

Arduino ESP32 Documentation Guide



Release 1.0.0-RC0-3734-g6aa7d51a89
Espressif Systems
Jun 25, 2026

Table of contents

Table of contents	i
1 Getting Started	3
1.1 About Arduino ESP32	3
1.2 First Things First	3
1.3 Supported SoC's	3
1.4 Arduino Core Reference	4
1.5 Supported Operating Systems	4
1.6 Supported IDEs	4
1.7 Support	5
1.8 Community	5
1.9 Issues Reporting	5
1.10 First Steps	5
1.10.1 Installing	5
1.10.2 Boards	16
1.11 Examples	26
1.12 Datasheet	26
1.13 Resources	26
2 Libraries	27
2.1 Supported Features and Peripherals	27
2.2 Datasheet	28
2.3 APIs	29
2.3.1 ADC	29
2.3.2 BLE	34
2.3.3 Bluetooth	38
2.3.4 Console	40
2.3.5 DAC	76
2.3.6 Deep Sleep	77
2.3.7 ESP-NOW	81
2.3.8 Ethernet	90
2.3.9 GPIO	93
2.3.10 I2C	96
2.3.11 I2S	105
2.3.12 ESP Insights	114
2.3.13 LED Control (LEDC)	117
2.3.14 Network API	126
2.3.15 Preferences	140
2.3.16 Pulse Counter	149
2.3.17 ESP Rainmaker	149
2.3.18 Reset Reason	158
2.3.19 Remote Control (RMT)	162
2.3.20 SDIO	180
2.3.21 SD MMC	180
2.3.22 Serial (UART)	185
2.3.23 SigmaDelta	210
2.3.24 SPI	211

2.3.25	Timer	213
2.3.26	TOUCH	218
2.3.27	USB API	221
2.3.28	Wi-Fi API	230
2.4	Matter APIs	245
2.4.1	Matter	245
2.5	OpenThread APIs	379
2.5.1	OpenThread	379
2.6	Zigbee APIs	418
2.6.1	Zigbee	418
3	Guides	561
3.1	Coding Guidelines	561
3.1.1	Code Formatting	561
3.1.2	Format Specifiers	563
3.2	Compatibility Guide for ESP32 Arduino Core	565
3.2.1	Introduction	565
3.2.2	Code Adaptations	565
3.2.3	Version Print	565
3.2.4	API Differences	565
3.2.5	Library Testing	566
3.3	Compile Arduino libs with ESP_LOGx	566
3.4	Documentation Contribution Guidelines	566
3.4.1	Introduction	566
3.4.2	About Documentation	566
3.4.3	How to Collaborate	567
3.4.4	Documentation Guide	567
3.4.5	Sections	568
3.4.6	Documentation Rules	569
3.4.7	Sphinx Basics	571
3.4.8	Additional Guidelines	572
3.5	Arduino IDE Tools Menu	572
3.5.1	Introduction	572
3.5.2	Arduino IDE	572
3.5.3	Tools Menu	572
3.5.4	Generic Options	572
3.5.5	USB Options	575
4	Tutorials	577
4.1	Basic Tutorial	577
4.1.1	Introduction	577
4.1.2	Requirements	577
4.1.3	Steps	577
4.1.4	Code	577
4.1.5	Log Output	579
4.1.6	Resources	579
4.2	Blink Interactive Tutorial	579
4.2.1	Introduction	579
4.2.2	About this Tutorial	579
4.2.3	Step by step	580
4.2.4	Simulation	580
4.2.5	Example Code	580
4.2.6	Resources	581
4.3	USB CDC and DFU Flashing	581
4.3.1	Introduction	581
4.3.2	USB DFU	581
4.3.3	USB CDC	582
4.3.4	Hardware	583

4.4	GPIO Matrix and Pin Mux	583
4.4.1	Introduction	583
4.4.2	GPIO Matrix and Pin Mux	583
4.4.3	Peripherals	584
4.4.4	Usage Examples	585
4.4.5	Datasheet	585
4.4.6	Resources	586
4.5	Partition Table	586
4.5.1	Introduction	586
4.5.2	Using a Custom Partition Scheme	587
4.5.3	Examples	588
4.5.4	Reference	589
4.6	Preferences	589
4.6.1	Introduction	589
4.6.2	Preferences Attributes	589
4.6.3	Library Overview	589
4.6.4	Workflow	590
4.6.5	Real World Example	593
4.6.6	Utility Functions	594
4.6.7	Working with Large Data	595
4.6.8	Multiple Namespaces	597
4.6.9	A Closer Look at <code>getX</code>	598
4.6.10	Advanced Item	599
4.6.11	Resources	599
4.6.12	Contribute	599
5	Advanced Utilities	601
5.1	Library Builder	601
5.1.1	About	601
5.1.2	Installing	601
5.1.3	Building	602
5.1.4	User Interface	603
5.1.5	Docker Image	604
5.2	Arduino as an ESP-IDF component	607
5.2.1	About	607
5.2.2	Installation	607
5.2.3	Configuration	608
5.2.4	Logging To Serial	610
5.2.5	FreeRTOS Tick Rate (Hz)	610
5.2.6	Compilation Errors	610
5.2.7	Adding arduino library	610
5.2.8	Tip	611
5.3	OTA Web Update	611
5.3.1	Requirements	611
5.3.2	Implementation	611
5.4	<code>makeEspArduino</code>	613
6	Third Party Tools	615
6.1	pioarduino - (p)eople (i)nitiated (o)ptimized (arduino)	615
6.1.1	About	615
6.2	Wokwi	615
6.2.1	About	615
6.2.2	Pricing	616
6.2.3	Learn more	616
7	Migration Guides	617
7.1	Migration from 2.x to 3.0	617
7.1.1	Introduction	617
7.1.2	Build System	617

7.1.3	APIs	617
8	Frequently Asked Questions	623
8.1	How to modify an sdkconfig option in Arduino?	623
8.2	How to compile libs with different debug level?	623
9	Troubleshooting	625
9.1	Common Issues	625
9.1.1	Installing	625
9.1.2	Building	625
9.1.3	Flashing	626
9.1.4	Hardware	629
9.1.5	Wi-Fi	629
9.1.6	SPIFFS mount failed	630
9.1.7	SD card mount fail	630
9.1.8	How to determine the module version:	631
9.1.9	Further Help	632
10	Contributions Guide	633
10.1	How to Contribute	633
10.2	Before Contributing	633
10.3	Pull Request Process	633
10.4	Example Contribution Guideline	633
10.4.1	Checklist	634
10.4.2	Header	634
10.4.3	README file	634
10.4.4	Inline Comments	634
10.4.5	Testing	634
10.4.6	Example Template	636
10.5	Documentation	636
10.6	Testing and CI	636
10.6.1	Examples	636
10.6.2	Library Changes	636
10.6.3	Core Changes	637
10.6.4	CI	637
10.6.5	Compilation Tests	637
10.6.6	Runtime Tests	637
10.6.7	Documentation Checks	642
10.6.8	Code Style Checks	642
10.7	Legal Part	643
11	External Libraries Testing	645
11.1	About	645
11.2	How to Add Library to Test	645
11.2.1	List of parameters:	645
11.2.2	Example of library addition from Arduino Library Manager with ESP32-S2 excluded:	646
11.2.3	Example of library addition from Github URL with required library:	646
11.2.4	Submit a PR	646
11.3	Test Results	646
11.3.1	Icons meaning	646
11.3.2	Scheduled test result	646
11.3.3	Pull Request test result	647

Here you will find all the relevant information about the project. This documentation is valid for the Arduino Core for ESP32 version 3.3.10 based on ESP-IDF 5.5.

Note: This is a work in progress documentation and we will appreciate your help! We are looking for contributors!

Chapter 1

Getting Started

1.1 About Arduino ESP32

Welcome to the Arduino ESP32 support documentation! Here you will find important information on how to use the project.

1.2 First Things First

Note: Before continuing, we must be clear that this project is supported by [Espressif Systems](#) and the community. Everyone is more than welcome to contribute back to this project.

ESP32 is a single 2.4 GHz Wi-Fi-and-Bluetooth SoC (System On a Chip) designed by [Espressif Systems](#).

ESP32 is designed for mobile, wearable electronics, and Internet-of-Things (IoT) applications. It features all the state-of-the-art characteristics of low-power chips, including fine-grained clock gating, multiple power modes, and dynamic power scaling. For instance, in a low-power IoT sensor hub application scenario, ESP32 is woken-up periodically and only when a specified condition is detected. Low-duty cycle is used to minimize the amount of energy that the chip expends.

The output of the power amplifier is also adjustable, thus contributing to an optimal trade-off between communication range, data rate and power consumption.

The ESP32 series is available as a chip or module.

1.3 Supported SoC's

Here are the ESP32 series supported by the Arduino-ESP32 project:

SoC	Stable	Development	Datasheet
ESP32	Yes	Yes	ESP32
ESP32-C3	Yes	Yes	ESP32-C3
ESP32-C5	Yes	Yes	ESP32-C5
ESP32-C6	Yes	Yes	ESP32-C6
ESP32-H2	Yes	Yes	ESP32-H2
ESP32-P4	Yes	Yes	ESP32-P4
ESP32-S2	Yes	Yes	ESP32-S2
ESP32-S3	Yes	Yes	ESP32-S3

Note: ESP32-C2 and ESP32-C61 are also supported by Arduino-ESP32 but require using Arduino as an ESP-IDF component or rebuilding the static libraries. For more information, see the [Arduino as an ESP-IDF component documentation](#) or the [Lib Builder documentation](#), respectively.

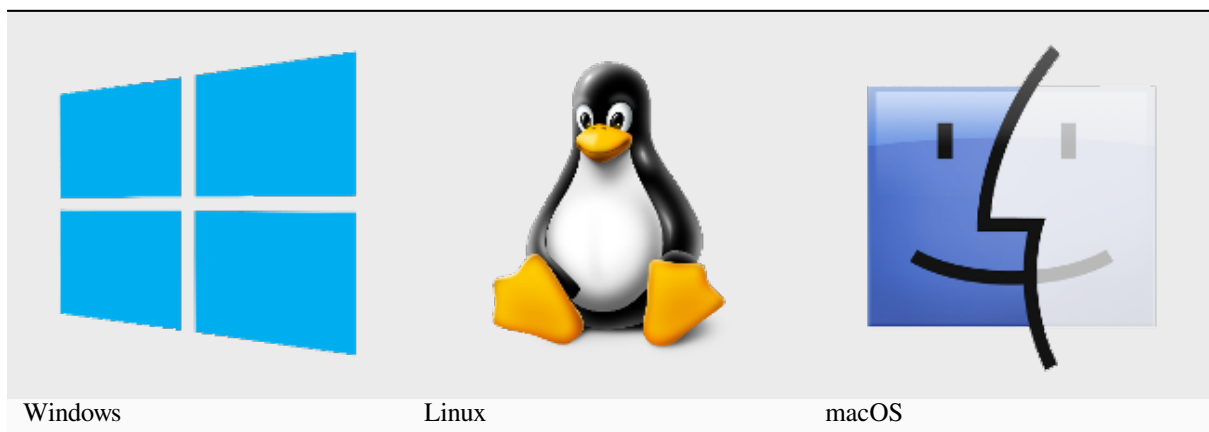
See [Boards](#) for more details about ESP32 development boards.

1.4 Arduino Core Reference

This documentation is built on the ESP32 and we are not going to cover the common Arduino API. To see the Arduino reference documentation, please consider reading the official documentation.

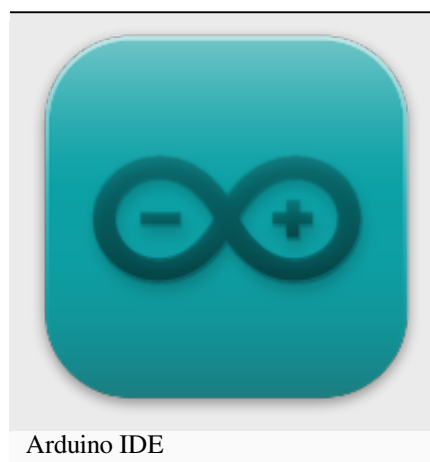
Arduino Official Documentation: [Arduino Reference](#).

1.5 Supported Operating Systems



1.6 Supported IDEs

Here is the list of supported IDE for Arduino ESP32 support integration.



See [Installing Guides](#) for more details on how to install the Arduino ESP32 support.

1.7 Support

This is an open project and it's supported by the community. Feel free to ask for help in one of the community channels.

1.8 Community

The Arduino community is huge! You can find a lot of useful content on the Internet. Here are some community channels where you may find information and ask for some help, if needed.

- [ESP32 Forum](#): Official Espressif Forum.
- [ESP32 Forum - Arduino](#): Official Espressif Forum for Arduino related discussions.
- [ESP32 Forum - Hardware](#): Official Espressif Forum for Hardware related discussions.
- [Espressif Developer Portal](#): Official Espressif Developer Portal with tutorials, examples, workshops, and more.
- [Arduino Core for Espressif \(Discord\)](#): Official Espressif Discord channel for the Arduino Core.
- [Espressif MCUs \(Discord\)](#)
- [ESP32 on Reddit](#)

1.9 Issues Reporting

Before opening a new issue, please read this:

Be sure to search for a similar reported issue. This avoids duplicating or creating noise in the GitHub Issues reporting. We also have the troubleshooting guide to save your time on the most common issues reported by users.

For more details about creating new Issue, see the [Issue Template](#).

If you have any new idea, see the [Feature request Template](#).

1.10 First Steps

Here are the first steps to get the Arduino ESP32 support running.

To install Arduino-ESP32, please see the dedicated section on the Installation guide. We recommend you install it using the boards manager.

1.10.1 Installing

This guide will show how to install the Arduino-ESP32 support.

Before Installing

We recommend you install the support using your favorite IDE, but other options are available depending on your operating system. To install Arduino-ESP32 support, you can use one of the following options.

Note: Users in China might have troubles with connection and download speeds using GitHub. Please use our Jihulab mirror as the repository source:

`https://jihulab.com/esp-mirror/espressif/arduino-esp32.git`



Installing using Arduino IDE

This is the way to install Arduino-ESP32 directly from the Arduino IDE.

Note: For overview of SoC's support, take a look on [Supported Soc's table](#) where you can find if the particular chip is under stable or development release.

- Stable release link:

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

- Development release link:

```
https://espressif.github.io/arduino-esp32/package_esp32_dev_index.json
```

Users in China might have troubles with connection and download speeds using the links above. Please use our Jihulab mirror:

- Stable release link:

```
https://jihulab.com/esp-mirror/espressif/arduino-esp32/-/raw/gh-pages/package_esp32_index_cn.json
```

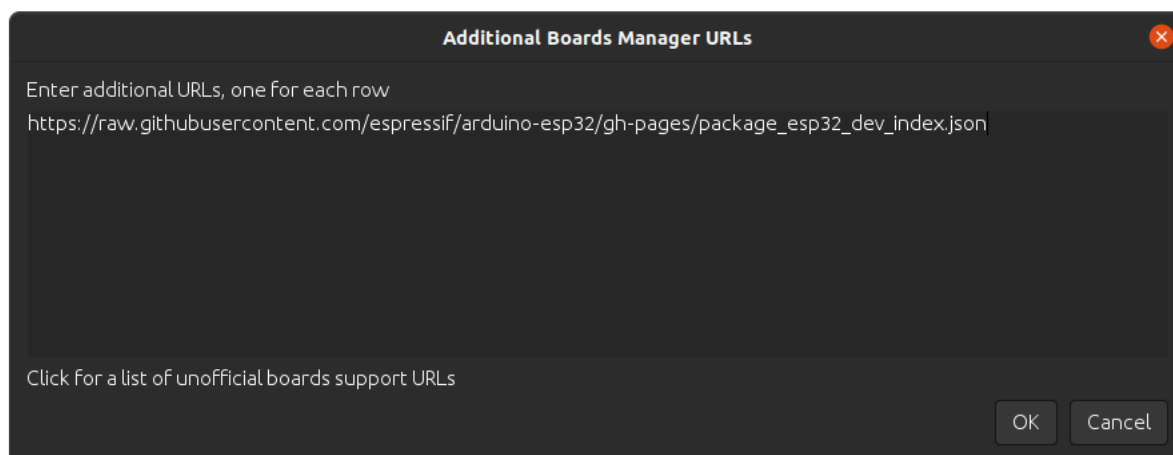
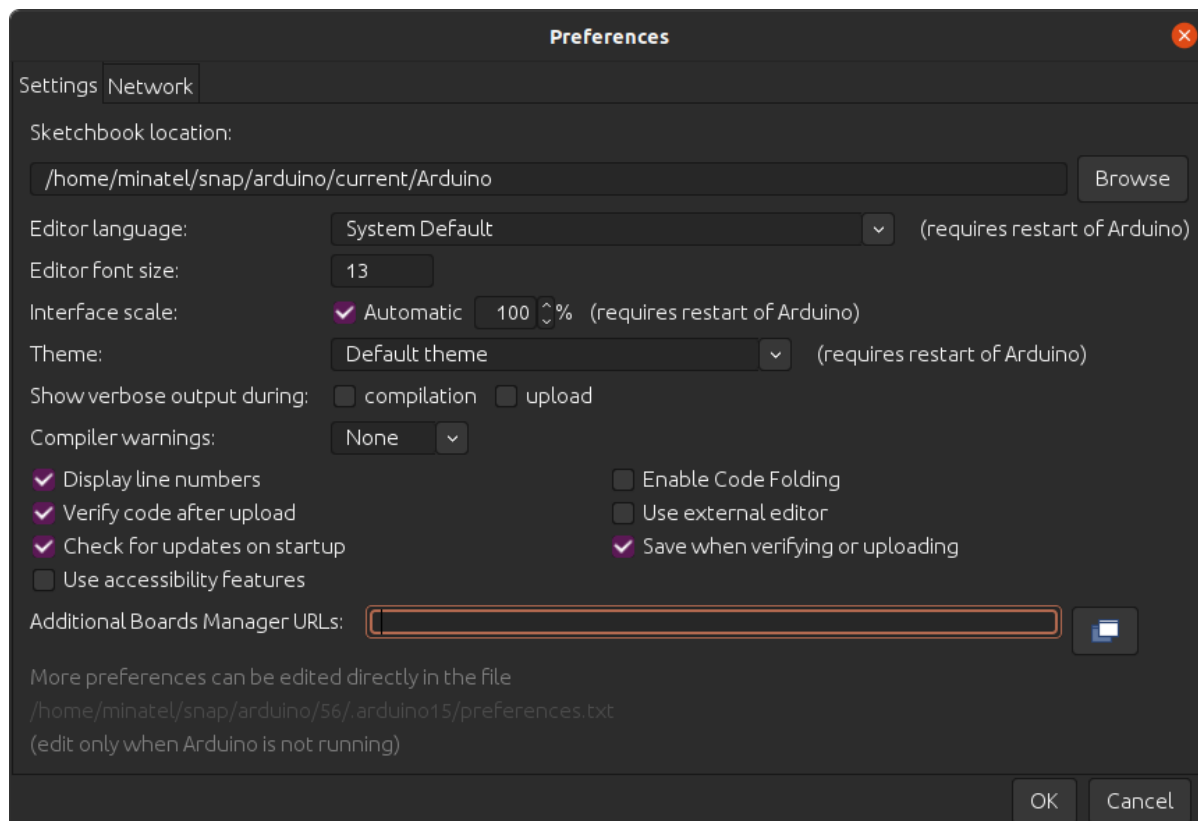
- Development release link:

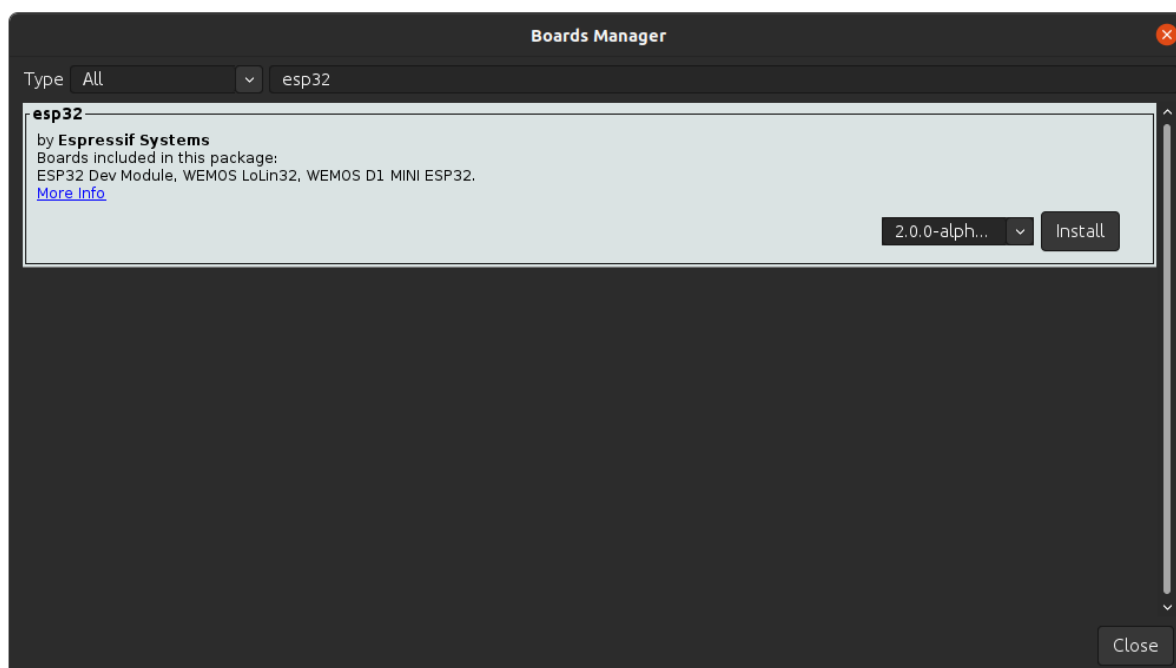
```
https://jihulab.com/esp-mirror/espressif/arduino-esp32/-/raw/gh-pages/package_esp32_dev_index_cn.json
```

Note: Starting with the Arduino IDE version 1.6.4, Arduino allows installation of third-party platform packages using Boards Manager. We have packages available for Windows, macOS, and Linux.

To start the installation process using the Boards Manager, follow these steps:

- Install the current upstream Arduino IDE at the 1.8 level or later. The current version is at the [arduino.cc](https://www.arduino.cc) website.
- Start Arduino and open the Preferences window.
- Enter one of the release links above into *Additional Board Manager URLs* field. You can add multiple URLs, separating them with commas.
- Open Boards Manager from Tools > Board menu and install *esp32* platform (and do not forget to select your ESP32 board from Tools > Board menu after installation). Users in China must select the package version with the “-cn” suffix and perform updates manually. Automatic updates are not supported in this region, as they target the default package without the “-cn” suffix, resulting in download failures.
- Restart Arduino IDE.





Windows (manual installation)

Warning: Arduino ESP32 core v2.x.x cannot be used on Windows 8.x x86 (32 bits), Windows 7 or earlier. The Windows 32 bits OS is no longer supported by this toolchain.

The Arduino ESP32 v1.0.6 still works on WIN32. You might want to install python 3.8.x because it is the latest release supported by Windows 7.

Steps to install Arduino ESP32 support on Windows:

Step 1

1. Download and install the latest Arduino IDE Windows Installer from [arduino.cc](https://www.arduino.cc/en/Main/Software)
2. Download and install Git from [git-scm.com](https://git-scm.com/download/win)
3. Start Git GUI and do the following steps:
 - Select Clone Existing Repository
 - **Select source and destination**
 - Sketchbook Directory: Usually C:/Users/[YOUR_USER_NAME]/Documents/Arduino and is listed underneath the “Sketchbook location” in Arduino preferences.
 - Source Location: https://github.com/espressif/arduino-esp32.git
 - Target Directory: [ARDUINO_SKETCHBOOK_DIR]/hardware/espressif/esp32
 - Click Clone to start cloning the repository

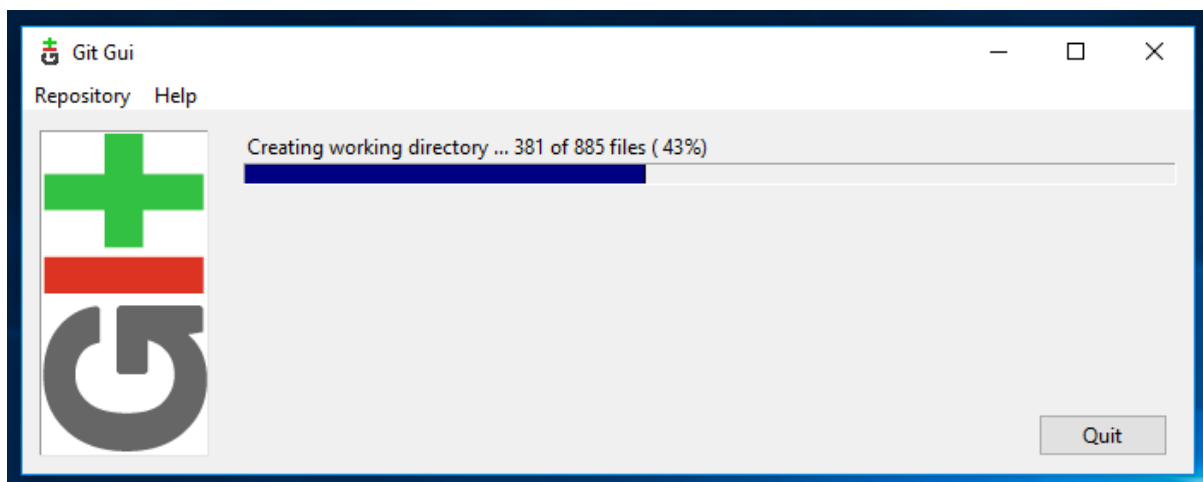
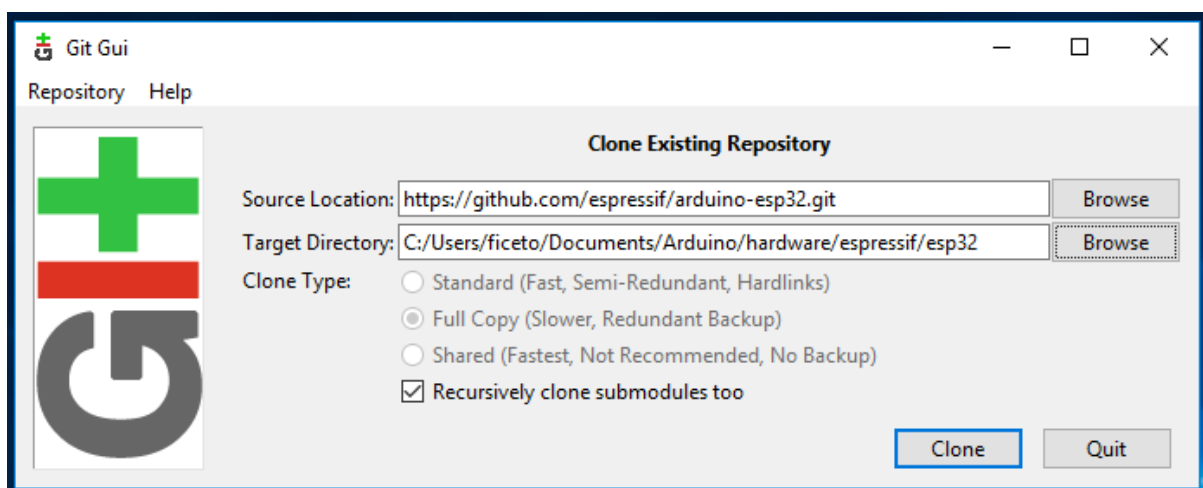
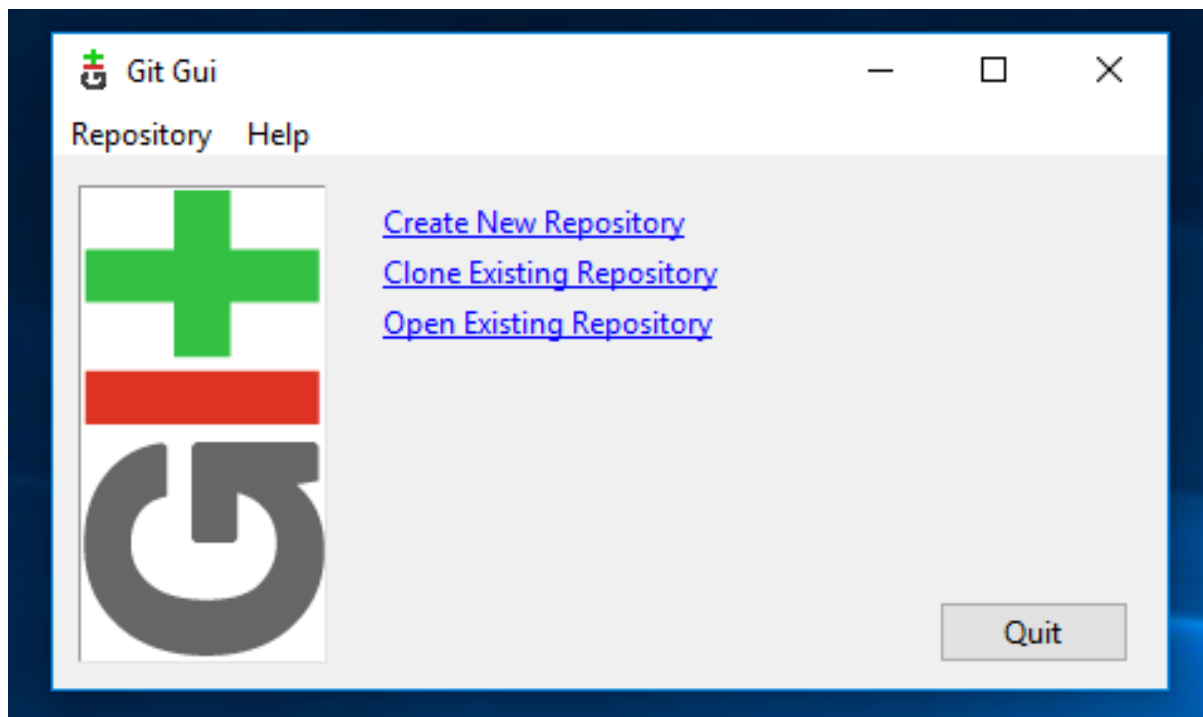
Step 2

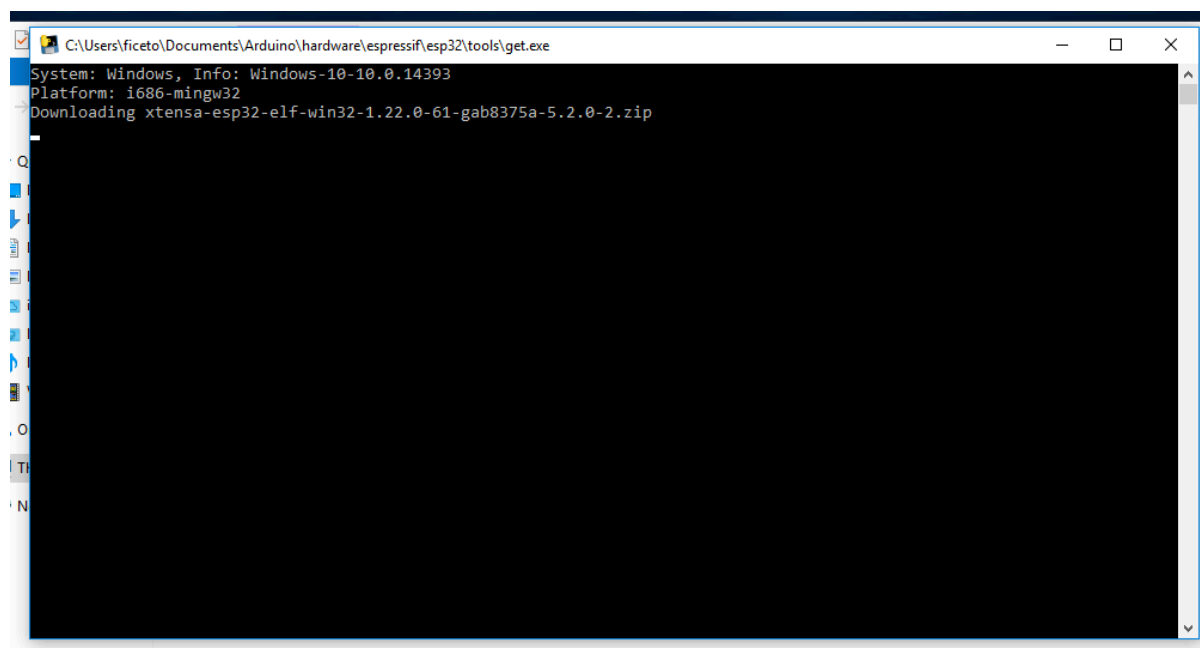
Step 3

- open a *Git Bash* session pointing to [ARDUINO_SKETCHBOOK_DIR]/hardware/espressif/esp32 and execute ``git submodule update --init --recursive``
- Open [ARDUINO_SKETCHBOOK_DIR]/hardware/espressif/esp32/tools and double-click `get.exe`

Step 4

- When ``get.exe`` finishes, you should see the following files in the directory





Step 5

1. Plug your ESP32 board and wait for the drivers to install (or install manually any that might be required)
2. Start Arduino IDE
3. Select your board in Tools > Board menu
4. Select the COM port that the board is attached to
5. Compile and upload (You might need to hold the boot button while uploading)

How to update to the latest code

1. Start Git GUI and you should see the repository under Open Recent Repository. Click on it!
1. From menu Remote select Fetch from> origin
1. Wait for git to pull any changes and close Git GUI
2. Open [ARDUINO_SKETCHBOOK_DIR]/hardware/espressif/esp32/tools and double-click get.exe

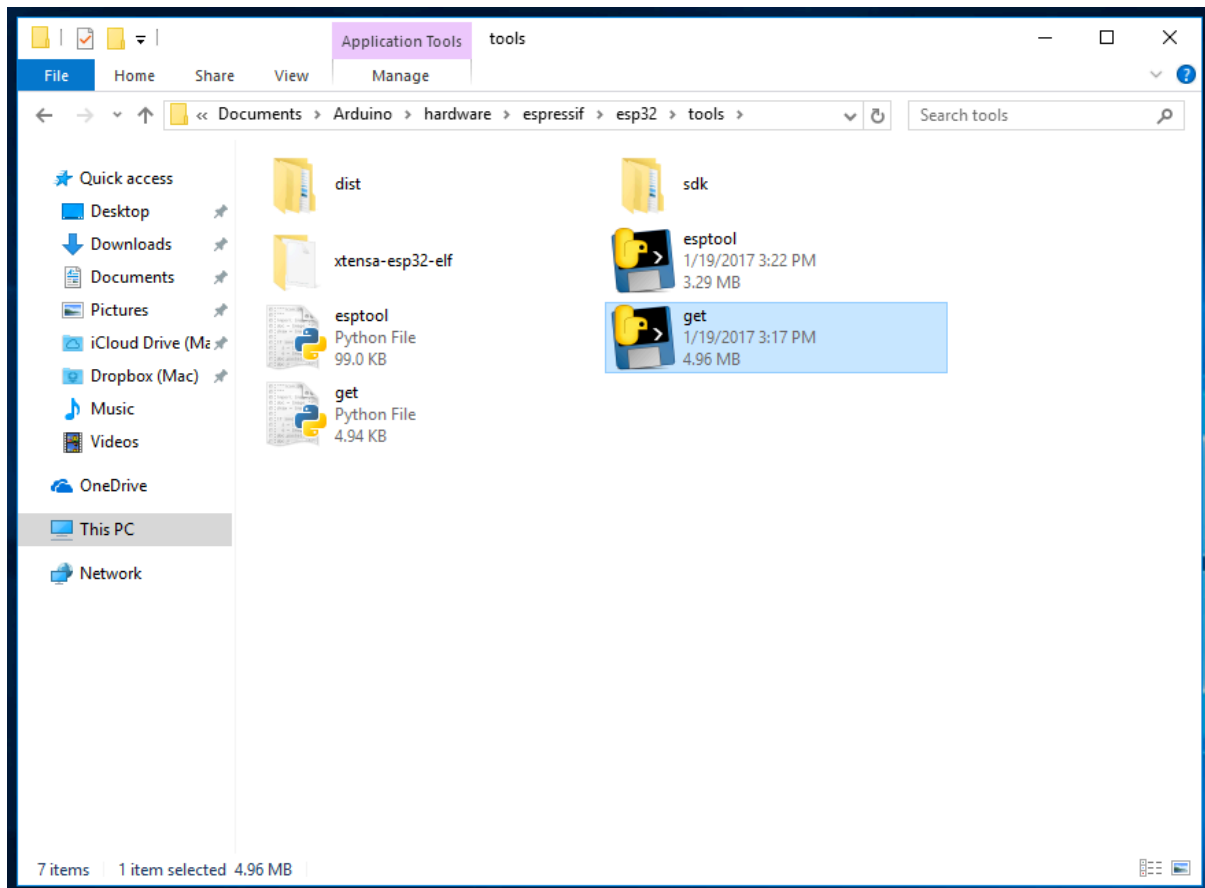
Linux

Debian/Ubuntu

- Install latest Arduino IDE from [arduino.cc](https://www.arduino.cc/).
- Open Terminal and execute the following command (copy -> paste and hit enter):

```
sudo usermod -a -G dialout $USER && \
sudo apt-get install git && \
wget https://bootstrap.pypa.io/get-pip.py && \
sudo python3 get-pip.py && \
sudo pip3 install pyserial && \
mkdir -p ~/Arduino/hardware/espressif && \
cd ~/Arduino/hardware/espressif && \
git clone https://github.com/espressif/arduino-esp32.git esp32 && \
cd esp32/tools && \
python3 get.py
```

- Restart Arduino IDE.



- If you have Arduino installed to `~/`, modify the installation as follows, beginning at `mkdir -p ~/Arduino/hardware`:

```
cd ~/Arduino/hardware
mkdir -p espressif && \
cd espressif && \
git clone https://github.com/espressif/arduino-esp32.git esp32 && \
cd esp32/tools && \
python3 get.py
```

Fedora

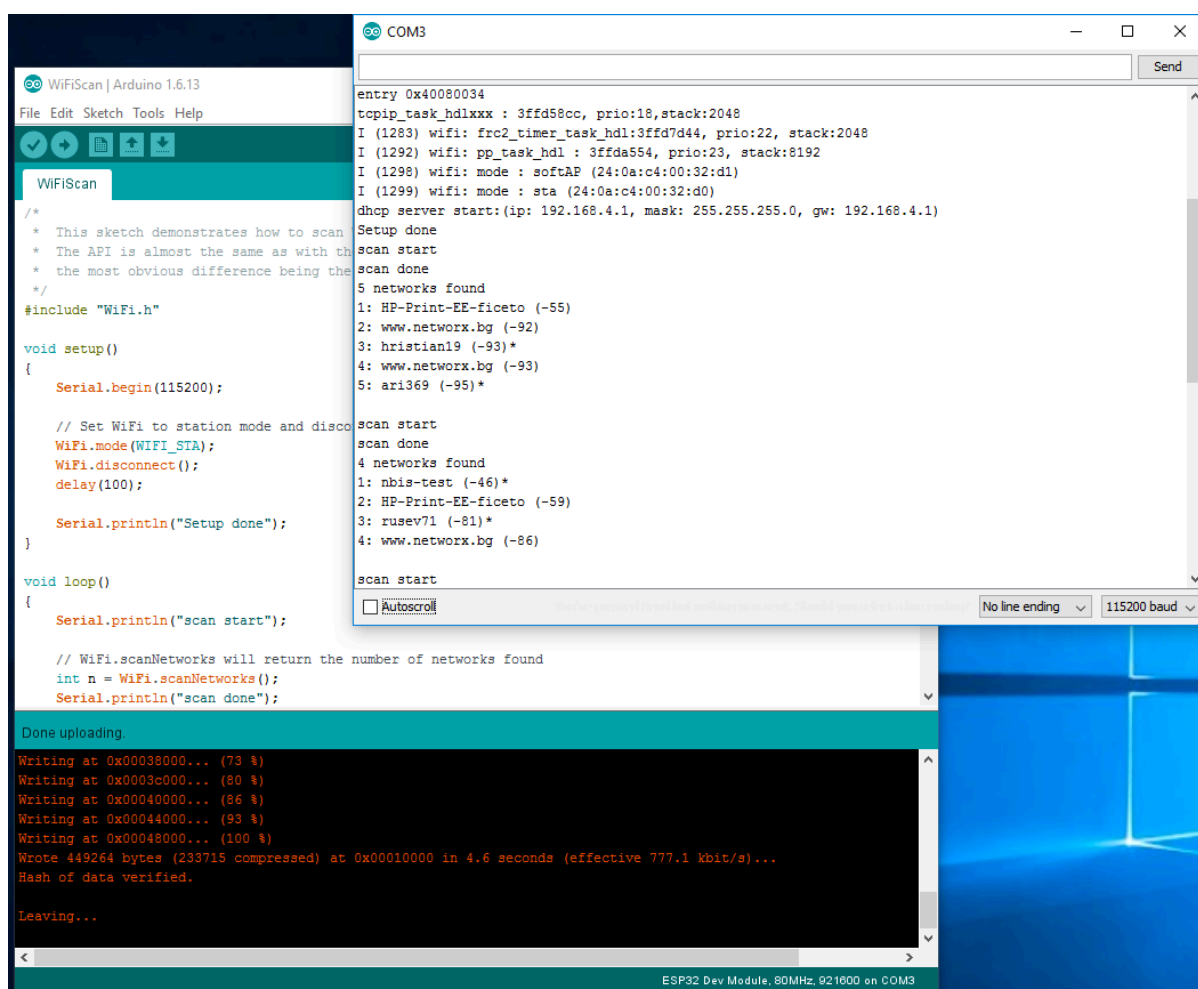
- Install the latest Arduino IDE from arduino.cc.

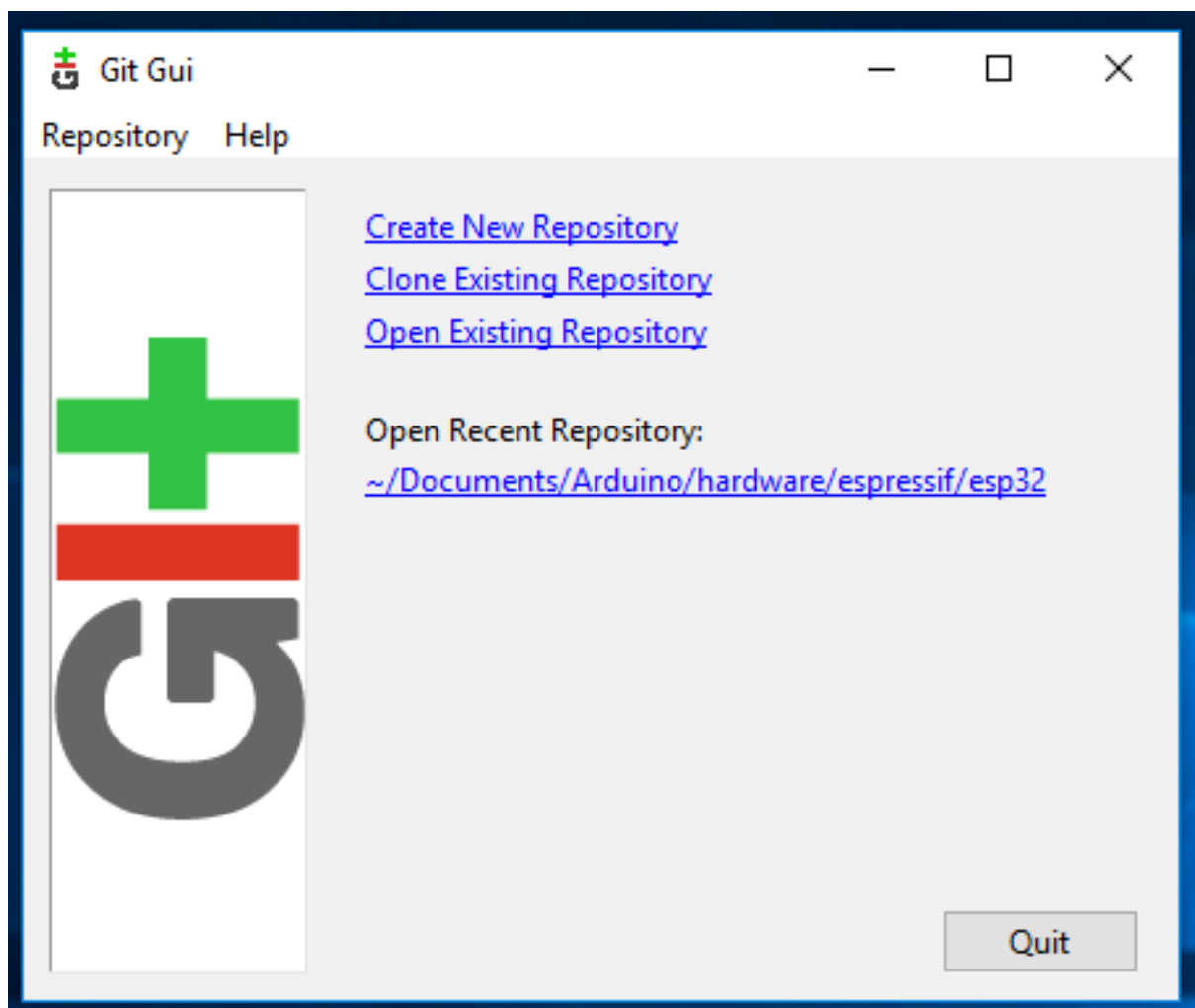
Note: Command `$ sudo dnf -y install arduino` will most likely install an older release.

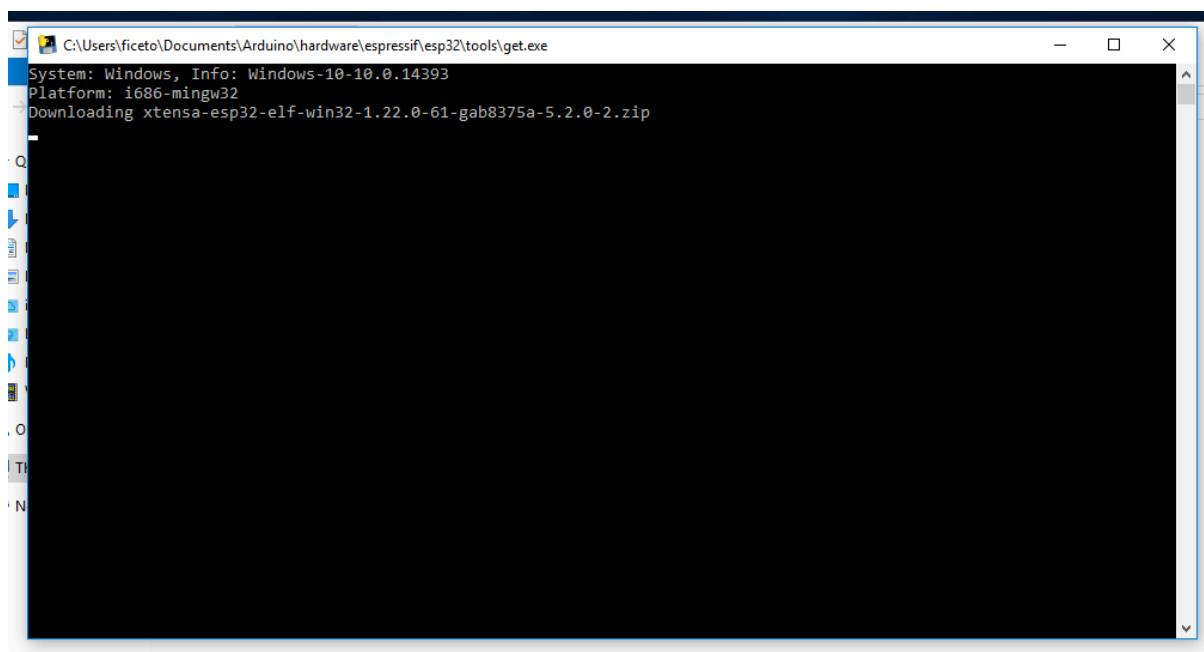
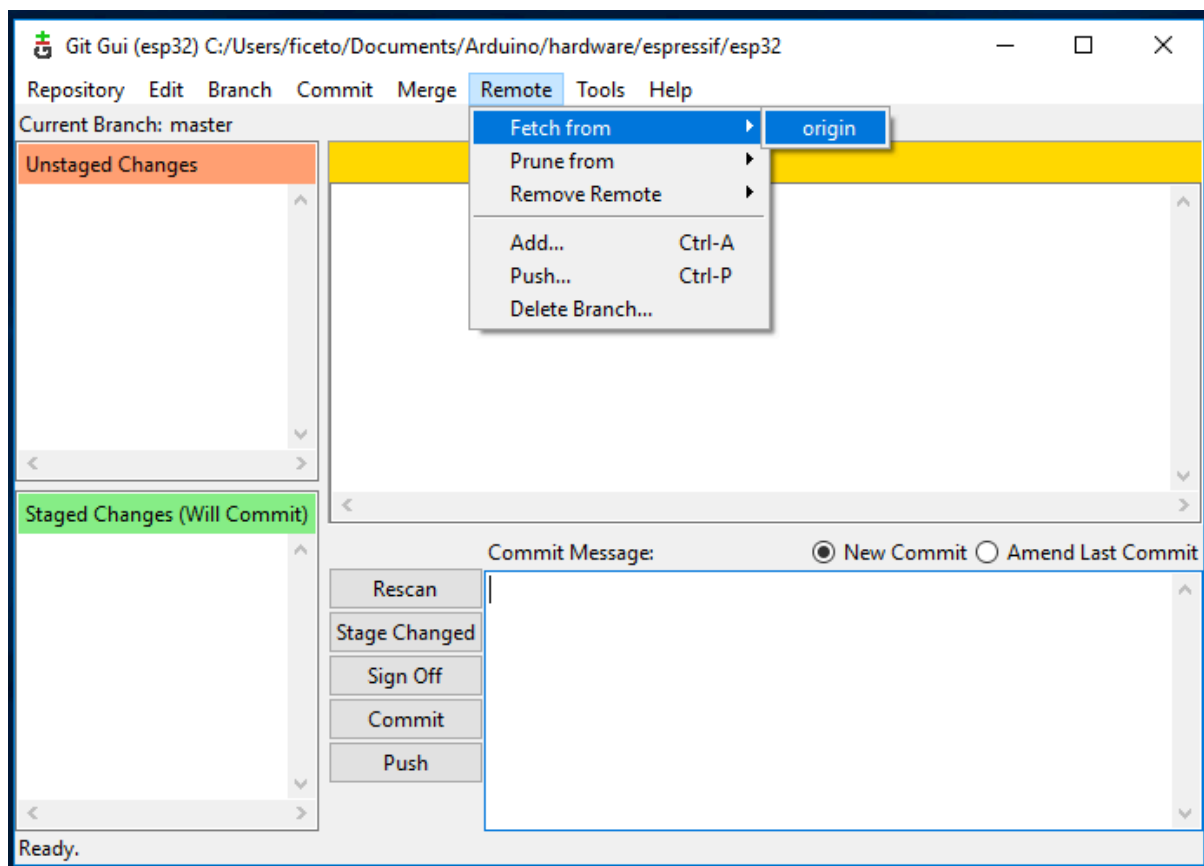
- Open Terminal and execute the following command (copy -> paste and hit enter):

```
sudo usermod -a -G dialout $USER && \
sudo dnf install git python3-pip python3-pyserial && \
mkdir -p ~/Arduino/hardware/espressif && \
cd ~/Arduino/hardware/espressif && \
git clone https://github.com/espressif/arduino-esp32.git esp32 && \
cd esp32/tools && \
python get.py
```

- Restart Arduino IDE.









openSUSE

- Install the latest Arduino IDE from arduino.cc.
- Open Terminal and execute the following command (copy -> paste and hit enter):

```
sudo usermod -a -G dialout $USER && \  
if [ `python --version 2>&1 | grep '2.7' | wc -l` = "1" ]; then \  
sudo zypper install git python-pip python-pyserial; \  
else \  
sudo zypper install git python3-pip python3-pyserial; \  
fi && \  
mkdir -p ~/Arduino/hardware/espressif && \  
cd ~/Arduino/hardware/espressif && \  
git clone https://github.com/espressif/arduino-esp32.git esp32 && \  
cd esp32/tools && \  
python get.py
```

- Restart Arduino IDE.

macOS

- Install the latest Arduino IDE from arduino.cc.
- Open Terminal and execute the following command (copy -> paste and hit enter):

```
mkdir -p ~/Documents/Arduino/hardware/espressif && \  
cd ~/Documents/Arduino/hardware/espressif && \  
git clone https://github.com/espressif/arduino-esp32.git esp32 && \  
cd esp32/tools && \  
python get.py
```

Where ~/Documents/Arduino represents your sketch book location as per “Arduino” > “Preferences” > “Sketchbook location” (in the IDE once started). Adjust the command above accordingly.

- If you get the error below, install through the command line dev tools with `xcode-select --install` and try the command above again:

```
xcrun: error: invalid active developer path (/Library/Developer/CommandLineTools),   
missing xcrun at: /Library/Developer/CommandLineTools/usr/bin/xcrun
```

- Run the command:

```
xcode-select --install
```

- Try python3 instead of python if you get the error: `IOError: [Errno socket error] [SSL: TLSV1_ALERT_PROTOCOL_VERSION] tlsv1 alert protocol version (_ssl.c:590)` when running `python get.py`

- If you get the following error when running `python get.py: urllib.error.URLError: <urlopen error SSL: CERTIFICATE_VERIFY_FAILED, go to Macintosh HD > Applications > Python3.6 folder (or any other python version), and run the following scripts: Install Certificates.command and Update Shell Profile.command`
- Restart Arduino IDE.

1.10.2 Boards

Development Boards

You will need a development board or a custom board with the ESP32 (see Supported SoC's) to start playing. There is a bunch of different types and models widely available on the Internet. You need to choose one that covers all your requirements.

To help you on this selection, we point out some facts about choosing the proper boards to help you to save money and time.

One ESP32 to rule them all!

One important information that usually bring about some confusion is regarding the different models of the ESP32 SoC and modules.

The ESP32 is divided by family:

- ESP32**
 - Wi-Fi, BT and BLE 4
- ESP32-C2¹**
 - Wi-Fi and BLE 5
- ESP32-C3**
 - Wi-Fi and BLE 5
- ESP32-C5**
 - Dual-band Wi-Fi 6, BLE 5 and IEEE 802.15.4
- ESP32-C6**
 - Wi-Fi 6, BLE 5 and IEEE 802.15.4
- ESP32-C61¹**
 - Wi-Fi 6, BLE 5
- ESP32-H2**
 - BLE 5 and IEEE 802.15.4
- ESP32-P4**
 - 400 MHz Dual Core RISC-V CPU, 40 MHz ULP Co-processor, single-precision FPU and AI extensions.
- ESP32-S2**
 - Wi-Fi only
- ESP32-S3**
 - Wi-Fi and BLE 5

For each family, we have SoC variants with some differentiation. The differences are more about the embedded flash and its size and the number of the cores (dual or single).

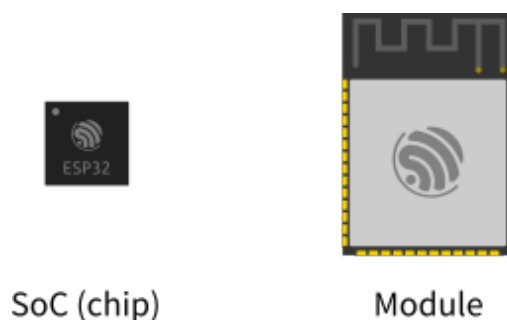
The modules use the SoC internally, including the external flash, PSRAM (in some models) and other essential electronic components. Essentially, all modules from the same family use the same SoC.

For example:

The SoC partnumber is the ESP32-D0WD-V3 and it's the same SoC used inside of the ESP32-WROVER (with PSRAM) and ESP32-WROOM modules. This means that the same characteristics are present in both modules' core.

For more detailed information regarding the SoC's and modules, see the [Espressif Product Selector](#).

¹ This SoC is only supported using Arduino as an ESP-IDF component or by rebuilding the static libraries.



Now that you know that the module can be different but the heart is the same, you can choose your development board.

Before buying: Keep in mind that for some “must have” features when choosing the best board for your needs:

- **Embedded USB-to-Serial**
 - This is very convenient for programming and monitoring the logs with the terminal via USB.
- **Breadboard friendly**
 - If you are prototyping, this will be very useful to connect your board directly on the breadboard.
- **open-source/open-hardware**
 - Check if the schematics are available for download. This helps a lot on prototyping.
- **Support**
 - Some of the manufacturers offer a very good level of support, with examples and demo projects.

Espressif



ESP32-DevKitC-1 The *ESP32-DevKitC-1* development board is one of Espressif’s official boards. This board is based on the [ESP32-WROVER-E](#) module, with the [ESP32](#) as the core.

Specifications

- Wi-Fi 802.11b/g/n (802.11n up to 150 Mbps)
- Bluetooth v4.2 BR/EDR and BLE specification
- Built around ESP32 series of SoCs
- Integrated 4 MB SPI flash
- Integrated 8 MB PSRAM
- **Peripherals**
 - SD card
 - UART
 - SPI
 - SDIO
 - I2C
 - LED PWM
 - Motor PWM
 - I2S
 - IR
 - Pulse Counter
 - GPIO
 - Capacitive Touch Sensor
 - ADC
 - DAC

- Two-Wire Automotive Interface (TWAI®, compatible with ISO11898-1)
- PCB antenna or external antenna connector

Header Block

Note: Not all of the chip pins are exposed to the pin headers.

J1

No.	Name	Type	Function
1	3V3	P	3.3 V power supply
2	EN	I	CHIP_PU, Reset
3	IO36	I	GPIO36, ADC1_CH0, S_VP
4	IO39	I	GPIO39, ADC1_CH3, S_VN
5	IO34	I	GPIO34, ADC1_CH6, VDET_1
6	IO35	I	GPIO35, ADC1_CH7, VDET_2
7	IO32	I/O	GPIO32, ADC1_CH4, TOUCH_CH9, XTAL_32K_P
8	IO33	I/O	GPIO33, ADC1_CH5, TOUCH_CH8, XTAL_32K_N
9	IO25	I/O	GPIO25, ADC1_CH8, DAC_1
10	IO26	I/O	GPIO26, ADC2_CH9, DAC_2
11	IO27	I/O	GPIO27, ADC2_CH7, TOUCH_CH7
12	IO14	I/O	GPIO14, ADC2_CH6, TOUCH_CH6, MTMS
13	IO12	I/O	GPIO12, ADC2_CH5, TOUCH_CH5, MTDI
14	GND	G	Ground
15	IO13	I/O	GPIO13, ADC2_CH4, TOUCH_CH4, MTCK
16	IO9	I/O	GPIO9, D2
17	IO10	I/O	GPIO10, D3
18	IO11	I/O	GPIO11, CMD
19	5V0	P	5 V power supply

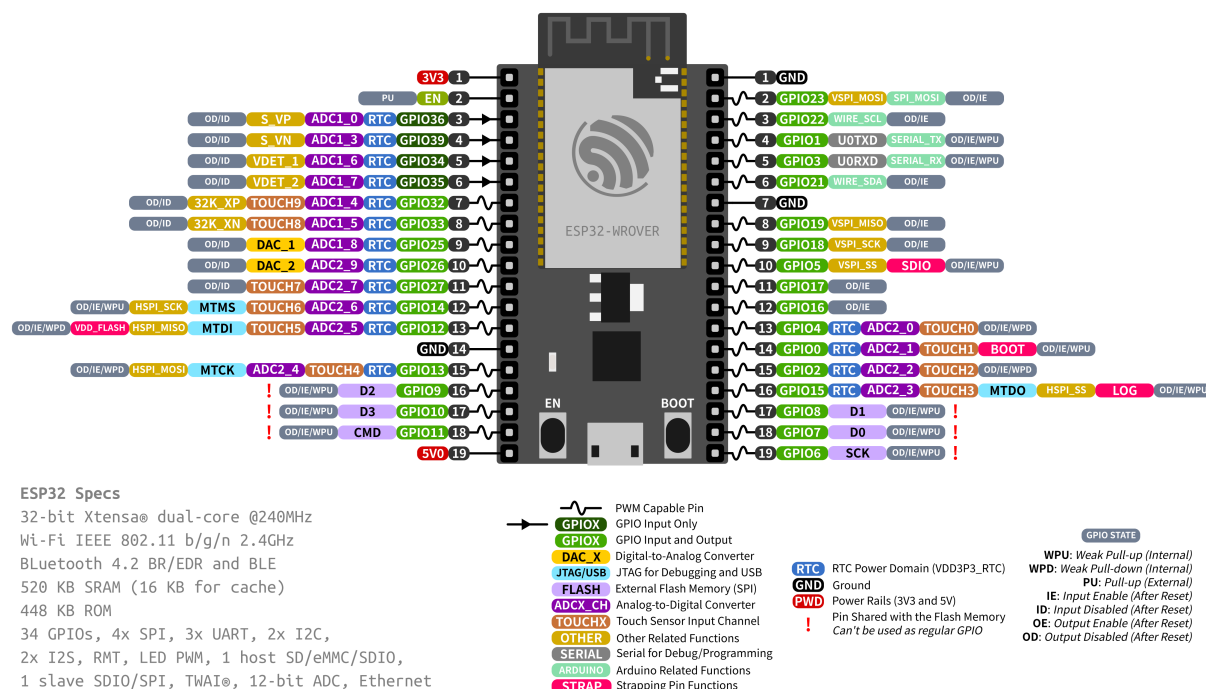
J3

No.	Name	Type	Function
1	GND	G	Ground
2	IO23	I/O	GPIO23
3	IO22	I/O	GPIO22
4	IO1	I/O	GPIO1, U0TXD
5	IO3	I/O	GPIO3, U0RXD
6	IO21	I/O	GPIO21
7	GND	G	Ground
8	IO19	I/O	GPIO19
9	IO18	I/O	GPIO18
10	IO5	I/O	GPIO5
11	IO17	I/O	GPIO17
12	IO16	I/O	GPIO16
13	IO4	I/O	GPIO4, ADC2_CH0, TOUCH_CH0
14	IO0	I/O	GPIO0, ADC2_CH1, TOUCH_CH1, Boot
16	IO2	I/O	GPIO2, ADC2_CH2, TOUCH_CH2
17	IO15	I/O	GPIO15, ADC2_CH3, TOUCH_CH3, MTDO
17	IO8	I/O	GPIO8, D1
18	IO7	I/O	GPIO7, D0
19	IO6	I/O	GPIO6, SCK

P: Power supply; I: Input; O: Output; T: High impedance.

Pin Layout

ESP32-DevKitC



Strapping Pins Some of the GPIO's have important features during the booting process. Here is the list of the strapping pins on the [ESP32](#).

GPIO	Default	Function	Pull-up	Pull-down
IO12	Pull-down	Voltage of Internal LDO (VDD_SDIO)	1.8 V	3.3 V
IO0	Pull-up	Bootting Mode	SPI Boot	Download Boot
IO2	Pull-down	Bootting Mode	Don't Care	Download Boot
IO15	Pull-up	Enabling/Disabling Log Print During Booting and Timing of SDIO Slave	U0TXD Active	U0TXD Silent
IO5	Pull-up	Timing of SDIO Slave	See ESP32	See ESP32

Be aware when choosing which pins to use.

Restricted Usage GPIO's Some of the GPIO's are used for the external flash and PSRAM. These GPIO's cannot be used:

GPIO	Shared Function
IO6	External SPI Flash
IO7	External SPI Flash
IO8	External SPI Flash
IO9	External SPI Flash
IO10	External SPI Flash
IO11	External SPI Flash

Other GPIO's are *INPUT ONLY* and cannot be used as output pin:

GPIO	Function
IO34	GPIO34, ADC1_CH6, VDET_1
IO35	GPIO35, ADC1_CH7, VDET_2
IO36	GPIO36, ADC1_CH0, S_VP
IO39	GPIO39, ADC1_CH3, S_VN

Resources

- [ESP32](#) (Datasheet)
- [ESP32-WROVER-E](#) (Datasheet)
- [ESP32-DevKitC](#) (Schematic)

ESP32-S2-Saola-1 The [ESP32-S2-Saola-1](#) development board is one of Espressif's official boards. This board is based on the [ESP32-S2-WROVER](#) module, with the [ESP32-S2](#) as the core.

Specifications

- Wi-Fi 802.11b/g/n (802.11n up to 150 Mbps)
- Built around ESP32-S2 series of SoCs Xtensa® single-core
- Integrated 4 MB SPI flash
- Integrated 2 MB PSRAM
- **Peripherals**
 - 43 × programmable GPIOs
 - 2 × 13-bit SAR ADCs, up to 20 channels
 - 2 × 8-bit DAC
 - 14 × touch sensing IOs
 - 4 × SPI
 - 1 × I2S
 - 2 × I2C
 - 2 × UART
 - RMT (TX/RX)
 - LED PWM controller, up to 8 channels
 - 1 × full-speed USB OTG
 - 1 × temperature sensor
 - 1 × DVP 8/16 camera interface, implemented using the hardware resources of I2S
 - 1 × LCD interface (8-bit serial RGB/8080/6800), implemented using the hardware resources of SPI2
 - 1 × LCD interface (8/16/24-bit parallel), implemented using the hardware resources of I2S
 - 1 × TWAI® controller (compatible with ISO 11898-1)
- PCB antenna or external antenna connector

Header Block

Note: Not all of the chip pins are exposed to the pin headers.

J2

No.	Name	Type	Function
1	3V3	P	3.3 V power supply
2	IO0	I/O	GPIO0, Boot
3	IO1	I/O	GPIO1, ADC1_CH0, TOUCH_CH1
4	IO2	I/O	GPIO2, ADC1_CH1, TOUCH_CH2
5	IO3	I/O	GPIO3, ADC1_CH2, TOUCH_CH3
6	IO4	I/O	GPIO4, ADC1_CH3, TOUCH_CH4
7	IO5	I/O	GPIO5, ADC1_CH4, TOUCH_CH5
8	IO6	I/O	GPIO6, ADC1_CH5, TOUCH_CH6
9	IO7	I/O	GPIO7, ADC1_CH6, TOUCH_CH7
10	IO8	I/O	GPIO8, ADC1_CH7, TOUCH_CH8
11	IO9	I/O	GPIO9, ADC1_CH8, TOUCH_CH9
12	IO10	I/O	GPIO10, ADC1_CH9, TOUCH_CH10
13	IO11	I/O	GPIO11, ADC2_CH0, TOUCH_CH11
14	IO12	I/O	GPIO12, ADC2_CH1, TOUCH_CH12
15	IO13	I/O	GPIO13, ADC2_CH2, TOUCH_CH13
16	IO14	I/O	GPIO14, ADC2_CH3, TOUCH_CH14
17	IO15	I/O	GPIO15, ADC2_CH4, XTAL_32K_P
18	IO16	I/O	GPIO16, ADC2_CH5, XTAL_32K_N
19	IO17	I/O	GPIO17, ADC2_CH6, DAC_1
20	5V0	P	5 V power supply
21	GND	G	Ground

J3

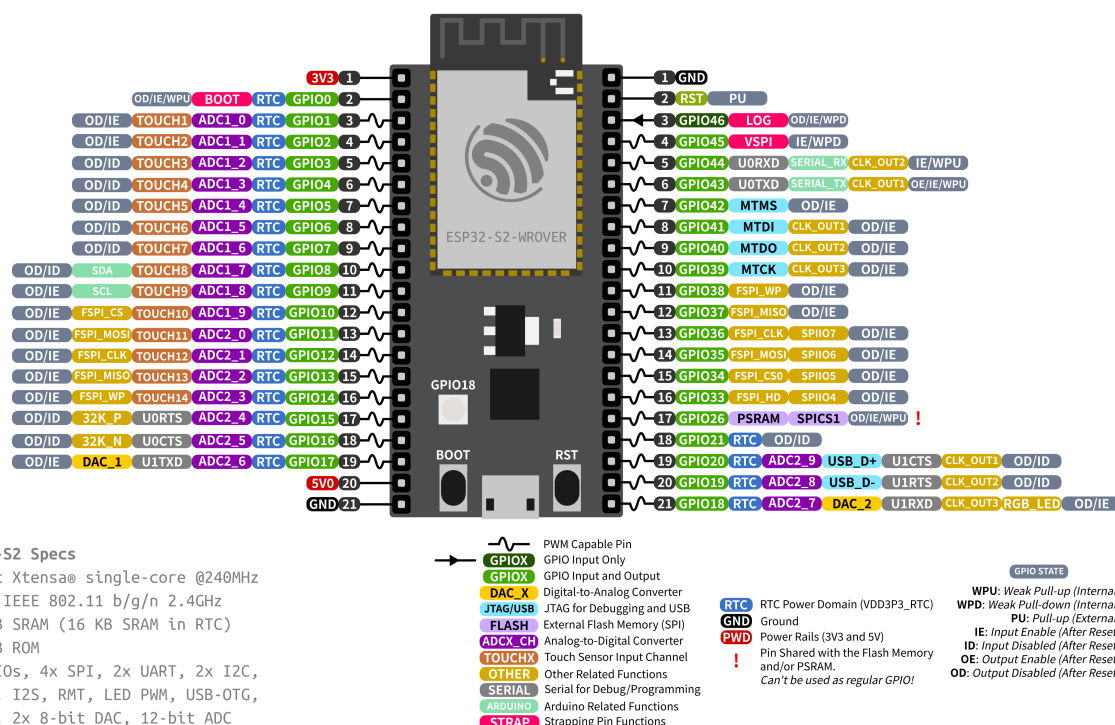
No.	Name	Type	Function
1	GND	G	Ground
2	RST	I	CHIP_PU, Reset
3	IO46	I	GPIO46
4	IO45	I/O	GPIO45
5	IO44	I/O	GPIO44, U0RXD
6	IO43	I/O	GPIO43, U0TXD
7	IO42	I/O	GPIO42, MTMS
8	IO41	I/O	GPIO41, MTDI
9	IO40	I/O	GPIO40, MTDO
10	IO39	I/O	GPIO39, MTCK
11	IO38	I/O	GPIO38
12	IO37	I/O	GPIO37
13	IO36	I/O	GPIO36
14	IO35	I/O	GPIO35
16	IO34	I/O	GPIO34
17	IO33	I/O	GPIO33
17	IO26	I/O	GPIO26
18	IO21	I/O	GPIO21
19	IO20	I/O	GPIO20, ADC2_CH3, USB_D+
20	IO19	I/O	GPIO19, ADC2_CH3, USB_D-
21	IO18	I/O	GPIO18, ADC2_CH3, DAC_2

P: Power supply; I: Input; O: Output; T: High impedance.

Pin Layout

Strapping Pins Some of the GPIO's have important features during the booting process. Here is the list of the strapping pins on the [ESP32-S2](#).

ESP32-S2-Saola-1



GPIO	Default	Function	Pull-up	Pull-down
IO45	Pull-down	Voltage of Internal LDO (VDD_SDIO)	1.8 V	3.3 V
IO0	Pull-up	Bootling Mode	SPI Boot	Download Boot
IO46	Pull-down	Bootling Mode	Don't Care	Download Boot
IO46	Pull-up	Enabling/Disabling Log Print During Bootling and Timing of SDIO Slave	U0TXD Active	U0TXD Silent

For more detailed information, see the [ESP32-S2](#) datasheet.

Restricted Usage GPIOs Some of the GPIO's are used for the external flash and PSRAM. These GPIO's cannot be used:

GPIO	Shared Function
IO26	Connected to PSRAM

Other GPIO's are *INPUT ONLY* and cannot be used as output pin:

GPIO	Function
IO46	GPIO46

Resources

- [ESP32-S2](#) (Datasheet)
- [ESP32-S2-WROVER](#) (Datasheet)
- [ESP32-S2-Saola-1](#) (Schematics)

ESP32-C3-DevKitM-1 The ESP32-C3-DevKitM-1 development board is one of Espressif's official boards. This board is based on the [ESP32-C3-MINI-1](#) module, with the [ESP32-C3](#) as the core.

Specifications

- Small sized 2.4 GHz Wi-Fi (802.11b/g/n) and Bluetooth® 5 module
 - Built around ESP32-C3 series of SoCs, RISC-V single-core microprocessor
 - 4 MB flash in chip package
 - 15 available GPIOs (module)
 - **Peripherals**
 - 22 × programmable GPIOs
 - Digital interfaces:
 - 3 × SPI
 - 2 × UART
 - 1 × I2C
 - 1 × I2S
 - Remote control peripheral, with 2 transmit channels and 2 receive channels
 - LED PWM controller, with up to 6 channels
 - Full-speed USB Serial/JTAG controller
 - General DMA controller (GDMA), with 3 transmit channels and 3 receive channels
 - 1 × TWAI® controller (compatible with ISO 11898-1)
 - **Analog interfaces:**
 - * 2 × 12-bit SAR ADCs, up to 6 channels
 - * 1 × temperature sensor
 - **Timers:**
 - * 2 × 54-bit general-purpose timers
 - * 3 × watchdog timers
 - * 1 × 52-bit system timer
- PCB antenna or external antenna connector

Header Block

Note: Not all of the chip pins are exposed to the pin headers.

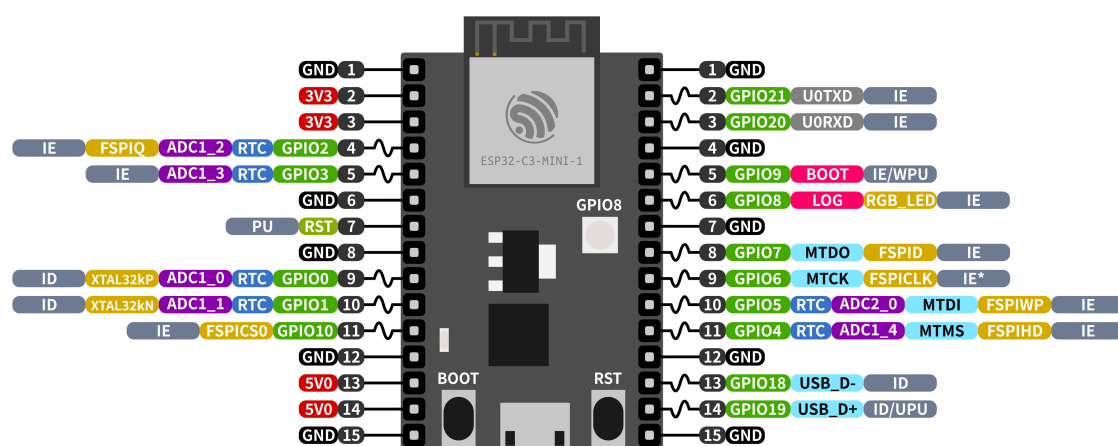
J1

No.	Name	Type	Page 24, 1	Function
1	GND	G		Ground
2	3V3	P		3.3 V power supply
3	3V3	P		3.3 V power supply
4	IO2	I/O/T		GPIO2 ² , ADC1_CH2, FSPIQ
5	IO3	I/O/T		GPIO3, ADC1_CH3
6	GND	G		Ground
7	RST	I		CHIP_PU
8	GND	G		Ground
9	IO0	I/O/T		GPIO0, ADC1_CH0, XTAL_32K_P
10	IO1	I/O/T		GPIO1, ADC1_CH1, XTAL_32K_N
11	IO10	I/O/T		GPIO10, FSPICS0
12	GND	G		Ground
13	5V	P		5 V power supply
14	5V	P		5 V power supply
15	GND	G		Ground

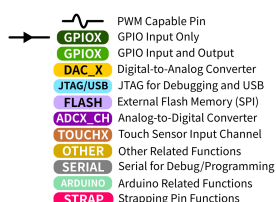
J3

No.	Name	Type ¹	Function
1	GND	G	Ground
2	TX	I/O/T	GPIO21, U0TXD
3	RX	I/O/T	GPIO20, U0RXD
4	GND	G	Ground
5	IO9	I/O/T	GPIO9 ²
6	IO8	I/O/T	GPIO8 ² , RGB LED
7	GND	G	Ground
8	IO7	I/O/T	GPIO7, FSPID, MTDO
9	IO6	I/O/T	GPIO6, FSPICLK, MTCK
10	IO5	I/O/T	GPIO5, ADC2_CH0, FSPWIP, MTDI
11	IO4	I/O/T	GPIO4, ADC1_CH4, FSPHID, MTMS
12	GND	G	Ground
13	IO18	I/O/T	GPIO18, USB_D-
14	IO19	I/O/T	GPIO19, USB_D+
15	GND	G	Ground

ESP32-C3-DevKitM-1

**ESP32-C3 Specs**

32-bit RISC-V single-core @160MHz
 Wi-Fi IEEE 802.11 b/g/n 2.4GHz
 Bluetooth LE 5
 400 KB SRAM (16 KB for cache)
 384 KB ROM
 22 GPIOs, 3x SPI, 2x UART, I2C,
 I2S, RMT, LED PWM, USB Serial/JTAG,
 GDMA, TWAI®, 12-bit ADC



RTC: RTC Power Domain (VDD3P3_RTC)
 GND: Ground
 PWD: Power Rails (3V3 and 5V)

GPIO STATE
 LPU: USB Weak Pull-up
 WPU: Weak Pull-up (Internal)
 WPD: Weak Pull-down (Internal)
 PU: Pull-up (External)
 IE: Input Enable (After Reset)
 IE*: Input Enable (Depends of FUSE_DIS_PAD, JTAG)
 ID: Input Disabled (After Reset)
 OE: Output Enable (After Reset)
 OD: Output Disabled (After Reset)

Pin Layout

Strapping Pins Some of the GPIO's have important features during the booting process. Here is the list of the strapping pins on the **ESP32-C3**.

¹ P: Power supply; I: Input; O: Output; T: High impedance.

² GPIO2, GPIO8, and GPIO9 are strapping pins of the ESP32-C3FN4 chip. During the chip's system reset, the latches of the strapping pins sample the voltage level as strapping bits, and hold these bits until the chip is powered down or shut down.

GPIO	Default	Function	Pull-up	Pull-down
IO2	N/A	Booting Mode	See ESP32-C3	See ESP32-C3
IO9	Pull-up	Booting Mode	SPI Boot	Download Boot
IO8	N/A	Booting Mode	Don' t Care	Download Boot
IO8	Pull-up	Enabling/Disabling Log Print	See ESP32-C3	See ESP32-C3

For more detailed information, see the [ESP32-C3](#) datasheet.

Resources

- [ESP32-C3](#) (Datasheet)
- [ESP32-C3-MINI-1](#) (Datasheet)

Note: Only a few development boards are described on this documentation page. For more information about other Espressif development boards please refer to the [Espressif website](#).

Third Party

Add here the third party boards, listed by vendors.

Note: All the information must be provided by the vendor. If your favorite board is not here, consider creating an [issue on GitHub](#) and directly link/mention the vendor in the issue description.

LOLIN

-
-

Generic Vendor

Generic ESP32 Boards Add here the board/kit specifications.

Header1

No.	Name	Type	Function
1	3V3	P	3.3 V power supply
2	IO0	I/O	GPIO0, Boot
3	5V0	P	5 V power supply
4	GND	G	Ground

Pin Layout Add here the pin layout image (not required).

- [ESP32](#) (Datasheet)
- [ESP32-C2](#) (Datasheet)
- [ESP32-C3](#) (Datasheet)
- [ESP32-C5](#) (Datasheet)
- [ESP32-C6](#) (Datasheet)
- [ESP32-C61](#) (Datasheet)
- [ESP32-H2](#) (Datasheet)
- [ESP32-P4](#) (Datasheet)
- [ESP32-S2](#) (Datasheet)

- [ESP32-S3](#) (Datasheet)

Datasheet

Note: Create one file per board or one file with multiple boards. Do not add board information/description on this file.

Datasheet

- [ESP32](#) (Datasheet)
- [ESP32-C2](#) (Datasheet)
- [ESP32-C3](#) (Datasheet)
- [ESP32-C5](#) (Datasheet)
- [ESP32-C6](#) (Datasheet)
- [ESP32-C61](#) (Datasheet)
- [ESP32-H2](#) (Datasheet)
- [ESP32-P4](#) (Datasheet)
- [ESP32-S2](#) (Datasheet)
- [ESP32-S3](#) (Datasheet)

Resources

1.11 Examples

After installing the toolchain into your environment, you will be able to see all the dedicated examples for the ESP32. These examples are located in the examples menu or inside each library folder.

<https://github.com/espressif/arduino-esp32/tree/master/libraries>

There is also a [list of examples](#) managed outside of Espressif, so check them out.

1.12 Datasheet

- [ESP32](#) (Datasheet)
- [ESP32-C2](#) (Datasheet)
- [ESP32-C3](#) (Datasheet)
- [ESP32-C5](#) (Datasheet)
- [ESP32-C6](#) (Datasheet)
- [ESP32-C61](#) (Datasheet)
- [ESP32-H2](#) (Datasheet)
- [ESP32-P4](#) (Datasheet)
- [ESP32-S2](#) (Datasheet)
- [ESP32-S3](#) (Datasheet)

1.13 Resources

Chapter 2

Libraries

Arduino libraries help you use the features of the ESP32 family of chips with the familiar Arduino API.

2.1 Supported Features and Peripherals

Currently, the Arduino ESP32 supports almost everything available on the ESP32 family with an Arduino-like API.

Not all features are available on all SoCs. Please check the [Product Selector](#) page for more details.

Here is a matrix of the library support status for the main features and peripherals per SoC:

- ✓ Supported through the Arduino Core
- ✗ Not supported through the Arduino Core. It can still be used through the ESP-IDF API, but might require rebuilding the static libraries.
- — Not available on the SoC

Feature	ESP32	C2	C3	C5	C6	C61	H2	P4	S2	S3
ADC ¹	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
BT Classic	✓	—	—	—	—	—	—	—	—	—
BLE ²	✓	✓	✓	✓	✓	✓	✓	✓	—	✓
Console	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
DAC	✓	—	—	—	—	—	—	—	✓	—
ESP-NOW ³	✓	✓	✓	✓	✓	✓	—	—	✓	✓
Ethernet ⁴	✓	—	—	—	—	—	—	✓	—	—
GPIO	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Hall Sensor	—	—	—	—	—	—	—	—	—	—
I2C	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
I2S	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
I3C	—	—	—	—	—	—	—	✗	—	—
LEDC	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Matter (Thread)	—	—	—	✓	✓	—	✓	—	—	—
Matter (Wi-Fi) ⁷	✓	✗	✓	✗	✓	✗	—	—	✓	✓

continues on next page

Table 1 – continued from previous page

Feature	ESP32	C2	C3	C5	C6	C61	H2	P4	S2	S3
MIPI CSI	—	—	—	—	—	—	—	✗	—	—
MIPI DSI	—	—	—	—	—	—	—	✗	—	—
Motor PWM	✗	—	—	✗	✗	—	✗	✗	—	✗
MSPI	—	—	—	—	—	—	—	✗	—	—
Pulse Counter	✗	✗	✗	✗	✗	✗	✗	✗	✗	✗
RMT	✓	✓	✓	✓	✓	—	✓	✓	✓	✓
SDIO	✗	—	—	✗	✗	✗	—	✗	—	✗
SD-MMC	✓	—	—	—	—	—	—	—	—	✓
PSRAM	✓	—	—	✓	—	✓	—	✓	✓	✓
Temp. Sensor	—	✓	✓	✓	✓	✓	✓	✓	✓	✓
Thread	—	—	—	✓	✓	—	✓	—	—	—
Timer	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
Touch	✓	—	—	—	—	—	—	✓	✓	✓
TWAI/C FD	✗	—	✗	✗	✗	—	✗	✗	✗	✗
UART	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓
USB OTG	—	—	—	—	—	—	—	✓	✓	✓
USB Serial	—	—	✓	✓	✓	✓	✓	✓	—	✓
Wi-Fi ¹	✓	✓	✓	✓	✓	✓	—	✓	✓	✓
Zigbee ⁵	—	—	—	✓	✓	—	✓	—	—	—

Note: The ESP32-C2 and ESP32-C61 are only supported using Arduino as an ESP-IDF component or by rebuilding the static libraries.

2.2 Datasheet

- [ESP32](#) (Datasheet)
- [ESP32-C2](#) (Datasheet)
- [ESP32-C3](#) (Datasheet)
- [ESP32-C5](#) (Datasheet)

¹ ESP32-P4 calibration schemes not supported yet in IDF and ADC Continuous also lacks IDF support.

² ESP32-P4 only supports Wi-Fi and BLE through another SoC by using `ESP-Hosted`.

³ ESP-NOW is not supported through `ESP-Hosted`.

⁴ SPI Ethernet is supported by all ESP32 families and RMII only for ESP32 and ESP32-P4.

⁶ Matter over Thread is supported by our library but is not included in the pre-compiled libraries for ESP32-C6 and ESP32-C5. In order to use Matter over Thread, you need to use Arduino as an ESP-IDF component or rebuild the static libraries. Check the [Arduino_ESP_Matter_over_OpenThread example](#) for more details.

⁷ Matter over Wi-Fi is supported by our library but is not included in the pre-compiled libraries for ESP32-C5. Although ESP32-C5 has Wi-Fi 2.4 GHz and 5 GHz support, the ESP32 Arduino Matter Library has been pre compiled using Thread only. In order to use Matter over Wi-Fi, you need to build the project using Arduino as an ESP-IDF component and disable Thread network, keeping only Wi-Fi station.

⁵ Non-native Zigbee SoCs can also run Zigbee, but must use another SoC (with Zigbee radio) as a RCP connected by UART/SPI. Check the [Gateway example](#) for more details.

- [ESP32-C6](#) (Datasheet)
- [ESP32-C61](#) (Datasheet)
- [ESP32-H2](#) (Datasheet)
- [ESP32-P4](#) (Datasheet)
- [ESP32-S2](#) (Datasheet)
- [ESP32-S3](#) (Datasheet)

2.3 APIs

The Arduino ESP32 offers some unique APIs, described in this section:

Note: Please be advised that we cannot ensure continuous compatibility between the Arduino Core ESP32 APIs and ESP8266 APIs, as well as Arduino-Core APIs (Arduino.cc). While our aim is to maintain harmony, the addition of new features may result in occasional divergence. We strive to achieve the best possible integration but acknowledge that perfect compatibility might not always be feasible. Please refer to the documentation for any specific considerations.

2.3.1 ADC

About

ADC (analog to digital converter) is a very common peripheral used to convert an analog signal such as voltage to a digital form so that it can be read and processed by a microcontroller.

ADCs are very useful in control and monitoring applications since most sensors (e.g., temperature, pressure, force) produce analog output voltages.

Note: Each SoC or module has a different number of ADC's with a different number of channels and pins available. Refer to datasheet of each board for more info.

Arduino-ESP32 ADC API

ADC OneShot mode The ADC OneShot mode API is fully compatible with Arduino's `analogRead` function. When you call the `analogRead` or `analogReadMilliVolts` function, it returns the result of a single conversion on the requested pin.

analogRead This function is used to get the ADC raw value for a given pin/ADC channel.

```
uint16_t analogRead(uint8_t pin);
```

- `pin` GPIO pin to read analog value

This function will return analog raw value (non-calibrated).

analogReadMilliVolts This function is used to get ADC raw value for a given pin/ADC channel and convert it to calibrated result in millivolts.

```
uint32_t analogReadMilliVolts(uint8_t pin);
```

- `pin` GPIO pin to read analog value

This function will return analog value in millivolts (calibrated).

analogReadResolution This function is used to set the resolution of `analogRead` return value. Default is 12 bits (range from 0 to 4095) for all chips. When different resolution is set, the values read will be shifted to match the given resolution.

Note: For **ESP32-S2 chip revision v0.0**, the default ADC resolution is 13 bits (0-8191) due to the [ADC-112 errata](#). This is fixed in later revisions (v1.0+), which use the standard 12-bit resolution.

Range is 1 - 16 .The default value will be used, if this function is not used.

Note: For the ESP32, the resolution is between 9 to 12 and it will change the ADC hardware resolution. Else value will be shifted.

```
void analogReadResolution(uint8_t bits);
```

- `bits` sets analog read resolution

analogSetAttenuation This function is used to set the attenuation for all channels.

Input voltages can be attenuated before being input to the ADCs. There are 4 available attenuation options, the higher the attenuation is, the higher the measurable input voltage could be.

The measurable input voltage differs for each chip, see table below for detailed information.

ESP32

Attenuation	Measurable input voltage range
ADC_ATTEN_DB_0	100 mV ~ 950 mV
ADC_ATTEN_DB_2_5	100 mV ~ 1250 mV
ADC_ATTEN_DB_6	150 mV ~ 1750 mV
ADC_ATTEN_DB_11	150 mV ~ 3100 mV

ESP32-S2

Attenuation	Measurable input voltage range
ADC_ATTEN_DB_0	0 mV ~ 750 mV
ADC_ATTEN_DB_2_5	0 mV ~ 1050 mV
ADC_ATTEN_DB_6	0 mV ~ 1300 mV
ADC_ATTEN_DB_11	0 mV ~ 2500 mV

ESP32-C3

Attenuation	Measurable input voltage range
ADC_ATTEN_DB_0	0 mV ~ 750 mV
ADC_ATTEN_DB_2_5	0 mV ~ 1050 mV
ADC_ATTEN_DB_6	0 mV ~ 1300 mV
ADC_ATTEN_DB_11	0 mV ~ 2500 mV

ESP32-S3

Attenuation	Measurable input voltage range
ADC_ATTEN_DB_0	0 mV ~ 950 mV
ADC_ATTEN_DB_2_5	0 mV ~ 1250 mV
ADC_ATTEN_DB_6	0 mV ~ 1750 mV
ADC_ATTEN_DB_11	0 mV ~ 3100 mV

```
void analogSetAttenuation(adc_attenuation_t attenuation);
```

- `attenuation` sets the attenuation.

analogSetPinAttenuation This function is used to set the attenuation for a specific pin/ADC channel. For more information refer to [analogSetAttenuation](#).

```
void analogSetPinAttenuation(uint8_t pin, adc_attenuation_t attenuation);
```

- `pin` selects specific pin for attenuation settings.
- `attenuation` sets the attenuation.

analogSetWidth

Note: This function is only available for ESP32 chip.

This function is used to set the hardware sample bits and read resolution. Default is 12 bits (0 - 4095). Range is 9 - 12.

```
void analogSetWidth(uint8_t bits);
```

ADC Continuous mode ADC Continuous mode is an API designed for performing analog conversions on multiple pins in the background, with the feature of receiving a callback upon completion of these conversions to access the results.

This API allows you to specify the desired number of conversions per pin within a single cycle, along with its corresponding sampling rate. The outcome of the `analogContinuousRead` function is an array of `adc_continuous_result_t` structures. These structures hold both the raw average value and the average value in millivolts for each pin.

analogContinuous This function is used to configure ADC continuous peripheral on selected pins.

```
bool analogContinuous(const uint8_t pins[], size_t pins_count, uint32_t conversions_per_pin, uint32_t sampling_freq_hz, void (*userFunc)(void));
```

- `pins[]` array of pins to be set up
- `pins_count` count of pins in array
- `conversions_per_pin` sets how many conversions per pin will run each ADC cycle
- `sampling_freq_hz` sets sampling frequency of ADC in Hz
- `userFunc` sets callback function to be called after adc conversion is done (can be set to NULL)

This function will return `true` if configuration is successful. If `false` is returned, error occurs and ADC continuous was not configured.

analogContinuousRead This function is used to read ADC continuous data to the result buffer. The result buffer is an array of `adc_continuous_result_t`.

```
typedef struct {
    uint8_t pin;           /*!<ADC pin */
    uint8_t channel;       /*!<ADC channel */
    int avg_read_raw;      /*!<ADC average raw data */
    int avg_read_mvvolts;  /*!<ADC average voltage in mV */
} adc_continuous_result_t;
```

```
bool analogContinuousRead(adc_continuous_result_t ** buffer, uint32_t timeout_ms);
```

- `buffer` conversion result buffer to read from ADC in `adc_continuous_result_t` format.
- `timeout_ms` time to wait for data in milliseconds.

This function will return `true` if reading is successful and `buffer` is filled with data. If `false` is returned, reading has failed and `buffer` is set to `NULL`.

analogContinuousStart This function is used to start ADC continuous conversions.

```
bool analogContinuousStart();
```

This function will return `true` if ADC continuous is successfully started. If `false` is returned, starting ADC continuous has failed.

analogContinuousStop This function is used to stop ADC continuous conversions.

```
bool analogContinuousStop();
```

This function will return `true` if ADC continuous is successfully stopped. If `false` is returned, stopping ADC continuous has failed.

analogContinuousDeinit This function is used to deinitialize ADC continuous peripheral.

```
bool analogContinuousDeinit();
```

This function will return `true` if ADC continuous is successfully deinitialized. If `false` is returned, deinitialization of ADC continuous has failed.

analogContinuousSetAtten This function is used to set the attenuation for ADC continuous peripheral. For more information refer to [analogSetAttenuation](#).

```
void analogContinuousSetAtten(adc_attenuation_t attenuation);
```

- `attenuation` sets the attenuation (default is 11 dB).

analogContinuousSetWidth This function is used to set the hardware resolution bits. Default value for all chips is 12 bits (0 - 4095).

Note: This function will take effect only for ESP32 chip, as it allows to set resolution in range 9-12 bits.

```
void analogContinuousSetWidth(uint8_t bits);
```

- `bits` sets resolution bits.

Example Applications Here is an example of how to use the ADC in OneShot mode or you can run Arduino example 01.Basics -> AnalogReadSerial.

```
#include <Arduino.h>

void setup() {
  // initialize serial communication at 115200 bits per second:
  Serial.begin(115200);

  //set the resolution to 12 bits (0-4095)
  analogReadResolution(12);
}

void loop() {
  // read the analog / millivolts value for pin 2:
  int analogValue = analogRead(2);
  int analogVolts = analogReadMilliVolts(2);

  // print out the values you read:
  Serial.printf("ADC analog value = %d\n", analogValue);
  Serial.printf("ADC millivolts value = %d\n", analogVolts);

  delay(100); // delay in between reads for clear read from serial
}
```

Here is an example of how to use the ADC in Continuous mode.

```
#include <Arduino.h>

// Define how many conversion per pin will happen and reading the data will be and
// average of all conversions
#define CONVERSIONS_PER_PIN 5

// Declare array of ADC pins that will be used for ADC Continuous mode - ONLY ADC1
// pins are supported
// Number of selected pins can be from 1 to ALL ADC1 pins.
#ifdef CONFIG_IDF_TARGET_ESP32
uint8_t adc_pins[] = {36, 39, 34, 35}; //some of ADC1 pins for ESP32
#else
uint8_t adc_pins[] = {1, 2, 3, 4}; //ADC1 common pins for ESP32S2/S3 + ESP32C3/C6
//+ ESP32H2
#endif

// Calculate how many pins are declared in the array - needed as input for the
// setup function of ADC Continuous
uint8_t adc_pins_count = sizeof(adc_pins) / sizeof(uint8_t);

// Flag which will be set in ISR when conversion is done
volatile bool adc_conversion_done = false;

// Result structure for ADC Continuous reading
adc_continuous_result_t *result = NULL;

// ISR Function that will be triggered when ADC conversion is done
void ARDUINO_ISR_ATTR adcComplete() {
  adc_conversion_done = true;
}

void setup() {
  // Initialize serial communication at 115200 bits per second:
  Serial.begin(115200);
```

(continues on next page)

(continued from previous page)

```

// Optional for ESP32: Set the resolution to 9-12 bits (default is 12 bits)
analogContinuousSetWidth(12);

// Optional: Set different attenuation (default is ADC_11db)
analogContinuousSetAtten(ADC_11db);

// Setup ADC Continuous with following input:
// array of pins, count of the pins, how many conversions per pin in one cycle_
↳will happen, sampling frequency, callback function
analogContinuous(adc_pins, adc_pins_count, CONVERSIONS_PER_PIN, 20000, &
↳adcComplete);

// Start ADC Continuous conversions
analogContinuousStart();
}

void loop() {
    // Check if conversion is done and try to read data
    if (adc_conversion_done == true) {
        // Set ISR flag back to false
        adc_conversion_done = false;
        // Read data from ADC
        if (analogContinuousRead(&result, 0)) {

            // Optional: Stop ADC Continuous conversions to have more time to process_
            ↳(print) the data
            analogContinuousStop();

            for (int i = 0; i < adc_pins_count; i++) {
                Serial.printf("\nADC PIN %u data:", result[i].pin);
                Serial.printf("\n    Avg raw value = %d", result[i].avg_read_raw);
                Serial.printf("\n    Avg millivolts value = %d", result[i].avg_read_mv);
            }

            // Delay for better readability of ADC data
            delay(1000);

            // Optional: If ADC was stopped, start ADC conversions and wait for callback_
            ↳function to set adc_conversion_done flag to true
            analogContinuousStart();
        } else {
            Serial.println("Error occurred during reading data. Set Core Debug Level to_
            ↳error or lower for more information.");
        }
    }
}
}

```

2.3.2 BLE

About

Note: This is a work in progress project and this section is still missing. If you want to contribute, please see the [Contributions Guide](#).

Examples

To get started with BLE, you can try:

BLE Scan

```

/*
   Based on Neil Kolban example for IDF: https://github.com/nkolban/esp32-snippets/
   ↪ blob/master/cpp_utils/tests/BLE%20Tests/SampleScan.cpp
   Ported to Arduino ESP32 by Evandro Copercini
*/

#include <Arduino.h>
#include <BLEDevice.h>
#include <BLEUtils.h>
#include <BLEScan.h>
#include <BLEAdvertisedDevice.h>

int scanTime = 5; //In seconds
BLEScan *pBLEScan;

class MyAdvertisedDeviceCallbacks : public BLEAdvertisedDeviceCallbacks {
    void onResult(BLEAdvertisedDevice advertisedDevice) {
        Serial.printf("Advertised Device: %s \n", advertisedDevice.toString().c_str());
    }
};

void setup() {
    Serial.begin(115200);
    Serial.println("Scanning...");

    BLEDevice::init("");
    pBLEScan = BLEDevice::getScan(); //create new scan
    pBLEScan->setAdvertisedDeviceCallbacks(new MyAdvertisedDeviceCallbacks());
    pBLEScan->setActiveScan(true); //active scan uses more power, but get results
    ↪ faster
    pBLEScan->setInterval(100);
    pBLEScan->setWindow(99); // less or equal setInterval value
}

void loop() {
    // put your main code here, to run repeatedly:
    BLEScanResults *foundDevices = pBLEScan->start(scanTime, false);
    Serial.print("Devices found: ");
    Serial.println(foundDevices->getCount());
    Serial.println("Scan done!");
    pBLEScan->clearResults(); // delete results from BLEScan buffer to release memory
    delay(2000);
}

```

BLE UART

```

/*
   Video: https://www.youtube.com/watch?v=oCMOYS71NIU
   Based on Neil Kolban example for IDF: https://github.com/nkolban/esp32-
   ↪ snippets/blob/master/cpp_utils/tests/BLE%20Tests/SampleNotify.cpp
   Ported to Arduino ESP32 by Evandro Copercini

   Create a BLE server that, once we receive a connection, will send periodic
   ↪ notifications.
   The service advertises itself as: 6E400001-B5A3-F393-E0A9-E50E24DCCA9E
   Has a characteristic of: 6E400002-B5A3-F393-E0A9-E50E24DCCA9E - used for
   ↪ receiving data with "WRITE"
   Has a characteristic of: 6E400003-B5A3-F393-E0A9-E50E24DCCA9E - used to send
   ↪ data with "NOTIFY"

```

(continues on next page)

(continued from previous page)

```

    The design of creating the BLE server is:
    1. Create a BLE Server
    2. Create a BLE Service
    3. Create a BLE Characteristic on the Service
    4. Create a BLE Descriptor on the characteristic
    5. Start the service.
    6. Start advertising.

    In this example rxValue is the data received (only accessible inside that_
    ↪function).
    And txValue is the data to be sent, in this example just a byte incremented_
    ↪every second.
    */

#include <Arduino.h>
#include <BLEDevice.h>
#include <BLEServer.h>
#include <BLEUtils.h>
#include <BLE2902.h>

BLEServer *pServer = NULL;
BLECharacteristic *pTxCharacteristic;
bool deviceConnected = false;
bool oldDeviceConnected = false;
uint8_t txValue = 0;

// See the following for generating UUIDs:
// https://www.uuidgenerator.net/

#define SERVICE_UUID          "6E400001-B5A3-F393-E0A9-E50E24DCCA9E" // UART_
    ↪service UUID
#define CHARACTERISTIC_UUID_RX "6E400002-B5A3-F393-E0A9-E50E24DCCA9E"
#define CHARACTERISTIC_UUID_TX "6E400003-B5A3-F393-E0A9-E50E24DCCA9E"

class MyServerCallbacks : public BLEServerCallbacks {
    void onConnect(BLEServer *pServer) {
        deviceConnected = true;
        Serial.println("Device connected");
    };

    void onDisconnect(BLEServer *pServer) {
        deviceConnected = false;
        Serial.println("Device disconnected");
    }
};

class MyCallbacks : public BLECharacteristicCallbacks {
    void onWrite(BLECharacteristic *pCharacteristic) {
        String rxValue = pCharacteristic->getValue();

        if (rxValue.length() > 0) {
            Serial.println("*****");
            Serial.print("Received Value: ");
            for (int i = 0; i < rxValue.length(); i++) {
                Serial.print(rxValue[i]);
            }

            Serial.println();
            Serial.println("*****");
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

};

void setup() {
  Serial.begin(115200);

  // Create the BLE Device
  BLEDevice::init("UART Service");

  // Create the BLE Server
  pServer = BLEDevice::createServer();
  pServer->setCallbacks(new MyServerCallbacks());

  // Create the BLE Service
  BLEService *pService = pServer->createService(SERVICE_UUID);

  // Create a BLE Characteristic
  pTxCharacteristic = pService->createCharacteristic(CHARACTERISTIC_UUID_TX,
↳BLECharacteristic::PROPERTY_NOTIFY);

  // Descriptor 2902 is not required when using NimBLE as it is automatically
↳added based on the characteristic properties
  pTxCharacteristic->addDescriptor(new BLE2902());

  BLECharacteristic *pRxCharacteristic = pService->
↳createCharacteristic(CHARACTERISTIC_UUID_RX, BLECharacteristic::PROPERTY_WRITE);

  pRxCharacteristic->setCallbacks(new MyCallbacks());

  // Start the service
  pService->start();

  // Start advertising
  pServer->getAdvertising()->start();
  Serial.println("Waiting a client connection to notify...");
}

void loop() {

  if (deviceConnected) {
    Serial.print("Notifying Value: ");
    Serial.println(txValue);
    pTxCharacteristic->setValue(&txValue, 1);
    pTxCharacteristic->notify();
    txValue++;
    delay(1000); // Notifying every 1 second
  }

  // disconnecting
  if (!deviceConnected && oldDeviceConnected) {
    delay(500); // give the bluetooth stack the chance to get
↳things ready
    pServer->startAdvertising(); // restart advertising
    Serial.println("Started advertising again...");
    oldDeviceConnected = false;
  }

  // connecting
  if (deviceConnected && !oldDeviceConnected) {
    // do stuff here on connecting
    oldDeviceConnected = true;
  }
}

```

Complete list of [BLE examples](#).

2.3.3 Bluetooth

About

Note: This is a work in progress project and this section is still missing. If you want to contribute, please see the [Contributions Guide](#).

Examples

To get started with Bluetooth, you can try:

Serial To Serial BT

```
// This example code is in the Public Domain (or CC0 licensed, at your option.)
// By Evandro Copercini - 2018
//
// This example creates a bridge between Serial and Classical Bluetooth (SPP)
// and also demonstrate that SerialBT have the same functionalities of a normal
↪Serial
// Note: Pairing is authenticated automatically by this device

#include <Arduino.h>
#include "BluetoothSerial.h"

String device_name = "ESP32-BT-Slave";

// Check if Bluetooth is available
#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth is not enabled! Please run `make menuconfig` to and enable it
#endif

// Check Serial Port Profile
#if !defined(CONFIG_BT_SPP_ENABLED)
#error Serial Port Profile for Bluetooth is not available or not enabled. It is
↪only available for the ESP32 chip.
#endif

BluetoothSerial SerialBT;

void setup() {
  Serial.begin(115200);
  SerialBT.begin(device_name); //Bluetooth device name
  //SerialBT.deleteAllBondedDevices(); // Uncomment this to delete paired devices;
↪Must be called after begin
  Serial.printf("The device with name \"%s\" is started.\nNow you can pair it with
↪Bluetooth!\n", device_name.c_str());
}

void loop() {
  if (Serial.available()) {
    SerialBT.write(Serial.read());
  }
  if (SerialBT.available()) {
    Serial.write(SerialBT.read());
  }
}
```

(continues on next page)

(continued from previous page)

```

    delay(20);
}

```

BT Classic Device Discovery

```

#include <Arduino.h>
#include <BluetoothSerial.h>

#if !defined(CONFIG_BT_ENABLED) || !defined(CONFIG_BLUEDROID_ENABLED)
#error Bluetooth is not enabled! Please run `make menuconfig` to and enable it
#endif

#if !defined(CONFIG_BT_SPP_ENABLED)
#error Serial Bluetooth not available or not enabled. It is only available for the_
↳ESP32 chip.
#endif

BluetoothSerial SerialBT;

#define BT_DISCOVER_TIME 10000

static bool btScanAsync = true;
static bool btScanSync = true;

void btAdvertisedDeviceFound(BTAdvertisedDevice *pDevice) {
    Serial.printf("Found a device asynchronously: %s\n", pDevice->toString().c_
↳str());
}

void setup() {
    Serial.begin(115200);
    SerialBT.begin("ESP32test"); //Bluetooth device name
    Serial.println("The device started, now you can pair it with bluetooth!");

    if (btScanAsync) {
        Serial.print("Starting asynchronous discovery... ");
        if (SerialBT.discoverAsync(btAdvertisedDeviceFound)) {
            Serial.println("Findings will be reported in \"btAdvertisedDeviceFound\"");
            delay(10000);
            Serial.print("Stopping discoverAsync... ");
            SerialBT.discoverAsyncStop();
            Serial.println("stopped");
        } else {
            Serial.println("Error on discoverAsync f.e. not working after a \"connect\"
↳");
        }
    }

    if (btScanSync) {
        Serial.println("Starting synchronous discovery... ");
        BTScanResults *pResults = SerialBT.discover(BT_DISCOVER_TIME);
        if (pResults) {
            pResults->dump(&Serial);
        } else {
            Serial.println("Error on BT Scan, no result!");
        }
    }
}

void loop() {

```

(continues on next page)

(continued from previous page)

```
    delay(100);  
}
```

Complete list of [Bluetooth examples](#).

2.3.4 Console

About

The Console library provides an interactive command-line interface (CLI) for ESP32 applications. It wraps the ESP-IDF `console` component and exposes an Arduino-style API for registering commands, starting a background REPL (Read-Eval-Print Loop) task, managing command history, and parsing typed arguments using the `argtable3` library.

Features include:

- Background REPL task with configurable stack, priority, and core affinity.
- Tab completion and inline hints powered by the `linenoise` library.
- Persistent command history saved to LittleFS or SPIFFS.
- Typed argument parsing via `argtable3` (integers, strings, optional arguments, etc.).
- Context-aware command callbacks for object-oriented design.
- Transport-agnostic: works with UART `Serial` and `HWCDC / USB-JTAG` (`HWCDCSerial`) —the same transport that `Serial` uses is detected automatically from the `ARDUINO_USB_MODE / ARDUINO_USB_CDC_ON_BOOT` build flags.

Note: The Console REPL currently does **not** support USB OTG (CDC via `TinyUSB / USBSerial`). On boards where USB OTG is selected as the `Serial` transport, the REPL will not receive input. Use UART or `HWCDC` instead.

Note that some features like history and tab completion are only available on terminals that support VT100 escape sequences. The Arduino IDE Serial Monitor does not support them. If needed, you can use any other terminal that supports VT100 escape sequences, such as PuTTY, Minicom or Picocom. If your terminal prints weird sequences of characters like `[5n`, it means that the terminal does not support VT100 escape sequences. To disable VT100 escape sequences, you can use the `setPlainMode()` method and use a plain text mode instead.

The Console library is usable by all ESP32 variants. Depending on how the board's USB and Serial are implemented, you may need to ensure that USB CDC is enabled on boot and use the Hardware CDC. For example, on the most recent ESP32-P4 revisions, you need to enable `USB CDC On Boot` and set the `USB Mode` to `Hardware CDC` and `JTAG`.

Header File

```
#include <Console.h>
```

Overview

The library exposes a single global object `Console` of type `ConsoleClass`.

Typical usage:

1. (Optional) Call configuration methods such as `setPrompt()`, `setMaxHistory()`, and `setHistoryFile()` **before** calling `begin()`.
2. Call `Console.begin()` to initialize the underlying `esp_console` module.
3. Register commands with `Console.addCmd()` or `Console.addCmdWithContext()`.
4. Call `Console.addHelpCmd()` to register the built-in help command.

5. Call `Console.attachToSerial(true)` to start the interactive session in a background task.

Alternatively, skip `attachToSerial(true)` and call `Console.run()` manually from your own input loop (see the `ConsoleManual` example).

Two function-pointer typedefs are provided for command callbacks:

```
// Simple callback
typedef int (*ConsoleCommandFunc)(int argc, char **argv);

// Context-aware callback (receives a user-supplied pointer as the first argument)
typedef int (*ConsoleCommandFuncWithCtx)(void *context, int argc, char **argv);
```

Both `argtable3` (<console/argtable3/argtable3.h>) and `linenoise` (<console/linenoise/linenoise.h>) headers are included by `Console.h` and are directly available in your sketch.

Argument Parsing with argtable3

The Console library re-exports the `argtable3` C library, which provides GNU-style argument parsing for your commands. Instead of manually parsing `argv` strings, you declare typed argument descriptors and let `argtable` handle validation, error messages, and automatic hint generation.

Declaring Arguments Each argument is created by calling a constructor function. The two most common variants are:

- `arg_xxx1(...)` —**required** argument (exactly 1 occurrence).
- `arg_xxx0(...)` —**optional** argument (0 or 1 occurrence).

Available types:

Constructor	Value field	Description
<code>arg_int0</code> <code>arg_int1</code>	<code>/ ->ival[0]</code>	Integer (decimal, hex <code>0x</code> , or octal <code>0</code> prefix)
<code>arg_db10</code> <code>arg_db11</code>	<code>/ ->dval[0]</code>	Double-precision floating point
<code>arg_str0</code> <code>arg_str1</code>	<code>/ ->sval[0]</code>	String (no whitespace unless quoted)
<code>arg_lit0</code> <code>arg_lit1</code>	<code>/ ->count</code>	Boolean flag (no value — presence means <i>true</i>)
<code>arg_end</code>	<i>(none)</i>	Sentinel that terminates the table and stores parse errors

Every constructor takes four parameters:

```
struct arg_xxx* arg_xxx1(
    const char *shortopts, // Short option letter, e.g. "p" for -p (or NULL)
    const char *longopts,  // Long option name, e.g. "pin" for --pin (or NULL)
    const char *datatype,  // Placeholder shown in hints, e.g. "<pin>"
    const char *glossary   // Description shown in help text
);
```

When both `shortopts` and `longopts` are `NULL`, the argument is **positional** (matched by order, not by a flag).

Building an Argument Table Group all argument descriptors into a `struct` that ends with `arg_end`:

```
// Declare a global struct for the argument table
static struct {
    struct arg_int *pin; // positional, required: GPIO pin number
```

(continues on next page)

(continued from previous page)

```

    struct arg_int *value;    // positional, required: 0 or 1
    struct arg_end *end;      // must always be last — tracks parse errors
} my_args;

void setup() {
    // Allocate and initialize each argument descriptor.
    // arg_int1: required int arg. (NULL, NULL) = positional (no -x/--xx flag).
    my_args.pin = arg_int1(NULL, NULL, "<pin>", "GPIO pin number");
    my_args.value = arg_int1(NULL, NULL, "<0|1>", "0 = LOW, 1 = HIGH");
    // arg_end(N): sentinel — N is the max number of errors to store.
    my_args.end = arg_end(2);

    // Register the command, passing &my_args as the argtable.
    // The Console library auto-generates the hint string from the table.
    Console.addCmd("gpio", "Write a GPIO pin", &my_args, cmd_gpio);
}

```

Parsing Inside a Command Handler Inside the command callback, call `arg_parse()` to validate the arguments and fill in the values:

```

static int cmd_gpio(int argc, char **argv) {
    // Parse argv against the argument table
    int nerrors = arg_parse(argc, argv, (void **)&my_args);
    if (nerrors != 0) {
        // Print human-readable error messages to stdout
        arg_print_errors(stdout, my_args.end, argv[0]);
        return 1;
    }

    // Access the parsed values through the typed fields
    int pin = my_args.pin->ival[0];    // first (and only) int value
    int val = my_args.value->ival[0];

    pinMode(pin, OUTPUT);
    digitalWrite(pin, val ? HIGH : LOW);
    printf("GPIO %d = %d\n", pin, val);
    return 0;
}

```

Optional Arguments and Flags Use `arg_xxx0` for optional parameters, and check `->count` to see if they were supplied:

```

static struct {
    struct arg_str *ssid;      // required positional string
    struct arg_str *password;  // optional positional string
    struct arg_lit *verbose;   // optional flag: -v / --verbose
    struct arg_end *end;
} connect_args;

void setup() {
    connect_args.ssid = arg_str1(NULL, NULL, "<ssid>", "Network name");
    connect_args.password = arg_str0(NULL, NULL, "[<password>]", "Password (optional)");
    connect_args.verbose = arg_lit0("v", "verbose", "Print extra info");
    connect_args.end = arg_end(3);
    // ...
}

```

(continues on next page)

(continued from previous page)

```
static int cmd_connect(int argc, char **argv) {
    int nerrors = arg_parse(argc, argv, (void **)&connect_args);
    if (nerrors != 0) {
        arg_print_errors(stdout, connect_args.end, argv[0]);
        return 1;
    }

    const char *ssid = connect_args.ssid->sval[0];

    // Check if the optional password was supplied
    const char *pass = (connect_args.password->count > 0)
        ? connect_args.password->sval[0]
        : "";

    // Check if the -v / --verbose flag was present
    bool verbose = (connect_args.verbose->count > 0);

    // ...
    return 0;
}
```

See the `ConsoleGPIO` and `ConsoleWiFi` examples for complete working sketches.

Arduino-esp32 Console API

begin

Initialize the console module. Must be called before any other method.

```
bool begin(size_t maxCmdLen = 256, size_t maxArgs = 32)
```

Parameters

- **maxCmdLen (Optional)**
 - Maximum length of a single command-line in bytes. Default: 256.
- **maxArgs (Optional)**
 - Maximum number of whitespace-separated tokens on a line. Default: 32.

Returns

- true on success; false if `esp_console_init()` fails.

Notes

- Sets up linenoise tab-completion and hints automatically.
- Loads history from the file set by `setHistoryFile()` if one was configured.
- Calling `begin()` again after it has already succeeded has no effect.

end

De-initialize the console module.

```
void end()
```

Notes

- Stops the REPL task if running.
- Saves command history to the configured file before freeing resources.

setPrompt

Set the prompt string displayed at the start of each input line.


```
void setPrompt(const char *prompt)
void setPrompt(const String &prompt)
```

Parameters

- `prompt` — Prompt string. Default: "esp> ".

Notes

- Must be called before `begin()` to take effect in the REPL task.

setMaxHistory

Set the maximum number of command history entries kept in RAM.

```
void setMaxHistory(uint32_t maxlen)
```

Parameters

- `maxLen` — Maximum history entries. Default: 32.

setHistoryFile

Set the filesystem and file path for persistent command history.

```
void setHistoryFile(fs::FS &fs, const char *path)
void setHistoryFile(fs::FS &fs, const String &path)
```

The method combines `fs.mountpoint()` and `path` internally to build the full VFS path used by `linenoise` for loading and saving history. You only need to pass the filesystem-relative path.

Parameters

- `path` — File path relative to the filesystem root, e.g. `"/history.txt"`.
- `fs` — A mounted Arduino filesystem object. Accepts `LittleFS`, `SPIFFS`, `FFat`, etc.

Notes

- History is loaded from this file at `begin()` and saved after every command.
- The filesystem must be mounted before `begin()` is called.

Example

```
LittleFS.begin(true);
Console.setHistoryFile(LittleFS, "/history.txt");
```

setTaskStackSize

Set the REPL background task stack size in bytes.

```
void setTaskStackSize(uint32_t size)
```

Parameters

- `size` — Stack size in bytes. Default: 4096.

setTaskPriority

Set the REPL background task priority.

```
void setTaskPriority(uint32_t priority)
```

Parameters

- `priority` — FreeRTOS task priority. Default: 2.

setTaskCore

Pin the REPL task to a specific CPU core.

```
void setTaskCore(BaseType_t core)
```

Parameters

- `core` — Core index (0 or 1) or `tskNO_AFFINITY`. Default: `tskNO_AFFINITY`.

usePsram

Route all Console PSRAM-eligible allocations to external SPI RAM.

```
void usePsram(bool enable)
```

When enabled:

- The `esp_console` command registry (`heap_alloc_caps` in `esp_console_config_t`) is allocated from PSRAM.
- The REPL FreeRTOS task stack is allocated from PSRAM via `xTaskCreatePinnedToCoreWithCaps()`.

This frees internal SRAM for other uses at the cost of slightly higher latency for stack and heap accesses. Has no effect if PSRAM is not available or not initialized at boot.

Parameters

- `enable` — `true` to use PSRAM, `false` for internal RAM (default: `true`).

Notes

- Must be called before `begin()` to affect heap allocation, and before `attachToSerial(true)` to affect the task stack.

Example

```
Console.usePsram(true); // before begin()
Console.begin();
Console.attachToSerial(true);
```

addCmd

Register a command. Three overloads are available depending on how the hint is provided.

```
// Auto-generated hint (no hint / hint from argtable)
bool addCmd(const char *name, const char *help, ConsoleCommandFunc func)
bool addCmd(const String &name, const String &help, ConsoleCommandFunc
↳func)

// Explicit hint string
bool addCmd(const char *name, const char *help, const char *hint,
↳ConsoleCommandFunc func)
bool addCmd(const String &name, const String &help, const String &hint,
↳ConsoleCommandFunc func)

// argtable3 argument table (hint auto-generated from the table)
bool addCmd(const char *name, const char *help, void *argtable,
↳ConsoleCommandFunc func)
```

Parameters

- **name (Required)**
 - Command name as typed at the prompt. Must not contain spaces. The pointer must remain valid until `end()` is called.
- **help (Required)**
 - Help text shown by the built-in `help` command.

- **hint (Optional)**

- Short argument synopsis shown inline while typing, e.g. "<pin> <value>".

- **argtable (Optional)**

- Pointer to an argtable3 argument table (a struct ending with `arg_end`). The hint is generated automatically from the table. The table is only read during registration.

- **func (Required)**

- Command handler. Receives `argc / argv` where `argv[0]` is the command name. Return 0 for success, non-zero for failure.

Returns

- true on success.

addCmdWithContext

Register a context-aware command.

```
bool addCmdWithContext(const char *name, const char *help,
                      ConsoleCommandFuncWithCtx func, void *ctx)
bool addCmdWithContext(const String &name, const String &help,
                      ConsoleCommandFuncWithCtx func, void *ctx)

bool addCmdWithContext(const char *name, const char *help, const char_
↳ *hint,
                      ConsoleCommandFuncWithCtx func, void *ctx)
bool addCmdWithContext(const String &name, const String &help, const_
↳ String &hint,
                      ConsoleCommandFuncWithCtx func, void *ctx)

bool addCmdWithContext(const char *name, const char *help, void *argtable,
                      ConsoleCommandFuncWithCtx func, void *ctx)
```

Parameters

- `func` — Callback of type `ConsoleCommandFuncWithCtx`. The context pointer is passed as the first argument on every call.
- `ctx` — Arbitrary pointer forwarded to `func` as its first argument.

All other parameters are the same as `addCmd`.

Returns

- true on success.

removeCmd

Unregister a previously registered command.

```
bool removeCmd(const char *name)
bool removeCmd(const String &name)
```

Parameters

- `name` — Name of the command to remove.

Returns

- true on success; false if the command was not found.

addHelpCmd

Register the built-in help command.

```
bool addHelpCmd()
```

When called with no arguments, help lists all registered commands with their hints and help text. When called with a command name (e.g. `help gpio`), it prints the details for that command only.

Returns

- `true` on success.

removeHelpCmd

Remove the built-in help command.

```
bool removeHelpCmd()
```

Returns

- `true` on success.

setHelpVerboseLevel

Control the verbosity of the built-in help command.

```
bool setHelpVerboseLevel(int level)
```

Parameters

- **level**
 - 0 —Brief: show command name and hint only.
 - 1 —Verbose: also show the full help text.

Returns

- `true` on success; `false` if `level` is out of range.

attachToSerial

Attach or detach the console from the serial port.

```
bool attachToSerial(bool enable)
```

When `enable` is `true`, starts the Read-Eval-Print Loop in a background FreeRTOS task. The task reads input through `Serial` (bypassing VFS `stdin` to avoid a race condition with the UART/USB driver ISR) and passes each line to `esp_console_run()`. Non-empty entries are added to the command history. Because reading goes through `Serial`, no transport-specific configuration is required —UART and HWCDC work automatically.

When `enable` is `false`, stops the background task and frees the serial port for other use.

Note: USB OTG (CDC via TinyUSB / USBSerial) is **not** currently supported as a REPL transport.

At startup, the task probes the terminal for VT100 support by sending a one-time Device Status Request (ESC[5n). If the terminal does not respond within 500 ms, plain mode is enabled automatically. Use `setPlainMode()` before `attachToSerial(true)` to override the probe result.

Parameters

- `enable` —`true` to attach and start the REPL task, `false` to detach and stop it.

Returns

- `true` on success.

Notes

- `begin()` must be called before `attachToSerial(true)`.
- Calling `attachToSerial(true)` a second time has no effect.
- Calling `attachToSerial(false)` when not attached has no effect.

isAttachedToSerial

Check whether the console is currently attached to the serial port.

```
bool isAttachedToSerial()
```

Returns

- true if the REPL background task is running.

run

Execute a command-line string directly without a REPL task.

```
int run(const char *cmdline)
int run(const String &cmdline)
```

Parameters

- cmdline — Full command-line (command name followed by arguments).

Returns

- The command handler's return code (0 = success).
- -1 if the command was not found or an esp_console error occurred.

Notes

- run() can be called from inside a command handler to invoke other commands (command composition). However, the underlying IDF function shares a single parse buffer, so the caller's argv pointers are **invalidated** after the call returns. Copy any argv values you need into local variables **before** calling run().

```
static int cmd_led(int argc, char **argv) {
    // Read argv BEFORE calling run() — it becomes invalid after.
    bool turnOn = (strcmp(argv[1], "on") == 0);

    if (turnOn) {
        Console.run("gpio write 2 1");
    }
    // Do NOT use argv[1] here.
    return 0;
}
```

- History is **not** updated by run(). Manage history manually when building a custom input loop (see the ConsoleManual example).

clearScreen

Clear the terminal screen.

```
void clearScreen()
```

setMultiLine

Enable or disable multi-line editing mode.

```
void setMultiLine(bool enable)
```

Parameters

- enable — true to allow the input line to wrap across multiple terminal rows. Disabled by default.

setPlainMode

Enable or disable plain (non-VT100) input mode.

```
void setPlainMode(bool enable, bool force = true)
```

In plain mode linenoise falls back to basic line input without cursor movement, arrow-key history navigation, or tab-completion rendering. Useful for terminals that do not support ANSI/VT100 escape sequences (such as the Arduino IDE Serial Monitor).

By default, `attachToSerial(true)` probes the terminal at startup (sending a one-time Device Status Request) and enables plain mode automatically if the terminal does not respond.

Parameters

- `enable` — `true` to enable plain mode, `false` to restore VT100 mode.
- `force` — When `true` (default), the automatic VT100 probe at `attachToSerial(true)` startup is skipped and the mode set here is kept unconditionally. When `false`, the mode is applied immediately but `attachToSerial(true)` may still override it based on the probe result.

Notes

- This wraps linenoise's internal “dumb mode”, a legacy term from the era of character-only display terminals. The Arduino API uses the friendlier name.

isPlainMode

Check whether plain (non-VT100) input mode is currently active.

```
bool isPlainMode()
```

Returns `true` if plain mode is enabled, either because it was set explicitly with `setPlainMode(true)` or because the automatic VT100 probe at `attachToSerial(true)` startup detected a non-VT100 terminal.

Returns

- `true` if plain mode is active, `false` otherwise.

splitArgv

Split a command-line string into argv-style tokens in place.

```
static size_t Console.splitArgv(char *line, char **argv, size_t argv_size)
```

Handles quoted strings (spaces preserved, quotes stripped) and backslash escapes. Modifies the input buffer.

Parameters

- `line` — Null-terminated input string, modified in place.
- `argv` — Output array; each element will point into `line`.
- `argv_size` — Size of the `argv` array. At most `argv_size - 1` tokens are returned; `argv[argc]` is always set to `NULL`.

Returns

- Number of tokens found (`argc`).

Example

```
char line[] = "gpio write 5 1";
char *argv[8];
size_t argc = Console.splitArgv(line, argv, 8);
// argc == 4, argv[0]=="gpio", argv[1]=="write", argv[2]=="5",
// argv[3]=="1"
```

Examples

ConsoleBasic Full interactive REPL with history, tab completion, and inline hints.

```

/*
 * ConsoleBasic — Basic interactive REPL with history and tab completion.
 *
 * This example demonstrates the basic usage of the Console library.
 * It uses the REPL (Read-Eval-Print Loop) API to create an interactive shell.
 * These are the available commands:
 *
 *   version — Print firmware version
 *   heap    — Show free heap memory
 *   restart — Restart the device
 *   echo    — Echo arguments back
 *
 * Connect with any serial terminal (115200 baud) and type "help" to list
 * available commands. Note that for full functionality (like history and tab
↪completion),
 * you need to use a serial terminal that supports VT100 escape sequences (Arduino↪
↪IDE does not).
 *
 * Board support: all ESP32 variants using UART or HWCDC (USB-JTAG).
 * Currently, the Console library does not support USB OTG (CDC via TinyUSB ↪
↪USBSerial).
 *
 * Created by lucasssvaz
 */

#include <Arduino.h>
#include <Console.h>

// -----
// Command: version
// -----
static int cmd_version(int argc, char **argv) {
    printf("arduino-esp32 %s\n", ESP_ARDUINO_VERSION_STR);
    return 0;
}

// -----
// Command: heap
// -----
static int cmd_heap(int argc, char **argv) {
    printf("Free heap : %lu bytes\n", (unsigned long)ESP.getFreeHeap());
    printf("Min heap  : %lu bytes\n", (unsigned long)ESP.getMinFreeHeap());
    return 0;
}

// -----
// Command: restart
// -----
static int cmd_restart(int argc, char **argv) {
    printf("Restarting in 1 second...\n");
    fflush(stdout);
    delay(1000);
    ESP.restart();
    return 0;
}

// -----
// Command: echo <message...>
// -----
static int cmd_echo(int argc, char **argv) {
    for (int i = 1; i < argc; i++) {
        printf("%s", argv[i]);
    }
}

```

(continues on next page)

(continued from previous page)

```

    if (i < argc - 1) {
        printf(" ");
    }
}
printf("\n");
return 0;
}

// -----
// setup / loop
// -----

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Configure Console
    Console.setPrompt("esp32> ");
    Console.setMaxHistory(32);

    // Initialize Console
    if (!Console.begin()) {
        Serial.println("Console init failed");
        return;
    }

    // Add commands
    Console.addCmd("version", "Print firmware version", cmd_version);
    Console.addCmd("heap", "Show free heap memory", cmd_heap);
    Console.addCmd("restart", "Restart the device", cmd_restart);
    Console.addCmd("echo", "Echo arguments back", "<message...>", cmd_echo);

    // Add built-in help command
    Console.addHelpCmd();

    // Begin Read-Eval-Print Loop
    if (!Console.attachToSerial(true)) {
        Serial.println("REPL start failed");
    }
}

void loop() {
    // Loop task is not used in this example, so we delete it to free up resources
    vTaskDelete(NULL);
}

```

ConsoleManual Manual command execution from a custom input loop in `loop()`, without a background task.

```

/*
 * ConsoleManual — Run commands from Serial without a background REPL task.
 *
 * Demonstrates using Console.run() inside loop() to execute commands read
 * from Serial. Useful when you need full control over the input loop or
 * want to integrate the console into an existing event-driven sketch.
 *
 * Type a command (e.g. "heap" or "help") in the Serial Monitor and press
 * Enter. No background task is created.
 */

```

(continues on next page)

(continued from previous page)

```

* Created by lucasssvaz
*/

#include <Arduino.h>
#include <Console.h>

// -----
// Command: heap
// -----
static int cmd_heap(int argc, char **argv) {
    printf("Free heap : %lu bytes\n", (unsigned long)ESP.getFreeHeap());
    return 0;
}

// -----
// Command: uptime
// -----
static int cmd_uptime(int argc, char **argv) {
    unsigned long ms = millis();
    unsigned long s = ms / 1000;
    unsigned long days = s / 86400;
    unsigned long hours = (s % 86400) / 3600;
    unsigned long mins = (s % 3600) / 60;
    unsigned long secs = s % 60;
    printf("up %lu day(s), %02lu:%02lu:%02lu  (%.3f s)\n", days, hours, mins, secs,
↪ms / 1000.0f);
    return 0;
}

// -----
// setup / loop
// -----

static String inputBuffer;

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Configure Console prompt
    Console.setPrompt(""); // No prompt — we print it manually below

    // Initialize Console
    if (!Console.begin()) {
        Serial.println("Console init failed");
        return;
    }

    // Add commands
    Console.addCmd("heap", "Show free heap", cmd_heap);
    Console.addCmd("uptime", "Show uptime", cmd_uptime);

    // Add built-in help command
    Console.addHelpCmd();

    Serial.println("Console ready. Type a command and press Enter.");
    Serial.print("> ");
}

```

(continues on next page)

(continued from previous page)

```
// In this example we manually implement the input loop instead of using the REPL
↳task.
// This is useful when you need full control over the input loop or want to
↳integrate the console into an existing event-driven sketch.
void loop() {
    while (Serial.available()) {
        char c = (char)Serial.read();
        if (c == '\r') {
            continue;
        }
        if (c == '\n') {
            inputBuffer.trim();
            if (inputBuffer.length() > 0) {
                Serial.println();
                Console.run(inputBuffer);
                inputBuffer = "";
            }
            Serial.print("> ");
        } else {
            Serial.print(c); // local echo
            inputBuffer += c;
        }
    }
}
```

ConsoleSysInfo System information commands: uptime, heap, tasks, version, restart.

```
/*
 * ConsoleSysInfo — System information CLI commands.
 *
 * Implements a small set of diagnostic commands inspired by common Unix tools:
 *
 *   uptime — elapsed time since boot
 *   heap   — free and minimum heap memory
 *   tasks  — list FreeRTOS tasks with state and stack watermark
 *   restart — restart the device
 *   version — firmware and IDF version strings
 *
 * Created by lucasssvaz
 */

#include <Arduino.h>
#include <Console.h>

// -----
// uptime
// -----

static int cmd_uptime(int argc, char **argv) {
    unsigned long ms = millis();
    unsigned long s = ms / 1000;
    unsigned long days = s / 86400;
    unsigned long hours = (s % 86400) / 3600;
    unsigned long mins = (s % 3600) / 60;
    unsigned long secs = s % 60;
    printf("up %lu day(s), %02lu:%02lu:%02lu  (%.3f s)\n", days, hours, mins, secs,
↳ms / 1000.0f);
    return 0;
}

// -----
```

(continues on next page)

(continued from previous page)

```

// heap
// -----
static int cmd_heap(int argc, char **argv) {
    printf("Free heap      : %7lu bytes\n", (unsigned long)ESP.getFreeHeap());
    printf("Min free heap: %7lu bytes\n", (unsigned long)ESP.getMinFreeHeap());
    printf("Max alloc      : %7lu bytes\n", (unsigned long)ESP.getMaxAllocHeap());
    return 0;
}

// -----
// tasks
// -----
static int cmd_tasks(int argc, char **argv) {
    uint32_t nTasks = uxTaskGetNumberOfTasks();
    TaskStatus_t *pxTaskStatusArray = (TaskStatus_t *)malloc(nTasks *
↪ sizeof(TaskStatus_t));
    if (!pxTaskStatusArray) {
        printf("Out of memory\n");
        return 1;
    }

    uint32_t ulTotalRunTime = 0;
    nTasks = uxTaskGetSystemState(pxTaskStatusArray, nTasks, &ulTotalRunTime);

    printf("%-20s %8s %5s %5s %4s\n", "Name", "State", "Prio", "Stack", "Core");
    printf("%-20s %8s %5s %5s %4s\n", "-----", "-----", "-----", "-----", "-----");
↪ printf("%-20s %8s %5s %5s %4s\n", "-----", "-----", "-----", "-----", "-----");

    static const char *stateNames[] = {"Running", "Ready", "Blocked", "Suspend",
↪ "Deleted", "Invalid"};

    for (uint32_t i = 0; i < nTasks; i++) {
        TaskStatus_t *t = &pxTaskStatusArray[i];
        int state = (int)t->eCurrentState;
        if (state > 5) {
            state = 5;
        }
        int core = (int)t->xCoreID;
        char coreStr[4];
        if (core == tskNO_AFFINITY) {
            snprintf(coreStr, sizeof(coreStr), "any");
        } else {
            snprintf(coreStr, sizeof(coreStr), "%" PRIu8, (uint8_t)core);
        }
        printf("%-20s %8s %5lu %5lu %4s\n", t->pcTaskName, stateNames[state],
↪ (unsigned long)t->uxCurrentPriority, (unsigned long)t->usStackHighWaterMark,
↪ coreStr);
    }

    free(pxTaskStatusArray);
    return 0;
}

// -----
// version
// -----
static int cmd_version(int argc, char **argv) {
    printf("Arduino-ESP32 : %s\n", ESP_ARDUINO_VERSION_STR);
    printf("ESP-IDF       : %s\n", esp_get_idf_version());
    printf("Chip          : %s rev %d\n", ESP.getChipModel(), ESP.getChipRevision());
    printf("CPU freq      : %lu MHz\n", (unsigned long)ESP.getCpuFreqMHz());
}

```

(continues on next page)

(continued from previous page)

```

printf("Flash size      : %lu bytes\n", (unsigned long)ESP.getFlashChipSize());
return 0;
}

// -----
// restart
// -----
static int cmd_restart(int argc, char **argv) {
    printf("Restarting in 1 second...\n");
    fflush(stdout);
    delay(1000);
    ESP.restart();
    return 0;
}

// -----
// setup / loop
// -----

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Configure Console prompt
    Console.setPrompt("sysinfo> ");

    // Initialize Console
    if (!Console.begin()) {
        Serial.println("Console init failed");
        return;
    }

    // Add commands
    Console.addCmd("uptime", "Show time since boot", cmd_uptime);
    Console.addCmd("heap", "Show heap memory statistics", cmd_heap);
    Console.addCmd("tasks", "List FreeRTOS tasks", cmd_tasks);
    Console.addCmd("version", "Print firmware/chip version", cmd_version);
    Console.addCmd("restart", "Restart the device", cmd_restart);

    // Add built-in help command
    Console.addHelpCmd();

    // Begin Read-Eval-Print Loop
    Console.attachToSerial(true);
}

void loop() {
    // Loop task is not used in this example, so we delete it to free up resources
    vTaskDelete(NULL);
}

```

ConsoleFS File system CLI over LittleFS: ls, cat, rm, mkdir, write, df.

```

/*
 * ConsoleFS — File system CLI commands over LittleFS.
 *
 * Implements a minimal shell for browsing and managing the LittleFS partition:
 *

```

(continues on next page)

(continued from previous page)

```

*   ls   [path]           — list directory contents
*   cat  <file>           — print file contents
*   rm   <file>           — delete a file
*   mkdir <dir>           — create a directory
*   write <file> <text>   — write (overwrite) text to a file
*   df                               — show filesystem usage
*
* Requires LittleFS to be mounted (LittleFS.begin() called in setup).
*
* Created by lucasssvaz
*/

#include <Arduino.h>
#include <Console.h>
#include <LittleFS.h>

// Ensure path starts with '/' so LittleFS.open() accepts it.
// Returns a pointer to either the original string or a static buffer.
static const char *normalizePath(const char *path) {
    if (path[0] == '/') {
        return path;
    }
    static char buf[256];
    snprintf(buf, sizeof(buf), "%s", path);
    return buf;
}

// -----
// ls [path]
// -----
static int cmd_ls(int argc, char **argv) {
    const char *path = (argc > 1) ? normalizePath(argv[1]) : "/";

    File dir = LittleFS.open(path);
    if (!dir || !dir.isDirectory()) {
        printf("ls: %s: not a directory\n", path);
        return 1;
    }

    File entry = dir.openNextFile();
    while (entry) {
        if (entry.isDirectory()) {
            printf("d  %s/\n", entry.name());
        } else {
            printf("f  %-40s  %zu bytes\n", entry.name(), entry.size());
        }
        entry = dir.openNextFile();
    }
    return 0;
}

// -----
// cat <file>
// -----
static int cmd_cat(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: cat <file>\n");
        return 1;
    }

    const char *path = normalizePath(argv[1]);

```

(continues on next page)

(continued from previous page)

```

File f = LittleFS.open(path, "r");
if (!f) {
    printf("cat: %s: no such file\n", path);
    return 1;
}

while (f.available()) {
    int c = f.read();
    putchar(c);
}
f.close();
printf("\n");
return 0;
}

// -----
// rm <file>
// -----
static int cmd_rm(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: rm <file>\n");
        return 1;
    }

    const char *path = normalizePath(argv[1]);
    if (!LittleFS.exists(path)) {
        printf("rm: %s: no such file\n", path);
        return 1;
    }

    if (!LittleFS.remove(path)) {
        printf("rm: %s: failed\n", path);
        return 1;
    }

    printf("removed '%s'\n", path);
    return 0;
}

// -----
// mkdir <dir>
// -----
static int cmd_mkdir(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: mkdir <dir>\n");
        return 1;
    }

    const char *path = normalizePath(argv[1]);
    if (!LittleFS.mkdir(path)) {
        printf("mkdir: %s: failed\n", path);
        return 1;
    }

    printf("created '%s'\n", path);
    return 0;
}

// -----
// write <file> <text...>
// -----

```

(continues on next page)

(continued from previous page)

```

static int cmd_write(int argc, char **argv) {
    if (argc < 3) {
        printf("Usage: write <file> <text...\n");
        return 1;
    }

    const char *path = normalizePath(argv[1]);
    File f = LittleFS.open(path, "w");
    if (!f) {
        printf("write: %s: cannot open\n", path);
        return 1;
    }

    for (int i = 2; i < argc; i++) {
        f.print(argv[i]);
        if (i < argc - 1) {
            f.print(' ');
        }
    }
    f.println();
    f.close();
    printf("written to '%s'\n", path);
    return 0;
}

// -----
// df
// -----
static int cmd_df(int argc, char **argv) {
    size_t total = LittleFS.totalBytes();
    size_t used = LittleFS.usedBytes();
    size_t free_ = total - used;
    printf("Filesystem Size      Used      Free      Use%%\n");
    printf("LittleFS      %-8zu %-8zu %-8zu %3zu%%\n", total, used, free_, total ?
↪ (used * 100 / total) : 0);
    return 0;
}

// -----
// setup / loop
// -----

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Mount LittleFS (format on first use)
    if (!LittleFS.begin(true)) {
        Serial.println("LittleFS mount failed");
        return;
    }
    Serial.println("LittleFS mounted");

    // Configure Console prompt
    Console.setPrompt("fs> ");

    // Initialize Console
    if (!Console.begin()) {
        Serial.println("Console init failed");
    }
}

```

(continues on next page)

(continued from previous page)

```

    return;
}

// Add commands
Console.addCmd("ls", "List directory contents", "[path]", cmd_ls);
Console.addCmd("cat", "Print file contents", "<file>", cmd_cat);
Console.addCmd("rm", "Delete a file", "<file>", cmd_rm);
Console.addCmd("mkdir", "Create a directory", "<dir>", cmd_mkdir);
Console.addCmd("write", "Write text to a file", "<file> <text...>", cmd_write);
Console.addCmd("df", "Show filesystem usage", cmd_df);

// Add built-in help command
Console.addHelpCmd();

// Begin Read-Eval-Print Loop
Console.attachToSerial(true);
}

void loop() {
    // Loop task is not used in this example, so we delete it to free up resources
    vTaskDelete(NULL);
}

```

ConsoleGPIO GPIO control with argtable3 argument parsing: gpio read, gpio write, gpio mode. LED control is supported with the led command on boards with a built-in LED.

```

/*
 * ConsoleGPIO — GPIO control via CLI using argtable3 argument parsing.
 *
 * Implements three sub-commands accessed as a single "gpio" command:
 *
 *   gpio read <pin>           — read and print pin level
 *   gpio write <pin> <0/1>    — write HIGH or LOW to a pin
 *   gpio mode <pin> <mode>   — set pin mode (in/out/in_pu/in_pd)
 *   led <on/off> [color]     — toggle LED (with RGB color on supported_
→boards)
 *
 * Demonstrates argtable3 usage: typed arguments with automatic hint
 * generation and built-in error reporting. The "led" command shows how to
 * reuse existing command handlers from a higher-level convenience command.
 *
 * Created by lucasssvaz
 */

#include <Arduino.h>
#include <Console.h>
#include "argtable3/argtable3.h"

#define RGB_BRIGHTNESS 64 // Change brightness (max 255)

// -----
// gpio read <pin>
// -----

static struct {
    struct arg_int *pin; // required int: GPIO pin number
    struct arg_end *end; // sentinel: tracks parse errors
} gpio_read_args;

static int cmd_gpio_read(int argc, char **argv) {

```

(continues on next page)

(continued from previous page)

```

// arg_parse() validates argv against the argument table, returns error count
int nerrors = arg_parse(argc, argv, (void **)&gpio_read_args);
if (nerrors != 0) {
    // Print human-readable error messages (e.g. "missing <pin> argument")
    arg_print_errors(stderr, gpio_read_args.end, argv[0]);
    return 1;
}

// Access the parsed integer value via ->ival[0]
int pin = gpio_read_args.pin->ival[0];
int val = digitalRead(pin);
Serial.printf("GPIO %d = %d (%s)\n", pin, val, val ? "HIGH" : "LOW");
return 0;
}

// -----
// gpio write <pin> <value>
// -----

static struct {
    struct arg_int *pin;    // required int: GPIO pin number
    struct arg_int *value;  // required int: 0 or 1
    struct arg_end *end;    // sentinel: tracks parse errors
} gpio_write_args;

static int cmd_gpio_write(int argc, char **argv) {
    int nerrors = arg_parse(argc, argv, (void **)&gpio_write_args);
    if (nerrors != 0) {
        arg_print_errors(stderr, gpio_write_args.end, argv[0]);
        return 1;
    }

    // Access the parsed integer values via ->ival[0]
    int pin = gpio_write_args.pin->ival[0];
    int val = gpio_write_args.value->ival[0];

    if (val != 0 && val != 1) {
        Serial.println("gpio write: value must be 0 or 1");
        return 1;
    }

    digitalWrite(pin, val);
    Serial.printf("GPIO %d set to %d (%s)\n", pin, val, val ? "HIGH" : "LOW");
    return 0;
}

// -----
// gpio mode <pin> <mode>
// -----

static struct {
    struct arg_int *pin;    // required int: GPIO pin number
    struct arg_str *mode;   // required string: mode name (in/out/in_pu/in_pd)
    struct arg_end *end;    // sentinel: tracks parse errors
} gpio_mode_args;

static int cmd_gpio_mode(int argc, char **argv) {
    int nerrors = arg_parse(argc, argv, (void **)&gpio_mode_args);
    if (nerrors != 0) {
        arg_print_errors(stderr, gpio_mode_args.end, argv[0]);
        return 1;
    }

```

(continues on next page)

(continued from previous page)

```

}

// Access the parsed integer value via ->ival[0]
int pin = gpio_mode_args.pin->ival[0];

// Access the parsed string value via ->sval[0]
const char *modeStr = gpio_mode_args.mode->sval[0];

uint8_t mode;
if (strcmp(modeStr, "in") == 0) {
    mode = INPUT;
} else if (strcmp(modeStr, "out") == 0) {
    mode = OUTPUT;
} else if (strcmp(modeStr, "in_pu") == 0) {
    mode = INPUT_PULLUP;
} else if (strcmp(modeStr, "in_pd") == 0) {
    mode = INPUT_PULLDOWN;
} else {
    Serial.printf("gpio mode: unknown mode '%s'. Use: in, out, in_pu, in_pd\n", modeStr);
    return 1;
}

pinMode(pin, mode);
Serial.printf("GPIO %d mode set to %s\n", pin, modeStr);
return 0;
}

// -----
// Top-level dispatcher: gpio <subcmd> ...
// -----

static int cmd_gpio(int argc, char **argv) {
    if (argc < 2) {
        Serial.println("Usage: gpio <read|write|mode> ...");
        Serial.println("  gpio read  <pin>");
        Serial.println("  gpio write <pin> <0|1>");
        Serial.println("  gpio mode  <pin> <in|out|in_pu|in_pd>");
        return 1;
    }

    // Shift argv so the sub-command is argv[0]
    if (strcmp(argv[1], "read") == 0) {
        return cmd_gpio_read(argc - 1, argv + 1);
    } else if (strcmp(argv[1], "write") == 0) {
        return cmd_gpio_write(argc - 1, argv + 1);
    } else if (strcmp(argv[1], "mode") == 0) {
        return cmd_gpio_mode(argc - 1, argv + 1);
    } else {
        Serial.printf("gpio: unknown sub-command '%s'\n", argv[1]);
        return 1;
    }
}

// -----
// led <on/off> [color] — convenience command that reuses the gpio handlers
//
// When RGB_BUILTIN is defined (boards with an addressable RGB LED), an
// optional color name is accepted: red, green, blue, white, yellow.
//
// This command will reuse the gpio commands if the board does not have an RGB LED.

```

(continues on next page)

(continued from previous page)

```

// Otherwise, it will use the rgbLedWrite function to set the color of the LED.
// -----

#ifdef LED_BUILTIN

#ifdef RGB_BUILTIN
struct NamedColor {
    const char *name;
    uint8_t r, g, b;
};

static const NamedColor colors[] = {
    {"red", RGB_BRIGHTNESS, 0, 0},
    {"green", 0, RGB_BRIGHTNESS, 0},
    {"blue", 0, 0, RGB_BRIGHTNESS},
    {"white", RGB_BRIGHTNESS, RGB_BRIGHTNESS, RGB_BRIGHTNESS},
    {"yellow", RGB_BRIGHTNESS, RGB_BRIGHTNESS, 0},
};

static const size_t numColors = sizeof(colors) / sizeof(colors[0]);
#else
static bool ledInitialized = false;
#endif

static void printLedUsage() {
#ifdef RGB_BUILTIN
    Serial.println("Usage: led <on [color]|off>");
    Serial.println("  Colors: red, green, blue, white, yellow");
#else
    Serial.println("Usage: led <on|off>");
#endif
}

static int cmd_led(int argc, char **argv) {
    if (argc < 2) {
        printLedUsage();
        return 1;
    }

#ifdef RGB_BUILTIN
    // Read argv values BEFORE calling Console.run() — nested run() calls
    // invalidate the caller's argv pointers (shared IDF parse buffer).
    if (strcmp(argv[1], "on") == 0) {
        // If the board has an RGB LED, and a color is specified, set the LED to the
        ↪ specified color.
        // If no color is specified, set the LED to white.
        // This will use the rgbLedWrite function to set the color of the LED.
        const char *colorName = (argc > 2) ? argv[2] : "white";
        for (size_t i = 0; i < numColors; i++) {
            if (strcasecmp(colorName, colors[i].name) == 0) {
                rgbLedWrite(LED_BUILTIN, colors[i].r, colors[i].g, colors[i].b);
                Serial.printf("RGB LED on: %s (%d, %d, %d)\n", colors[i].name, colors[i].r,
                ↪ colors[i].g, colors[i].b);
                return 0;
            }
        }
        Serial.printf("Unknown color '%s'. Available: ", colorName);
        for (size_t i = 0; i < numColors; i++) {
            Serial.printf("%s%s", colors[i].name, (i < numColors - 1) ? ", " : "\n");
        }
        return 1;
    }

```

(continues on next page)

(continued from previous page)

```

    } else if (strcmp(argv[1], "off") == 0) {
        rgbLedWrite(RGB_BUILTIN, 0, 0, 0);
        Serial.println("RGB LED off");
        return 0;
    }
#else
    // Read argv values BEFORE calling Console.run() — nested run() calls
    // invalidate the caller's argv pointers (shared IDF parse buffer).
    if (strcmp(argv[1], "on") == 0) {
        // If the board does not have an RGB LED, use the gpio commands to toggle the
        ↪LED.
        // This will call the gpio commands to configure the pin as an output and set
        ↪the LED to HIGH.
        if (!ledInitialized) {
            Console.run(String("gpio mode ") + LED_BUILTIN + " out");
            ledInitialized = true;
        }
        return Console.run(String("gpio write ") + LED_BUILTIN + " 1");
    } else if (strcmp(argv[1], "off") == 0) {
        if (!ledInitialized) {
            Console.run(String("gpio mode ") + LED_BUILTIN + " out");
            ledInitialized = true;
        }
        return Console.run(String("gpio write ") + LED_BUILTIN + " 0");
    }
#endif

    printLedUsage();
    return 1;
}

#endif

// -----
// setup / loop
// -----

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Initialize argtable argument structures for each GPIO sub-command.
    // These structures help parse and validate command line arguments for the
    ↪Console commands.
    //
    // arg_int1(shortopts, longopts, datatype, glossary)
    //   - Adds a required integer argument.
    // arg_str1(...)
    //   - Adds a required string argument.
    // arg_end(N)
    //   - Terminates the argument table and provides space for up to N error
    ↪messages.
    //
    // Passing NULL for both shortopts and longopts makes arguments positional (not
    ↪using flags like -x or --foo).
    // The glossary string is a description that appears in the help message.

    // Arguments for: gpio read <pin>
    gpio_read_args.pin = arg_int1(NULL, NULL, "<pin>", "GPIO pin number to read

```

(continues on next page)

(continued from previous page)

```

↪(required)");
    gpio_read_args.end = arg_end(1); // Up to 1 error message for reading.

    // Arguments for: gpio write <pin> <0|1>
    gpio_write_args.pin = arg_int1(NULL, NULL, "<pin>", "GPIO pin number to write.");
    ↪(required)");
    gpio_write_args.value = arg_int1(NULL, NULL, "<0|1>", "Value to write: 0 = LOW, ↪
    ↪1 = HIGH (required)");
    gpio_write_args.end = arg_end(2); // Up to 2 error messages for writing.

    // Arguments for: gpio mode <pin> <in/out/in_pu/in_pd>
    gpio_mode_args.pin = arg_int1(NULL, NULL, "<pin>", "GPIO pin number to configure.");
    ↪(required)");
    gpio_mode_args.mode = arg_str1(NULL, NULL, "<in|out|in_pu|in_pd>", "Pin mode: in,
    ↪out, in_pu, or in_pd (required)");
    gpio_mode_args.end = arg_end(2); // Up to 2 error messages for mode.

    // Configure Console prompt
    Console.setPrompt("gpio> ");

    // Initialize Console
    if (!Console.begin()) {
        Serial.println("Console init failed");
        return;
    }

    // Add commands
    Console.addCmd("gpio", "Control GPIO pins", "<read|write|mode> ...", cmd_gpio);

#ifdef LED_BUILTIN
#ifdef RGB_BUILTIN
    Console.addCmd("led", "Control RGB LED", "<on [color]|off>", cmd_led);
#else
    Console.addCmd("led", "Toggle LED_BUILTIN", "<on|off>", cmd_led);
#endif
#endif

    // Add built-in help command
    Console.addHelpCmd();

    // Begin Read-Eval-Print Loop
    Console.attachToSerial(true);
}

void loop() {
    // Loop task is not used in this example, so we delete it to free up resources
    vTaskDelete(NULL);
}

```

ConsoleAdvanced Demonstrates advanced APIs: context-aware commands (`addCmdWithContext`), dynamic command registration and removal (`removeCmd`), REPL pause/resume (`attachToSerial(false/true)`), help verbosity (`setHelpVerboseLevel`), clear screen, set plain mode, set multi line, and task tuning (`setTaskStackSize`, `setTaskPriority`, `setTaskCore`).

```

/*
 * ConsoleAdvanced — Demonstrates advanced Console API features.
 *
 * Covers APIs not shown in the basic examples:
 *
 * - addCmdWithContext() — pass state to a command via a context pointer

```

(continues on next page)

(continued from previous page)

```

*   - removeCmd()           — unregister a command at runtime
*   - attachToSerial(false/true) — pause and resume the REPL
*   - setHelpVerboseLevel()   — toggle brief / verbose help output
*   - clearScreen()          — clear the terminal
*   - setPlainMode()         — force plain (non-VT100) input
*   - setMultiLine()         — enable multi-line editing
*   - Task tuning: setTaskStackSize(), setTaskPriority(), setTaskCore()
*
* Created by lucasssvaz
*/

#include <Arduino.h>
#include <Console.h>

// -----
// Context-aware command: addCmdWithContext() demo
//
// A context pointer lets multiple commands share or isolate state without
// global variables. The pointer is passed as the first argument to the
// callback every time the command is invoked.
// -----

struct LedCtx {
    int pin;
    bool state;
    const char *name;
};

static LedCtx led1 = {2, false, "LED1"};
static LedCtx led2 = {15, false, "LED2"};

static int cmd_toggle(void *context, int argc, char **argv) {
    LedCtx *led = (LedCtx *)context;
    led->state = !led->state;
    pinMode(led->pin, OUTPUT);
    digitalWrite(led->pin, led->state ? HIGH : LOW);
    printf("%s (GPIO %d) = %s\n", led->name, led->pin, led->state ? "ON" : "OFF");
    return 0;
}

// -----
// removeCmd demo: register/unregister a "secret" command at runtime
// -----

static int cmd_secret(int argc, char **argv) {
    printf("You found the secret command!\n");
    return 0;
}

static bool secretRegistered = false;

static int cmd_lock(int argc, char **argv) {
    if (secretRegistered) {
        Console.removeCmd("secret");
        secretRegistered = false;
        printf("'secret' command removed.\n");
    } else {
        printf("'secret' is not registered.\n");
    }
    return 0;
}

```

(continues on next page)

(continued from previous page)

```

static int cmd_unlock(int argc, char **argv) {
    if (!secretRegistered) {
        Console.addCmd("secret", "A hidden command", cmd_secret);
        secretRegistered = true;
        printf("'secret' command registered. Try it!\n");
    } else {
        printf("'secret' is already available.\n");
    }
    return 0;
}

// -----
// attachToSerial(false/true) demo: pause the Read-Eval-Print Loop for 5 seconds
// -----

static int cmd_pause(int argc, char **argv) {
    printf("Stopping Read-Eval-Print Loop for 5 seconds...\n");
    fflush(stdout);
    Console.attachToSerial(false);
    delay(5000);
    Console.attachToSerial(true);
    return 0;
}

// -----
// setHelpVerboseLevel demo
// -----

static int cmd_verbose_help(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: verbose_help <0|1>\n");
        printf("  0 = brief (names and hints only)\n");
        printf("  1 = verbose (includes help text)\n");
        return 1;
    }
    int level = atoi(argv[1]);
    if (Console.setHelpVerboseLevel(level)) {
        printf("Help verbose level set to %d.\n", level);
    } else {
        printf("Invalid level. Use 0 or 1.\n");
    }
    return 0;
}

// -----
// clearScreen demo
// -----

static int cmd_clear(int argc, char **argv) {
    Console.clearScreen();
    return 0;
}

// -----
// setPlainMode / setMultiLine toggles
// -----

static int cmd_plain(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: plain <on|off>\n");
    }
}

```

(continues on next page)

(continued from previous page)

```

    return 1;
}
bool enable = (strcmp(argv[1], "on") == 0);
Console.setPlainMode(enable);
printf("Plain mode %s.\n", enable ? "enabled" : "disabled");
return 0;
}

static int cmd_multiline(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: multiline <on|off>\n");
        return 1;
    }
    bool enable = (strcmp(argv[1], "on") == 0);
    Console.setMultiLine(enable);
    printf("Multi-line mode %s.\n", enable ? "enabled" : "disabled");
    return 0;
}

// -----
// setup / loop
// -----

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Console task configuration — must be called before attachToSerial(true)

    // Set the task stack size to 4096 bytes
    Console.setTaskStackSize(4096);
    // Set the task priority to 2
    Console.setTaskPriority(2);
    // Set the task core to 1 (core 1)
    Console.setTaskCore(1); // pin REPL to core 1

    // Configure Console prompt
    Console.setPrompt("adv> ");

    // Initialize Console
    if (!Console.begin()) {
        Serial.println("Console init failed");
        return;
    }

    // Context-aware commands: same callback, different context pointer.
    // Each LED struct carries its own pin number, state, and label.
    Console.addCmdWithContext("led1", "Toggle LED1", cmd_toggle, &led1);
    Console.addCmdWithContext("led2", "Toggle LED2", cmd_toggle, &led2);

    // Dynamic command management
    Console.addCmd("unlock", "Register the 'secret' command", cmd_unlock);
    Console.addCmd("lock", "Unregister the 'secret' command", cmd_lock);

    // REPL control
    Console.addCmd("pause", "Stop the Read-Eval-Print Loop for 5 seconds", cmd_
↵ pause);

    // Help verbosity

```

(continues on next page)

(continued from previous page)

```

Console.addCmd("verbose_help", "Set help verbosity", "<0|1>", cmd_verbose_help);

// Terminal utilities
Console.addCmd("clear", "Clear the terminal screen", cmd_clear);
Console.addCmd("plain", "Toggle plain (non-VT100) mode", "<on|off>", cmd_plain);
Console.addCmd("multiline", "Toggle multi-line editing", "<on|off>", cmd_
↪multiline);

// Add built-in help command
Console.addHelpCmd();

// Begin Read-Eval-Print Loop
if (!Console.attachToSerial(true)) {
    Serial.println("Failed to attach to serial");
}
}

void loop() {
    // Loop task is not used in this example, so we delete it to free up resources
    vTaskDelete(NULL);
}

```

ConsoleHistory Persistent command history across reboots using LittleFS (setHistoryFile), PSRAM allocation (usePsrAm), and the splitArgv tokenising utility.

```

/*
 * ConsoleHistory — Persistent history, PSRAM allocation, and splitArgv demo.
 *
 * Demonstrates APIs not covered by the basic examples:
 *
 * - setHistoryFile() — save/load history to a filesystem across reboots
 * - setMaxHistory() — limit number of stored history lines
 * - usePsrAm() — allocate Console heap and REPL task stack in PSRAM
 * - splitArgv() — split a raw string into argv-style tokens
 *
 * On boot the REPL loads the previous session's history. Use the up/down
 * arrow keys (VT100 terminals) or the "history" command to inspect entries.
 * Power-cycle the board to verify history persists.
 *
 * Created by lucasssvaz
 */

#include <Arduino.h>
#include <Console.h>
#include <LittleFS.h>

static const char *HISTORY_PATH = "/console_history.txt";

// -----
// Command: history — print the current history buffer
// -----

static int cmd_history(int argc, char **argv) {
    File f = LittleFS.open(HISTORY_PATH, "r");
    if (!f) {
        printf("No history saved yet.\n");
        return 0;
    }

    int index = 1;
    while (f.available()) {

```

(continues on next page)

(continued from previous page)

```

    String line = f.readStringUntil('\n');
    line.trim();
    if (line.length() > 0) {
        printf("  %3d  %s\n", index++, line.c_str());
    }
}
f.close();

if (index == 1) {
    printf("History is empty.\n");
}
return 0;
}

// -----
// Command: tokenize — demonstrate Console.splitArgv()
//
// splitArgv() splits a raw string into argc/argv tokens, handling quotes
// and backslash escapes. Useful when you receive a full command line from
// a non-REPL source (MQTT, BLE, HTTP, etc.) and need to parse it.
// -----
static int cmd_tokenize(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: tokenize <string>\n");
        printf("  Splits the string into tokens. Use quotes for multi-word tokens.\n
↪");
        printf("  Example: tokenize hello \"big world\" foo\n");
        return 1;
    }

    // Reconstruct the argument string from argv[1..] so the user can type
    // something like: tokenize hello "big world" foo
    String raw;
    for (int i = 1; i < argc; i++) {
        if (i > 1) {
            raw += ' ';
        }
        raw += argv[i];
    }

    // splitArgv() works in-place — it modifies the buffer and sets pointers
    // into it. We need a mutable copy.
    char *buf = strdup(raw.c_str());
    if (!buf) {
        printf("Out of memory\n");
        return 1;
    }

    char *tokens[16];
    size_t count = Console.splitArgv(buf, tokens, 16);

    printf("Input : \"%s\"\n", raw.c_str());
    printf("Tokens: %zu\n", count);
    for (size_t i = 0; i < count; i++) {
        printf(" [%zu] \"%s\"\n", i, tokens[i]);
    }

    free(buf);
    return 0;
}

```

(continues on next page)

(continued from previous page)

```

// -----
// Command: heap — show free memory
// -----
static int cmd_heap(int argc, char **argv) {
    printf("Free heap : %lu bytes\n", (unsigned long)ESP.getFreeHeap());
    if (psramFound()) {
        printf("Free PSRAM: %lu bytes\n", (unsigned long)ESP.getFreePsram());
    }
    return 0;
}

// -----
// setup / loop
// -----

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Mount LittleFS (format on first use)
    if (!LittleFS.begin(true)) {
        Serial.println("LittleFS mount failed — history will not persist.");
    } else {
        Serial.println("LittleFS mounted");
        // Persistent history: lines are stored in a plain text file on LittleFS.
        // On begin(), existing history is loaded; after every command, the file
        // is updated automatically.
        Console.setHistoryFile(LittleFS, HISTORY_PATH);
    }

    Console.setMaxHistory(50);

    // If PSRAM is available, move Console allocations there to save internal
    // SRAM. Falls back to internal RAM if PSRAM is not available.
    Console.usePsram(true);

    // Configure Console prompt
    Console.setPrompt("hist> ");

    // Initialize Console
    if (!Console.begin()) {
        Serial.println("Console init failed");
        return;
    }

    // Add commands
    Console.addCmd("history", "Show history info", cmd_history);
    Console.addCmd("tokenize", "Split a string into tokens", "<string>", cmd_
    →tokenize);
    Console.addCmd("heap", "Show free memory", cmd_heap);

    // Add built-in help command
    Console.addHelpCmd();

    if (!Console.attachToSerial(true)) {
        Serial.println("REPL start failed");
    }
}

```

(continues on next page)

(continued from previous page)

```

void loop() {
    // Loop task is not used in this example, so we delete it to free up resources
    vTaskDelete(NULL);
}

```

ConsoleWiFi Wi-Fi management: wifi scan, wifi connect, wifi disconnect, wifi status, wifi ping.

```

/*
 * ConsoleWiFi — Wi-Fi management CLI commands.
 *
 * Implements a set of commands for scanning, connecting, and monitoring Wi-Fi:
 *
 *   wifi scan                — scan and list nearby access points
 *   wifi connect <ssid> [<pass>] — connect to a network
 *   wifi disconnect          — disconnect from the current network
 *   wifi status              — show connection status and IP info
 *   ping <host> [-c <count>] — ICMP ping a host by name or IP
 *
 * Demonstrates argtable3 with optional arguments (password is optional)
 * and optional flags (-c for ping count).
 *
 * Created by lucasssvaz
 */

#include <Arduino.h>
#include <Console.h>
#include <WiFi.h>
#include "ping/ping_sock.h"
#include "argtable3/argtable3.h"

// -----
// wifi scan
// -----

static int cmd_wifi_scan(int argc, char **argv) {
    printf("Scanning...\n");
    int n = WiFi.scanNetworks();
    if (n == 0) {
        printf("No networks found.\n");
        return 0;
    }

    printf("%-4s  %-32s  %5s  %4s  %s\n", "#", "SSID", "RSSI", "Ch", "Encryption");
    printf("%-4s  %-32s  %5s  %4s  %s\n", "---", "-----", "-----", "-----", "-----");
    ↪ "-----", "-----", "-----");

    for (int i = 0; i < n; i++) {
        const char *enc;
        switch (WiFi.encryptionType(i)) {
            case WIFI_AUTH_OPEN:      enc = "Open"; break;
            case WIFI_AUTH_WEP:       enc = "WEP"; break;
            case WIFI_AUTH_WPA_PSK:   enc = "WPA"; break;
            case WIFI_AUTH_WPA2_PSK:  enc = "WPA2"; break;
            case WIFI_AUTH_WPA_WPA2_PSK: enc = "WPA/WPA2"; break;
            case WIFI_AUTH_WPA3_PSK:  enc = "WPA3"; break;
            default:                  enc = "Unknown"; break;
        }
        printf("%-4d  %-32s  %5d  %4d  %s\n", i + 1, WiFi.SSID(i).c_str(), (int)WiFi.
        ↪ RSSI(i), (int)WiFi.channel(i), enc);
    }
}

```

(continues on next page)

(continued from previous page)

```

    }

    WiFi.scanDelete();
    return 0;
}

// -----
// wifi connect <ssid> [<password>]
// -----

static struct {
    struct arg_str *ssid;      // required positional string: network name
    struct arg_str *password;  // optional positional string: network password
    struct arg_end *end;       // sentinel: tracks parse errors
} wifi_connect_args;

static int cmd_wifi_connect(int argc, char **argv) {
    int nerrors = arg_parse(argc, argv, (void **)&wifi_connect_args);
    if (nerrors != 0) {
        arg_print_errors(stdout, wifi_connect_args.end, argv[0]);
        return 1;
    }

    const char *ssid = wifi_connect_args.ssid->sval[0]; // access parsed string via
    ↪->sval[0]
    // For optional args, check ->count to see if it was provided
    const char *pass = (wifi_connect_args.password->count > 0) ? wifi_connect_args.
    ↪password->sval[0] : NULL;

    printf("Connecting to '%s'...\n", ssid);
    WiFi.STA.connect(ssid, pass);

    uint32_t deadline = millis() + 15000;
    while (!WiFi.STA.connected() && millis() < deadline) {
        delay(500);
        printf(".");
        fflush(stdout);
    }
    printf("\n");

    if (WiFi.STA.connected()) {
        printf("Connected. IP: %s\n", WiFi.STA.localIP().toString().c_str());
        return 0;
    } else {
        printf("Connection failed (status %d)\n", (int)WiFi.STA.status());
        return 1;
    }
}

// -----
// wifi disconnect
// -----

static int cmd_wifi_disconnect(int argc, char **argv) {
    WiFi.STA.disconnect();
    printf("Disconnected.\n");
    return 0;
}

// -----
// wifi status

```

(continues on next page)

(continued from previous page)

```
// -----

static int cmd_wifi_status(int argc, char **argv) {
    wl_status_t s = WiFi.STA.status();
    const char *statusStr;
    switch (s) {
        case WL_CONNECTED:      statusStr = "Connected"; break;
        case WL_NO_SSID_AVAIL:   statusStr = "SSID not found"; break;
        case WL_CONNECT_FAILED: statusStr = "Connect failed"; break;
        case WL_DISCONNECTED:    statusStr = "Disconnected"; break;
        case WL_IDLE_STATUS:     statusStr = "Idle"; break;
        default:                 statusStr = "Unknown"; break;
    }

    printf("Status      : %s\n", statusStr);

    if (s == WL_CONNECTED) {
        printf("SSID        : %s\n", WiFi.STA.SSID().c_str());
        printf("BSSID       : %s\n", WiFi.STA.BSSIDstr().c_str());
        printf("RSSI        : %d dBm\n", (int)WiFi.STA.RSSI());
        printf("IP          : %s\n", WiFi.STA.localIP().toString().c_str());
        printf("Gateway     : %s\n", WiFi.STA.gatewayIP().toString().c_str());
        printf("DNS         : %s\n", WiFi.STA.dnsIP().toString().c_str());
        printf("Subnet      : %s\n", WiFi.STA.subnetMask().toString().c_str());
    }

    printf("MAC         : %s\n", WiFi.STA.macAddress().c_str());
    return 0;
}

// -----
// Top-level dispatcher: wifi <subcmd> ...
// -----

static int cmd_wifi(int argc, char **argv) {
    if (argc < 2) {
        printf("Usage: wifi <scan|connect|disconnect|status>\n");
        return 1;
    }

    if (strcmp(argv[1], "scan") == 0) {
        return cmd_wifi_scan(argc - 1, argv + 1);
    } else if (strcmp(argv[1], "connect") == 0) {
        return cmd_wifi_connect(argc - 1, argv + 1);
    } else if (strcmp(argv[1], "disconnect") == 0) {
        return cmd_wifi_disconnect(argc - 1, argv + 1);
    } else if (strcmp(argv[1], "status") == 0) {
        return cmd_wifi_status(argc - 1, argv + 1);
    } else {
        printf("wifi: unknown sub-command '%s'\n", argv[1]);
        return 1;
    }
}

// -----
// ping <host> [-c <count>]
// -----

static struct {
    struct arg_str *host;    // required positional: hostname or IP address
    struct arg_int *count;   // optional: -c <count> (default 5)
}
```

(continues on next page)

(continued from previous page)

```

    struct arg_end *end;
} ping_args;

static SemaphoreHandle_t ping_done_sem = NULL;

static void ping_on_success(esp_ping_handle_t hdl, void *args) {
    uint32_t elapsed;
    uint32_t size;
    uint32_t seqno;
    uint8_t ttl;
    ip_addr_t addr;
    esp_ping_get_profile(hdl, ESP_PING_PROF_SEQNO, &seqno, sizeof(seqno));
    esp_ping_get_profile(hdl, ESP_PING_PROF_TTL, &ttl, sizeof(ttl));
    esp_ping_get_profile(hdl, ESP_PING_PROF_SIZE, &size, sizeof(size));
    esp_ping_get_profile(hdl, ESP_PING_PROF_TIMEGAP, &elapsed, sizeof(elapsed));
    esp_ping_get_profile(hdl, ESP_PING_PROF_IPADDR, &addr, sizeof(addr));
    printf(
        "%lu bytes from %s: icmp_seq=%lu ttl=%d time=%lu ms\n", (unsigned long)size,
        ipaddr_ntoa(&addr), (unsigned long)seqno, (int)ttl, (unsigned long)elapsed
    );
}

static void ping_on_timeout(esp_ping_handle_t hdl, void *args) {
    uint32_t seqno;
    ip_addr_t addr;
    esp_ping_get_profile(hdl, ESP_PING_PROF_SEQNO, &seqno, sizeof(seqno));
    esp_ping_get_profile(hdl, ESP_PING_PROF_IPADDR, &addr, sizeof(addr));
    printf("From %s icmp_seq=%lu timeout\n", ipaddr_ntoa(&addr), (unsigned
    long)seqno);
}

static void ping_on_end(esp_ping_handle_t hdl, void *args) {
    uint32_t transmitted;
    uint32_t received;
    uint32_t total_ms;
    esp_ping_get_profile(hdl, ESP_PING_PROF_REQUEST, &transmitted,
    sizeof(transmitted));
    esp_ping_get_profile(hdl, ESP_PING_PROF_REPLY, &received, sizeof(received));
    esp_ping_get_profile(hdl, ESP_PING_PROF_DURATION, &total_ms, sizeof(total_ms));
    uint32_t loss = transmitted ? ((transmitted - received) * 100 / transmitted) : 0;
    printf("\n--- ping statistics ---\n");
    printf(
        "%lu packets transmitted, %lu received, %lu%% packet loss, time %lu ms\n",
        (unsigned long)transmitted, (unsigned long)received, (unsigned long)loss,
        (unsigned long)total_ms
    );
    xSemaphoreGive(ping_done_sem);
}

static int cmd_ping(int argc, char **argv) {
    int nerrors = arg_parse(argc, argv, (void **)&ping_args);
    if (nerrors != 0) {
        arg_print_errors(stdout, ping_args.end, argv[0]);
        return 1;
    }

    if (!WiFi.STA.connected()) {
        printf("Not connected to Wi-Fi.\n");
        return 1;
    }
}

```

(continues on next page)

(continued from previous page)

```

const char *host = ping_args.host->sval[0];
int count = (ping_args.count->count > 0) ? ping_args.count->ival[0] : 5;
if (count < 1) {
    count = 1;
}

IPAddress addr;
if (!Network.hostByName(host, addr)) {
    printf("ping: unknown host '%s'\n", host);
    return 1;
}

ip_addr_t target_addr;
target_addr.type = IPADDR_TYPE_V4;
target_addr.u_addr.ip4.addr = (uint32_t)addr;

printf("PING %s (%s): 64 bytes of data, count=%d\n", host, addr.toString().c_
↪str(), count);
fflush(stdout);

esp_ping_config_t ping_config = ESP_PING_DEFAULT_CONFIG();
ping_config.target_addr = target_addr;
ping_config.count = count;

esp_ping_callbacks_t cbs = {};
cbs.on_ping_success = ping_on_success;
cbs.on_ping_timeout = ping_on_timeout;
cbs.on_ping_end = ping_on_end;

esp_ping_handle_t ping_handle = NULL;
if (esp_ping_new_session(&ping_config, &cbs, &ping_handle) != ESP_OK) {
    printf("Failed to create ping session\n");
    return 1;
}

esp_ping_start(ping_handle);
xSemaphoreTake(ping_done_sem, portMAX_DELAY);
esp_ping_delete_session(ping_handle);

return 0;
}

// -----
// setup / loop
// -----

void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }

    // Initialize WiFi
    WiFi.STA.begin();

    // Create semaphore for ping done
    ping_done_sem = xSemaphoreCreateBinary();

    // Initialize argtable.
    // arg_str1() = required string. arg_str0() = optional string.
    // NULL, NULL for shortopts/longopts makes the argument positional.

```

(continues on next page)

(continued from previous page)

```

wifi_connect_args.ssid = arg_str1(NULL, NULL, "<ssid>", "Network SSID");
wifi_connect_args.password = arg_str0(NULL, NULL, "[<password>]", "Network_
↪password (optional for open networks)");
wifi_connect_args.end = arg_end(2); // sentinel — stores up to 2 parse errors

// arg_int0() = optional integer. "c" is the short option (-c), "count"
// is the long option (--count). Default ping count is 5 if not given.
ping_args.host = arg_str1(NULL, NULL, "<host>", "Hostname or IP address to ping
↪");
ping_args.count = arg_int0("c", "count", "<n>", "Number of pings (default 5)");
ping_args.end = arg_end(2);

// Configure Console prompt
Console.setPrompt("wifi> ");

// Initialize Console
if (!Console.begin()) {
    Serial.println("Console init failed");
    return;
}

// Add commands
Console.addCmd("wifi", "Manage Wi-Fi connections", "
↪<scan|connect|disconnect|status>", cmd_wifi);
Console.addCmd("ping", "ICMP ping a host", (void *)&ping_args, cmd_ping);

// Add built-in help command
Console.addHelpCmd();

// Begin Read-Eval-Print Loop
Console.attachToSerial(true);
}

void loop() {
    // Loop task is not used in this example, so we delete it to free up resources
    vTaskDelete(NULL);
}

```

2.3.5 DAC

About

DAC (digital to analog converter) is a very common peripheral used to convert a digital signal to an analog form.

ESP32 and ESP32-S2 have two 8-bit DAC channels. The DAC driver allows these channels to be set to arbitrary voltages.

DACs can be used for generating a specific (and dynamic) reference voltage for external sensors, controlling transistors, etc.

ESP32 SoC	DAC_1 pin	DAC_2 pin
ESP32	GPIO 25	GPIO 26
ESP32-S2	GPIO 17	GPIO 18

Arduino-ESP32 DAC API

dacWrite This function is used to set the DAC value for a given pin/DAC channel.

```
void dacWrite(uint8_t pin, uint8_t value);
```

- pin GPIO pin.
- value to be set. Range is 0 - 255 (equals 0 V - 3.3 V).

dacDisable This function is used to disable DAC output on a given pin/DAC channel.

```
void dacDisable(uint8_t pin);
```

- pin GPIO pin.

2.3.6 Deep Sleep

About

Note: This is a work in progress project and this section is still missing. If you want to contribute, please see the [Contributions Guide](#).

Examples

To get started with Hall sensor, you can try:

ExternalWakeUp

```
/*
  Deep Sleep with External Wake Up
  =====
  This code displays how to use deep sleep with
  an external trigger as a wake up source and how
  to store data in RTC memory to use it over reboots

  This code is under Public Domain License.

  Hardware Connections
  =====
  Push Button to GPIO 33 pulled down with a 10K Ohm
  resistor

  NOTE:
  =====
  Only RTC IO can be used as a source for external wake
  source. They are pins: 0,2,4,12-15,25-27,32-39.

  Author:
  Pranav Cherukupalli <cherukupallip@gmail.com>
*/

#include <Arduino.h>
#include "driver/rtc_io.h"

#define BUTTON_PIN_BITMASK(GPIO) (1ULL << GPIO) // 2 ^ GPIO_NUMBER in hex
#define USE_EXT0_WAKEUP          1                // 1 = EXT0 wakeup, 0 = EXT1
↪wakeup
#define WAKEUP_GPIO              GPIO_NUM_33      // Only RTC IO are allowed
↪ESP32 Pin example
```

(continues on next page)

(continued from previous page)

```

RTC_DATA_ATTR int bootCount = 0;

/*
 * Method to print the reason by which ESP32
 * has been awoken from sleep
 */
void print_wakeup_reason() {
    esp_sleep_wakeup_cause_t wakeup_reason;

    wakeup_reason = esp_sleep_get_wakeup_cause();

    switch (wakeup_reason) {
        case ESP_SLEEP_WAKEUP_EXT0:      Serial.println("Wakeup caused by external_
↪signal using RTC_IO"); break;
        case ESP_SLEEP_WAKEUP_EXT1:      Serial.println("Wakeup caused by external_
↪signal using RTC_CNTL"); break;
        case ESP_SLEEP_WAKEUP_TIMER:      Serial.println("Wakeup caused by timer");
↪break;
        case ESP_SLEEP_WAKEUP_TOUCHPAD:  Serial.println("Wakeup caused by touchpad");
↪break;
        case ESP_SLEEP_WAKEUP_ULP:        Serial.println("Wakeup caused by ULP program");
↪break;
        default:                          Serial.printf("Wakeup was not caused by deep_
↪sleep: %d\n", wakeup_reason); break;
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000); //Take some time to open up the Serial Monitor

    //Increment boot number and print it every reboot
    ++bootCount;
    Serial.println("Boot number: " + String(bootCount));

    //Print the wakeup reason for ESP32
    print_wakeup_reason();

    /*
     * First we configure the wake up source
     * We set our ESP32 to wake up for an external trigger.
     * There are two types for ESP32, ext0 and ext1 .
     * ext0 uses RTC_IO to wakeup thus requires RTC peripherals
     * to be on while ext1 uses RTC Controller so does not need
     * peripherals to be powered on.
     * Note that using internal pullups/pulldowns also requires
     * RTC peripherals to be turned on.
     */
    #if USE_EXT0_WAKEUP
        esp_sleep_enable_ext0_wakeup(WAKEUP_GPIO, 1); //1 = High, 0 = Low
        // Configure pullup/downs via RTCIO to tie wakeup pins to inactive level during_
↪deepsleep.
        // EXT0 resides in the same power domain (RTC_PERIPH) as the RTC IO pullup/downs.
        // No need to keep that power domain explicitly, unlike EXT1.
        rtc_gpio_pullup_dis(WAKEUP_GPIO);
        rtc_gpio_pulldown_en(WAKEUP_GPIO);
    #else // EXT1 WAKEUP
        //If you were to use ext1, you would use it like
        esp_sleep_enable_ext1_wakeup_io(BUTTON_PIN_BITMASK(WAKEUP_GPIO), ESP_EXT1_WAKEUP_
↪ANY_HIGH);
    #endif
}

```

(continues on next page)

(continued from previous page)

```

/*
   If there are no external pull-up/downs, tie wakeup pins to inactive level with
   ↳ internal pull-up/downs via RTC IO
      during deepsleep. However, RTC IO relies on the RTC_PERIPH power domain.
   ↳ Keeping this power domain on will
      increase some power consumption. However, if we turn off the RTC_PERIPH
   ↳ domain or if certain chips lack the RTC_PERIPH
      domain, we will use the HOLD feature to maintain the pull-up and pull-
   ↳ down on the pins during sleep.
*/
rtc_gpio_pulldown_en(WAKEUP_GPIO); // GPIO33 is tie to GND in order to wake up
↳ in HIGH
rtc_gpio_pullup_dis(WAKEUP_GPIO); // Disable PULL_UP in order to allow it to
↳ wakeup on HIGH
#endif
//Go to sleep now
Serial.println("Going to sleep now");
esp_deep_sleep_start();
Serial.println("This will never be printed");
}

void loop() {
    //This is not going to be called
}

```

Timer Wake Up

```

/*
Simple Deep Sleep with Timer Wake Up
=====
ESP32 offers a deep sleep mode for effective power
saving as power is an important factor for IoT
applications. In this mode CPUs, most of the RAM,
and all the digital peripherals which are clocked
from APB_CLK are powered off. The only parts of
the chip which can still be powered on are:
RTC controller, RTC peripherals ,and RTC memories

This code displays the most basic deep sleep with
a timer to wake it up and how to store data in
RTC memory to use it over reboots

This code is under Public Domain License.

Author:
Pranav Cherukupalli <cherukupallip@gmail.com>
*/

#include <Arduino.h>

#define uS_TO_S_FACTOR 1000000ULL /* Conversion factor for micro seconds to
↳ seconds */
#define TIME_TO_SLEEP  5           /* Time ESP32 will go to sleep (in seconds) */

RTC_DATA_ATTR int bootCount = 0;

/*
Method to print the reason by which ESP32
has been awoken from sleep
*/

```

(continues on next page)

(continued from previous page)

```

void print_wakeup_reason() {
    esp_sleep_wakeup_cause_t wakeup_reason;

    wakeup_reason = esp_sleep_get_wakeup_cause();

    switch (wakeup_reason) {
        case ESP_SLEEP_WAKEUP_EXT0:      Serial.println("Wakeup caused by external_
↪signal using RTC_IO"); break;
        case ESP_SLEEP_WAKEUP_EXT1:      Serial.println("Wakeup caused by external_
↪signal using RTC_CNTL"); break;
        case ESP_SLEEP_WAKEUP_TIMER:      Serial.println("Wakeup caused by timer");
↪break;
        case ESP_SLEEP_WAKEUP_TOUCHPAD:  Serial.println("Wakeup caused by touchpad");
↪break;
        case ESP_SLEEP_WAKEUP_ULP:        Serial.println("Wakeup caused by ULP program");
↪break;
        default:                          Serial.printf("Wakeup was not caused by deep_
↪sleep: %d\n", wakeup_reason); break;
    }
}

void setup() {
    Serial.begin(115200);
    delay(1000); //Take some time to open up the Serial Monitor

    //Increment boot number and print it every reboot
    ++bootCount;
    Serial.println("Boot number: " + String(bootCount));

    //Print the wakeup reason for ESP32
    print_wakeup_reason();

    /*
    First we configure the wake up source
    We set our ESP32 to wake up every 5 seconds
    */
    esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);
    Serial.println("Setup ESP32 to sleep for every " + String(TIME_TO_SLEEP) + "
↪Seconds");

    /*
    Next we decide what all peripherals to shut down/keep on
    By default, ESP32 will automatically power down the peripherals
    not needed by the wakeup source, but if you want to be a poweruser
    this is for you. Read in detail at the API docs
    http://esp-idf.readthedocs.io/en/latest/api-reference/system/deep_sleep.html
    Left the line commented as an example of how to configure peripherals.
    The line below turns off all RTC peripherals in deep sleep.
    */
    //esp_deep_sleep_pd_config(ESP_PD_DOMAIN_RTC_PERIPH, ESP_PD_OPTION_OFF);
    //Serial.println("Configured all RTC Peripherals to be powered down in sleep");

    /*
    Now that we have setup a wake cause and if needed setup the
    peripherals state in deep sleep, we can now start going to
    deep sleep.
    In the case that no wake up sources were provided but deep
    sleep was started, it will sleep forever unless hardware
    reset occurs.
    */
    Serial.println("Going to sleep now");
}

```

(continues on next page)

(continued from previous page)

```
Serial.flush();
esp_deep_sleep_start();
Serial.println("This will never be printed");
}

void loop() {
    //This is not going to be called
}
```

2.3.7 ESP-NOW

About

ESP-NOW is a communication protocol designed for low-power, low-latency, and high-throughput communication between ESP32 devices without the need for an access point (AP). It is ideal for scenarios where devices need to communicate directly with each other in a local network. ESP-NOW can be used for smart lights, remote control devices, sensors and many other applications.

This library provides an easy-to-use interface for setting up ESP-NOW communication, adding and removing peers, and sending and receiving data packets.

Arduino-ESP32 ESP-NOW API

ESP-NOW Class The *ESP_NOW_Class* is the main class used for managing ESP-NOW communication.

begin Initialize the ESP-NOW communication. This function must be called before using any other ESP-NOW functionalities.

```
bool begin(const uint8_t *pmk = NULL);
```

- `pmk`: Optional. Pass the pairwise master key (PMK) if encryption is enabled.

Returns `true` if initialization is successful, `false` otherwise.

end End the ESP-NOW communication. This function releases all resources used by the ESP-NOW library.

```
bool end();
```

Returns `true` if the operation is successful, `false` otherwise.

getTotalPeerCount Get the total number of peers currently added.

```
int getTotalPeerCount();
```

Returns the total number of peers, or `-1` if an error occurs.

getEncryptedPeerCount Get the number of peers using encryption.

```
int getEncryptedPeerCount();
```

Returns the number of peers using encryption, or `-1` if an error occurs.

onNewPeer You can register a callback function to handle incoming data from new peers using the *onNewPeer* function.

```
void onNewPeer(void (*cb)(const esp_now_recv_info_t *info, const uint8_t *data,
↳ int len, void *arg), void *arg);
```

- *cb*: Pointer to the callback function.
- *arg*: Optional. Pointer to user-defined argument to be passed to the callback function.

cb function signature:

```
void cb(const esp_now_recv_info_t *info, const uint8_t *data, int len, void *arg);
```

info: Information about the received packet. *data*: Pointer to the received data. *len*: Length of the received data. *arg*: User-defined argument passed to the callback function.

ESP-NOW Peer Class The *ESP_NOW_Peer* class represents a peer device in the ESP-NOW network. It is an abstract class that must be inherited by a child class that properly handles the peer connections and implements the *_onReceive* and *_onSent* methods.

Constructor Create an instance of the *ESP_NOW_Peer* class.

```
ESP_NOW_Peer(const uint8_t *mac_addr, uint8_t channel, wifi_interface_t iface,
↳ const uint8_t *lmk);
```

- *mac_addr*: MAC address of the peer device.
- *channel*: Communication channel.
- *iface*: Wi-Fi interface.
- *lmk*: Optional. Pass the local master key (LMK) if encryption is enabled.

add Add the peer to the ESP-NOW network.

```
bool add();
```

Returns *true* if the peer is added successfully, *false* otherwise.

remove Remove the peer from the ESP-NOW network.

```
bool remove();
```

Returns *true* if the peer is removed successfully, *false* otherwise.

send Send data to the peer.

```
size_t send(const uint8_t *data, int len);
```

- *data*: Pointer to the data to be sent.
- *len*: Length of the data in bytes.

Returns the number of bytes sent, or 0 if an error occurs.

addr Get the MAC address of the peer.

```
const uint8_t * addr() const;
```

Returns a pointer to the MAC address.

addr Set the MAC address of the peer.

```
void addr(const uint8_t *mac_addr);
```

- `mac_addr`: MAC address of the peer.

getChannel Get the communication channel of the peer.

```
uint8_t getChannel() const;
```

Returns the communication channel.

setChannel Set the communication channel of the peer.

```
void setChannel(uint8_t channel);
```

- `channel`: Communication channel.

getInterface Get the Wi-Fi interface of the peer.

```
wifi_interface_t getInterface() const;
```

Returns the Wi-Fi interface.

setInterface Set the Wi-Fi interface of the peer.

```
void setInterface(wifi_interface_t iface);
```

- `iface`: Wi-Fi interface.

isEncrypted Check if the peer is using encryption.

```
bool isEncrypted() const;
```

Returns `true` if the peer is using encryption, `false` otherwise.

setKey Set the local master key (LMK) for the peer.

```
void setKey(const uint8_t *lmk);
```

- `lmk`: Local master key.

onReceive Callback function to handle incoming data from the peer. This is a virtual method can be implemented by the upper class for custom handling.

```
void onReceive(const uint8_t *data, int len, bool broadcast);
```

- `data`: Pointer to the received data.
- `len`: Length of the received data.
- `broadcast`: `true` if the data is broadcasted, `false` otherwise.

onSent Callback function to handle the completion of sending data to the peer. This is a virtual method can be implemented by the upper class for custom handling.


```
void onSent (bool success);
```

- success: true if the data is sent successfully, false otherwise.

Examples

Set of 2 examples of the ESP-NOW library to send and receive data using broadcast messages between multiple ESP32 devices (multiple masters, multiple slaves).

1. ESP-NOW Broadcast Master Example:

```
/*
   ESP-NOW Broadcast Master
   Lucas Saavedra Vaz - 2024

   This sketch demonstrates how to broadcast messages to all devices within the
   ↪ESP-NOW network.
   This example is intended to be used with the ESP-NOW Broadcast Slave example.

   The master device will broadcast a message every 5 seconds to all devices
   ↪within the network.
   This will be done using by registering a peer object with the broadcast
   ↪address.

   The slave devices will receive the broadcasted messages and print them to the
   ↪Serial Monitor.
*/

#include <Arduino.h>
#include "ESP32_NOW.h"
#include "WiFi.h"

#include <esp_mac.h> // For the MAC2STR and MACSTR macros

/* Definitions */

#define ESPNOW_WIFI_CHANNEL 6

/* Classes */

// Creating a new class that inherits from the ESP_NOW_Peer class is required.

class ESP_NOW_Broadcast_Peer : public ESP_NOW_Peer {
public:
    // Constructor of the class using the broadcast address
    ESP_NOW_Broadcast_Peer(uint8_t channel, wifi_interface_t iface, const uint8_t
    ↪*lmk) : ESP_NOW_Peer(ESP_NOW.BROADCAST_ADDR, channel, iface, lmk) {}

    // Destructor of the class
    ~ESP_NOW_Broadcast_Peer() {
        remove();
    }

    // Function to properly initialize the ESP-NOW and register the broadcast peer
    bool begin() {
        if (!ESP_NOW.begin() || !add()) {
            log_e("Failed to initialize ESP-NOW or register the broadcast peer");
            return false;
        }
        return true;
    }
}
```

(continues on next page)

(continued from previous page)

```

// Function to send a message to all devices within the network
bool send_message(const uint8_t *data, size_t len) {
    if (!send(data, len)) {
        log_e("Failed to broadcast message");
        return false;
    }
    return true;
}

};

/* Global Variables */

uint32_t msg_count = 0;

// Create a broadcast peer object
ESP_NOW_Broadcast_Peer broadcast_peer(ESP_NOW_WIFI_CHANNEL, WIFI_IF_STA, nullptr);

/* Main */

void setup() {
    Serial.begin(115200);

    // Initialize the Wi-Fi module
    WiFi.mode(WIFI_STA);
    WiFi.setChannel(ESP_NOW_WIFI_CHANNEL);
    while (!WiFi.STA.started()) {
        delay(100);
    }

    Serial.println("ESP-NOW Example - Broadcast Master");
    Serial.println("Wi-Fi parameters:");
    Serial.println("  Mode: STA");
    Serial.println("  MAC Address: " + WiFi.macAddress());
    Serial.printf("  Channel: %u\n", ESP_NOW_WIFI_CHANNEL);

    // Register the broadcast peer
    if (!broadcast_peer.begin()) {
        Serial.println("Failed to initialize broadcast peer");
        Serial.println("Rebooting in 5 seconds...");
        delay(5000);
        ESP.restart();
    }

    Serial.printf("ESP-NOW version: %d, max data length: %d\n", ESP_NOW.getVersion(),
↳ ESP_NOW.getMaxDataLen());

    Serial.println("Setup complete. Broadcasting messages every 5 seconds.");
}

void loop() {
    // Broadcast a message to all devices within the network
    char data[32];
    snprintf(data, sizeof(data), "Hello, World! #%d" PRIu32, msg_count++);

    Serial.printf("Broadcasting message: %s\n", data);

    if (!broadcast_peer.send_message((uint8_t *)data, sizeof(data))) {
        Serial.println("Failed to broadcast message");
    }
}

```

(continues on next page)

(continued from previous page)

```

    delay(5000);
}

```

2. ESP-NOW Broadcast Slave Example:

```

/*
   ESP-NOW Broadcast Slave
   Lucas Saavedra Vaz - 2024

   This sketch demonstrates how to receive broadcast messages from a master_
   ↪device using the ESP-NOW protocol.

   The master device will broadcast a message every 5 seconds to all devices_
   ↪within the network.

   The slave devices will receive the broadcasted messages. If they are not from_
   ↪a known master, they will be registered as a new master
   using a callback function.
*/

#include <Arduino.h>
#include "ESP32_NOW.h"
#include "WiFi.h"

#include <esp_mac.h> // For the MAC2STR and MACSTR macros

#include <vector>

/* Definitions */

#define ESPNOW_WIFI_CHANNEL 6

/* Classes */

// Creating a new class that inherits from the ESP_NOW_Peer class is required.

class ESP_NOW_Peer_Class : public ESP_NOW_Peer {
public:
    // Constructor of the class
    ESP_NOW_Peer_Class(const uint8_t *mac_addr, uint8_t channel, wifi_interface_t_
    ↪iface, const uint8_t *lmk) : ESP_NOW_Peer(mac_addr, channel, iface, lmk) {}

    // Destructor of the class
    ~ESP_NOW_Peer_Class() {}

    // Function to register the master peer
    bool add_peer() {
        if (!add()) {
            log_e("Failed to register the broadcast peer");
            return false;
        }
        return true;
    }

    // Function to print the received messages from the master
    void onReceive(const uint8_t *data, size_t len, bool broadcast) {
        Serial.printf("Received a message from master " MACSTR " (%s)\n", _
        ↪MAC2STR(addr()), broadcast ? "broadcast" : "unicast");
        Serial.printf("  Message: %s\n", (char *)data);
    }
};

```

(continues on next page)

(continued from previous page)

```

/* Global Variables */

// List of all the masters. It will be populated when a new master is registered
// Note: Using pointers instead of objects to prevent dangling pointers when the
↪vector reallocates
std::vector<ESP_NOW_Peer_Class *> masters;

/* Callbacks */

// Callback called when an unknown peer sends a message
void register_new_master(const esp_now_recv_info_t *info, const uint8_t *data, int_
↪len, void *arg) {
    if (memcmp(info->des_addr, ESP_NOW.BROADCAST_ADDR, 6) == 0) {
        Serial.printf("Unknown peer " MACSTR " sent a broadcast message\n", _
↪MAC2STR(info->src_addr));
        Serial.println("Registering the peer as a master");

        ESP_NOW_Peer_Class *new_master = new ESP_NOW_Peer_Class(info->src_addr, ESPNOW_
↪WIFI_CHANNEL, WIFI_IF_STA, nullptr);
        if (!new_master->add_peer()) {
            Serial.println("Failed to register the new master");
            delete new_master;
            return;
        }
        masters.push_back(new_master);
        Serial.printf("Successfully registered master " MACSTR " (total masters: %lu)\n
↪", MAC2STR(new_master->addr()), (unsigned long)masters.size());
    } else {
        // The slave will only receive broadcast messages
        log_v("Received a unicast message from " MACSTR, MAC2STR(info->src_addr));
        log_v("Ignoring the message");
    }
}

/* Main */

void setup() {
    Serial.begin(115200);

    // Initialize the Wi-Fi module
    WiFi.mode(WIFI_STA);
    WiFi.setChannel(ESPNOW_WIFI_CHANNEL);
    while (!WiFi.STA.started()) {
        delay(100);
    }

    Serial.println("ESP-NOW Example - Broadcast Slave");
    Serial.println("Wi-Fi parameters:");
    Serial.println("  Mode: STA");
    Serial.println("  MAC Address: " + WiFi.macAddress());
    Serial.printf("  Channel: %u\n", ESPNOW_WIFI_CHANNEL);

    // Initialize the ESP-NOW protocol
    if (!ESP_NOW.begin()) {
        Serial.println("Failed to initialize ESP-NOW");
        Serial.println("Rebooting in 5 seconds...");
        delay(5000);
        ESP.restart();
    }
}

```

(continues on next page)

(continued from previous page)

```

Serial.printf("ESP-NOW version: %d, max data length: %d\n", ESP_NOW.getVersion(),
↳ ESP_NOW.getMaxDataLen());

// Register the new peer callback
ESP_NOW.onNewPeer(register_new_master, nullptr);

Serial.println("Setup complete. Waiting for a master to broadcast a message...");
}

void loop() {
  // Print debug information every 10 seconds
  static unsigned long last_debug = 0;
  if (millis() - last_debug > 10000) {
    last_debug = millis();
    Serial.printf("Registered masters: %lu\n", (unsigned long)masters.size());
    for (size_t i = 0; i < masters.size(); i++) {
      if (masters[i]) {
        Serial.printf(" Master %lu: " MACSTR "\n", (unsigned long)i,
↳MAC2STR(masters[i]->addr()));
      }
    }
  }

  delay(100);
}

```

Example of the ESP-NOW Serial library to send and receive data as a stream between 2 ESP32 devices using the serial monitor:

```

/*
  ESP-NOW Serial Example - Unicast transmission
  Lucas Saavedra Vaz - 2024
  Send data between two ESP32s using the ESP-NOW protocol in one-to-one
↳(unicast) configuration.
  Note that different MAC addresses are used for different interfaces.
  The devices can be in different modes (AP or Station) and still communicate
↳using ESP-NOW.
  The only requirement is that the devices are on the same Wi-Fi channel.
  Set the peer MAC address according to the device that will receive the data.

  Example setup:
  - Device 1: AP mode with MAC address F6:12:FA:42:B6:E8
    Peer MAC address set to the Station MAC address of Device 2
↳(F4:12:FA:40:64:4C)
  - Device 2: Station mode with MAC address F4:12:FA:40:64:4C
    Peer MAC address set to the AP MAC address of Device 1
↳(F6:12:FA:42:B6:E8)

  The device running this sketch will also receive and print data from any
↳device that has its MAC address set as the peer MAC address.
  To properly visualize the data being sent, set the line ending in the Serial
↳Monitor to "Both NL & CR".
*/

#include <Arduino.h>
#include "ESP32_NOW_Serial.h"
#include "MacAddress.h"
#include "WiFi.h"

#include "esp_wifi.h"

```

(continues on next page)

(continued from previous page)

```

// 0: AP mode, 1: Station mode
#define ESPNOW_WIFI_MODE_STATION 1

// Channel to be used by the ESP-NOW protocol
#define ESPNOW_WIFI_CHANNEL 1

#if ESPNOW_WIFI_MODE_STATION // ESP-NOW using WiFi Station mode
#define ESPNOW_WIFI_MODE WIFI_STA // WiFi Mode
#define ESPNOW_WIFI_IF WIFI_IF_STA // WiFi Interface
#else // ESP-NOW using WiFi AP mode
#define ESPNOW_WIFI_MODE WIFI_AP // WiFi Mode
#define ESPNOW_WIFI_IF WIFI_IF_AP // WiFi Interface
#endif

// Set the MAC address of the device that will receive the data
// For example: F4:12:FA:40:64:4C
const MacAddress peer_mac({0xF4, 0x12, 0xFA, 0x40, 0x64, 0x4C});

ESP_NOW_Serial_Class NowSerial(peer_mac, ESPNOW_WIFI_CHANNEL, ESPNOW_WIFI_IF);

void setup() {
    Serial.begin(115200);

    Serial.print("WiFi Mode: ");
    Serial.println(ESPNOW_WIFI_MODE == WIFI_AP ? "AP" : "Station");
    WiFi.mode(ESPNOW_WIFI_MODE);

    Serial.print("Channel: ");
    Serial.println(ESPNOW_WIFI_CHANNEL);
    WiFi.setChannel(ESPNOW_WIFI_CHANNEL, WIFI_SECOND_CHAN_NONE);

    while (!(WiFi.STA.started() || WiFi.AP.started())) {
        delay(100);
    }

    Serial.print("MAC Address: ");
    Serial.println(ESPNOW_WIFI_MODE == WIFI_AP ? WiFi.softAPmacAddress() : WiFi.
↳ macAddress());

    // Start the ESP-NOW communication
    Serial.println("ESP-NOW communication starting...");
    NowSerial.begin(115200);
    Serial.printf("ESP-NOW version: %d, max data length: %d\n", ESP_NOW.getVersion(),
↳ ESP_NOW.getMaxDataLen());
    Serial.println("You can now send data to the peer device using the Serial_
↳ Monitor.\n");
}

void loop() {
    while (NowSerial.available()) {
        Serial.write(NowSerial.read());
    }

    while (Serial.available() && NowSerial.availableForWrite()) {
        if (NowSerial.write(Serial.read()) <= 0) {
            Serial.println("Failed to send data");
            break;
        }
    }
}

delay(1);

```

(continues on next page)

(continued from previous page)

}

2.3.8 Ethernet

About

Note: This is a work in progress project and this section is still missing. If you want to contribute, please see the [Contributions Guide](#).

Examples

To get started with Ethernet, you can try:

LAN8720

```
/*
   This sketch shows the Ethernet event usage
*/

// Important to be defined BEFORE including ETH.h for ETH.begin() to work.
// Example RMII LAN8720 (Olimex, etc.)
#include <Arduino.h>
#ifndef ETH_PHY_MDC
#define ETH_PHY_TYPE ETH_PHY_LAN8720
#endif
#define CONFIG_IDF_TARGET_ESP32
#define ETH_PHY_ADDR 0
#define ETH_PHY_MDC 23
#define ETH_PHY_MDIO 18
#define ETH_PHY_POWER -1
#define ETH_CLK_MODE ETH_CLOCK_GPIO0_IN
#elif CONFIG_IDF_TARGET_ESP32P4
#define ETH_PHY_ADDR 0
#define ETH_PHY_MDC 31
#define ETH_PHY_MDIO 52
#define ETH_PHY_POWER 51
#define ETH_CLK_MODE EMAC_CLK_EXT_IN
#endif
#include <ETH.h>

static bool eth_connected = false;

// WARNING: onEvent is called from a separate FreeRTOS task (thread)!
void onEvent(arduino_event_id_t event) {
    switch (event) {
        case ARDUINO_EVENT_ETH_START:
            Serial.println("ETH Started");
            // The hostname must be set after the interface is started, but needs
            // to be set before DHCP, so set it from the event handler thread.
            ETH.setHostname("esp32-ethernet");
            break;
        case ARDUINO_EVENT_ETH_CONNECTED: Serial.println("ETH Connected"); break;
        case ARDUINO_EVENT_ETH_GOT_IP:
```

(continues on next page)

(continued from previous page)

```

        Serial.println("ETH Got IP");
        Serial.println(ETH);
        eth_connected = true;
        break;
    case ARDUINO_EVENT_ETH_LOST_IP:
        Serial.println("ETH Lost IP");
        eth_connected = false;
        break;
    case ARDUINO_EVENT_ETH_DISCONNECTED:
        Serial.println("ETH Disconnected");
        eth_connected = false;
        break;
    case ARDUINO_EVENT_ETH_STOP:
        Serial.println("ETH Stopped");
        eth_connected = false;
        break;
    default: break;
}
}

void testClient(const char *host, uint16_t port) {
    Serial.print("\nconnecting to ");
    Serial.println(host);

    NetworkClient client;
    if (!client.connect(host, port)) {
        Serial.println("connection failed");
        return;
    }
    client.printf("GET / HTTP/1.1\r\nHost: %s\r\n\r\n", host);
    while (client.connected() && !client.available());
    while (client.available()) {
        Serial.write(client.read());
    }

    Serial.println("closing connection\n");
    client.stop();
}

void setup() {
    Serial.begin(115200);
    Network.onEvent(onEvent);
    ETH.begin();
}

void loop() {
    if (eth_connected) {
        testClient("google.com", 80);
    }
    delay(10000);
}

```

TLK110

```

/*
   This sketch shows the Ethernet event usage
*/

#include <Arduino.h>

```

(continues on next page)

(continued from previous page)

```

#include <ETH.h>

#ifndef ETH_PHY_MDC
#define ETH_PHY_TYPE ETH_PHY_TLK110
#if CONFIG_IDF_TARGET_ESP32
#define ETH_PHY_ADDR 31
#define ETH_PHY_MDC 23
#define ETH_PHY_MDIO 18
#define ETH_PHY_POWER 17
#define ETH_CLK_MODE ETH_CLOCK_GPIO0_IN
#elif CONFIG_IDF_TARGET_ESP32P4
#define ETH_PHY_ADDR 1
#define ETH_PHY_MDC 31
#define ETH_PHY_MDIO 52
#define ETH_PHY_POWER 51
#define ETH_CLK_MODE EMAC_CLK_EXT_IN
#endif
#endif

static bool eth_connected = false;

// WARNING: onEvent is called from a separate FreeRTOS task (thread)!
void onEvent(arduino_event_id_t event) {
    switch (event) {
        case ARDUINO_EVENT_ETH_START:
            Serial.println("ETH Started");
            // The hostname must be set after the interface is started, but needs
            // to be set before DHCP, so set it from the event handler thread.
            ETH.setHostname("esp32-ethernet");
            break;
        case ARDUINO_EVENT_ETH_CONNECTED: Serial.println("ETH Connected"); break;
        case ARDUINO_EVENT_ETH_GOT_IP:
            Serial.println("ETH Got IP");
            Serial.println(ETH);
            eth_connected = true;
            break;
        case ARDUINO_EVENT_ETH_LOST_IP:
            Serial.println("ETH Lost IP");
            eth_connected = false;
            break;
        case ARDUINO_EVENT_ETH_DISCONNECTED:
            Serial.println("ETH Disconnected");
            eth_connected = false;
            break;
        case ARDUINO_EVENT_ETH_STOP:
            Serial.println("ETH Stopped");
            eth_connected = false;
            break;
        default: break;
    }
}

void testClient(const char *host, uint16_t port) {
    Serial.print("\nconnecting to ");
    Serial.println(host);

    NetworkClient client;
    if (!client.connect(host, port)) {
        Serial.println("connection failed");
        return;
    }
}

```

(continues on next page)

(continued from previous page)

```

client.printf("GET / HTTP/1.1\r\nHost: %s\r\n\r\n", host);
while (client.connected() && !client.available());
while (client.available()) {
    Serial.write(client.read());
}

Serial.println("closing connection\n");
client.stop();
}

void setup() {
    Serial.begin(115200);
    Network.onEvent(onEvent); // Will call onEvent() from another thread.
    ETH.begin(ETH_PHY_TYPE, ETH_PHY_ADDR, ETH_PHY_MDC, ETH_PHY_MDIO, ETH_PHY_POWER,
    ↪ETH_CLK_MODE);
}

void loop() {
    if (eth_connected) {
        testClient("google.com", 80);
    }
    delay(10000);
}

```

Complete list of [Ethernet examples](#).

2.3.9 GPIO

About

One of the most used and versatile peripheral in a microcontroller is the GPIO. The GPIO is commonly used to write and read the pin state.

GPIO stands to General Purpose Input Output, and is responsible to control or read the state of a specific pin in the digital world. For example, this peripheral is widely used to create the LED blinking or to read a simple button.

Note: There are some GPIOs with special restrictions, and not all GPIOs are accessible through the development board. For more information about it, see the corresponding board pin layout information.

GPIOs Modes There are two different modes in the GPIO configuration:

- **Input Mode**

In this mode, the GPIO will receive the digital state from a specific device. This device could be a button or a switch.

- **Output Mode**

For the output mode, the GPIO will change the GPIO digital state to a specific device. You can drive an LED for example.

GPIO API

Here is the common functions used for the GPIO peripheral.

pinMode The `pinMode` function is used to define the GPIO operation mode for a specific pin.

```
void pinMode(uint8_t pin, uint8_t mode);
```

- `pin` defines the GPIO pin number.
- `mode` sets operation mode.

The following modes are supported for the basic *input* and *output*:

- **INPUT** sets the GPIO as input without pullup or pulldown (high impedance).
- **OUTPUT** sets the GPIO as output/read mode.
- **INPUT_PULLDOWN** sets the GPIO as input with the internal pulldown.
- **INPUT_PULLUP** sets the GPIO as input with the internal pullup.

Internal Pullup and Pulldown The ESP32 SoC families supports the internal pullup and pulldown through a 45kR resistor, that can be enabled when configuring the GPIO mode as **INPUT** mode. If the pullup or pulldown mode is not defined, the pin will stay in the high impedance mode.

digitalWrite The function `digitalWrite` sets the state of the selected GPIO to **HIGH** or **LOW**. This function is only used if the `pinMode` was configured as **OUTPUT**.

```
void digitalWrite(uint8_t pin, uint8_t val);
```

- `pin` defines the GPIO pin number.
- `val` set the output digital state to **HIGH** or **LOW**.

digitalRead To read the state of a given pin configured as **INPUT**, the function `digitalRead` is used.

```
int digitalRead(uint8_t pin);
```

- `pin` select GPIO

This function will return the logical state of the selected pin as **HIGH** or **LOW**.

Interrupts

The GPIO peripheral on the ESP32 supports interruptions.

attachInterrupt The function `attachInterrupt` is used to attach the interrupt to the defined pin.

```
attachInterrupt(uint8_t pin, voidFuncPtr handler, int mode);
```

- `pin` defines the GPIO pin number.
- `handler` set the handler function.
- `mode` set the interrupt mode.

Here are the supported interrupt modes:

- **DISABLED**
- **RISING**
- **FALLING**
- **CHANGE**
- **ONLOW**
- **ONHIGH**
- **ONLOW_WE**
- **ONHIGH_WE**

attachInterruptArg The function `attachInterruptArg` is used to attach the interrupt to the defined pin using arguments.

```
attachInterruptArg(uint8_t pin, voidFuncPtrArg handler, void * arg, int mode);
```

- `pin` defines the GPIO pin number.
- `handler` set the handler function.
- `arg` pointer to the interrupt arguments.
- `mode` set the interrupt mode.

detachInterrupt To detach the interruption from a specific pin, use the `detachInterrupt` function giving the GPIO to be detached.

```
detachInterrupt(uint8_t pin);
```

- `pin` defines the GPIO pin number.

Example Code

GPIO Input and Output Modes

```
#define LED 12
#define BUTTON 2

uint8_t stateLED = 0;

void setup() {
    pinMode(LED, OUTPUT);
    pinMode(BUTTON, INPUT_PULLUP);
}

void loop() {

    if(!digitalRead(BUTTON)) {
        stateLED = stateLED^1;
        digitalWrite(LED, stateLED);
    }
}
```

GPIO Interrupt

```
#include <Arduino.h>

struct Button {
    const uint8_t PIN;
    uint32_t numberKeyPresses;
    bool pressed;
};

Button button1 = {23, 0, false};
Button button2 = {18, 0, false};

void ARDUINO_ISR_ATTR isr(void *arg) {
    Button *s = static_cast<Button *>(arg);
    s->numberKeyPresses += 1;
    s->pressed = true;
}

void ARDUINO_ISR_ATTR isr() {
    button2.numberKeyPresses += 1;
```

(continues on next page)

(continued from previous page)

```

    button2.pressed = true;
}

void setup() {
    Serial.begin(115200);
    pinMode(button1.PIN, INPUT_PULLUP);
    attachInterruptArg(button1.PIN, isr, &button1, FALLING);
    pinMode(button2.PIN, INPUT_PULLUP);
    attachInterrupt(button2.PIN, isr, FALLING);
}

void loop() {
    if (button1.pressed) {
        Serial.printf("Button 1 has been pressed %" PRIu32 " times\n", button1.
↪numberKeyPresses);
        button1.pressed = false;
    }
    if (button2.pressed) {
        Serial.printf("Button 2 has been pressed %" PRIu32 " times\n", button2.
↪numberKeyPresses);
        button2.pressed = false;
    }
    static uint32_t lastMillis = 0;
    if (millis() - lastMillis > 10000) {
        lastMillis = millis();
        detachInterrupt(button1.PIN);
    }
}

```

2.3.10 I2C

About

I2C (Inter-Integrated Circuit) / TWI (Two-wire Interface) is a widely used serial communication to connect devices in a short distance. This is one of the most common peripherals used to connect sensors, EEPROMs, RTC, ADC, DAC, displays, OLED, and many other devices and microcontrollers.

This serial communication is considered as a low-speed bus, and multiple devices can be connected on the same two-wires bus, each with a unique 7-bits address (up to 128 devices). These two wires are called SDA (serial data line) and SCL (serial clock line).

Note: The SDA and SCL lines require pull-up resistors. See the device datasheet for more details about the resistors' values and the operating voltage.

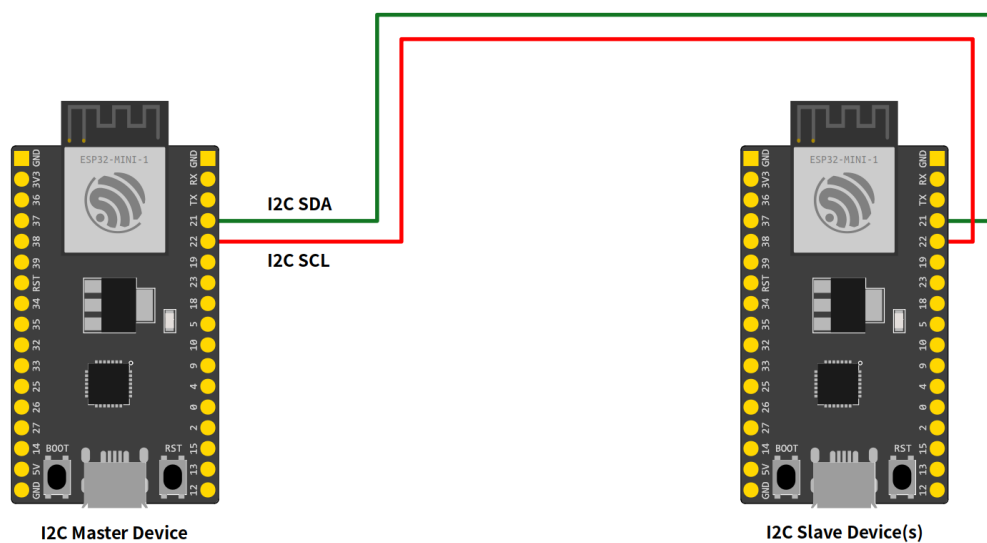
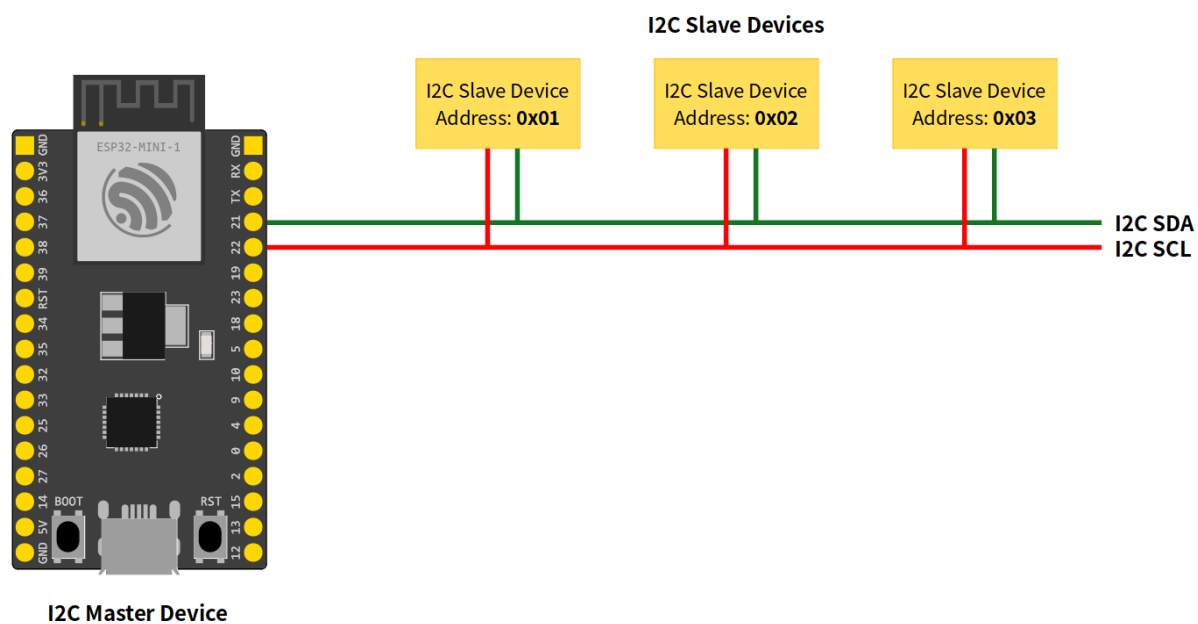
I2C Modes The I2C can be used in two different modes:

- I2C Master Mode**

- In this mode, the ESP32 generates the clock signal and initiates the communication with the slave device.

- I2C Slave Mode**

- The slave mode, the clock is generated by the master device and responds to the master if the destination address is the same as the destination.



Arduino-ESP32 I2C API

The ESP32 I2C library is based on the [Arduino Wire Library](#) and implements a few more APIs, described in this documentation.

I2C Common API Here are the common functions used for master and slave modes.

begin This function is used to start the peripheral using the default configuration.

```
bool begin();
```

This function will return `true` if the peripheral was initialized correctly.

setPins This function is used to define the SDA and SCL pins.

Note: Call this function before `begin` to change the pins from the default ones.

```
bool setPins(int sdaPin, int sclPin);
```

- `sdaPin` sets the GPIO to be used as the I2C peripheral data line.
- `sclPin` sets the GPIO to be used as the I2C peripheral clock line.

The default pins may vary from board to board. On the *Generic ESP32* the default I2C pins are:

- `sdaPin` **GPIO21**
- `sclPin` **GPIO22**

This function will return `true` if the peripheral was configured correctly.

setClock Use this function to set the bus clock. The default value will be used if this function is not used.

```
bool setClock(uint32_t frequency);
```

- `frequency` sets the bus frequency clock.

This function will return `true` if the clock was configured correctly.

getClock Use this function to get the bus clock.

```
uint32_t getClock();
```

This function will return the current frequency configuration.

setTimeout Set the bus timeout given in milliseconds. The default value is 50 ms.

```
void setTimeout(uint16_t timeOutMillis);
```

- `timeOutMillis` sets the timeout in ms.

getTimeOut Get the bus timeout in milliseconds.

```
uint16_t getTimeOut();
```

This function will return the current timeout configuration.

write This function writes data to the buffer.

```
size_t write(uint8_t);
```

or

```
size_t write(const uint8_t *, size_t);
```

The return will be the size of the data added to the buffer.

end This function will finish the communication and release all the allocated resources. After calling `end` you need to use `begin` again in order to initialize the I2C driver again.

```
bool end();
```

I2C Master Mode This mode is used to initiate communication to the slave.

Basic Usage To start using I2C master mode on the Arduino, the first step is to include the `Wire.h` header to the sketch.

```
#include "Wire.h"
```

Now, we can start the peripheral configuration by calling `begin` function.

```
Wire.begin();
```

By using `begin` without any arguments, all the settings will be done by using the default values. To set the values by your own, see the function description. This function is described here: [i2c begin](#)

After calling `begin`, we can start the transmission by calling `beginTransaction` and passing the I2C slave address:

```
Wire.beginTransaction(I2C_DEV_ADDR);
```

To write some bytes to the slave, use the `write` function.

```
Wire.write(x);
```

You can pass different data types using `write` function. This function is described here: [i2c write](#)

Note: The `write` function does not write directly to the slave device but adds to the I2C buffer. To do so, you need to use the `endTransmission` function to send the buffered bytes to the slave device.

```
Wire.endTransmission(true);
```

After calling `endTransmission`, the data stored in the I2C buffer will be transmitted to the slave device.

Now you can request a reading from the slave device. The `requestFrom` will ask for a readout to the selected device by giving the address and the size.

```
Wire.requestFrom(I2C_DEV_ADDR, SIZE);
```

and the `readBytes` will read it.

```
Wire.readBytes(temp, error);
```

I2C Master APIs Here are the I2C master APIs. These function are intended to be used only for master mode.

begin In master mode, the `begin` function can be used by passing the **pins** and **bus frequency**. Use this function only for the master mode.

```
bool begin(int sdaPin, int sclPin, uint32_t frequency)
```

Alternatively, you can use the `begin` function without any argument to use all default values.

This function will return `true` if the peripheral was initialized correctly.

beginTransaction This function is used to start a communication process with the slave device. Call this function by passing the slave address before writing the message to the buffer.

```
void beginTransmission(uint16_t address)
```

endTransmission After writing to the buffer using *i2c write*, use the function `endTransmission` to send the message to the slave device address defined on the `beginTransaction` function.

```
uint8_t endTransmission(bool sendStop);
```

- `sendStop` enables (**true**) or disables (**false**) the stop signal (*only used in master mode*).

Calling the this function without `sendStop` is equivalent to `sendStop = true`.

```
uint8_t endTransmission(void);
```

This function will return the error code.

requestFrom To read from the slave device, use the `requestFrom` function.

```
uint8_t requestFrom(uint16_t address, uint8_t size, bool sendStop)
```

- `address` set the device address.
- `size` define the size to be requested.
- `sendStop` enables (`true`) or disables (`false`) the stop signal.

This function will return the number of bytes read from the device.

Example Application - WireMaster.ino Here is an example of how to use the I2C in Master Mode.

```
#include <Arduino.h>
#include "Wire.h"

#define I2C_DEV_ADDR 0x55

uint32_t i = 0;

void setup() {
  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Wire.begin();
}

void loop() {
  delay(5000);

  //Write message to the slave
  Wire.beginTransaction(I2C_DEV_ADDR);
  Wire.printf("Hello World! %" PRIu32, i++);
  uint8_t error = Wire.endTransmission(true);
```

(continues on next page)

(continued from previous page)

```
Serial.printf("endTransmission: %u\n", error);

//Read 16 bytes from the slave
uint8_t bytesReceived = Wire.requestFrom(I2C_DEV_ADDR, 16);
Serial.printf("requestFrom: %u\n", bytesReceived);
if ((bool)bytesReceived) { //If received more than zero bytes
    uint8_t temp[bytesReceived];
    Wire.readBytes(temp, bytesReceived);
    log_print_buf(temp, bytesReceived);
}
}
```

I2C Slave Mode This mode is used to accept communication from the master.

Basic Usage To start using I2C as slave mode on the Arduino, the first step is to include the `Wire.h` header to the sketch.

```
#include "Wire.h"
```

Before calling `begin` we must create two callback functions to handle the communication with the master device.

```
Wire.onReceive(onReceive);
```

and

```
Wire.onRequest(onRequest);
```

The `onReceive` will handle the request from the master device upon a slave read request and the `onRequest` will handle the answer to the master.

Now, we can start the peripheral configuration by calling `begin` function with the device address.

```
Wire.begin((uint8_t) I2C_DEV_ADDR);
```

By using `begin` without any arguments, all the settings will be done by using the default values. To set the values by your own, see the function description. This function is described here: [i2c begin](#)

For ESP32 only!

Use the function `slaveWrite` in order to pre-write to the slave response buffer. This is used only for the ESP32 in order to add the slave capability on the chip and keep compatibility with Arduino.

```
Wire.slaveWrite((uint8_t *)message, strlen(message));
```

I2C Slave APIs Here are the I2C slave APIs. These function are intended to be used only for slave mode.

begin In slave mode, the `begin` function must be used by passing the **slave address**. You can also define the **pins** and the **bus frequency**.

```
bool Wire.begin(uint8_t addr, int sdaPin, int sclPin, uint32_t frequency)
```

This function will return `true` if the peripheral was initialized correctly.

onReceive The `onReceive` function is used to define the callback for data received from the master device.

```
void onReceive(const std::function<void(int)>& callback);
```

Function Signature:

The callback function must have the signature `void(int numBytes)` where `numBytes` indicates how many bytes were received from the master.

Usage Examples:

```
// Method 1: Regular function
void handleReceive(int numBytes) {
    Serial.printf("Received %d bytes: ", numBytes);
    while (Wire.available()) {
        char c = Wire.read();
        Serial.print(c);
    }
    Serial.println();
}
Wire.onReceive(handleReceive);

// Method 2: Lambda function
Wire.onReceive([](int numBytes) {
    Serial.printf("Master sent %d bytes\n", numBytes);
    while (Wire.available()) {
        uint8_t data = Wire.read();
        // Process received data
        Serial.printf("Data: 0x%02X\n", data);
    }
});

// Method 3: Lambda with capture (for accessing variables)
int deviceId = 42;
Wire.onReceive([deviceId](int numBytes) {
    Serial.printf("Device %d received %d bytes\n", deviceId, numBytes);
    // Process data...
});

// Method 4: Using std::function variable
std::function<void(int)> receiveHandler = [](int bytes) {
    Serial.printf("Handling %d received bytes\n", bytes);
};
Wire.onReceive(receiveHandler);

// Method 5: Class member function (using lambda wrapper)
class I2CDevice {
private:
    int deviceAddress;
public:
    I2CDevice(int addr) : deviceAddress(addr) {}

    void handleReceive(int numBytes) {
        Serial.printf("Device 0x%02X received %d bytes\n", deviceAddress, ↵
↵numBytes);
    }

    void setup() {
        Wire.onReceive([this](int bytes) {
            this->handleReceive(bytes);
        });
    }
};
```

Note: The `onReceive` callback is triggered when the I2C master sends data to this slave device. Use `Wire.available()` and `Wire.read()` inside the callback to retrieve the received data.

onRequest The `onRequest` function is used to define the callback for responding to master read requests.

```
void onRequest(const std::function<void()>& callback);
```

Function Signature:

The callback function must have the signature `void()` with no parameters. This callback is triggered when the master requests data from this slave device.

Usage Examples:

```
// Method 1: Regular function
void handleRequest() {
    static int counter = 0;
    Wire.printf("Response #%d", counter++);
}
Wire.onRequest(handleRequest);

// Method 2: Lambda function
Wire.onRequest([]() {
    // Send sensor data to master
    int sensorValue = analogRead(A0);
    Wire.write(sensorValue >> 8); // High byte
    Wire.write(sensorValue & 0xFF); // Low byte
});

// Method 3: Lambda with capture (for accessing variables)
int deviceStatus = 1;
String deviceName = "Sensor1";
Wire.onRequest([&deviceStatus, &deviceName]() {
    Wire.write(deviceStatus);
    Wire.write(deviceName.c_str(), deviceName.length());
});

// Method 4: Using std::function variable
std::function<void()> requestHandler = []() {
    Wire.write("Hello Master!");
};
Wire.onRequest(requestHandler);

// Method 5: Class member function (using lambda wrapper)
class TemperatureSensor {
private:
    float temperature;
public:
    void updateTemperature() {
        temperature = 25.5; // Read from actual sensor
    }

    void sendTemperature() {
        // Convert float to bytes and send
        uint8_t* tempBytes = (uint8_t*)&temperature;
        Wire.write(tempBytes, sizeof(float));
    }

    void setup() {
        Wire.onRequest([this]() {
```

(continues on next page)

(continued from previous page)

```

        this->sendTemperature();
    });
}
};

```

Note: The `onRequest` callback is triggered when the I2C master requests data from this slave device. Use `Wire.write()` inside the callback to send response data back to the master.

slaveWrite The `slaveWrite` function writes on the slave response buffer before receiving the response message. This function is only used for adding the slave compatibility for the ESP32.

Warning: This function is only required for the ESP32. You **don't** need to use for ESP32-S2 and ESP32-C3.

```
size_t slaveWrite(const uint8_t *, size_t);
```

Example Application - WireSlave.ino Here is an example of how to use the I2C in Slave Mode.

```

#include <Arduino.h>
#include "Wire.h"

#define I2C_DEV_ADDR 0x55

uint32_t i = 0;

void onRequest() {
    Wire.print(i++);
    Wire.print(" Packets.");
    Serial.println("onRequest");
}

void onReceive(int len) {
    Serial.printf("onReceive[%d]: ", len);
    while (Wire.available()) {
        Serial.write(Wire.read());
    }
    Serial.println();
}

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    Wire.onReceive(onReceive);
    Wire.onRequest(onRequest);
    Wire.begin((uint8_t) I2C_DEV_ADDR);

#ifdef CONFIG_IDF_TARGET_ESP32
    char message[64];
    snprintf(message, 64, "%u PRIu32 Packets.", i++);
    Wire.slaveWrite((uint8_t *)message, strlen(message));
#endif
}

void loop() {}

```

2.3.11 I2S

About

I2S - Inter-IC Sound, correctly written I²S pronounced “eye-squared-ess” , alternative notation is IIS. I²S is an electrical serial bus interface standard used for connecting digital audio devices together.

It is used to communicate PCM (Pulse-Code Modulation) audio data between integrated circuits in an electronic device. The I²S bus separates clock and serial data signals, resulting in simpler receivers than those required for asynchronous communications systems that need to recover the clock from the data stream.

Despite the similar name, I²S is unrelated and incompatible with the bidirectional I²C (IIC) bus.

The I²S bus consists of at least three lines:

Note: All lines can be attached to almost any pin and this change can occur even during operation.

- **Bit clock line**
 - Officially “continuous serial clock (SCK)” . Typically written “bit clock (BCLK)” .
 - In this library function parameter `sck`.
- **Word clock line**
 - Officially “word select (WS)” . Typically called “left-right clock (LRCLK)” or “frame sync (FS)” .
 - 0 = Left channel, 1 = Right channel
 - In this library function parameter `ws`.
- **Data line**
 - Officially “serial data (SD)” , but can be called `SDATA`, `SDIN`, `SDOUT`, `DACDAT`, `ADCDAT`, etc.
 - Unlike Arduino I2S with single data pin switching between input and output, in ESP core driver use separate data line for input and output.
 - Output data line is called `dout` for function parameter.
 - Input data line is called `din` for function parameter.

It may also include a **Master clock** line:

- **Master clock**
 - Officially “master clock (MCLK)” .
 - This is not a part of I2S bus, but is used to synchronize multiple I2S devices.
 - In this library function parameter `mclk`.

Note: Please check the [ESP-IDF documentation](#) for more details on the I2S peripheral for each ESP32 chip.

I2S Configuration

Master / Slave Mode In **Master mode** (default), the device generates the clock signal `sck` and word select signal on `ws`.

In **Slave mode**, the device listens on attached pins for the clock signal and word select driven by an external master. The role is selected via the `role` parameter of `begin()` (default `I2S_ROLE_MASTER`).

I2S Port By default, the I2S driver automatically selects an available I2S controller. Applications that need a specific controller, such as dual-I2S audio designs, can select it before initialization using `setPort()` or the `I2SClass` constructor.

Operation Modes Setting the operation mode is done with function `begin` and is set by function parameter `mode`.

•I2S_MODE_STD

In standard mode, there are always two sound channels, i.e., the left and right channels, which are called “slots”. These slots support 8/16/24/32-bit width sample data. The communication format for the slots follows the Philips standard.

•I2S_MODE_TDM

In Time Division Multiplexing mode (TDM), the number of sound channels is variable, and the width of each channel is fixed.

•I2S_MODE_PDM_TX

PDM (Pulse-density Modulation) mode for the TX channel can convert PCM data into PDM format which always has left and right slots. PDM TX is only supported on I2S0 and it only supports 16-bit width sample data. It needs at least a CLK pin for clock signal and a DOUT pin for data signal.

•I2S_MODE_PDM_RX

PDM (Pulse-density Modulation) mode for RX channel can receive PDM-format data and convert the data into PCM format. PDM RX is only supported on I2S0, and it only supports 16-bit width sample data. PDM RX needs at least a CLK pin for clock signal and a DIN pin for data signal.

Simplex / Duplex Mode Due to the different clock sources the PDM modes are always in **Simplex** mode, using only one data pin.

The STD and TDM modes operate in the **Duplex** mode, using two separate data pins:

- Output pin `dout` for function parameter
- Input pin `din` for function parameter

In this mode, the driver is able to read and write simultaneously on each line and is suitable for applications like walkie-talkie or phone.

Data Bit Width This is the number of bits in a channel sample. The data bit width is set by function parameter `bits_cfg`. The current supported values are:

- `I2S_DATA_BIT_WIDTH_8BIT` (see note below)
- `I2S_DATA_BIT_WIDTH_16BIT`
- `I2S_DATA_BIT_WIDTH_24BIT`, requires the MCLK multiplier to be manually set to 384
- `I2S_DATA_BIT_WIDTH_32BIT`

Note: 8-bit data width on ESP32 (HW v1): The original ESP32’s I2S peripheral uses `uint16_t`-aligned DMA words for 8-bit data, with only the high 8 bits valid. The Arduino library handles this transparently: `write()` packs each `int8_t` sample into the high byte of a `uint16_t`, and `readBytes()` unpacks it back. For mono 8-bit, the existing stereo workaround is also applied (hardware runs in stereo mode, software converts). This is fully automatic and requires no special handling in application code. See the [ESP-IDF I2S STD TX Mode documentation](#) for background on the hardware behavior.

Sample Rate The sample rate is set by function parameter `rate`. It is the number of samples per second in Hz.

Slot Mode The slot mode is set by function parameter `ch`. The current supported values are:

•I2S_SLOT_MODE_MONO

I2S channel slot format mono. Transmit the same data in all slots for TX mode. Only receive the data in the first slot for RX mode.

•I2S_SLOT_MODE_STEREO

I2S channel slot format stereo. Transmit different data in different slots for TX mode. Receive the data in all slots for RX mode.

Note: On the original ESP32 (I2S hardware version 1), 8-bit and 16-bit mono modes have a known hardware limitation: the FIFO packing logic scrambles data during transmission and reception. The library works around this automatically by configuring the hardware in stereo mode and converting between mono and stereo in software.

`write()` duplicates each mono sample into both left and right slots, and `readBytes()` extracts the selected channel to produce mono output. For 8-bit, the library also handles the `uint16_t` high-byte packing required by the hardware. This is fully transparent to application code.

Slot Mask The `slot_mask` parameter selects which channel slot(s) to use. It is available in `begin()`, `configureTX()`, and `configureRX()`. When set to `-1` (default), the library chooses a sensible default based on the slot mode.

The supported values are:

- `I2S_STD_SLOT_LEFT` (1) –Use the left slot only.
- `I2S_STD_SLOT_RIGHT` (2) –Use the right slot only.
- `I2S_STD_SLOT_BOTH` (3) –Use both left and right slots.

TX behavior (mono mode):

- `LEFT` –Transmit mono data on the left wire slot; right is silent.
- `RIGHT` –Transmit mono data on the right wire slot; left is silent.
- `BOTH` –Transmit mono data on both wire slots. Useful for driving a stereo DAC from a single-channel source.

TX behavior (stereo mode):

- `LEFT` –Transmit only the left-channel data from the buffer on the left wire slot; right is silent.
- `RIGHT` –Transmit only the right-channel data from the buffer on the right wire slot; left is silent.
- `BOTH` –Transmit interleaved stereo data normally.

RX behavior (mono mode):

- `LEFT` –Receive the left channel only.
- `RIGHT` –Receive the right channel only.
- `BOTH` –On the original ESP32 (HW v1), averages the left and right channels. On other targets, causes sample duplication in the DMA buffer (not recommended).

RX behavior (stereo mode):

The library always forces `slot_mask` to `BOTH` for stereo RX, regardless of the value passed by the user. This is because the ESP-IDF does not reliably ignore `slot_mask` in stereo RX mode on all targets —some hardware duplicates the selected channel instead of delivering normal interleaved data. The library overrides the mask to guarantee correct interleaved `[L, R, L, R, ...]` output.

Note: On the original ESP32 (HW v1) with mono 8-bit or 16-bit mode, `slot_mask` controls which channel the software workaround extracts: `LEFT` extracts the left channel, `RIGHT` extracts the right channel, and `BOTH` averages the two channels. This can be changed at runtime via `configureRX()`.

Arduino-ESP32 I2S API

Initialization and deinitialization Before initialization, set which pins you want to use.

I2SClass Create an I2S object. The optional `port` parameter selects the I2S controller. If it is not specified, the driver uses `I2S_NUM_AUTO`.

```
I2SClass I2S(i2s_port_t port=I2S_NUM_AUTO)
```

setPort Set the I2S controller to use. This function must be called before `begin()`.

```
bool setPort(i2s_port_t port)
```

Parameters:

- [in] port is the I2S controller, for example I2S_NUM_0, another SoC-specific port such as I2S_NUM_1, or I2S_NUM_AUTO.

This function will return `true` on success or `false` in case of failure.

getPort Get the configured I2S controller. After `begin()`, this returns the controller allocated by the driver. Before `begin()`, this returns the configured controller value, such as `I2S_NUM_AUTO`.

```
i2s_port_t getPort()
```

begin Initialize the I2S driver. Before calling `begin()`, choose which pins you want to use with `setPins()` or the PDM pin-setup functions.

```
bool begin(i2s_mode_t mode, uint32_t rate, i2s_data_bit_width_t bits_cfg, i2s_slot_
↪mode_t ch, int8_t slot_mask=-1, i2s_role_t role=I2S_ROLE_MASTER)
```

Parameters:

- [in] mode one of above mentioned operation mode, for example `I2S_MODE_STD`.
- [in] rate is the sampling rate in Hz, for example 16000.
- [in] bits_cfg is the number of bits in a channel sample, for example `I2S_DATA_BIT_WIDTH_16BIT`.
- [in] ch is the slot mode, for example `I2S_SLOT_MODE_STEREO`.
- [in] slot_mask selects which slot(s) to use (see [Slot Mask](#) section).
This parameter is optional and defaults to -1 (library default: LEFT for mono TX/RX, BOTH for stereo).
For stereo RX, the library always forces BOTH regardless of this value.
- [in] role is the I2S role. Use `I2S_ROLE_MASTER` (default) to generate clock and word-select signals, or `I2S_ROLE_SLAVE` to receive them from an external master.

This function will return `true` on success or `false` in case of failure.

When failed, an error message will be printed if the correct log level is set.

end Performs safe deinitialization - free buffers, destroy task, end driver operation, etc.

```
void end()
```

Pin setup The function to set the pins will depend on the operation mode.

setPins Set the pins for the I2S interface when using the standard or TDM mode.

```
void setPins(int8_t bclk, int8_t ws, int8_t dout, int8_t din=-1, int8_t mclk=-1)
```

Parameters:

- [in] bclk is the bit clock pin.
- [in] ws is the word select pin.
- [in] dout is the data output pin. Can be set to -1 if not used.
- [in] din is the data input pin. This parameter is optional and defaults to -1 (not used).
- [in] mclk is the master clock pin. This parameter is optional and defaults to -1 (not used).

setPinsPdmTx Set the pins for the I2S interface when using the PDM TX mode.

```
void setPinsPdmTx(int8_t clk, int8_t dout0, int8_t dout1=-1)
```

Parameters:

- [in] clk is the clock pin.

- [in] `dout0` is the data output pin 0.
- [in] `dout1` is the data output pin 1. This parameter is optional and defaults to `-1` (not used).

setPinsPdmRx Set the pins for the I2S interface when using the PDM RX mode.

```
void setPinsPdmRx(int8_t clk, int8_t din0, int8_t din1=-1, int8_t din2=-1, int8_t
↳din3=-1)
```

Parameters:

- [in] `clk` is the clock pin.
- [in] `din0` is the data input pin 0.
- [in] `din1` is the data input pin 1. This parameter is optional and defaults to `-1` (not used).
- [in] `din2` is the data input pin 2. This parameter is optional and defaults to `-1` (not used).
- [in] `din3` is the data input pin 3. This parameter is optional and defaults to `-1` (not used).

setInverted Set which pins have inverted logic when using the standard or TDM mode. Data pins cannot be inverted.

```
void setInverted(bool bclk, bool ws, bool mclk=false)
```

Parameters:

- [in] `bclk` true if the bit clock pin is inverted. False otherwise.
- [in] `ws` true if the word select pin is inverted. False otherwise.
- [in] `mclk` true if the master clock pin is inverted. False otherwise. This parameter is optional and defaults to `false`.

setInvertedPdm Set which pins have inverted logic when using the PDM mode. Data pins cannot be inverted.

```
void setInvertedPdm(bool clk)
```

Parameters:

- [in] `clk` true if the clock pin is inverted. False otherwise.

I2S Configuration The I2S configuration can be changed during operation.

configureTX Configure the I2S TX channel.

```
bool configureTX(uint32_t rate, i2s_data_bit_width_t bits_cfg, i2s_slot_mode_t ch,
↳int8_t slot_mask=-1)
```

Parameters:

- [in] `rate` is the sampling rate in Hz, for example 16000.
- [in] `bits_cfg` is the number of bits in a channel sample, for example `I2S_DATA_BIT_WIDTH_16BIT`.
- [in] `ch` is the slot mode, for example `I2S_SLOT_MODE_STEREO`.
- [in] **slot_mask** selects which slot(s) to use for TX (see [Slot Mask](#) section).
This parameter is optional and defaults to `-1` (library default).

This function will return `true` on success or `false` in case of failure.

When failed, an error message will be printed if the correct log level is set.

configureRX Configure the I2S RX channel. This function can change the sample rate, bit width, slot mode, RX transform, and slot mask at runtime without calling `end()`/`begin()`. If only the `slot_mask` changes (rate, bits, and channel mode remain the same), the library performs a lightweight slot-only reconfiguration.

```
bool configureRX(uint32_t rate, i2s_data_bit_width_t bits_cfg, i2s_slot_mode_t ch,
↳ i2s_rx_transform_t transform=I2S_RX_TRANSFORM_NONE, int8_t slot_mask=-1)
```

Parameters:

- [in] `rate` is the sampling rate in Hz, for example 16000.
- [in] `bits_cfg` is the number of bits in a channel sample, for example `I2S_DATA_BIT_WIDTH_16BIT`.
- [in] `ch` is the slot mode, for example `I2S_SLOT_MODE_STEREO`.
- [in] **transform is the transform mode, for example `I2S_RX_TRANSFORM_NONE`.**
This can be used to apply a transformation/conversion to the received data. The supported values are: `I2S_RX_TRANSFORM_NONE` (no transformation), `I2S_RX_TRANSFORM_32_TO_16` (convert from 32 bits of data width to 16 bits) and `I2S_RX_TRANSFORM_16_STEREO_TO_MONO` (convert from stereo to mono when using 16 bits of data width). When using `I2S_RX_TRANSFORM_16_STEREO_TO_MONO`, the channel extracted depends on the configured `slot_mask`: `LEFT` extracts the left channel, `RIGHT` extracts the right channel, and `BOTH` averages both channels.
- [in] **slot_mask is the slot mask for RX channel selection (see [Slot Mask](#) section).**
This parameter is optional and defaults to `-1` (use current setting). In mono mode, this selects which channel to receive (`LEFT`, `RIGHT`, or `BOTH`). In stereo mode, this parameter is ignored—the library always uses `BOTH`.

This function will return `true` on success or `false` in case of failure.

When failed, an error message will be printed if the correct log level is set.

txChan Get the TX channel handler pointer.

```
i2s_chan_handle_t txChan()
```

txSampleRate Get the TX sample rate.

```
uint32_t txSampleRate()
```

txDataWidth Get the TX data width (8, 16 or 32 bits).

```
i2s_data_bit_width_t txDataWidth()
```

txSlotMode Get the TX slot mode (stereo or mono).

```
i2s_slot_mode_t txSlotMode()
```

rxChan Get the RX channel handler pointer.

```
i2s_chan_handle_t rxChan()
```

rxSampleRate Get the RX sample rate.

```
uint32_t rxSampleRate()
```

rxDataWidth Get the RX data width (8, 16 or 32 bits).

```
i2s_data_bit_width_t rxDataWidth()
```

rxSlotMode Get the RX slot mode (stereo or mono).

```
i2s_slot_mode_t rxSlotMode()
```

I/O Operations

readBytes Read a certain amount of data bytes from the I2S interface.

```
size_t readBytes(char *buffer, size_t size)
```

Parameters:

- [in] `buffer` is the buffer to store the read data. The buffer must be at least `size` bytes long.
- [in] `size` is the number of bytes to read.

This function will return the number of bytes read.

read Read the next available byte from the I2S interface.

```
int read()
```

This function will return the next available byte or `-1` if no data is available or an error occurred.

write PCM data must be written in **sample-aligned** chunks. The minimum number of bytes per call depends on the data bit width configured in `begin()` or `configureTX()`:

- `I2S_DATA_BIT_WIDTH_8BIT` —1 byte per sample
- `I2S_DATA_BIT_WIDTH_16BIT` —2 bytes per sample
- `I2S_DATA_BIT_WIDTH_32BIT` —4 bytes per sample

For stereo, samples must reach the driver in **interleaved order** (left, then right). Each `write()` call must contain at least one full sample (2 bytes for 16-bit), but a complete stereo frame may be sent in one call or as separate per-channel calls.

Writes smaller than one sample are ignored: the function returns 0 and does not set `lastError()`. Enable debug logging to see a message when this happens.

Per-channel writes —clear when learning or generating one sample at a time:

```
// 16-bit stereo — left then right
int16_t left = ...;
int16_t right = ...;
i2s.write(&left, sizeof(left));
i2s.write(&right, sizeof(right));
```

Frame buffer —preferred for streaming or bulk PCM:

```
// 16-bit stereo — one interleaved frame (4 bytes)
int16_t frame[2] = {left, right};
i2s.write(frame, sizeof(frame));
```

There are two versions of the write function:

The first version writes a buffer of PCM bytes to the I2S interface.

```
size_t write(const void *buffer, size_t size)
```

Parameters:

- [in] `buffer` is the buffer containing the data to be written.
- [in] `size` is the number of bytes to write from the buffer. Must be at least the per-sample minimum above. For streaming, `size` should be a multiple of the full frame size (sample size × number of active channels).

This function will return the number of bytes written, or 0 if the buffer is empty, the TX channel is not initialized, or `size` is smaller than one sample.

The second version is inherited from `Print` and forwards a single byte to the buffer version above. For bit widths greater than 8, a one-byte write is below the sample minimum and is therefore ignored (returns 0 without setting an error).

```
size_t write(uint8_t d)
```

Parameters:

- [in] `d` is the byte to be written.

This function returns 1 only when the byte is accepted (8-bit mode). For 16-bit and wider formats it returns 0 because a single byte is not a complete sample.

available Get if there is data available to be read.

```
int available()
```

This function will return `I2S_READ_CHUNK_SIZE` if there is data available to be read or -1 if not.

peek Get the next available byte from the I2S interface without removing it from the buffer. Currently not implemented.

```
int peek()
```

This function will currently always return -1.

lastError Get the last error code for an I/O operation on the I2S interface.

```
int lastError()
```

recordWAV Record a short PCM WAV to memory with the current RX settings. Returns a buffer that must be freed by the user.

```
uint8_t * recordWAV(size_t rec_seconds, size_t * out_size)
```

Parameters:

- [in] `rec_seconds` is the number of seconds to record.
- [out] `out_size` is the size of the returned buffer in bytes.

This function will return a pointer to the buffer containing the recorded WAV data or `NULL` if an error occurred.

playWAV Play a PCM WAV from memory with the current TX settings.

```
void playWAV(uint8_t * data, size_t len)
```

Parameters:

- [in] data is the buffer containing the WAV data.
- [in] len is the size of the buffer in bytes.

playMP3 Play a MP3 from memory with the current TX settings.

```
bool playMP3(uint8_t *src, size_t src_len)
```

Parameters:

- [in] src is the buffer containing the MP3 data.
- [in] src_len is the size of the buffer in bytes.

This function will return `true` on success or `false` in case of failure.

When failed, an error message will be printed if the correct log level is set.

Sample code

Master mode (default)

```
#include <ESP_I2S.h>

const int buff_size = 128;
int available_bytes, read_bytes;
uint8_t buffer[buff_size];
I2SClass I2S;

void setup() {
    I2S.setPins(5, 25, 26, 35, 0); //SCK, WS, SDOUT, SDIN, MCLK
    I2S.begin(I2S_MODE_STD, 16000, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_STEREO);
    I2S.read();
    available_bytes = I2S.available();
    if(available_bytes < buff_size) {
        read_bytes = I2S.readBytes(buffer, available_bytes);
    } else {
        read_bytes = I2S.readBytes(buffer, buff_size);
    }
    I2S.write(buffer, read_bytes);
    I2S.end();
}

void loop() {}
```

Slave mode

```
#include <ESP_I2S.h>

const int buff_size = 128;
uint8_t buffer[buff_size];
I2SClass I2S;

void setup() {
    I2S.setPins(5, 25, -1, 35); //SCK, WS, SDOUT (not used), SDIN
    I2S.begin(I2S_MODE_STD, 16000, I2S_DATA_BIT_WIDTH_16BIT, I2S_SLOT_MODE_MONO, -1, I2S_ROLE_SLAVE);
    size_t read_bytes = I2S.readBytes((char *)buffer, buff_size);
    // process received data...
    I2S.end();
}

void loop() {}
```

2.3.12 ESP Insights

About

ESP Insights is a remote diagnostics solution that allows users to remotely monitor the health of Espressif devices in the field.

Developers normally prefer debugging issues by physically probing them using gdb or observing the logs. This surely helps debug issues, but there are often cases wherein issues are seen only in specific environments under specific conditions. Even things like casings and placement of the product can affect the behavior. A few examples are

- Wi-Fi disconnections for a smart switch concealed in a wall.
- Smart speakers crashing during some specific usage pattern.
- Appliance frequently rebooting due to power supply issues.

Additional information about ESP Insights can be found [here](#).

ESP Insights Agent API

Insights.begin This initializes the ESP Insights agent.

```
bool begin(const char *auth_key, const char *node_id = NULL, uint32_t log_type = 0xFFFFFFFF, bool alloc_ext_ram = false);
```

- `auth_key` Auth key generated using Insights dashboard
- `log_type` Type of logs to be captured (value can be a mask of `ESP_DIAG_LOG_TYPE_ERROR`, `ESP_DIAG_LOG_TYPE_WARNING` and `ESP_DIAG_LOG_TYPE_EVENT`)

This function will return

1. `true` : On success
2. `false` in case of failure

Insights.send Read insights data from buffers and send it to the cloud. Call to this function is asynchronous, it may take some time to send the data.

```
bool sendData()
```

This function will return

1. `true` : On success
2. `false` in case of failure

Insights.end Deinitialize ESP Insights.

```
void end();
```

Insights.disable Disable ESP Insights.

```
void disable();
```

ESP Insights Metrics API

metrics object of *Insights* class expose API's for using metrics.

Insights.metrics.addX Register a metric of type X, where X is one of: Bool, Int, Uint, Float, String, IPv4 or MAC

```
bool addX(const char *tag, const char *key, const char *label, const char *path);
```

- tag : Tag of metrics
- key : Unique key for the metrics
- label : Label for the metrics
- path : Hierarchical path for key, must be separated by ‘.’ for more than one level

This function will return

1. true : On success
2. false in case of failure

Insights.metrics.remove Unregister a diagnostics metrics

```
bool remove(const char *key);
```

- key : Key for the metrics

This function will return

1. true : On success
2. false in case of failure

Insights.metrics.removeAll Unregister all previously registered metrics

```
bool removeAll();
```

This function will return

1. true : On success
2. false in case of failure

Insights.metrics.setX Add metrics of type X to storage, where X is one of: Bool, Int, Uint, Float, String, IPv4 or MAC

```
bool setX(const char *key, const void val);
```

- key : Key of metrics
- val : Value of metrics

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

Insights.metrics.dumpHeap Dumps the heap metrics and prints them to the console. This API collects and reports metrics value at any give point in time.

```
bool dumpHeap();
```

This function will return

1. true : On success
2. false in case of failure

Insights.metrics.dumpWiFi Dumps the Wi-Fi metrics and prints them to the console. This API can be used to collect Wi-Fi metrics at any given point in time.

```
bool dumpWiFi();
```

This function will return

1. true : On success
2. false in case of failure

Insights.metrics.setHeapPeriod Reset the periodic interval By default, heap metrics are collected every 30 seconds, this function can be used to change the interval. If the interval is set to 0, heap metrics collection disabled.

```
void setHeapPeriod(uint32_t period);
```

- period : Period interval in seconds

Insights.metrics.setWiFiPeriod Reset the periodic interval By default, Wi-Fi metrics are collected every 30 seconds, this function can be used to change the interval. If the interval is set to 0, Wi-Fi metrics collection disabled.

```
void setHeapPeriod(uint32_t period);
```

- period : Period interval in seconds

ESP Insights Variables API

variables object of *Insights* class expose API's for using variables.

Insights.variables.addX Register a variable of type X, where X is one of: Bool, Int, Uint, Float, String, IPv4 or MAC

```
bool addX(const char *tag, const char *key, const char *label, const char *path);
```

- tag : Tag of variable
- key : Unique key for the variable
- label : Label for the variable
- path : Hierarchical path for key, must be separated by '.' for more than one level

This function will return

1. true : On success
2. false in case of failure

Insights.variables.remove Unregister a diagnostics variable

```
bool remove(const char *key);
```

- key : Key for the variable

This function will return

1. true : On success
2. false in case of failure

Insights.variables.removeAll Unregister all previously registered variables

```
bool unregisterAll();
```

This function will return

1. true : On success
2. false in case of failure

Insights.variables.setX Add variable of type X to storage, where X is one of: Bool, Int, Uint, Float, String, IPv4 or MAC

```
bool setX(const char *key, const void val);
```

- key : Key of metrics
- val : Value of metrics

This function will return

1. true : On success
2. false in case of failure

Example

To get started with Insights, you can try:

```
#include <Arduino.h>
#include "Insights.h"
#include "WiFi.h"

const char insights_auth_key[] = "<ENTER YOUR AUTH KEY>";

#define WIFI_SSID      "<ENTER YOUR SSID>"
#define WIFI_PASSPHRASE "<ENTER YOUR PASSWORD>"

void setup() {
    Serial.begin(115200);
    WiFi.mode(WIFI_STA);
    WiFi.begin(WIFI_SSID, WIFI_PASSPHRASE);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("");
    Serial.println("WiFi connected");
    if (!Insights.begin(insights_auth_key)) {
        return;
    }
    Serial.println("=====");
    Serial.printf("ESP Insights enabled Node ID %s\n", Insights.nodeID());
    Serial.println("=====");
}

void loop() {
    delay(1000);
}
```

2.3.13 LED Control (LEDC)

About

The LED control (LEDC) peripheral is primarily designed to control the intensity of LEDs, although it can also be used to generate PWM signals for other purposes.

ESP32 SoCs has from 6 to 16 channels (varies on socs, see table below) which can generate independent waveforms, that can be used for example to drive RGB LED devices.

ESP32 SoC	Number of LEDC channels
ESP32	16
ESP32-S2	8
ESP32-S3	8
ESP32-C3	6
ESP32-C6	6
ESP32-H2	6

Arduino-ESP32 LEDC API

ledcSetClockSource This function is used to set the LEDC peripheral clock source. Must be called before any LEDC channel is used. The default clock source is XTAL clock (LEDC_USE_XTAL_CLK) if supported by the SoC, otherwise it is AUTO clock (LEDC_AUTO_CLK).

```
bool ledcSetClockSource(ledc_clk_cfg_t source);
```

- `source` select the clock source for LEDC peripheral.
 - LEDC_APB_CLK - APB clock.
 - LEDC_REF_CLK - REF clock.

This function will return `true` if setting the clock source is successful, otherwise it will return `false`.

ledcGetClockSource This function is used to get the LEDC peripheral clock source.

```
ledc_clk_cfg_t ledcGetClockSource(void);
```

This function will return the clock source for the LEDC peripheral.

ledcAttach This function is used to setup LEDC pin with given frequency and resolution. LEDC channel will be selected automatically.

```
bool ledcAttach(uint8_t pin, uint32_t freq, uint8_t resolution);
```

- `pin` select LEDC pin.
- `freq` select frequency of pwm.
- `resolution` select resolution for LEDC channel.
 - range is 1-14 bits (1-20 bits for ESP32).

This function will return `true` if configuration is successful. If `false` is returned, error occurs and LEDC channel was not configured.

ledcAttachChannel This function is used to setup LEDC pin with given frequency, resolution and channel. Attaching multiple pins to the same channel will make them share the same duty cycle. Given frequency, resolution will be ignored if channel is already configured.

```
bool ledcAttachChannel(uint8_t pin, uint32_t freq, uint8_t resolution, int8_t channel);
```

- `pin` select LEDC pin.
- `freq` select frequency of pwm.
- `resolution` select resolution for LEDC channel.
 - range is 1-14 bits (1-20 bits for ESP32).
- `channel` select LEDC channel.

This function will return `true` if configuration is successful. If `false` is returned, error occurs and LEDC channel was not configured.

ledcWrite This function is used to set duty for the LEDC pin.

```
bool ledcWrite(uint8_t pin, uint32_t duty);
```

- `pin` select LEDC pin.
- `duty` select duty to be set for selected LEDC pin.

This function will return `true` if setting duty is successful. If `false` is returned, error occurs and duty was not set.

ledcWriteChannel This function is used to set duty for the LEDC channel.

```
bool ledcWriteChannel(uint8_t channel, uint32_t duty);
```

- `channel` select LEDC channel.
- `duty` select duty to be set for selected LEDC channel.

This function will return `true` if setting duty is successful. If `false` is returned, error occurs and duty was not set.

ledcRead This function is used to get configured duty for the LEDC pin.

```
uint32_t ledcRead(uint8_t pin);
```

- `pin` select LEDC pin to read the configured LEDC duty.

This function will return `duty` set for selected LEDC pin.

ledcReadFreq This function is used to get configured frequency for the LEDC channel.

```
uint32_t ledcReadFreq(uint8_t pin);
```

- `pin` select LEDC pin to read the configured frequency.

This function will return `frequency` configured for selected LEDC pin.

ledcWriteTone This function is used to setup the LEDC pin to 50 % PWM tone on selected frequency.

```
uint32_t ledcWriteTone(uint8_t pin, uint32_t freq);
```

- `pin` select LEDC pin.
- `freq` select frequency of pwm signal. If frequency is 0, duty will be set to 0.

This function will return `frequency` set for LEDC pin. If 0 is returned, error occurs and LEDC pin was not configured.

ledcWriteNote This function is used to setup the LEDC pin to specific note.

```
uint32_t ledcWriteNote(uint8_t pin, note_t note, uint8_t octave);
```

- `pin` select LEDC pin.

- `note` select note to be set.

NOTE_C	NOTE_Cs	NOTE_D	NOTE_Eb	NOTE_E	NOTE_F
NOTE_Fs	NOTE_G	NOTE_Gs	NOTE_A	NOTE_Bb	NOTE_B

- `octave` select octave for note.

This function will return `frequency` configured for the LEDC pin according to note and octave inputs. If 0 is returned, error occurs and the LEDC channel was not configured.

ledcDetach This function is used to detach the pin from LEDC.

```
bool ledcDetach(uint8_t pin);
```

- `pin` select LEDC pin.

This function returns `true` if detaching was successful. If `false` is returned, an error occurred and the pin was not detached.

ledcChangeFrequency This function is used to set frequency for the LEDC pin.

```
uint32_t ledcChangeFrequency(uint8_t pin, uint32_t freq, uint8_t resolution);
```

- `pin` select LEDC pin.
- `freq` select frequency of pwm.
- `resolution` select resolution for LEDC channel.
 - range is 1-14 bits (1-20 bits for ESP32).

This function will return `frequency` configured for the LEDC channel. If 0 is returned, error occurs and the LEDC channel frequency was not set.

ledcOutputInvert This function is used to set inverting output for the LEDC pin.

```
bool ledcOutputInvert(uint8_t pin, bool out_invert);
```

- `pin` select LEDC pin.
- `out_invert` select, if output should be inverted (`true` = inverting output).

This function returns `true` if setting inverting output was successful. If `false` is returned, an error occurred and the inverting output was not set.

ledcFade This function is used to setup and start fade for the LEDC pin.

```
bool ledcFade(uint8_t pin, uint32_t start_duty, uint32_t target_duty, int max_fade_time_ms);
```

- `pin` select LEDC pin.
- `start_duty` select starting duty of fade.
- `target_duty` select target duty of fade.
- `max_fade_time_ms` select maximum time for fade.

This function will return `true` if configuration is successful. If `false` is returned, error occurs and LEDC fade was not configured / started.

ledcFadeWithInterrupt This function is used to setup and start fade for the LEDC pin with interrupt.

```
bool ledcFadeWithInterrupt(uint8_t pin, uint32_t start_duty, uint32_t target_duty,
↳ int max_fade_time_ms, void (*userFunc) (void));
```

- pin select LEDC pin.
- start_duty select starting duty of fade.
- target_duty select target duty of fade.
- max_fade_time_ms select maximum time for fade.
- userFunc function to be called when interrupt is triggered.

This function will return `true` if configuration is successful and fade start. If `false` is returned, error occurs and LEDC fade was not configured / started.

ledcFadeWithInterruptArg This function is used to setup and start fade for the LEDC pin with interrupt using arguments.

```
bool ledcFadeWithInterruptArg(uint8_t pin, uint32_t start_duty, uint32_t target_
↳ duty, int max_fade_time_ms, void (*userFunc) (void*), void * arg);
```

- pin select LEDC pin.
- start_duty select starting duty of fade.
- target_duty select target duty of fade.
- max_fade_time_ms select maximum time for fade.
- userFunc function to be called when interrupt is triggered.
- arg pointer to the interrupt arguments.

This function will return `true` if configuration is successful and fade start. If `false` is returned, error occurs and LEDC fade was not configured / started.

analogWrite This function is used to write an analog value (PWM wave) on the pin. It is compatible with Arduinos `analogWrite` function.

```
void analogWrite(uint8_t pin, int value);
```

- pin select the GPIO pin.
- value select the duty cycle of pwm.
 - range is from 0 (always off) to 255 (always on).

analogWriteResolution This function is used to set resolution for selected `analogWrite` pin.

```
void analogWriteResolution(uint8_t pin, uint8_t resolution);
```

- pin select the GPIO pin.
- resolution select resolution for analog channel.

analogWriteFrequency This function is used to set frequency for selected `analogWrite` pin.

```
void analogWriteFrequency(uint8_t pin, uint32_t freq);
```

- pin select the GPIO pin.
- freq select frequency of pwm.

Example Applications LEDC fade example:

```

/* LEDC Fade Arduino Example

   This example code is in the Public Domain (or CC0 licensed, at your option.)
   Unless required by applicable law or agreed to in writing, this
   software is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
   CONDITIONS OF ANY KIND, either express or implied.
*/

#include <Arduino.h>

// use 12 bit precision for LEDC timer
#define LEDC_TIMER_12_BIT 12

// use 5000 Hz as a LEDC base frequency
#define LEDC_BASE_FREQ 5000

// fade LED PIN (replace with LED_BUILTIN constant for built-in LED)
#define LED_PIN 4

// define starting duty, target duty and maximum fade time
#define LEDC_START_DUTY (0)
#define LEDC_TARGET_DUTY (4095)
#define LEDC_FADE_TIME (3000)

bool fade_ended = false; // status of LED fade
bool fade_in = true;

void ARDUINO_ISR_ATTR LED_FADE_ISR() {
    fade_ended = true;
}

void setup() {
    // Initialize serial communication at 115200 bits per second:
    Serial.begin(115200);

    // Setup timer with given frequency, resolution and attach it to a led pin with
    ↪ auto-selected channel
    ledcAttach(LED_PIN, LEDC_BASE_FREQ, LEDC_TIMER_12_BIT);

    // Setup and start fade on led (duty from 0 to 4095)
    ledcFade(LED_PIN, LEDC_START_DUTY, LEDC_TARGET_DUTY, LEDC_FADE_TIME);
    Serial.println("LED Fade on started.");

    // Wait for fade to end
    delay(LEDC_FADE_TIME);

    // Setup and start fade off led and use ISR (duty from 4095 to 0)
    ledcFadeWithInterrupt(LED_PIN, LEDC_TARGET_DUTY, LEDC_START_DUTY, LEDC_FADE_TIME,
    ↪ LED_FADE_ISR);
    Serial.println("LED Fade off started.");
}

void loop() {
    // Check if fade_ended flag was set to true in ISR
    if (fade_ended) {
        Serial.println("LED fade ended");
        fade_ended = false;

        // Check what fade should be started next
        if (fade_in) {
            ledcFadeWithInterrupt(LED_PIN, LEDC_START_DUTY, LEDC_TARGET_DUTY, LEDC_FADE_
            ↪ TIME, LED_FADE_ISR);

```

(continues on next page)

(continued from previous page)

```

    Serial.println("LED Fade in started.");
    fade_in = false;
  } else {
    ledcFadeWithInterrupt(LED_PIN, LEDC_TARGET_DUTY, LEDC_START_DUTY, LEDC_FADE_
→TIME, LED_FADE_ISR);
    Serial.println("LED Fade out started.");
    fade_in = true;
  }
}
}

```

LEDC software fade example:

```

/*
  LEDC Software Fade

  This example shows how to software fade LED
  using the ledcWrite function.

  Code adapted from original Arduino Fade example:
  https://www.arduino.cc/en/Tutorial/Fade

  This example code is in the public domain.
  */

#include <Arduino.h>

// use 12 bit precision for LEDC timer
#define LEDC_TIMER_12_BIT 12

// use 5000 Hz as a LEDC base frequency
#define LEDC_BASE_FREQ 5000

// fade LED PIN (replace with LED_BUILTIN constant for built-in LED)
#define LED_PIN 5

int brightness = 0; // how bright the LED is
int fadeAmount = 5; // how many points to fade the LED by

// Arduino like analogWrite
// value has to be between 0 and valueMax
void ledcAnalogWrite(uint8_t pin, uint32_t value, uint32_t valueMax = 255) {
  // calculate duty, 4095 from 2 ^ 12 - 1
  uint32_t duty = (4095 / valueMax) * min(value, valueMax);

  // write duty to LEDC
  ledcWrite(pin, duty);
}

void setup() {
  // Setup timer and attach timer to a led pin
  ledcAttach(LED_PIN, LEDC_BASE_FREQ, LEDC_TIMER_12_BIT);
}

void loop() {
  // set the brightness on LEDC channel 0
  ledcAnalogWrite(LED_PIN, brightness);

  // change the brightness for next time through the loop:
  brightness = brightness + fadeAmount;

```

(continues on next page)

(continued from previous page)

```

// reverse the direction of the fading at the ends of the fade:
if (brightness <= 0 || brightness >= 255) {
  fadeAmount = -fadeAmount;
}
// wait for 30 milliseconds to see the dimming effect
delay(30);
}

```

LEDC Write RGB example:

```

/*
  ledcWrite_RGB.ino
  Runs through the full 255 color spectrum for an rgb led
  Demonstrate ledcWrite functionality for driving leds with PWM on ESP32

  This example code is in the public domain.

  Some basic modifications were made by vseven, mostly commenting.
  */

#include <Arduino.h>

// Set up the rgb led names
uint8_t ledR = 0;
uint8_t ledG = 2;
uint8_t ledB = 4;

const boolean invert = true; // set true if common anode, false if common cathode

uint8_t color = 0;           // a value from 0 to 255 representing the hue
uint32_t R, G, B;            // the Red Green and Blue color components
uint8_t brightness = 255;    // 255 is maximum brightness, but can be changed.
// Might need 256 for common anode to fully turn off.

// the setup routine runs once when you press reset:
void setup() {
  Serial.begin(115200);
  delay(10);

  // Initialize pins as LEDC channels
  // resolution 1-16 bits, freq limits depend on resolution, channel is
  // automatically selected
  ledcAttach(ledR, 12000, 8); // 12 kHz PWM, 8-bit resolution
  ledcAttach(ledG, 12000, 8);
  ledcAttach(ledB, 12000, 8);
}

// void loop runs over and over again
void loop() {
  Serial.println("Send all LEDs a 255 and wait 2 seconds.");
  // If your RGB LED turns off instead of on here you should check if the LED is
  // common anode or cathode.
  // If it doesn't fully turn off and is common anode try using 256.
  ledcWrite(ledR, 255);
  ledcWrite(ledG, 255);
  ledcWrite(ledB, 255);
  delay(2000);
  Serial.println("Send all LEDs a 0 and wait 2 seconds.");
  ledcWrite(ledR, 0);
  ledcWrite(ledG, 0);
  ledcWrite(ledB, 0);
}

```

(continues on next page)

(continued from previous page)

```

delay(2000);

Serial.println("Starting color fade loop.");

for (color = 0; color < 255; color++) { // Slew through the color spectrum

    hueToRGB(color, brightness); // call function to convert hue to RGB

    // write the RGB values to the pins
    ledcWrite(ledR, R); // write red component to channel 1, etc.
    ledcWrite(ledG, G);
    ledcWrite(ledB, B);

    delay(100); // full cycle of rgb over 256 colors takes 26 seconds
}
}

// Courtesy http://www.instructables.com/id/How-to-Use-an-RGB-LED/?ALLSTEPS
// function to convert a color to its Red, Green, and Blue components.

void hueToRGB(uint8_t hue, uint8_t brightness) {
    uint16_t scaledHue = (hue * 6);
    uint8_t segment = scaledHue / 256; // segment 0 to 5 around
    // the // color wheel
    uint16_t segmentOffset = scaledHue - (segment * 256); // position within the
    // segment

    uint8_t complement = 0;
    uint16_t prev = (brightness * (255 - segmentOffset)) / 256;
    uint16_t next = (brightness * segmentOffset) / 256;

    if (invert) {
        brightness = 255 - brightness;
        complement = 255;
        prev = 255 - prev;
        next = 255 - next;
    }

    switch (segment) {
        case 0: // red
            R = brightness;
            G = next;
            B = complement;
            break;
        case 1: // yellow
            R = prev;
            G = brightness;
            B = complement;
            break;
        case 2: // green
            R = complement;
            G = brightness;
            B = next;
            break;
        case 3: // cyan
            R = complement;
            G = prev;
            B = brightness;
            break;
        case 4: // blue

```

(continues on next page)

(continued from previous page)

```

    R = next;
    G = complement;
    B = brightness;
    break;
case 5: // magenta
default:
    R = brightness;
    G = complement;
    B = prev;
    break;
}
}

```

2.3.14 Network API

About

The Network API provides a unified, object-oriented interface for managing multiple network interfaces on ESP32 devices. The Network Manager acts as a central coordinator that manages Wi-Fi (Station and Access Point modes), Ethernet, and PPP interfaces, providing a consistent API regardless of the underlying network technology.

Key Features:

- **Unified Interface Management:** Single API for all network types (Wi-Fi, Ethernet, PPP)
- **Event-Driven Architecture:** Centralized event handling through NetworkEvents
- **Multiple Interface Support:** Simultaneous operation of multiple network interfaces
- **Default Interface Selection:** Automatic or manual selection of the default network interface
- **IPv4 and IPv6 Support:** Full support for both IP protocol versions
- **Network Communication Classes:** Unified Client, Server, and UDP classes that work with any interface

Architecture

The Network library follows a tree-like hierarchical structure:

```

NetworkManager (NetworkEvents)
├── NetworkInterface (Base Class)
│   ├── Wi-Fi
│   │   ├── STAClass WiFi.STA (Station Mode)
│   │   └── APClass WiFi.AP (Access Point Mode)
│   ├── Ethernet
│   │   └── ETHClass ETH
│   └── PPP
│       └── PPPClass PPP
└── Network Communication
    ├── NetworkClient
    ├── NetworkServer
    └── NetworkUdp

```

Class Hierarchy:

- **NetworkManager:** Extends `NetworkEvents` and `Printable`. Manages all network interfaces and provides global network functions.
- **NetworkEvents:** Provides event callback registration and handling for all network events.

- **NetworkInterface**: Base class that all network interfaces extend. Provides common functionality for IP configuration, status checking, and network information.
- **Interface Implementations**: * STAClass: Wi-Fi Station (client) mode * APClass: Wi-Fi Access Point mode * ETHClass: Ethernet interface * PPPClass: Point-to-Point Protocol (modem/cellular)

Network Manager

The `NetworkManager` class (global instance: `Network`) is the central coordinator for all network interfaces. It extends `NetworkEvents` to provide event handling capabilities and implements `Printable` for status output.

Initialization

```
bool begin();
```

Initializes the Network Manager and the underlying ESP-IDF network interface system. This must be called before using any network interfaces. Returns `true` on success, `false` on failure.

Example:

```
#include "Network.h"

void setup() {
  Serial.begin(115200);
  Network.begin(); // Initialize Network Manager
}
```

Check if Online

```
bool isOnline();
```

Check if the device is online by verifying if any network interface (except AP) has an assigned IPv4 or global IPv6 address. Returns `true` if such interface has been found, `false` if no interface has assigned IP.

Example:

```
#include "Network.h"

void setup() {
  Serial.begin(115200);
  Network.begin(); // Initialize Network Manager

  // Connect to WiFi AP
  WiFi.STA.begin();
  WiFi.STA.connect("ssid", "password");

  // Wait for WiFi to connect
  while (!WiFi.STA.hasIP()) {
    delay(500);
  }

  if (Network.isOnline()) {
    Serial.println("Network is Online");
  }
}
```

Default Interface Management

```
bool setDefaultInterface(NetworkInterface &ifc);
NetworkInterface *getDefaultInterface();
```

Sets or gets the default network interface. The default interface is used for network operations when no specific interface is specified.

Example:

```
#include "Network.h"
#include "WiFi.h"

void setup() {
  Network.begin();
  WiFi.STA.begin();
  WiFi.STA.connect("ssid", "password");

  // Wait for WiFi to connect
  while (!WiFi.STA.hasIP()) {
    delay(500);
  }

  // Set WiFi as default interface
  Network.setDefaultInterface(WiFi.STA);

  // Get current default interface
  NetworkInterface *defaultIf = Network.getDefaultInterface();
  Serial.print("Default interface: ");
  Serial.println(defaultIf->impl_name());
}
```

Hostname Management

```
static const char *getHostname();
static bool setHostname(const char *hostname);
static bool hostname(const String &aHostname);
```

Gets or sets the system hostname. The hostname is used for network identification and mDNS.

Example:

```
// Set hostname
Network.setHostname("my-esp32-device");

// Get hostname
const char *hostname = Network.getHostname();
Serial.println(hostname);
```

DNS Resolution

```
int hostByName(const char *aHostname, IPAddress &aResult);
```

Resolves a hostname to an IP address using DNS. Returns 1 on success, error code on failure.

Example:

```
IPAddress ip;
if (Network.hostByName("www.example.com", ip) == 1) {
  Serial.print("Resolved IP: ");
  Serial.println(ip);
}
```

MAC Address

```
uint8_t *macAddress(uint8_t *mac);
String macAddress();
```

Gets the MAC address of the default network interface.

Example:

```
uint8_t mac[6];
Network.macAddress(mac);
Serial.print("MAC: ");
for (int i = 0; i < 6; i++) {
    if (i > 0) Serial.print(":");
    Serial.print(mac[i], HEX);
}
Serial.println();

// Or as String
String macStr = Network.macAddress();
Serial.println(macStr);
```

Network Events

The `NetworkEvents` class provides a centralized event handling system for all network-related events. `NetworkManager` extends `NetworkEvents`, so you can register event callbacks directly on the `Network` object.

Event Types Network events are defined in `arduino_event_id_t` enum:

Ethernet Events: * `ARDUINO_EVENT_ETH_START` * `ARDUINO_EVENT_ETH_STOP` * `ARDUINO_EVENT_ETH_CONNECTED` * `ARDUINO_EVENT_ETH_DISCONNECTED` * `ARDUINO_EVENT_ETH_GOT_IP` * `ARDUINO_EVENT_ETH_LOST_IP` * `ARDUINO_EVENT_ETH_GOT_IP6`

Wi-Fi Station Events: * `ARDUINO_EVENT_WIFI_STA_START` * `ARDUINO_EVENT_WIFI_STA_STOP` * `ARDUINO_EVENT_WIFI_STA_CONNECTED` * `ARDUINO_EVENT_WIFI_STA_DISCONNECTED` * `ARDUINO_EVENT_WIFI_STA_GOT_IP` * `ARDUINO_EVENT_WIFI_STA_LOST_IP` * `ARDUINO_EVENT_WIFI_STA_GOT_IP6`

Wi-Fi AP Events: * `ARDUINO_EVENT_WIFI_AP_START` * `ARDUINO_EVENT_WIFI_AP_STOP` * `ARDUINO_EVENT_WIFI_AP_STA_CONNECTED` * `ARDUINO_EVENT_WIFI_AP_STA_DISCONNECTED` * `ARDUINO_EVENT_WIFI_AP_STA_IP_ASSIGNED` * `ARDUINO_EVENT_WIFI_AP_GOT_IP6`

PPP Events: * `ARDUINO_EVENT_PPP_START` * `ARDUINO_EVENT_PPP_STOP` * `ARDUINO_EVENT_PPP_CONNECTED` * `ARDUINO_EVENT_PPP_DISCONNECTED` * `ARDUINO_EVENT_PPP_GOT_IP` * `ARDUINO_EVENT_PPP_LOST_IP` * `ARDUINO_EVENT_PPP_GOT_IP6`

Registering Event Callbacks Three types of callback functions are supported:

1. Simple Event Callback (Event ID only):

```
typedef void (*NetworkEventCb)(arduino_event_id_t event);
network_event_handle_t onEvent(NetworkEventCb cbEvent, arduino_event_id_t event =
↳ ARDUINO_EVENT_MAX);
```

2. Functional Callback (Event ID and Info):

```
typedef std::function<void(arduino_event_id_t event, arduino_event_info_t info)>
↳ NetworkEventFuncCb;
network_event_handle_t onEvent(NetworkEventFuncCb cbEvent, arduino_event_id_t
↳ event = ARDUINO_EVENT_MAX);
```

3. System Callback (Event Structure):

```
typedef void (*NetworkEventSysCb)(arduino_event_t *event);
network_event_handle_t onEvent(NetworkEventSysCb cbEvent, arduino_event_id_t event
↳ = ARDUINO_EVENT_MAX);
```

Example - Simple Callback:

```
void onNetworkEvent(arduino_event_id_t event) {
    Serial.print("Network event: ");
    Serial.println(NetworkEvents::eventName(event));

    if (event == ARDUINO_EVENT_WIFI_STA_GOT_IP) {
        Serial.println("WiFi connected!");
    }
}

void setup() {
    Network.begin();
    Network.onEvent(onNetworkEvent);
}
```

Example - Functional Callback with Event Info:

```
void setup() {
    Network.begin();

    Network.onEvent([](arduino_event_id_t event, arduino_event_info_t info) {
        if (event == ARDUINO_EVENT_WIFI_STA_GOT_IP) {
            Serial.print("IP Address: ");
            Serial.println(IPAddress(info.got_ip.ip_info.ip.addr));
            Serial.print("Gateway: ");
            Serial.println(IPAddress(info.got_ip.ip_info.gw.addr));
        }
    });
}
```

Example - System Callback:

```
void onNetworkEventSys(arduino_event_t *event) {
    Serial.print("Event: ");
    Serial.println(NetworkEvents::eventName(event->event_id));

    if (event->event_id == ARDUINO_EVENT_WIFI_STA_GOT_IP) {
        IPAddress ip = IPAddress(event->event_info.got_ip.ip_info.ip.addr);
        Serial.print("Got IP: ");
        Serial.println(ip);
    }
}

void setup() {
    Network.begin();
    Network.onEvent(onNetworkEventSys);
}
```

Removing Event Callbacks

```
void removeEvent(NetworkEventCb cbEvent, arduino_event_id_t event = ARDUINO_EVENT_
↳MAX);
void removeEvent(NetworkEventFuncCb cbEvent, arduino_event_id_t event = ARDUINO_
↳EVENT_MAX);
void removeEvent(NetworkEventSysCb cbEvent, arduino_event_id_t event = ARDUINO_
↳EVENT_MAX);
void removeEvent(network_event_handle_t event_handle);
```

Remove event callbacks by function pointer or by handle (recommended).

Example:

```
network_event_handle_t eventHandle;

void setup() {
    Network.begin();
    // Register and save handle
    eventHandle = Network.onEvent(onNetworkEvent);
}

void loop() {
    // Later, remove by handle
    Network.removeEvent(eventHandle);
}
```

Event Information

```
static const char *eventName(arduino_event_id_t id);
```

Returns a human-readable name for an event ID.

Example:

```
Serial.println(NetworkEvents::eventName(ARDUINO_EVENT_WIFI_STA_GOT_IP));
// Output: "WIFI_STA_GOT_IP"
```

Network Interface Base Class

The `NetworkInterface` class is the base class for all network interfaces (Wi-Fi STA, Wi-Fi AP, Ethernet, PPP). It provides common functionality for IP configuration, status checking, and network information retrieval.

All network interfaces inherit from `NetworkInterface` and can be used polymorphically.

IP Configuration

```
bool config(
    IPAddress local_ip = (uint32_t)0x00000000,
    IPAddress gateway = (uint32_t)0x00000000,
    IPAddress subnet = (uint32_t)0x00000000,
    IPAddress dns1 = (uint32_t)0x00000000,
    IPAddress dns2 = (uint32_t)0x00000000,
    IPAddress dns3 = (uint32_t)0x00000000
);
bool dnsIP(uint8_t dns_no, IPAddress ip);
```

Configures static IP address, gateway, subnet mask, and DNS servers. For server interfaces (Wi-Fi AP), `dns1` is the DHCP lease range start and `dns2` is the DNS server.

Example:

```
#include "Network.h"
#include "WiFi.h"

void setup() {
    Network.begin();

    // Configure static IP
    IPAddress local_ip(192, 168, 1, 100);
    IPAddress gateway(192, 168, 1, 1);
    IPAddress subnet(255, 255, 255, 0);
    IPAddress dns1(8, 8, 8, 8);
    IPAddress dns2(8, 8, 4, 4);
```

(continues on next page)

(continued from previous page)

```
WiFi.STA.begin();
WiFi.STA.config(local_ip, gateway, subnet, dns1, dns2);
WiFi.STA.connect("ssid", "password");
}
```

Hostname

```
const char *getHostname() const;
bool setHostname(const char *hostname) const;
```

Gets or sets the hostname for the specific interface.

Example:

```
WiFi.STA.setHostname("my-wifi-device");
Serial.println(WiFi.STA.getHostname());
```

Status Checking

```
bool started() const;
bool connected() const;
bool hasIP() const;
bool hasLinkLocalIPv6() const;
bool hasGlobalIPv6() const;
bool linkUp() const;
```

Check the status of the network interface.

Example:

```
if (WiFi.STA.started()) {
    Serial.println("WiFi interface started");
}

if (WiFi.STA.connected()) {
    Serial.println("WiFi connected");
}

if (WiFi.STA.hasIP()) {
    Serial.println("WiFi has IP address");
}
```

IPv6 Support

```
bool enableIPv6(bool en = true);
```

Enables or disables IPv6 support on the interface.

Example:

```
WiFi.STA.enableIPv6(true);

if (WiFi.STA.hasGlobalIPv6()) {
    IPAddress ipv6 = WiFi.STA.globalIPv6();
    Serial.print("Global IPv6: ");
    Serial.println(ipv6);
}
```

IP Address Information

```
IPAddress localIP() const;
IPAddress subnetMask() const;
IPAddress gatewayIP() const;
IPAddress dnsIP(uint8_t dns_no = 0) const;
IPAddress broadcastIP() const;
IPAddress networkID() const;
uint8_t subnetCIDR() const;
IPAddress linkLocalIPv6() const; // IPv6 only
IPAddress globalIPv6() const;   // IPv6 only
```

Get IP address information from the interface.

Example:

```
Serial.print("Local IP: ");
Serial.println(WiFi.STA.localIP());

Serial.print("Subnet Mask: ");
Serial.println(WiFi.STA.subnetMask());

Serial.print("Gateway: ");
Serial.println(WiFi.STA.gatewayIP());

Serial.print("DNS: ");
Serial.println(WiFi.STA.dnsIP());

Serial.print("Broadcast: ");
Serial.println(WiFi.STA.broadcastIP());

Serial.print("Network ID: ");
Serial.println(WiFi.STA.networkID());

Serial.print("Subnet CIDR: /");
Serial.println(WiFi.STA.subnetCIDR());
```

MAC Address

```
uint8_t *macAddress(uint8_t *mac) const;
String macAddress() const;
```

Get the MAC address of the interface.

Example:

```
uint8_t mac[6];
WiFi.STA.macAddress(mac);
Serial.print("MAC: ");
for (int i = 0; i < 6; i++) {
    if (i > 0) Serial.print(":");
    Serial.print(mac[i], HEX);
}
Serial.println();

// Or as String
Serial.println(WiFi.STA.macAddress());
```

Default Interface

```
bool setDefault();
bool isDefault() const;
```

Set this interface as the default network interface, or check if it is the default.

Example:

```
// Set WiFi STA as default
WiFi.STA.setDefault();

// Check if it's default
if (WiFi.STA.isDefault()) {
    Serial.println("WiFi STA is the default interface");
}
```

Route Priority

```
int getRoutePrio() const;
int setRoutePrio(int prio); // ESP-IDF 5.5+
```

Gets or sets the route priority for the interface. Higher priority interfaces are preferred for routing.

Example:

```
#if ESP_IDF_VERSION >= ESP_IDF_VERSION_VAL(5, 5, 0)
// Set higher priority for Ethernet
ETH.setRoutePrio(100);
WiFi.STA.setRoutePrio(50);
#endif
```

Status Bits and Waiting

```
int getStatusBits() const;
int waitStatusBits(int bits, uint32_t timeout_ms) const;
```

Get current status bits or wait for specific status bits to be set.

Status Bits: * ESP_NETIF_STARTED_BIT: Interface has been started *
ESP_NETIF_CONNECTED_BIT: Interface is connected * ESP_NETIF_HAS_IP_BIT: Inter-
face has an IP address * ESP_NETIF_HAS_LOCAL_IP6_BIT: Interface has link-local IPv6 *
ESP_NETIF_HAS_GLOBAL_IP6_BIT: Interface has global IPv6 * ESP_NETIF_WANT_IP6_BIT: In-
terface wants IPv6 * ESP_NETIF_HAS_STATIC_IP_BIT: Interface has static IP configuration

Example:

```
// Wait for WiFi STA to get IP address (with 10 second timeout)
if (WiFi.STA.waitStatusBits(ESP_NETIF_HAS_IP_BIT, 10000) & ESP_NETIF_HAS_IP_BIT) {
    Serial.println("WiFi STA got IP address!");
} else {
    Serial.println("Timeout waiting for IP address");
}
```

Interface Information

```
const char *ifkey() const;
const char *desc() const;
String impl_name() const;
int impl_index() const;
esp_netif_t *netif();
```

Get interface identification and description information.

Example:

```
Serial.print("Interface key: ");
Serial.println(WiFi.STA.ifkey());

Serial.print("Description: ");
Serial.println(WiFi.STA.desc());

Serial.print("Implementation name: ");
Serial.println(WiFi.STA.impl_name());

Serial.print("Implementation index: ");
Serial.println(WiFi.STA.impl_index());
```

Network Interface Implementations

Wi-Fi Station (STA) The `STA` class (accessed via `WiFi.STA` object) extends `NetworkInterface` and provides Wi-Fi Station (client) mode functionality.

Key Features: * Connect to Wi-Fi access points * Automatic reconnection * WPA2/WPA3 security support * Enterprise Wi-Fi support (WPA2-Enterprise) * Scanning for available networks

Example:

```
#include "WiFi.h"

void setup() {
  Network.begin();
  WiFi.STA.begin();
  WiFi.STA.connect("ssid", "password");

  while (WiFi.STA.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.print("Connected! IP: ");
  Serial.println(WiFi.STA.localIP());
}
```

For detailed Wi-Fi Station API documentation, see [Wi-Fi API](#).

Wi-Fi Access Point (AP) The `AP` class (accessed via `WiFi.AP` object) extends `NetworkInterface` and provides Wi-Fi Access Point mode functionality.

Key Features: * Create Wi-Fi access points * DHCP server for connected stations * Network Address Translation (NAT) * Captive portal support

Example:

```
#include "WiFi.h"

void setup() {
  Network.begin();
  WiFi.AP.begin();
  WiFi.AP.create("MyESP32AP", "password123");

  Serial.print("AP IP: ");
  Serial.println(WiFi.AP.localIP());
}
```

For detailed Wi-Fi AP API documentation, see [Wi-Fi API](#).

Ethernet The `ETHClass` (accessed via `ETH` object) extends `NetworkInterface` and provides Ethernet connectivity.

Key Features: * Support for multiple Ethernet PHY types * SPI-based and EMAC-based Ethernet controllers * Automatic link detection * Full-duplex operation

Example:

```
#include "ETH.h"

void setup() {
  Network.begin();
  ETH.begin(ETH_PHY_TYPE, ETH_PHY_ADDR, ETH_PHY_MDC, ETH_PHY_MDIO, ETH_PHY_POWER, ↵
  ↵ETH_CLK_MODE);

  while (!ETH.hasIP()) {
    delay(500);
  }

  Serial.print("Ethernet IP: ");
  Serial.println(ETH.localIP());
}
```

For detailed Ethernet API documentation, see [Ethernet](#).

PPP (Point-to-Point Protocol) The `PPPClass` (accessed via `PPP` object) extends `NetworkInterface` and provides PPP connectivity, typically used with cellular modems.

Key Features: * Cellular modem support (SIM7000, SIM7600, BG96, etc.) * APN configuration * PIN code support * Hardware flow control

Example:

```
#include "PPP.h"

void setup() {
  Network.begin();
  PPP.begin(PPP_MODEM_SIM7600, 1, 115200);
  PPP.setApn("your.apn.here");

  while (!PPP.hasIP()) {
    delay(500);
  }

  Serial.print("PPP IP: ");
  Serial.println(PPP.localIP());
}
```

For detailed PPP API documentation, see the PPP library documentation.

Network Communication Classes

The Network library provides unified communication classes that work with any network interface: `NetworkClient`, `NetworkServer`, and `NetworkUdp`.

NetworkClient The `NetworkClient` class provides TCP client functionality that works with any network interface.

Key Features: * Connect to TCP servers * Read/write data * Timeout configuration * Socket options

Example:

```
#include "Network.h"
#include "WiFi.h"

NetworkClient client;

void setup() {
  Network.begin();
  WiFi.STA.begin();
  WiFi.STA.connect("ssid", "password");
  while (WiFi.STA.status() != WL_CONNECTED) delay(500);

  if (client.connect("www.example.com", 80)) {
    client.println("GET / HTTP/1.1");
    client.println("Host: www.example.com");
    client.println();

    while (client.available()) {
      char c = client.read();
      Serial.print(c);
    }

    client.stop();
  }
}
```

NetworkServer The NetworkServer class provides TCP server functionality that works with any network interface.

Key Features: * Accept incoming TCP connections * Multiple client support * Timeout configuration

Example:

```
#include "Network.h"
#include "WiFi.h"

NetworkServer server(80);

void setup() {
  Network.begin();
  WiFi.STA.begin();
  WiFi.STA.connect("ssid", "password");
  while (WiFi.STA.status() != WL_CONNECTED) delay(500);

  server.begin();
  Serial.print("Server started on: ");
  Serial.println(WiFi.STA.localIP());
}

void loop() {
  NetworkClient client = server.accept();
  if (client) {
    Serial.println("New client connected");
    client.println("Hello from ESP32!");
    client.stop();
  }
}
```

NetworkUdp The NetworkUdp class provides UDP communication that works with any network interface.

Key Features: * Send/receive UDP packets * Multicast support * Remote IP and port information

Example:

```
#include "Network.h"
#include "WiFi.h"

NetworkUDP udp;

void setup() {
  Network.begin();
  WiFi.STA.begin();
  WiFi.STA.connect("ssid", "password");
  while (WiFi.STA.status() != WL_CONNECTED) delay(500);

  udp.begin(1234);
}

void loop() {
  int packetSize = udp.parsePacket();
  if (packetSize) {
    Serial.print("Received packet from: ");
    Serial.print(udp.remoteIP());
    Serial.print(":");
    Serial.println(udp.remotePort());

    char buffer[255];
    int len = udp.read(buffer, 255);
    if (len > 0) {
      buffer[len] = 0;
      Serial.println(buffer);
    }
  }
}
```

Multiple Interface Management

The Network Manager allows you to manage multiple network interfaces simultaneously and switch between them as needed.

Example - Multiple Interfaces:

```
#include "Network.h"
#include "WiFi.h"
#include "ETH.h"

void setup() {
  Network.begin();

  // Start Ethernet
  ETH.begin(ETH_PHY_TYPE, ETH_PHY_ADDR, ETH_PHY_MDC, ETH_PHY_MDIO, ETH_PHY_POWER,
↳ETH_CLK_MODE);

  // Start WiFi as backup
  WiFi.STA.begin();
  WiFi.STA.connect("ssid", "password");

  // Wait for either interface to connect
  while (!ETH.connected() && WiFi.STA.status() != WL_CONNECTED) {
    delay(500);
  }

  // Set the connected interface as default
```

(continues on next page)

(continued from previous page)

```

if (ETH.connected()) {
    Network.setDefaultInterface(ETH);
    Serial.println("Using Ethernet");
} else if (WiFi.STA.status() == WL_CONNECTED) {
    Network.setDefaultInterface(WiFi.STA);
    Serial.println("Using WiFi");
}
}

```

Example - Interface Priority:

```

void setup() {
    Network.begin();

    // Start both interfaces
    ETH.begin(...);
    WiFi.STA.begin();
    WiFi.STA.connect("ssid", "password");

    // Set route priorities (ESP-IDF 5.5+)
    #if ESP_IDF_VERSION >= ESP_IDF_VERSION_VAL(5, 5, 0)
    ETH.setRoutePrio(100); // Higher priority
    WiFi.STA.setRoutePrio(50); // Lower priority
    #endif

    // Ethernet will be preferred for routing when both are connected
}

```

Event Handling Examples**Example - Monitor All Network Events:**

```

#include "Network.h"

void onNetworkEvent(arduino_event_id_t event) {
    Serial.print("[Network Event] ");
    Serial.println(NetworkEvents::eventName(event));

    switch (event) {
        case ARDUINO_EVENT_WIFI_STA_GOT_IP:
            Serial.println("WiFi Station got IP!");
            break;
        case ARDUINO_EVENT_ETH_GOT_IP:
            Serial.println("Ethernet got IP!");
            break;
        case ARDUINO_EVENT_PPP_GOT_IP:
            Serial.println("PPP got IP!");
            break;
        default:
            break;
    }
}

void setup() {
    Serial.begin(115200);
    Network.begin();
    Network.onEvent(onNetworkEvent);
}

```

Example - Interface-Specific Event Handling:


```
#include "Network.h"
#include "WiFi.h"

void onWiFiEvent(arduino_event_id_t event, arduino_event_info_t info) {
    if (event == ARDUINO_EVENT_WIFI_STA_GOT_IP) {
        IPAddress ip = IPAddress(info.got_ip.ip_info.ip.addr);
        IPAddress gateway = IPAddress(info.got_ip.ip_info.gw.addr);
        IPAddress subnet = IPAddress(info.got_ip.ip_info.netmask.addr);

        Serial.println("WiFi Connected!");
        Serial.print("IP: ");
        Serial.println(ip);
        Serial.print("Gateway: ");
        Serial.println(gateway);
        Serial.print("Subnet: ");
        Serial.println(subnet);
    }
}

void setup() {
    Network.begin();
    Network.onEvent(onWiFiEvent, ARDUINO_EVENT_WIFI_STA_GOT_IP);
    WiFi.STA.begin();
    WiFi.STA.connect("ssid", "password");
}
```

Troubleshooting

Interface Not Starting: * Ensure `Network.begin()` is called before using any interface * Check that the interface-specific initialization is correct * Verify hardware connections (for Ethernet/PPP)

Default Interface Not Working: * Explicitly set the default interface using `Network.setDefaultInterface()` * Check interface status using `started()`, `connected()`, and `hasIP()` * Verify route priorities if using multiple interfaces

Events Not Firing: * Ensure `Network.begin()` is called to initialize the event system * Check that callbacks are registered before the events occur * Use `NetworkEvents::eventName()` to verify event IDs

IP Configuration Issues: * For static IP, ensure all parameters (IP, gateway, subnet) are provided * Check that the IP address is not already in use on the network * Verify DNS server addresses are correct

Multiple Interface Conflicts: * Set appropriate route priorities to control which interface is used * Use `setDefaultInterface()` to explicitly select the default * Monitor events to see which interface gets IP addresses first

Related Documentation

- [ESP-IDF Network Interface Documentation](#)

2.3.15 Preferences

About

The Preferences library is unique to `arduino-esp32`. It should be considered as the replacement for the Arduino EEPROM library.

It uses a portion of the on-board non-volatile memory (NVS) of the ESP32 to store data. This data is retained across restarts and loss of power events to the system.

Preferences works best for storing many small values, rather than a few large values. If large amounts of data are to be stored, consider using a file system library such as LittleFS.

The Preferences library is usable by all ESP32 variants.

Header File

```
#include <Preferences.h>
```

Overview

Library methods are provided to:

- create a namespace;
- open and close a namespace;
- store and retrieve data within a namespace for supported data types;
- determine if a key value has been initialized;
- delete a key-value pair;
- delete all key-value pairs in a namespace;
- determine data types stored against a key;
- determine the number of key entries in the namespace.

Preferences directly supports the following data types:

Table 2: **Table 1 — Preferences Data Types**

Preferences Type	Data Type	Size (bytes)
Bool	bool	1
Char	int8_t	1
UChar	uint8_t	1
Short	int16_t	2
UShort	uint16_t	2
Int	int32_t	4
UInt	uint32_t	4
Long	int32_t	4
ULong	uint32_t	4
Float	float_t	4
Long64	int64_t	8
ULong64	uint64_t	8
Double	double_t	8
String	const char* String	variable
Bytes	uint8_t	variable

String values can be stored and retrieved either as an Arduino String or as a null terminated char array (c-string).

Bytes type is used for storing and retrieving an arbitrary number of bytes in a namespace.

Arduino-esp32 Preferences API

begin

Open non-volatile storage with a given namespace name from an NVS partition.

```
bool begin(const char * name, bool readOnly=false, const char* partition_
↳ label=NULL)
```

Parameters

•name (Required)

- Namespace name. Maximum length is 15 characters.

•readOnly (Optional)

- `false` will open the namespace in read-write mode.
- `true` will open the namespace in read-only mode.
- if omitted, the namespace is opened in read-write mode.

•partition_label (Optional)

- name of the NVS partition in which to open the namespace.
- if omitted, the namespace is opened in the “`nvs`” partition.

Returns

- `true` if the namespace was opened successfully; `false` otherwise.

Notes

- If the namespace does not exist within the partition, it is first created.
- Attempting to write a key value to a namespace open in read-only mode will fail.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

end

Close the currently opened namespace.

```
void end()
```

Parameters

- None

Returns

- Nothing

Note

- After closing a namespace, methods used to access it will fail.

clear

Delete all keys and values from the currently opened namespace.

```
bool clear()
```

Parameters

- None

Returns

- `true` if all keys and values were deleted; `false` otherwise.

Note

- the namespace name still exists afterward.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

remove

Delete a key-value pair from the currently open namespace.

```
bool remove(const char * key)
```

Parameters**•key (Required)**

- the name of the key to be deleted.

Returns

- `true` if key-value pair was deleted; `false` otherwise.

Note

- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

isKey

Check if a key-value pair from the currently open namespace exists.

```
bool isKey(const char * key)
```

Parameters

- **key (Required)**
 - the name of the key to be checked.

Returns

- `true` if key-value pair exists; `false` otherwise.

Note

- Attempting to check a key without a namespace being open will return false.

putChar, putUChar

Store a value against a given key in the currently open namespace.

```
size_t putChar(const char* key, int8_t value)
size_t putUChar(const char* key, uint8_t value)
```

Parameters

- **key (Required)**
 - if the key does not exist in the currently opened namespace it is first created.
- **value (Required)**
 - must match the data type of the method.

Returns

- 1 (the number of bytes stored for these data types) if the call is successful; 0 otherwise.

Notes

- Attempting to store a value without a namespace being open in read-write mode will fail.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

putShort, putUShort

Store a value against a given key in the currently open namespace.

```
size_t putShort(const char* key, int16_t value)
size_t putUShort(const char* key, uint16_t value)
```

Parameters

- **key (Required)**
 - if the key does not exist in the currently opened namespace it is first created.
- **value (Required)**
 - must match the data type of the method.

Returns

- 2 (the number of bytes stored for these data types) if the call is successful; 0 otherwise.

Notes

- Attempting to store a value without a namespace being open in read-write mode will fail.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

putInt, putUInt

putLong, putULong

putFloat

Store a value against a given key in the currently open namespace.

```
size_t putInt(const char* key, int32_t value)
size_t putUInt(const char* key, uint32_t value)
size_t putLong(const char* key, int32_t value)
size_t putULong(const char* key, uint32_t value)
size_t putFloat(const char* key, float_t value)
```

Parameters

- key (Required)**
 - if the key does not exist in the currently opened namespace it is first created.
- value (Required)**
 - must match the data type of the method.

Returns

- 4 (the number of bytes stored for these data types) if the call is successful; 0 otherwise.

Notes

- Attempting to store a value without a namespace being open in read-write mode will fail.
- A message providing the reason for a failed call is sent to the arduino-esp32 log_e facility.

putLong64, putULong64**putDouble**

Store a value against a given key in the currently open namespace.

```
size_t putLong64(const char* key, int64_t value)
size_t putULong64(const char* key, uint64_t value)
size_t putDouble(const char* key, double_t value)
```

Parameters

- key (Required)**
 - if the key does not exist in the currently opened namespace it is first created.
- value (Required)**
 - must match the data type of the method.

Returns

- 8 (the number of bytes stored for these data types) if the call is successful; 0 otherwise.

Notes

- Attempting to store a value without a namespace being open in read-write mode will fail.
- A message providing the reason for a failed call is sent to the arduino-esp32 log_e facility.

putBool

Store a value against a given key in the currently open namespace.

```
size_t putBool(const char* key, bool value)
```

Parameters

- key (Required)**
 - if the key does not exist in the currently opened namespace it is first created.
- value (Required)**
 - must match the data type of the method.

Returns

- true if successful; false otherwise.

Notes

- Attempting to store a value without a namespace being open in read-write mode will fail.
- A message providing the reason for a failed call is sent to the arduino-esp32 log_e facility.

putString

Store a variable length value against a given key in the currently open namespace.

```
size_t putString(const char* key, const char* value);  
size_t putString(const char* key, String value);
```

Parameters

- **key (Required)**
 - if the key does not exist in the currently opened namespace it is first created.
- **value (Required)**
 - if `const char*`, a null-terminated (c-string) character array.
 - if `String`, a valid Arduino String type.

Returns

- if successful: the number of bytes stored; 0 otherwise.

Notes

- Attempting to store a value without a namespace being open in read-write mode will fail.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

putBytes

Store a variable number of bytes against a given key in the currently open namespace.

```
size_t putBytes(const char* key, const void* value, size_t len);
```

Parameters

- **key (Required)**
 - if the key does not exist in the currently opened namespace it is first created.
- **value (Required)**
 - pointer to an array or buffer containing the bytes to be stored.
- **len (Required)**
 - the number of bytes from `value` to be stored.

Returns

- if successful: the number of bytes stored; 0 otherwise.

Notes

- Attempting to store a value without a namespace being open in read-write mode will fail.
- This method operates on the bytes used by the underlying data type, not the number of elements of a given data type. The data type of `value` is not retained by the Preferences library afterward.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

getChar, getUChar

Retrieve a value stored against a given key in the currently open namespace.

```
int8_t getChar(const char* key, int8_t defaultValue = 0)  
uint8_t getUChar(const char* key, uint8_t defaultValue = 0)
```

Parameters

- **key (Required)**
- **defaultValue (Optional)**
 - must match the data type of the method if provided.

Returns

- the value stored against `key` if the call is successful.
- `defaultValue`, if it is provided; 0 otherwise.

Notes

- Attempting to retrieve a key without a namespace being available will fail.
- Attempting to retrieve value from a non-existent key will fail.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

getShort, getUShort

Retrieve a value stored against a given key in the currently open namespace.

```
int16_t getShort(const char* key, int16_t defaultValue = 0)
uint16_t getUShort(const char* key, uint16_t defaultValue = 0)
```

Except for the data type returned, behaves exactly like `getChar`.

getInt, getUInt

Retrieve a value stored against a given key in the currently open namespace.

```
int32_t getInt(const char* key, int32_t defaultValue = 0)
uint32_t getUInt(const char* key, uint32_t defaultValue = 0)
```

Except for the data type returned, behaves exactly like `getChar`.

getLong, getULong

Retrieve a value stored against a given key in the currently open namespace.

```
int32_t getLong(const char* key, int32_t defaultValue = 0)
uint32_t getULong(const char* key, uint32_t defaultValue = 0)
```

Except for the data type returned, behaves exactly like `getChar`.

getLong64, getULong64

Retrieve a value stored against a given key in the currently open namespace.

```
int64_t getLong64(const char* key, int64_t defaultValue = 0)
uint64_t getULong64(const char* key, uint64_t defaultValue = 0)
```

Except for the data type returned, behaves exactly like `getChar`.

getFloat

Retrieve a value stored against a given key in the currently open namespace.

```
float_t getFloat(const char* key, float_t defaultValue = NAN)
```

Except for the data type returned and the value of `defaultValue`, behaves exactly like `getChar`.

getDouble

Retrieve a value stored against a given key in the currently open namespace.

```
double_t getDouble(const char* key, double_t defaultValue = NAN)
```

Except for the data type returned and the value of `defaultValue`, behaves exactly like `getChar`.

getBool

Retrieve a value stored against a given key in the currently open namespace.

```
bool getBool(const char* key, bool defaultValue = false);
```

Except for the data type returned, behaves exactly like `getChar`.

getString

Copy a string of `char` stored against a given key in the currently open namespace to a buffer.

```
size_t getString(const char* key, char* value, size_t maxlen);
```

Parameters

- `key` (Required)
- **value (Required)**
 - buffer to receive the null-terminated string
- **maxLen (Required)**
 - maximum number of bytes (including the null terminator) that can be written to `value`

Returns

- if successful: that many bytes (including the null terminator) are written to the buffer pointed to by `value`, and the method returns that number of bytes.
- if the method fails: nothing is written to the buffer and the method returns 0.

Notes

- `maxLen` must be greater than or equal to the number of bytes stored against the key (including the null terminator) or the call will fail. Use `getStringLength` to obtain the required size before calling.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

getString

Retrieve an Arduino String value stored against a given key in the currently open namespace.

```
String getString(const char* key, String defaultValue = String());
```

Parameters

- `key` (Required)
- `defaultValue` (Optional)

Returns

- the value stored against `key` if the call is successful
- if the method fails: it returns `defaultValue`, if provided; "" (an empty String) otherwise.

Notes

- `defaultValue` must be of type String.

getStringLength

Get the number of bytes (including the null terminator) stored for a string value against a key in the currently open namespace. Use this to size a buffer before calling `getString(key, value, maxlen)`.

```
size_t getStringLength(const char* key);
```

Parameters

- `key` (Required)

Returns

- if successful: the number of bytes in the value stored against `key` (including the null terminator); 0 otherwise.

Notes

- This method will fail (return 0) if the key does not exist or is not of type String.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

getBytes Copy a series of bytes stored against a given key in the currently open namespace to a buffer.


```
size_t getBytes(const char* key, void * buf, size_t maxLen);
```

Parameters

- **key** (Required)
- **buf** (Required)
 - buffer to receive the bytes
- **maxLen** (Required)
 - maximum number of bytes that can be written to buf

Returns

- if successful: that many bytes are written to the buffer pointed to by buf, and the method returns that number of bytes.
- if the method fails: nothing is written to the buffer and the method returns 0.

Notes

- maxLen must be greater than or equal to the number of bytes stored against the key or the call will fail. Use getBytesLength to obtain the required size before calling.
- A message providing the reason for a failed call is sent to the arduino-esp32 log_e facility.

getBytesLength Get the number of bytes stored in the value against a key of type Bytes in the currently open namespace.

```
size_t getBytesLength(const char* key)
```

Parameters

- **key** (Required)

Returns

- if successful: the number of bytes in the value stored against key; 0 otherwise.

Notes

- This method will fail if key is not of type Bytes.
- A message providing the reason for a failed call is sent to the arduino-esp32 log_e facility.

getType Get the Preferences data type of a given key within the currently open namespace.

```
PreferenceType getType(const char* key)
```

Parameters

- **key** (Required)

Returns

- an int value as per Table 2 below.
- a value of 10 (PT_INVALID) if the call fails.

Notes

- The return values are enumerated in Preferences.h. Table 2 includes the enumerated values for information.
- A return value can map to more than one Prefs Type.
- The method will fail if: the namespace is not open; the key does not exist; the provided key exceeds 15 characters.

Table 3: Table 2 —getType Return Values

Return value	Prefs Type	Data Type	Enumerated Value
0	Char	int8_t	PT_I8
1	UChar	uint8_t	PT_U8
	Bool	bool	
2	Short	int16_t	PT_I16
3	UShort	uint16_t	PT_U16
4	Int	int32_t	PT_I32
	Long		
5	UInt	uint32_t	PT_U32
	ULong		
6	Long64	int64_t	PT_I64
7	ULong64	uint64_t	PT_U64
8	String	String	PT_STR
		*char	
9	Double	double_t	PT_BLOB
	Float	float_t	
	Bytes	uint8_t	
10	-	-	PT_INVALID

freeEntries Get the number of free entries available in the key table of the currently open namespace.

```
size_t freeEntries()
```

Parameters

- none

Returns

- if successful: the number of free entries available in the key table of the currently open namespace; 0 otherwise.

Notes

- keys storing values of type `Bool`, `Char`, `UChar`, `Short`, `UShort`, `Int`, `UInt`, `Long`, `ULong`, `Long64`, `ULong64` use one entry in the key table.
- keys storing values of type `Float` and `Double` use three entries in the key table.
- Arduino or c-string `String` types use a minimum of two key table entries with the number of entries increasing with the length of the string.
- keys storing values of type `Bytes` use a minimum of three key table entries with the number of entries increasing with the number of bytes stored.
- A message providing the reason for a failed call is sent to the `arduino-esp32 log_e` facility.

2.3.16 Pulse Counter

About

Note: This peripheral is not supported yet by the Arduino API's.

2.3.17 ESP Rainmaker

About

This library allows to work with ESP RainMaker.

ESP RainMaker is an end-to-end solution offered by Espressif to enable remote control and monitoring for ESP32-S2 and ESP32 based products without any configuration required in the Cloud. The primary components of this solution are:

- Claiming Service (to get the Cloud connectivity credentials)
- RainMaker library (i.e. this library, to develop the firmware)
- RainMaker Cloud (backend, offering remote connectivity)
- RainMaker Phone App/CLI (Client utilities for remote access)

The key features of ESP RainMaker are:

1. Ability to define own devices and parameters, of any type, in the firmware.
2. Zero configuration required on the Cloud.
3. Phone apps that dynamically render the UI as per the device information.

Additional information about ESP RainMaker can be found [here](#).

ESP RainMaker Agent API

RMaker.initNode This initializes the ESP RainMaker agent, Wi-Fi and creates the node.

You can also set the configuration of the node using the following API

```
RMaker.setTimeSync (bool val)
```

NOTE: If you want to set the configuration for the node then these configuration API must be called before *RMaker.initNode()*.

```
Node initNode(const char *name, const char *type);
```

- name Name of the node
- type Type of the node

This function will return object of Node.

RMaker.start It starts the ESP RainMaker agent.

NOTE:

1. ESP RainMaker agent should be initialized before this call.
2. Once ESP RainMaker agent starts, compulsorily call `WiFi.beginProvision()` API.

```
esp_err_t start();
```

This function will return *ESP_OK* on success or *Error* in case of failure.

RMaker.stop It stops the ESP RainMaker agent which was started using *RMaker.start()*.

```
esp_err_t stop()
```

This function will return

1. *ESP_OK* : On success
2. Error in case of failure.

RMaker.deinitNode It deinitializes the ESP RainMaker agent and the node created using *RMaker.initNode()*.

```
esp_err_t deinitNode(Node node)
```

- node : Node object created using *RMaker.initNode()*

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

RMaker.enableOTA It enables OTA as per the ESP RainMaker Specification. For more details refer ESP RainMaker documentation, check [here](#).

```
esp_err_t enableOTA(ota_type_t type);
```

•**type**

[The OTA workflow type.]

- OTA_USING_PARAMS
- OTA_USING_TOPICS

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

RMaker.enableSchedule This API enables the scheduling service for the node. For more information, check [here](#).

```
esp_err_t enableSchedule();
```

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

RMaker.enableScenes This API enables the Scenes service for the node. It should be called after *RMaker.initNode()* and before *RMaker.start()*. For more information, check [here](#).

```
esp_err_t enableScenes()
```

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

RMaker.enableSystemService This API enables the System service for the node. It should be called after *RMaker.initNode()* and before *RMaker.start()*. For more information, check [here](#).

```
esp_err_t enableSystemService(uint16_t flags, int8_t reboot_seconds, int8_t reset_
↪seconds, int8_t reset_reboot_seconds)
```

- **flags** : Logical OR of system service flags (SYSTEM_SERV_FLAG_REBOOT, SYSTEM_SERV_FLAG_FACTORY_RESET, SYSTEM_SERV_FLAG_WIFI_RESET) as required or SYSTEM_SERV_FLAGS_ALL.
- **reboot_seconds** Time in seconds after which the device should reboot. Recommended value: 2
- **reset_seconds** Time in seconds after which the device should reset(Wi-Fi or Factory). Recommended value: 2
- **reset_reboot_seconds** Time in seconds after which the device should reboot after it has been reset. Zero as a value would mean there won't be any reboot after the reset. Recommended value: 2

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

RMaker.setTimeZone This API set's the timezone as a user friendly location string. Check [here](#) for a list of valid values.

NOTE : default value is "Asia/Shanghai" .

This API comes into picture only when working with scheduling.

```
esp_err_t setTimeZone(const char *tz);
```

- tz : Valid values as specified in documentation.

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

ESP RainMaker Node API

Node class expose API's for node.

NOTE : my_node is the object of Node class.

my_node.getNodeID It returns the unique node_id assigned to the node. This node_id is usually the MAC address of the board.

```
char * getNodeID();
```

- tz : Valid values as specified in documentation.

This function will return

1. *char ** : Pointer to a NULL terminated node_id string.

my_node.getNodeInfo It returns pointer to the node_info_t as configured during node initialization.

```
node_info_t * getNodeInfo();
```

This function will return

1. *node_info_t* : Pointer to the structure node_info_t on success.
2. *NULL* : On failure.

ESP RainMaker node info

It has following data member

1. *char * name*
2. *char * type*
3. *char * fw_version*
4. *char * model*

my_node.addNodeAttr It adds a new attribute as the metadata to the node.

NOTE : Only string values are allowed.

```
esp_err_t addNodeAttr(const char *attr_name, const char *val);
```

- attr_name : Name of the attribute
- val : Value of the attribute

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

my_node.addDevice It adds a device to the node.

NOTE :

- This is the mandatory API to register device to node.
- Single Node can have multiple devices.
- Device name should be unique for each device.

```
esp_err_t addDevice(Device device);
```

- device : Device object

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

my_node.removeDevice It removes a device from the node.

```
esp_err_t removeDevice(Device device);
```

- device : Device object

This function will return

1. *ESP_OK* : On success
2. Error in case of failure

ESP RainMaker Device API

Device class expose API' s for virtual devices on the node. Parameterized constructor is defined which creates the virtual device on the node. Using Device class object you can create your own device.

NOTE : my_device is the object of Device class

```
Device my_device(const char *dev_name, const char *dev_type, void *priv_data);
```

- dev_name : Unique device name
- **dev_type**
[Optional device type. It can be kept NULL.]
 - **Standard Device Types**
 - * ESP_RMAKER_DEVICE_SWITCH
 - * ESP_RMAKER_DEVICE_LIGHTBULB
 - * ESP_RMAKER_DEVICE_FAN
 - * ESP_RMAKER_DEVICE_TEMP_SENSOR
- priv_data : Private data associated with the device. This will be passed to the callbacks.

NOTE : This created device should be added to the node using `my_node.addDevice(my_device)` ;

- Sample example

```
Device my_device("Switch");  
Device my_device("Switch1", NULL, NULL);
```

- Here, dev_name is compulsory, rest are optional.
- Node can have multiple device, each device should have unique device name.

Standard Devices

- Classes are defined for the standard devices.
- Creating object of these class creates the standard device with default parameters to it.
- **Class for standard devices**
 - Switch

- LightBulb
- TemperatureSensor
- Fan

```
Switch my_switch(const char *dev_name, void *priv_data, bool power);
```

- `dev_name` : Unique device name by default it is “switch” for switch device.
- `priv_data` : Private data associated with the device. This will be passed to the callbacks.
- `power` : It is the value that can be set for primary parameter.

Sample example for standard device.

```
Switch switch1;  
Switch switch2("switch2", NULL, true);
```

- “switch2” : Name for standard device.
- `NULL` : Private data for the device, which will be used in callback.
- `true` : Default value for the primary param, in case of switch it is power.

NOTE: No parameter are compulsory for standard devices. However if you are creating two objects of same standard class then in that case you will have to set the device name, if not then both device will have same name which is set by default, hence device will not get create. *Device name should be unique for each device.*

my_device.getDeviceName It returns the name of the Device.

```
const char * getDeviceName();
```

- `device` : Device object

This function will return

- `char *`: Returns Device name.

NOTE: Each device on the node should have unique device name.

my_device.addDeviceAttr It adds attribute to the device. Device attributes are reported only once after a boot-up as part of the node configuration. Eg. Serial Number

```
esp_err_t addDeviceAttr(const char *attr_name, const char *val);
```

- `attr_name` : Name of the attribute
- `val` : Value of the attribute

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

my_device.deleteDevice It deletes the device created using parameterized constructor.

This device should be first removed from the node using `my_node.removeDevice(my_device)`.

```
esp_err_t deleteDevice();
```

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

my_device.addXParam It adds standard parameter to the device.

NOTE: X is the default name by which parameter is referred, you can specify your own name to each parameter.

- Eg. `my_device.addPowerParam(true)` here power parameter is referred with name Power.
- Eg. `my_device.addHueParam(12)` here hue parameter is referred with name Hue.

You can specify your own name to each parameter

- Eg. `my_device.addNameParam( "NickName" )` here name parameter is referred with name NickName.
- Eg. `my_device.addPowerParam(true, "FanPower" )` here power parameter is referred with name FanPower.

Standard Parameters

•These are the standard parameters.

- Name : ESP_RMAKER_DEF_NAME_PARAM
- Power : ESP_RMAKER_DEF_POWER_NAME
- Brightness : ESP_RMAKER_DEF_BRIGHTNESS_NAME
- Hue : ESP_RMAKER_DEF_HUE_NAME
- Saturation : ESP_RMAKER_DEF_SATURATION_NAME
- Intensity : ESP_RMAKER_DEF_INTENSITY_NAME
- CCT : ESP_RMAKER_DEF_CCT_NAME
- Direction : ESP_RMAKER_DEF_DIRECTION_NAME
- Speed : ESP_RMAKER_DEF_SPEED_NAME
- Temperature : ESP_RMAKER_DEF_TEMPERATURE_NAME

```
esp_err_t addNameParam(const char *param_name = ESP_RMAKER_DEF_NAME_PARAM);
esp_err_t addPowerParam(bool val, const char *param_name = ESP_RMAKER_DEF_POWER_
↳NAME);
esp_err_t addBrightnessParam(int val, const char *param_name = ESP_RMAKER_DEF_
↳BRIGHTNESS_NAME);
esp_err_t addHueParam(int val, const char *param_name = ESP_RMAKER_DEF_HUE_NAME);
esp_err_t addSaturationParam(int val, const char *param_name = ESP_RMAKER_DEF_
↳SATURATION_NAME);
esp_err_t addIntensityParam(int val, const char *param_name = ESP_RMAKER_DEF_
↳INTENSITY_NAME);
esp_err_t addCCTParam(int val, const char *param_name = ESP_RMAKER_DEF_CCT_NAME);
esp_err_t addDirectionParam(int val, const char *param_name = ESP_RMAKER_DEF_
↳DIRECTION_NAME);
esp_err_t addSpeedParam(int val, const char *param_name = ESP_RMAKER_DEF_SPEED_
↳NAME);
esp_err_t addTempratureParam(float val, const char *param_name = ESP_RMAKER_DEF_
↳TEMPERATURE_NAME);
```

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

NOTE : Care should be taken while accessing name of parameter. Above mentioned are the two ways using which default name of parameters can be accessed. Either LHS or RHS.

my_device.assignPrimaryParam It assigns a parameter (already added using `addXParam()` or `addParam()`) as a primary parameter, which can be used by clients (phone apps specifically) to give prominence to it.

```
esp_err_t assignPrimaryParam(param_handle_t *param);
```

- param : Handle of the parameter. It is obtained using `my_device.getParamByName()`.

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

my_device.getParamByName

```
param_handle_t * getParamByName(const char *param_name);
```

- `param_name` : It is the name of the parameter which was added using `addXparam()` or `addParam()`.

This function will return object of the parameter.

my_device.addParam It allows user to add custom parameter to the device created using *Param* class.

```
esp_err_t addParam(Param parameter);
```

- `parameter` : Object of *Param*

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

NOTE: *Param* class exposes API's to create the custom parameter.

my_device.updateAndReportParam It updates the parameter associated with particular device on ESP RainMaker cloud.

```
esp_err_t updateAndReportParam(const char *param_name, value);
```

- `param_name` : Name of the parameter
- `value` : Value to be updated. It can be int, bool, char *, float.

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

my_device.addCb It registers read and write callback for the device which will be invoked as per requests received from the cloud (or other paths as may be added in future).

```
void addCb(deviceWriteCb write_cb, deviceReadCb read_cb);
```

- **write_cb**
[Function with signature] `func_name(Device *device, Param *param, const param_val_t val, void *priv_data, write_ctx_t *ctx);`
- **read_cb**
[Function with signature] `func_name(Device *device, Param *param, void *priv_data, read_ctx_t *ctx);`

Parameters**param_val_t val**

Value can be accessed as below

1. *bool* : `val.val.b`
2. *integer* : `val.val.i`
3. *float* : `val.val.f`
4. *char ** : `val.val.s`

ESP RainMaker Param API

Param class expose API's for creating custom parameters for the devices and report and update values associated with parameter to the ESP RainMaker cloud. Parameterized constructor is defined which creates custom parameter.

NOTE : *my_param* is the object of *Param* class.

```
Param my_param(const char *param_name, const char *param_type, param_val_t val,
↳uint8_t properties);
```

- `param_name` : Name of the parameter
- `param_type` : Type of the parameter. It is optional can be kept NULL.
- `val` : Define the default value for the parameter. It should be defined using `value(int ival)` , `value(bool bval)` , `value(float fval)` , `value(char *sval)`.

•properties

[Properties of the parameter, which will be a logical OR of flags.]

–Flags

- * PROP_FLAG_WRITE
- * PROP_FLAG_READ
- * PROP_FLAG_TIME_SERIES
- * PROP_FLAG_PERSIST

Sample example :

```
Param my_param(const char *param_name, const char *param_type, param_val_t val,
↳uint8_t properties);
Param my_param("bright", NULL, value(30), PROP_FLAG_READ | PROP_FLAG_WRITE | PROP_
↳FLAG_PERSIST);
```

NOTE : Parameter created using Param class should be added to the device using `my_device.addParam(my_param)`;

my_param.addUIType Add a UI type to the parameter. This will be used by the Phone apps (or other clients) to render appropriate UI for the given parameter. Please refer the RainMaker documentation [here](#) for supported UI Types.

```
esp_err_t addUIType(const char *ui_type);
```

•ui_type

[String describing the UI Type.]

–Standard UI Types

- * ESP_RMAKER_UI_TOGGLE
- * ESP_RMAKER_UI_SLIDER
- * ESP_RMAKER_UI_DROPDOWN
- * ESP_RMAKER_UI_TEXT

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

my_param.addBounds Add bounds for an integer/float parameter. This can be used to add bounds (min/max values) for a given integer/float parameter. Eg. brightness will have bounds as 0 and 100 if it is a percentage.

```
esp_err_t addBounds(param_val_t min, param_val_t max, param_val_t step);
```

- `min` : Minimum value
- `max` : Maximum value
- `step` : step Minimum stepping

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

Sample example : `my_param.addBounds(value(0), value(100), value(5));`

my_param.updateAndReport It updates the parameter and report it to ESP RainMaker cloud. This is called in callback.

```
esp_err_t updateAndReport(param_val_t val);
```

- `val` : New value of the parameter

This function will return

1. `ESP_OK` : On success
2. Error in case of failure

NOTE:

- This API should always be called inside device write callback, if you aimed at updating n reporting parameter values, changed via RainMaker Client (Phone App), to the ESP RainMaker cloud.
- If not called then parameter values will not be updated to the ESP RainMaker cloud.

printQR This API displays QR code, which is used in provisioning.

```
printQR(const char *serv_name, const char *pop, const char *transport);
```

- `name` : Service name used in provisioning API.
- `pop` : Proof of possession used in provisioning API.
- **transport** :
 1. *softap* : In case of provisioning using SOFTAP.
 2. *ble* : In case of provisioning using BLE.

RMakerFactoryReset Reset the device to factory defaults.

```
RMakerFactoryReset(int seconds);
```

- `seconds` : Time in seconds after which the chip should reboot after doing a factory reset.

RMakerWiFiReset Reset Wi-Fi credentials.

```
RMakerWiFiReset(int seconds);
```

- `seconds` : Time in seconds after which the chip should reboot after doing a Wi-Fi reset.

2.3.18 Reset Reason

About

Note: This is a work in progress project and this section is still missing. If you want to contribute, please see the [Contributions Guide](#).

Example

To get started with Reset Reason, you can try:

Reset Reason

```

/*
 * Print last reset reason of ESP32
 * =====
 *
 * Use either of the methods print_reset_reason
 * or verbose_print_reset_reason to display the
 * cause for the last reset of this device.
 *
 * Public Domain License.
 *
 * Author:
 * Evandro Luis Copercini - 2017
 */

#include <Arduino.h>

#if CONFIG_IDF_TARGET_ESP32 // ESP32/PICO-D4
#include "esp32/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32S2
#include "esp32s2/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32C2
#include "esp32c2/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32C3
#include "esp32c3/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32S3
#include "esp32s3/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32C6
#include "esp32c6/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32H2
#include "esp32h2/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32P4
#include "esp32p4/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32C5
#include "esp32c5/rom/rtc.h"
#elif CONFIG_IDF_TARGET_ESP32C61
#include "esp32c61/rom/rtc.h"
#else
#error Target CONFIG_IDF_TARGET is not supported
#endif

#define uS_TO_S_FACTOR 1000000 /* Conversion factor for micro seconds to seconds */

void print_reset_reason(int reason) {
    switch (reason) {
        case 1: Serial.println("POWERON_RESET"); break;           /**<1, Vbat power_
        ↪on reset*/
        case 3: Serial.println("SW_RESET"); break;               /**<3, Software_
        ↪reset digital core*/
        case 4: Serial.println("OWDT_RESET"); break;             /**<4, Legacy watch_
        ↪dog reset digital core*/
        case 5: Serial.println("DEEPSLEEP_RESET"); break;         /**<5, Deep Sleep_
        ↪reset digital core*/
        case 6: Serial.println("SDIO_RESET"); break;             /**<6, Reset by SLC_
        ↪module, reset digital core*/
        case 7: Serial.println("TG0WDT_SYS_RESET"); break;        /**<7, Timer Group0_
        ↪Watch dog reset digital core*/
        case 8: Serial.println("TG1WDT_SYS_RESET"); break;        /**<8, Timer Group1_
        ↪Watch dog reset digital core*/
        case 9: Serial.println("RTCWDT_SYS_RESET"); break;        /**<9, RTC Watch_
        ↪dog Reset digital core*/
        case 10: Serial.println("INTRUSION_RESET"); break;        /**<10, Instrusion_
        ↪tested to reset CPU*/
    }

```

(continues on next page)

(continued from previous page)

```

    case 11: Serial.println("TGWDT_CPU_RESET"); break;          /**<11, Time Group
↪reset CPU*/
    case 12: Serial.println("SW_CPU_RESET"); break;            /**<12, Software
↪reset CPU*/
    case 13: Serial.println("RTCWDT_CPU_RESET"); break;        /**<13, RTC Watch
↪dog Reset CPU*/
    case 14: Serial.println("EXT_CPU_RESET"); break;           /**<14, for APP CPU,
↪reset by PRO CPU*/
    case 15: Serial.println("RTCWDT_BROWN_OUT_RESET"); break;  /**<15, Reset when
↪the vdd voltage is not stable*/
    case 16: Serial.println("RTCWDT_RTC_RESET"); break;        /**<16, RTC Watch
↪dog reset digital core and rtc module*/
    default: Serial.println("NO_MEAN");
}
}

void verbose_print_reset_reason(int reason) {
    switch (reason) {
        case 1: Serial.println("Vbat power on reset"); break;
        case 3: Serial.println("Software reset digital core"); break;
        case 4: Serial.println("Legacy watch dog reset digital core"); break;
        case 5: Serial.println("Deep Sleep reset digital core"); break;
        case 6: Serial.println("Reset by SLC module, reset digital core"); break;
        case 7: Serial.println("Timer Group0 Watch dog reset digital core"); break;
        case 8: Serial.println("Timer Group1 Watch dog reset digital core"); break;
        case 9: Serial.println("RTC Watch dog Reset digital core"); break;
        case 10: Serial.println("Intrusion tested to reset CPU"); break;
        case 11: Serial.println("Time Group reset CPU"); break;
        case 12: Serial.println("Software reset CPU"); break;
        case 13: Serial.println("RTC Watch dog Reset CPU"); break;
        case 14: Serial.println("for APP CPU, reset by PRO CPU"); break;
        case 15: Serial.println("Reset when the vdd voltage is not stable"); break;
        case 16: Serial.println("RTC Watch dog reset digital core and rtc module");
↪break;
        default: Serial.println("NO_MEAN");
    }
}

void setup() {
    // put your setup code here, to run once:
    Serial.begin(115200);
    delay(2000);

    Serial.println("CPU0 reset reason:");
    print_reset_reason(rtc_get_reset_reason(0));
    verbose_print_reset_reason(rtc_get_reset_reason(0));

    Serial.println("CPU1 reset reason:");
    print_reset_reason(rtc_get_reset_reason(1));
    verbose_print_reset_reason(rtc_get_reset_reason(1));

#ifdef CONFIG_IDF_TARGET_ESP32H2
    // Set ESP32 to go to deep sleep to see a variation
    // in the reset reason. Device will sleep for 5 seconds.
    #if CONFIG_IDF_TARGET_ESP32C3 || CONFIG_IDF_TARGET_ESP32H2
        esp_sleep_pd_config(ESP_PD_DOMAIN_RC_FAST, ESP_PD_OPTION_OFF);
    #else
        esp_sleep_pd_config(ESP_PD_DOMAIN_RTC_PERIPH, ESP_PD_OPTION_OFF);
    #endif
    Serial.println("Going to sleep");
    esp_deep_sleep(5 * uS_TO_S_FACTOR);

```

(continues on next page)

(continued from previous page)

```

#endif
}

void loop() {
    // put your main code here, to run repeatedly:
}

/*
Example Serial Log:
=====

rst:0x10 (RTCWDT_RTC_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0x00
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0008,len:8
load:0x3fff0010,len:160
load:0x40078000,len:10632
load:0x40080000,len:252
entry 0x40080034
CPU0 reset reason:
RTCWDT_RTC_RESET
RTC Watch dog reset digital core and rtc module
CPU1 reset reason:
EXT_CPU_RESET
for APP CPU, reset by PRO CPU
Going to sleep
ets Jun  8 2016 00:22:57

rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0x00
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0008,len:8
load:0x3fff0010,len:160
load:0x40078000,len:10632
load:0x40080000,len:252
entry 0x40080034
CPU0 reset reason:
DEEPSLEEP_RESET
Deep Sleep reset digital core
CPU1 reset reason:
EXT_CPU_RESET
for APP CPU, reset by PRO CPU
Going to sleep
ets Jun  8 2016 00:22:57

rst:0x5 (DEEPSLEEP_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
configsip: 0, SPIWP:0x00
clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
mode:DIO, clock div:1
load:0x3fff0008,len:8
load:0x3fff0010,len:160
load:0x40078000,len:10632
load:0x40080000,len:252
entry 0x40080034
CPU0 reset reason:
DEEPSLEEP_RESET
Deep Sleep reset digital core
CPU1 reset reason:
EXT_CPU_RESET

```

(continues on next page)

(continued from previous page)

```
for APP CPU, reset by PRO CPU
Going to sleep

*/
```

2.3.19 Remote Control (RMT)

About

The Remote Control Transceiver (RMT) peripheral was originally designed to act as an infrared transceiver, but it can be used to generate or receive many other types of digital signals with precise timing.

The RMT peripheral is capable of transmitting and receiving digital signals with precise timing control, making it ideal for protocols that require specific pulse widths and timing, such as:

- **Infrared (IR) remote control protocols** (NEC, RC5, Sony, etc.)
- **WS2812/NeoPixel RGB LED control** (requires precise timing)
- **Custom communication protocols** with specific timing requirements
- **Pulse generation** for various applications

RMT operates by encoding digital signals as sequences of high/low pulses with specific durations. Each RMT symbol represents two consecutive pulses (level0/duration0 and level1/duration1).

RMT Memory Blocks

RMT channels use memory blocks to store signal data. The number of available memory blocks varies by SoC:

ESP32 SoC	Memory Blocks	Notes
ESP32	8 blocks total	Shared between TX and RX channels
ESP32-S2	4 blocks total	Shared between TX and RX channels
ESP32-S3	4 blocks TX + 4 RX	Separate memory for TX and RX channels
ESP32-C3	2 blocks TX + 2 RX	Separate memory for TX and RX channels
ESP32-C5	2 blocks TX + 2 RX	Separate memory for TX and RX channels
ESP32-C6	2 blocks TX + 2 RX	Separate memory for TX and RX channels
ESP32-H2	2 blocks TX + 2 RX	Separate memory for TX and RX channels
ESP32-P4	4 blocks TX + 4 RX	Separate memory for TX and RX channels

Each memory block can store `RMT_SYMBOLS_PER_CHANNEL_BLOCK` symbols (64 for ESP32/ESP32-S2, 48 for ESP32-S3/ESP32-C3/ESP32-C5/ESP32-C6/ESP32-H2/ESP32-P4).

Note: Each RMT symbol is 4 bytes (32 bits), containing two pulses with their durations and levels.

Arduino-ESP32 RMT API

rmtInit Initializes an RMT channel for a specific GPIO pin with the specified direction, memory size, and frequency.

```
bool rmtInit(int pin, rmt_ch_dir_t channel_direction, rmt_reserve_memsize_t_
↪ memsize, uint32_t frequency_Hz);
```

- `pin` - GPIO pin number to use for RMT
- `channel_direction` - Channel direction:
 - `RMT_RX_MODE` - Receive mode (for reading signals)
 - `RMT_TX_MODE` - Transmit mode (for sending signals)
- `memsize` - Number of memory blocks to reserve for this channel:

- RMT_MEM_NUM_BLOCKS_1 - 1 block
- RMT_MEM_NUM_BLOCKS_2 - 2 blocks
- RMT_MEM_NUM_BLOCKS_3 - 3 blocks (ESP32 only)
- RMT_MEM_NUM_BLOCKS_4 - 4 blocks (ESP32 only)
- RMT_MEM_NUM_BLOCKS_5 through RMT_MEM_NUM_BLOCKS_8 - 5-8 blocks (ESP32 only)
- frequency_Hz - RMT channel frequency in Hz (tick frequency). Must be between 312.5 kHz and 80 MHz. The frequency determines the resolution of pulse durations. For example:
 - 10 MHz (100 ns tick) - High precision, suitable for WS2812 LEDs
 - 1 MHz (1 µs tick) - Good for most IR protocols
 - 400 kHz (2.5 µs tick) - Suitable for slower protocols

This function returns `true` if initialization is successful, `false` otherwise.

Note: The RMT tick is set by the frequency parameter. Example: 100 ns tick => 10 MHz, thus frequency will be 10,000,000 Hz.

rmtDeinit Deinitializes the RMT channel and releases all allocated resources for the specified pin.

```
bool rmtDeinit(int pin);
```

- pin - GPIO pin number that was initialized with RMT

This function returns `true` if deinitialization is successful, `false` otherwise.

rmtSetEOT Sets the End of Transmission (EOT) level for the RMT pin when transmission ends. This function affects how `rmtWrite()`, `rmtWriteAsync()`, or `rmtWriteLooping()` will set the pin after writing the data.

```
bool rmtSetEOT(int pin, uint8_t EOT_Level);
```

- pin - GPIO pin number configured for RMT TX mode
- EOT_Level - End of transmission level:
 - 0 (LOW) - Pin will be set to LOW after transmission (default)
 - Non-zero (HIGH) - Pin will be set to HIGH after transmission

Note: This only affects the transmission process. The pre-transmission idle level can be set manually using `digitalWrite(pin, level)`.

This function returns `true` if EOT level is set successfully, `false` otherwise.

rmtWrite Sends RMT data in blocking mode. The function waits until all data is transmitted or until timeout occurs.

```
bool rmtWrite(int pin, rmt_data_t *data, size_t num_rmt_symbols, uint32_t timeout_
↪ms);
```

- pin - GPIO pin number configured for RMT TX mode
- data - Pointer to array of `rmt_data_t` symbols to transmit
- num_rmt_symbols - Number of RMT symbols to transmit
- timeout_ms - Timeout in milliseconds. Use `RMT_WAIT_FOR_EVER` for indefinite wait

Blocking mode: The function only returns after sending all data or by timeout.

This function returns `true` if transmission is successful, `false` on error or timeout.

Example:

```
rmt_data_t symbols[] = {
    {8, 1, 4, 0}, // High for 8 ticks, Low for 4 ticks (bit '1')
    {4, 1, 8, 0}  // High for 4 ticks, Low for 8 ticks (bit '0')
};
```

(continues on next page)

(continued from previous page)

```
rmtWrite(pin, symbols, 2, RMT_WAIT_FOR_EVER);
```

rmtWriteAsync Sends RMT data in non-blocking (asynchronous) mode. The function returns immediately after starting the transmission.

```
bool rmtWriteAsync(int pin, rmt_data_t *data, size_t num_rmt_symbols);
```

- `pin` - GPIO pin number configured for RMT TX mode
- `data` - Pointer to array of `rmt_data_t` symbols to transmit
- `num_rmt_symbols` - Number of RMT symbols to transmit

Non-blocking mode: Returns immediately after execution. Use `rmtTransmitCompleted()` to check if transmission is finished.

Note: If `rmtWriteAsync()` is called while a previous transmission is still in progress, it will return `false` immediately to indicate failure; it does not wait for the previous transmission to finish.

This function returns `true` on execution success, `false` otherwise.

rmtWriteLooping Sends RMT data in infinite looping mode. The data will be transmitted continuously until stopped.

```
bool rmtWriteLooping(int pin, rmt_data_t *data, size_t num_rmt_symbols);
```

- `pin` - GPIO pin number configured for RMT TX mode
- `data` - Pointer to array of `rmt_data_t` symbols to transmit
- `num_rmt_symbols` - Number of RMT symbols to transmit

Looping mode: The data is transmitted continuously in a loop. To stop looping, call `rmtWrite()` or `rmtWriteAsync()` with new data, or call `rmtWriteLooping()` with `NULL` data or zero size.

Note: Looping mode needs a zero-ending data symbol `{0, 0, 0, 0}` to mark the end of data.

This function returns `true` on execution success, `false` otherwise.

rmtWriteRepeated Sends RMT data a fixed number of times (repeated transmission).

```
bool rmtWriteRepeated(int pin, rmt_data_t *data, size_t num_rmt_symbols, uint32_t loop_count);
```

- `pin` - GPIO pin number configured for RMT TX mode
- `data` - Pointer to array of `rmt_data_t` symbols to transmit
- `num_rmt_symbols` - Number of RMT symbols to transmit
- `loop_count` - Number of times to repeat the transmission (must be at least 1)

Note: `* loop_count == 0` is invalid (no transmission) `* loop_count == 1` transmits once (no looping) `* loop_count > 1` transmits the data repeatedly

Note: Loop count feature is only supported on certain SoCs. On unsupported SoCs, this function will return `false`.

This function returns `true` on execution success, `false` otherwise.

rmtTransmitCompleted Checks if the RMT transmission is completed and the channel is ready for transmitting new data.

```
bool rmtTransmitCompleted(int pin);
```

- `pin` - GPIO pin number configured for RMT TX mode

This function returns `true` when all data has been sent and the channel is ready for a new transmission, `false` otherwise.

Note: * If `rmtWrite()` times out or `rmtWriteAsync()` is called, this function will return `false` until all data is sent out. * `rmtTransmitCompleted()` will always return `true` when `rmtWriteLooping()` is active, because it has no effect in such case.

rmtRead Initiates blocking receive operation. Reads RMT data and stores it in the provided buffer.

```
bool rmtRead(int pin, rmt_data_t *data, size_t *num_rmt_symbols, uint32_t timeout_
↪ms);
```

- `pin` - GPIO pin number configured for RMT RX mode
- `data` - Pointer to buffer where received RMT symbols will be stored
- `num_rmt_symbols` - Pointer to variable containing maximum number of symbols to read. On return, this variable will contain the actual number of symbols read.
- `timeout_ms` - Timeout in milliseconds. Use `RMT_WAIT_FOR_EVER` for indefinite wait

Blocking mode: The function waits until data is received or timeout occurs.

If the reading operation times out, `num_rmt_symbols` won't change and `rmtReceiveCompleted()` can be used later to check if data is available.

This function returns `true` when data is successfully read, `false` on error or timeout.

rmtReadAsync Initiates non-blocking (asynchronous) receive operation. Returns immediately after starting the receive process.

```
bool rmtReadAsync(int pin, rmt_data_t *data, size_t *num_rmt_symbols);
```

- `pin` - GPIO pin number configured for RMT RX mode
- `data` - Pointer to buffer where received RMT symbols will be stored
- `num_rmt_symbols` - Pointer to variable containing maximum number of symbols to read. On completion, this variable will be updated with the actual number of symbols read.

Non-blocking mode: Returns immediately after execution. Use `rmtReceiveCompleted()` to check if data is available.

This function returns `true` on execution success, `false` otherwise.

rmtReceiveCompleted Checks if RMT data reception is completed and new data is available for processing.

```
bool rmtReceiveCompleted(int pin);
```

- `pin` - GPIO pin number configured for RMT RX mode

This function returns `true` when data has been received and is available in the buffer, `false` otherwise.

Note: The data reception information is reset when a new `rmtRead()` or `rmtReadAsync()` function is called.

rmtSetCarrier Sets carrier frequency modulation/demodulation for RMT TX or RX channel.

```
bool rmtSetCarrier(int pin, bool carrier_en, bool carrier_level, uint32_t_
↪frequency_Hz, float duty_percent);
```

- `pin` - GPIO pin number configured for RMT
- `carrier_en` - Enable/disable carrier modulation (TX) or demodulation (RX)
- `carrier_level` - Carrier polarity level:
 - `true` - Positive polarity (active high)
 - `false` - Negative polarity (active low)
- `frequency_Hz` - Carrier frequency in Hz (e.g., 38000 for 38 kHz IR carrier)

- `duty_percent` - Duty cycle as a float from 0.0 to 1.0 (e.g., 0.33 for 33% duty cycle, 0.5 for 50% square wave)

Note: Parameters changed in Arduino Core 3: low and high (ticks) are now expressed in Carrier Frequency in Hz and duty cycle in percentage (float 0.0 to 1.0).

Example: 38.5 kHz carrier with 33% duty cycle: `rmtSetCarrier(pin, true, true, 38500, 0.33)`

This function returns `true` if carrier is set successfully, `false` otherwise.

rmtSetRxMinThreshold Sets the minimum pulse width filter threshold for RX channel. Pulses smaller than this threshold will be ignored as noise.

```
bool rmtSetRxMinThreshold(int pin, uint8_t filter_pulse_ticks);
```

- `pin` - GPIO pin number configured for RMT RX mode
- `filter_pulse_ticks` - Minimum pulse width in RMT ticks. Pulses (high or low) smaller than this will be filtered out.
Set to 0 to disable the filter.

Note: The filter threshold is specified in RMT ticks, which depends on the RMT frequency set during `rmtInit()`.

This function returns `true` if filter threshold is set successfully, `false` otherwise.

rmtSetRxMaxThreshold Sets the maximum idle threshold for RX channel. When no edge is detected for longer than this threshold, the receiving process is finished.

```
bool rmtSetRxMaxThreshold(int pin, uint16_t idle_thres_ticks);
```

- `pin` - GPIO pin number configured for RMT RX mode
- `idle_thres_ticks` - Maximum idle time in RMT ticks. When no edge is detected for longer than this time, reception ends.
This threshold also defines how many low/high bits are read at the end of the received data.

Note: The idle threshold is specified in RMT ticks, which depends on the RMT frequency set during `rmtInit()`.

This function returns `true` if idle threshold is set successfully, `false` otherwise.

RMT Data Structure

rmt_data_t RMT data structure representing a single RMT symbol (two consecutive pulses).

```
typedef union {
    struct {
        uint32_t duration0 : 15; // Duration of first pulse in RMT ticks
        uint32_t level0 : 1; // Level of first pulse (0 = LOW, 1 = HIGH)
        uint32_t duration1 : 15; // Duration of second pulse in RMT ticks
        uint32_t level1 : 1; // Level of second pulse (0 = LOW, 1 = HIGH)
    };
    uint32_t val; // Access as 32-bit value
} rmt_data_t;
```

Each RMT symbol contains two pulses: * **First pulse:** `level0` for `duration0` ticks * **Second pulse:** `level1` for `duration1` ticks

Example:

```
// Create a symbol: HIGH for 8 ticks, then LOW for 4 ticks
rmt_data_t symbol = {
    .duration0 = 8,
    .level0 = 1,
```

(continues on next page)

(continued from previous page)

```

        .duration1 = 4,
        .level1 = 0
    };

    // Or using struct initialization
    rmt_data_t symbol2 = {8, 1, 4, 0};

```

Helper Macros

RMT_SYMBOLS_OF Helper macro to calculate the number of RMT symbols in an array.

```
#define RMT_SYMBOLS_OF(x) (sizeof(x) / sizeof(rmt_data_t))
```

Example:

```

rmt_data_t data[] = {
    {8, 1, 4, 0},
    {4, 1, 8, 0}
};

size_t num_symbols = RMT_SYMBOLS_OF(data); // Returns 2

```

RMT_WAIT_FOR_EVER Constant for indefinite timeout in blocking operations.

```
#define RMT_WAIT_FOR_EVER ((uint32_t)portMAX_DELAY)
```

Use this constant as the `timeout_ms` parameter in `rmtWrite()` or `rmtRead()` to wait indefinitely.

Example:

```
rmtWrite(pin, data, num_symbols, RMT_WAIT_FOR_EVER);
```

RMT_SYMBOLS_PER_CHANNEL_BLOCK Constant defining the number of RMT symbols per memory block.

- ESP32/ESP32-S2: 64 symbols per block
- ESP32-S3/ESP32-C3/ESP32-C5/ESP32-C6/ESP32-H2/ESP32-P4: 48 symbols per block

Example:

```

// Allocate buffer for 1 memory block
rmt_data_t buffer[RMT_SYMBOLS_PER_CHANNEL_BLOCK];

```

Example Applications

RMT Write RGB LED (WS2812):

```

// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software

```

(continues on next page)

(continued from previous page)

```
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates usage of RGB LED driven by RMT
 *
 * The output is a visual WS2812 RGB LED color moving in a 8 x 4 LED matrix
 * Parameters can be changed by the user. In a single LED circuit, it will just
 * blink.
 */

#include <Arduino.h>

// The effect seen in (Espressif devkits) ESP32C6, ESP32H2, ESP32C3, ESP32S2 and
// ESP32S3 is like a Blink of RGB LED
#ifdef PIN_LED_RGB
#define BUILTIN_RGBLED_PIN PIN_LED_RGB
#else
#define BUILTIN_RGBLED_PIN 21 // ESP32 has no builtin RGB LED (PIN_LED_RGB)
#endif

#define NR_OF_LEDS      8 * 4
#define NR_OF_ALL_BITS 24 * NR_OF_LEDS

//
// Note: This example uses a board with 32 WS2812b LEDs chained one
//       after another, each RGB LED has its 24 bit value
//       for color configuration (8b for each color)
//
//       Bits encoded as pulses as follows:
//
//       "0":
//       +-----+           +--
//       |         |         |
//       |         |         |
//       |         |         |
//       ---|         |-----|
//       +         +         +
//       | 0.4us | 0.85 0us |
//
//       "1":
//       +-----+           +--
//       |         |         |
//       |         |         |
//       |         |         |
//       |         |         |
//       ---+         +-----+
//       | 0.8us | 0.4us |
//
rmt_data_t led_data[NR_OF_ALL_BITS];

void setup() {
  Serial.begin(115200);
  if (!rmtInit(BUILTIN_RGBLED_PIN, RMT_TX_MODE, RMT_MEM_NUM_BLOCKS_1, 10000000)) {
    Serial.println("init sender failed\n");
  }
  Serial.println("real tick set to: 100ns");
}
```

(continues on next page)

(continued from previous page)

```

int color[] = {0x55, 0x11, 0x77}; // Green Red Blue values
int led_index = 0;

void loop() {
    // Init data with only one led ON
    int led, col, bit;
    int i = 0;
    for (led = 0; led < NR_OF_LEDS; led++) {
        for (col = 0; col < 3; col++) {
            for (bit = 0; bit < 8; bit++) {
                if ((color[col] & (1 << (7 - bit))) && (led == led_index)) {
                    led_data[i].level0 = 1;
                    led_data[i].duration0 = 8;
                    led_data[i].level1 = 0;
                    led_data[i].duration1 = 4;
                } else {
                    led_data[i].level0 = 1;
                    led_data[i].duration0 = 4;
                    led_data[i].level1 = 0;
                    led_data[i].duration1 = 8;
                }
                i++;
            }
        }
    }
    // make the led travel in the panel
    if ((++led_index) >= NR_OF_LEDS) {
        led_index = 0;
    }
    // Send the data and wait until it is done
    rmtWrite(BUILTIN_RGBLED_PIN, led_data, NR_OF_ALL_BITS, RMT_WAIT_FOR_EVER);
    delay(100);
}

```

RMT LED Blink:

```

// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrate how to use RMT to just blink a regular LED_
 * ↳ (GPIO)
 * It uses all the different RMT Writing APIs to blink the LED by hardware, not_
 * ↳ being
 * necessary the regular Blink code in Arduino.
 *
 * The output is the Blinking LED in the GPIO and a serial output describing what_
 * ↳ is
 * going on, along the execution.

```

(continues on next page)

[illegible]

(continued from previous page)

```

// 400,000 x 2.5us = 1 second OFF
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
// Looping mode needs a Zero ending data to mark the EOF
{0, 0, 0, 0}
};

// RMT is at 400KHz with a 2.5us tick
// This RMT data sends a 1Hz pulse with 500ms High and 500ms Low signal
rmt_data_t blink_500ms_rmt_data[] = {
    // 200,000 x 2.5us = 0.5 second ON
    {
        25000,
        1,
        25000,
    }
};

```

(continues on next page)

(continued from previous page)

```

    1,
},
{
    25000,
    1,
    25000,
    1,
},
{
    25000,
    1,
    25000,
    1,
},
{
    25000,
    1,
    25000,
    1,
},
// 200,000 x 2.5us = 0.5 second OFF
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
// Looping mode needs a Zero ending data to mark the EOF
{0, 0, 0, 0}
};

// RMT is at 400KHz with a 2.5us tick
// This RMT data sends a 2Hz pulse with 250ms High and 250ms Low signal
rmt_data_t blink_250ms_rmt_data[] = {
    // 100,000 x 2.5us = 0.25 second ON
    {
        25000,
        1,
        25000,
        1,
    },
    {
        25000,

```

(continues on next page)

(continued from previous page)

```

    1,
    25000,
    1,
},
// 100,000 x 2.5us = 0.25 second OFF
{
    25000,
    0,
    25000,
    0,
},
{
    25000,
    0,
    25000,
    0,
},
// Looping mode needs a Zero ending data to mark the EOF
{0, 0, 0, 0}
};

void RMT_Mixed_Write_Blink() {
    Serial.println("==> rmtWriteLooping() to Blink the LED.");
    Serial.println("Blinking at 1s on + 1s off :: 3 blinks");
    if (!rmtWriteLooping(BLINK_GPIO, blink_1s_rmt_data, RMT_SYMBOLS_OF(blink_1s_rmt_
    ↪data))) {
        Serial.println("==> rmtWriteLooping Blink 1s Error!");
    }
    delay(6000); // blinking happens here, done by hardware!

    Serial.println("==> rmtWrite() (Blocking Mode) to Blink the LED.");
    Serial.println("Blinking at 500ms on + 500ms off :: 4 blinks");
    for (uint8_t i = 0; i < 4; i++) {
        if (!rmtWrite(BLINK_GPIO, blink_500ms_rmt_data, RMT_SYMBOLS_OF(blink_500ms_rmt_
    ↪data) - 1, RMT_WAIT_FOR_EVER)) {
            Serial.println("==> rmtWrite Blink 0.5s Error!");
        }
    }

    Serial.println("==> rmtWriteAsync() (Non-Blocking Mode) to Blink the LED.");
    Serial.println("Blinking at 250ms on + 250ms off :: 5 blinks");
    for (uint8_t i = 0; i < 5; i++) {
        if (!rmtWriteAsync(BLINK_GPIO, blink_250ms_rmt_data, RMT_SYMBOLS_OF(blink_
    ↪250ms_rmt_data) - 1)) {
            Serial.println("==> rmtWrite Blink 0.25s Error!");
        }
        // wait (blocks) until all the data is sent out
        while (!rmtTransmitCompleted(BLINK_GPIO));
    }
    Serial.println("Blinking OFF for 1 seconds");
    delay(1000);
}

void RMT_Loop_Write_Blink() {
    Serial.println("Using RMT Writing loop to blink an LED.");
    Serial.println("Blinking at 1s on + 1s off :: 3 blinks");
    if (!rmtWriteLooping(BLINK_GPIO, blink_1s_rmt_data, RMT_SYMBOLS_OF(blink_1s_rmt_
    ↪data))) {
        Serial.println("==> rmtWriteLooping Blink 1s Error!");
    }
    delay(6000);
}

```

(continues on next page)

(continued from previous page)

```

Serial.println("Blinking at 500ms on + 500ms off :: 5 blinks");
if (!rmtWriteLooping(BLINK_GPIO, blink_500ms_rmt_data, RMT_SYMBOLS_OF(blink_
↪500ms_rmt_data))) {
    Serial.println("==> rmtWriteLooping Blink 0.5s Error!");
}
delay(5000);
Serial.println("Blinking at 250ms on + 250ms off :: 10 blinks");
if (!rmtWriteLooping(BLINK_GPIO, blink_250ms_rmt_data, RMT_SYMBOLS_OF(blink_
↪250ms_rmt_data))) {
    Serial.println("==> rmtWriteLooping Blink 0.25s Error!");
}
delay(5000);

Serial.println("Blinking OFF for 2 seconds");
rmt_data_t blink_STOP_rmt_data[] = {{0, 0, 0, 0}};
if (!rmtWrite(BLINK_GPIO, blink_STOP_rmt_data, RMT_SYMBOLS_OF(blink_STOP_rmt_
↪data), RMT_WAIT_FOR_EVER)) {
    Serial.println("==> rmtWrite Blink STOP Error!");
}
delay(2000);
}

void RMT_Repeated_Write_Blink() {
    Serial.println("Using RMT Writing repeated N times to blink an LED.");
    Serial.println("Blinking at 1s on + 1s off :: 2 blinks");
    // repeating blink_1s_rmt_data (1s on + 1s off) 2 times for 2 blinks
    if (!rmtWriteRepeated(BLINK_GPIO, blink_1s_rmt_data, RMT_SYMBOLS_OF(blink_1s_rmt_
↪data), 2)) {
        Serial.println("==> rmtWrite Blink 1s Error!");
    }
    delay(4000 + 1000); // it should blink 2 times and stop automatically
    Serial.println("Blinking at 500ms on + 500ms off :: 4 blinks");
    // repeating blink_500ms_rmt_data (500ms on + 500ms off) 4 times for 4 blinks
    if (!rmtWriteRepeated(BLINK_GPIO, blink_500ms_rmt_data, RMT_SYMBOLS_OF(blink_
↪500ms_rmt_data), 4)) {
        Serial.println("==> rmtWrite Blink 0.5s Error!");
    }
    delay(4000 + 1000); // it should blink 4 times and stop automatically
    Serial.println("Blinking at 250ms on + 250ms off :: 8 blinks");
    // repeating blink_250ms_rmt_data (250ms on + 250ms off) 8 times for 8 blinks
    if (!rmtWriteRepeated(BLINK_GPIO, blink_250ms_rmt_data, RMT_SYMBOLS_OF(blink_
↪250ms_rmt_data), 8)) {
        Serial.println("==> rmtWrite Blink 0.25s Error!");
    }
    delay(4000 + 1000); // it should blink 8 times and stop automatically
    Serial.println("Blinking is OFF for 2 seconds");
    delay(2000);
}

void RMT_Single_Write_Blocking_Blink() {
    Serial.println("Using RMT Writing and its Completion to blink an LED.");
    Serial.println("Blinking at 1s on + 1s off :: 2 blinks");
    for (uint8_t i = 0; i < 2; i++) {
        if (!rmtWrite(BLINK_GPIO, blink_1s_rmt_data, RMT_SYMBOLS_OF(blink_1s_rmt_data),
↪1, RMT_WAIT_FOR_EVER)) {
            Serial.println("==> rmtWrite Blink 1s Error!");
        }
    }
    Serial.println("Blinking at 500ms on + 500ms off :: 4 blinks");
    for (uint8_t i = 0; i < 4; i++) {
        if (!rmtWrite(BLINK_GPIO, blink_500ms_rmt_data, RMT_SYMBOLS_OF(blink_500ms_rmt_

```

(continues on next page)

(continued from previous page)

```

    ↪data) - 1, RMT_WAIT_FOR_EVER)) {
        Serial.println("==> rmtWrite Blink 0.5s Error!");
    }
}
Serial.println("Blinking at 250ms on + 250ms off :: 8 blinks");
for (uint8_t i = 0; i < 8; i++) {
    if (!rmtWrite(BLINK_GPIO, blink_250ms_rmt_data, RMT_SYMBOLS_OF(blink_250ms_rmt_
    ↪data) - 1, RMT_WAIT_FOR_EVER)) {
        Serial.println("==> rmtWrite Blink 0.25s Error!");
    }
}
Serial.println("Blinking OFF for 3 seconds");
delay(3000);
}

void RMT_Write_Aync_Non_Blocking_Blink() {
    Serial.println("Using RMT Async Writing and its Completion to blink an LED.");
    Serial.println("Blinking at 1s on + 1s off :: 5 blinks");
    for (uint8_t i = 0; i < 5; i++) {
        if (!rmtWriteAsync(BLINK_GPIO, blink_1s_rmt_data, RMT_SYMBOLS_OF(blink_1s_rmt_
    ↪data) - 1)) {
            Serial.println("==> rmtWrite Blink 1s Error!");
        }
        // wait (blocks) until all the data is sent out
        while (!rmtTransmitCompleted(BLINK_GPIO));
    }
    Serial.println("Blinking at 500ms on + 500ms off :: 5 blinks");
    for (uint8_t i = 0; i < 5; i++) {
        if (!rmtWriteAsync(BLINK_GPIO, blink_500ms_rmt_data, RMT_SYMBOLS_OF(blink_
    ↪500ms_rmt_data) - 1)) {
            Serial.println("==> rmtWrite Blink 0.5s Error!");
        }
        // wait (blocks) until all the data is sent out
        while (!rmtTransmitCompleted(BLINK_GPIO));
    }
    Serial.println("Blinking at 250ms on + 250ms off :: 5 blinks");
    for (uint8_t i = 0; i < 5; i++) {
        if (!rmtWriteAsync(BLINK_GPIO, blink_250ms_rmt_data, RMT_SYMBOLS_OF(blink_
    ↪250ms_rmt_data) - 1)) {
            Serial.println("==> rmtWrite Blink 0.25s Error!");
        }
        // wait (blocks) until all the data is sent out
        while (!rmtTransmitCompleted(BLINK_GPIO));
    }
    Serial.println("Blinking OFF for 1 seconds");
    delay(1000);
}

void setup() {
    Serial.begin(115200);
    Serial.println("Starting Blink testing...");
    Serial.flush();

    // 1 RMT Block has 64 RMT_SYMBOLS (ESP32/ESP32S2) or 48 RMT_SYMBOLS_
    ↪(ESP32C3/ESP32S3)
    if (!rmtInit(BLINK_GPIO, RMT_TX_MODE, RMT_MEM_NUM_BLOCKS_1, 400000)) { //2.5us_
    ↪tick
        Serial.println("==> rmtInit Error!");
    } else {
        Serial.println("==> rmtInit OK! Tick = 2.5us - OK for testing");
    }
}

```

(continues on next page)

(continued from previous page)

```

Serial.println("\n=====");
Serial.println("All set. Starting RMT testing Routine.");
Serial.println("=====\n");

RMT_Mixed_Write_Blink();
Serial.println("End of Mixed Calls testing");

Serial.println("\n=====");
Serial.println("Starting a Blinking sequence...");
Serial.println("=====\n");
}

void loop() {
  RMT_Write_Aync_Non_Blocking_Blink();
  RMT_Loop_Write_Blink();
  RMT_Repeated_Write_Blink();
  RMT_Single_Write_Blocking_Blink();
  Serial.println("\nStarting OVER...\n");
}

```

RMT Read XJT Protocol:

```

// Copyright 2023 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates usage of RMT for receiving XJT D16 data
 *
 * The output is the RMT data read and processed
 */
#include <Arduino.h>

//
// Note: This example uses a FrSKY device communication
//       using XJT D16 protocol
//
// ; 0 bit = 6us low/10us high
// ; 1 bit = 14us low/10us high
// ;
// ; -----+           +-----+           +-----+
// ;         |         |         |         |         |
// ;         |  0  |         |         |  1  |         |
// ;         |         |         |         |         |
// ;         |         |         |         |         |
// ;         +-----+           +-----+           +-----+
// ;
// ;         |  6us      10us   |         |  14us      10us   |

```

(continues on next page)

(continued from previous page)

```

// ;          |-----|-----|-----|-----|-----|
// ;          |          16us          |          24us          |

// Typedef of received frame
//
// ; 0x00 - Sync, 0x7E (sync header ID)
// ; 0x01 - Rx ID, 0x?? (receiver ID number, 0x00-0x??)
// ; 0x02 - Flags 1, 0x?? (used for failsafe and binding)
// ; 0x03 - Flags 2, 0x00 (reserved)
// ; 0x04-0x06, Channels 1/9 and 2/10
// ; 0x07-0x09, Channels 3/11 and 4/12
// ; 0x0A-0x0C, Channels 5/13 and 6/14
// ; 0x0D-0x0F, Channels 7/15 and 8/16
// ; 0x10 - 0x00, always zero
// ; 0x11 - CRC-16 High
// ; 0x12 - CRC-16 Low
// ; 0x13 - Tail, 0x7E (tail ID)
typedef union {
    struct {
        uint8_t head;           //0x7E
        uint8_t rxid;           //Receiver Number
        uint8_t flags;          //Range:0x20, Bind:0x01
        uint8_t reserved0;      //0x00
        union {
            struct {
                uint8_t ch0_l;
                uint8_t ch0_h : 4;
                uint8_t ch1_l : 4;
                uint8_t ch1_h;
            };
            uint8_t bytes[3];
        } channels[4];
        uint8_t reserved1;      //0x00
        uint8_t crc_h;
        uint8_t crc_l;
        uint8_t tail;          //0x7E
    };
    uint8_t buffer[20];
} xjt_packet_t;

#define XJT_VALID(i) (i->level0 && !i->level1 && i->duration0 >= 8 && i->duration0
↪ <= 11)

static uint32_t *s_channels;
static uint32_t channels[16];
static uint8_t xjt_flags = 0x0;
static uint8_t xjt_rxid = 0x0;

static bool xjtReceiveBit(size_t index, bool bit) {
    static xjt_packet_t xjt;
    static uint8_t xjt_bit_index = 8;
    static uint8_t xht_byte_index = 0;
    static uint8_t xht_ones = 0;

    if (!index) {
        xjt_bit_index = 8;
        xht_byte_index = 0;
        xht_ones = 0;
    }

    if (xht_byte_index > 19) {

```

(continues on next page)

(continued from previous page)

```

    //fail!
    return false;
}
if (bit) {
    xht_ones++;
    if (xht_ones > 5 && xht_byte_index && xht_byte_index < 19) {
        //fail!
        return false;
    }
    //add bit
    xjt.buffer[xht_byte_index] |= (1 << --xjt_bit_index);
} else if (xht_ones == 5 && xht_byte_index && xht_byte_index < 19) {
    xht_ones = 0;
    //skip bit
    return true;
} else {
    xht_ones = 0;
    //add bit
    xjt.buffer[xht_byte_index] &= ~(1 << --xjt_bit_index);
}
if ((!xjt_bit_index) || (xjt_bit_index == 1 && xht_byte_index == 19)) {
    xjt_bit_index = 8;
    if (!xht_byte_index && xjt.buffer[0] != 0x7E) {
        //fail!
        return false;
    }
    xht_byte_index++;
    if (xht_byte_index == 20) {
        //done
        if (xjt.buffer[19] != 0x7E) {
            //fail!
            return false;
        }
    }
    //check crc?

    xjt_flags = xjt.flags;
    xjt_rxid = xjt.rxid;
    for (int i = 0; i < 4; i++) {
        uint16_t ch0 = xjt.channels[i].ch0_l | ((uint16_t)(xjt.channels[i].ch0_h &
↪0x7) << 8);
        ch0 = ((ch0 * 2) + 2452) / 3;
        uint16_t ch1 = xjt.channels[i].ch1_l | ((uint16_t)(xjt.channels[i].ch1_h &
↪0x7F) << 4);
        ch1 = ((ch1 * 2) + 2452) / 3;
        uint8_t c0n = i * 2;
        if (xjt.channels[i].ch0_h & 0x8) {
            c0n += 8;
        }
        uint8_t c1n = i * 2 + 1;
        if (xjt.channels[i].ch1_h & 0x80) {
            c1n += 8;
        }
        s_channels[c0n] = ch0;
        s_channels[c1n] = ch1;
    }
}
}
return true;
}

void parseRmt(rmt_data_t *items, size_t len, uint32_t *channels) {

```

(continues on next page)

(continued from previous page)

```

bool valid = true;
rmt_data_t *it = NULL;

if (!channels) {
    log_e("Please provide data block for storing channel info");
    return;
}
s_channels = channels;

it = &items[0];
for (size_t i = 0; i < len; i++) {

    if (!valid) {
        break;
    }
    it = &items[i];
    if (XJT_VALID(it)) {
        if (it->duration1 >= 5 && it->duration1 <= 8) {
            valid = xjtReceiveBit(i, false);
        } else if (it->duration1 >= 13 && it->duration1 <= 16) {
            valid = xjtReceiveBit(i, true);
        } else {
            valid = false;
        }
    } else if (!it->duration1 && !it->level1 && it->duration0 >= 5 && it->
↳duration0 <= 8) {
        valid = xjtReceiveBit(i, false);
    }
}

// Change the RMT reading GPIO here:
#define RMT_GPIO 21

void setup() {
    Serial.begin(115200);
    // Initialize the channel to capture up to 64*2 or 48*2 items - 1us tick
    if (!rmtInit(RMT_GPIO, RMT_RX_MODE, RMT_MEM_NUM_BLOCKS_2, 1000000)) {
        Serial.println("init receiver failed\n");
    }
    Serial.println("real tick set to: 1us");
}

void loop() {
    static rmt_data_t data[RMT_MEM_NUM_BLOCKS_2 * RMT_SYMBOLS_PER_CHANNEL_BLOCK];
    static size_t data_symbols = RMT_MEM_NUM_BLOCKS_2 * RMT_SYMBOLS_PER_CHANNEL_
↳BLOCK;

    // Blocking read with timeout of 500ms
    // If data is read, data_symbols will have the number of RMT symbols effectively.
↳read
    // to check if something was read and it didn't just timeout, use
↳rmtReceiveCompleted()
    rmtRead(RMT_GPIO, data, &data_symbols, 500);

    // If read something, process the data
    if (rmtReceiveCompleted(RMT_GPIO)) {
        Serial.printf("Got %lu RMT Symbols. Parsing data...\n", (unsigned long)data_
↳symbols);
        parseRmt(data, data_symbols, channels);
    } else {

```

(continues on next page)

(continued from previous page)

```

Serial.println("No RMT data read...");
}

// printout some of the channels every 500ms
Serial.printf("%04" PRIx32 " %04" PRIx32 " %04" PRIx32 " %04" PRIx32 "\n",
↳ channels[0], channels[1], channels[2], channels[3]);
}

```

Complete list of [RMT examples](#).

2.3.20 SDIO

About

Note: This peripheral is not supported yet by the Arduino API's.

2.3.21 SD MMC

About

Note: This is a work in progress project and this section is still missing. If you want to contribute, please see the [Contributions Guide](#).

Example

To get started with SD_MMC, you can try:

SDMMC Test

```

/*
 * pin 1 - D2           | Micro SD card |
 * pin 2 - D3           |         |
 * pin 3 - CMD          |         |
 * pin 4 - VDD (3.3V)   |         |
 * pin 5 - CLK          | 8 7 6 5 4 3 2 1 |
 * pin 6 - VSS (GND)    | 0 0 0 0 0 0 0 0 |
 * pin 7 - D0           | 0 0 0 0 0 0 0 0 |
 * pin 8 - D1           |         |
 *
 *                      0 0 0 0 0 0 0 0
 *                      0 0 0 0 0 0 0 0
 *                      0 0 0 0 0 0 0 0
 *
 * Connections for      0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 * full-sized           0 0 0 0 0 0 0 0 0 0 0 0 0 0
 * SD card              0 0 0 0 0 0 0 0 0 0 0 0
 *
 * ESP32-P4 Func EV | 40 39 GND 43 3V3 GND 44 42 41 | SLOT 0 (IO_MUX)
 * ESP32-S3 DevKit  | 21 47 GND 39 3V3 GND 40 41 42 |
 * ESP32-S3-USB-OTG | 38 37 GND 36 3V3 GND 35 34 33 |
 * ESP32            | 4 2 GND 14 3V3 GND 15 13 12 |
 * Pin name         | D1 D0 VSS CLK VDD VSS CMD D3 D2 |
 * SD pin number    | 8 7 6 5 4 3 2 1 9 |
 *
 *                  | 0 /

```

(continues on next page)

(continued from previous page)

```

*           |_2_2_2_2_2_2_2_2_/
* WARNING: ALL data pins must be pulled up to 3.3V with an external 10k Ohm
↳ resistor!
* Note to ESP32 pin 2 (D0): Add a 1K Ohm pull-up resistor to 3.3V after flashing
*
* SD Card | ESP32
*   D2      12
*   D3      13
*   CMD     15
*   VSS     GND
*   VDD     3.3V
*   CLK     14
*   VSS     GND
*   D0      2   (add 1K pull up after flashing)
*   D1      4
*
*   For more info see file README.md in this library or on URL:
*   https://github.com/espressif/arduino-esp32/tree/master/libraries/SD_MMC
*/

#include <Arduino.h>
#include "FS.h"
#include "SD_MMC.h"

#ifdef CONFIG_IDF_TARGET_ESP32S3
// Default pins for ESP-S3
// Warning: ESP32-S3-WROOM-2 is using most of the default GPIOs (33-37) to
↳ interface with on-board OPI flash.
// If the SD_MMC is initialized with default pins it will result in rebooting
↳ loop - please
// reassign the pins elsewhere using the mentioned command `setPins`.
// Note: ESP32-S3-WROOM-1 does not have GPIO 33 and 34 broken out.
// Note: if it's ok to use default pins, you do not need to call the setPins
int clk = 36;
int cmd = 35;
int d0 = 37;
int d1 = 38;
int d2 = 33;
int d3 = 39; // GPIO 34 is not broken-out on ESP32-S3-DevKitC-1 v1.1
#endif

void listDir(fs::FS &fs, const char *dirname, uint8_t levels) {
  Serial.printf("Listing directory: %s\n", dirname);

  File root = fs.open(dirname);
  if (!root) {
    Serial.println("Failed to open directory");
    return;
  }
  if (!root.isDirectory()) {
    Serial.println("Not a directory");
    return;
  }

  File file = root.openNextFile();
  while (file) {
    if (file.isDirectory()) {
      Serial.print(" DIR : ");
      Serial.println(file.name());
      if (levels) {
        listDir(fs, file.path(), levels - 1);
      }
    } else {
      Serial.print("  FILE: ");
      Serial.print(file.name());
      Serial.print("  SIZE: ");
      Serial.println(file.size());
    }
    file = root.openNextFile();
  }
}

```

(continues on next page)

(continued from previous page)

```

    }
    } else {
        Serial.print("  FILE: ");
        Serial.print(file.name());
        Serial.print("  SIZE: ");
        Serial.println(file.size());
    }
    file = root.openNextFile();
}
}

void createDir(fs::FS &fs, const char *path) {
    Serial.printf("Creating Dir: %s\n", path);
    if (fs.mkdir(path)) {
        Serial.println("Dir created");
    } else {
        Serial.println("mkdir failed");
    }
}

void removeDir(fs::FS &fs, const char *path) {
    Serial.printf("Removing Dir: %s\n", path);
    if (fs.rmdir(path)) {
        Serial.println("Dir removed");
    } else {
        Serial.println("rmdir failed");
    }
}

void readFile(fs::FS &fs, const char *path) {
    Serial.printf("Reading file: %s\n", path);

    File file = fs.open(path);
    if (!file) {
        Serial.println("Failed to open file for reading");
        return;
    }

    Serial.print("Read from file: ");
    while (file.available()) {
        Serial.write(file.read());
    }
}

void writeFile(fs::FS &fs, const char *path, const char *message) {
    Serial.printf("Writing file: %s\n", path);

    File file = fs.open(path, FILE_WRITE);
    if (!file) {
        Serial.println("Failed to open file for writing");
        return;
    }
    if (file.print(message)) {
        Serial.println("File written");
    } else {
        Serial.println("Write failed");
    }
}

void appendFile(fs::FS &fs, const char *path, const char *message) {
    Serial.printf("Appending to file: %s\n", path);

```

(continues on next page)

(continued from previous page)

```

File file = fs.open(path, FILE_APPEND);
if (!file) {
    Serial.println("Failed to open file for appending");
    return;
}
if (file.print(message)) {
    Serial.println("Message appended");
} else {
    Serial.println("Append failed");
}
}

void renameFile(fs::FS &fs, const char *path1, const char *path2) {
    Serial.printf("Renaming file %s to %s\n", path1, path2);
    if (fs.rename(path1, path2)) {
        Serial.println("File renamed");
    } else {
        Serial.println("Rename failed");
    }
}

void deleteFile(fs::FS &fs, const char *path) {
    Serial.printf("Deleting file: %s\n", path);
    if (fs.remove(path)) {
        Serial.println("File deleted");
    } else {
        Serial.println("Delete failed");
    }
}

void testFileIO(fs::FS &fs, const char *path) {
    File file = fs.open(path);
    static uint8_t buf[512];
    size_t len = 0;
    uint32_t start = millis();
    uint32_t end = start;
    if (file) {
        len = file.size();
        size_t flen = len;
        start = millis();
        while (len) {
            size_t toRead = len;
            if (toRead > 512) {
                toRead = 512;
            }
            file.read(buf, toRead);
            len -= toRead;
        }
        end = millis() - start;
        Serial.printf("%lu bytes read for %" PRIu32 " ms\n", (unsigned long)flen, end);
        file.close();
    } else {
        Serial.println("Failed to open file for reading");
    }

    file = fs.open(path, FILE_WRITE);
    if (!file) {
        Serial.println("Failed to open file for writing");
        return;
    }
}

```

(continues on next page)

(continued from previous page)

```

size_t i;
start = millis();
for (i = 0; i < 2048; i++) {
    file.write(buf, 512);
}
end = millis() - start;
Serial.printf("%u bytes written for %" PRIu32 " ms\n", 2048 * 512, end);
file.close();
}

void setup() {
    Serial.begin(115200);
    /*
        // If you want to change the pin assignment or you get an error that some pins
        // are not assigned on ESP32-S3/ESP32-P4 uncomment this block and the
    ↪appropriate
        // line depending if you want to use 1-bit or 4-bit line.
        // Please note that ESP32 does not allow pin change and setPins() will always
    ↪fail.
        //if(! SD_MMC.setPins(clk, cmd, d0)){
        //if(! SD_MMC.setPins(clk, cmd, d0, d1, d2, d3)){
        //    Serial.println("Pin change failed!");
        //    return;
        //}
    */

    if (!SD_MMC.begin()) {
        Serial.println("Card Mount Failed");
        return;
    }
    uint8_t cardType = SD_MMC.cardType();

    if (cardType == CARD_NONE) {
        Serial.println("No SD_MMC card attached");
        return;
    }

    Serial.print("SD_MMC Card Type: ");
    if (cardType == CARD_MMC) {
        Serial.println("MMC");
    } else if (cardType == CARD_SD) {
        Serial.println("SDSC");
    } else if (cardType == CARD_SDHC) {
        Serial.println("SDHC");
    } else {
        Serial.println("UNKNOWN");
    }

    uint64_t cardSize = SD_MMC.cardSize() / (1024 * 1024);
    Serial.print("SD_MMC Card Size: ");
    Serial.print(cardSize);
    Serial.println("MB");

    listDir(SD_MMC, "/", 0);
    createDir(SD_MMC, "/mydir");
    listDir(SD_MMC, "/", 0);
    removeDir(SD_MMC, "/mydir");
    listDir(SD_MMC, "/", 2);
    writeFile(SD_MMC, "/hello.txt", "Hello ");
    appendFile(SD_MMC, "/hello.txt", "World!\n");

```

(continues on next page)

(continued from previous page)

```
readFile(SD_MMC, "/hello.txt");
deleteFile(SD_MMC, "/foo.txt");
renameFile(SD_MMC, "/hello.txt", "/foo.txt");
readFile(SD_MMC, "/foo.txt");
testFileIO(SD_MMC, "/test.txt");

Serial.print("Total space: ");
Serial.print(SD_MMC.totalBytes() / (1024 * 1024));
Serial.println("MB");
Serial.print("Used space: ");
Serial.print(SD_MMC.usedBytes() / (1024 * 1024));
Serial.println("MB");
}

void loop() {
  delay(10);
}
```

Complete list of [SD MMC examples](#).

2.3.22 Serial (UART)

About

The Serial (UART - Universal Asynchronous Receiver-Transmitter) peripheral provides asynchronous serial communication, allowing the ESP32 to communicate with other devices such as computers, sensors, displays, and other microcontrollers.

UART is a simple, two-wire communication protocol that uses a TX (transmit) and RX (receive) line for full-duplex communication. The ESP32 Arduino implementation provides a `HardwareSerial` class that is compatible with the standard Arduino Serial API, with additional features for advanced use cases.

Key Features:

- **Full-duplex communication:** Simultaneous transmission and reception
- **Configurable baud rates:** From 300 to 5,000,000+ baud
- **Multiple data formats:** Configurable data bits, parity, and stop bits
- **Hardware flow control:** Support for RTS/CTS signals
- **RS485 support:** Half-duplex RS485 communication mode
- **Low-power UART:** Some SoCs support LP (Low-Power) UART for ultra-low power applications
- **Baud rate detection:** Automatic baud rate detection (ESP32, ESP32-S2 only)
- **Event callbacks:** Receive and error event callbacks
- **Configurable buffers:** Adjustable RX and TX buffer sizes

Note: In case that both pins, RX and TX are detached from UART, the driver will be stopped. Detaching may occur when, for instance, starting another peripheral using RX and TX pins, such as `Wire.begin(RX0, TX0)`.

UART Availability

The number of UART peripherals available varies by ESP32 SoC:

ESP32 SoC	HP UARTs	LP UARTs
ESP32	3	0
ESP32-S2	2	0
ESP32-S3	3	0
ESP32-C3	2	0
ESP32-C5	2	1
ESP32-C6	2	1
ESP32-H2	2	0
ESP32-P4	5	1

Note: * HP (High-Performance) UARTs are the standard UART peripherals * LP (Low-Power) UARTs are available on some SoCs for ultra-low power applications * UART0 is typically used for programming and debug output (Serial Monitor) * Additional UARTs (Serial1, Serial2, etc.) are available for general-purpose communication, including LP UARTs when available. The ESP32 Arduino Core automatically creates HardwareSerial objects for all available UARTs:

- Serial0 (or Serial) - UART0 (HP UART, typically used for programming and debug output)
- Serial1, Serial2, etc. - Additional HP UARTs (numbered sequentially)
- Additional Serial objects - LP UARTs, when available (numbered after HP UARTs)

Example: The ESP32-C6 has 2 HP UARTs and 1 LP UART. The Arduino Core creates Serial0 and Serial1 (HP UARTs) plus Serial2 (LP UART) HardwareSerial objects.

Important: LP UARTs can be used as regular UART ports, but they have fixed GPIO pins for RX, TX, CTS, and RTS. It is not possible to change the pins for LP UARTs using `setPins()`.

Arduino-ESP32 Serial API

begin Initializes the Serial port with the specified baud rate and configuration.

```
void begin(unsigned long baud, uint32_t config = SERIAL_8N1, int8_t rxPin = -1,
  ↳ int8_t txPin = -1, bool invert = false, unsigned long timeout_ms = 20000UL,
  ↳ uint8_t rxfifo_full_thrhd = 120);
```

- `baud` - Baud rate (bits per second). Common values: 9600, 115200, 230400, etc.
Special value: 0 enables baud rate detection (ESP32, ESP32-S2 only). The function will attempt to detect the baud rate for up to `timeout_ms` milliseconds. See the [Baud Rate Detection Example](#) for usage details.
- `config` - Serial configuration (data bits, parity, stop bits):
 - SERIAL_8N1 - 8 data bits, no parity, 1 stop bit (default)
 - SERIAL_8N2 - 8 data bits, no parity, 2 stop bits
 - SERIAL_8E1 - 8 data bits, even parity, 1 stop bit
 - SERIAL_8E2 - 8 data bits, even parity, 2 stop bits
 - SERIAL_8O1 - 8 data bits, odd parity, 1 stop bit
 - SERIAL_8O2 - 8 data bits, odd parity, 2 stop bits
 - SERIAL_7N1, SERIAL_7N2, SERIAL_7E1, SERIAL_7E2, SERIAL_7O1, SERIAL_7O2 - 7 data bits variants
 - SERIAL_6N1, SERIAL_6N2, SERIAL_6E1, SERIAL_6E2, SERIAL_6O1, SERIAL_6O2 - 6 data bits variants
 - SERIAL_5N1, SERIAL_5N2, SERIAL_5E1, SERIAL_5E2, SERIAL_5O1, SERIAL_5O2 - 5 data bits variants
- `rxPin` - RX pin number. Use -1 to keep the default pin or current pin assignment.
- `txPin` - TX pin number. Use -1 to keep the default pin or current pin assignment.
- `invert` - If true, inverts the RX and TX signal polarity.
- `timeout_ms` - Timeout in milliseconds for baud rate detection (when `baud` = 0). Default: 20000 ms (20 seconds).
- `rxfifo_full_thrhd` - RX FIFO full threshold (1-127 bytes). When the FIFO reaches this threshold, data is copied to the RX buffer. Default: 120 bytes.

Example:

```
// Basic initialization with default pins
Serial.begin(115200);

// Initialize with custom pins
Serial1.begin(9600, SERIAL_8N1, 4, 5);

// Initialize with baud rate detection (ESP32, ESP32-S2 only)
Serial.begin(0, SERIAL_8N1, -1, -1, false, 20000);
```

end Stops the Serial port and releases all resources.

```
void end(void);
```

This function disables the UART peripheral and frees all associated resources.

available Returns the number of bytes available for reading from the Serial port.

```
int available(void);
```

Returns: The number of bytes available in the RX buffer, or 0 if no data is available.

Example:

```
if (Serial.available() > 0) {
    char data = Serial.read();
}
```

availableForWrite Returns the number of bytes that can be written to the Serial port without blocking.

```
int availableForWrite(void);
```

Returns: The number of bytes that can be written to the TX buffer without blocking.

read Reads a single byte from the Serial port.

```
int read(void);
```

Returns: The byte read (0-255), or -1 if no data is available.

Example:

```
int data = Serial.read();
if (data != -1) {
    Serial.printf("Received: %c\n", (char)data);
}
```

read (buffer) Reads multiple bytes from the Serial port into a buffer.

```
size_t read(uint8_t *buffer, size_t size);
size_t read(char *buffer, size_t size);
```

- **buffer** - Pointer to the buffer where data will be stored
- **size** - Maximum number of bytes to read

Returns: The number of bytes actually read.

Example:


```
uint8_t buffer[64];
size_t bytesRead = Serial.read(buffer, sizeof(buffer));
Serial.printf("Read %lu bytes\n", (unsigned long)bytesRead);
```

readBytes Reads multiple bytes from the Serial port, blocking until the specified number of bytes is received or timeout occurs.

```
size_t readBytes(uint8_t *buffer, size_t length);
size_t readBytes(char *buffer, size_t length);
```

- `buffer` - Pointer to the buffer where data will be stored
- `length` - Number of bytes to read

Returns: The number of bytes actually read (may be less than `length` if timeout occurs).

Note: This function overrides `Stream::readBytes()` for better performance using ESP-IDF functions.

write Writes data to the Serial port.

```
size_t write(uint8_t);
size_t write(const uint8_t *buffer, size_t size);
size_t write(const char *buffer, size_t size);
size_t write(const char *s);
size_t write(unsigned long n);
size_t write(long n);
size_t write(unsigned int n);
size_t write(int n);
```

- Single byte: `write(uint8_t)` - Writes a single byte
- Buffer: `write(buffer, size)` - Writes multiple bytes from a buffer
- String: `write(const char *s)` - Writes a null-terminated string
- Number: `write(n)` - Writes a number as a single byte

Returns: The number of bytes written.

Example:

```
Serial.write('A');
Serial.write("Hello");
Serial.write(buffer, 10);
Serial.write(65); // Writes byte value 65
```

peek Returns the next byte in the RX buffer without removing it.

```
int peek(void);
```

Returns: The next byte (0-255), or -1 if no data is available.

Note: Unlike `read()`, `peek()` does not remove the byte from the buffer.

flush Waits for all data in the TX buffer to be transmitted.

```
void flush(void);
void flush(bool txOnly);
```

- `txOnly` - If `true`, only flushes the TX buffer. If `false` (default), also clears the RX buffer.

Note: This function blocks until all data in the TX buffer has been sent.

baudRate Returns the current baud rate of the Serial port.

```
uint32_t baudRate(void);
```

Returns: The configured baud rate in bits per second.

Note: When using baud rate detection (`begin(0)`), this function returns the detected baud rate, which may be slightly rounded (e.g., 115200 may return 115201).

updateBaudRate Updates the baud rate of an already initialized Serial port.

```
void updateBaudRate(unsigned long baud);
```

- `baud` - New baud rate

Note: This function can be called after `begin()` to change the baud rate without reinitializing the port.

setPins Sets or changes the RX, TX, CTS, and RTS pins for the Serial port.

```
bool setPins(int8_t rxPin, int8_t txPin, int8_t ctsPin = -1, int8_t rtsPin = -1);
```

- `rxPin` - RX pin number. Use `-1` to keep current pin.
- `txPin` - TX pin number. Use `-1` to keep current pin.
- `ctsPin` - CTS (Clear To Send) pin for hardware flow control. Use `-1` to keep current pin or disable.
- `rtsPin` - RTS (Request To Send) pin for hardware flow control. Use `-1` to keep current pin or disable.

Returns: `true` if pins are set successfully, `false` otherwise.

Note: This function can be called before or after `begin()`. When pins are changed, the previous pins are automatically detached.

setRxBufferSize Sets the size of the RX buffer.

```
size_t setRxBufferSize(size_t new_size);
```

- `new_size` - New RX buffer size in bytes

Returns: The actual buffer size set, or 0 on error.

Note: This function must be called **before** `begin()` to take effect. Default RX buffer size is 256 bytes.

setTxBufferSize Sets the size of the TX buffer.

```
size_t setTxBufferSize(size_t new_size);
```

- `new_size` - New TX buffer size in bytes

Returns: The actual buffer size set, or 0 on error.

Note: This function must be called **before** `begin()` to take effect. Default TX buffer size is 0 (no buffering).

setRxTimeout Sets the RX timeout threshold in UART symbol periods.

```
bool setRxTimeout(uint8_t symbols_timeout);
```

- `symbols_timeout` - Timeout threshold in UART symbol periods. Setting 0 disables timeout-based callbacks.

The timeout is calculated based on the current baud rate and serial configuration. For example:

- For `SERIAL_8N1` (10 bits per symbol), a timeout of 3 symbols at 9600 baud = $3 / (9600 / 10) = 3.125$ ms

- Maximum timeout is calculated automatically by ESP-IDF based on the serial configuration

Returns: `true` if timeout is set successfully, `false` otherwise.

Note: * When RX timeout occurs, the `onReceive()` callback is triggered * For ESP32 and ESP32-S2, when using `REF_TICK` clock source (baud rates ≤ 250000), RX timeout is limited to 1 symbol * To use higher RX timeout values on ESP32/ESP32-S2, set the clock source to APB using `setClockSource(UART_CLK_SRC_APB)` before `begin()`

setRxFIFOFull Sets the RX FIFO full threshold that triggers data transfer from FIFO to RX buffer.

```
bool setRxFIFOFull(uint8_t fifoBytes);
```

- `fifoBytes` - Number of bytes (1-127) that will trigger the FIFO full interrupt
When the UART FIFO reaches this threshold, data is copied to the RX buffer and the `onReceive()` callback is triggered.

Returns: `true` if threshold is set successfully, `false` otherwise.

Note: * Lower values (e.g., 1) provide byte-by-byte reception but consume more CPU time * Higher values (e.g., 120) provide better performance but introduce latency * Default value depends on baud rate: 1 byte for ≤ 115200 baud, 120 bytes for > 115200 baud

onReceive Sets a callback function that is called when data is received.

```
void onReceive(OnReceiveCb function, bool onlyOnTimeout = false);
```

- `function` - Callback function to call when data is received. Use `NULL` to disable the callback.
- `onlyOnTimeout` - If `true`, callback is only called on RX timeout. If `false` (default), callback is called on both FIFO full and RX timeout events.

Callback Signature:

```
typedef std::function<void(void)> OnReceiveCb;
```

Note: * When `onlyOnTimeout = false`, the callback is triggered when FIFO reaches the threshold (set by `setRxFIFOFull()`) or on RX timeout * When `onlyOnTimeout = true`, the callback is only triggered on RX timeout, ensuring all data in a stream is available at once * Using `onlyOnTimeout = true` may cause RX overflow if the RX buffer size is too small for the incoming data stream * The callback is executed in a separate task, allowing non-blocking data processing

Example:

```
void onReceiveCallback() {
    while (Serial1.available()) {
        char c = Serial1.read();
        Serial.print(c);
    }
}

void setup() {
    Serial1.begin(115200);
    Serial1.onReceive(onReceiveCallback);
}
```

onReceiveError Sets a callback function that is called when a UART error occurs.

```
void onReceiveError(OnReceiveErrorCb function);
```

- `function` - Callback function to call when an error occurs. Use `NULL` to disable the callback.

Callback Signature:

```
typedef std::function<void(hardwareSerial_error_t)> OnReceiveErrorCb;
```

Error Types:

- UART_NO_ERROR - No error
- UART_BREAK_ERROR - Break condition detected
- UART_BUFFER_FULL_ERROR - RX buffer is full
- UART_FIFO_OVF_ERROR - UART FIFO overflow
- UART_FRAME_ERROR - Frame error (invalid stop bit)
- UART_PARITY_ERROR - Parity error

Example:

```
void onErrorCallback(hardwareSerial_error_t error) {  
    Serial.printf("UART Error: %d\n", error);  
}  
  
void setup() {  
    Serial1.begin(115200);  
    Serial1.onReceiveError(onErrorCallback);  
}
```

eventQueueReset Clears all events in the event queue (events that trigger `onReceive()` and `onReceiveError()`).

```
void eventQueueReset(void);
```

This function can be useful in some use cases where you want to clear pending events.

setHwFlowCtrlMode Enables or disables hardware flow control using RTS and/or CTS pins.

```
bool setHwFlowCtrlMode(SerialHwFlowCtrl mode = UART_HW_FLOWCTRL_CTS_RTS, uint8_t  
↳ threshold = 64);
```

- mode - Hardware flow control mode:
 - UART_HW_FLOWCTRL_DISABLE (0x0) - Disable hardware flow control
 - UART_HW_FLOWCTRL_RTS (0x1) - Enable RX hardware flow control (RTS)
 - UART_HW_FLOWCTRL_CTS (0x2) - Enable TX hardware flow control (CTS)
 - UART_HW_FLOWCTRL_CTS_RTS (0x3) - Enable full hardware flow control (default)
- threshold - Flow control threshold (default: 64, which is half of the FIFO length)

Returns: `true` if flow control mode is set successfully, `false` otherwise.

Note: CTS and RTS pins must be set using `setPins()` before enabling hardware flow control.

setMode Sets the UART operating mode.

```
bool setMode(SerialMode mode);
```

- mode - UART mode:
 - UART_MODE_UART (0x00) - Regular UART mode (default)
 - UART_MODE_RS485_HALF_DUPLEX (0x01) - Half-duplex RS485 mode (RTS pin controls transceiver)
 - UART_MODE_IRDA (0x02) - IrDA UART mode
 - UART_MODE_RS485_COLLISION_DETECT (0x03) - RS485 collision detection mode (for testing)
 - UART_MODE_RS485_APP_CTRL (0x04) - Application-controlled RS485 mode (for testing)

Returns: `true` if mode is set successfully, `false` otherwise.

Note: For RS485 half-duplex mode, the RTS pin must be configured using `setPins()` to control the transceiver.

setIrdaDirection Sets the IrDA transmission direction (TX or RX mode). Can only be used after `setMode(UART_MODE_IRDA)` is called.

```
bool setIrdaDirection(esp32_uart_irda_direction_t irdaDirection);
```

- `irdaDirection` - Direction for IrDA mode:
 - `ESP32_UART_IRDA_TX` - IrDA TX mode (transmit only). The UART will transmit data in IrDA format. RX is disabled.
 - `ESP32_UART_IRDA_RX` - IrDA RX mode (receive only). The UART will receive data in IrDA format. TX is disabled.

Returns: `true` if IrDA mode direction is set successfully, `false` otherwise.

Note:

- IrDA mode works in exclusive directions: the UART can either transmit or receive, but not both simultaneously.
 - The `setMode(UART_MODE_IRDA)` function must be called before using `setIrdaDirection()`.
 - Switching between TX and RX modes can be done by calling `setIrdaDirection()` with different parameters.
 - The ESP32 UART hardware automatically handles IrDA pulse timing and encoding/decoding.
 - Hardware requirements: IR LED (950 nm) + resistor for TX, IR photodiode + amplifier for RX. No external SIR transceiver module needed.
-

Example:

```
Serial1.begin(9600, SERIAL_8N1, RX1, TX1);
Serial1.setMode(UART_MODE_IRDA);                                     // Enable IrDA mode

// Switch to TX mode
Serial1.setIrdaDirection(ESP32_UART_IRDA_TX);
Serial1.println("Transmit data");

// Switch to RX mode
Serial1.setIrdaDirection(ESP32_UART_IRDA_RX);
while (Serial1.available()) {
    char c = Serial1.read();
    Serial.print(c);
}
```

Related Examples:

- **IrdaMode_DualUART_Demo** - Single-board demonstration using two UARTs with internal loopback. Requires ESP32 with 3+ UARTs (ESP32, ESP32-S3, ESP32-P4). No external hardware needed. Ideal for testing IrDA mode functionality.
- **IrdaMode_TwoBoard_Demo** - Two-board peer-to-peer IrDA communication with user-selectable TX/RX modes via Serial Monitor. Works on any ESP32 variant. Requires IR LED (TX side) and IR receiver (RX side) connected between two boards. Demonstrates real infrared communication.

setClockSource Sets the UART clock source. Must be called **before** `begin()` to take effect.

```
bool setClockSource(SerialClkSrc clkSrc);
```

- `clkSrc` - Clock source:
 - `UART_CLK_SRC_DEFAULT` - Default clock source (varies by SoC)
 - `UART_CLK_SRC_APB` - APB clock (ESP32, ESP32-S2, ESP32-C3, ESP32-S3)

- UART_CLK_SRC_PLL - PLL clock (ESP32-C2, ESP32-C5, ESP32-C6, ESP32-C61, ESP32-H2, ESP32-P4)
- UART_CLK_SRC_XTAL - XTAL clock (ESP32-C2, ESP32-C3, ESP32-C5, ESP32-C6, ESP32-C61, ESP32-H2, ESP32-S3, ESP32-P4)
- UART_CLK_SRC_RTC - RTC clock (ESP32-C2, ESP32-C3, ESP32-C5, ESP32-C6, ESP32-C61, ESP32-H2, ESP32-S3, ESP32-P4)
- UART_CLK_SRC_REF_TICK - REF_TICK clock (ESP32, ESP32-S2)

Note: * Clock source availability varies by SoC. * PLL frequency varies by SoC: ESP32-C2 (40 MHz), ESP32-H2 (48 MHz), ESP32-C5/C6/C61/P4 (80 MHz). * ESP32-C5, ESP32-C6, ESP32-C61, and ESP32-P4 have LP UART that uses only RTC_FAST or XTAL/2 as clock source. * For ESP32 and ESP32-S2, REF_TICK is used by default for baud rates ≤ 250000 to avoid baud rate changes when CPU frequency changes, but this limits RX timeout to 1 symbol.

Returns: `true` if clock source is set successfully, `false` otherwise.

setRxInvert Enables or disables RX signal inversion.

```
bool setRxInvert(bool invert);
```

- `invert` - If `true`, inverts the RX signal polarity

Returns: `true` if inversion is set successfully, `false` otherwise.

setTxInvert Enables or disables TX signal inversion.

```
bool setTxInvert(bool invert);
```

- `invert` - If `true`, inverts the TX signal polarity

Returns: `true` if inversion is set successfully, `false` otherwise.

setCtsInvert Enables or disables CTS signal inversion.

```
bool setCtsInvert(bool invert);
```

- `invert` - If `true`, inverts the CTS signal polarity

Returns: `true` if inversion is set successfully, `false` otherwise.

setRtsInvert Enables or disables RTS signal inversion.

```
bool setRtsInvert(bool invert);
```

- `invert` - If `true`, inverts the RTS signal polarity

Returns: `true` if inversion is set successfully, `false` otherwise.

setDebugOutput Enables or disables debug output on this Serial port.

```
void setDebugOutput(bool enable);
```

- `enable` - If `true`, enables debug output (ESP-IDF log messages will be sent to this Serial port)

Note: By default, debug output is sent to UART0 (Serial0).

operator bool Returns whether the Serial port is initialized and ready.

```
operator bool() const;
```

Returns: `true` if the Serial port is initialized, `false` otherwise.

Example:

```
Serial1.begin(115200);  
while (!Serial1) {  
    delay(10); // Wait for Serial1 to be ready  
}
```

Testing and Helper Functions

The following HAL-level functions are available for testing UART functionality without external hardware connections. These functions use the ESP32 GPIO matrix to create internal loopback connections.

uart_internal_loopback Creates an internal loopback connection from a UART's TX signal to a specified RX pin, allowing testing without external wires.

```
void uart_internal_loopback(uint8_t uartNum, int8_t rxPin);
```

- `uartNum` - UART number (0, 1, 2, etc.)
- `rxPin` - GPIO pin number to receive the TX signal

Note: * This function uses the ESP32 GPIO matrix to internally connect the UART's TX output to the specified RX pin. * LP (Low-Power) UARTs are not supported for loopback. * This is useful for testing UART communication without physical connections.

Example:

```
Serial1.begin(115200);  
Serial1.setPins(RX1, TX1);  
uart_internal_loopback(1, RX1); // Connect TX1 to RX1 internally
```

uart_internal_hw_flow_ctrl_loopback Creates an internal loopback connection from a UART's RTS signal to a specified CTS pin, allowing hardware flow control testing without external wires.

```
void uart_internal_hw_flow_ctrl_loopback(uint8_t uartNum, int8_t ctsPin);
```

- `uartNum` - UART number (0, 1, 2, etc.)
- `ctsPin` - GPIO pin number (CTS pin) to receive the RTS signal

Note: * This function uses the ESP32 GPIO matrix to internally connect the UART's RTS output to the specified CTS pin. * LP (Low-Power) UARTs are not supported for hardware flow control loopback. * This is useful for testing hardware flow control (RTS/CTS) without physical connections. * Must be used together with `uart_internal_loopback()` for complete loopback testing with flow control.

Example:

```
Serial1.begin(115200);  
Serial1.setPins(RX1, TX1, CTS_PIN, RTS_PIN);  
Serial1.setHwFlowCtrlMode(UART_HW_FLOWCTRL_CTS_RTS);  
uart_internal_loopback(1, RX1); // Connect TX1 to RX1 internally  
uart_internal_hw_flow_ctrl_loopback(1, CTS_PIN); // Connect RTS to CTS internally
```

Serial Configuration Constants

The following constants are used for serial configuration in the `begin()` function:

Data Bits, Parity, Stop Bits

- SERIAL_5N1, SERIAL_5N2, SERIAL_5E1, SERIAL_5E2, SERIAL_5O1, SERIAL_5O2 - 5 data bits
- SERIAL_6N1, SERIAL_6N2, SERIAL_6E1, SERIAL_6E2, SERIAL_6O1, SERIAL_6O2 - 6 data bits
- SERIAL_7N1, SERIAL_7N2, SERIAL_7E1, SERIAL_7E2, SERIAL_7O1, SERIAL_7O2 - 7 data bits
- SERIAL_8N1, SERIAL_8N2, SERIAL_8E1, SERIAL_8E2, SERIAL_8O1, SERIAL_8O2 - 8 data bits

Where: * First number = data bits (5, 6, 7, or 8) * Letter = parity: N (None), E (Even), O (Odd) * Last number = stop bits (1 or 2)

Example Applications Baud Rate Detection Example:

```

/*
   This Sketch demonstrates how to detect and set the baud rate when the UART0 is
   ↪connected to
       some port that is sending data. It can be used with the Arduino IDE Serial
   ↪Monitor to send the data.

   Serial.begin(0) will start the baud rate detection. Valid range is 300 to
   ↪230400 baud.
   It will try to detect for 20 seconds, by default, while reading RX.
   This timeout of 20 seconds can be changed in the begin() function through <
   ↪<timeout_ms>> parameter:

   void HardwareSerial::begin(baud, config, rxPin, txPin, invert, <<timeout_ms>>,
   ↪rxfifo_full_thrhd)

   It is necessary that the other end sends some data within <<timeout_ms>>,
   ↪otherwise the detection won't work.

   IMPORTANT NOTE: baud rate detection seem to only work with ESP32 and ESP32-S2.
   In other other SoCs, it doesn't work.

*/

#include <Arduino.h>

// Open the Serial Monitor with testing baud start typing and sending characters
void setup() {
  Serial.begin(0); // it will try to detect the baud rate for 20 seconds

  Serial.print("\n==>The baud rate is ");
  Serial.println(Serial.baudRate());

  //after 20 seconds timeout, when not detected, it will return zero - in this
  ↪case, we set it back to 115200.
  if (Serial.baudRate() == 0) {
    // Trying to set Serial to a safe state at 115200
    Serial.end();
    Serial.begin(115200);
    Serial.setDebugOutput(true);
    delay(1000);
    log_e("Baud rate detection failed.");
  }
}

void loop() {}

```

OnReceive Callback Example:

```

/*

```

(continues on next page)

(continued from previous page)

```

This Sketch demonstrates how to use onReceive(callbackFunc) with HardwareSerial

void HardwareSerial::onReceive(OnReceiveCb function, bool onlyOnTimeout = false)

It is possible to register a UART callback function that will be called
every time that UART receives data and an associated interrupt is generated.

The receiving data interrupt can occur because of two possible events:

1- UART FIFO FULL: it happens when internal UART FIFO reaches a certain number
↳ of bytes.
    Its full capacity is 127 bytes. The FIFO Full threshold for the interrupt
↳ can be changed
    using HardwareSerial::setRxFIFOFull(uint8_t fifoFull).
    Default FIFO Full Threshold is set at the UART initialization using
↳ HardwareSerial::begin()
    This will depend on the baud rate set with when begin() is executed.
    For a baudrate of 115200 or lower, it is just 1 byte, mimicking original
↳ Arduino UART driver.
    For a baudrate over 115200 it will be 120 bytes for higher performance.
    Anyway it can be changed by the application at anytime.

2- UART RX Timeout: it happens, based on a timeout equivalent to a number of
↳ symbols at
    the current baud rate. If the UART line is idle for this timeout, it will
↳ raise an interrupt.
    This time can be changed by HardwareSerial::setRxTimeout(uint8_t rxTimeout)

When any of those two interrupts occur, IDF UART driver will copy FIFO data to
↳ its internal
    RingBuffer and then Arduino can read such data. At the same time, Arduino Layer
↳ will execute
    the callback function defined with HardwareSerial::onReceive().

<bool onlyOnTimeout> parameter (default false) can be used by the application
↳ to tell Arduino to
    only execute the callback when the second event above happens (Rx Timeout). At
↳ this time all
    received data will be available to be read by the Arduino application. But if
↳ the number of
    received bytes is higher than the FIFO space, it will generate an error of FIFO
↳ overflow.
    In order to avoid such problem, the application shall set an appropriate RX
↳ buffer size using
    HardwareSerial::setRxBufferSize(size_t new_size) before executing begin() for
↳ the Serial port.

In summary, HardwareSerial::onReceive() works like an RX Interrupt callback,
↳ that can be adjusted
    using HardwareSerial::setRxFIFOFull() and HardwareSerial::setRxTimeout().

*/

#include <Arduino.h>

// There are two ways to make this sketch work:
// By physically connecting the pins 4 and 5 and then create a physical UART
↳ loopback,
// Or by using the internal IO_MUX to connect the TX signal to the RX pin,
↳ creating the
// same loopback internally.

```

(continues on next page)

(continued from previous page)

```

#define USE_INTERNAL_PIN_LOOPBACK 1 // 1 uses the internal loopback, 0 for wiring
↳ pins 4 and 5 externally

#define DATA_SIZE 26 // 26 bytes is a lower than RX FIFO size (127 bytes)
#define BAUD 9600 // Any baudrate from 300 to 115200
#define TEST_UART 1 // Serial1 will be used for the loopback testing with
↳ different RX FIFO FULL values
#define RXPIN 4 // GPIO 4 => RX for Serial1
#define TXPIN 5 // GPIO 5 => TX for Serial1

uint8_t fifoFullTestCases[] = {120, 20, 5, 1};
// volatile declaration will avoid any compiler optimization when reading variable
↳ values
volatile size_t sent_bytes = 0, received_bytes = 0;

void onReceiveFunction(void) {
    // This is a callback function that will be activated on UART RX events
    size_t available = Serial1.available();
    received_bytes = received_bytes + available;
    Serial.printf("onReceive Callback:: There are %lu bytes available: ", (unsigned
↳ long)available);
    while (available--> 0) {
        Serial.print((char)Serial1.read());
    }
    Serial.println();
}

void setup() {
    // UART0 will be used to log information into Serial Monitor
    Serial.begin(115200);

    // UART1 will have its RX<->TX cross connected
    // GPIO4 <-> GPIO5 using external wire
    Serial1.begin(BAUD, SERIAL_8N1, RXPIN, TXPIN); // Rx = 4, Tx = 5 will work for
↳ ESP32, S2, S3 and C3
    #if USE_INTERNAL_PIN_LOOPBACK
        uart_internal_loopback(TEST_UART, RXPIN);
    #endif

    for (uint8_t i = 0; i < sizeof(fifoFullTestCases); i++) {
        Serial.printf("\n\n===== \nTest Case # %u\
↳ n===== \n", i + 1);
        // onReceive callback will be called on FIFO Full and RX timeout - default
↳ behavior
        testAndReport(fifoFullTestCases[i], false);
    }

    Serial.printf("\n\n===== \nTest Case #6\
↳ n===== \n");
    // onReceive callback will be called just on RX timeout - using onlyOnTimeout =
↳ true
    // FIFO Full parameter (5 bytes) won't matter for the execution of this test case
    // because onReceive() uses only RX Timeout to be activated
    testAndReport(5, true);
}

void loop() {}

void testAndReport(uint8_t fifoFull, bool onlyOnTimeOut) {
    // Let's send 125 bytes from Serial1 rx<->tx and mesaure time using different
↳ FIFO Full configurations

```

(continues on next page)

(continued from previous page)

```

received_bytes = 0;
sent_bytes = DATA_SIZE; // 26 characters

uint8_t dataSent[DATA_SIZE + 1];
dataSent[DATA_SIZE] = '\0'; // string null terminator, for easy printing.

// initialize all data
for (uint8_t i = 0; i < DATA_SIZE; i++) {
    dataSent[i] = 'A' + i; // fill it with characters A..Z
}

Serial.printf("\nTesting onReceive for receiving %lu bytes at %u baud, using RX
↳FIFO Full = %u.\n", (unsigned long)sent_bytes, BAUD, fifoFull);
if (onlyOnTimeout) {
    Serial.println("onReceive is called just on RX Timeout!");
} else {
    Serial.println("onReceive is called on both FIFO Full and RX Timeout events.");
}
Serial.flush(); // wait Serial FIFO to be
↳empty and then spend almost no time processing it
Serial1.setRxFIFOFull(fifoFull); // testing different
↳result based on FIFO Full setup
Serial1.onReceive(onReceiveFunction, onlyOnTimeout); // sets a RX callback
↳function for Serial 1

sent_bytes = Serial1.write(dataSent, DATA_SIZE); // ESP32 TX FIFO is about 128
↳bytes, 125 bytes will fit fine
Serial.printf("\nSent String: %s\n", dataSent);
while (received_bytes < sent_bytes) {
    // just wait for receiving all byte in the callback...
}

Serial.printf("\nIt has sent %lu bytes from Serial1 TX to Serial1 RX\n",
↳(unsigned long)sent_bytes);
Serial.printf("onReceive() has read a total of %lu bytes\n", (unsigned
↳long)received_bytes);
Serial.println("=====\nFinished!");

Serial1.onReceive(NULL); // resets/disables the RX callback function for Serial
↳1
}

```

RS485 Communication Example:

```

/*
    This Sketch demonstrates how to use the Hardware Serial peripheral to
    ↳communicate over an RS485 bus.

    Data received on the primary serial port is relayed to the bus acting as an
    ↳RS485 interface and vice versa.

    UART to RS485 translation hardware (e.g., MAX485, MAX33046E, ADM483) is assumed
    ↳to be configured in half-duplex
    mode with collision detection as described in
    https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/
    ↳peripherals/uart.html#circuit-a-collision-detection-circuit

    To use the script open the Arduino serial monitor (or alternative serial monitor
    ↳on the Arduino port). Then,
    using an RS485 transceiver, connect another serial monitor to the RS485 port.
    ↳Entering data on one terminal

```

(continues on next page)

(continued from previous page)

```

    should be displayed on the other terminal.
*/

#include <Arduino.h>
#include "hal/uart_types.h"

#define RS485_RX_PIN 16
#define RS485_TX_PIN 5
#define RS485_RTS_PIN 4

#define RS485 Serial1

void setup() {
    Serial.begin(115200);

    RS485.begin(9600, SERIAL_8N1, RS485_RX_PIN, RS485_TX_PIN);
    while (!RS485) {
        delay(10);
    }
    if (!RS485.setPins(-1, -1, -1, RS485_RTS_PIN)) {
        Serial.print("Failed to set RS485 pins");
    }

    // Certain versions of Arduino core don't define MODE_RS485_HALF_DUPLEX and so
    ↪ fail to compile.
    // By using UART_MODE_RS485_HALF_DUPLEX defined in hal/uart_types.h we work
    ↪ around this problem.
    // If using a newer IDF and Arduino core you can omit including hal/uart_types.h
    ↪ and use MODE_RS485_HALF_DUPLEX
    // defined in esp32-hal-uart.h (included during other build steps) instead.
    if (!RS485.setMode(UART_MODE_RS485_HALF_DUPLEX)) {
        Serial.print("Failed to set RS485 mode");
    }
}

void loop() {
    if (RS485.available()) {
        Serial.write(RS485.read());
    }
    if (Serial.available()) {
        RS485.write(Serial.read());
    }
}

```

IrDA Mode Examples:**Dual-UART Example (Single Board with 3+ UARTs):**

```

/*
    IrdaMode_DualUART_Demo.ino

    Demonstrates UART IrDA direction switching using two UARTs on a single ESP32
    ↪ board.
    This example is for SoCs with 3+ UARTs (ESP32, ESP32-S3, etc.).

    UART Configuration:
    - UART1: IrDA TX mode (transmitter)
    - UART2: IrDA RX mode (receiver)
    - Internal loopback: UART1 TX → UART2 RX (no external hardware needed)

    Functions used:
    - setMode(UART_MODE_IRDA) - Enable IrDA mode

```

(continues on next page)

(continued from previous page)

```

- setIrdaDirection(ESP32_UART_IRDA_TX / ESP32_UART_IRDA_RX) - Set direction

Hardware notes:
- Single ESP32 board with 3+ UARTs required
- Internal GPIO matrix loopback (no external connections needed)
- Perfect for development and testing IrDA functionality
*/

#include <Arduino.h>

#ifdef SOC_UART_HP_NUM
#if SOC_UART_HP_NUM >= 3
#define HAS_UART2 1
#else
#define HAS_UART2 0
#endif
#else
#define HAS_UART2 0
#endif
#endif

// UART pin definitions
#define UART1_RX_PIN RX1 // RX UART1 pin
#define UART1_TX_PIN TX1 // TX UART1 pin
#if HAS_UART2
#define UART2_RX_PIN RX2 // RX UART2 pin
#define UART2_TX_PIN TX2 // TX UART2 pin
#endif
#define BAUD_RATE 9600
#define RX_TIMEOUT 300

HardwareSerial &uart_tx = Serial1;
#if HAS_UART2
HardwareSerial &uart_rx = Serial2;
String received = ""; // Buffer for received message
const uint16_t MAX_MSG_SIZE = 64; // Limit message size
#endif

void setup() {
    Serial.begin(115200);
    delay(100);

    Serial.println("\n=== UART IrDA Dual-UART Demo (Single Board) ===");

    #if HAS_UART2
        Serial.println("✓ Board has 3+ UARTs: Using internal cross-UART loopback");
        setupDualUART();
        received.reserve(MAX_MSG_SIZE); // Pre-allocate for efficiency
    #else
        Serial.println("✗ ERROR: This example requires 3+ UARTs (UART0, UART1, UART2)");
        Serial.println("Your board only has 2 UARTs.");
        Serial.println("Use IrdaMode_TwoBoard_Demo.ino instead for two-board setup.");
        while (1) {
            delay(1000);
        }
    #endif
}

```

(continues on next page)

(continued from previous page)

```

#if HAS_UART2
void setupDualUART() {
    // Initialize UART1 in TX mode
    uart_tx.begin(BAUD_RATE, SERIAL_8N1, UART1_RX_PIN, UART1_TX_PIN);
    if (!uart_tx.setMode(UART_MODE_IRDA)) {
        Serial.println("ERROR: Failed to set UART1 to IRDA mode");
        while (1) {
            delay(100);
        }
    }
    if (!uart_tx.setIrdaDirection(ESP32_UART_IRDA_TX)) {
        Serial.println("ERROR: Failed to set UART1 to IRDA TX mode");
        while (1) {
            delay(100);
        }
    }
    Serial.println("✓ UART1 initialized in IRDA TX mode");

    // Initialize UART2 in RX mode
    uart_rx.begin(BAUD_RATE, SERIAL_8N1, UART2_RX_PIN, UART2_TX_PIN);
    if (!uart_rx.setMode(UART_MODE_IRDA)) {
        Serial.println("ERROR: Failed to set UART2 to IRDA mode");
        while (1) {
            delay(100);
        }
    }
    if (!uart_rx.setIrdaDirection(ESP32_UART_IRDA_RX)) {
        Serial.println("ERROR: Failed to set UART2 to IRDA RX mode");
        while (1) {
            delay(100);
        }
    }
    Serial.println("✓ UART2 initialized in IRDA RX mode");

    // Set up internal loopback: UART1 TX -> UART2 RX
    uart_internal_loopback(1, UART2_RX_PIN);
    Serial.println("✓ Internal loopback configured (UART1 TX → UART2 RX)");

    Serial.println("\nDemo will alternate between:");
    Serial.println("1) UART1 transmits message in IRDA TX mode");
    Serial.println("2) UART2 receives message in IRDA RX mode");
    Serial.println();
}
#endif

void loop() {
    static uint32_t counter = 0;

    #if HAS_UART2
        loopDualUART(counter);
    #endif

    counter++;
    delay(500);
}

#if HAS_UART2
void loopDualUART(uint32_t counter) {
    // UART1 TX mode: transmit
    Serial.printf("TX (UART1) -> PING %lu\n", (unsigned long)counter);

```

(continues on next page)

(continued from previous page)

```

uart_tx.printf("PING %lu\n", (unsigned long)counter);
uart_tx.flush();
delay(100);

// UART2 RX mode: receive
uint32_t start = millis();
bool gotReply = false;

while ((millis() - start) < RX_TIMEOUT) {
    while (uart_rx.available()) {
        char c = (char)uart_rx.read();
        if (received.length() < MAX_MSG_SIZE) {
            received += c;
        } else {
            Serial.printf("WARNING: Received message truncated (exceeds max size = %u)\n", MAX_MSG_SIZE);
        }
        gotReply = true;
    }
    delay(1);
}

if (gotReply) {
    Serial.print("RX (UART2) <- ");
    Serial.println(received);
    received = ""; // Clear buffer for new message
} else {
    Serial.println("RX (UART2) <- (no data received)");
}
}
#endif

```

Two-Board Example (Peer-to-Peer Communication):

```

/*
  IrdaMode_TwoBoard_Demo.ino

  Demonstrates UART IrDA communication between two ESP32 boards.
  This example is for SoCs with only 2 UARTs (UART0 + UART1).

  Setup Required:
  - 2 ESP32 boards connected via infrared (IR LED on one, IR receiver on other)
  - Serial Monitor for mode selection
  - User selects TX or RX mode at startup

  UART Configuration:
  - UART0 (Serial): Used for Serial Monitor communication and mode selection
  - UART1 (Serial1): Configured for IrDA in user-selected mode (TX or RX)

  Features:
  - Simple mode selection via Serial Monitor
  - One board transmits (TX mode), other receives (RX mode)
  - Demonstrates bidirectional IrDA communication concept
  - Real infrared hardware required (IR LED for TX, IR receiver for RX)

  Hardware Requirements:
  - Board 1: IR LED (950nm) + resistor (100-330Ω) connected to TX pin
  - Board 2: IR photodiode + amplifier circuit connected to RX pin
  - Align IR optical components for line-of-sight communication
*/

```

(continues on next page)

(continued from previous page)

```

#include <Arduino.h>

// UART1 pin definitions
#define UART1_RX_PIN RX1 // RX UART1 pin
#define UART1_TX_PIN TX1 // TX UART1 pin
#define BAUD_RATE 9600

// IrDA mode selection
#define MODE_NOT_SET 0
#define MODE_TX 1
#define MODE_RX 2

HardwareSerial &irda_uart = Serial1;
uint8_t irda_mode = MODE_NOT_SET;

String received = ""; // Buffer for received message
const uint16_t MAX_MSG_SIZE = 64; // Limit message size

void setup() {
  Serial.begin(115200);
  delay(100);

  Serial.println("\n=== UART IrDA Two-Board Demo ===");
  Serial.println("This example requires 2 ESP32 boards connected via infrared\n");

  selectMode();
}

void selectMode() {
  Serial.println("Select IrDA mode:");
  Serial.println("  T - Transmit mode (this board sends IrDA data)");
  Serial.println("  R - Receive mode (this board receives IrDA data)");
  Serial.println("\nWait for mode selection from Serial Monitor...");

  // Wait for user input from Serial Monitor
  while (irda_mode == MODE_NOT_SET) {
    if (Serial.available()) {
      char cmd = Serial.read();
      if (cmd == 'T' || cmd == 't') {
        irda_mode = MODE_TX;
        Serial.println("\n✓ Mode selected: TRANSMIT (TX)");
        setupTXMode();
        break;
      } else if (cmd == 'R' || cmd == 'r') {
        irda_mode = MODE_RX;
        received.reserve(MAX_MSG_SIZE); // Pre-allocate for efficiency
        Serial.println("\n✓ Mode selected: RECEIVE (RX)");
        setupRXMode();
        break;
      }
    }
    delay(10);
  }
}

void setupTXMode() {
  // Initialize UART1 for IrDA TX
  irda_uart.begin(BAUD_RATE, SERIAL_8N1, UART1_RX_PIN, UART1_TX_PIN);
  if (!irda_uart.setMode(UART_MODE_IRDA)) {
    Serial.println("ERROR: Failed to set IRDA mode");
    while (1) {

```

(continues on next page)

(continued from previous page)

```

        delay(100);
    }
}
if (!irda_uart.setIrdaDirection(ESP32_UART_IRDA_TX)) {
    Serial.println("ERROR: Failed to set IRDA TX mode");
    while (1) {
        delay(100);
    }
}
Serial.println("UART1 configured in IRDA TX mode");
Serial.println("Transmitting IrDA frames...\n");
}

void setupRXMode() {
    // Initialize UART1 for IrDA RX
    irda_uart.begin(BAUD_RATE, SERIAL_8N1, UART1_RX_PIN, UART1_TX_PIN);
    if (!irda_uart.setMode(UART_MODE_IRDA)) {
        Serial.println("ERROR: Failed to set IRDA mode");
        while (1) {
            delay(100);
        }
    }
    if (!irda_uart.setIrdaDirection(ESP32_UART_IRDA_RX)) {
        Serial.println("ERROR: Failed to set IRDA RX mode");
        while (1) {
            delay(100);
        }
    }
    Serial.println("UART1 configured in IRDA RX mode");
    Serial.println("Waiting for IrDA frames...\n");
}

void loop() {
    if (irda_mode == MODE_TX) {
        loopTXMode();
    } else if (irda_mode == MODE_RX) {
        loopRXMode();
    }
}

void loopTXMode() {
    static uint32_t counter = 0;

    // Send IrDA data
    Serial.printf("TX -> Sending frame %lu\n", (unsigned long)counter);
    irda_uart.printf("FRAME_%lu", (unsigned long)counter);
    irda_uart.print("\n"); // Add newline as delimiter
    irda_uart.flush();

    counter++;
    delay(1000); // Send new frame every 1 second
}

// Receive IrDA data
void loopRXMode() {
    uint32_t start = millis();
    bool gotData = false;

    while ((millis() - start) < 5000) { // Wait 5 seconds for data
        while (irda_uart.available()) {
            char c = (char)irda_uart.read();

```

(continues on next page)

(continued from previous page)

```

    if (c == '\n') {
        // End of frame
        if (received.length() > 0) {
            Serial.printf("RX <- Received: %s\n", received.c_str());
            gotData = true;
        }
        received = "";
    } else if (c > 31 && c < 127) {
        // Printable character
        if (received.length() < MAX_MSG_SIZE) {
            received += c;
        } else {
            Serial.printf("So far RX <- Received: %s%c\n", received.c_str(), c);
            Serial.printf("WARNING: Received message truncated (exceeds max size =
↪ %u)\r\n", MAX_MSG_SIZE);
            received = ""; // Clear buffer for new message
        }
    }
}

if (gotData) {
    // Optional: Send acknowledgment back
    // irda_uart.println("ACK");
    // irda_uart.flush();
    delay(100);
    break;
}
delay(1);
}

if (!gotData) {
    Serial.println("RX <- (waiting for IrDA frames from peer TX mode)");
}

delay(500);
}

```

Hardware Flow Control Example:

```

/*
Hardware Flow Control Demo for ESP32

This sketch demonstrates UART hardware flow control using RTS (Request To Send)
and CTS (Clear To Send) signals with UART1 (HardwareSerial Serial1).

CONFIGURATION:
=====

Set USE_INTERNAL_MATRIX_PIN_LOOPBACK to 1 for internal GPIO matrix connections
(no external wires needed). Set to 0 to use external wire connections.

PIN CONNECTIONS (when USE_INTERNAL_MATRIX_PIN_LOOPBACK = 0):
=====

For basic loopback with hardware flow control:
- Connect GPIO2 (RTS1) to GPIO4 (CTS1) - Flow control loopback
- Connect TX1 pin to RX1 pin - Data loopback

For GPIO-controlled flow control demonstration:
- Connect TX1 pin to RX1 pin - Data loopback
- Connect GPIO2 (RTS1) to GPIO5 (GPIO_RTS_MONITOR) - Monitor RTS state

```

(continues on next page)

(continued from previous page)

- Connect GPIO4 (CTS1) to GPIO13 (GPIO_CTS_CTRL) - Control CTS signal
- Use GPIO13 to manually control CTS signal (LOW = allow, HIGH = block)

HARDWARE FLOW CONTROL EXPLANATION:

=====

RTS (Request To Send):

- Output signal from UART (GPIO2 in this example)
- Asserted LOW when UART is ready to receive data (RX buffer has space)
- De-asserted HIGH when RX buffer is getting full (threshold reached)

CTS (Clear To Send):

- Input signal to UART (GPIO4 in this example)
- UART will only transmit when CTS is LOW (asserted)
- UART will pause transmission when CTS is HIGH (de-asserted)

OPERATION:

=====

The sketch demonstrates:

- Periodic transmission of messages every second
- Automatic flow control when USE_GPIO_CONTROL = false
- Manual CTS control when USE_GPIO_CONTROL = true
- Loopback reception of transmitted data
- Status monitoring of RTS/CTS pin states

NOTE: When USE_INTERNAL_MATRIX_PIN_LOOPBACK = 1, no external connections are needed as the ESP32 GPIO matrix handles the loopback internally.

*/

#include <Arduino.h>

```
// setting it to 1 will allow internal matrix pin connection for RX1<->TX1 and RTS1
<->CTS1
```

```
// otherwise it needs a wire for cross connecting the pins
```

#define USE_INTERNAL_MATRIX_PIN_LOOPBACK 1

// Pin definitions for UART1

#define UART1_RX_PIN RX1 // Default GPIO - UART1 RX pin

#define UART1_TX_PIN TX1 // Default GPIO - UART1 TX pin

#define UART1_RTS_PIN 2 // GPIO2 - UART1 RTS pin (output from UART)

#define UART1_CTS_PIN 4 // GPIO4 - UART1 CTS pin (input to UART)

// Optional: GPIO pins for manual flow control demonstration

// If using GPIO-controlled flow control, connect:

// - RTS1 to GPIO_RTS_MONITOR (to monitor RTS state)

// - CTS1 to GPIO_CTS_CTRL (to control CTS signal)

#define GPIO_RTS_MONITOR 5 // GPIO5 - Monitor RTS signal (connect RTS1 to this)

#define GPIO_CTS_CTRL 13 // GPIO13 - Control CTS signal (connect CTS1 to this)

// Set to true to use GPIO-controlled flow control, false for simple loopback

```
// Note: this control will be overridden by USE_INTERNAL_MATRIX_PIN_LOOPBACK when
<->it is 1
```

#define USE_GPIO_CONTROL false

// Variables for demonstration

unsigned long lastSendTime = 0;

unsigned long lastStatusTime = 0;

const unsigned long sendInterval = 1000; // Send data every 1 second

const unsigned long statusInterval = 2000; // Print status every 2 seconds

int sendCounter = 0;

(continues on next page)

(continued from previous page)

```

void printPinStatus() {
    Serial.println("\n== UART1 Pin Status ==");
    Serial.printf("RX Pin (GPIO%d): Receiving data\n", UART1_RX_PIN);
    Serial.printf("TX Pin (GPIO%d): Transmitting data\n", UART1_TX_PIN);

    if (USE_GPIO_CONTROL) {
        // Read RTS state from monitor GPIO (connected to RTS1)
        bool rtsState = digitalRead(GPIO_RTS_MONITOR);
        Serial.printf("RTS Pin (GPIO%d): %s (LOW = ready to receive)\n", UART1_RTS_PIN,
        ↪ rtsState == LOW ? "LOW (Ready)" : "HIGH (Busy)");

        // Read CTS state from control GPIO (connected to CTS1)
        bool ctsState = digitalRead(GPIO_CTS_CTRL);
        Serial.printf("CTS Pin (GPIO%d): %s (LOW = can transmit)\n", UART1_CTS_PIN, ↪
        ↪ ctsState == LOW ? "LOW (Clear)" : "HIGH (Blocked)");
    } else {
        Serial.printf("RTS Pin (GPIO%d): Hardware controlled (LOW = ready to receive)\n
        ↪", UART1_RTS_PIN);
        Serial.printf("CTS Pin (GPIO%d): Hardware controlled (LOW = can transmit)\n", ↪
        ↪ UART1_CTS_PIN);
        Serial.println("Note: RTS/CTS pins are hardware-controlled. Connect RTS1 to ↪
        ↪ CTS1 for loopback.");
    }

    Serial.printf("Available for write: %d bytes\n", Serial1.availableForWrite());
    Serial.printf("Available to read: %d bytes\n", Serial1.available());
    Serial.println("=====\n");
}

void setup() {
    // Initialize Serial (USB) for debugging
    Serial.begin(115200);
    delay(1000);

    Serial.println("\n\n=====");
    Serial.println("ESP32 Hardware Flow Control Demo");
    Serial.println("=====\n");

    // Configure GPIOs for flow control (only if using GPIO-controlled mode)
    if (USE_GPIO_CONTROL) {
        // Configure CTS control GPIO - this will control the CTS signal
        pinMode(GPIO_CTS_CTRL, OUTPUT);
        digitalWrite(GPIO_CTS_CTRL, LOW); // Start with CTS LOW (clear to send)

        // Configure RTS monitor GPIO - this will monitor the RTS signal
        pinMode(GPIO_RTS_MONITOR, INPUT);
        Serial.println("Using GPIO-controlled flow control mode");
    } else {
        Serial.println("Using hardware-controlled flow control (simple loopback)");
    }

    // Initialize UART1 with hardware flow control
    Serial.println("Initializing UART1...");

    // Begin UART1 with 115200 baud, 8N1 configuration
    Serial1.begin(115200);

    // Set all pins for UART1
    if (!Serial1.setPins(UART1_RX_PIN, UART1_TX_PIN, UART1_CTS_PIN, UART1_RTS_PIN)) {
        Serial.println("ERROR: Failed to set CTS and RTS UART1 pins!");
    }
}

```

(continues on next page)

(continued from previous page)

```

    while (1) {
        delay(1000);
    }
}

Serial.println("Enabling hardware flow control...");
if (!Serial1.setHwFlowCtrlMode(UART_HW_FLOWCTRL_CTS_RTS, 64)) {
    Serial.println("ERROR: Failed to enable hardware flow control!");
    while (1) {
        delay(1000);
    }
}

#if USE_INTERNAL_MATRIX_PIN_LOOPBACK
    uart_internal_loopback(1, UART1_RX_PIN);
    uart_internal_hw_flow_ctrl_loopback(1, UART1_CTS_PIN);
#endif

    // Diagnostic: Check initial state after enabling flow control
    Serial.println("\nPost-initialization diagnostics:");
    Serial.printf("    Serial1.available(): %d bytes\n", Serial1.available());
    Serial.printf("    Serial1.availableForWrite(): %d bytes\n", Serial1.
↪availableForWrite());
    if (USE_GPIO_CONTROL) {
        Serial.printf("    GPIO%d (CTS control): %s\n", GPIO_CTS_CTRL, digitalRead(GPIO_
↪CTS_CTRL) == LOW ? "LOW" : "HIGH");
        Serial.printf("    GPIO%d (RTS monitor): %s\n", GPIO_RTS_MONITOR,
↪digitalRead(GPIO_RTS_MONITOR) == LOW ? "LOW" : "HIGH");
    }
    Serial.println();

    Serial.println("UART1 initialized successfully!");
    Serial.println("Hardware flow control: ENABLED (RTS + CTS)");
    Serial.printf("RX Pin: GPIO%d\n", UART1_RX_PIN);
    Serial.printf("TX Pin: GPIO%d\n", UART1_TX_PIN);
    Serial.printf("RTS Pin: GPIO%d (output from UART)\n", UART1_RTS_PIN);
    Serial.printf("CTS Pin: GPIO%d (input to UART)\n", UART1_CTS_PIN);

    #if USE_INTERNAL_MATRIX_PIN_LOOPBACK
        Serial.println("\nNO EXTERNAL PIN CONNECTIONS ARE REQUIRED:");
        Serial.println("-----");
        Serial.println("Internal GPIO Matrix connection with flow control mode:");
        Serial.printf("    1. Automatic Internal Connection of GPIO%d (TX1) to GPIO%d
↪(RX1) - Loopback\n", UART1_TX_PIN, UART1_RX_PIN);
        Serial.printf("    2. Automatic Internal Connection of GPIO%d (RTS1) to GPIO%d
↪(CTS1) - Flow control loopback\n", UART1_RTS_PIN, UART1_CTS_PIN);
        Serial.println("\n    Note: In this mode, RTS/CTS are automatically controlled by
↪hardware.");
        Serial.println("    RTS goes LOW when ready to receive, HIGH when buffer is full.
↪");
        Serial.println("    CTS must be LOW for transmission to proceed.");
    #else
        Serial.println("\nPIN CONNECTIONS REQUIRED:");
        Serial.println("-----");
        if (USE_GPIO_CONTROL) {
            Serial.println("GPIO-controlled flow control mode:");
            Serial.printf("    1. Connect GPIO%d (TX1) to GPIO%d (RX1) - Loopback\n", UART1_
↪TX_PIN, UART1_RX_PIN);
            Serial.printf("    2. Connect GPIO%d (RTS1) to GPIO%d - Monitor RTS state\n",
↪UART1_RTS_PIN, GPIO_RTS_MONITOR);
            Serial.printf("    3. Connect GPIO%d (CTS1) to GPIO%d - Control CTS signal\n",
↪

```

(continues on next page)

(continued from previous page)

```

    ↪UART1_CTS_PIN, GPIO_CTS_CTRL);
    } else {
        Serial.println("Hardware-controlled flow control (simple loopback):");
        Serial.printf(" 1. Connect GPIO%d (TX1) to GPIO%d (RX1) - Loopback\n", UART1_
    ↪TX_PIN, UART1_RX_PIN);
        Serial.printf(" 2. Connect GPIO%d (RTS1) to GPIO%d (CTS1) - Flow control
    ↪loopback\n", UART1_RTS_PIN, UART1_CTS_PIN);
        Serial.println("\n Note: In this mode, RTS/CTS are automatically controlled
    ↪by hardware.");
        Serial.println("  RTS goes LOW when ready to receive, HIGH when buffer is full.
    ↪");
        Serial.println("  CTS must be LOW for transmission to proceed.");
    }
#endif
    Serial.println("\nStarting demonstration in 2 seconds...\n");
    delay(2000);
}

void loop() {
    unsigned long currentTime = millis();

    // Print status periodically
    if (currentTime - lastStatusTime >= statusInterval) {
        lastStatusTime = currentTime;
        printPinStatus();

        // Demonstrate flow control by toggling CTS (only in GPIO-controlled mode)
        if (USE_GPIO_CONTROL) {
            static bool ctsState = false;
            ctsState = !ctsState;

            if (ctsState) {
                Serial.println(">>> Blocking transmission (CTS HIGH)...");
                digitalWrite(GPIO_CTS_CTRL, HIGH); // Block transmission
            } else {
                Serial.println(">>> Allowing transmission (CTS LOW)...");
                digitalWrite(GPIO_CTS_CTRL, LOW); // Allow transmission
            }
        }
    }

    // Send data periodically
    if (currentTime - lastSendTime >= sendInterval) {
        lastSendTime = currentTime;
        sendCounter++;

        // Check if we can transmit (CTS must be LOW)
        // In GPIO-controlled mode, check the control GPIO; otherwise hardware handles
    ↪it
        bool canTransmit = true;
        if (USE_GPIO_CONTROL) {
            canTransmit = (digitalRead(GPIO_CTS_CTRL) == LOW);
        }

        if (canTransmit) {
            char message[64];
            snprintf(message, sizeof(message), "Message #%d: Hello from UART1! Time: %lu
    ↪ms\r\n", sendCounter, currentTime);

            Serial.print("Sending: ");
            Serial.print(message);

```

(continues on next page)

(continued from previous page)

```

    size_t bytesWritten = Serial1.write((const uint8_t *)message, ↵
↵strlen(message));
    Serial.printf(" -> Written: %lu bytes\n", (unsigned long)bytesWritten);

    // Flush to ensure data is sent
    Serial1.flush();
} else {
    Serial.println("!!! Transmission blocked - CTS is HIGH !!!");
}
}

// Read and echo received data
if (Serial1.available()) {
    Serial.print("Received: ");
    while (Serial1.available()) {
        char c = Serial1.read();
        Serial.write(c);
    }
    Serial.println();
}

// Small delay to prevent tight loop
delay(10);
}

```

Complete list of [Serial examples](#).

2.3.23 SigmaDelta

About

ESP32 provides a second-order sigma delta modulation module and 8 (4 for ESP32-C3) independent modulation channels. The channels are capable to output 1-bit signals (output index: 100 ~ 107) with sigma delta modulation.

ESP32 SoC	Number of SigmaDelta channels
ESP32	8
ESP32-S2	8
ESP32-S3	8
ESP32-C3	4
ESP32-C6	4
ESP32-H2	4

Arduino-ESP32 SigmaDelta API

sigmaDeltaAttach This function is used to set up the SigmaDelta channel with the selected frequency and attach it to the selected pin.

```
bool sigmaDeltaAttach(uint8_t pin, uint32_t freq);
```

- pin select GPIO pin.
- freq select frequency.
 - range is 1-14 bits (1-20 bits for ESP32).

This function returns `true` if the configuration was successful. If `false` is returned, an error occurred and the SigmaDelta channel was not configured.

sigmaDeltaWrite This function is used to set duty for the SigmaDelta pin.

```
bool sigmaDeltaWrite(uint8_t pin, uint8_t duty);
```

- pin selects the GPIO pin.
- duty selects the duty to be set for selected pin.

This function returns `true` if setting the duty was successful. If `false` is returned, error occurs and duty was not set.

sigmaDeltaDetach This function is used to detach a pin from SigmaDelta and deinitialize the channel that was attached to the pin.

```
bool sigmaDeltaDetach(uint8_t pin);
```

- pin select GPIO pin.

This function returns `true` if detaching was successful. If `false` is returned, an error occurred and pin was not detached.

Example Applications Here is example use of SigmaDelta:

```
#include <Arduino.h>

void setup() {
  //setup on pin 18 with frequency 312500 Hz
  sigmaDeltaAttach(18, 312500);
  //set pin 18 to off
  sigmaDeltaWrite(18, 0);
}

void loop() {
  //slowly ramp-up the value
  //will overflow at 256
  static uint8_t i = 0;
  sigmaDeltaWrite(18, i++);
  delay(100);
}
```

2.3.24 SPI

About

For some APIs, the reference to be used is the same as the Arduino Core.

Arduino API Reference

[SPI Reference](#)

[SPI Description](#)

Example

To get started with SPI, you can try:

SPI Multiple Buses

```

/* The ESP32 has four SPi buses, however as of right now only two of
 * them are available to use, HSPI and VSPI. Simply using the SPI API
 * as illustrated in Arduino examples will use VSPI, leaving HSPI unused.
 *
 * However if we simply initialize two instance of the SPI class for both
 * of these buses both can be used. However when just using these the Arduino
 * way only will actually be outputting at a time.
 *
 * Logic analyzer capture is in the same folder as this example as
 * "multiple_bus_output.png"
 *
 * created 30/04/2018 by Alistair Symonds
 */
#include <Arduino.h>
#include <SPI.h>

// Define ALTERNATE_PINS to use non-standard GPIO pins for SPI bus

#ifdef ALTERNATE_PINS
#define VSPI_MISO 2
#define VSPI_MOSI 4
#define VSPI_SCLK 0
#define VSPI_SS 33

#define HSPI_MISO 26
#define HSPI_MOSI 27
#define HSPI_SCLK 25
#define HSPI_SS 32
#else
#define VSPI_MISO MISO
#define VSPI_MOSI MOSI
#define VSPI_SCLK SCK
#define VSPI_SS SS

#define HSPI_MISO 12
#define HSPI_MOSI 13
#define HSPI_SCLK 14
#define HSPI_SS 15
#endif

#if !defined(CONFIG_IDF_TARGET_ESP32)
#define VSPI FSPI
#endif

static const int spiClk = 1000000; // 1 MHz

//uninitialized pointers to SPI objects
SPIClass *vspi = NULL;
SPIClass *hspi = NULL;

void setup() {
    //initialize two instances of the SPIClass attached to VSPI and HSPI respectively
    vspi = new SPIClass(VSPI);
    hspi = new SPIClass(HSPI);

    //clock miso mosi ss

#ifdef ALTERNATE_PINS
    //initialize vspi with default pins
    //SCLK = 18, MISO = 19, MOSI = 23, SS = 5
    vspi->begin();

```

(continues on next page)

(continued from previous page)

```

#else
    //alternatively route through GPIO pins of your choice
    vspi->begin(VSPI_SCLK, VSPI_MISO, VSPI_MOSI, VSPI_SS); //SCLK, MISO, MOSI, SS
#endif

#ifndef ALTERNATE_PINS
    //initialize hspi with default pins
    //SCLK = 14, MISO = 12, MOSI = 13, SS = 15
    hspi->begin();
#else
    //alternatively route through GPIO pins
    hspi->begin(HSPI_SCLK, HSPI_MISO, HSPI_MOSI, HSPI_SS); //SCLK, MISO, MOSI, SS
#endif

    //set up slave select pins as outputs as the Arduino API
    //doesn't handle automatically pulling SS low
    pinMode(vspi->pinSS(), OUTPUT); //VSPI SS
    pinMode(hspi->pinSS(), OUTPUT); //HSPI SS
}

// the loop function runs over and over again until power down or reset
void loop() {
    //use the SPI buses
    spiCommand(vspi, 0b01010101); // junk data to illustrate usage
    spiCommand(hspi, 0b11001100);
    delay(100);
}

void spiCommand(SPIClass *spi, byte data) {
    //use it as you would the regular arduino SPI API
    spi->beginTransaction(SPISettings(spiClk, MSBFIRST, SPI_MODE0));
    digitalWrite(spi->pinSS(), LOW); //pull SS low to prep other end for transfer
    spi->transfer(data);
    digitalWrite(spi->pinSS(), HIGH); //pull ss high to signify end of data transfer
    spi->endTransaction();
}

```

2.3.25 Timer

About

The ESP32 SoCs contains from 2 to 4 hardware timers. They are all 64-bit (54-bit for ESP32-C3) generic timers based on 16-bit pre-scalers and 64-bit (54-bit for ESP32-C3) up / down counters which are capable of being auto-reloaded.

ESP32 SoC	Number of timers
ESP32	4
ESP32-S2	4
ESP32-S3	4
ESP32-C3	2
ESP32-C6	2
ESP32-H2	2

Arduino-ESP32 Timer API

timerBegin This function is used to configure the timer. After successful setup the timer will automatically start.

```
hw_timer_t * timerBegin(uint32_t frequency);
```

- `frequency` select timer frequency in Hz. Sets how quickly the timer counter is “ticking” .

This function will return `timer` structure if configuration is successful. If `NULL` is returned, error occurs and the timer was not configured.

timerEnd This function is used to end timer.

```
void timerEnd(hw_timer_t * timer);
```

- `timer` timer struct.

timerStart This function is used to start counter of the timer.

```
void timerStart(hw_timer_t * timer);
```

- `timer` timer struct.

timerStop This function is used to stop counter of the timer.

```
void timerStop(hw_timer_t * timer);
```

- `timer` timer struct.

timerRestart This function is used to restart counter of the timer.

```
void timerRestart(hw_timer_t * timer);
```

- `timer` timer struct.

timerWrite This function is used to set counter value of the timer.

```
void timerWrite(hw_timer_t * timer, uint64_t val);
```

- `timer` timer struct.
- `val` counter value to be set.

timerRead This function is used to read counter value of the timer.

```
uint64_t timerRead(hw_timer_t * timer);
```

- `timer` timer struct.

This function will return `counter` value of the timer.

timerReadMicros This function is used to read counter value in microseconds of the timer.

```
uint64_t timerReadMicros(hw_timer_t * timer);
```

- `timer` timer struct.

This function will return `counter` value of the timer in microseconds.

timerReadMillis This function is used to read counter value in milliseconds of the timer.

```
uint64_t timerReadMillis(hw_timer_t * timer);
```

- timer timer struct.

This function will return counter value of the timer in milliseconds.

timerReadSeconds This function is used to read counter value in seconds of the timer.

```
double timerReadSeconds(hw_timer_t * timer);
```

- timer timer struct.

This function will return counter value of the timer in seconds.

timerGetFrequency This function is used to get resolution in Hz of the timer.

```
uint16_t timerGetFrequency(hw_timer_t * timer);
```

- timer timer struct.

This function will return frequency in Hz of the timer.

timerAttachInterrupt This function is used to attach interrupt to timer.

```
void timerAttachInterrupt(hw_timer_t * timer, void (*userFunc)(void));
```

- timer timer struct.
- userFunc function to be called when interrupt is triggered.

timerAttachInterruptArg This function is used to attach interrupt to timer using arguments.

```
void timerAttachInterruptArg(hw_timer_t * timer, void (*userFunc)(void*), void *arg);
```

- timer timer struct.
- userFunc function to be called when interrupt is triggered.
- arg pointer to the interrupt arguments.

timerDetachInterrupt This function is used to detach interrupt from timer.

```
void timerDetachInterrupt(hw_timer_t * timer);
```

- timer timer struct.

timerAlarm This function is used to configure alarm value and autoreload of the timer. Alarm is automatically enabled.

```
void timerAlarm(hw_timer_t * timer, uint64_t alarm_value, bool autoreload, uint64_t reload_count);
```

- timer timer struct.
- alarm_value alarm value to generate event.
- autoreload enabled/disabled autoreload.
- reload_count number of autoreloads (0 = unlimited). Has no effect if autoreload is disabled.

Example Applications There are 2 examples uses of Timer:

Repeat timer example:

```

/*
Repeat timer example

This example shows how to use hardware timer in ESP32. The timer calls onTimer
function every second. The timer can be stopped with button attached to PIN 0
(IO0).

This example code is in the public domain.
*/

#include <Arduino.h>

// Stop button is attached to PIN 0 (IO0)
#define BTN_STOP_ALARM 0

hw_timer_t *timer = NULL;
volatile SemaphoreHandle_t timerSemaphore;
portMUX_TYPE timerMux = portMUX_INITIALIZER_UNLOCKED;

volatile uint32_t isrCounter = 0;
volatile uint32_t lastIsrAt = 0;

void ARDUINO_ISR_ATTR onTimer() {
    // Increment the counter and set the time of ISR
    portENTER_CRITICAL_ISR(&timerMux);
    isrCounter = isrCounter + 1;
    lastIsrAt = millis();
    portEXIT_CRITICAL_ISR(&timerMux);
    // Give a semaphore that we can check in the loop
    xSemaphoreGiveFromISR(timerSemaphore, NULL);
    // It is safe to use digitalWrite/Write here if you want to toggle an output
}

void setup() {
    Serial.begin(115200);

    // Set BTN_STOP_ALARM to input mode
    pinMode(BTN_STOP_ALARM, INPUT_PULLUP);

    // Create semaphore to inform us when the timer has fired
    timerSemaphore = xSemaphoreCreateBinary();

    // Set timer frequency to 1Mhz
    timer = timerBegin(1000000);

    // Attach onTimer function to our timer.
    timerAttachInterrupt(timer, &onTimer);

    // Set alarm to call onTimer function every second (value in microseconds).
    // Repeat the alarm (third parameter) with unlimited count = 0 (fourth
    // parameter).
    timerAlarm(timer, 1000000, true, 0);
}

void loop() {
    // If Timer has fired
    if (xSemaphoreTake(timerSemaphore, 0) == pdTRUE) {
        uint32_t isrCount = 0, isrTime = 0;
        // Read the interrupt count and time
    }
}

```

(continues on next page)

(continued from previous page)

```

    portENTER_CRITICAL(&timerMux);
    isrCount = isrCounter;
    isrTime = lastIsrAt;
    portEXIT_CRITICAL(&timerMux);
    // Print it
    Serial.print("onTimer no. ");
    Serial.print(isrCount);
    Serial.print(" at ");
    Serial.print(isrTime);
    Serial.println(" ms");
}
// If button is pressed
if (digitalRead(BTN_STOP_ALARM) == LOW) {
    // If timer is still running
    if (timer) {
        // Stop and free timer
        timerEnd(timer);
        timer = NULL;
    }
}
}
}

```

Watchdog timer example:

```

#include <Arduino.h>
#include "esp_system.h"
#include "rom/ets_sys.h"

const int button = 0;           //gpio to use to trigger delay
const int wdtTimeout = 3000;    //time in ms to trigger the watchdog
hw_timer_t *timer = NULL;

void ARDUINO_ISR_ATTR resetModule() {
    ets_printf("reboot\n");
    esp_restart();
}

void setup() {
    Serial.begin(115200);
    Serial.println();
    Serial.println("running setup");

    pinMode(button, INPUT_PULLUP);           //init control pin
    timer = timerBegin(1000000);              //timer 1Mhz resolution
    timerAttachInterrupt(timer, &resetModule); //attach callback
    timerAlarm(timer, wdtTimeout * 1000, false, 0); //set time in us
}

void loop() {
    Serial.println("running main loop");

    timerWrite(timer, 0); //reset timer (feed watchdog)
    long loopTime = millis();
    //while button is pressed, delay up to 3 seconds to trigger the timer
    while (!digitalRead(button)) {
        Serial.println("button pressed");
        delay(500);
    }
    delay(1000); //simulate work
    loopTime = millis() - loopTime;
}

```

(continues on next page)

(continued from previous page)

```
Serial.print("loop time is = ");
Serial.println(loopTime); //should be under 3000
}
```

2.3.26 TOUCH

About

Touch sensor is a peripheral, that has an internal oscillator circuit and it measures charge/discharge frequency over a fixed period of time on respective GPIO pins. Therefore these touch sensors are also known as capacitive sensors. For example, if you touch any of these pins, finger electrical charge will change this number of cycles, by changing the RC circuit attached to the touch sensor. The `TouchRead()` will return the number of cycles (charges/discharges) in a certain time (meas). The change of this count will be used to validate if a touch has happened or not. These pins can be easily integrated into capacitive pads, and replace mechanical buttons.

Note: Touch peripheral is not present in every SoC. Refer to datasheet of each chip for more info.

Arduino-ESP32 TOUCH API

TOUCH common API

touchRead This function gets the touch sensor data. Each touch sensor has a counter to count the number of charge/discharge cycles. When the pad is ‘touched’, the value in the counter will change because of the larger equivalent capacitance. The change of the data determines if the pad has been touched or not.

```
touch_value_t touchRead(uint8_t pin);
```

- pin GPIO pin to read TOUCH value

This function will return touch pad value as `uint16_t` (ESP32) or `uint32_t` (ESP32-S2/S3).

touchSetCycles This function is used to set cycles that measurement operation takes. The result from `touchRead`, threshold and detection accuracy depend on these values. The defaults are setting `touchRead` to take ~0.5 ms.

```
void touchSetCycles(uint16_t measure, uint16_t sleep);
```

- measure Sets the time that it takes to measure touch sensor value
- sleep Sets waiting time before next measure cycle

touchAttachInterrupt This function is used to attach interrupt to the touch pad. The function will be called if a touch sensor value falls below the given threshold for ESP32 or rises above the given threshold for ESP32-S2/S3. To determine a proper threshold value between touched and untouched state, use `touchRead()` function.

```
void touchAttachInterrupt(uint8_t pin, void (*userFunc)(void), touch_value_t
↪threshold);
```

- pin GPIO TOUCH pad pin
- userFunc Function to be called when interrupt is triggered
- threshold Sets the threshold when to call interrupt

touchAttachInterruptArg This function is used to attach interrupt to the touch pad. In the function called by ISR you have the given arguments available.

```
void touchAttachInterruptArg(uint8_t pin, void (*userFunc)(void*), void *arg,   
↪touch_value_t threshold);
```

- pin GPIO TOUCH pad pin
- userFunc Function to be called when interrupt is triggered
- arg Sets arguments to the interrupt
- threshold Sets the threshold when to call interrupt

touchDetachInterrupt This function is used to detach interrupt from the touch pad.

```
void touchDetachInterrupt(uint8_t pin);
```

- pin GPIO TOUCH pad pin.

touchSleepWakeUpEnable This function is used to setup touch pad as the wake up source from the deep sleep.

Note: ESP32-S2 and ESP32-S3 only support one sleep wake up touch pad.

```
void touchSleepWakeUpEnable(uint8_t pin, touch_value_t threshold);
```

- pin GPIO TOUCH pad pin
- threshold Sets the threshold when to wake up

TOUCH API specific for ESP32 chip (TOUCH_V1)

touchInterruptSetThresholdDirection This function is used to tell the driver if it shall activate the interrupt if the sensor is lower or higher than the threshold value. Default is lower.

```
void touchInterruptSetThresholdDirection(bool mustbeLower);
```

TOUCH API specific for ESP32-S2 and ESP32-S3 chip (TOUCH_V2)

touchInterruptGetLastStatus This function is used get the latest ISR status for the touch pad.

```
bool touchInterruptGetLastStatus(uint8_t pin);
```

This function returns true if the touch pad has been and continues pressed or false otherwise.

Example Applications Example of reading the touch sensor.

```
// ESP32 Touch Test  
// Just test touch pin - Touch2 is T2 which is on GPIO 2.  
  
#include <Arduino.h>  
  
void setup() {  
  Serial.begin(115200);  
  delay(1000); // give me time to bring up serial monitor  
  Serial.println("ESP32 Touch Test");  
}
```

(continues on next page)

(continued from previous page)

```
void loop() {
  Serial.println(touchRead(T2)); // get value using T2
  delay(1000);
}
```

A usage example for the touch interrupts.

```
/*
This is an example how to use Touch Interrerupts
The bigger the threshold, the more sensible is the touch
*/

#include <Arduino.h>

int threshold = 0; // if 0 is used, benchmark value is used. Its by default 1,5%
↳change, can be changed by touchSetDefaultThreshold(float percentage)

bool touch1detected = false;
bool touch2detected = false;

void gotTouch1() {
  touch1detected = true;
}

void gotTouch2() {
  touch2detected = true;
}

void setup() {
  Serial.begin(115200);
  delay(1000); // give me time to bring up serial monitor

  //Optional: Set the threshold to 5% of the benchmark value. Only effective if
  ↳threshold = 0.
  touchSetDefaultThreshold(5);

  Serial.println("ESP32 Touch Interrupt Test");
  touchAttachInterrupt(T2, gotTouch1, threshold);
  touchAttachInterrupt(T3, gotTouch2, threshold);
}

void loop() {
  if (touch1detected) {
    touch1detected = false;
    Serial.println("Touch 1 detected");
  }
  if (touch2detected) {
    touch2detected = false;
    Serial.println("Touch 2 detected");
  }
}
```

More examples can be found in our repository -> [Touch examples](#).

2.3.27 USB API

Note: This feature is only supported on ESP chips that have USB peripheral, like the ESP32-S2 and ESP32-S3. Some chips, like the ESP32-C3 include native CDC+JTAG peripheral that is not covered here.

About

The **Universal Serial Bus** is a widely used peripheral to exchange data between devices. USB was introduced on the ESP32, supporting both device and host mode.

To learn about the USB, see the [USB.org](https://www.usb.org) for developers.

USB as Device In the device mode, the ESP32 acts as an USB device, like a mouse or keyboard to be connected to a host device, like your computer or smartphone.

USB as Host The USB host mode, you can connect devices on the ESP32, like external modems, mouse and keyboards.

Note: This mode is still under development for the ESP32.

API Description

This is the common USB API description.

For more supported USB classes implementation, see the following sections:

USB CDC

About USB Communications Device Class API. This class is used to enable communication between the host and the device.

This class is often used to enable serial communication and can be used to flash the firmware on the ESP32 without the external USB to Serial chip.

APIs

onEvent Event handling functions.

```
void onEvent(esp_event_handler_t callback);
```

```
void onEvent(arduino_usb_cdc_event_t event, esp_event_handler_t callback);
```

Where event can be:

- ARDUINO_USB_CDC_ANY_EVENT
- ARDUINO_USB_CDC_CONNECTED_EVENT
- ARDUINO_USB_CDC_DISCONNECTED_EVENT
- ARDUINO_USB_CDC_LINE_STATE_EVENT
- ARDUINO_USB_CDC_LINE_CODING_EVENT
- ARDUINO_USB_CDC_RX_EVENT
- ARDUINO_USB_CDC_TX_EVENT

- ARDUINO_USB_CDC_RX_OVERFLOW_EVENT
- ARDUINO_USB_CDC_MAX_EVENT

setRxBufferSize The `setRxBufferSize` function is used to set the size of the RX buffer.

```
size_t setRxBufferSize(size_t size);
```

setTxTimeoutMs This function is used to define the time to reach the timeout for the TX.

```
void setTxTimeoutMs(uint32_t timeout);
```

begin This function is used to start the peripheral using the default CDC configuration.

```
void begin(unsigned long baud);
```

Where:

- `baud` is the baud rate.

end This function will finish the peripheral as CDC and release all the allocated resources. After calling `end` you need to use `begin` again in order to initialize the USB CDC driver again.

```
void end();
```

available This function will return if there are messages in the queue.

```
int available(void);
```

The return is the number of bytes available to read.

availableForWrite This function will return if the hardware is available to write data.

```
int availableForWrite(void);
```

peek This function is used to peek messages from the queue.

```
int peek(void);
```

read This function is used to read the bytes available.

```
size_t read(uint8_t *buffer, size_t size);
```

Where:

- `buffer` is the pointer to the buffer to be read.
- `size` is the number of bytes to be read.

write This function is used to write the message.

```
size_t write(const uint8_t *buffer, size_t size);
```

Where:

- `buffer` is the pointer to the buffer to be written.
- `size` is the number of bytes to be written.

flush This function is used to flush the data.

```
void flush(void);
```

baudRate This function is used to get the baudRate.

```
uint32_t baudRate();
```

setDebugOutput This function will enable the debug output, usually from the *UART0*, to the USB CDC.

```
void setDebugOutput(bool);
```

enableReboot This function enables the device to reboot by the DTR as RTS signals.

```
void enableReboot(bool enable);
```

rebootEnabled This function will return if the reboot is enabled.

```
bool rebootEnabled(void);
```

Example Code Here is an example of how to use the USB CDC.

USBSerial

```
#include <Arduino.h>
#ifndef ARDUINO_USB_MODE
#error This ESP32 SoC has no Native USB interface
#elif ARDUINO_USB_MODE == 1
#warning This sketch should be used when USB is in OTG mode
void setup() {}
void loop() {}
#else
#include "USB.h"

#if !ARDUINO_USB_CDC_ON_BOOT
USB CDC USBSerial;
#endif

static void usbEventCallback(void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data) {
    if (event_base == ARDUINO_USB_EVENTS) {
        arduino_usb_event_data_t *data = (arduino_usb_event_data_t *)event_data;
        switch (event_id) {
            case ARDUINO_USB_STARTED_EVENT: Serial.println("USB PLUGGED"); break;
            case ARDUINO_USB_STOPPED_EVENT: Serial.println("USB UNPLUGGED"); break;
```

(continues on next page)

(continued from previous page)

```

        case ARDUINO_USB_SUSPEND_EVENT: Serial.printf("USB SUSPENDED: remote_wakeup_
↳en: %u\n", data->suspend.remote_wakeup_en); break;
        case ARDUINO_USB_RESUME_EVENT: Serial.println("USB RESUMED"); break;

        default: break;
    }
} else if (event_base == ARDUINO_USB_CDC_EVENTS) {
    arduino_usb_cdc_event_data_t *data = (arduino_usb_cdc_event_data_t *)event_
↳data;
    switch (event_id) {
        case ARDUINO_USB_CDC_CONNECTED_EVENT: Serial.println("CDC CONNECTED");
↳break;
        case ARDUINO_USB_CDC_DISCONNECTED_EVENT: Serial.println("CDC DISCONNECTED");
↳break;
        case ARDUINO_USB_CDC_LINE_STATE_EVENT: Serial.printf("CDC LINE STATE: dtr:
↳%u, rts: %u\n", data->line_state.dtr, data->line_state.rts); break;
        case ARDUINO_USB_CDC_LINE_CODING_EVENT:
            Serial.printf(
                "CDC LINE CODING: bit_rate: %" PRIu32 " ", data_bits: %u, stop_bits: %u,
↳parity: %u\n", data->line_coding.bit_rate, data->line_coding.data_bits,
                data->line_coding.stop_bits, data->line_coding.parity
            );
            break;
        case ARDUINO_USB_CDC_RX_EVENT:
            Serial.printf("CDC RX [%lu]:", (unsigned long)data->rx.len);
            {
                uint8_t buf[data->rx.len];
                size_t len = USBSerial.read(buf, data->rx.len);
                Serial.write(buf, len);
            }
            Serial.println();
            break;
        case ARDUINO_USB_CDC_RX_OVERFLOW_EVENT: Serial.printf("CDC RX Overflow of
↳%lu bytes", (unsigned long)data->rx_overflow.dropped_bytes); break;

        default: break;
    }
}
}

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);

    USB.onEvent(usbEventCallback);
    USBSerial.onEvent(usbEventCallback);

    USBSerial.begin();
    USB.begin();
}

void loop() {
    while (Serial.available()) {
        size_t l = Serial.available();
        uint8_t b[l];
        l = Serial.read(b, l);
        USBSerial.write(b, l);
    }
}
#endif /* ARDUINO_USB_MODE */

```

USB MSC

About USB Mass Storage Class API. This class makes the device accessible as a mass storage device and allows you to transfer data between the host and the device.

One of the examples for this mode is to flash the device by dropping the firmware binary like a flash memory device when connecting the ESP32 to the host computer.

APIs

begin This function is used to start the peripheral using the default MSC configuration.

```
bool begin(uint32_t block_count, uint16_t block_size);
```

Where:

- `block_count` set the disk sector count.
- `block_size` set the disk sector size.

This function will return `true` if the configuration was successful.

end This function will finish the peripheral as MSC and release all the allocated resources. After calling `end` you need to use `begin` again in order to initialize the USB MSC driver again.

```
void end();
```

vendorID This function is used to define the vendor ID.

```
void vendorID(const char * vid); //max 8 chars
```

productID This function is used to define the product ID.

```
void productID(const char * pid); //max 16 chars
```

productRevision This function is used to define the product revision.

```
void productRevision(const char * ver); //max 4 chars
```

mediaPresent Set the `mediaPresent` configuration.

```
void mediaPresent(bool media_present);
```

onStartStop Set the `onStartStop` callback function.

```
void onStartStop(msc_start_stop_cb cb);
```

onRead Set the `onRead` callback function.

```
void onRead(msc_read_cb cb);
```

onWrite Set the onWrite callback function.

```
void onWrite(msc_write_cb cb);
```

Example Code Here is an example of how to use the USB MSC.

FirmwareMSC

```
#include <Arduino.h>
#ifndef ARDUINO_USB_MODE
#error This ESP32 SoC has no Native USB interface
#elif ARDUINO_USB_MODE == 1
#warning This sketch should be used when USB is in OTG mode
void setup() {}
void loop() {}
#else
#include "USB.h"
#include "FirmwareMSC.h"

#if !ARDUINO_USB_MSC_ON_BOOT
FirmwareMSC MSC_Update;
#endif

static void usbEventCallback(void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data) {
    if (event_base == ARDUINO_USB_EVENTS) {
        arduino_usb_event_data_t *data = (arduino_usb_event_data_t *)event_data;
        switch (event_id) {
            case ARDUINO_USB_STARTED_EVENT: Serial.println("USB PLUGGED"); break;
            case ARDUINO_USB_STOPPED_EVENT: Serial.println("USB UNPLUGGED"); break;
            case ARDUINO_USB_SUSPEND_EVENT: Serial.printf("USB SUSPENDED: remote_wakeup_en: %u\n", data->suspend.remote_wakeup_en); break;
            case ARDUINO_USB_RESUME_EVENT: Serial.println("USB RESUMED"); break;

            default: break;
        }
    } else if (event_base == ARDUINO_FIRMWARE_MSC_EVENTS) {
        arduino_firmware_msc_event_data_t *data = (arduino_firmware_msc_event_data_t *)event_data;
        switch (event_id) {
            case ARDUINO_FIRMWARE_MSC_START_EVENT: Serial.println("MSC Update Start"); break;
            case ARDUINO_FIRMWARE_MSC_WRITE_EVENT:
                //Serial.printf("MSC Update Write %lu bytes at offset %lu\n", (unsigned long)data->write.size, (unsigned long)data->write.offset);
                Serial.print(".");
                break;
            case ARDUINO_FIRMWARE_MSC_END_EVENT: Serial.printf("\nMSC Update End: %lu bytes\n", (unsigned long)data->end.size); break;
            case ARDUINO_FIRMWARE_MSC_ERROR_EVENT: Serial.printf("MSC Update ERROR! Progress: %lu bytes\n", (unsigned long)data->error.size); break;
            case ARDUINO_FIRMWARE_MSC_POWER_EVENT:
                Serial.printf("MSC Update Power: power: %u, start: %u, eject: %u", data->power.power_condition, data->power.start, data->power.load_eject);
                break;

            default: break;
        }
    }
}
```

(continues on next page)

(continued from previous page)

```
void setup() {
  Serial.begin(115200);
  Serial.setDebugOutput(true);

  USB.onEvent(usbEventCallback);
  MSC_Update.onEvent(usbEventCallback);
  MSC_Update.begin();
  USB.begin();
}

void loop() {
  // put your main code here, to run repeatedly
}
#endif /* ARDUINO_USB_MODE */
```

USB Common These are the common APIs for the USB driver.

onEvent Event handling function to set the callback.

```
void onEvent(esp_event_handler_t callback);
```

Event handling function for the specific event.

```
void onEvent(arduino_usb_event_t event, esp_event_handler_t callback);
```

Where event can be:

- ARDUINO_USB_ANY_EVENT
- ARDUINO_USB_STARTED_EVENT
- ARDUINO_USB_STOPPED_EVENT
- ARDUINO_USB_SUSPEND_EVENT
- ARDUINO_USB_RESUME_EVENT
- ARDUINO_USB_MAX_EVENT

VID Set the Vendor ID. This 16 bits identification is used to identify the company that develops the product.

Note: You can't define your own VID. If you need your own VID, you need to buy one. See <https://www.usb.org/getting-vendor-id> for more details.

```
bool VID(uint16_t v);
```

Get the Vendor ID.

```
uint16_t VID(void);
```

Returns the Vendor ID. The default value for the VID is: 0x303A.

PID Set the Product ID. This 16 bits identification is used to identify the product.

```
bool PID(uint16_t p);
```

Get the Product ID.

```
uint16_t PID(void);
```

Returns the Product ID. The default PID is: 0x0002.

firmwareVersion Set the firmware version. This is a 16 bits unsigned value.

```
bool firmwareVersion(uint16_t version);
```

Get the firmware version.

```
uint16_t firmwareVersion(void);
```

Return the 16 bits unsigned value. The default value is: 0x100.

usbVersion Set the USB version.

```
bool usbVersion(uint16_t version);
```

Get the USB version.

```
uint16_t usbVersion(void);
```

Return the USB version. The default value is: 0x200 (USB 2.0).

usbPower Set the USB power as mA (current).

Note: This configuration does not change the physical power output. This is only used for the USB device information.

```
bool usbPower(uint16_t mA);
```

Get the USB power configuration.

```
uint16_t usbPower(void);
```

Return the current in mA. The default value is: 0x500 (500 mA).

usbClass Set the USB class.

```
bool usbClass(uint8_t _class);
```

Get the USB class.

```
uint8_t usbClass(void);
```

Return the USB class. The default value is: TUSB_CLASS_MISC.

usbSubClass Set the USB sub-class.

```
bool usbSubClass(uint8_t subClass);
```

Get the USB sub-class.

```
uint8_t usbSubClass(void);
```

Return the USB sub-class. The default value is: MISC_SUBCLASS_COMMON.

usbProtocol Define the USB protocol.

```
bool usbProtocol(uint8_t protocol);
```

Get the USB protocol.

```
uint8_t usbProtocol(void);
```

Return the USB protocol. The default value is: MISC_PROTOCOL_IAD

usbAttributes Set the USB attributes.

```
bool usbAttributes(uint8_t attr);
```

Get the USB attributes.

```
uint8_t usbAttributes(void);
```

Return the USB attributes. The default value is: TUSB_DESC_CONFIG_ATT_SELF_POWERED

webUSB This function is used to enable the webUSB functionality.

```
bool webUSB(bool enabled);
```

This function is used to get the webUSB setting.

```
bool webUSB(void);
```

Return the webUSB setting (*Enabled* or *Disabled*)

productName This function is used to define the product name.

```
bool productName(const char * name);
```

This function is used to get the product' s name.

```
const char * productName(void);
```

manufacturerName This function is used to define the manufacturer name.

```
bool manufacturerName(const char * name);
```

This function is used to get the manufacturer' s name.

```
const char * manufacturerName(void);
```

serialNumber This function is used to define the serial number.

```
bool serialNumber(const char * name);
```

This function is used to get the serial number.

```
const char * serialNumber(void);
```

The default serial number is: 0.

webUSBURL This function is used to define the webUSBURL.

```
bool webUSBURL(const char * name);
```

This function is used to get the webUSBURL.

```
const char * webUSBURL(void);
```

The default webUSBURL is: https://docs.espressif.com/projects/arduino-esp32/en/latest/_static/webusb.html

enableDFU This function is used to enable the DFU capability.

```
bool enableDFU();
```

begin This function is used to start the peripheral using the default configuration.

```
bool begin();
```

Example Code

There are a collection of USB device examples on the project GitHub, including Firmware MSC update, USB CDC, HID and composite device.

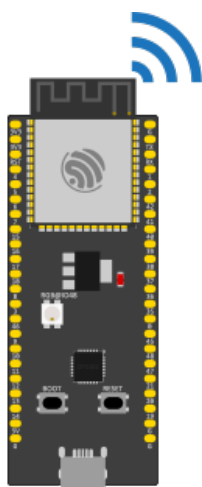
2.3.28 Wi-Fi API

About

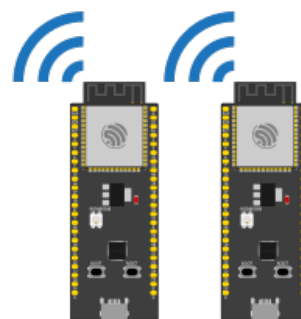
The Wi-Fi API provides support for the 802.11b/g/n protocol driver. This API includes:

- Station mode (STA mode or Wi-Fi client mode). ESP32 connects to an access point
- AP mode (aka Soft-AP mode or Access Point mode). Devices connect to the ESP32
- Security modes (WPA2, WPA3 etc.)
- Scanning for access points

Working as AP In this mode, the ESP32 is configured as an Access Point (AP) and it's capable of receiving incoming connections from other devices (stations) by providing a Wi-Fi network.



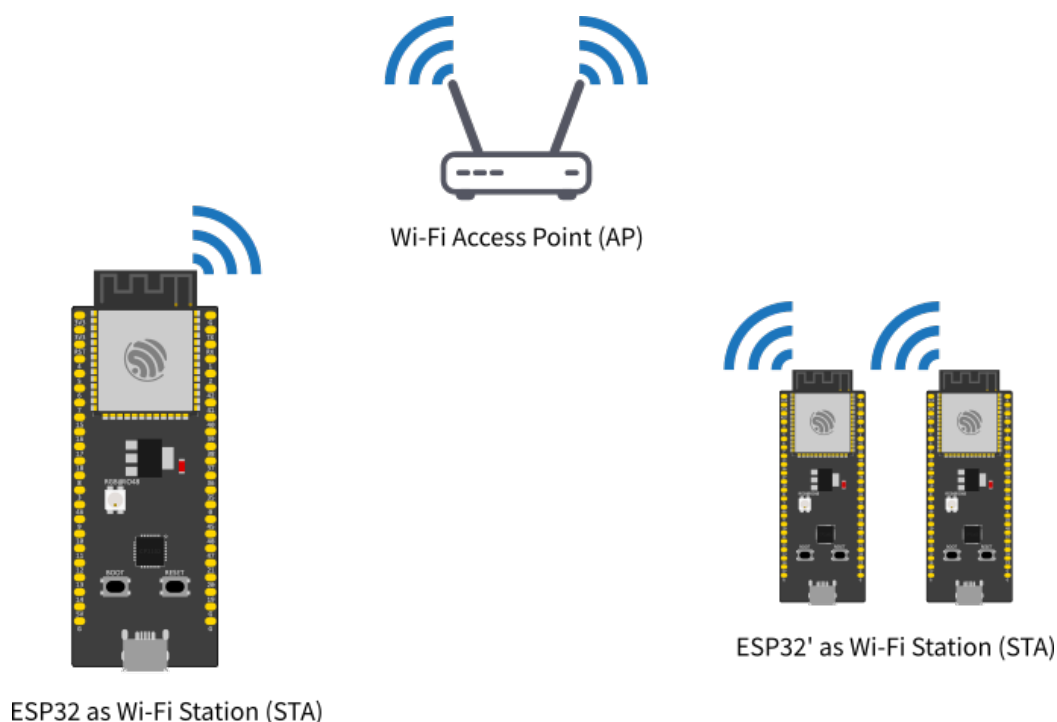
ESP32 as Wi-Fi Access Point (AP)



ESP32' as Wi-Fi Station (STA)

This mode can be used for serving an HTTP or HTTPS server inside the ESP32, for example.

Working as STA The STA mode is used to connect the ESP32 to a Wi-Fi network, provided by an Access Point.



This is the mode to be used if you want to connect your project to the Internet.

API Description

Here is the description of the Wi-Fi API.

Common API

Here are the common APIs that are used for both modes, AP and STA.

onEvent (and removeEvent) Registers a caller-supplied function to be called when Wi-Fi events occur. Several forms are available.

Function pointer callback taking the event ID:

```
typedef void (*WiFiEventCb) (arduino_event_id_t);
wifi_event_id_t onEvent (WiFiEventCb, arduino_event_id_t = ARDUINO_EVENT_MAX);
```

Function pointer callback taking an event-ID-and-info struct:

```
typedef struct{
    arduino_event_id_t event_id;
    arduino_event_info_t event_info;
} arduino_event_t;

typedef void (*WiFiEventSysCb) (arduino_event_t *);
wifi_event_id_t onEvent (WiFiEventSysCb, arduino_event_id_t = ARDUINO_EVENT_MAX);
```

Callback using std::function taking event ID and info separately:

```
typedef std::function<void(arduino_event_id_t, arduino_event_info_t)>_
↳WiFiEventFuncCb;
wifi_event_id_t onEvent (WiFiEventFuncCb, arduino_event_id_t = ARDUINO_EVENT_MAX);
```

A similar set of functions are available to remove callbacks:

```
void removeEvent(WiFiEventCb, arduino_event_id_t = ARDUINO_EVENT_MAX);  
void removeEvent(WiFiEventSysCb, arduino_event_id_t = ARDUINO_EVENT_MAX);  
void removeEvent(wifi_event_id_t = ARDUINO_EVENT_MAX);
```

In all cases, the subscribing function accepts an optional event type to invoke the callback only for that specific event; with the default `ARDUINO_EVENT_MAX`, the callback will be invoked for all Wi-Fi events.

Any callback function is given the event type in a parameter. Some of the possible callback function formats also take an `arduino_event_info_t` (or use `arduino_event_t` which includes both ID and info) which is a union of structs with additional information about different event types.

See [WiFiGeneric.h](#) for the list of event types and “info” substructures, and also see a full example of event handling: [events example](#).

Warning: Event callback functions are invoked on a separate [thread \(FreeRTOS task\)](#) independent of the main application thread that runs `setup()` and `loop()`. Callback functions must therefore be [thread-safe](#); they must not access shared/global variables directly without locking, and must only call similarly thread-safe functions.

Some core operations like `Serial.print()` are thread-safe but many functions are not. Notably, `WiFi.onEvent()` and `WiFi.removeEvent()` are not thread-safe and should never be invoked from a callback thread.

setHostname (and getHostname) Sets the name the DHCP client uses to identify itself. In a typical network setup this will be the name that shows up in the Wi-Fi router’s device list. The hostname must be no longer than 32 characters.

```
setHostname(const char *hostname);
```

If the hostname is never specified, a default one will be assigned based on the chip type and MAC address. The current hostname (default or custom) may be retrieved:

```
const char *getHostname();
```

Warning: The `setHostname()` function must be called BEFORE Wi-Fi is started with `WiFi.begin()`, `WiFi.softAP()`, `WiFi.mode()`, or `WiFi.run()`. To change the name, reset Wi-Fi with `WiFi.mode(WIFI_MODE_NULL)`, then proceed with `WiFi.setHostname(...)` and restart Wi-Fi from scratch.

useStaticBuffers This function is used to set the memory allocation mode for the Wi-Fi buffers.

```
static void useStaticBuffers(bool bufferMode);
```

- Set `true` to use the Wi-Fi buffers memory allocation as **static**.
- Set `false` to set the buffers memory allocation to **dynamic**.

The use of dynamic allocation is recommended to save memory and reduce resources usage. However, the dynamic performs slightly slower than the static allocation. Use static allocation if you want to have more performance and if your application is multi-tasking.

By default, the memory allocation will be set to **dynamic** if this function is not being used.

setDualAntennaConfig Configures the Dual antenna functionality. This function should be used only on the **ESP32-WROOM-DA** module or any other ESP32 with RF switch.

```
bool setDualAntennaConfig(uint8_t gpio_ant1, uint8_t gpio_ant2, wifi_rx_ant_t rx_  
↪mode, wifi_tx_ant_t tx_mode);
```

- `gpio_ant1` Configure the GPIO number for the antenna 1 connected to the RF switch (default GPIO2 on ESP32-WROOM-DA)
- `gpio_ant2` Configure the GPIO number for the antenna 2 connected to the RF switch (default GPIO25 on ESP32-WROOM-DA)
- `rx_mode` Set the RX antenna mode. See `wifi_rx_ant_t` for the options.
- `tx_mode` Set the TX antenna mode. See `wifi_tx_ant_t` for the options.

Return `true` if the configuration was successful.

For the `rx_mode` you can use the following configuration:

- `WIFI_RX_ANT0` Selects the antenna 1 for all RX activity.
- `WIFI_RX_ANT1` Selects the antenna 2 for all RX activity.
- `WIFI_RX_ANT_AUTO` Selects the antenna for RX automatically.

For the `tx_mode` you can use the following configuration:

- `WIFI_TX_ANT0` Selects the antenna 1 for all TX activity.
- `WIFI_TX_ANT1` Selects the antenna 2 for all TX activity.
- `WIFI_TX_ANT_AUTO` Selects the antenna for TX automatically.

WiFiAP

The `WiFiAP` is used to configure and manage the Wi-Fi as an Access Point. This is where you can find the related functions for the AP.

Basic Usage To start the Wi-Fi as an Access Point.

```
WiFi.softAP(ssid, password);
```

Please see the full `WiFiAP` example in: [ap example](#).

AP Configuration

softAP Use the function `softAP` to configure the Wi-Fi AP characteristics:

```
bool softAP(const char* ssid, const char* passphrase = NULL, int channel = 1, int ↵
↳ssid_hidden = 0, int max_connection = 4, bool ftm_responder = false);
```

Where:

- `ssid` sets the Wi-Fi network SSID.
- `passphrase` sets the Wi-Fi network password. If the network is open, set as `NULL`.
- `channel` configures the Wi-Fi channel.
- `ssid_hidden` sets the network as hidden.
- `max_connection` sets the maximum number of simultaneous connections. The default is 4.
- `ftm_responder` sets the Wi-Fi FTM responder feature. **Only for ESP32-S2 and ESP32-C3 SoC!**

Return `true` if the configuration was successful.

softAPConfig Function used to configure the IP as static (fixed) as well as the gateway and subnet.

```
bool softAPConfig(IPAddress local_ip, IPAddress gateway, IPAddress subnet);
```

Where:

- `local_ip` sets the local IP address.
- `gateway` sets the gateway IP.
- `subnet` sets the subnet mask.

The function will return `true` if the configuration is successful.

AP Connection

softAPdisconnect Function used to force the AP disconnection.

```
bool softAPdisconnect (bool wifioff = false);
```

Where:

- `wifioff` sets the Wi-Fi off if `true`.

The function will return `true` if the configuration is successful.

softAPgetStationNum This function returns the number of clients connected to the AP.

```
uint8_t softAPgetStationNum();
```

softAPIP Function to get the AP IPv4 address.

```
IPAddress softAPIP();
```

The function will return the AP IP address in `IPAddress` format.

softAPBroadcastIP Function to get the AP IPv4 broadcast address.

```
IPAddress softAPBroadcastIP();
```

The function will return the AP broadcast address in `IPAddress` format.

softAPNetworkID Get the softAP network ID.

```
IPAddress softAPNetworkID();
```

The function will return the AP network address in `IPAddress` format.

softAPSubnetCIDR Get the softAP subnet CIDR.

```
uint8_t softAPSubnetCIDR();
```

softAPSubnetMask Get the softAP subnet mask.

```
IPAddress softAPSubnetMask();
```

softAPenableIPv6 Function used to enable the IPv6 support.

```
bool softAPenableIPv6 (bool enable=true);
```

The function will return `true` if the configuration is successful.

softAPIPv6 Function to get the IPv6 address.

```
IPAddress softAPIPv6();
```

The function will return the AP IPv6 address in `IPAddress` format.

softAPgetHostname Function to get the AP hostname.

```
const char * softAPgetHostname();
```

softAPsetHostname Function to set the AP hostname.

```
bool softAPsetHostname(const char * hostname);
```

Where:

- `hostname` sets the device hostname.

The function will return `true` if the configuration is successful.

softAPmacAddress Function to define the AP MAC address.

```
uint8_t* softAPmacAddress(uint8_t* mac);
```

Where:

- `mac` sets the new MAC address.

Function to get the AP MAC address.

```
String softAPmacAddress(void);
```

softAPSSID Function to get the AP SSID.

```
String softAPSSID(void) const;
```

Returns the AP SSID.

WiFiSTA

The `WiFiSTA` is used to configure and manage the Wi-Fi as Station. The related functions for the STA are here.

Basic Usage The following code shows the basic usage of the `WifiSTA` functionality.

```
WiFi.begin(ssid, password);
```

Where the `ssid` and `password` are from the network you want to connect the ESP32.

To check if the connection is successful, you can use:

```
while (WiFi.status() != WL_CONNECTED) {  
    delay(500);  
    Serial.print(".");  
}
```

After a successful connection, you can print the IP address given by the network.

```
Serial.println("IP address: ");  
Serial.println(WiFi.localIP());
```

Please see the full example of the `WiFiSTA` in: [sta example](#).

STA Configuration

begin

- Functions `begin` are used to configure and start the Wi-Fi.

```
wl_status_t begin(const char* ssid, const char *passphrase = NULL, int32_t channel_
↳= 0, const uint8_t* bssid = NULL, bool tryConnect = true);
```

Where:

- `ssid` sets the AP SSID.
- `passphrase` sets the AP password. Set as `NULL` for open networks.
- `channel` sets the Wi-Fi channel.
- `uint8_t* bssid` sets the AP BSSID.
- `tryConnect` sets `true` to connect to the configured network automatically.

```
wl_status_t begin(char* ssid, char *passphrase = NULL, int32_t channel = 0, const_
↳uint8_t* bssid = NULL, bool tryConnect = true);
```

Where:

- `ssid` sets the AP SSID.
- `passphrase` sets the AP password. Set as `NULL` for open networks.
- `channel` sets the Wi-Fi channel.
- `bssid` sets the AP BSSID.
- `tryConnect` sets `true` to connect to the configured network automatically.

Note: The `begin` function also accepts `String` types for the `ssid` and `passphrase` parameters, providing convenient overloads for use with Arduino `String` objects.

Function to start the connection after being configured.

```
wl_status_t begin();
```

connect Function to connect to a Wi-Fi network. This is called internally by `begin` but can also be used directly for more control.

```
bool connect(const char* ssid, const char *passphrase = NULL, int32_t channel = 0,
↳const uint8_t* bssid = NULL, bool tryConnect = true);
```

Where:

- `ssid` sets the AP SSID.
- `passphrase` sets the AP password. Set as `NULL` for open networks.
- `channel` sets the Wi-Fi channel.
- `bssid` sets the AP BSSID.
- `tryConnect` sets `true` to connect to the configured network automatically.

Note: The `connect` function also accepts `String` types for the `ssid` and `passphrase` parameters, providing convenient overloads for use with Arduino `String` objects.

config Function `config` is used to configure Wi-Fi. After configuring, you can call function `begin` to start the Wi-Fi process.

```
bool config(IPAddress local_ip, IPAddress gateway, IPAddress subnet, IPAddress_
↳dns1 = (uint32_t)0x00000000, IPAddress dns2 = (uint32_t)0x00000000);
```

Where:

- `local_ip` sets the local IP.
- `gateway` sets the gateway IP.
- `subnet` sets the subnet mask.
- `dns1` sets the DNS.
- `dns2` sets the DNS alternative option.

The function will return `true` if the configuration is successful.

The `IPAddress` format is defined by 4 bytes as described here:

```
IPAddress(uint8_t first_octet, uint8_t second_octet, uint8_t third_octet, uint8_t
↪ fourth_octet);
```

Example:

```
IPAddress local_ip(192, 168, 10, 20);
```

See the `WiFiClientStaticIP.ino` for more details on how to use this feature.

STA Connection

reconnect Function used to reconnect the Wi-Fi connection.

```
bool reconnect();
```

disconnect Function to force disconnection.

```
bool disconnect(bool wifioff = false, bool eraseap = false);
```

Where:

- `wifioff` use `true` to turn the Wi-Fi radio off.
- `eraseap` use `true` to erase the AP configuration from the NVS memory.

The function will return `true` if the configuration is successful.

isConnected Function used to get the connection state.

```
bool isConnected();
```

Return the connection state.

setAutoConnect Function is deprecated.

getAutoConnect Function is deprecated.

setAutoReconnect Function used to set the automatic reconnection if the connection is lost.

```
bool setAutoReconnect(bool autoReconnect);
```

Where:

- `autoConnect` is set to `true` to enable this option.

getAutoReconnect Function used to get the automatic reconnection if the connection is lost.

```
bool getAutoReconnect();
```

The function will return `true` if this setting is enabled.

setMinSecurity Function used to set the minimum security for AP to be considered connectable.

```
bool setMinSecurity(wifi_auth_mode_t minSecurity);
```

Where:

- `minSecurity` is the minimum security for AP to be considered connectable. Default is `WIFI_AUTH_WPA2_PSK`.

WiFiMulti

The `WiFiMulti` allows you to add more than one option for the AP connection while running as a station.

To add the AP, use the following function. You can add multiple AP's and this library will handle the connection.

```
bool addAP(const char *ssid, const char *passphrase = NULL);
```

Or, if you have WPA2-Enterprise support enabled (`CONFIG_ESP_WIFI_ENTERPRISE_SUPPORT`):

```
bool addAP(const char *ssid, const char *passphrase = NULL, const char *username =   
↳ NULL, const char *identity = NULL);
```

For:

- Open networks, `ssid` must be set.
- Networks using a passphrase, e.g. WPA2-PSK, `passphrase` must be set.
- Networks using WPA2-Enterprise with PEAPv0/EAP-MSCHAPv2, `username` and `passphrase` must be set to the username and password required for inner authentication (MSCHAPv2), respectively. `identity` must be set to the “anonymous identity” used for PEAP, if required by the network, or must be set to an empty string otherwise. Note that WPA2-Enterprise networks using certificate-based authentication (e.g. EAP-TLS) are currently not supported. It is also currently not possible to supply a CA certificate to validate the server certificate for PEAP. Given these limitations, you should consider whether this is secure enough for your needs.

After adding the AP's, run by the following function.

```
uint8_t run(uint32_t connectTimeout=5000);
```

To see how to use the `WiFiMulti`, take a look at the `WiFiMulti.ino`, `WiFiMultiAdvanced.ino` and `WiFiMultiEnterprise.ino` examples available.

WiFiScan

To perform the Wi-Fi scan for networks, you can use the following functions:

Start scan Wi-Fi networks available.

```
int16_t scanNetworks(bool async = false, bool show_hidden = false, bool passive =   
↳ false, uint32_t max_ms_per_chan = 300, uint8_t channel = 0);
```

Called to get the scan state in Async mode.

```
int16_t scanComplete();
```

Delete last scan result from RAM.

```
void scanDelete();
```

Loads all infos from a scanned Wi-Fi in to the ptr parameters.

```
bool getNetworkInfo(uint8_t networkItem, String &ssid, uint8_t &encryptionType,
↳int32_t &RSSI, uint8_t* &BSSID, int32_t &channel);
```

To see how to use the `WiFiScan`, take a look at the `WiFiScan.ino` or `WiFiScanAsync.ino` example available.

Examples

[Complete list of Wi-Fi examples.](#)

Wi-Fi AP Example

```
/*
  WiFiAccessPoint.ino creates a WiFi access point and provides a web server on it.

  Steps:
  1. Connect to the access point "yourAp"
  2. Point your web browser to http://192.168.4.1/H to turn the LED on or http://
↳192.168.4.1/L to turn it off
  OR
  Run raw TCP "GET /H" and "GET /L" on PuTTY terminal with 192.168.4.1 as IP_
↳address and 80 as port

  Created for arduino-esp32 on 04 July, 2018
  by Elochukwu Ifediora (fedy0)
*/

#include <Arduino.h>
#include <WiFi.h>
#include <NetworkClient.h>
#include <WiFiAP.h>

#ifndef LED_BUILTIN
#define LED_BUILTIN 2 // Set the GPIO pin where you connected your test LED or_
↳comment this line out if your dev board has a built-in LED
#endif

// Set these to your desired credentials.
const char *ssid = "yourAP";
const char *password = "yourPassword";

NetworkServer server(80);

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);

  Serial.begin(115200);
  Serial.println();
  Serial.println("Configuring access point...");

  // You can remove the password parameter if you want the AP to be open.
  // a valid password must have more than 7 characters
  if (!WiFi.softAP(ssid, password)) {
    log_e("Soft AP creation failed.");
    while (1);
  }
}
```

(continues on next page)

(continued from previous page)

```

IPAddress myIP = WiFi.softAPIP();
Serial.print("AP IP address: ");
Serial.println(myIP);
server.begin();

Serial.println("Server started");
}

void loop() {
  NetworkClient client = server.accept(); // listen for incoming clients

  if (client) { // if you get a client,
    Serial.println("New Client."); // print a message out the serial port
    String currentLine = ""; // make a String to hold incoming data from
    the client
    while (client.connected()) { // loop while the client's connected
      if (client.available()) { // if there's bytes to read from the client,
        char c = client.read(); // read a byte, then
        Serial.write(c); // print it out the serial monitor
        if (c == '\n') { // if the byte is a newline character

          // if the current line is blank, you got two newline characters in a row.
          // that's the end of the client HTTP request, so send a response:
          if (currentLine.length() == 0) {
            // HTTP headers always start with a response code (e.g. HTTP/1.1 200
            OK)
            // and a content-type so the client knows what's coming, then a blank
            line:
            client.println("HTTP/1.1 200 OK");
            client.println("Content-type:text/html");
            client.println();

            // the content of the HTTP response follows the header:
            client.print("Click <a href=\"/H\">here</a> to turn ON the LED.<br>");
            client.print("Click <a href=\"/L\">here</a> to turn OFF the LED.<br>");

            // The HTTP response ends with another blank line:
            client.println();
            // break out of the while loop:
            break;
          } else { // if you got a newline, then clear currentLine:
            currentLine = "";
          }
        } else if (c != '\r') { // if you got anything else but a carriage return
        character,
            currentLine += c; // add it to the end of the currentLine
          }

          // Check to see if the client request was "GET /H" or "GET /L":
          if (currentLine.endsWith("GET /H")) {
            digitalWrite(LED_BUILTIN, HIGH); // GET /H turns the LED on
          }
          if (currentLine.endsWith("GET /L")) {
            digitalWrite(LED_BUILTIN, LOW); // GET /L turns the LED off
          }
        }
      }
    }
    // close the connection:
    client.stop();
    Serial.println("Client Disconnected.");
  }
}

```

(continues on next page)

(continued from previous page)

}

Wi-Fi STA Example

```

/*
  Go to thingspeak.com and create an account if you don't have one already.
  After logging in, click on the "New Channel" button to create a new channel.
  ↳ for your data. This is where your data will be stored and displayed.
  Fill in the Name, Description, and other fields for your channel as desired,
  ↳ then click the "Save Channel" button.
  Take note of the "Write API Key" located in the "API keys" tab, this is the
  ↳ key you will use to send data to your channel.
  Replace the channelID from tab "Channel Settings" and privateKey with "Read
  ↳ API Keys" from "API Keys" tab.
  Replace the host variable with the thingspeak server hostname "api.thingspeak.
  ↳ com"
  Upload the sketch to your ESP32 board and make sure that the board is
  ↳ connected to the internet. The ESP32 should now send data to your Thingspeak
  ↳ channel at the intervals specified by the loop function.
  Go to the channel view page on thingspeak and check the "Field1" for the new
  ↳ incoming data.
  You can use the data visualization and analysis tools provided by Thingspeak
  ↳ to display and process your data in various ways.
  Please note, that Thingspeak accepts only integer values.

  You can later check the values at https://thingspeak.com/channels/2005329
  Please note that this public channel can be accessed by anyone and it is
  ↳ possible that more people will write their values.
*/

#include <Arduino.h>
#include <WiFi.h>

const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password

const char *host = "api.thingspeak.com";   // This should not be changed
const int httpPort = 80;                   // This should not be changed
const String channelID = "2005329";        // Change this to your channel ID
const String writeApiKey = "V6YOTILH9I7D51F9"; // Change this to your Write API
↳ key
const String readApiKey = "34W6LGLIFXD56MPM"; // Change this to your Read API key

// The default example accepts one data filed named "field1"
// For your own server you can ofcourse create more of them.
int field1 = 0;

int numberOfResults = 3; // Number of results to be read
int fieldNumber = 1;     // Field number which will be read out

void setup() {
  Serial.begin(115200);

  // We start by connecting to a WiFi network

  Serial.println();
  Serial.println("*****");
  Serial.print("Connecting to ");
  Serial.println(ssid);

```

(continues on next page)

(continued from previous page)

```

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}

Serial.println("");
Serial.println("WiFi connected");
Serial.println("IP address: ");
Serial.println(WiFi.localIP());
}

void readResponse(NetworkClient *client) {
    unsigned long timeout = millis();
    while (client->available() == 0) {
        if (millis() - timeout > 5000) {
            Serial.println(">>> Client Timeout !");
            client->stop();
            return;
        }
    }

    // Read all the lines of the reply from server and print them to Serial
    while (client->available()) {
        String line = client->readStringUntil('\r');
        Serial.print(line);
    }

    Serial.printf("\nClosing connection\n\n");
}

void loop() {
    NetworkClient client;
    String footer = String(" HTTP/1.1\r\n") + "Host: " + String(host) + "\r\n" +
    ↪ "Connection: close\r\n\r\n";

    // WRITE -----
    ↪ -----
    if (!client.connect(host, httpPort)) {
        return;
    }

    client.print("GET /update?api_key=" + writeApiKey + "&field1=" + field1 +
    ↪ footer);
    readResponse(&client);

    // READ -----
    ↪ -----

    String readRequest = "GET /channels/" + channelID + "/fields/" + fieldNumber + ".
    ↪ json?results=" + numberOfResults + " HTTP/1.1\r\n" + "Host: " + host + "\r\n"
        + "Connection: close\r\n\r\n";

    if (!client.connect(host, httpPort)) {
        return;
    }

    client.print(readRequest);
    readResponse(&client);
}

```

(continues on next page)

(continued from previous page)

```

// -----
↪-----

++field1;
delay(10000);
}

```

Wi-Fi Events Example

```

/*
 * This sketch shows the WiFi event usage
 *
 */

/*
 * WiFi Events

0  ARDUINO_EVENT_WIFI_READY          < ESP32 WiFi ready
1  ARDUINO_EVENT_WIFI_SCAN_DONE     < ESP32 finish scanning AP
2  ARDUINO_EVENT_WIFI_STA_START     < ESP32 station start
3  ARDUINO_EVENT_WIFI_STA_STOP      < ESP32 station stop
4  ARDUINO_EVENT_WIFI_STA_CONNECTED < ESP32 station connected to AP
5  ARDUINO_EVENT_WIFI_STA_DISCONNECTED < ESP32 station disconnected from AP
6  ARDUINO_EVENT_WIFI_STA_AUTHMODE_CHANGE < the auth mode of AP connected by
↪ESP32 station changed
7  ARDUINO_EVENT_WIFI_STA_GOT_IP     < ESP32 station got IP from
↪connected AP
8  ARDUINO_EVENT_WIFI_STA_LOST_IP    < ESP32 station lost IP and the IP
↪is reset to 0
9  ARDUINO_EVENT_WPS_ER_SUCCESS      < ESP32 station wps succeeds in enrollee mode
10 ARDUINO_EVENT_WPS_ER_FAILED       < ESP32 station wps fails in enrollee mode
11 ARDUINO_EVENT_WPS_ER_TIMEOUT      < ESP32 station wps timeout in enrollee mode
12 ARDUINO_EVENT_WPS_ER_PIN          < ESP32 station wps pin code in enrollee mode
13 ARDUINO_EVENT_WIFI_AP_START       < ESP32 soft-AP start
14 ARDUINO_EVENT_WIFI_AP_STOP        < ESP32 soft-AP stop
15 ARDUINO_EVENT_WIFI_AP_STACONNECTED < a station connected to ESP32 soft-
↪AP
16 ARDUINO_EVENT_WIFI_AP_STADISCONNECTED < a station disconnected from ESP32
↪soft-AP
17 ARDUINO_EVENT_WIFI_AP_STAIPASSIGNED < ESP32 soft-AP assign an IP to a
↪connected station
18 ARDUINO_EVENT_WIFI_AP_PROBEREQRECVED < Receive probe request packet in
↪soft-AP interface
19 ARDUINO_EVENT_WIFI_AP_GOT_IP6     < ESP32 ap interface v6IP addr is
↪preferred
19 ARDUINO_EVENT_WIFI_STA_GOT_IP6     < ESP32 station interface v6IP addr
↪is preferred
20 ARDUINO_EVENT_ETH_START           < ESP32 ethernet start
21 ARDUINO_EVENT_ETH_STOP            < ESP32 ethernet stop
22 ARDUINO_EVENT_ETH_CONNECTED       < ESP32 ethernet phy link up
23 ARDUINO_EVENT_ETH_DISCONNECTED    < ESP32 ethernet phy link down
24 ARDUINO_EVENT_ETH_GOT_IP          < ESP32 ethernet got IP from connected AP
19 ARDUINO_EVENT_ETH_GOT_IP6         < ESP32 ethernet interface v6IP addr is
↪preferred
25 ARDUINO_EVENT_MAX

*/

#include <Arduino.h>
#include <WiFi.h>

```

(continues on next page)

(continued from previous page)

```

const char *ssid = "your-ssid";
const char *password = "your-password";

// WARNING: This function is called from a separate FreeRTOS task (thread)!
void WiFiEvent(WiFiEvent_t event) {
    Serial.printf("[WiFi-event] event: %d\n", event);

    switch (event) {
        case ARDUINO_EVENT_WIFI_READY:                Serial.println("WiFi interface_
↪ready"); break;
        case ARDUINO_EVENT_WIFI_SCAN_DONE:            Serial.println("Completed scan_
↪for access points"); break;
        case ARDUINO_EVENT_WIFI_STA_START:            Serial.println("WiFi client_
↪started"); break;
        case ARDUINO_EVENT_WIFI_STA_STOP:             Serial.println("WiFi clients_
↪stopped"); break;
        case ARDUINO_EVENT_WIFI_STA_CONNECTED:        Serial.println("Connected to_
↪access point"); break;
        case ARDUINO_EVENT_WIFI_STA_DISCONNECTED:      Serial.println("Disconnected from_
↪WiFi access point"); break;
        case ARDUINO_EVENT_WIFI_STA_AUTHMODE_CHANGE:  Serial.println("Authentication_
↪mode of access point has changed"); break;
        case ARDUINO_EVENT_WIFI_STA_GOT_IP:           Serial.print("Obtained IP address: ");
                                                         Serial.println(WiFi.localIP());
                                                         break;
        case ARDUINO_EVENT_WIFI_STA_LOST_IP:          Serial.println("Lost IP address_
↪and IP address is reset to 0"); break;
        case ARDUINO_EVENT_WPS_ER_SUCCESS:            Serial.println("WiFi Protected_
↪Setup (WPS): succeeded in enrollee mode"); break;
        case ARDUINO_EVENT_WPS_ER_FAILED:            Serial.println("WiFi Protected_
↪Setup (WPS): failed in enrollee mode"); break;
        case ARDUINO_EVENT_WPS_ER_TIMEOUT:           Serial.println("WiFi Protected_
↪Setup (WPS): timeout in enrollee mode"); break;
        case ARDUINO_EVENT_WPS_ER_PIN:               Serial.println("WiFi Protected_
↪Setup (WPS): pin code in enrollee mode"); break;
        case ARDUINO_EVENT_WIFI_AP_START:            Serial.println("WiFi access point_
↪started"); break;
        case ARDUINO_EVENT_WIFI_AP_STOP:             Serial.println("WiFi access point _
↪stopped"); break;
        case ARDUINO_EVENT_WIFI_AP_STACONNECTED:      Serial.println("Client connected");
        ↪ break;
        case ARDUINO_EVENT_WIFI_AP_STADISCONNECTED:  Serial.println("Client disconnected
        ↪"); break;
        case ARDUINO_EVENT_WIFI_AP_STAIPASSIGNED:    Serial.println("Assigned IP_
        ↪address to client"); break;
        case ARDUINO_EVENT_WIFI_AP_PROBEREQRECVED:   Serial.println("Received probe_
        ↪request"); break;
        case ARDUINO_EVENT_WIFI_AP_GOT_IP6:          Serial.println("AP IPv6 is_
        ↪preferred"); break;
        case ARDUINO_EVENT_WIFI_STA_GOT_IP6:         Serial.println("STA IPv6 is_
        ↪preferred"); break;
        case ARDUINO_EVENT_ETH_GOT_IP6:             Serial.println("Ethernet IPv6 is_
        ↪preferred"); break;
        case ARDUINO_EVENT_ETH_START:               Serial.println("Ethernet started");
        ↪ break;
        case ARDUINO_EVENT_ETH_STOP:               Serial.println("Ethernet stopped");
        ↪ break;
        case ARDUINO_EVENT_ETH_CONNECTED:           Serial.println("Ethernet connected
        ↪"); break;
        case ARDUINO_EVENT_ETH_DISCONNECTED:         Serial.println("Ethernet_

```

(continues on next page)

(continued from previous page)

```

    disconnected"); break;
    case ARDUINO_EVENT_ETH_GOT_IP:          Serial.println("Obtained IP address
    "); break;
    default:                                break;
  }
}

// WARNING: This function is called from a separate FreeRTOS task (thread)!
void WiFiGotIP(WiFiEvent_t event, WiFiEventInfo_t info) {
  Serial.println("WiFi connected");
  Serial.println("IP address: ");
  Serial.println(IPAddress(info.got_ip.ip_info.ip.addr));
}

void setup() {
  Serial.begin(115200);

  // delete old config
  WiFi.disconnect(true);

  delay(1000);

  // Examples of different ways to register wifi events;
  // these handlers will be called from another thread.
  WiFi.onEvent(WiFiEvent);
  WiFi.onEvent(WiFiGotIP, WiFiEvent_t::ARDUINO_EVENT_WIFI_STA_GOT_IP);
  WiFiEventId_t eventID = WiFi.onEvent(
    [](WiFiEvent_t event, WiFiEventInfo_t info) {
      Serial.print("WiFi lost connection. Reason: ");
      Serial.println(info.wifi_sta_disconnected.reason);
    },
    WiFiEvent_t::ARDUINO_EVENT_WIFI_STA_DISCONNECTED
  );

  // Remove WiFi event
  Serial.print("WiFi Event ID: ");
  Serial.println(eventID);
  // WiFi.removeEvent(eventID);

  WiFi.begin(ssid, password);

  Serial.println();
  Serial.println();
  Serial.println("Wait for WiFi... ");
}

void loop() {
  delay(1000);
}

```

2.4 Matter APIs

2.4.1 Matter

About

The Matter library provides support for creating Matter-compatible devices including:

- Support for Wi-Fi and Thread connectivity

- Matter commissioning via QR code or manual pairing code
- Multiple endpoint types for various device categories
- Event monitoring and callback support
- Integration with Apple HomeKit, Amazon Alexa, and Google Home
- Smart home ecosystem compatibility

The Matter library is built on top of [ESP Matter SDK](#) and provides a high-level Arduino-style interface for creating Matter devices.

Building and Flashing Matter Examples

Before uploading any Matter example sketch, it is necessary to configure the Arduino IDE with the following settings:

1. **Partition Scheme:** Select **“Huge APP (3 MB No OTA / 1 MB SPIFFS)”** from **Tools > Partition Scheme** menu.

Partition Scheme: "Huge APP (3MB No OTA/1MB SPIFFS)" >

2. **Erase Flash:** Enable **“Erase All Flash Before Sketch Upload”** option from **Tools** menu.

Erase All Flash Before Sketch Upload: "Enabled" >

These settings are required for the following reasons:

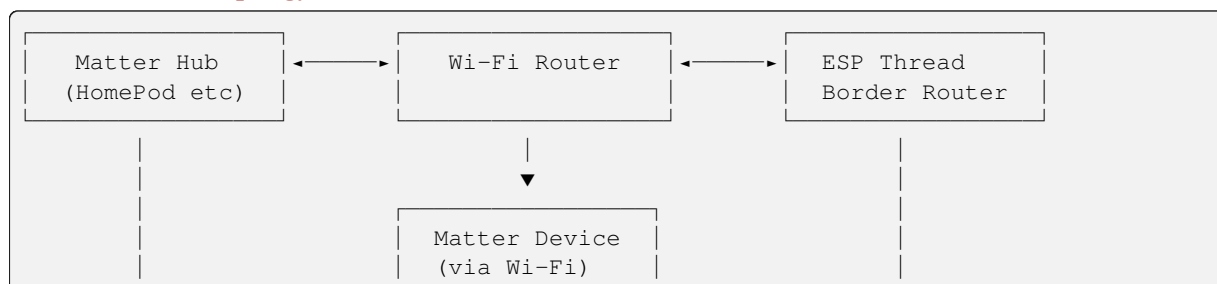
- **Partition Scheme:** Matter firmware requires a large application partition (3 MB) to accommodate the Matter stack and application code.
- **Erase Flash:** Erasing flash is necessary to remove any leftover Wi-Fi or Matter configuration from the NVS (Non-Volatile Storage) partition. Without erasing, previous network credentials, Matter fabric information, or device commissioning data may interfere with the new firmware, causing commissioning failures or connectivity issues.

Matter Protocol Overview Matter (formerly Project CHIP - Connected Home over IP) is an open-source connectivity standard for smart home devices. It enables seamless communication between devices from different manufacturers, allowing them to work together within a single ecosystem.

Key Features:

- **Multi-Protocol Support:** Works over Wi-Fi, Thread, and Ethernet
- **Interoperability:** Devices from different manufacturers work together
- **Security:** Built-in security features including encryption and authentication
- **Local Control:** Devices can communicate locally without requiring cloud connectivity
- **Simple Setup:** Easy commissioning via QR code or pairing code

Matter Network Topology



(continues on next page)

(continued from previous page)

**Network Interfaces:**

- **Wi-Fi:** High-bandwidth connection for devices that require constant power
- **Thread:** Low-power mesh networking for battery-operated devices
- **Ethernet:** Wired connection for stationary devices

Matter Library Structure**The library is split into three main components:**

- **Matter:** The main class that manages the Matter network and device commissioning
- **MatterEndPoint:** The base class for all Matter endpoints, which provides common functionality for all endpoint types
- **Specific endpoint classes:** The classes for all Matter endpoints, which provides the specific functionality for each endpoint type

Matter The `Matter` class is the main entry point for all Matter operations. It serves as the central manager that handles:

- **Device Commissioning:** Managing the commissioning process via QR code or manual pairing code
- **Network Connectivity:** Checking and managing Wi-Fi and Thread connections
- **Event Handling:** Monitoring Matter events and device state changes
- **Device Management:** Decommissioning and factory reset functionality

The `Matter` class is implemented as a singleton, meaning there's only one instance available globally. You access it directly as `Matter` without creating an instance.

The `Matter` class provides the following key methods:

- `begin()`: Initializes the Matter stack
- `isDeviceCommissioned()`: Checks if the device is commissioned
- `isWiFiConnected()`: Checks Wi-Fi connection status
- `isThreadConnected()`: Checks Thread connection status
- `isDeviceConnected()`: Checks overall device connectivity
- `isWiFiStationEnabled()`: Checks if Wi-Fi Station mode is supported and enabled
- `isWiFiAccessPointEnabled()`: Checks if Wi-Fi AP mode is supported and enabled
- `isThreadEnabled()`: Checks if Thread network is supported and enabled
- `isBLECommissioningEnabled()`: Checks if BLE commissioning is supported and enabled
- `decommission()`: Factory resets the device
- `getManualPairingCode()`: Gets the manual pairing code for commissioning
- `getOnboardingQRCodeUrl()`: Gets the QR code URL for commissioning
- `onEvent()`: Sets a callback for Matter events

MatterEndPoint The `MatterEndPoint` class is the base class for all Matter endpoints. It provides common functionality for all endpoint types.

- **Endpoint Management:** Each endpoint has a unique endpoint ID for identification
- **Attribute Access:** Methods to get and set attribute values from Matter clusters
- **Identify Cluster:** Support for device identification (visual feedback)
- **Secondary Network Interfaces:** Support for multiple network interfaces (Wi-Fi, Thread, Ethernet)
- **Attribute Change Callbacks:** Base framework for handling attribute changes from Matter controllers

MatterEndPoint

About The `MatterEndPoint` class is the base class for all Matter endpoints. It provides common functionality for all endpoint types.

- **Endpoint Management:** Each endpoint has a unique endpoint ID for identification within the Matter network
- **Attribute Access:** Methods to get and set attribute values from Matter clusters
- **Identify Cluster:** Support for device identification (visual feedback like LED blinking)
- **Secondary Network Interfaces:** Support for multiple network interfaces (Wi-Fi, Thread, Ethernet)
- **Attribute Change Callbacks:** Base framework for handling attribute changes from Matter controllers

All Matter endpoint classes inherit from `MatterEndPoint`, providing a consistent interface and common functionality across all device types.

MatterEndPoint APIs

Endpoint Management

getEndPointId Gets the current Matter Accessory endpoint ID.

```
uint16_t getEndPointId();
```

This function will return the endpoint number (typically 1-254).

setEndPointId Sets the current Matter Accessory endpoint ID.

```
void setEndPointId(uint16_t ep);
```

- `ep` - Endpoint number to set

Secondary Network Interface

createSecondaryNetworkInterface Creates a secondary network interface endpoint. This can be used for devices that support multiple network interfaces, such as Ethernet, Thread and Wi-Fi.

```
bool createSecondaryNetworkInterface();
```

This function will return `true` if successful, `false` otherwise.

getSecondaryNetworkEndPointId Gets the secondary network interface endpoint ID.

```
uint16_t getSecondaryNetworkEndPointId();
```

This function will return the secondary network endpoint ID, or 0 if not created.

Attribute Management

getAttribute Gets a pointer to an attribute from its cluster ID and attribute ID.

```
esp_matter::attribute_t *getAttribute(uint32_t cluster_id, uint32_t attribute_id);
```

- `cluster_id` - Cluster ID (e.g., `OnOff::Attributes::OnOff::Id`)
- `attribute_id` - Attribute ID (e.g., `OnOff::Attributes::OnOff::Id`)

This function will return a pointer to the attribute, or `NULL` if not found.

getAttributeVal Gets the value of an attribute from its cluster ID and attribute ID.

```
bool getAttributeVal(uint32_t cluster_id, uint32_t attribute_id, esp_matter_attr_val_t *attrVal);
```

- cluster_id - Cluster ID
- attribute_id - Attribute ID
- attrVal - Pointer to store the attribute value

This function will return `true` if successful, `false` otherwise.

setAttributeVal Sets the value of an attribute from its cluster ID and attribute ID.

```
bool setAttributeVal(uint32_t cluster_id, uint32_t attribute_id, esp_matter_attr_val_t *attrVal);
```

- cluster_id - Cluster ID
- attribute_id - Attribute ID
- attrVal - Pointer to the attribute value to set

This function will return `true` if successful, `false` otherwise.

updateAttributeVal Updates the value of an attribute from its cluster ID. This is typically used for read-only attributes that are updated by the device itself (e.g., sensor readings).

```
bool updateAttributeVal(uint32_t cluster_id, uint32_t attribute_id, esp_matter_attr_val_t *attrVal);
```

- cluster_id - Cluster ID
- attribute_id - Attribute ID
- attrVal - Pointer to the attribute value to update

This function will return `true` if successful, `false` otherwise.

Identify Cluster

onIdentify Sets a callback function for the Identify cluster. This callback is invoked when clients interact with the Identify Cluster of a specific endpoint.

```
void onIdentify(EndPointIdentifyCB onEndPointIdentifyCB);
```

- onEndPointIdentifyCB - Function pointer to the callback function. The callback receives a boolean parameter indicating if identify is enabled.

The callback signature is:

```
bool identifyCallback(bool identifyIsEnabled);
```

When `identifyIsEnabled` is `true`, the device should provide visual feedback (e.g., blink an LED). When `false`, the device should stop the identification feedback.

Example usage:

```
myEndpoint.onIdentify([](bool identifyIsEnabled) {
    if (identifyIsEnabled) {
        // Start blinking LED
        digitalWrite(LED_PIN, HIGH);
    } else {
        // Stop blinking LED
    }
});
```

(continues on next page)

(continued from previous page)

```

        digitalWrite(LED_PIN, LOW);
    }
    return true;
});

```

Attribute Change Callback

attributeChangeCB This function is called by the Matter internal event processor when an attribute changes. It can be overridden by the application if necessary.

```

virtual bool attributeChangeCB(uint16_t endpoint_id, uint32_t cluster_id, uint32_t attribute_id, esp_matter_attr_val_t *val);

```

- `endpoint_id` - Endpoint ID where the attribute changed
- `cluster_id` - Cluster ID of the changed attribute
- `attribute_id` - Attribute ID that changed
- `val` - Pointer to the new attribute value

This function should return `true` if the change was handled successfully, `false` otherwise.

All endpoint classes implement this function to handle attribute changes specific to their device type. You typically don't need to override this unless you need custom behavior.

Supported Endpoints The Matter library provides specialized endpoint classes that inherit from `MatterEndpoint`. Each endpoint type includes specific clusters and functionality relevant to that device category.

MatterColorLight

About The `MatterColorLight` class provides a color light endpoint for Matter networks with RGB color control using the HSV color model. This endpoint implements the Matter lighting standard for full-color lighting control.

Features: * On/off control * RGB color control with HSV color model * State persistence support * Callback support for state and color changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * RGB smart lights * Color-changing lights * Mood lighting * Entertainment lighting control * Smart home color automation

API Reference

Constructor

MatterColorLight Creates a new Matter color light endpoint.

```

MatterColorLight();

```

Initialization

begin Initializes the Matter color light endpoint with optional initial state and color.

```
bool begin(bool initialState = false, espHsvColor_t colorHSV = {0, 254, 31});
```

- `initialState` - Initial on/off state (default: `false` = off)
- `colorHSV` - Initial HSV color (default: red 12% intensity HSV(0, 254, 31))

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter light events.

```
void end();
```

On/Off Control

setOnOff Sets the on/off state of the light.

```
bool setOnOff(bool newState);
```

getOnOff Gets the current on/off state.

```
bool getOnOff();
```

toggle Toggles the on/off state.

```
bool toggle();
```

Color Control

setColorRGB Sets the color using RGB values.

```
bool setColorRGB(espRgbColor_t rgbColor);
```

- `rgbColor` - RGB color structure with red, green, and blue values (0-255 each)

getColorRGB Gets the current color as RGB values.

```
espRgbColor_t getColorRGB();
```

setColorHSV Sets the color using HSV values.

```
bool setColorHSV(espHsvColor_t hsvColor);
```

- `hsvColor` - HSV color structure with hue (0-360), saturation (0-254), and value/brightness (0-254)

getColorHSV Gets the current color as HSV values.

```
espHsvColor_t getColorHSV();
```

Event Handling

onChange Sets a callback for when any parameter changes.

```
void onChange(EndPointCB onChangeCB);
```

The callback signature is:

```
bool onChangeCallback(bool newState, espHsvColor_t newColor);
```

onChangeOnOff Sets a callback for on/off state changes.

```
void onChangeOnOff(EndPointOnOffCB onChangeCB);
```

onChangeColorHSV Sets a callback for color changes.

```
void onChangeColorHSV(EndPointRGBColorCB onChangeCB);
```

updateAccessory Updates the physical light state using current Matter internal state.

```
void updateAccessory();
```

Operators

bool operator Returns current on/off state.

```
operator bool();
```

Assignment operator Turns light on or off.

```
void operator=(bool state);
```

Example

Color Light

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
//
// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
```

(continues on next page)

(continued from previous page)

```

#include <WiFi.h>
#endif
#include <Preferences.h>

// List of Matter Endpoints for this Node
// Color Light Endpoint
MatterColorLight ColorLight;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// it will keep last OnOff & HSV Color state stored, using Preferences
Preferences matterPref;
const char *onOffPrefKey = "OnOff";
const char *hsvColorPrefKey = "HSV";

// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↳BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                 // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

// Set the RGB LED Light based on the current state of the Color Light
bool setLightState(bool state, espHsvColor_t colorHSV) {

    if (state) {
#ifdef RGB_BUILTIN
        espRgbColor_t rgbColor = espHsvColorToRgbColor(colorHSV);
        // set the RGB LED
        rgbLedWrite(ledPin, rgbColor.r, rgbColor.g, rgbColor.b);
#else
        // No Color RGB LED, just use the HSV value (brightness) to control the LED
        analogWrite(ledPin, colorHSV.v);
#endif
    } else {
#ifdef RGB_BUILTIN
        // after analogWrite(), it is necessary to set the GPIO to digital mode first
        pinMode(ledPin, OUTPUT);
#endif
        digitalWrite(ledPin, LOW);
    }
    // store last HSV Color and OnOff state for when the Light is restarted / power_
↳goes off

```

(continues on next page)

(continued from previous page)

```

    matterPref.putBool(onOffPrefKey, state);
    matterPref.putUInt(hsvColorPrefKey, colorHSV.h << 16 | colorHSV.s << 8 | ↵
↵colorHSV.v);
    // This callback must return the success state to Matter core
    return true;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will act as a toggle switch
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the LED (light) GPIO and Matter End Point
    pinMode(ledPin, OUTPUT);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↵Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // We start by connecting to a WiFi network
        Serial.print("Connecting to ");
        Serial.println(ssid);
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println("\r\nWiFi connected");
        Serial.println("IP address: ");
        Serial.println(WiFi.localIP());
        delay(500);
    #endif

    // Initialize Matter EndPoint
    matterPref.begin("MatterPrefs", false);
    // default OnOff state is ON if not stored before
    bool lastOnOffState = matterPref.getBool(onOffPrefKey, true);
    // default HSV color is blue HSV(169, 254, 254)
    uint32_t prefHsvColor = matterPref.getUInt(hsvColorPrefKey, 169 << 16 | 254 << 8_
↵| 254);
    espHsvColor_t lastHsvColor = {uint8_t(prefHsvColor >> 16), uint8_t(prefHsvColor >
↵> 8), uint8_t(prefHsvColor)};
    ColorLight.begin(lastOnOffState, lastHsvColor);
    // set the callback function to handle the Light state change
    ColorLight.onChange(setLightState);

    // lambda functions are used to set the attribute change callbacks
    ColorLight.onChangeOnOff([](bool state) {
        Serial.printf("Light OnOff changed to %s\r\n", state ? "ON" : "OFF");
        return true;
    });
    ColorLight.onChangeColorHSV([](HsvColor_t hsvColor) {
        Serial.printf("Light HSV Color changed to (%u,%u,%u)\r\n", hsvColor.h, ↵
↵hsvColor.s, hsvColor.v);
        return true;
    });

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();
    // This may be a restart of a already commissioned Matter accessory

```

(continues on next page)

(continued from previous page)

```

    if (Matter.isDeviceCommissioned()) {
        Serial.println("Matter Node is commissioned and connected to the network.
↪Ready for use.");
        Serial.printf(
            "Initial state: %s | RGB Color: (%u,%u,%u) \r\n", ColorLight ? "ON" : "OFF",
↪ColorLight.getColorRGB().r, ColorLight.getColorRGB().g,
            ColorLight.getColorRGB().b
        );
        // configure the Light based on initial on-off state and its color
        ColorLight.updateAccessory();
    }
}

void loop() {
    // Check Matter Light Commissioning state, which may change during execution of
↪loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code
↪or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↪str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Light Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for
↪commissioning.");
            }
        }
        Serial.printf(
            "Initial state: %s | RGB Color: (%u,%u,%u) \r\n", ColorLight ? "ON" : "OFF",
↪ColorLight.getColorRGB().r, ColorLight.getColorRGB().g,
            ColorLight.getColorRGB().b
        );
        // configure the Light based on initial on-off state and its color
        ColorLight.updateAccessory();
        Serial.println("Matter Node is commissioned and connected to the network.
↪Ready for use.");
    }

    // A button is also used to control the light
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    // Onboard User Button is used as a Light toggle switch or to decommission it
    uint32_t time_diff = millis() - button_time_stamp;
    if (digitalRead(buttonPin) == HIGH && button_state && time_diff > debounceTime) {
        // Toggle button is released - toggle the light
        Serial.println("User button released. Toggling Light!");
        ColorLight.toggle(); // Matter Controller also can see the change
        button_state = false; // released
    }
}

```

(continues on next page)

(continued from previous page)

```

// Onboard User Button is kept pressed for longer than 5 seconds in order to
↳decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning the Light Matter Accessory. It shall be
↳commissioned again.");
    ColorLight = false; // turn the light off
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot
↳takes a second or so
}
}

```

MatterColorTemperatureLight

About The `MatterColorTemperatureLight` class provides a color temperature light endpoint for Matter networks with brightness and color temperature control. This endpoint implements the Matter lighting standard for lights that support color temperature adjustment (warm white to cool white).

Features: * On/off control * Brightness level control (0-255) * Color temperature control (100-500 mireds) * State persistence support * Callback support for state, brightness, and temperature changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Tunable white lights * Color temperature adjustable lights * Smart lighting with warm/cool control * Circadian lighting * Smart home lighting automation

API Reference

Constructor

MatterColorTemperatureLight Creates a new Matter color temperature light endpoint.

```
MatterColorTemperatureLight();
```

Initialization

begin Initializes the Matter color temperature light endpoint with optional initial state, brightness, and color temperature.

```

bool begin(bool initialState = false, uint8_t brightness = 64, uint16_t
↳colorTemperature = 370);

```

- `initialState` - Initial on/off state (default: `false` = off)
- `brightness` - Initial brightness level (0-255, default: 64 = 25%)
- `colorTemperature` - Initial color temperature in mireds (100-500, default: 370 = Soft White)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter light events.

```
void end();
```

Constants

MAX_BRIGHTNESS Maximum brightness value (255).

```
static const uint8_t MAX_BRIGHTNESS = 255;
```

MAX_COLOR_TEMPERATURE Maximum color temperature value (500 mireds = cool white).

```
static const uint16_t MAX_COLOR_TEMPERATURE = 500;
```

MIN_COLOR_TEMPERATURE Minimum color temperature value (100 mireds = warm white).

```
static const uint16_t MIN_COLOR_TEMPERATURE = 100;
```

On/Off Control

setOnOff Sets the on/off state of the light.

```
bool setOnOff(bool newState);
```

getOnOff Gets the current on/off state.

```
bool getOnOff();
```

toggle Toggles the on/off state.

```
bool toggle();
```

Brightness Control

setBrightness Sets the brightness level.

```
bool setBrightness(uint8_t newBrightness);
```

- newBrightness - Brightness level (0-255)

getBrightness Gets the current brightness level.

```
uint8_t getBrightness();
```

Color Temperature Control

setColorTemperature Sets the color temperature.

```
bool setColorTemperature(uint16_t newTemperature);
```

- newTemperature - Color temperature in mireds (100-500)

Note: Color temperature is measured in mireds (micro reciprocal degrees). Lower values (100-200) are warm white, higher values (400-500) are cool white.

getColorTemperature Gets the current color temperature.

```
uint16_t getColorTemperature();
```

Event Handling

onChange Sets a callback for when any parameter changes.

```
void onChange(EndPointCB onChangeCB);
```

The callback signature is:

```
bool onChangeCallback(bool newState, uint8_t newBrightness, uint16_t  
↪ newTemperature);
```

onChangeOnOff Sets a callback for on/off state changes.

```
void onChangeOnOff(EndPointOnOffCB onChangeCB);
```

onChangeBrightness Sets a callback for brightness changes.

```
void onChangeBrightness(EndPointBrightnessCB onChangeCB);
```

onChangeColorTemperature Sets a callback for color temperature changes.

```
void onChangeColorTemperature(EndPointTemperatureCB onChangeCB);
```

updateAccessory Updates the physical light state using current Matter internal state.

```
void updateAccessory();
```

Operators

bool operator Returns current on/off state.

```
operator bool();
```

Assignment operator Turns light on or off.

```
void operator=(bool state);
```

Example

Color Temperature Light

```

// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif
#include <Preferences.h>

// List of Matter Endpoints for this Node
// Color Temperature CW/WW Light Endpoint
MatterColorTemperatureLight CW_WW_Light;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// it will keep last OnOff & Brightness state stored, using Preferences
Preferences matterPref;
const char *onOffPrefKey = "OnOff";
const char *brightnessPrefKey = "Brightness";
const char *temperaturePrefKey = "Temperature";

// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↪BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                 // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

// Set the RGB LED Light based on the current state of the Color Temperature Light
bool setLightState(bool state, uint8_t brightness, uint16_t temperature_Mireds) {

```

(continues on next page)

(continued from previous page)

```

    if (state) {
#ifdef RGB_BUILTIN
        espRgbColor_t rgb_ct = espCTToRgbColor(temperature_Mireds);
        // simple intensity correction
        float brightnessPercent = (float)brightness / MatterColorTemperatureLight::MAX_
↳BRIGHTNESS;
        rgb_ct.r = brightnessPercent * rgb_ct.r;
        rgb_ct.g = brightnessPercent * rgb_ct.g;
        rgb_ct.b = brightnessPercent * rgb_ct.b;
        // set the RGB LED
        rgbLedWrite(ledPin, rgb_ct.r, rgb_ct.g, rgb_ct.b);
#else
        // No Color RGB LED, just use the brightness to control the LED
        analogWrite(ledPin, brightness);
#endif
    } else {
#ifdef RGB_BUILTIN
        // after analogWrite(), it is necessary to set the GPIO to digital mode first
        pinMode(ledPin, OUTPUT);
#endif
        digitalWrite(ledPin, LOW);
    }
    // store last Brightness and OnOff state for when the Light is restarted / power_
↳goes off
    matterPref.putUChar(brightnessPrefKey, brightness);
    matterPref.putBool(onOffPrefKey, state);
    matterPref.putUShort(temperaturePrefKey, temperature_Mireds);
    // This callback must return the success state to Matter core
    return true;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will act as a toggle switch
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the LED (light) GPIO and Matter End Point
    pinMode(ledPin, OUTPUT);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // We start by connecting to a WiFi network
        Serial.print("Connecting to ");
        Serial.println(ssid);
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println("\r\nWiFi connected");
        Serial.println("IP address: ");
        Serial.println(WiFi.localIP());
        delay(500);
    #endif

    // Initialize Matter EndPoint
    matterPref.begin("MatterPrefs", false);

```

(continues on next page)

(continued from previous page)

```

// default OnOff state is ON if not stored before
bool lastOnOffState = matterPref.getBool(onOffPrefKey, true);
// default brightness ~= 6% (15/255)
uint8_t lastBrightness = matterPref.getUChar(brightnessPrefKey, 15);
// default temperature ~= 454 Mireds (Warm White)
uint16_t lastTemperature = matterPref.getUShort(temperaturePrefKey, WARM_WHITE_
↪COLOR_TEMPERATURE.ctMireds);
CW_WW_Light.begin(lastOnOffState, lastBrightness, lastTemperature);
// set the callback function to handle the Light state change
CW_WW_Light.onChange(setLightState);

// lambda functions are used to set the attribute change callbacks
CW_WW_Light.onChangeOnOff([](bool state) {
    Serial.printf("Light OnOff changed to %s\r\n", state ? "ON" : "OFF");
    return true;
});
CW_WW_Light.onChangeBrightness([](uint8_t level) {
    Serial.printf("Light Brightness changed to %u\r\n", level);
    return true;
});
CW_WW_Light.onChangeColorTemperature([](uint16_t temperature) {
    Serial.printf("Light Color Temperature changed to %u\r\n", temperature);
    return true;
});

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();
// This may be a restart of a already commissioned Matter accessory
if (Matter.isDeviceCommissioned()) {
    Serial.println("Matter Node is commissioned and connected to the network.↪
↪Ready for use.");
    Serial.printf(
        "Initial state: %s | brightness: %u | Color Temperature: %u mireds \r\n", CW_
↪WW_Light ? "ON" : "OFF", CW_WW_Light.getBrightness(),
        CW_WW_Light.getColorTemperature()
    );
    // configure the Light based on initial on-off state and brightness
    CW_WW_Light.updateAccessory();
}
}

void loop() {
    // Check Matter Light Commissioning state, which may change during execution of↪
↪loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code↪
↪or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↪str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Light Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for↪
↪commissioning.");
            }
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    Serial.printf(
        "Initial state: %s | brightness: %u | Color Temperature: %u mireds \r\n", CW_
↪WW_Light ? "ON" : "OFF", CW_WW_Light.getBrightness(),
        CW_WW_Light.getColorTemperature()
    );
    // configure the Light based on initial on-off state and brightness
    CW_WW_Light.updateAccessory();
    Serial.println("Matter Node is commissioned and connected to the network.↪
↪Ready for use.");
}

// A button is also used to control the light
// Check if the button has been pressed
if (digitalRead(buttonPin) == LOW && !button_state) {
    // deals with button debouncing
    button_time_stamp = millis(); // record the time while the button is pressed.
    button_state = true;          // pressed.
}

// Onboard User Button is used as a Light toggle switch or to decommission it
uint32_t time_diff = millis() - button_time_stamp;
if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
    button_state = false; // released
    // Toggle button is released - toggle the light
    Serial.println("User button released. Toggling Light!");
    CW_WW_Light.toggle(); // Matter Controller also can see the change
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to↪
↪decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning the Light Matter Accessory. It shall be↪
↪commissioned again.");
    CW_WW_Light = false; // turn the light off
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot↪
↪takes a second or so
}
}

```

MatterContactSensor

About The `MatterContactSensor` class provides a contact sensor endpoint for Matter networks. This endpoint implements the Matter contact sensing standard for detecting open/closed states (e.g., doors, windows).

Features: * Contact state reporting (open/closed) * Simple boolean state * Read-only sensor (no control functionality) * Automatic state updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Door/window sensors * Contact switches * Security systems * Access control * Smart home automation triggers

API Reference

Constructor

MatterContactSensor Creates a new Matter contact sensor endpoint.

```
MatterContactSensor();
```

Initialization

begin Initializes the Matter contact sensor endpoint with an initial contact state.

```
bool begin(bool _contactState = false);
```

- `_contactState` - Initial contact state (`true` = closed, `false` = open, default: `false`)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter contact sensor events.

```
void end();
```

Contact State Control

setContact Sets the contact state.

```
bool setContact(bool _contactState);
```

- `_contactState` - Contact state (`true` = closed, `false` = open)

This function will return `true` if successful, `false` otherwise.

getContact Gets the current contact state.

```
bool getContact();
```

This function will return `true` if closed, `false` if open.

Operators

bool operator Returns the current contact state.

```
operator bool();
```

Example:

```
if (mySensor) {  
    Serial.println("Contact is closed");  
} else {  
    Serial.println("Contact is open");  
}
```

Assignment operator Sets the contact state.

```
void operator=(bool _contactState);
```

Example:

```
mySensor = true;    // Set contact to closed
mySensor = false;   // Set contact to open
```

Example

Contact Sensor

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
↳ Controller messages.
 *
 * The example will create a Matter Contact Sensor Device.
 * The Contact Sensor state can be toggled by pressing the onboard button.
 * The Contact Sensor state will be indicated by the onboard LED.
 * The Contact Sensor state will be simulated to change every 20 seconds.
 *
 * The onboard button can be kept pressed for 5 seconds to decommission the Matter
↳ Node.
 * The example will also show the manual commissioning code and QR code to be used
↳ in the Matter environment.
 */

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Contact Sensor Endpoint
MatterContactSensor ContactSensor;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
↳ Network
#if !CONFIG_ENABLE_CHIPOBLE
```

(continues on next page)

(continued from previous page)

```

// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// LED will be used to indicate the Contact Sensor state
// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↪BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here - decommissioning and Manual Contact Sensor_
↪toggle button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission_
↪the Matter Node
    // The button will also be used to manually toggle the Contact Sensor state
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the LED (light) GPIO and Matter End Point
    pinMode(ledPin, OUTPUT);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    #endif

    // set initial contact sensor state as false (default)
    ContactSensor.begin();
    digitalWrite(ledPin, LOW); // LED OFF

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();

    // Check Matter Accessory Commissioning state, which may change during execution_
↪of loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
    }

```

(continues on next page)

(continued from previous page)

```

    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code_
↳or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Contact Sensor Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for_
↳commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network._
↳Ready for use.");
}

bool simulatedHWContactSensor() {
    // Simulated Contact Sensor
    static bool contactState = false;
    static uint32_t lastTime = 0;

    // Simulate a Contact Sensor state change every 20 seconds
    if (millis() - lastTime > 20000) {
        contactState = !contactState;
        lastTime = millis();
    }
    return contactState;
}

void loop() {
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }

    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
        button_state = false; // released
        // button is released - toggle Contact State (Open/Closed)
        ContactSensor.setContact(!ContactSensor.getContact()); // same as_
↳ContactSensor = !ContactSensor;
        Serial.printf("User button released. Setting the Contact Sensor to %s.\r\n", _
↳ContactSensor ? "Closed" : "Open");
        // LED will indicate the Contact Sensor state
        if (ContactSensor) {
            digitalWrite(ledPin, HIGH); // LED ON
        } else {
            digitalWrite(ledPin, LOW); // LED OFF
        }
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to_
↳decommission matter node
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Contact Sensor Matter Accessory. It shall be_

```

(continues on next page)

(continued from previous page)

```

↪ commissioned again.");
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot_
↪ takes a second or so
}

// Simulated Contact Sensor
ContactSensor.setContact(simulatedHWContactSensor());

delay(50);
}

```

MatterDimmableLight

About The `MatterDimmableLight` class provides a dimmable light endpoint for Matter networks. This endpoint implements the Matter lighting standard for lights with brightness control.

Features: * On/off control * Brightness level control (0-255) * State persistence support * Callback support for state and brightness changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Dimmable smart lights * Brightness control switches * Smart home lighting automation * Variable brightness lighting

API Reference

Constructor

MatterDimmableLight Creates a new Matter dimmable light endpoint.

```
MatterDimmableLight();
```

Initialization

begin Initializes the Matter dimmable light endpoint with optional initial state and brightness.

```
bool begin(bool initialState = false, uint8_t brightness = 64);
```

- `initialState` - Initial on/off state (`true` = on, `false` = off, default: `false`)
- `brightness` - Initial brightness level (0-255, default: 64 = 25%)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter light events.

```
void end();
```

On/Off Control

setOnOff Sets the on/off state of the light.

```
bool setOnOff(bool newState);
```

- `newState` - New state (`true` = on, `false` = off)

This function will return `true` if successful, `false` otherwise.

getOnOff Gets the current on/off state of the light.

```
bool getOnOff();
```

This function will return `true` if the light is on, `false` if off.

toggle Toggles the on/off state of the light.

```
bool toggle();
```

This function will return `true` if successful, `false` otherwise.

Brightness Control

setBrightness Sets the brightness level of the light.

```
bool setBrightness(uint8_t newBrightness);
```

- `newBrightness` - New brightness level (0-255, where 0 = off, 255 = maximum brightness)

This function will return `true` if successful, `false` otherwise.

getBrightness Gets the current brightness level of the light.

```
uint8_t getBrightness();
```

This function will return the brightness level (0-255).

Constants

MAX_BRIGHTNESS Maximum brightness value constant.

```
static const uint8_t MAX_BRIGHTNESS = 255;
```

Operators

bool operator Returns the current on/off state of the light.

```
operator bool();
```

Assignment operator Turns the light on or off.

```
void operator=(bool state);
```

Event Handling

onChange Sets a callback function to be called when any parameter changes.

```
void onChange(EndPointCB onChangeCB);
```

- onChangeCB - Function to call when state changes

The callback signature is:

```
bool onChangeCallback(bool newState, uint8_t newBrightness);
```

- newState - New on/off state
- newBrightness - New brightness level (0-255)

onChangeOnOff Sets a callback function to be called when the on/off state changes.

```
void onChangeOnOff(EndPointOnOffCB onChangeCB);
```

- onChangeCB - Function to call when on/off state changes

The callback signature is:

```
bool onChangeCallback(bool newState);
```

onChangeBrightness Sets a callback function to be called when the brightness level changes.

```
void onChangeBrightness(EndPointBrightnessCB onChangeCB);
```

- onChangeCB - Function to call when brightness changes

The callback signature is:

```
bool onChangeCallback(uint8_t newBrightness);
```

updateAccessory Updates the state of the light using the current Matter Light internal state.

```
void updateAccessory();
```

Example

Dimmable Light

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
```

(continues on next page)

(continued from previous page)

```

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif
#include <Preferences.h>

// List of Matter Endpoints for this Node
// Dimmable Light Endpoint
MatterDimmableLight DimmableLight;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid"; // Change this to your WiFi SSID
const char *password = "your-password"; // Change this to your WiFi password
#endif

// it will keep last OnOff & Brightness state stored, using Preferences
Preferences matterPref;
const char *onOffPrefKey = "OnOff";
const char *brightnessPrefKey = "Brightness";

// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↪BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0; // debouncing control
bool button_state = false; // false = released | true = pressed
const uint32_t debounceTime = 250; // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

// Set the RGB LED Light based on the current state of the Dimmable Light
bool setLightState(bool state, uint8_t brightness) {
    if (state) {
#ifdef RGB_BUILTIN
        rgbLedWrite(ledPin, brightness, brightness, brightness);
    #else
        analogWrite(ledPin, brightness);
    #endif
    } else {
#ifdef RGB_BUILTIN
        // after analogWrite(), it is necessary to set the GPIO to digital mode first
        pinMode(ledPin, OUTPUT);
    #endif
        digitalWrite(ledPin, LOW);
    }
    // store last Brightness and OnOff state for when the Light is restarted / power_

```

(continues on next page)

(continued from previous page)

```

↪ goes off
matterPref.putUChar(brightnessPrefKey, brightness);
matterPref.putBool(onOffPrefKey, state);
// This callback must return the success state to Matter core
return true;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will act as a toggle switch
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the LED (light) GPIO and Matter End Point
    pinMode(ledPin, OUTPUT);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
    ↪ Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // We start by connecting to a WiFi network
        Serial.print("Connecting to ");
        Serial.println(ssid);
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println("\r\nWiFi connected");
        Serial.println("IP address: ");
        Serial.println(WiFi.localIP());
        delay(500);
    #endif

    // Initialize Matter EndPoint
    matterPref.begin("MatterPrefs", false);
    // default OnOff state is ON if not stored before
    bool lastOnOffState = matterPref.getBool(onOffPrefKey, true);
    // default brightness ~= 6% (15/255)
    uint8_t lastBrightness = matterPref.getUChar(brightnessPrefKey, 15);
    DimmableLight.begin(lastOnOffState, lastBrightness);
    // set the callback function to handle the Light state change
    DimmableLight.onChange(setLightState);

    // lambda functions are used to set the attribute change callbacks
    DimmableLight.onChangeOnOff([](bool state) {
        Serial.printf("Light OnOff changed to %s\r\n", state ? "ON" : "OFF");
        return true;
    });
    DimmableLight.onChangeBrightness([](uint8_t level) {
        Serial.printf("Light Brightness changed to %u\r\n", level);
        return true;
    });

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();
    // This may be a restart of a already commissioned Matter accessory
    if (Matter.isDeviceCommissioned()) {
        Serial.println("Matter Node is commissioned and connected to the network.
↪ Ready for use.");
        Serial.printf("Initial state: %s | brightness: %u\r\n", DimmableLight ? "ON" :

```

(continues on next page)

(continued from previous page)

```

↪ "OFF", DimmableLight.getBrightness());
    // configure the Light based on initial on-off state and brightness
    DimmableLight.updateAccessory();
}
}

void loop() {
    // Check Matter Light Commissioning state, which may change during execution of ↪
    ↪ loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code ↪
        ↪ or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
        ↪ str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Light Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for ↪
                ↪ commissioning.");
            }
        }
        Serial.printf("Initial state: %s | brightness: %u\r\n", DimmableLight ? "ON" :
        ↪ "OFF", DimmableLight.getBrightness());
        // configure the Light based on initial on-off state and brightness
        DimmableLight.updateAccessory();
        Serial.println("Matter Node is commissioned and connected to the network. ↪
        ↪ Ready for use.");
    }

    // A button is also used to control the light
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    // Onboard User Button is used as a Light toggle switch or to decommission it
    uint32_t time_diff = millis() - button_time_stamp;
    if (digitalRead(buttonPin) == HIGH && button_state && time_diff > debounceTime) {
        // Toggle button is released - toggle the light
        Serial.println("User button released. Toggling Light!");
        DimmableLight.toggle(); // Matter Controller also can see the change
        button_state = false; // released
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to ↪
    ↪ decommission matter node
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning the Light Matter Accessory. It shall be ↪
        ↪ commissioned again.");
        DimmableLight = false; // turn the light off
        Matter.decommission();
        button_time_stamp = millis(); // avoid running decommissioning again, reboot ↪
        ↪ takes a second or so
    }
}

```

(continues on next page)

(continued from previous page)

```
}  
}
```

MatterDimmablePlugin

About The `MatterDimmablePlugin` class provides a dimmable plugin unit endpoint for Matter networks. This endpoint implements the Matter dimmable plugin standard for controlling power outlets, relays, and other dimmable devices with power level control.

Features: * On/off control * Power level control (0-255) * State persistence support * Callback support for state and level changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Dimmable smart power outlets * Variable power control * Smart dimmer plugs * Level-controlled device control

API Reference

Constructor

MatterDimmablePlugin Creates a new Matter dimmable plugin endpoint.

```
MatterDimmablePlugin();
```

Initialization

begin Initializes the Matter dimmable plugin endpoint with optional initial state and level.

```
bool begin(bool initialState = false, uint8_t level = 64);
```

- `initialState` - Initial on/off state (`true` = on, `false` = off, default: `false`)
- `level` - Initial power level (0-255, default: 64 = 25%)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter plugin events.

```
void end();
```

On/Off Control

setOnOff Sets the on/off state of the plugin.

```
bool setOnOff(bool newState);
```

- `newState` - New state (`true` = on, `false` = off)

This function will return `true` if successful, `false` otherwise.

getOnOff Gets the current on/off state of the plugin.

```
bool getOnOff();
```

This function will return `true` if the plugin is on, `false` if off.

toggle Toggles the on/off state of the plugin.

```
bool toggle();
```

This function will return `true` if successful, `false` otherwise.

Level Control

setLevel Sets the power level of the plugin.

```
bool setLevel(uint8_t newLevel);
```

- `newLevel` - New power level (0-255, where 0 = off, 255 = maximum level)

This function will return `true` if successful, `false` otherwise.

getLevel Gets the current power level of the plugin.

```
uint8_t getLevel();
```

This function will return the power level (0-255).

Constants

MAX_LEVEL Maximum power level value constant.

```
static const uint8_t MAX_LEVEL = 255;
```

Operators

bool operator Returns the current on/off state of the plugin.

```
operator bool();
```

Example:

```
if (myPlugin) {  
    Serial.println("Plugin is on");  
}
```

Assignment operator Turns the plugin on or off.

```
void operator=(bool state);
```

Example:

```
myPlugin = true;    // Turn plugin on  
myPlugin = false;   // Turn plugin off
```

Event Handling

onChange Sets a callback function to be called when any parameter changes.

```
void onChange(EndPointCB onChangeCB);
```

- onChangeCB - Function to call when state changes

The callback signature is:

```
bool onChangeCallback(bool newState, uint8_t newLevel);
```

- newState - New on/off state (true = on, false = off)
- newLevel - New power level (0-255)

onChangeOnOff Sets a callback function to be called when the on/off state changes.

```
void onChangeOnOff(EndPointOnOffCB onChangeCB);
```

- onChangeCB - Function to call when on/off state changes

The callback signature is:

```
bool onChangeCallback(bool newState);
```

- newState - New on/off state (true = on, false = off)

onChangeLevel Sets a callback function to be called when the power level changes.

```
void onChangeLevel(EndPointLevelCB onChangeCB);
```

- onChangeCB - Function to call when level changes

The callback signature is:

```
bool onChangeCallback(uint8_t newLevel);
```

- newLevel - New power level (0-255)

updateAccessory Updates the state of the plugin using the current Matter Plugin internal state.

```
void updateAccessory();
```

This function will call the registered callback with the current state.

Example

Dimmable Plugin

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
```

(continues on next page)

(continued from previous page)

```

// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif
#include <Preferences.h>

// List of Matter Endpoints for this Node
// Dimmable Plugin Endpoint
MatterDimmablePlugin DimmablePlugin;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// it will keep last OnOff & Level state stored, using Preferences
Preferences matterPref;
const char *onOffPrefKey = "OnOff";
const char *levelPrefKey = "Level";

// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t pluginPin = RGB_BUILTIN;    // Using built-in RGB LED for visualization
#else
const uint8_t pluginPin = 2;              // Set your pin here if your board has not defined_
↳RGB_BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN;       // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

// Set the RGB LED Plugin output based on the current state and level
bool setPluginState(bool state, uint8_t level) {
    Serial.printf("User Callback :: New Plugin State = %s, Level = %u\r\n", state ?
↳"ON" : "OFF", level);
    if (state) {
        // Plugin is ON - set RGB LED level (0-255 maps to 0-100% power)
#ifdef RGB_BUILTIN
        rgbLedWrite(pluginPin, level, level, level);
#else
        analogWrite(pluginPin, level);
#endif
    }
}

```

(continues on next page)

(continued from previous page)

```

    } else {
        // Plugin is OFF - turn off output
#ifdef RGB_BUILTIN
        // After analogWrite(), it is necessary to set the GPIO to digital mode first
        pinMode(pluginPin, OUTPUT);
#endif
        digitalWrite(pluginPin, LOW);
    }
    // store last Level and OnOff state for when the Plugin is restarted / power_
    ↪ goes off
    matterPref.putUChar(levelPrefKey, level);
    matterPref.putBool(onOffPrefKey, state);
    // This callback must return the success state to Matter core
    return true;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will act as a toggle switch
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the RGB LED (plugin) GPIO
    pinMode(pluginPin, OUTPUT);
    digitalWrite(pluginPin, LOW); // Start with plugin off

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
    ↪ Network
#ifdef !CONFIG_ENABLE_CHIPOBLE
    // We start by connecting to a WiFi network
    Serial.print("Connecting to ");
    Serial.println(ssid);
    // Manually connect to WiFi
    WiFi.begin(ssid, password);
    // Wait for connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\r\nWiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
    delay(500);
#endif

    // Initialize Matter EndPoint
    matterPref.begin("MatterPrefs", false);
    // default OnOff state is OFF if not stored before
    bool lastOnOffState = matterPref.getBool(onOffPrefKey, false);
    // default level ~= 25% (64/255)
    uint8_t lastLevel = matterPref.getUChar(levelPrefKey, 64);
    DimmablePlugin.begin(lastOnOffState, lastLevel);
    // set the callback function to handle the Plugin state change
    DimmablePlugin.onChange(setPluginState);

    // lambda functions are used to set the attribute change callbacks
    DimmablePlugin.onChangeOnOff([](bool state) {
        Serial.printf("Plugin OnOff changed to %s\r\n", state ? "ON" : "OFF");
        return true;
    });
    DimmablePlugin.onChangeLevel([](uint8_t level) {
        Serial.printf("Plugin Level changed to %u\r\n", level);
    });

```

(continues on next page)

(continued from previous page)

```

    return true;
});

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();
// This may be a restart of a already commissioned Matter accessory
if (Matter.isDeviceCommissioned()) {
    Serial.println("Matter Node is commissioned and connected to the network.↵
↵Ready for use.");
    Serial.printf("Initial state: %s | level: %u↵↵", DimmablePlugin ? "ON" : "OFF
↵", DimmablePlugin.getLevel());
    // configure the Plugin based on initial on-off state and level
    DimmablePlugin.updateAccessory();
}
}

void loop() {
    // Check Matter Plugin Commissioning state, which may change during execution of↵
    ↵loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code↵
↵or QR code");
        Serial.printf("Manual pairing code: %s↵↵", Matter.getManualPairingCode().c_
↵str());
        Serial.printf("QR code URL: %s↵↵", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Plugin Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for↵
↵commissioning.");
            }
        }
        Serial.printf("Initial state: %s | level: %u↵↵", DimmablePlugin ? "ON" : "OFF
↵", DimmablePlugin.getLevel());
        // configure the Plugin based on initial on-off state and level
        DimmablePlugin.updateAccessory();
        Serial.println("Matter Node is commissioned and connected to the network.↵
↵Ready for use.");
    }

    // A button is also used to control the plugin
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }

    // Onboard User Button is used as a Plugin toggle switch or to decommission it
    uint32_t time_diff = millis() - button_time_stamp;
    if (digitalRead(buttonPin) == HIGH && button_state && time_diff > debounceTime) {
        // Toggle button is released - toggle the plugin
        Serial.println("User button released. Toggling Plugin!");
        DimmablePlugin.toggle(); // Matter Controller also can see the change
        button_state = false;    // released
    }
}

```

(continues on next page)

(continued from previous page)

```

// Onboard User Button is kept pressed for longer than 5 seconds in order to
↳decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning the Plugin Matter Accessory. It shall be
↳commissioned again.");
    DimmablePlugin = false; // turn the plugin off
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot
↳takes a second or so
}
}

```

MatterEnhancedColorLight

About The `MatterEnhancedColorLight` class provides an enhanced color light endpoint for Matter networks with full RGB color control, brightness, and color temperature. This endpoint implements the Matter lighting standard for advanced color lighting with all features.

Features: * On/off control * RGB color control with HSV color model * Brightness level control (0-255) * Color temperature control (100-500 mireds) * State persistence support * Callback support for all parameter changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Full-featured RGB smart lights * Advanced color and temperature control * Mood lighting with all features * Entertainment lighting * Circadian lighting with color temperature * Smart home advanced lighting automation

API Reference

Constructor

MatterEnhancedColorLight Creates a new Matter enhanced color light endpoint.

```
MatterEnhancedColorLight();
```

Initialization

begin Initializes the Matter enhanced color light endpoint with optional initial state, color, brightness, and color temperature.

```

bool begin(bool initialState = false, espHsvColor_t colorHSV = {21, 216, 25},
↳uint8_t newBrightness = 25, uint16_t colorTemperature = 454);

```

- `initialState` - Initial on/off state (default: `false` = off)
- `colorHSV` - Initial HSV color (default: HSV(21, 216, 25) = warm white)
- `newBrightness` - Initial brightness level (0-255, default: 25 = 10%)
- `colorTemperature` - Initial color temperature in mireds (100-500, default: 454 = Warm White)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter light events.

```
void end();
```

Constants

MAX_BRIGHTNESS Maximum brightness value (255).

```
static const uint8_t MAX_BRIGHTNESS = 255;
```

MAX_COLOR_TEMPERATURE Maximum color temperature value (500 mireds = cool white).

```
static const uint16_t MAX_COLOR_TEMPERATURE = 500;
```

MIN_COLOR_TEMPERATURE Minimum color temperature value (100 mireds = warm white).

```
static const uint16_t MIN_COLOR_TEMPERATURE = 100;
```

On/Off Control

setOnOff Sets the on/off state of the light.

```
bool setOnOff(bool newState);
```

getOnOff Gets the current on/off state.

```
bool getOnOff();
```

toggle Toggles the on/off state.

```
bool toggle();
```

Color Control

setColorRGB Sets the color using RGB values.

```
bool setColorRGB(espRgbColor_t rgbColor);
```

getColorRGB Gets the current color as RGB values.

```
espRgbColor_t getColorRGB();
```

setColorHSV Sets the color using HSV values.

```
bool setColorHSV(espHsvColor_t hsvColor);
```

getColorHSV Gets the current color as HSV values.

```
espHsvColor_t getColorHSV();
```

Brightness Control

setBrightness Sets the brightness level.

```
bool setBrightness(uint8_t newBrightness);
```

getBrightness Gets the current brightness level.

```
uint8_t getBrightness();
```

Color Temperature Control

setColorTemperature Sets the color temperature.

```
bool setColorTemperature(uint16_t newTemperature);
```

getColorTemperature Gets the current color temperature.

```
uint16_t getColorTemperature();
```

Event Handling

onChange Sets a callback for when any parameter changes.

```
void onChange(EndPointCB onChangeCB);
```

The callback signature is:

```
bool onChangeCallback(bool newState, espHsvColor_t newColor, uint8_t newBrightness,  
↔ uint16_t newTemperature);
```

onChangeOnOff Sets a callback for on/off state changes.

```
void onChangeOnOff(EndPointOnOffCB onChangeCB);
```

onChangeBrightness Sets a callback for brightness changes.

```
void onChangeBrightness(EndPointBrightnessCB onChangeCB);
```

onChangeColorHSV Sets a callback for color changes.

```
void onChangeColorHSV(EndPointRGBColorCB onChangeCB);
```

onChangeColorTemperature Sets a callback for color temperature changes.

```
void onChangeColorTemperature (EndPointTemperatureCB onChangeCB);
```

updateAccessory Updates the physical light state using current Matter internal state.

```
void updateAccessory();
```

Operators

bool operator Returns current on/off state.

```
operator bool();
```

Assignment operator Turns light on or off.

```
void operator=(bool state);
```

Example

Enhanced Color Light

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif
#include <Preferences.h>

// List of Matter Endpoints for this Node
// Color Light Endpoint
MatterEnhancedColorLight EnhancedColorLight;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif
```

(continues on next page)

(continued from previous page)

```

// It will use HSV color to control all Matter Attribute Changes
HsvColor_t currentHSVColor = {0, 0, 0};

// it will keep last OnOff & HSV Color state stored, using Preferences
Preferences matterPref;
const char *onOffPrefKey = "OnOff";
const char *hsvColorPrefKey = "HSV";

// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↪BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0; // debouncing control
bool button_state = false; // false = released | true = pressed
const uint32_t debounceTime = 250; // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

// Set the RGB LED Light based on the current state of the Enhanced Color Light
bool setLightState(bool state, espHsvColor_t colorHSV, uint8_t brightness, uint16_
↪t temperature_Mireds) {

    if (state) {
#ifdef RGB_BUILTIN
        // currentHSVColor keeps final color result
        espRgbColor_t rgbColor = espHsvColorToRgbColor(currentHSVColor);
        // set the RGB LED
        rgbLedWrite(ledPin, rgbColor.r, rgbColor.g, rgbColor.b);
#else
        // No Color RGB LED, just use the HSV value (brightness) to control the LED
        analogWrite(ledPin, colorHSV.v);
#endif
    } else {
#ifdef RGB_BUILTIN
        // after analogWrite(), it is necessary to set the GPIO to digital mode first
        pinMode(ledPin, OUTPUT);
#endif
        digitalWrite(ledPin, LOW);
    }
    // store last HSV Color and OnOff state for when the Light is restarted / power_
↪goes off
    matterPref.putBool(onOffPrefKey, state);
    matterPref.putUInt(hsvColorPrefKey, currentHSVColor.h << 16 | currentHSVColor.s <
↪ < 8 | currentHSVColor.v);
    // This callback must return the success state to Matter core
    return true;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will act as a toggle switch
    pinMode(buttonPin, INPUT_PULLUP);

```

(continues on next page)

(continued from previous page)

```

// Initialize the LED (light) GPIO and Matter End Point
pinMode(ledPin, OUTPUT);

Serial.begin(115200);

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#ifdef CONFIG_ENABLE_CHIPOBLE
// We start by connecting to a WiFi network
Serial.print("Connecting to ");
Serial.println(ssid);
// Manually connect to WiFi
WiFi.begin(ssid, password);
// Wait for connection
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println("\r\nWiFi connected");
Serial.println("IP address: ");
Serial.println(WiFi.localIP());
delay(500);
#endif

// Initialize Matter EndPoint
matterPref.begin("MatterPrefs", false);
// default OnOff state is ON if not stored before
bool lastOnOffState = matterPref.getBool(onOffPrefKey, true);
// default HSV color is (21, 216, 25) - Warm White Color at 10% intensity
uint32_t prefHsvColor = matterPref.getUInt(hsvColorPrefKey, 21 << 16 | 216 << 8,
↳| 25);
currentHSVColor = {uint8_t(prefHsvColor >> 16), uint8_t(prefHsvColor >> 8),
↳uint8_t(prefHsvColor)};
EnhancedColorLight.begin(lastOnOffState, currentHSVColor);
// set the callback function to handle the Light state change
EnhancedColorLight.onChange(setLightState);

// lambda functions are used to set the attribute change callbacks
EnhancedColorLight.onChangeOnOff([](bool state) {
    Serial.printf("Light OnOff changed to %s\r\n", state ? "ON" : "OFF");
    return true;
});
EnhancedColorLight.onChangeColorTemperature([](uint16_t colorTemperature) {
    Serial.printf("Light Color Temperature changed to %u\r\n", colorTemperature);
    // get correspondent Hue and Saturation of the color temperature
    HsvColor_t hsvTemperature =
↳espRgbColorToHsvColor(espCTToRgbColor(colorTemperature));
    // keep previous the brightness and just change the Hue and Saturation
    currentHSVColor.h = hsvTemperature.h;
    currentHSVColor.s = hsvTemperature.s;
    return true;
});
EnhancedColorLight.onChangeBrightness([](uint8_t brightness) {
    Serial.printf("Light brightness changed to %u\r\n", brightness);
    // change current brightness (HSV value)
    currentHSVColor.v = brightness;
    return true;
});
EnhancedColorLight.onChangeColorHSV([](HsvColor_t hsvColor) {
    Serial.printf("Light HSV Color changed to (%u,%u,%u)\r\n", hsvColor.h,
↳hsvColor.s, hsvColor.v);

```

(continues on next page)

(continued from previous page)

```

    // keep the current brightness and just change Hue and Saturation
    currentHsvColor.h = hsvColor.h;
    currentHsvColor.s = hsvColor.s;
    return true;
});

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();
// This may be a restart of a already commissioned Matter accessory
if (Matter.isDeviceCommissioned()) {
    Serial.println("Matter Node is commissioned and connected to the network.␣
↳ Ready for use.");
    Serial.printf(
        "Initial state: %s | RGB Color: (%u,%u,%u) \r\n", EnhancedColorLight ? "ON"␣
↳: "OFF", EnhancedColorLight.getColorRGB().r,
        EnhancedColorLight.getColorRGB().g, EnhancedColorLight.getColorRGB().b
    );
    // configure the Light based on initial on-off state and its color
    EnhancedColorLight.updateAccessory();
}
}

void loop() {
    // Check Matter Light Commissioning state, which may change during execution of␣
↳ loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code␣
↳ or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_␣
↳ str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Light Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for␣
↳ commissioning.");
            }
        }
        Serial.printf(
            "Initial state: %s | RGB Color: (%u,%u,%u) \r\n", EnhancedColorLight ? "ON"␣
↳: "OFF", EnhancedColorLight.getColorRGB().r,
            EnhancedColorLight.getColorRGB().g, EnhancedColorLight.getColorRGB().b
        );
        // configure the Light based on initial on-off state and its color
        EnhancedColorLight.updateAccessory();
        Serial.println("Matter Node is commissioned and connected to the network.␣
↳ Ready for use.");
    }

    // A button is also used to control the light
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }
}

```

(continues on next page)

(continued from previous page)

```

// Onboard User Button is used as a Light toggle switch or to decommission it
uint32_t time_diff = millis() - button_time_stamp;
if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
    button_state = false; // released
    // Toggle button is released - toggle the light
    Serial.println("User button released. Toggling Light!");
    EnhancedColorLight.toggle(); // Matter Controller also can see the change
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to
↳decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning the Light Matter Accessory. It shall be
↳commissioned again.");
    EnhancedColorLight = false; // turn the light off
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot
↳takes a second or so
}
}

```

MatterFan

About The MatterFan class provides a fan endpoint for Matter networks with speed and mode control. This endpoint implements the Matter fan control standard.

Features: * On/off control * Fan speed control (0-100%) * Fan mode control (OFF, LOW, MEDIUM, HIGH, ON, AUTO, SMART) * Fan mode sequence configuration * Callback support for state, speed, and mode changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Smart ceiling fans * Exhaust fans * Ventilation fans * Fan speed controllers * HVAC fan control

API Reference

Constructor

MatterFan Creates a new Matter fan endpoint.

```
MatterFan();
```

Initialization

begin Initializes the Matter fan endpoint with optional initial speed, mode, and mode sequence.

```

bool begin(uint8_t percent = 0, FanMode_t fanMode = FAN_MODE_OFF, FanModeSequence_
↳t fanModeSeq = FAN_MODE_SEQ_OFF_HIGH);

```

- percent - Initial speed percentage (0-100, default: 0)
- fanMode - Initial fan mode (default: FAN_MODE_OFF)
- fanModeSeq - Fan mode sequence configuration (default: FAN_MODE_SEQ_OFF_HIGH)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter fan events.

```
void end();
```

Constants

MAX_SPEED Maximum speed value (100%).

```
static const uint8_t MAX_SPEED = 100;
```

MIN_SPEED Minimum speed value (1%).

```
static const uint8_t MIN_SPEED = 1;
```

OFF_SPEED Speed value when fan is off (0%).

```
static const uint8_t OFF_SPEED = 0;
```

Fan Modes

FanMode_t Fan mode enumeration:

- FAN_MODE_OFF - Fan is off
- FAN_MODE_LOW - Low speed
- FAN_MODE_MEDIUM - Medium speed
- FAN_MODE_HIGH - High speed
- FAN_MODE_ON - Fan is on
- FAN_MODE_AUTO - Auto mode
- FAN_MODE_SMART - Smart mode

Fan Mode Sequences

FanModeSequence_t Fan mode sequence enumeration:

- FAN_MODE_SEQ_OFF_LOW_MED_HIGH - OFF, LOW, MEDIUM, HIGH
- FAN_MODE_SEQ_OFF_LOW_HIGH - OFF, LOW, HIGH
- FAN_MODE_SEQ_OFF_LOW_MED_HIGH_AUTO - OFF, LOW, MEDIUM, HIGH, AUTO
- FAN_MODE_SEQ_OFF_LOW_HIGH_AUTO - OFF, LOW, HIGH, AUTO
- FAN_MODE_SEQ_OFF_HIGH_AUTO - OFF, HIGH, AUTO
- FAN_MODE_SEQ_OFF_HIGH - OFF, HIGH

On/Off Control

setOnOff Sets the on/off state of the fan.

```
bool setOnOff(bool newState, bool performUpdate = true);
```

- newState - New state (true = on, false = off)
- performUpdate - Perform update after setting (default: true)

getOnOff Gets the current on/off state.

```
bool getOnOff();
```

toggle Toggles the on/off state.

```
bool toggle(bool performUpdate = true);
```

Speed Control

setSpeedPercent Sets the fan speed percentage.

```
bool setSpeedPercent(uint8_t newPercent, bool performUpdate = true);
```

- newPercent - Speed percentage (0-100)
- performUpdate - Perform update after setting (default: true)

getSpeedPercent Gets the current speed percentage.

```
uint8_t getSpeedPercent();
```

Mode Control

setMode Sets the fan mode.

```
bool setMode(FanMode_t newMode, bool performUpdate = true);
```

- newMode - Fan mode to set
- performUpdate - Perform update after setting (default: true)

getMode Gets the current fan mode.

```
FanMode_t getMode();
```

getFanModeString Gets a friendly string for the fan mode.

```
static const char *getFanModeString(uint8_t mode);
```

Event Handling

onChange Sets a callback for when any parameter changes.

```
void onChange(EndPointCB onChangeCB);
```

The callback signature is:

```
bool onChangeCallback(FanMode_t newMode, uint8_t newPercent);
```

onChangeMode Sets a callback for mode changes.

```
void onChangeMode(EndPointModeCB onChangeCB);
```

onChangeSpeedPercent Sets a callback for speed changes.

```
void onChangeSpeedPercent(EndPointSpeedCB onChangeCB);
```

updateAccessory Updates the physical fan state using current Matter internal state.

```
void updateAccessory();
```

Operators

uint8_t operator Returns the current speed percentage.

```
operator uint8_t();
```

Assignment operator Sets the speed percentage.

```
void operator=(uint8_t speedPercent);
```

Example

Fan Control

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Fan Endpoint - On/Off control + Speed Percent Control + Fan Modes
MatterFan Fan;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
```

(continues on next page)

(continued from previous page)

```

#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here - used for toggling On/Off and decommission
↳the Matter Node
const uint8_t buttonPin = BOOT_PIN;  // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                 // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

// set your board Analog Pin here - used for changing the Fan speed
const uint8_t analogPin = A0;  // Analog Pin depends on each board

// set your board PWM Pin here - used for controlling the Fan speed (DC motor
↳example)
// for this example, it will use the builtin board RGB LED to simulate the Fan DC
↳motor using its brightness
#ifdef RGB_BUILTIN
const uint8_t dcMotorPin = RGB_BUILTIN;
#else
const uint8_t dcMotorPin = 2;  // Set your pin here if your board has not defined
↳LED_BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

void fanDCMotorDrive(bool fanState, uint8_t speedPercent) {
    // drive the Fan DC motor
    if (fanState == false) {
        // turn off the Fan
#ifdef RGB_BUILTIN
        // after analogWrite(), it is necessary to set the GPIO to digital mode first
        pinMode(dcMotorPin, OUTPUT);
#endif
        digitalWrite(dcMotorPin, LOW);
    } else {
        // set the Fan speed
        uint8_t fanDCMotorPWM = map(speedPercent, 0, 100, 0, 255);
#ifdef RGB_BUILTIN
        rgbLedWrite(dcMotorPin, fanDCMotorPWM, fanDCMotorPWM, fanDCMotorPWM);
#else
        analogWrite(dcMotorPin, fanDCMotorPWM);
#endif
    }
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will toggle the Fan (On/
↳Off) and decommission the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the Analog Pin A0 used to read input voltage and to set the Fan
↳speed accordingly
    pinMode(analogPin, INPUT);
    analogReadResolution(10); // 10 bits resolution reading 0..1023
    // Initialize the PWM output pin for a Fan DC motor

```

(continues on next page)

(continued from previous page)

```

pinMode(dcMotorPin, OUTPUT);

Serial.begin(115200);

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#ifdef CONFIG_ENABLE_CHIPOBLE
    // We start by connecting to a WiFi network
    Serial.print("Connecting to ");
    Serial.println(ssid);
    // Manually connect to WiFi
    WiFi.begin(ssid, password);
    // Wait for connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\r\nWiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
    delay(500);
#endif

    // On Boot or Reset, Fan is set at 0% speed, OFF, changing between OFF, ON,
↳SMART and HIGH
    Fan.begin(0, MatterFan::FAN_MODE_OFF, MatterFan::FAN_MODE_SEQ_OFF_HIGH);

    // callback functions would control Fan motor
    // the Matter Controller will send new data whenever the User APP or Automation_
↳request

    // single feature callbacks take place before the generic (all features) callback
    // This callback will be executed whenever the speed percent matter attribute is_
↳updated
    Fan.onChangeSpeedPercent([](uint8_t speedPercent) {
        // setting speed to Zero, while the Fan is ON, shall turn the Fan OFF
        if (speedPercent == MatterFan::OFF_SPEED && Fan.getMode() != MatterFan::FAN_
↳MODE_OFF) {
            // ATTR_SET do not update the attribute, just SET it to avoid infinite loop
            return Fan.setOnOff(false, Fan.ATTR_SET);
        }
        // changing the speed to higher than Zero, while the Fan is OFF, shall turn_
↳the Fan ON
        if (speedPercent > MatterFan::OFF_SPEED && Fan.getMode() == MatterFan::FAN_
↳MODE_OFF) {
            // ATTR_SET do not update the attribute, just SET it to avoid infinite loop
            return Fan.setOnOff(true, Fan.ATTR_SET);
        }
        // for other case, just return true
        return true;
    });

    // This callback will be executed whenever the fan mode matter attribute is_
↳updated
    // This will take action when user APP starts the Fan by changing the mode
    Fan.onChangeMode([](MatterFan::FanMode_t fanMode) {
        // when the Fan is turned ON using Mode Selection, while it is OFF, shall_
↳start it by setting the speed to 50%
        if (Fan.getSpeedPercent() == MatterFan::OFF_SPEED && fanMode != MatterFan::FAN_
↳MODE_OFF) {
            Serial.printf("Fan set to %s mode -- speed percentage will go to 50%\r\n",

```

(continues on next page)

(continued from previous page)

```

↪Fan.getFanModeString(fanMode));
    // ATTR_SET do not update the attribute, just SET it to avoid infinite loop
    return Fan.setSpeedPercent(50, Fan.ATTR_SET);
}
return true;
});

// Generic callback will be executed as soon as a single feature callback is done
// In this example, it will just print status messages
Fan.onChange([](MatterFan::FanMode_t fanMode, uint8_t speedPercent) {
    // just report state
    Serial.printf("Fan State: Mode %s | %u%% speed.\r\n", Fan.
↪getFanModeString(fanMode), speedPercent);
    // drive the Fan DC motor
    fanDCMotorDrive(fanMode != MatterFan::FAN_MODE_OFF, speedPercent);
    // returns success
    return true;
});

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();
// This may be a restart of a already commissioned Matter accessory
if (Matter.isDeviceCommissioned()) {
    Serial.println("Matter Node is commissioned and connected to the network.
↪Ready for use.");
}
}

void loop() {
    // Check Matter Accessory Commissioning state, which may change during execution
↪of loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code
↪or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↪str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Generic Switch Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for
↪commissioning.");
            }
        }
        Serial.println("Matter Node is commissioned and connected to the network.
↪Ready for use.");
    }

    // A builtin button is used to trigger and send a command to the Matter
↪Controller
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }
}

```

(continues on next page)

(continued from previous page)

```

// Onboard User Button is used as a smart button or to decommission it
uint32_t time_diff = millis() - button_time_stamp;
if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
    button_state = false; // released
    // button is released - toggle Fan On/Off
    Fan.toggle();
    Serial.printf("User button released. Setting the Fan %s.\r\n", Fan > 0 ? "ON" :
↳: "OFF");
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to
↳decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning Fan Matter Accessory. It shall be commissioned
↳again.");
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot
↳takes a second or so
}

// checks Analog pin and adjust the speed only if it has changed
static int lastRead = 0;
// analog values (0..1023) / 103 => mapped into 10 steps (0..9)
int anaVal = analogRead(analogPin) / 103;
if (lastRead != anaVal) {
    // speed percent moves in steps of 10. Range is 10..100
    if (Fan.setSpeedPercent((anaVal + 1) * 10)) {
        lastRead = anaVal;
    }
}
}

```

MatterGenericSwitch

About The `MatterGenericSwitch` class provides a generic switch endpoint for Matter networks. This endpoint works as a smart button that can send click events to the Matter controller, enabling automation triggers.

Features: * Click event reporting to Matter controller * Simple button functionality * Automation trigger support * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Smart buttons * Automation triggers * Remote controls * Scene activation buttons * Event generators for smart home automation

API Reference

Constructor

MatterGenericSwitch Creates a new Matter generic switch endpoint.

```
MatterGenericSwitch();
```

Initialization

begin Initializes the Matter generic switch endpoint.

```
bool begin();
```

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter switch events.

```
void end();
```

Event Generation

click Sends a click event to the Matter controller.

```
void click();
```

This function sends a click event that can be used to trigger automations in smart home apps. The event is sent immediately and the Matter controller will receive it.

Usage Example:

When a physical button is pressed and released, call this function to send the click event:

```
if (buttonReleased) {  
    mySwitch.click();  
    Serial.println("Click event sent to Matter controller");  
}
```

Example

Generic Switch (Smart Button)

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD  
//  
// Licensed under the Apache License, Version 2.0 (the "License");  
// you may not use this file except in compliance with the License.  
// You may obtain a copy of the License at  
  
//      http://www.apache.org/licenses/LICENSE-2.0  
//  
// Unless required by applicable law or agreed to in writing, software  
// distributed under the License is distributed on an "AS IS" BASIS,  
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
// See the License for the specific language governing permissions and  
// limitations under the License.  
  
// Matter Manager  
#include <Arduino.h>  
#include <Matter.h>  
#if !CONFIG_ENABLE_CHIPOBLE  
// if the device can be commissioned using BLE, WiFi is not used - save flash space  
#include <WiFi.h>  
#endif  
  
// List of Matter Endpoints for this Node  
// Generic Switch Endpoint - works as a smart button with a single click  
MatterGenericSwitch SmartButton;
```

(continues on next page)

(continued from previous page)

```

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN;  // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommisioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommision

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will act as a smart button_
    ↳or to decommision the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
    ↳Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // We start by connecting to a WiFi network
        Serial.print("Connecting to ");
        Serial.println(ssid);

        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println("\r\nWiFi connected");
        Serial.println("IP address: ");
        Serial.println(WiFi.localIP());
        delay(500);
    #endif

    // Initialize the Matter EndPoint
    SmartButton.begin();

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();
    // This may be a restart of a already commissioned Matter accessory
    if (Matter.isDeviceCommissioned()) {
        Serial.println("Matter Node is commissioned and connected to the network_
        ↳Ready for use.");
    }
}

void loop() {
    // Check Matter Accessory Commissioning state, which may change during execution_
    ↳of loop()
    if (!Matter.isDeviceCommissioned()) {

```

(continues on next page)

(continued from previous page)

```

Serial.println("");
Serial.println("Matter Node is not commissioned yet.");
Serial.println("Initiate the device discovery in your Matter environment.");
Serial.println("Commission it to your Matter hub with the manual pairing code_
↳or QR code");
Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
// waits for Matter Generic Switch Commissioning.
uint32_t timeCount = 0;
while (!Matter.isDeviceCommissioned()) {
    delay(100);
    if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
        Serial.println("Matter Node not commissioned yet. Waiting for_
↳commissioning.");
    }
}
Serial.println("Matter Node is commissioned and connected to the network._
↳Ready for use.");
}

// A builtin button is used to trigger a command to the Matter Controller
// Check if the button has been pressed
if (digitalRead(buttonPin) == LOW && !button_state) {
    // deals with button debouncing
    button_time_stamp = millis(); // record the time while the button is pressed.
    button_state = true; // pressed.
}

// Onboard User Button is used as a smart button or to decommission it
uint32_t time_diff = millis() - button_time_stamp;
if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
    button_state = false; // released
    // builtin button is released - send a click event to the Matter Controller
    Serial.println("User button released. Sending Click to the Matter Controller!
↳");
    // Matter Controller will receive an event and, if programmed, it will trigger_
↳an action
    SmartButton.click();
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to_
↳decommission matter node
if (button_state && time_diff > decommmissioningTimeout) {
    Serial.println("Decommissioning the Generic Switch Matter Accessory. It shall_
↳be commissioned again.");
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot_
↳takes a second or so
}
}

```

MatterHumiditySensor

About The `MatterHumiditySensor` class provides a humidity sensor endpoint for Matter networks. This endpoint implements the Matter humidity sensing standard for read-only humidity reporting.

Features: * Humidity measurement reporting (0-100%) * 1/100th percent precision * Read-only sensor (no control functionality) * Automatic humidity updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Room humidity monitoring * Weather stations * HVAC systems * Humidity logging * Smart home climate monitoring

API Reference

Constructor

MatterHumiditySensor Creates a new Matter humidity sensor endpoint.

```
MatterHumiditySensor();
```

Initialization

begin Initializes the Matter humidity sensor endpoint with an initial humidity value.

```
bool begin(double humidityPercent = 0.00);
```

- humidityPercent - Initial humidity percentage (0.0-100.0, default: 0.00)

This function will return `true` if successful, `false` otherwise.

Note: The implementation stores humidity with 1/100th percent precision internally. Values must be between 0.0 and 100.0.

end Stops processing Matter humidity sensor events.

```
void end();
```

Humidity Control

setHumidity Sets the reported humidity percentage.

```
bool setHumidity(double humidityPercent);
```

- humidityPercent - Humidity percentage (0.0-100.0)

This function will return `true` if successful, `false` otherwise.

Note: Humidity is stored with 1/100th percent precision. Values outside the 0.0-100.0 range will be rejected.

getHumidity Gets the current reported humidity percentage.

```
double getHumidity();
```

This function will return the humidity percentage with 1/100th percent precision.

Operators

double operator Returns the current humidity percentage.

```
operator double();
```

Example:

```
double humidity = mySensor; // Get current humidity
```

Assignment operator Sets the humidity percentage.

```
void operator=(double humidityPercent);
```

Example:

```
mySensor = 45.5; // Set humidity to 45.5%
```

Example

Humidity Sensor

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
 * ↪Controller messages.
 */
// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Humidity Sensor Endpoint
MatterHumiditySensor SimulatedHumiditySensor;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid"; // Change this to your WiFi SSID
```

(continues on next page)

(continued from previous page)

```

const char *password = "your-password"; // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here - decommissioning button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control - decommission the Matter Node
uint32_t button_time_stamp = 0; // debouncing control
bool button_state = false; // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳ or longer, to decommission

// Simulate a humidity sensor - add your preferred humidity sensor library code
↳ here
float getSimulatedHumidity() {
    // The Endpoint implementation keeps an uint16_t as internal value information,
    // which stores data in 1/100th of humidity percent
    static float simulatedHumidityHWSensor = 10.0;

    // it will increase from 10% to 30% in 0.5% steps to simulate a humidity sensor
    simulatedHumidityHWSensor = simulatedHumidityHWSensor + 0.5;
    if (simulatedHumidityHWSensor > 30) {
        simulatedHumidityHWSensor = 10;
    }

    return simulatedHumidityHWSensor;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission
    ↳ the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
    ↳ Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    #endif

    // set initial humidity sensor measurement
    // Simulated Sensor - it shall initially print 95% and then move to the 10% to 30
    ↳ % humidity range
    SimulatedHumiditySensor.begin(95.00);

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();

    // Check Matter Accessory Commissioning state, which may change during execution
    ↳ of loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
    }

```

(continues on next page)

(continued from previous page)

```

Serial.println("Initiate the device discovery in your Matter environment.");
Serial.println("Commission it to your Matter hub with the manual pairing code_
↳or QR code");
Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
// waits for Matter Humidity Sensor Commissioning.
uint32_t timeCount = 0;
while (!Matter.isDeviceCommissioned()) {
    delay(100);
    if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
        Serial.println("Matter Node not commissioned yet. Waiting for_
↳commissioning.");
    }
}
Serial.println("Matter Node is commissioned and connected to the network._
↳Ready for use.");
}
}

void loop() {
    static uint32_t timeCounter = 0;

    // Print the current humidity value every 5s
    if (!(timeCounter++ % 10)) { // delaying for 500ms x 10 = 5s
        // Print the current humidity value
        Serial.printf("Current Humidity is %.02f%\r\n", SimulatedHumiditySensor.
↳getHumidity());
        // Update Humidity from the (Simulated) Hardware Sensor
        // Matter APP shall display the updated humidity percent
        SimulatedHumiditySensor.setHumidity(getSimulatedHumidity());
    }

    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    if (digitalRead(buttonPin) == HIGH && button_state) {
        button_state = false; // released
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to_
↳decommission matter node
    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Humidity Sensor Matter Accessory. It shall be_
↳commissioned again.");
        Matter.decommission();
    }

    delay(500);
}

```

MatterLightSensor

About The `MatterLightSensor` class provides an illuminance sensor endpoint for Matter networks. This endpoint implements the Matter illuminance sensing standard for read-only illuminance reporting.

Features: * Illuminance measurement reporting (1 lx to 3.576 Mlx) * Read-only sensor (no control functionality) * Automatic illuminance updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Room illuminance monitoring * Weather stations * Illuminance logging * Smart home climate monitoring

API Reference

Constructor

MatterLightSensor Creates a new Matter light sensor endpoint.

```
MatterLightSensor();
```

Initialization

begin Initializes the Matter light sensor endpoint with an initial illuminance value.

```
bool begin(double illuminance = 1.00)
```

- `illuminance` - Initial illuminance value (1 lx to 3.576 Mlx, default: 1.00)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter light sensor events.

```
void end();
```

Illuminance Control

setIlluminance Sets the reported illuminance value.

```
bool setIlluminance(double illuminance);
```

- `illuminance` - Illuminance value to report (1 lx to 3.576 Mlx)

This function will return `true` if successful, `false` otherwise.

getIlluminance Gets the current reported illuminance value.

```
double getIlluminance();
```

This function will return the illuminance value.

Operators

double operator Returns the current illuminance value.

```
operator double();
```

Example:

```
double illuminance = mySensor; // Get current illuminance
```

Assignment operator Sets the illuminance value.

```
void operator=(double illuminance);
```

Example:

```
mySensor = 45.5; // Set illuminance to 45.5lx
```

Example

Light Sensor

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
 * ↪Controller messages.
 */
// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Light Sensor Endpoint
MatterLightSensor SimulatedLightSensor;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid"; // Change this to your WiFi SSID
```

(continues on next page)

(continued from previous page)

```

const char *password = "your-password"; // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here - decommissioning button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control - decommission the Matter Node
uint32_t button_time_stamp = 0; // debouncing control
bool button_state = false; // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳ or longer, to decommission

// Simulate an illuminance sensor - add your preferred illuminance sensor library.
↳ code here
// Returns the illuminance in lux; the endpoint converts it to the Matter-style.
↳ logarithmic value internally.
float getSimulatedIlluminance() {
    static float simulatedIlluminanceHWSensor = 100.0;

    // it will increase from 100lx to 300lx in 10lx steps to simulate an illuminance.
    ↳ sensor
    simulatedIlluminanceHWSensor = simulatedIlluminanceHWSensor + 10.0;
    if (simulatedIlluminanceHWSensor > 300) {
        simulatedIlluminanceHWSensor = 100;
    }

    return simulatedIlluminanceHWSensor;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission.
    ↳ the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter.
    ↳ Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    #endif

    // set initial illuminance sensor measurement
    // Simulated Sensor - it shall initially print 2150lx and then move to the 100lx.
    ↳ to 300lx illuminance range
    SimulatedLightSensor.begin(2150.0);

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();

    // Check Matter Accessory Commissioning state, which may change during execution.
    ↳ of loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
    }

```

(continues on next page)

(continued from previous page)

```

    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code_
↳or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Illuminance Sensor Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for_
↳commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network._
↳Ready for use.");
}

void loop() {
    static uint32_t timeCounter = 0;

    // Print the current illuminance value every 5s
    if (!(timeCounter++ % 10)) { // delaying for 500ms x 10 = 5s
        // Print the current illuminance value
        Serial.printf("Current Illuminance is %.02flx\r\n", SimulatedLightSensor.
↳getIlluminance());
        // Update Illuminance from the (Simulated) Hardware Sensor
        // Matter APP shall display the updated illuminance value
        SimulatedLightSensor.setIlluminance(getSimulatedIlluminance());
    }

    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    if (digitalRead(buttonPin) == HIGH && button_state) {
        button_state = false; // released
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to_
↳decommission matter node
    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Light Sensor Matter Accessory. It shall be_
↳commissioned again.");
        Matter.decommission();
    }

    delay(500);
}

```

MatterOccupancySensor

About The `MatterOccupancySensor` class provides an occupancy sensor endpoint for Matter networks. This endpoint implements the Matter occupancy sensing standard for detecting occupied/unoccupied states (e.g., motion sensors, PIR sensors).

Features: * Occupancy state reporting (occupied/unoccupied) * Multiple sensor type support (PIR, Ultrasonic, Physical Contact) * HoldTime attribute for configuring how long the sensor holds the “occupied” state * HoldTimeLimits (min, max, default) for validation and controller guidance * HoldTime change callback for real-time updates from Matter controllers * Simple boolean state * Read-only sensor (no control functionality) * Automatic state updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Motion sensors (PIR) * Occupancy detection * Security systems * Smart lighting automation * Energy management (turn off lights when unoccupied)

API Reference

Constructor

MatterOccupancySensor Creates a new Matter occupancy sensor endpoint.

```
MatterOccupancySensor();
```

Initialization

begin Initializes the Matter occupancy sensor endpoint with an initial occupancy state and sensor type.

```
bool begin(bool _occupancyState = false, OccupancySensorType_t _  
↳occupancySensorType = OCCUPANCY_SENSOR_TYPE_PIR);
```

- `_occupancyState` - Initial occupancy state (true = occupied, false = unoccupied, default: false)
- `_occupancySensorType` - Sensor type (default: `OCCUPANCY_SENSOR_TYPE_PIR`)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter occupancy sensor events.

```
void end();
```

Sensor Types

OccupancySensorType_t Occupancy sensor type enumeration:

- `OCCUPANCY_SENSOR_TYPE_PIR` - Passive Infrared (PIR) sensor
- `OCCUPANCY_SENSOR_TYPE_ULTRASONIC` - Ultrasonic sensor
- `OCCUPANCY_SENSOR_TYPE_PIR_AND_ULTRASONIC` - Combined PIR and Ultrasonic
- `OCCUPANCY_SENSOR_TYPE_PHYSICAL_CONTACT` - Physical contact sensor

Occupancy State Control

setOccupancy Sets the occupancy state.

```
bool setOccupancy(bool _occupancyState);
```

- `_occupancyState` - Occupancy state (true = occupied, false = unoccupied)

This function will return `true` if successful, `false` otherwise.

getOccupancy Gets the current occupancy state.

```
bool getOccupancy();
```

This function will return `true` if occupied, `false` if unoccupied.

HoldTime Control

setHoldTime Sets the HoldTime value (in seconds). The HoldTime determines how long the sensor maintains the “occupied” state after the last detection.

```
bool setHoldTime(uint16_t _holdTime_seconds);
```

- `_holdTime_seconds` - HoldTime value in seconds

Important: This function must be called after `Matter.begin()` has been called, as it requires the Matter event loop to be running.

This function will return `true` if successful, `false` otherwise.

getHoldTime Gets the current HoldTime value (in seconds).

```
uint16_t getHoldTime();
```

This function will return the current HoldTime value in seconds.

setHoldTimeLimits Sets the HoldTime limits (minimum, maximum, and default values). These limits define the valid range for HoldTime values and provide metadata for Matter controllers.

```
bool setHoldTimeLimits(uint16_t _holdTimeMin_seconds, uint16_t _holdTimeMax_
↪seconds, uint16_t _holdTimeDefault_seconds);
```

- `_holdTimeMin_seconds` - Minimum HoldTime value in seconds
- `_holdTimeMax_seconds` - Maximum HoldTime value in seconds
- `_holdTimeDefault_seconds` - Default/recommended HoldTime value in seconds (informational metadata for controllers)

Important: * This function must be called after `Matter.begin()` has been called, as it requires the Matter event loop to be running. * The `holdTimeDefault_seconds` parameter is informational metadata for Matter controllers (recommended default value). It does NOT automatically set the HoldTime attribute - use `setHoldTime()` to set the actual value. * If the current HoldTime value is outside the new limits, it will be automatically adjusted to the nearest limit (minimum or maximum).

This function will return `true` if successful, `false` otherwise.

onHoldTimeChange Sets a callback function that will be called when the HoldTime value is changed by a Matter Controller.

```
void onHoldTimeChange(HoldTimeChangeCB onHoldTimeChangeCB);
```

- `onHoldTimeChangeCB` - Callback function of type `HoldTimeChangeCB`

The callback function signature is:

```
using HoldTimeChangeCB = std::function<bool(uint16_t holdTime_seconds)>;
```

The callback receives the new HoldTime value and can return `true` to accept the change or `false` to reject it.

Example:

```
OccupancySensor.onHoldTimeChange([] (uint16_t holdTime_seconds) -> bool {
    Serial.printf("HoldTime changed to %u seconds\n", holdTime_seconds);
    return true; // Accept the change
});
```

Operators

bool operator Returns the current occupancy state.

```
operator bool();
```

Example:

```
if (mySensor) {
    Serial.println("Room is occupied");
} else {
    Serial.println("Room is unoccupied");
}
```

Assignment operator Sets the occupancy state.

```
void operator=(bool _occupancyState);
```

Example:

```
mySensor = true; // Set to occupied
mySensor = false; // Set to unoccupied
```

Example

Basic Occupancy Sensor

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
 * ↪Controller messages.
 *
 * The example will create a Matter Occupancy Sensor Device.
 * The Occupancy Sensor will be simulated to change its state every 2 minutes.
 */
```

(continues on next page)

(continued from previous page)

```

* The onboard button can be kept pressed for 5 seconds to decommission the Matter_
↳Node.
* The example will also show the manual commissioning code and QR code to be used_
↳in the Matter environment.
*
*/

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Occupancy Sensor Endpoint
MatterOccupancySensor OccupancySensor;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here - decommissioning only
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission_
↳the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    #endif

    // set initial occupancy sensor state as false and connected to a PIR sensor_
↳type (default)
    OccupancySensor.begin();

    // Matter beginning - Last step, after all EndPoints are initialized

```

(continues on next page)

(continued from previous page)

```

Matter.begin();

// Check Matter Accessory Commissioning state, which may change during execution
of loop()
if (!Matter.isDeviceCommissioned()) {
    Serial.println("");
    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code
or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Occupancy Sensor Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for
commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network.
Ready for use.");
}
}

bool simulatedHWOccupancySensor() {
    // Simulated Occupancy Sensor
    static bool occupancyState = false;
    static uint32_t lastTime = millis();
    const uint32_t occupancyTimeout = 120000; // 2 minutes to toggle the state

    // Simulate a Occupancy Sensor state change every 2 minutes
    if (millis() - lastTime > occupancyTimeout) {
        occupancyState = !occupancyState;
        lastTime = millis();
    }
    return occupancyState;
}

void loop() {
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    if (button_state && digitalRead(buttonPin) == HIGH) {
        button_state = false; // released
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to
decommission matter node
    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Occupancy Sensor Matter Accessory. It shall be
commissioned again.");
        Matter.decommission();
        button_time_stamp = millis(); // avoid running decommissioning again, reboot

```

(continues on next page)

(continued from previous page)

```

    ↳takes a second or so
  }

  // Check Simulated Occupancy Sensor and set Matter Attribute
  OccupancySensor.setOccupancy(simulatedHWOccupancySensor());

  delay(50);
}

```

Occupancy Sensor with HoldTime For an example that demonstrates HoldTime functionality, see:

```

// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
↳Controller messages.
 *
 * The example will create a Matter Occupancy Sensor Device.
 * The Occupancy Sensor will be simulated to toggle occupancy every 2 minutes.
 * When occupancy is detected, the sensor holds the "occupied" state for the
↳HoldTime duration
 * (default: 30 seconds, configurable via Matter Controller). After HoldTime
↳expires,
 * the sensor automatically switches to "unoccupied" state.
 *
 * The HoldTime attribute allows you to adjust how long the active status is
↳retained
 * after the person leaves. The HoldTime can be changed by a Matter Controller,
↳and the
 * onHoldTimeChange() callback is used to update the simulated sensor
↳functionality.
 *
 * The HoldTime value is persisted to Preferences (NVS) and restored on reboot, so
↳the
 * last configured HoldTime value is maintained across device restarts.
 *
 * The onboard button can be kept pressed for 5 seconds to decommission the Matter
↳Node.
 * The example will also show the manual commissioning code and QR code to be used
↳in the Matter environment.
 *
 */

// Matter Manager
#include <Arduino.h>

```

(continues on next page)

(continued from previous page)

```

#include <Matter.h>
// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif
#include <Preferences.h>

// HoldTime configuration constants
const uint16_t HOLD_TIME_MIN = 0;         // Minimum HoldTime in seconds
const uint16_t HOLD_TIME_MAX = 3600;      // Maximum HoldTime in seconds (1 hour)
const uint16_t HOLD_TIME_DEFAULT = 30;    // Default HoldTime in seconds

// List of Matter Endpoints for this Node
// Matter Occupancy Sensor Endpoint
MatterOccupancySensor OccupancySensor;

// Preferences to store HoldTime value across reboots
Preferences matterPref;
const char *holdTimePrefKey = "HoldTime";

// set your board USER BUTTON pin here - decommissioning only
const uint8_t buttonPin = BOOT_PIN;       // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

// Simulated Occupancy Sensor with HoldTime support
// When occupancy is detected, it holds the "occupied" state for HoldTime seconds
// After HoldTime expires, it automatically switches to "unoccupied"
//
// Behavior for different HoldTime vs detectionInterval relationships:
// - holdTime_ms < detectionInterval: State switches to unoccupied after HoldTime,
↳then waits for next detection
// - holdTime_ms == detectionInterval: If detections keep coming, timer resets_
↳(continuous occupancy)
// - holdTime_ms > detectionInterval: If detections keep coming, timer resets_
↳(continuous occupancy)
// If detections stop, HoldTime expires after the last detection
bool simulatedHWOccupancySensor() {
    static bool occupancyState = false;
    static uint32_t lastDetectionTime = 0;
    static uint32_t lastDetectionEvent = millis();
    const uint32_t detectionInterval = 120000; // Simulate detection every 2 minutes

    // Get current HoldTime from the sensor (can be changed by Matter Controller)
    uint32_t holdTime_ms = OccupancySensor.getHoldTime() * 1000; // Convert seconds_
↳to milliseconds

    // Check HoldTime expiration FIRST (before processing new detections)
    // This ensures HoldTime can expire even if a detection occurs in the same_
↳iteration
    if (occupancyState && (millis() - lastDetectionTime > holdTime_ms)) {
        occupancyState = false;
    }

```

(continues on next page)

(continued from previous page)

```

    // Reset detection interval counter so next detection can happen immediately
    // This makes the simulation more responsive after the room becomes unoccupied
    lastDetectionEvent = millis();
    Serial.println("HoldTime expired. Switching to unoccupied state.");
}

// Simulate periodic occupancy detection (e.g., motion detected)
// Check this AFTER HoldTime expiration so new detections can immediately re-
→ trigger occupancy
if (millis() - lastDetectionEvent > detectionInterval) {
    // New detection event occurred
    lastDetectionEvent = millis();

    if (!occupancyState) {
        // Transition from unoccupied to occupied - start hold timer
        occupancyState = true;
        lastDetectionTime = millis();
        Serial.printf("Occupancy detected! Holding state for %u seconds (HoldTime)\n
→ ", OccupancySensor.getHoldTime());
    } else {
        // Already occupied - new detection extends the hold period by resetting the
→ timer
        // This simulates continuous occupancy (person still present)
        lastDetectionTime = millis();
        Serial.printf("Occupancy still detected. Resetting hold timer to %u seconds
→ (HoldTime)\n", OccupancySensor.getHoldTime());
    }
}

return occupancyState;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommisio
→ n the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
→ Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    #endif

    // Initialize Preferences and read stored HoldTime value
    matterPref.begin("MatterPrefs", false);
    uint16_t storedHoldTime = matterPref.getUShort(holdTimePrefKey, HOLD_TIME_
→ DEFAULT);

    // Validate stored value is within limits
    if (storedHoldTime < HOLD_TIME_MIN || storedHoldTime > HOLD_TIME_MAX) {
        uint16_t invalidValue = storedHoldTime;
        storedHoldTime = HOLD_TIME_DEFAULT;
    }
}

```

(continues on next page)

(continued from previous page)

```

    Serial.printf("Invalid stored HoldTime (%u), using default: %u seconds\n",
↳invalidValue, HOLD_TIME_DEFAULT);
    } else if (storedHoldTime != HOLD_TIME_DEFAULT) {
        Serial.printf("Restored HoldTime from Preferences: %u seconds\n",
↳storedHoldTime);
    }

    // Register callback for HoldTime changes from Matter Controller
    OccupancySensor.onHoldTimeChange([](uint16_t holdTime_seconds) -> bool {
        Serial.printf("HoldTime changed to %u seconds by Matter Controller\n",
↳holdTime_seconds);
        // Store the new HoldTime value to Preferences for persistence across reboots
        matterPref.putUShort(holdTimePrefKey, holdTime_seconds);
        // The callback can return false to reject the change, or true to accept it
        // In this case, we always accept the change and update the simulator
        return true;
    });

    // set initial occupancy sensor state as false and connected to a PIR sensor
↳type (default)
    OccupancySensor.begin();

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();

    // Set HoldTimeLimits after Matter.begin() (optional, but recommended for
↳validation)
    if (!OccupancySensor.setHoldTimeLimits(HOLD_TIME_MIN, HOLD_TIME_MAX, HOLD_TIME_
↳DEFAULT)) {
        Serial.println("Warning: Failed to set HoldTimeLimits");
    } else {
        Serial.printf("HoldTimeLimits set: Min=%u, Max=%u, Default=%u seconds\n", HOLD_
↳TIME_MIN, HOLD_TIME_MAX, HOLD_TIME_DEFAULT);
    }

    // Set initial HoldTime (use stored value if valid, otherwise use default)
    // This must be done after Matter.begin() because setHoldTime() requires the
↳Matter event loop
    if (!OccupancySensor.setHoldTime(storedHoldTime)) {
        Serial.printf("Warning: Failed to set HoldTime to %u seconds\n",
↳storedHoldTime);
    } else {
        Serial.printf("HoldTime set to: %u seconds\n", storedHoldTime);
    }

    Serial.printf("Initial HoldTime: %u seconds\n", OccupancySensor.getHoldTime());

    // Check Matter Accessory Commissioning state, which may change during execution
↳of loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code
↳or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Occupancy Sensor Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {

```

(continues on next page)

(continued from previous page)

```

        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for_
↪commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network._
↪Ready for use.");
}
}

void loop() {
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }

    if (button_state && digitalRead(buttonPin) == HIGH) {
        button_state = false; // released
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to_
↪decommission matter node
    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Occupancy Sensor Matter Accessory. It shall be_
↪commissioned again.");
        Matter.decommission();
        button_time_stamp = millis(); // avoid running decommissioning again, reboot_
↪takes a second or so
    }

    // Check Simulated Occupancy Sensor and set Matter Attribute
    OccupancySensor.setOccupancy(simulatedHWOccupancySensor());

    delay(50);
}

```

This example shows: * How to configure HoldTimeLimits after Matter.begin() * How to set and persist HoldTime values * How to use the onHoldTimeChange() callback * How to implement HoldTime expiration logic in sensor simulation

MatterOnOffLight

About The MatterOnOffLight class provides a simple on/off light endpoint for Matter networks. This endpoint implements the Matter lighting standard for basic light control without dimming or color features.

Features: * Simple on/off control * State persistence support * Callback support for state changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Simple smart lights * On/off switches * Basic lighting control * Smart home automation

API Reference

Constructor

MatterOnOffLight Creates a new Matter on/off light endpoint.

```
MatterOnOffLight();
```

Initialization

begin Initializes the Matter on/off light endpoint with an optional initial state.

```
bool begin(bool initialState = false);
```

- `initialState` - Initial on/off state (`true` = on, `false` = off, default: `false`)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter light events. This will just stop processing Light Matter events but won't remove the endpoint.

```
void end();
```

On/Off Control

setOnOff Sets the on/off state of the light.

```
bool setOnOff(bool newState);
```

- `newState` - New state (`true` = on, `false` = off)

This function will return `true` if successful, `false` otherwise.

getOnOff Gets the current on/off state of the light.

```
bool getOnOff();
```

This function will return `true` if the light is on, `false` if off.

toggle Toggles the on/off state of the light.

```
bool toggle();
```

This function will return `true` if successful, `false` otherwise.

Operators

bool operator Returns the current on/off state of the light.

```
operator bool();
```

Example:

```
if (myLight) {  
    Serial.println("Light is on");  
}
```


Assignment operator Turns the light on or off.

```
void operator=(bool state);
```

Example:

```
myLight = true;    // Turn light on
myLight = false;   // Turn light off
```

Event Handling

onChange Sets a callback function to be called when the light state changes.

```
void onChange(EndPointCB onChangeCB);
```

- onChangeCB - Function to call when state changes

The callback signature is:

```
bool onChangeCallback(bool newState);
```

- newState - New on/off state (true = on, false = off)

The callback should return true if the change was handled successfully.

onChangeOnOff Sets a callback function to be called when the on/off state changes (same as onChange).

```
void onChangeOnOff(EndPointCB onChangeCB);
```

- onChangeCB - Function to call when state changes

updateAccessory Updates the state of the light using the current Matter Light internal state. It is necessary to set a user callback function using onChange () to handle the physical light state.

```
void updateAccessory();
```

This function will call the registered callback with the current state.

Example

Basic On/Off Light

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
//
// Matter Manager
```

(continues on next page)

(continued from previous page)

```

#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif
#include <Preferences.h>

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// List of Matter Endpoints for this Node
// On/Off Light Endpoint
MatterOnOffLight OnOffLight;

// it will keep last OnOff state stored, using Preferences
Preferences matterPref;
const char *onOffPrefKey = "OnOff";

// set your board LED pin here
#ifdef LED_BUILTIN
const uint8_t ledPin = LED_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↪BUILTIN
#warning "Do not forget to set the LED pin"
#endif

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                 // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

// Matter Protocol Endpoint Callback
bool setLightOnOff(bool state) {
    Serial.printf("User Callback :: New Light State = %s\r\n", state ? "ON" : "OFF");
    if (state) {
        digitalWrite(ledPin, HIGH);
    } else {
        digitalWrite(ledPin, LOW);
    }
    // store last OnOff state for when the Light is restarted / power goes off
    matterPref.putBool(onOffPrefKey, state);
    // This callback must return the success state to Matter core
    return true;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO that will act as a toggle switch
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the LED (light) GPIO and Matter End Point

```

(continues on next page)

(continued from previous page)

```

pinMode(ledPin, OUTPUT);

Serial.begin(115200);

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
#ifdef CONFIG_ENABLE_CHIPOBLE
    // We start by connecting to a WiFi network
    Serial.print("Connecting to ");
    Serial.println(ssid);
    // Manually connect to WiFi
    WiFi.begin(ssid, password);
    // Wait for connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\r\nWiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
    delay(500);
#endif

    // Initialize Matter EndPoint
    matterPref.begin("MatterPrefs", false);
    bool lastOnOffState = matterPref.getBool(onOffPrefKey, true);
    OnOffLight.begin(lastOnOffState);
    OnOffLight.onChange(setLightOnOff);

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();
    // This may be a restart of a already commissioned Matter accessory
    if (Matter.isDeviceCommissioned()) {
        Serial.println("Matter Node is commissioned and connected to the network_
↪Ready for use.");
        Serial.printf("Initial state: %s\r\n", OnOffLight.getOnOff() ? "ON" : "OFF");
        OnOffLight.updateAccessory(); // configure the Light based on initial state
    }
}

void loop() {
    // Check Matter Light Commissioning state, which may change during execution of_
↪loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code_
↪or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↪str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Light Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for_
↪commissioning.");
            }
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

Serial.printf("Initial state: %s\r\n", OnOffLight.getOnOff() ? "ON" : "OFF");
OnOffLight.updateAccessory(); // configure the Light based on initial state
Serial.println("Matter Node is commissioned and connected to the network.␣
↪Ready for use.");
}

// A button is also used to control the light
// Check if the button has been pressed
if (digitalRead(buttonPin) == LOW && !button_state) {
    // deals with button debouncing
    button_time_stamp = millis(); // record the time while the button is pressed.
    button_state = true;          // pressed.
}

// Onboard User Button is used as a Light toggle switch or to decommission it
uint32_t time_diff = millis() - button_time_stamp;
if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
    button_state = false; // released
    // Toggle button is released - toggle the light
    Serial.println("User button released. Toggling Light!");
    OnOffLight.toggle(); // Matter Controller also can see the change
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to␣
↪decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning the Light Matter Accessory. It shall be␣
↪commissioned again.");
    OnOffLight.setOnOff(false); // turn the light off
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot␣
↪takes a second or so
}
}

```

MatterOnOffPlugin

About The MatterOnOffPlugin class provides an on/off plugin unit endpoint for Matter networks. This endpoint implements the Matter on/off plugin standard for controlling power outlets, relays, and other on/off devices.

Features: * Simple on/off control * State persistence support * Callback support for state changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Smart power outlets * Relay control * Power switches * Smart plugs * On/off device control

API Reference

Constructor

MatterOnOffPlugin Creates a new Matter on/off plugin endpoint.

```
MatterOnOffPlugin();
```

Initialization

begin Initializes the Matter on/off plugin endpoint with an optional initial state.

```
bool begin(bool initialState = false);
```

- `initialState` - Initial on/off state (`true` = on, `false` = off, default: `false`)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter plugin events.

```
void end();
```

On/Off Control

setOnOff Sets the on/off state of the plugin.

```
bool setOnOff(bool newState);
```

- `newState` - New state (`true` = on, `false` = off)

This function will return `true` if successful, `false` otherwise.

getOnOff Gets the current on/off state of the plugin.

```
bool getOnOff();
```

This function will return `true` if the plugin is on, `false` if off.

toggle Toggles the on/off state of the plugin.

```
bool toggle();
```

This function will return `true` if successful, `false` otherwise.

Operators

bool operator Returns the current on/off state of the plugin.

```
operator bool();
```

Example:

```
if (myPlugin) {  
    Serial.println("Plugin is on");  
}
```

Assignment operator Turns the plugin on or off.

```
void operator=(bool state);
```

Example:

```
myPlugin = true;    // Turn plugin on  
myPlugin = false;   // Turn plugin off
```

Event Handling

onChange Sets a callback function to be called when the plugin state changes.

```
void onChange(EndPointCB onChangeCB);
```

- onChangeCB - Function to call when state changes

The callback signature is:

```
bool onChangeCallback(bool newState);
```

- newState - New on/off state (true = on, false = off)

onChangeOnOff Sets a callback function to be called when the on/off state changes (same as onChange).

```
void onChangeOnOff(EndPointCB onChangeCB);
```

updateAccessory Updates the state of the plugin using the current Matter Plugin internal state.

```
void updateAccessory();
```

This function will call the registered callback with the current state.

Example

On/Off Plugin

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif
#include <Preferences.h>

// List of Matter Endpoints for this Node
// On/Off Plugin Endpoint
MatterOnOffPlugin OnOffPlugin;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
```

(continues on next page)

(continued from previous page)

```

// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// it will keep last OnOff state stored, using Preferences
Preferences matterPref;
const char *onOffPrefKey = "OnOff";

// set your board Power Relay pin here - this example uses the built-in LED for
↳easy visualization
#ifdef LED_BUILTIN
const uint8_t onoffPin = LED_BUILTIN;
#else
const uint8_t onoffPin = 2; // Set your pin here - usually a power relay
#warning "Do not forget to set the Power Relay pin"
#endif

// board USER BUTTON pin necessary for Decommissioning
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                 // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

// Matter Protocol Endpoint Callback
bool setPluginOnOff(bool state) {
    Serial.printf("User Callback :: New Plugin State = %s\r\n", state ? "ON" : "OFF
↳");
    if (state) {
        digitalWrite(onoffPin, HIGH);
    } else {
        digitalWrite(onoffPin, LOW);
    }
    // store last OnOff state for when the Plugin is restarted / power goes off
    matterPref.putBool(onOffPrefKey, state);
    // This callback must return the success state to Matter core
    return true;
}

void setup() {
    // Initialize the USER BUTTON
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the Power Relay (plugin) GPIO
    pinMode(onoffPin, OUTPUT);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
↳Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // We start by connecting to a WiFi network
        Serial.print("Connecting to ");
        Serial.println(ssid);
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");

```

(continues on next page)

(continued from previous page)

```

}
Serial.println("\r\nWiFi connected");
Serial.println("IP address: ");
Serial.println(WiFi.localIP());
delay(500);
#endif

// Initialize Matter EndPoint
matterPref.begin("MatterPrefs", false);
bool lastOnOffState = matterPref.getBool(onOffPrefKey, false);
OnOffPlugin.begin(lastOnOffState);
OnOffPlugin.onChange(setPluginOnOff);

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();
// This may be a restart of a already commissioned Matter accessory
if (Matter.isDeviceCommissioned()) {
    Serial.println("Matter Node is commissioned and connected to the network.
↳ Ready for use.");
    Serial.printf("Initial state: %s\r\n", OnOffPlugin.getOnOff() ? "ON" : "OFF");
    OnOffPlugin.updateAccessory(); // configure the Plugin based on initial state
}
}

void loop() {
    // Check Matter Plugin Commissioning state, which may change during execution of
    ↳ loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code.
↳ or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳ str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Plugin Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for
↳ commissioning.");
            }
        }
        Serial.printf("Initial state: %s\r\n", OnOffPlugin.getOnOff() ? "ON" : "OFF");
        OnOffPlugin.updateAccessory(); // configure the Plugin based on initial state
        Serial.println("Matter Node is commissioned and connected to the network.
↳ Ready for use.");
    }

    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    // Onboard User Button is used to decommission the Matter Node
    if (button_state && digitalRead(buttonPin) == HIGH) {
        button_state = false; // released
    }
}

```

(continues on next page)

(continued from previous page)

```

    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to
    ↳ decommission matter node
    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning the Plugin Matter Accessory. It shall be
    ↳ commissioned again.");
        OnOffPlugin.setOnOff(false); // turn the plugin off
        Matter.decommission();
        button_time_stamp = millis(); // avoid running decommissioning again, reboot
    ↳ takes a second or so
    }
}

```

MatterPressureSensor

About The `MatterPressureSensor` class provides a pressure sensor endpoint for Matter networks. This endpoint implements the Matter pressure sensing standard for read-only pressure reporting.

Features: * Pressure measurement reporting in hectopascals (hPa) * Read-only sensor (no control functionality) * Automatic pressure updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Atmospheric pressure monitoring * Weather stations * Barometric pressure sensors * Pressure logging * Smart home weather monitoring

API Reference

Constructor

MatterPressureSensor Creates a new Matter pressure sensor endpoint.

```
MatterPressureSensor();
```

Initialization

begin Initializes the Matter pressure sensor endpoint with an initial pressure value.

```
bool begin(double pressure = 0.00);
```

- `pressure` - Initial pressure in hectopascals (hPa, default: 0.00)

This function will return `true` if successful, `false` otherwise.

Note: Typical atmospheric pressure at sea level is around 1013.25 hPa. Pressure values typically range from 950-1050 hPa.

end Stops processing Matter pressure sensor events.

```
void end();
```

Pressure Control

setPressure Sets the reported pressure value.

```
bool setPressure(double pressure);
```

- pressure - Pressure in hectopascals (hPa)

This function will return `true` if successful, `false` otherwise.

getPressure Gets the current reported pressure value.

```
double getPressure();
```

This function will return the pressure in hectopascals (hPa).

Operators

double operator Returns the current pressure.

```
operator double();
```

Example:

```
double pressure = mySensor; // Get current pressure
```

Assignment operator Sets the pressure value.

```
void operator=(double pressure);
```

Example:

```
mySensor = 1013.25; // Set pressure to 1013.25 hPa
```

Example

Pressure Sensor

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
 * ↪ Controller messages.
 */
```

(continues on next page)

(continued from previous page)

```

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Pressure Sensor Endpoint
MatterPressureSensor SimulatedPressureSensor;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here - decommissioning button
const uint8_t buttonPin = BOOT_PIN;       // Set your pin here. Using BOOT Button.

// Button control - decommission the Matter Node
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

// Simulate a pressure sensor - add your preferred pressure sensor library code_
↳here
float getSimulatedPressure() {
    // The Endpoint implementation keeps an uint16_t as internal value information,
    // which stores data in hPa (pressure measurement unit)
    static float simulatedPressureHWSensor = 950;

    // it will increase from 950 to 1100 hPa in steps of 10 hPa to simulate a_
↳pressure sensor
    simulatedPressureHWSensor = simulatedPressureHWSensor + 10;
    if (simulatedPressureHWSensor > 1100) {
        simulatedPressureHWSensor = 950;
    }

    return simulatedPressureHWSensor;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission_
↳the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {

```

(continues on next page)

(continued from previous page)

```

    delay(500);
    Serial.print(".");
}
Serial.println();
#endif

// set initial pressure sensor measurement
// Simulated Sensor - it shall initially print 900hPa and then move to the 950_
↳to 1100 hPa as pressure range
SimulatedPressureSensor.begin(900.00);

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();

// Check Matter Accessory Commissioning state, which may change during execution_
↳of loop()
if (!Matter.isDeviceCommissioned()) {
    Serial.println("");
    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code_
↳or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Pressure Sensor Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for_
↳commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network._
↳Ready for use.");
}
}

void loop() {
    static uint32_t timeCounter = 0;

    // Print the current pressure value every 5s
    if (!(timeCounter++ % 10)) { // delaying for 500ms x 10 = 5s
        // Print the current pressure value
        Serial.printf("Current Pressure is %.02fhPa\r\n", SimulatedPressureSensor.
↳getPressure());
        // Update Pressure from the (Simulated) Hardware Sensor
        // Matter APP shall display the updated pressure percent
        SimulatedPressureSensor.setPressure(getSimulatedPressure());
    }

    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    if (digitalRead(buttonPin) == HIGH && button_state) {
        button_state = false; // released
    }
}

```

(continues on next page)

(continued from previous page)

```

    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to
    ↳ decommission matter node
    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > decommissioningTimeout) {
        // Factory reset is triggered if the button is pressed longer than 5 seconds
        Serial.println("Decommissioning Pressure Sensor Matter Accessory. It shall be
    ↳ commissioned again.");
        Matter.decommission();
    }

    delay(500);
}

```

MatterRainSensor

About The `MatterRainSensor` class provides a rain sensor endpoint for Matter networks. This endpoint implements the Matter rain sensing standard for detecting rain presence (detected/not detected states).

Features: * Rain detection state reporting (detected/not detected) * Simple boolean state * Read-only sensor (no control functionality) * Automatic state updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Weather monitoring systems * Irrigation control systems * Outdoor sensor networks * Smart home automation triggers * Rain detection for automated systems

API Reference

Constructor

MatterRainSensor Creates a new Matter rain sensor endpoint.

```
MatterRainSensor();
```

Initialization

begin Initializes the Matter rain sensor endpoint with an initial rain detection state.

```
bool begin(bool _rainState = false);
```

- `_rainState` - Initial rain detection state (`true` = detected, `false` = not detected, default: `false`)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter rain sensor events.

```
void end();
```

Rain Detection State Control

setRain Sets the rain detection state.

```
bool setRain(bool _rainState);
```

- `_rainState` - Rain detection state (`true` = detected, `false` = not detected)

This function will return `true` if successful, `false` otherwise.

getRain Gets the current rain detection state.

```
bool getRain();
```

This function will return `true` if rain is detected, `false` if not detected.

Operators

bool operator Returns the current rain detection state.

```
operator bool();
```

Example:

```
if (mySensor) {  
    Serial.println("Rain is detected");  
} else {  
    Serial.println("Rain is not detected");  
}
```

Assignment operator Sets the rain detection state.

```
void operator=(bool _rainState);
```

Example:

```
mySensor = true;    // Set rain detection to detected  
mySensor = false;   // Set rain detection to not detected
```

Example

Rain Sensor

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD  
//  
// Licensed under the Apache License, Version 2.0 (the "License");  
// you may not use this file except in compliance with the License.  
// You may obtain a copy of the License at  
  
//     http://www.apache.org/licenses/LICENSE-2.0  
//  
// Unless required by applicable law or agreed to in writing, software  
// distributed under the License is distributed on an "AS IS" BASIS,  
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.  
// See the License for the specific language governing permissions and  
// limitations under the License.  
  
/*  
 * This example is an example code that will create a Matter Device which can be
```

(continues on next page)

(continued from previous page)

```

* commissioned and controlled from a Matter Environment APP.
* Additionally the ESP32 will send debug messages indicating the Matter activity.
* Turning DEBUG Level ON may be useful to following Matter Accessory and
↳Controller messages.
*
* The example will create a Matter Rain Sensor Device.
* The Rain Sensor state can be toggled by pressing the onboard button.
* The Rain Sensor state will be indicated by the onboard LED.
* The Rain Sensor state will be simulated to change every 20 seconds.
*
* The onboard button can be kept pressed for 5 seconds to decommission the Matter.
↳Node.
* The example will also show the manual commissioning code and QR code to be used
↳in the Matter environment.
*
*/

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Rain Sensor Endpoint
MatterRainSensor RainSensor;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter.
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// LED will be used to indicate the Rain Sensor state
// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↳BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here - decommissioning and Manual Rain Sensor.
↳toggle button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t debounceTime = 250;       // button debouncing time (ms)
const uint32_t decommisioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission.
↳the Matter Node

```

(continues on next page)

(continued from previous page)

```

// The button will also be used to manually toggle the Rain Sensor state
pinMode(buttonPin, INPUT_PULLUP);
// Initialize the LED (light) GPIO and Matter End Point
pinMode(ledPin, OUTPUT);

Serial.begin(115200);

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// Manually connect to WiFi
WiFi.begin(ssid, password);
// Wait for connection
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println();
#endif

// set initial rain sensor state as false (default)
RainSensor.begin();
digitalWrite(ledPin, LOW); // LED OFF

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();

// Check Matter Accessory Commissioning state, which may change during execution_
↳of loop()
if (!Matter.isDeviceCommissioned()) {
    Serial.println("");
    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code_
↳or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Rain Sensor Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for_
↳commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network._
↳Ready for use.");
}
}

bool simulatedHWRainSensor() {
    // Simulated Rain Sensor
    static bool rainState = false;
    static uint32_t lastTime = 0;

    // Simulate a Rain Sensor state change every 20 seconds
    if (millis() - lastTime > 20000) {
        rainState = !rainState;
        lastTime = millis();
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    return rainState;
}

void loop() {
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }

    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
        button_state = false; // released
        // button is released - toggle Rain State (Not Detected/Detected)
        RainSensor.setRain(!RainSensor.getRain()); // same as RainSensor = !
        ↪RainSensor;
        Serial.printf("User button released. Setting the Rain Sensor to %s.\r\n",
        ↪RainSensor ? "Detected" : "Not Detected");
        // LED will indicate the Rain Sensor state
        if (RainSensor) {
            digitalWrite(ledPin, HIGH); // LED ON
        } else {
            digitalWrite(ledPin, LOW); // LED OFF
        }
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to
    ↪decommission matter node
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Rain Sensor Matter Accessory. It shall be
        ↪commissioned again.");
        Matter.decommission();
        button_time_stamp = millis(); // avoid running decommissioning again, reboot
        ↪takes a second or so
    }

    // Simulated Rain Sensor
    RainSensor.setRain(simulatedHWRainSensor());

    delay(50);
}

```

MatterTemperatureControlledCabinet

About The `MatterTemperatureControlledCabinet` class provides a temperature controlled cabinet endpoint for Matter networks. This endpoint implements the Matter temperature control standard for managing temperature setpoints with min/max limits, step control (always enabled), or temperature level support.

Features:

- Two initialization modes:
 - **Temperature Number Mode** (`begin(tempSetpoint, minTemp, maxTemp, step)`): Temperature setpoint control with min/max limits and step control
 - **Temperature Level Mode** (`begin(supportedLevels, levelCount, selectedLevel)`): Temperature level control with array of supported levels
- 1/100th degree Celsius precision (for temperature_number mode)
- Min/max temperature limits with validation (temperature_number mode)
- Temperature step control (temperature_number mode, always enabled)

- Temperature level array support (temperature_level mode)
- Automatic setpoint validation against limits
- Feature validation - methods return errors if called with wrong feature mode
- Integration with Apple HomeKit, Amazon Alexa, and Google Home
- Matter standard compliance

Important: The `temperature_number` and `temperature_level` features are **mutually exclusive**. Only one can be enabled at a time. Use `begin(tempSetpoint, minTemp, maxTemp, step)` for `temperature_number` mode or `begin(supportedLevels, levelCount, selectedLevel)` for `temperature_level` mode.

Use Cases:

- Refrigerators and freezers
- Wine coolers
- Medical storage cabinets
- Laboratory equipment
- Food storage systems
- Temperature-controlled storage units

API Reference

Constructor

MatterTemperatureControlledCabinet Creates a new Matter temperature controlled cabinet endpoint.

```
MatterTemperatureControlledCabinet();
```

Initialization

begin Initializes the Matter temperature controlled cabinet endpoint with **temperature_number** feature mode. This enables temperature setpoint control with min/max limits and optional step.

Note: This mode is mutually exclusive with `temperature_level` mode. Use `begin(supportedLevels, levelCount, selectedLevel)` for temperature level control.

```
bool begin(double tempSetpoint = 0.00, double minTemperature = -10.0, double ↵
↵maxTemperature = 32.0, double step = 0.50);
```

- `tempSetpoint` - Initial temperature setpoint in Celsius (default: 0.00)
- `minTemperature` - Minimum allowed temperature in Celsius (default: -10.0)
- `maxTemperature` - Maximum allowed temperature in Celsius (default: 32.0)
- `step` - Initial temperature step value in Celsius (default: 0.50)

This function will return `true` if successful, `false` otherwise.

Note: The implementation stores temperature with 1/100th degree Celsius precision internally. The `temperature_step` feature is always enabled for `temperature_number` mode, allowing `setStep()` to be called later even if step is not provided in `begin()`.

begin (overloaded) Initializes the Matter temperature controlled cabinet endpoint with **temperature_level** feature mode. This enables temperature level control with an array of supported levels.

Note: This mode is mutually exclusive with `temperature_number` mode. Use `begin(tempSetpoint, minTemp, maxTemp, step)` for temperature setpoint control.

```
bool begin(uint8_t *supportedLevels, uint16_t levelCount, uint8_t selectedLevel ↵
↵0);
```

- `supportedLevels` - Pointer to array of temperature level values (`uint8_t`, 0-255)
- `levelCount` - Number of levels in the array (maximum: 16)
- `selectedLevel` - Initial selected temperature level (default: 0)

This function will return `true` if successful, `false` otherwise.

Note: The maximum number of supported levels is 16 (defined by `temperature_control::k_max_temp_level_count`). The array is copied internally, so it does not need to remain valid after the function returns. This method uses a custom endpoint implementation that properly supports the `temperature_level` feature.

end Stops processing Matter temperature controlled cabinet events.

```
void end();
```

Temperature Setpoint Control

setTemperatureSetpoint Sets the temperature setpoint value. The setpoint will be validated against min/max limits.

Requires: `temperature_number` feature mode (use `begin()`)

```
bool setTemperatureSetpoint(double temperature);
```

- `temperature` - Temperature setpoint in Celsius

This function will return `true` if successful, `false` otherwise (e.g., if temperature is out of range or wrong feature mode).

Note: Temperature is stored with 1/100th degree Celsius precision. The setpoint must be within the min/max temperature range. This method will return `false` and log an error if called when using `temperature_level` mode.

getTemperatureSetpoint Gets the current temperature setpoint value.

```
double getTemperatureSetpoint();
```

This function will return the temperature setpoint in Celsius with 1/100th degree precision.

Min/Max Temperature Control

setMinTemperature Sets the minimum allowed temperature.

Requires: `temperature_number` feature mode (use `begin()`)

```
bool setMinTemperature(double temperature);
```

- `temperature` - Minimum temperature in Celsius

This function will return `true` if successful, `false` otherwise. Will return `false` and log an error if called when using `temperature_level` mode.

getMinTemperature Gets the minimum allowed temperature.

```
double getMinTemperature();
```

This function will return the minimum temperature in Celsius with 1/100th degree precision.

setMaxTemperature Sets the maximum allowed temperature.

Requires: temperature_number feature mode (use `begin()`)

```
bool setMaxTemperature(double temperature);
```

- temperature - Maximum temperature in Celsius

This function will return `true` if successful, `false` otherwise. Will return `false` and log an error if called when using temperature_level mode.

getMaxTemperature Gets the maximum allowed temperature.

```
double getMaxTemperature();
```

This function will return the maximum temperature in Celsius with 1/100th degree precision.

Temperature Step Control

setStep Sets the temperature step value.

Requires: temperature_number feature mode (use `begin()`)

```
bool setStep(double step);
```

- step - Temperature step value in Celsius

This function will return `true` if successful, `false` otherwise.

Note: The temperature_step feature is always enabled when using temperature_number mode, so this method can be called at any time to set or change the step value, even if step was not provided in `begin()`. This method will return `false` and log an error if called when using temperature_level mode.

getStep Gets the current temperature step value.

```
double getStep();
```

This function will return the temperature step in Celsius with 1/100th degree precision.

Temperature Level Control (Optional)

setSelectedTemperatureLevel Sets the selected temperature level.

Requires: temperature_level feature mode (use `begin(supportedLevels, levelCount, selectedLevel)`)

```
bool setSelectedTemperatureLevel(uint8_t level);
```

- level - Temperature level (0-255)

This function will return `true` if successful, `false` otherwise.

Note: Temperature level and temperature number features are mutually exclusive. This method will return `false` and log an error if called when using temperature_number mode.

getSelectedTemperatureLevel Gets the current selected temperature level.

```
uint8_t getSelectedTemperatureLevel();
```

This function will return the selected temperature level (0-255).

setSupportedTemperatureLevels Sets the supported temperature levels array.

Requires: temperature_level feature mode (use begin(supportedLevels, levelCount, selectedLevel))

```
bool setSupportedTemperatureLevels(uint8_t *levels, uint16_t count);
```

- levels - Pointer to array of temperature level values (array is copied internally)
- count - Number of levels in the array (maximum: 16)

This function will return true if successful, false otherwise.

Note: The maximum number of supported levels is 16. The array is copied internally, so it does not need to remain valid after the function returns. This method will return false and log an error if called when using temperature_number mode or if count exceeds the maximum.

getSupportedTemperatureLevelsCount Gets the count of supported temperature levels.

```
uint16_t getSupportedTemperatureLevelsCount();
```

This function will return the number of supported temperature levels.

Example

Temperature Controlled Cabinet The example demonstrates the temperature_number mode with dynamic temperature updates. The temperature setpoint automatically cycles between the minimum and maximum limits every 1 second using the configured step value, allowing Matter controllers to observe real-time changes. The example also monitors and logs when the initial setpoint is reached or overpassed in each direction.

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example demonstrates the Temperature Number mode of the Matter Temperature_
↳ Controlled Cabinet Device.
 *
 * This example will create a Matter Device which can be commissioned and_
↳ controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and_
↳ Controller messages.
 *
 * The example will create a Matter Temperature Controlled Cabinet Device using_
↳ temperature_number feature.
 * The Temperature Controlled Cabinet can be controlled via Matter controllers to_
↳ set
 * temperature setpoint with min/max limits and optional step control.
 *
 * This mode is mutually exclusive with temperature_level mode.
```

(continues on next page)

(continued from previous page)

```

* See MatterTemperatureControlledCabinetLevels example for temperature level
↪control.
*/

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Temperature Controlled Cabinet Endpoint
MatterTemperatureControlledCabinet TemperatureCabinet;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here - decommissioning button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control - decommission the Matter Node
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

// Temperature control state
struct TemperatureControlState {
    bool initialized;
    bool increasing;
    double currentSetpoint;
    double initialSetpoint;
    bool setpointReachedIncreasing;
    bool setpointReachedDecreasing;
};

static TemperatureControlState tempState = {
    .initialized = false,
    .increasing = true,
    .currentSetpoint = 0.0,
    .initialSetpoint = 0.0,
    .setpointReachedIncreasing = false,
    .setpointReachedDecreasing = false
};

// Initialize temperature control state
void initTemperatureControl() {
    if (!tempState.initialized) {
        tempState.currentSetpoint = TemperatureCabinet.getTemperatureSetpoint();
        tempState.initialSetpoint = tempState.currentSetpoint;
        tempState.initialized = true;
    }
}

```

(continues on next page)

(continued from previous page)

```

// Check and log when initial setpoint is reached/overpassed
void checkSetpointReached(double newSetpoint, bool isIncreasing, bool
↪directionChanged) {
    if (directionChanged) {
        // Reset flags when direction changes
        tempState.setpointReachedIncreasing = false;
        tempState.setpointReachedDecreasing = false;
        return;
    }

    if (isIncreasing && !tempState.setpointReachedIncreasing && newSetpoint >=
↪tempState.initialSetpoint) {
        Serial.printf("*** Temperature setpoint %.02f°C reached/overpassed while
↪increasing ***\r\n", tempState.initialSetpoint);
        tempState.setpointReachedIncreasing = true;
    } else if (!isIncreasing && !tempState.setpointReachedDecreasing && newSetpoint
↪<= tempState.initialSetpoint) {
        Serial.printf("*** Temperature setpoint %.02f°C reached/overpassed while
↪decreasing ***\r\n", tempState.initialSetpoint);
        tempState.setpointReachedDecreasing = true;
    }
}

// Update temperature setpoint with cycling logic
void updateTemperatureSetpoint() {
    double minTemp = TemperatureCabinet.getMinTemperature();
    double maxTemp = TemperatureCabinet.getMaxTemperature();
    double step = TemperatureCabinet.getStep();

    // Calculate next setpoint based on direction and step
    bool directionChanged = false;

    if (tempState.increasing) {
        tempState.currentSetpoint += step;
        if (tempState.currentSetpoint >= maxTemp) {
            tempState.currentSetpoint = maxTemp;
            tempState.increasing = false; // Reverse direction
            directionChanged = true;
        }
    } else {
        tempState.currentSetpoint -= step;
        if (tempState.currentSetpoint <= minTemp) {
            tempState.currentSetpoint = minTemp;
            tempState.increasing = true; // Reverse direction
            directionChanged = true;
        }
    }

    // Check if setpoint has been reached or overpassed
    checkSetpointReached(tempState.currentSetpoint, tempState.increasing,
↪directionChanged);

    // Update the temperature setpoint
    if (TemperatureCabinet.setTemperatureSetpoint(tempState.currentSetpoint)) {
        Serial.printf("Temperature setpoint updated to: %.02f°C (Range: %.02f°C to %.
↪02f°C)\r\n", tempState.currentSetpoint, minTemp, maxTemp);
    } else {
        Serial.printf("Failed to update temperature setpoint to: %.02f°C\r\n",
↪tempState.currentSetpoint);
    }
}

```

(continues on next page)

(continued from previous page)

```

// Print current temperature status
void printTemperatureStatus() {
    Serial.printf(
        "Current Temperature Setpoint: %.02f°C (Range: %.02f°C to %.02f°C)\r\n",
        ↪TemperatureCabinet.getTemperatureSetpoint(),
        TemperatureCabinet.getMinTemperature(), TemperatureCabinet.getMaxTemperature()
    );
}

// Handle button press for decommissioning
void handleButtonPress() {
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }

    if (digitalRead(buttonPin) == HIGH && button_state) {
        button_state = false; // released
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to
    ↪decommission matter node
    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Temperature Controlled Cabinet Matter
        ↪Accessory. It shall be commissioned again.");
        Matter.decommission();
        button_time_stamp = millis(); // avoid running decommissioning again, reboot
        ↪takes a second or so
    }
}

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission
    ↪the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
    ↪Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    #endif

    // Initialize Temperature Controlled Cabinet with:
    // - Initial setpoint: 4.0°C (typical refrigerator temperature)
    // - Min temperature: -10.0°C
    // - Max temperature: 10.0°C
    // - Step: 0.5°C (optional, for temperature_step feature)
    TemperatureCabinet.begin(4.0, -10.0, 10.0, 0.5);

```

(continues on next page)

(continued from previous page)

```

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();

// Check Matter Accessory Commissioning state, which may change during execution.
of loop()
if (!Matter.isDeviceCommissioned()) {
    Serial.println("");
    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code.
or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Temperature Controlled Cabinet Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for.
commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network.
Ready for use.");
}

// Print initial configuration
Serial.println("\nTemperature Controlled Cabinet Configuration:");
Serial.printf("  Setpoint: %.02f°C\n", TemperatureCabinet.
getTemperatureSetpoint());
Serial.printf("  Min Temperature: %.02f°C\n", TemperatureCabinet.
getMinTemperature());
Serial.printf("  Max Temperature: %.02f°C\n", TemperatureCabinet.
getMaxTemperature());
Serial.printf("  Step: %.02f°C\n", TemperatureCabinet.getStep());
}

void loop() {
    static uint32_t timeCounter = 0;
    static uint32_t lastUpdateTime = 0;

    // Initialize temperature control state on first run
    initTemperatureControl();

    // Update temperature setpoint dynamically every 1 second
    uint32_t currentTime = millis();
    if (currentTime - lastUpdateTime >= 1000) { // 1 second interval
        lastUpdateTime = currentTime;
        updateTemperatureSetpoint();
    }

    // Print the current temperature setpoint every 5s
    if (!(timeCounter++ % 10)) { // delaying for 500ms x 10 = 5s
        printTemperatureStatus();
    }

    // Handle button press for decommissioning
    handleButtonPress();
}

```

(continues on next page)

(continued from previous page)

```
    delay(500);  
}
```

Temperature Controlled Cabinet (Level Mode) A separate example demonstrates the `temperature_level` mode with dynamic level updates. The temperature level automatically cycles through all supported levels every 1 second in both directions (increasing and decreasing), allowing Matter controllers to observe real-time changes. The example also monitors and logs when the initial level is reached or overpassed in each direction.

See `MatterTemperatureControlledCabinetLevels` example for the temperature level mode implementation.

MatterTemperatureSensor

About The `MatterTemperatureSensor` class provides a temperature sensor endpoint for Matter networks. This endpoint implements the Matter temperature sensing standard for read-only temperature reporting.

Features: * Temperature measurement reporting in Celsius * 1/100th degree Celsius precision * Read-only sensor (no control functionality) * Automatic temperature updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Room temperature monitoring * Weather stations * HVAC systems * Temperature logging * Smart home climate monitoring

API Reference

Constructor

MatterTemperatureSensor Creates a new Matter temperature sensor endpoint.

```
MatterTemperatureSensor();
```

Initialization

begin Initializes the Matter temperature sensor endpoint with an initial temperature value.

```
bool begin(double temperature = 0.00);
```

- `temperature` - Initial temperature in Celsius (default: 0.00)

This function will return `true` if successful, `false` otherwise.

Note: The implementation stores temperature with 1/100th degree Celsius precision internally.

end Stops processing Matter temperature sensor events.

```
void end();
```

Temperature Control

setTemperature Sets the reported temperature value.

```
bool setTemperature(double temperature);
```

- temperature - Temperature in Celsius

This function will return `true` if successful, `false` otherwise.

Note: Temperature is stored with 1/100th degree Celsius precision. The valid range is typically -273.15°C (absolute zero) to 327.67°C.

getTemperature Gets the current reported temperature value.

```
double getTemperature();
```

This function will return the temperature in Celsius with 1/100th degree precision.

Operators

double operator Returns the current temperature.

```
operator double();
```

Example:

```
double temp = mySensor; // Get current temperature
```

Assignment operator Sets the temperature value.

```
void operator=(double temperature);
```

Example:

```
mySensor = 25.5; // Set temperature to 25.5°C
```

Example

Temperature Sensor

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
```

(continues on next page)

(continued from previous page)

```

* Turning DEBUG Level ON may be useful to following Matter Accessory and
↳ Controller messages.
*/

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Temperature Sensor Endpoint
MatterTemperatureSensor SimulatedTemperatureSensor;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
↳ Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// set your board USER BUTTON pin here - decommissioning button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control - decommission the Matter Node
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳ or longer, to decommission

// Simulate a temperature sensor - add your preferred temperature sensor library
↳ code here
float getSimulatedTemperature() {
    // The Endpoint implementation keeps an int16_t as internal value information,
    // which stores data in 1/100th Celsius.
    static float simulatedTempHWSensor = -10.0;

    // it will increase from -10C to 10C in 0.5C steps to simulate a temperature
    ↳ sensor
    simulatedTempHWSensor = simulatedTempHWSensor + 0.5;
    if (simulatedTempHWSensor > 10) {
        simulatedTempHWSensor = -10;
    }

    return simulatedTempHWSensor;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission
    ↳ the Matter Node
    pinMode(buttonPin, INPUT_PULLUP);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
    ↳ Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi

```

(continues on next page)

(continued from previous page)

```

WiFi.begin(ssid, password);
// Wait for connection
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println();
#endif

// set initial temperature sensor measurement
// Simulated Sensor - it shall initially print -25C and then move to the -10C to
↪10C range
SimulatedTemperatureSensor.begin(-25.00);

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();

// Check Matter Accessory Commissioning state, which may change during execution
↪of loop()
if (!Matter.isDeviceCommissioned()) {
    Serial.println("");
    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code
↪or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↪str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Temperature Sensor Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for
↪commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network.
↪Ready for use.");
}
}

void loop() {
    static uint32_t timeCounter = 0;

    // Print the current temperature value every 5s
    if (!(timeCounter++ % 10)) { // delaying for 500ms x 10 = 5s
        // Print the current temperature value
        Serial.printf("Current Temperature is %.02fC\r\n", SimulatedTemperatureSensor.
↪getTemperature());
        // Update Temperature from the (Simulated) Hardware Sensor
        // Matter APP shall display the updated temperature percent
        SimulatedTemperatureSensor.setTemperature(getSimulatedTemperature());
    }

    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }
}

```

(continues on next page)

(continued from previous page)

```

if (digitalRead(buttonPin) == HIGH && button_state) {
    button_state = false; // released
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to
↳ decommission matter node
uint32_t time_diff = millis() - button_time_stamp;
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning Temperature Sensor Matter Accessory. It shall
↳ be commissioned again.");
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot
↳ takes a second or so
}

delay(500);
}

```

MatterThermostat

About The MatterThermostat class provides a thermostat endpoint for Matter networks with temperature control, setpoints, and multiple operating modes. This endpoint implements the Matter thermostat standard.

Features: * Multiple operating modes (OFF, HEAT, COOL, AUTO, etc.) * Heating and cooling setpoint control * Local temperature reporting * Automatic temperature regulation * Deadband control for AUTO mode * Callback support for mode, temperature, and setpoint changes * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * HVAC systems * Smart thermostats * Temperature control systems * Climate control automation * Energy management systems

API Reference

Constructor

MatterThermostat Creates a new Matter thermostat endpoint.

```
MatterThermostat();
```

Initialization

begin Initializes the Matter thermostat endpoint with control sequence and auto mode settings.

```

bool begin(ControlSequenceOfOperation_t controlSequence = THERMOSTAT_SEQ_OP_
↳ COOLING, ThermostatAutoMode_t autoMode = THERMOSTAT_AUTO_MODE_DISABLED);

```

- controlSequence - Control sequence of operation (default: THERMOSTAT_SEQ_OP_COOLING)
- autoMode - Auto mode enabled/disabled (default: THERMOSTAT_AUTO_MODE_DISABLED)

This function will return true if successful, false otherwise.

end Stops processing Matter thermostat events.

```
void end();
```

Control Sequences

ControlSequenceOfOperation_t Control sequence enumeration:

- THERMOSTAT_SEQ_OP_COOLING - Cooling only
- THERMOSTAT_SEQ_OP_COOLING_REHEAT - Cooling with reheat
- THERMOSTAT_SEQ_OP_HEATING - Heating only
- THERMOSTAT_SEQ_OP_HEATING_REHEAT - Heating with reheat
- THERMOSTAT_SEQ_OP_COOLING_HEATING - Cooling and heating
- THERMOSTAT_SEQ_OP_COOLING_HEATING_REHEAT - Cooling and heating with reheat

Thermostat Modes

ThermostatMode_t Thermostat mode enumeration:

- THERMOSTAT_MODE_OFF - Off
- THERMOSTAT_MODE_AUTO - Auto mode
- THERMOSTAT_MODE_COOL - Cooling mode
- THERMOSTAT_MODE_HEAT - Heating mode
- THERMOSTAT_MODE_EMERGENCY_HEAT - Emergency heat
- THERMOSTAT_MODE_PRECOOLING - Precooling
- THERMOSTAT_MODE_FAN_ONLY - Fan only
- THERMOSTAT_MODE_DRY - Dry mode
- THERMOSTAT_MODE_SLEEP - Sleep mode

Mode Control

setMode Sets the thermostat mode.

```
bool setMode(ThermostatMode_t mode);
```

getMode Gets the current thermostat mode.

```
ThermostatMode_t getMode();
```

getThermostatModeString Gets a friendly string for the thermostat mode.

```
static const char *getThermostatModeString(uint8_t mode);
```

Temperature Control

setLocalTemperature Sets the local temperature reading.

```
bool setLocalTemperature(double temperature);
```

- temperature - Temperature in Celsius

getLocalTemperature Gets the current local temperature.

```
double getLocalTemperature();
```

Setpoint Control

setCoolingHeatingSetpoints Sets both cooling and heating setpoints.

```
bool setCoolingHeatingSetpoints(double _setpointHeatingTemperature, double _  
    ↪ setpointCoolingTemperature);
```

- `_setpointHeatingTemperature` - Heating setpoint in Celsius (or 0xffff to keep current)
- `_setpointCoolingTemperature` - Cooling setpoint in Celsius (or 0xffff to keep current)

Note: Heating setpoint must be lower than cooling setpoint. In AUTO mode, cooling setpoint must be at least 2.5°C higher than heating setpoint (deadband).

setHeatingSetpoint Sets the heating setpoint.

```
bool setHeatingSetpoint(double _setpointHeatingTemperature);
```

getHeatingSetpoint Gets the heating setpoint.

```
double getHeatingSetpoint();
```

setCoolingSetpoint Sets the cooling setpoint.

```
bool setCoolingSetpoint(double _setpointCoolingTemperature);
```

getCoolingSetpoint Gets the cooling setpoint.

```
double getCoolingSetpoint();
```

Setpoint Limits

getMinHeatSetpoint Gets the minimum heating setpoint limit.

```
float getMinHeatSetpoint();
```

getMaxHeatSetpoint Gets the maximum heating setpoint limit.

```
float getMaxHeatSetpoint();
```

getMinCoolSetpoint Gets the minimum cooling setpoint limit.

```
float getMinCoolSetpoint();
```

getMaxCoolSetpoint Gets the maximum cooling setpoint limit.


```
float getMaxCoolSetpoint();
```

getDeadBand Gets the deadband value (minimum difference between heating and cooling setpoints in AUTO mode).

```
float getDeadBand();
```

Event Handling

onChange Sets a callback for when any parameter changes.

```
void onChange(EndPointCB onChangeCB);
```

onChangeMode Sets a callback for mode changes.

```
void onChangeMode(EndPointModeCB onChangeCB);
```

onChangeLocalTemperature Sets a callback for local temperature changes.

```
void onChangeLocalTemperature(EndPointTemperatureCB onChangeCB);
```

onChangeCoolingSetpoint Sets a callback for cooling setpoint changes.

```
void onChangeCoolingSetpoint(EndPointCoolingSetpointCB onChangeCB);
```

onChangeHeatingSetpoint Sets a callback for heating setpoint changes.

```
void onChangeHeatingSetpoint(EndPointHeatingSetpointCB onChangeCB);
```

Example

Thermostat

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
    This example is an example code that will create a Matter Device which can be
    commissioned and controlled from a Matter Environment APP.
    Additionally the ESP32 will send debug messages indicating the Matter activity.
```

(continues on next page)

(continued from previous page)

```

    Turning DEBUG Level ON may be useful to following Matter Accessory and
    ↳ Controller messages.
    */

    // Matter Manager
    #include <Arduino.h>
    #include <Matter.h>
    #if !CONFIG_ENABLE_CHIPOBLE
    // if the device can be commissioned using BLE, WiFi is not used - save flash space
    #include <WiFi.h>
    #endif

    // List of Matter Endpoints for this Node
    // Matter Thermostat Endpoint
    MatterThermostat SimulatedThermostat;

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
    ↳ Network
    #if !CONFIG_ENABLE_CHIPOBLE
    // WiFi is manually set and started
    const char *ssid = "your-ssid";           // Change this to your WiFi SSID
    const char *password = "your-password";   // Change this to your WiFi password
    #endif

    // set your board USER BUTTON pin here - decommissioning button
    const uint8_t buttonPin = BOOT_PIN;      // Set your pin here. Using BOOT Button.

    // Button control - decommission the Matter Node
    uint32_t button_time_stamp = 0;           // debouncing control
    bool button_state = false;                // false = released | true = pressed
    const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
    ↳ or longer, to decommission

    // Simulate a system that will activate heating/cooling in addition to a
    ↳ temperature sensor - add your preferred code here
    float getSimulatedTemperature(bool isHeating, bool isCooling) {
        // read sensor temperature and apply heating/cooling
        float simulatedTempHWSensor = SimulatedThermostat.getLocalTemperature();

        if (isHeating) {
            // it will increase to simulate a heating system
            simulatedTempHWSensor = simulatedTempHWSensor + 0.5;
        }
        if (isCooling) {
            // it will decrease to simulate a colling system
            simulatedTempHWSensor = simulatedTempHWSensor - 0.5;
        }
        // otherwise, it will keep the temperature stable
        return simulatedTempHWSensor;
    }

    void setup() {
        // Initialize the USER BUTTON (Boot button) that will be used to decommission
        ↳ the Matter Node
        pinMode(buttonPin, INPUT_PULLUP);

        Serial.begin(115200);

        // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
        ↳ Network
        #if !CONFIG_ENABLE_CHIPOBLE

```

(continues on next page)

(continued from previous page)

```

// Manually connect to WiFi
WiFi.begin(ssid, password);
// Wait for connection
while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println();
#endif

// Simulated Thermostat in COOLING and HEATING mode with Auto Mode to keep the
↳temperature between setpoints
// Auto Mode can only be used when the control sequence of operation is Cooling &
↳Heating
SimulatedThermostat.begin(MatterThermostat::THERMOSTAT_SEQ_OP_COOLING_HEATING,
↳MatterThermostat::THERMOSTAT_AUTO_MODE_ENABLED);

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();

// Check Matter Accessory Commissioning state, which may change during execution
↳of loop()
if (!Matter.isDeviceCommissioned()) {
    Serial.println("");
    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code
↳or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↳str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Thermostat Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for
↳commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network.
↳Ready for use.");

    // after commissioning, set initial thermostat parameters
    // start the thermostat in AUTO mode
    SimulatedThermostat.setMode(MatterThermostat::THERMOSTAT_MODE_AUTO);
    // cooling setpoint must be lower than heating setpoint by at least 2.5C
↳(deadband), in auto mode
    SimulatedThermostat.setCoolingHeatingSetpoints(20.0, 23.00); // the target
↳cooler and heating setpoint
    // set the local temperature sensor in Celsius
    SimulatedThermostat.setLocalTemperature(12.50);

    Serial.println();
    Serial.printf(
        "Initial Setpoints are %.01fC to %.01fC with a minimum 2.5C difference\r\n",
↳SimulatedThermostat.getHeatingSetpoint(),
        SimulatedThermostat.getCoolingSetpoint()
    );
    Serial.printf("Auto mode is ON. Initial Temperature of %.01fC \r\n",
↳SimulatedThermostat.getLocalTemperature());

```

(continues on next page)

(continued from previous page)

```

        Serial.println("Local Temperature Sensor will be simulated every 10 seconds_
↪and changed by a simulated heater and cooler to move in between setpoints.");
    }
}

// This will simulate the thermostat control system (heating and cooling)
// User can set a local temperature using the Serial input (type a number and_
↪press Enter)
// New temperature can be an positive or negative temperature in Celsius, between -
↪50C and 50C
// Initial local temperature is 10C as defined in getSimulatedTemperature()_
↪function
void readSerialForNewTemperature() {
    static String newTemperatureStr;

    while (Serial.available()) {
        char c = Serial.read();
        if (c == '\n' || c == '\r') {
            if (newTemperatureStr.length() > 0) {
                // convert the string to a float value
                float newTemperature = newTemperatureStr.toFloat();
                // check if the new temperature is valid
                if (newTemperature >= -50.0 && newTemperature <= 50.0) {
                    // set the new temperature
                    SimulatedThermostat.setLocalTemperature(newTemperature);
                    Serial.printf("New Temperature is %.01fC\r\n", newTemperature);
                } else {
                    Serial.println("Invalid Temperature value. Please type a number between -
↪50 and 50");
                }
                newTemperatureStr = "";
            }
            else {
                if (c == '+' || c == '-' || (c >= '0' && c <= '9') || c == '.') {
                    newTemperatureStr += c;
                } else {
                    Serial.println("Invalid character. Please type a number between -50 and 50
↪");
                    newTemperatureStr = "";
                }
            }
        }
    }
}

// loop will simulate the thermostat control system
// User can set a local temperature using the Serial input (type a number and_
↪press Enter)
// User can change the thermostat mode using the Matter APP (smartphone)
// The loop will simulate a heating and cooling system and the associated local_
↪temperature change
void loop() {
    static uint32_t timeCounter = 0;

    // Simulate the heating and cooling systems
    static bool isHeating = false;
    static bool isCooling = false;

    // check if a new temperature is typed in the Serial Monitor
    readSerialForNewTemperature();

    // simulate thermostat with heating/cooling system and the associated local_

```

(continues on next page)

(continued from previous page)

```

↪temperature change, every 10s
  if (!(timeCounter++ % 20)) { // delaying for 500ms x 20 = 10s
    float localTemperature = getSimulatedTemperature(isHeating, isCooling);
    // Print the current thermostat local temperature value
    Serial.printf("Current Local Temperature is %.01fC\r\n", localTemperature);
    SimulatedThermostat.setLocalTemperature(localTemperature); // publish the new
↪temperature value

    // Simulate the thermostat control system - User has 4 modes: OFF, HEAT, COOL,
↪AUTO
    switch (SimulatedThermostat.getMode()) {
      case MatterThermostat::THERMOSTAT_MODE_OFF:
        // turn off the heating and cooling systems
        isHeating = false;
        isCooling = false;
        break;
      case MatterThermostat::THERMOSTAT_MODE_AUTO:
        // User APP has set the thermostat to AUTO mode -- keeping the temperature
↪between both setpoints
        // check if the heating system should be turned on or off
        if (localTemperature < SimulatedThermostat.getHeatingSetpoint() +
↪SimulatedThermostat.getDeadBand()) {
          // turn on the heating system and turn off the cooling system
          isHeating = true;
          isCooling = false;
        }
        if (localTemperature > SimulatedThermostat.getCoolingSetpoint() -
↪SimulatedThermostat.getDeadBand()) {
          // turn off the heating system and turn on the cooling system
          isHeating = false;
          isCooling = true;
        }
        break;
      case MatterThermostat::THERMOSTAT_MODE_HEAT:
        // Simulate the heating system - User has turned the heating system ON
        isHeating = true;
        isCooling = false; // keep the cooling system off as it is in heating mode
        // when the heating system is in HEATING mode, it will be turned off as
↪soon as the local temperature is above the setpoint
        if (localTemperature > SimulatedThermostat.getHeatingSetpoint()) {
          // turn off the heating system
          isHeating = false;
        }
        break;
      case MatterThermostat::THERMOSTAT_MODE_COOL:
        // Simulate the cooling system - User has turned the cooling system ON
        if (SimulatedThermostat.getMode() == MatterThermostat::THERMOSTAT_MODE_
↪COOL) {
          isCooling = true;
          isHeating = false; // keep the heating system off as it is in cooling
↪mode
          // when the cooling system is in COOLING mode, it will be turned off as
↪soon as the local temperature is bellow the setpoint
          if (localTemperature < SimulatedThermostat.getCoolingSetpoint()) {
            // turn off the cooling system
            isCooling = false;
          }
        }
        break;
      default: log_e("Invalid Thermostat Mode %d", SimulatedThermostat.getMode());
    }
  }

```

(continues on next page)

(continued from previous page)

```

// Reporting Heating and Cooling status
Serial.printf(
    "\tThermostat Mode: %s >>> Heater is %s -- Cooler is %s\r\n",
    MatterThermostat::getThermostatModeString(SimulatedThermostat.getMode()),
    isHeating ? "ON" : "OFF", isCooling ? "ON" : "OFF"
);
}
// Check if the button has been pressed
if (digitalRead(buttonPin) == LOW && !button_state) {
    // deals with button debouncing
    button_time_stamp = millis(); // record the time while the button is pressed.
    button_state = true;          // pressed.
}

if (digitalRead(buttonPin) == HIGH && button_state) {
    button_state = false; // released
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to
// decommission matter node
uint32_t time_diff = millis() - button_time_stamp;
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning Thermostat Matter Accessory. It shall be
    commissioned again.");
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot
    // takes a second or so
}

delay(500);
}

```

MatterWaterFreezeDetector

About The `MatterWaterFreezeDetector` class provides a water freeze detector endpoint for Matter networks. This endpoint implements the Matter water freeze detection standard for detecting water freeze conditions (detected/not detected states).

Features: * Water freeze detection state reporting (detected/not detected) * Simple boolean state * Read-only sensor (no control functionality) * Automatic state updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Water pipe freeze monitoring * Outdoor water system protection * HVAC freeze detection * Smart home automation triggers * Preventative maintenance systems

API Reference

Constructor

MatterWaterFreezeDetector Creates a new Matter water freeze detector endpoint.

```
MatterWaterFreezeDetector();
```

Initialization

begin Initializes the Matter water freeze detector endpoint with an initial freeze detection state.

```
bool begin(bool _freezeState = false);
```

- `_freezeState` - Initial water freeze detection state (`true` = detected, `false` = not detected, default: `false`)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter water freeze detector events.

```
void end();
```

Water Freeze Detection State Control

setFreeze Sets the water freeze detection state.

```
bool setFreeze(bool _freezeState);
```

- `_freezeState` - Water freeze detection state (`true` = detected, `false` = not detected)

This function will return `true` if successful, `false` otherwise.

getFreeze Gets the current water freeze detection state.

```
bool getFreeze();
```

This function will return `true` if water freeze is detected, `false` if not detected.

Operators

bool operator Returns the current water freeze detection state.

```
operator bool();
```

Example:

```
if (myDetector) {  
    Serial.println("Water freeze is detected");  
} else {  
    Serial.println("Water freeze is not detected");  
}
```

Assignment operator Sets the water freeze detection state.

```
void operator=(bool _freezeState);
```

Example:

```
myDetector = true;    // Set water freeze detection to detected  
myDetector = false;   // Set water freeze detection to not detected
```

Example

Water Freeze Detector

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
↳Controller messages.
 *
 * The example will create a Matter Water Freeze Detector Device.
 * The Water Freeze Detector state can be toggled by pressing the onboard button.
 * The Water Freeze Detector state will be indicated by the onboard LED.
 * The Water Freeze Detector state will be simulated to change every 20 seconds.
 *
 * The onboard button can be kept pressed for 5 seconds to decommission the Matter.
↳Node.
 * The example will also show the manual commissioning code and QR code to be used
↳in the Matter environment.
 */

// Matter Manager
#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Water Freeze Detector Endpoint
MatterWaterFreezeDetector WaterFreezeDetector;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// LED will be used to indicate the Water Freeze Detector state
// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↳BUILTIN
#warning "Do not forget to set the RGB LED pin"
```

(continues on next page)

(continued from previous page)

```

#endif

// set your board USER BUTTON pin here - decommissioning and Manual Water Freeze_
↪Detector toggle button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0; // debouncing control
bool button_state = false; // false = released | true = pressed
const uint32_t debounceTime = 250; // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission_
↪the Matter Node
    // The button will also be used to manually toggle the Water Freeze Detector_
↪state
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the LED (light) GPIO and Matter End Point
    pinMode(ledPin, OUTPUT);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    #endif

    // set initial water freeze detector state as false (default)
    WaterFreezeDetector.begin();
    digitalWrite(ledPin, LOW); // LED OFF

    // Matter beginning - Last step, after all EndPoints are initialized
    Matter.begin();

    // Check Matter Accessory Commissioning state, which may change during execution_
↪of loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code_
↪or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
↪str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Water Freeze Detector Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for_

```

(continues on next page)

(continued from previous page)

```

    ↪commissioning.");
    }
    }
    Serial.println("Matter Node is commissioned and connected to the network.↵
    ↪Ready for use.");
    }
}

bool simulatedHWWaterFreezeDetector() {
    // Simulated Water Freeze Detector
    static bool freezeState = false;
    static uint32_t lastTime = 0;

    // Simulate a Water Freeze Detector state change every 20 seconds
    if (millis() - lastTime > 20000) {
        freezeState = !freezeState;
        lastTime = millis();
    }
    return freezeState;
}

void loop() {
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true;          // pressed.
    }

    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
        button_state = false; // released
        // button is released - toggle Freeze State (Not Detected/Detected)
        WaterFreezeDetector.setFreeze(!WaterFreezeDetector.getFreeze()); // same as↵
    ↪WaterFreezeDetector = !WaterFreezeDetector;
        Serial.printf("User button released. Setting the Water Freeze Detector to %s.↵
    ↪r\n", WaterFreezeDetector ? "Detected" : "Not Detected");
        // LED will indicate the Water Freeze Detector state
        if (WaterFreezeDetector) {
            digitalWrite(ledPin, HIGH); // LED ON
        } else {
            digitalWrite(ledPin, LOW); // LED OFF
        }
    }

    // Onboard User Button is kept pressed for longer than 5 seconds in order to↵
    ↪decommission matter node
    if (button_state && time_diff > decommissioningTimeout) {
        Serial.println("Decommissioning Water Freeze Detector Matter Accessory. It↵
    ↪shall be commissioned again.");
        Matter.decommission();
        button_time_stamp = millis(); // avoid running decommissioning again, reboot↵
    ↪takes a second or so
    }

    // Simulated Water Freeze Detector
    WaterFreezeDetector.setFreeze(simulatedHWWaterFreezeDetector());

    delay(50);
}

```

MatterWaterLeakDetector

About The `MatterWaterLeakDetector` class provides a water leak detector endpoint for Matter networks. This endpoint implements the Matter water leak detection standard for detecting water leak conditions (detected/not detected states).

Features: * Water leak detection state reporting (detected/not detected) * Simple boolean state * Read-only sensor (no control functionality) * Automatic state updates * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Use Cases: * Water leak monitoring * Basement flood detection * Appliance leak detection * Smart home automation triggers * Preventative maintenance systems

API Reference

Constructor

MatterWaterLeakDetector Creates a new Matter water leak detector endpoint.

```
MatterWaterLeakDetector();
```

Initialization

begin Initializes the Matter water leak detector endpoint with an initial leak detection state.

```
bool begin(bool _leakState = false);
```

- `_leakState` - Initial water leak detection state (`true` = detected, `false` = not detected, default: `false`)

This function will return `true` if successful, `false` otherwise.

end Stops processing Matter water leak detector events.

```
void end();
```

Water Leak Detection State Control

setLeak Sets the water leak detection state.

```
bool setLeak(bool _leakState);
```

- `_leakState` - Water leak detection state (`true` = detected, `false` = not detected)

This function will return `true` if successful, `false` otherwise.

getLeak Gets the current water leak detection state.

```
bool getLeak();
```

This function will return `true` if water leak is detected, `false` if not detected.

Operators

bool operator Returns the current water leak detection state.

```
operator bool();
```

Example:

```
if (myDetector) {
    Serial.println("Water leak is detected");
} else {
    Serial.println("Water leak is not detected");
}
```

Assignment operator Sets the water leak detection state.

```
void operator=(bool _leakState);
```

Example:

```
myDetector = true;    // Set water leak detection to detected
myDetector = false;   // Set water leak detection to not detected
```

Example

Water Leak Detector

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/*
 * This example is an example code that will create a Matter Device which can be
 * commissioned and controlled from a Matter Environment APP.
 * Additionally the ESP32 will send debug messages indicating the Matter activity.
 * Turning DEBUG Level ON may be useful to following Matter Accessory and
 * ↪ Controller messages.
 *
 * The example will create a Matter Water Leak Detector Device.
 * The Water Leak Detector state can be toggled by pressing the onboard button.
 * The Water Leak Detector state will be indicated by the onboard LED.
 * The Water Leak Detector state will be simulated to change every 20 seconds.
 *
 * The onboard button can be kept pressed for 5 seconds to decommission the Matter
 * ↪ Node.
 * The example will also show the manual commissioning code and QR code to be used
 * ↪ in the Matter environment.
 */
// Matter Manager
```

(continues on next page)

(continued from previous page)

```

#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif

// List of Matter Endpoints for this Node
// Matter Water Leak Detector Endpoint
MatterWaterLeakDetector WaterLeakDetector;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// LED will be used to indicate the Water Leak Detector state
// set your board RGB LED pin here
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined LED_
↳BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

// set your board USER BUTTON pin here - decommissioning and Manual Water Leak_
↳Detector toggle button
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t debounceTime = 250;       // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↳or longer, to decommission

void setup() {
    // Initialize the USER BUTTON (Boot button) that will be used to decommission_
↳the Matter Node
    // The button will also be used to manually toggle the Water Leak Detector state
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the LED (light) GPIO and Matter End Point
    pinMode(ledPin, OUTPUT);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↳Network
    #if !CONFIG_ENABLE_CHIPOBLE
        // Manually connect to WiFi
        WiFi.begin(ssid, password);
        // Wait for connection
        while (WiFi.status() != WL_CONNECTED) {
            delay(500);
            Serial.print(".");
        }
        Serial.println();
    
```

(continues on next page)

(continued from previous page)

```

#endif

// set initial water leak detector state as false (default)
WaterLeakDetector.begin();
digitalWrite(ledPin, LOW); // LED OFF

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();

// Check Matter Accessory Commissioning state, which may change during execution.
of loop()
if (!Matter.isDeviceCommissioned()) {
    Serial.println("");
    Serial.println("Matter Node is not commissioned yet.");
    Serial.println("Initiate the device discovery in your Matter environment.");
    Serial.println("Commission it to your Matter hub with the manual pairing code.
or QR code");
    Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
str());
    Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
    // waits for Matter Water Leak Detector Commissioning.
    uint32_t timeCount = 0;
    while (!Matter.isDeviceCommissioned()) {
        delay(100);
        if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
            Serial.println("Matter Node not commissioned yet. Waiting for.
commissioning.");
        }
    }
    Serial.println("Matter Node is commissioned and connected to the network.
Ready for use.");
}
}

bool simulatedHWWaterLeakDetector() {
    // Simulated Water Leak Detector
    static bool leakState = false;
    static uint32_t lastTime = 0;

    // Simulate a Water Leak Detector state change every 20 seconds
    if (millis() - lastTime > 20000) {
        leakState = !leakState;
        lastTime = millis();
    }
    return leakState;
}

void loop() {
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    uint32_t time_diff = millis() - button_time_stamp;
    if (button_state && time_diff > debounceTime && digitalRead(buttonPin) == HIGH) {
        button_state = false; // released
        // button is released - toggle Leak State (Not Detected/Detected)
        WaterLeakDetector.setLeak(!WaterLeakDetector.getLeak()); // same as
WaterLeakDetector = !WaterLeakDetector;
}

```

(continues on next page)

(continued from previous page)

```

Serial.printf("User button released. Setting the Water Leak Detector to %s.\r\n",
↳ WaterLeakDetector ? "Detected" : "Not Detected");
// LED will indicate the Water Leak Detector state
if (WaterLeakDetector) {
    digitalWrite(ledPin, HIGH); // LED ON
} else {
    digitalWrite(ledPin, LOW); // LED OFF
}
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to_
↳ decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning Water Leak Detector Matter Accessory. It shall_
↳ be commissioned again.");
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot_
↳ takes a second or so
}

// Simulated Water Leak Detector
WaterLeakDetector.setLeak(simulatedHWWaterLeakDetector());

delay(50);
}

```

MatterWindowCovering

About The MatterWindowCovering class provides a window covering endpoint for Matter networks. This endpoint implements the Matter window covering standard for motorized blinds, shades, and other window coverings with lift and tilt control.

Features: * Lift position and percentage control (0-100%) * Tilt position and percentage control (0-100%) * Multiple window covering types support * Callback support for open, close, lift, tilt, and stop commands * Integration with Apple HomeKit, Amazon Alexa, and Google Home * Matter standard compliance

Supported Window Covering Types: * ROLLERSHADE - Lift support * ROLLERSHADE_2_MOTOR - Lift support * ROLLERSHADE_EXTERIOR - Lift support * ROLLERSHADE_EXTERIOR_2_MOTOR - Lift support * DRAPERY - Lift support * AWNING - Lift support * SHUTTER - Tilt support * BLIND_TILT_ONLY - Tilt support * BLIND_LIFT_AND_TILT - Lift and Tilt support * PROJECTOR_SCREEN - Lift support

Use Cases: * Motorized blinds * Automated shades * Smart window coverings * Projector screens * Awnings and drapes

API Reference

Constructor

MatterWindowCovering Creates a new Matter window covering endpoint.

```
MatterWindowCovering();
```

Initialization

begin Initializes the Matter window covering endpoint with optional initial positions and covering type.

```
bool begin(uint8_t liftPercent = 100, uint8_t tiltPercent = 0, WindowCoveringType_
↳t coveringType = ROLLERSHADE);
```

- `liftPercent` - Initial lift percentage (0-100, default: 100 = fully open)
- `tiltPercent` - Initial tilt percentage (0-100, default: 0)
- `coveringType` - Window covering type (default: ROLLERSHADE). This determines which features (lift, tilt, or both) are enabled.

This function will return `true` if successful, `false` otherwise.

Note: Lift percentage 0 means fully closed, 100 means fully open. Tilt percentage 0 means fully closed, 100 means fully open. The covering type must be specified during initialization to ensure the correct features (lift and/or tilt) are enabled.

end Stops processing Matter window covering events.

```
void end();
```

Lift Position Control

setLiftPosition Sets the window covering lift position.

```
bool setLiftPosition(uint16_t liftPosition);
```

- `liftPosition` - Lift position value

This function will return `true` if successful, `false` otherwise.

getLiftPosition Gets the current lift position.

```
uint16_t getLiftPosition();
```

This function will return the current lift position.

setLiftPercentage Sets the window covering lift position as a percentage. This method updates the `CurrentPositionLiftPercent100ths` attribute, which reflects the device's actual position. The `TargetPositionLiftPercent100ths` attribute is set by Matter commands/apps when a new target is requested.

```
bool setLiftPercentage(uint8_t liftPercent);
```

- `liftPercent` - Lift percentage (0-100, where 0 is fully closed, 100 is fully open)

This function will return `true` if successful, `false` otherwise.

Note: When the device reaches the target position, call `setOperationalState(LIFT, STALL)` to indicate that movement is complete.

getLiftPercentage Gets the current lift percentage.

```
uint8_t getLiftPercentage();
```

This function will return the current lift percentage (0-100).

Tilt Position Control

setTiltPosition Sets the window covering tilt position. Note that tilt is a rotation, not a linear measurement. This method converts the absolute position to percentage using the installed limits.

```
bool setTiltPosition(uint16_t tiltPosition);
```

- tiltPosition - Tilt position value (absolute value for conversion, not a physical unit)

This function will return `true` if successful, `false` otherwise.

getTiltPosition Gets the current tilt position. Note that tilt is a rotation, not a linear measurement.

```
uint16_t getTiltPosition();
```

This function will return the current tilt position (absolute value for conversion, not a physical unit).

setTiltPercentage Sets the window covering tilt position as a percentage. This method updates the `CurrentPositionTiltPercent100ths` attribute, which reflects the device's actual position. The `TargetPositionTiltPercent100ths` attribute is set by Matter commands/apps when a new target is requested.

```
bool setTiltPercentage(uint8_t tiltPercent);
```

- tiltPercent - Tilt percentage (0-100, where 0 is fully closed, 100 is fully open)

This function will return `true` if successful, `false` otherwise.

Note: When the device reaches the target position, call `setOperationalState(TILT, STALL)` to indicate that movement is complete.

getTiltPercentage Gets the current tilt percentage.

```
uint8_t getTiltPercentage();
```

This function will return the current tilt percentage (0-100).

Window Covering Type

setCoveringType Sets the window covering type.

```
bool setCoveringType(WindowCoveringType_t coveringType);
```

- coveringType - Window covering type (see Window Covering Types enum)

This function will return `true` if successful, `false` otherwise.

getCoveringType Gets the current window covering type.

```
WindowCoveringType_t getCoveringType();
```

This function will return the current window covering type.

Installed Limit Control

setInstalledOpenLimitLift Sets the installed open limit for lift (centimeters). This defines the physical position when the window covering is fully open.

```
bool setInstalledOpenLimitLift (uint16_t openLimit);
```

- openLimit - Open limit position (centimeters)

This function will return `true` if successful, `false` otherwise.

getInstalledOpenLimitLift Gets the installed open limit for lift.

```
uint16_t getInstalledOpenLimitLift();
```

This function will return the installed open limit for lift (centimeters).

setInstalledClosedLimitLift Sets the installed closed limit for lift (centimeters). This defines the physical position when the window covering is fully closed.

```
bool setInstalledClosedLimitLift (uint16_t closedLimit);
```

- closedLimit - Closed limit position (centimeters)

This function will return `true` if successful, `false` otherwise.

getInstalledClosedLimitLift Gets the installed closed limit for lift.

```
uint16_t getInstalledClosedLimitLift();
```

This function will return the installed closed limit for lift (centimeters).

setInstalledOpenLimitTilt Sets the installed open limit for tilt (absolute value for conversion, not a physical unit). This is used for converting between absolute position and percentage.

```
bool setInstalledOpenLimitTilt (uint16_t openLimit);
```

- openLimit - Open limit absolute value

This function will return `true` if successful, `false` otherwise.

Note: Tilt is a rotation, not a linear measurement. These limits are used for position conversion only.

getInstalledOpenLimitTilt Gets the installed open limit for tilt.

```
uint16_t getInstalledOpenLimitTilt();
```

This function will return the installed open limit for tilt (absolute value).

setInstalledClosedLimitTilt Sets the installed closed limit for tilt (absolute value for conversion, not a physical unit). This is used for converting between absolute position and percentage.

```
bool setInstalledClosedLimitTilt (uint16_t closedLimit);
```

- closedLimit - Closed limit absolute value

This function will return `true` if successful, `false` otherwise.

Note: Tilt is a rotation, not a linear measurement. These limits are used for position conversion only.

getInstalledClosedLimitTilt Gets the installed closed limit for tilt.

```
uint16_t getInstalledClosedLimitTilt();
```

This function will return the installed closed limit for tilt (absolute value).

Target Position Control

setTargetLiftPercent100ths Sets the target lift position in percent100ths (0-10000, where 0 is fully closed, 10000 is fully open).

```
bool setTargetLiftPercent100ths(uint16_t liftPercent100ths);
```

- liftPercent100ths - Target lift position in percent100ths (0-10000)

This function will return `true` if successful, `false` otherwise.

Note: This sets the target position that the device should move towards. The actual position should be updated using `setLiftPercentage()`.

getTargetLiftPercent100ths Gets the current target lift position in percent100ths.

```
uint16_t getTargetLiftPercent100ths();
```

This function will return the current target lift position in percent100ths (0-10000).

setTargetTiltPercent100ths Sets the target tilt position in percent100ths (0-10000, where 0 is fully closed, 10000 is fully open).

```
bool setTargetTiltPercent100ths(uint16_t tiltPercent100ths);
```

- tiltPercent100ths - Target tilt position in percent100ths (0-10000)

This function will return `true` if successful, `false` otherwise.

Note: This sets the target position that the device should move towards. The actual position should be updated using `setTiltPercentage()`.

getTargetTiltPercent100ths Gets the current target tilt position in percent100ths.

```
uint16_t getTargetTiltPercent100ths();
```

This function will return the current target tilt position in percent100ths (0-10000).

Operational Status Control

setOperationalStatus Sets the full operational status bitmap.

```
bool setOperationalStatus(uint8_t operationalStatus);
```

- operationalStatus - Full operational status bitmap value

This function will return `true` if successful, `false` otherwise.

Note: It is recommended to use `setOperationalState()` to set individual field states instead of setting the full bitmap directly.

getOperationalStatus Gets the full operational status bitmap.

```
uint8_t getOperationalStatus();
```

This function will return the current operational status bitmap value.

setOperationalState Sets the operational state for a specific field (LIFT or TILT). The GLOBAL field is automatically updated based on priority (LIFT > TILT).

```
bool setOperationalState(OperationalStatusField_t field, OperationalState_t state);
```

- **field** - Field to set (LIFT or TILT). GLOBAL cannot be set directly.
- **state** - Operational state (STALL, MOVING_UP_OR_OPEN, or MOVING_DOWN_OR_CLOSE)

This function will return `true` if successful, `false` otherwise.

Note: Only LIFT and TILT fields can be set directly. The GLOBAL field is automatically updated based on the active field (LIFT has priority over TILT).

getOperationalState Gets the operational state for a specific field.

```
OperationalState_t getOperationalState(OperationalStatusField_t field);
```

- **field** - Field to get (GLOBAL, LIFT, or TILT)

This function will return the operational state for the specified field (STALL, MOVING_UP_OR_OPEN, or MOVING_DOWN_OR_CLOSE).

Event Handling The `MatterWindowCovering` class automatically detects Matter commands and calls the appropriate callbacks when registered. There are two types of callbacks:

Target Position Callbacks (triggered when `TargetPosition` attributes change): * `onOpen()` - called when `UpOrOpen` command is received (sets target to 0% = fully open) * `onClose()` - called when `DownOrClose` command is received (sets target to 100% = fully closed) * `onStop()` - called when `StopMotion` command is received (sets target to current position) * `onGoToLiftPercentage()` - called when `TargetPositionLiftPercent100ths` changes (from any command, `setTargetLiftPercent100ths()`, or direct attribute write) * `onGoToTiltPercentage()` - called when `TargetPositionTiltPercent100ths` changes (from any command, `setTargetTiltPercent100ths()`, or direct attribute write)

Current Position Callback (triggered when `CurrentPosition` attributes change): * `onChange()` - called when `CurrentPositionLiftPercent100ths` or `CurrentPositionTiltPercent100ths` change (after `setLiftPercentage()/setTiltPercentage()` are called or when a Matter controller updates these attributes directly)

Important: `onChange()` is **not** automatically called when Matter commands are executed. Commands modify `TargetPosition`, not `CurrentPosition`. To trigger `onChange()`, your `onGoToLiftPercentage()` or `onGoToTiltPercentage()` callback must call `setLiftPercentage()` or `setTiltPercentage()` when the physical device actually moves.

Note: All callbacks are optional. If a specific callback is not registered, only the generic `onGoToLiftPercentage()` or `onGoToTiltPercentage()` callbacks will be called (if registered).

onOpen Sets a callback function to be called when the `UpOrOpen` command is received from a Matter controller. This command sets the target position to 0% (fully open).

```
void onOpen(EndPointOpenCB onChangeCB);
```

- **onChangeCB** - Function to call when `UpOrOpen` command is received

The callback signature is:

```
bool onChangeCallback();
```

onClose Sets a callback function to be called when the `DownOrClose` command is received from a Matter controller. This command sets the target position to 100% (fully closed).

```
void onClose(EndPointCloseCB onChangeCB);
```

- `onChangeCB` - Function to call when `DownOrClose` command is received

The callback signature is:

```
bool onChangeCallback();
```

onGoToLiftPercentage Sets a callback function to be called when `TargetPositionLiftPercent100ths` changes. This is triggered by: * Matter commands: `UpOrOpen`, `DownOrClose`, `StopMotion`, `GoToLiftPercentage` * Calling `setTargetLiftPercent100ths()` * Direct attribute writes to `TargetPositionLiftPercent100ths`

This callback is always called when the target lift position changes, regardless of which command or method was used to change it.

Note: This callback receives the **target** position. To update the **current** position (which triggers `onChange()`), call `setLiftPercentage()` when the physical device actually moves.

```
void onGoToLiftPercentage(EndPointLiftCB onChangeCB);
```

- `onChangeCB` - Function to call when target lift percentage changes

The callback signature is:

```
bool onChangeCallback(uint8_t liftPercent);
```

- `liftPercent` - Target lift percentage (0-100, where 0 is fully closed, 100 is fully open)

onGoToTiltPercentage Sets a callback function to be called when `TargetPositionTiltPercent100ths` changes. This is triggered by: * Matter commands: `UpOrOpen`, `DownOrClose`, `StopMotion`, `GoToTiltPercentage` * Calling `setTargetTiltPercent100ths()` * Direct attribute writes to `TargetPositionTiltPercent100ths`

This callback is always called when the target tilt position changes, regardless of which command or method was used to change it.

Note: This callback receives the **target** position. To update the **current** position (which triggers `onChange()`), call `setTiltPercentage()` when the physical device actually moves.

```
void onGoToTiltPercentage(EndPointTiltCB onChangeCB);
```

- `onChangeCB` - Function to call when target tilt percentage changes

The callback signature is:

```
bool onChangeCallback(uint8_t tiltPercent);
```

- `tiltPercent` - Target tilt percentage (0-100, where 0 is fully closed, 100 is fully open)

onStop Sets a callback function to be called when the `StopMotion` command is received from a Matter controller. This command sets the target position to the current position, effectively stopping any movement.

```
void onStop(EndPointStopCB onChangeCB);
```

- onChangeCB - Function to call when StopMotion command is received

The callback signature is:

```
bool onChangeCallback();
```

onChange Sets a callback function to be called when CurrentPositionLiftPercent100ths or CurrentPositionTiltPercent100ths attributes change. This is different from onGoToLiftPercentage() and onGoToTiltPercentage(), which are called when TargetPosition attributes change.

When ``onChange()`` is called: * When CurrentPositionLiftPercent100ths changes (after setLiftPercentage() is called or when a Matter controller updates this attribute directly) * When CurrentPositionTiltPercent100ths changes (after setTiltPercentage() is called or when a Matter controller updates this attribute directly)

Important: onChange() is **not** automatically called when Matter commands are executed. Commands modify TargetPosition attributes, which trigger onGoToLiftPercentage() or onGoToTiltPercentage() callbacks instead. To trigger onChange() after a command, your onGoToLiftPercentage() or onGoToTiltPercentage() callback must call setLiftPercentage() or setTiltPercentage() to update the CurrentPosition attributes when the physical device actually moves.

```
void onChange(EndPointCB onChangeCB);
```

- onChangeCB - Function to call when current position attributes change

The callback signature is:

```
bool onChangeCallback(uint8_t liftPercent, uint8_t tiltPercent);
```

- liftPercent - Current lift percentage (0-100)
- tiltPercent - Current tilt percentage (0-100)

updateAccessory Updates the state of the window covering using the current Matter internal state.

```
void updateAccessory();
```

This function will call the registered callback with the current state.

Example

Window Covering

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
//
// Matter Manager
```

(continues on next page)

(continued from previous page)

```

#include <Arduino.h>
#include <Matter.h>
#if !CONFIG_ENABLE_CHIPOBLE
// if the device can be commissioned using BLE, WiFi is not used - save flash space
#include <WiFi.h>
#endif
#include <Preferences.h>

// List of Matter Endpoints for this Node
// Window Covering Endpoint
MatterWindowCovering WindowBlinds;

// CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter_
↪Network
#if !CONFIG_ENABLE_CHIPOBLE
// WiFi is manually set and started
const char *ssid = "your-ssid";           // Change this to your WiFi SSID
const char *password = "your-password";   // Change this to your WiFi password
#endif

// it will keep last Lift & Tilt state stored, using Preferences
Preferences matterPref;
const char *liftPercentPrefKey = "LiftPercent";
const char *tiltPercentPrefKey = "TiltPercent";

// set your board USER BUTTON pin here
const uint8_t buttonPin = BOOT_PIN; // Set your pin here. Using BOOT Button.

// Button control
uint32_t button_time_stamp = 0;           // debouncing control
bool button_state = false;                // false = released | true = pressed
const uint32_t debounceTime = 250;        // button debouncing time (ms)
const uint32_t decommissioningTimeout = 5000; // keep the button pressed for 5s,
↪or longer, to decommission

// Window covering limits
// Lift limits in centimeters (physical position)
const uint16_t MAX_LIFT = 200; // Maximum lift position (fully open)
const uint16_t MIN_LIFT = 0;   // Minimum lift position (fully closed)

// Tilt limits (absolute values for conversion, not physical units)
// Tilt is a rotation, not a linear measurement
const uint16_t MAX_TILT = 90; // Maximum tilt absolute value
const uint16_t MIN_TILT = 0;  // Minimum tilt absolute value

// Current window covering state
// These will be initialized in setup() based on installed limits and saved_
↪percentages
uint16_t currentLift = 0; // Lift position in cm
uint8_t currentLiftPercent = 100;
uint8_t currentTiltPercent = 0; // Tilt rotation percentage (0-100%)

// Visualize window covering position using RGB LED
// Lift percentage controls brightness (0% = off, 100% = full brightness)
#ifdef RGB_BUILTIN
const uint8_t ledPin = RGB_BUILTIN;
#else
const uint8_t ledPin = 2; // Set your pin here if your board has not defined RGB_
↪BUILTIN
#warning "Do not forget to set the RGB LED pin"
#endif

```

(continues on next page)

(continued from previous page)

```

void visualizeWindowBlinds(uint8_t liftPercent, uint8_t tiltPercent) {
#ifdef RGB_BUILTIN
    // Use RGB LED to visualize lift position (brightness) and tilt (color shift)
    float brightness = (float)liftPercent / 100.0; // 0.0 to 1.0
    // Tilt affects color: 0% = red, 100% = blue
    uint8_t red = (uint8_t)(map(tiltPercent, 0, 100, 255, 0) * brightness);
    uint8_t blue = (uint8_t)(map(tiltPercent, 0, 100, 0, 255) * brightness);
    uint8_t green = 0;
    rgbLedWrite(ledPin, red, green, blue);
#else
    // For non-RGB boards, just use brightness
    uint8_t brightnessValue = map(liftPercent, 0, 100, 0, 255);
    analogWrite(ledPin, brightnessValue);
#endif
}

// Window Covering Callbacks
bool fullOpen() {
    // This is where you would trigger your motor to go to full open state
    // For simulation, we update instantly
    uint16_t openLimit = WindowBlinds.getInstalledOpenLimitLift();
    currentLift = openLimit;
    currentLiftPercent = 100;
    Serial.printf("Opening window covering to full open (position: %u cm)\r\n",
↳currentLift);

    // Update CurrentPosition to reflect actual position (setLiftPercentage now only
↳updates CurrentPosition)
    WindowBlinds.setLiftPercentage(currentLiftPercent);

    // Set operational status to STALL when movement is complete
    WindowBlinds.setOperationalState(MatterWindowCovering::LIFT,
↳MatterWindowCovering::STALL);

    // Store state
    matterPref.putUChar(liftPercentPrefKey, currentLiftPercent);

    return true;
}

bool fullClose() {
    // This is where you would trigger your motor to go to full close state
    // For simulation, we update instantly
    uint16_t closedLimit = WindowBlinds.getInstalledClosedLimitLift();
    currentLift = closedLimit;
    currentLiftPercent = 0;
    Serial.printf("Closing window covering to full close (position: %u cm)\r\n",
↳currentLift);

    // Update CurrentPosition to reflect actual position (setLiftPercentage now only
↳updates CurrentPosition)
    WindowBlinds.setLiftPercentage(currentLiftPercent);

    // Set operational status to STALL when movement is complete
    WindowBlinds.setOperationalState(MatterWindowCovering::LIFT,
↳MatterWindowCovering::STALL);

    // Store state
    matterPref.putUChar(liftPercentPrefKey, currentLiftPercent);

```

(continues on next page)

(continued from previous page)

```

    return true;
}

bool goToLiftPercentage(uint8_t liftPercent) {
    // update Lift operational state
    if (liftPercent > currentLiftPercent) {
        // Set operational status to OPEN
        WindowBlinds.setOperationalState(MatterWindowCovering::LIFT,
↪MatterWindowCovering::MOVING_UP_OR_OPEN);
    }
    if (liftPercent < currentLiftPercent) {
        // Set operational status to CLOSE
        WindowBlinds.setOperationalState(MatterWindowCovering::LIFT,
↪MatterWindowCovering::MOVING_DOWN_OR_CLOSE);
    }

    // This is where you would trigger your motor to go towards liftPercent
    // For simulation, we update instantly
    // Calculate absolute position based on installed limits
    uint16_t openLimit = WindowBlinds.getInstalledOpenLimitLift();
    uint16_t closedLimit = WindowBlinds.getInstalledClosedLimitLift();

    // Linear interpolation: 0% = openLimit, 100% = closedLimit
    if (openLimit < closedLimit) {
        currentLift = openLimit + ((closedLimit - openLimit) * liftPercent) / 100;
    } else {
        currentLift = openLimit - ((openLimit - closedLimit) * liftPercent) / 100;
    }
    currentLiftPercent = liftPercent;
    Serial.printf("Moving lift to %u%% (position: %u cm)\r\n", currentLiftPercent,
↪currentLift);

    // Update CurrentPosition to reflect actual position (setLiftPercentage now only
↪updates CurrentPosition)
    WindowBlinds.setLiftPercentage(currentLiftPercent);

    // Set operational status to STALL when movement is complete
    WindowBlinds.setOperationalState(MatterWindowCovering::LIFT,
↪MatterWindowCovering::STALL);

    // Store state
    matterPref.putUChar(liftPercentPrefKey, currentLiftPercent);

    return true;
}

bool goToTiltPercentage(uint8_t tiltPercent) {
    // update Tilt operational state
    if (tiltPercent < currentTiltPercent) {
        // Set operational status to OPEN
        WindowBlinds.setOperationalState(MatterWindowCovering::TILT,
↪MatterWindowCovering::MOVING_UP_OR_OPEN);
    }
    if (tiltPercent > currentTiltPercent) {
        // Set operational status to CLOSE
        WindowBlinds.setOperationalState(MatterWindowCovering::TILT,
↪MatterWindowCovering::MOVING_DOWN_OR_CLOSE);
    }

    // This is where you would trigger your motor to rotate the shade to tiltPercent
    // For simulation, we update instantly

```

(continues on next page)

(continued from previous page)

```

currentTiltPercent = tiltPercent;
Serial.printf("Rotating tilt to %u%%\r\n", currentTiltPercent);

// Update CurrentPosition to reflect actual position
WindowBlinds.setTiltPercentage(currentTiltPercent);

// Set operational status to STALL when movement is complete
WindowBlinds.setOperationalState(MatterWindowCovering::TILT, ␣
↪MatterWindowCovering::STALL);

// Store state
matterPref.putUChar(tiltPercentPrefKey, currentTiltPercent);

return true;
}

bool stopMotor() {
    // Motor can be stopped while moving cover toward current target
    Serial.println("Stopping window covering motor");

    // Update CurrentPosition to reflect actual position when stopped
    // (setLiftPercentage and setTiltPercentage now only update CurrentPosition)
    WindowBlinds.setLiftPercentage(currentLiftPercent);
    WindowBlinds.setTiltPercentage(currentTiltPercent);

    // Set operational status to STALL for both lift and tilt
    WindowBlinds.setOperationalState(MatterWindowCovering::LIFT, ␣
↪MatterWindowCovering::STALL);
    WindowBlinds.setOperationalState(MatterWindowCovering::TILT, ␣
↪MatterWindowCovering::STALL);

    return true;
}

void setup() {
    // Initialize the USER BUTTON (Boot button) GPIO
    pinMode(buttonPin, INPUT_PULLUP);
    // Initialize the RGB LED GPIO
    pinMode(ledPin, OUTPUT);
    digitalWrite(ledPin, LOW);

    Serial.begin(115200);

    // CONFIG_ENABLE_CHIPOBLE is enabled when BLE is used to commission the Matter␣
↪Network
#ifdef CONFIG_ENABLE_CHIPOBLE
    // We start by connecting to a WiFi network
    Serial.print("Connecting to ");
    Serial.println(ssid);
    // Manually connect to WiFi
    WiFi.begin(ssid, password);
    // Wait for connection
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\r\nWiFi connected");
    Serial.println("IP address: ");
    Serial.println(WiFi.localIP());
    delay(500);
#endif
}

```

(continues on next page)

(continued from previous page)

```

// Initialize Matter EndPoint
matterPref.begin("MatterPrefs", false);
// default lift percentage is 100% (fully open) if not stored before
uint8_t lastLiftPercent = matterPref.getUChar(liftPercentPrefKey, 100);
// default tilt percentage is 0% if not stored before
uint8_t lastTiltPercent = matterPref.getUChar(tiltPercentPrefKey, 0);

// Initialize window covering with BLIND_LIFT_AND_TILT type
WindowBlinds.begin(lastLiftPercent, lastTiltPercent, MatterWindowCovering::BLIND_
↪ LIFT_AND_TILT);

// Configure installed limits for lift and tilt
WindowBlinds.setInstalledOpenLimitLift(MIN_LIFT);
WindowBlinds.setInstalledClosedLimitLift(MAX_LIFT);
WindowBlinds.setInstalledOpenLimitTilt(MIN_TILT);
WindowBlinds.setInstalledClosedLimitTilt(MAX_TILT);

// Initialize current positions based on percentages and installed limits
uint16_t openLimitLift = WindowBlinds.getInstalledOpenLimitLift();
uint16_t closedLimitLift = WindowBlinds.getInstalledClosedLimitLift();
currentLiftPercent = lastLiftPercent;
if (openLimitLift < closedLimitLift) {
    currentLift = openLimitLift + ((closedLimitLift - openLimitLift) *
↪ lastLiftPercent) / 100;
} else {
    currentLift = openLimitLift - ((openLimitLift - closedLimitLift) *
↪ lastLiftPercent) / 100;
}

currentTiltPercent = lastTiltPercent;

Serial.printf(
    "Window Covering limits configured: Lift [%u-%u cm], Tilt [%u-%u]\r\n",
↪ WindowBlinds.getInstalledOpenLimitLift(),
    WindowBlinds.getInstalledClosedLimitLift(), WindowBlinds.
↪ getInstalledOpenLimitTilt(), WindowBlinds.getInstalledClosedLimitTilt()
);
Serial.printf("Initial positions: Lift=%u cm (%u%%), Tilt=%u%%\r\n", currentLift,
↪ currentLiftPercent, currentTiltPercent);

// Set callback functions
WindowBlinds.onOpen(fullOpen);
WindowBlinds.onClose(fullClose);
WindowBlinds.onGoToLiftPercentage(goToLiftPercentage);
WindowBlinds.onGoToTiltPercentage(goToTiltPercentage);
WindowBlinds.onStop(stopMotor);

// Generic callback for Lift or Tilt change
WindowBlinds.onChange([](uint8_t liftPercent, uint8_t tiltPercent) {
    Serial.printf("Window Covering changed: Lift=%u%%, Tilt=%u%%\r\n", liftPercent,
↪ tiltPercent);
    visualizeWindowBlinds(liftPercent, tiltPercent);
    return true;
});

// Matter beginning - Last step, after all EndPoints are initialized
Matter.begin();
// This may be a restart of a already commissioned Matter accessory
if (Matter.isDeviceCommissioned()) {
    Serial.println("Matter Node is commissioned and connected to the network.
↪

```

(continues on next page)

(continued from previous page)

```

    ↪Ready for use.");
    Serial.printf("Initial state: Lift=%u%%, Tilt=%u%%\r\n", WindowBlinds.
    ↪getLiftPercentage(), WindowBlinds.getTiltPercentage());
    // Update visualization based on initial state
    visualizeWindowBlinds(WindowBlinds.getLiftPercentage(), WindowBlinds.
    ↪getTiltPercentage());
    }
}

void loop() {
    // Check Matter Window Covering Commissioning state, which may change during
    ↪execution of loop()
    if (!Matter.isDeviceCommissioned()) {
        Serial.println("");
        Serial.println("Matter Node is not commissioned yet.");
        Serial.println("Initiate the device discovery in your Matter environment.");
        Serial.println("Commission it to your Matter hub with the manual pairing code
    ↪or QR code");
        Serial.printf("Manual pairing code: %s\r\n", Matter.getManualPairingCode().c_
    ↪str());
        Serial.printf("QR code URL: %s\r\n", Matter.getOnboardingQRCodeUrl().c_str());
        // waits for Matter Window Covering Commissioning.
        uint32_t timeCount = 0;
        while (!Matter.isDeviceCommissioned()) {
            delay(100);
            if ((timeCount++ % 50) == 0) { // 50*100ms = 5 sec
                Serial.println("Matter Node not commissioned yet. Waiting for
    ↪commissioning.");
            }
        }
        Serial.printf("Initial state: Lift=%u%%, Tilt=%u%%\r\n", WindowBlinds.
    ↪getLiftPercentage(), WindowBlinds.getTiltPercentage());
        // Update visualization based on initial state
        visualizeWindowBlinds(WindowBlinds.getLiftPercentage(), WindowBlinds.
    ↪getTiltPercentage());
        Serial.println("Matter Node is commissioned and connected to the network.
    ↪Ready for use.");
    }

    // A button is also used to control the window covering
    // Check if the button has been pressed
    if (digitalRead(buttonPin) == LOW && !button_state) {
        // deals with button debouncing
        button_time_stamp = millis(); // record the time while the button is pressed.
        button_state = true; // pressed.
    }

    // Onboard User Button is used to manually change lift percentage or to
    ↪decommission
    uint32_t time_diff = millis() - button_time_stamp;
    if (digitalRead(buttonPin) == HIGH && button_state && time_diff > debounceTime) {
        // Button is released - cycle lift percentage by 20%
        button_state = false; // released
        uint8_t targetLiftPercent = currentLiftPercent;
        // go to the closest next 20% or move 20% more
        if ((targetLiftPercent % 20) != 0) {
            targetLiftPercent = ((targetLiftPercent / 20) + 1) * 20;
        } else {
            targetLiftPercent += 20;
        }
        if (targetLiftPercent > 100) {

```

(continues on next page)

(continued from previous page)

```

        targetLiftPercent = 0;
    }
    Serial.printf("User button released. Setting lift to %u%%\r\n",
↪targetLiftPercent);
    WindowBlinds.setTargetLiftPercent100ths(targetLiftPercent * 100);
}

// Onboard User Button is kept pressed for longer than 5 seconds in order to
↪decommission matter node
if (button_state && time_diff > decommissioningTimeout) {
    Serial.println("Decommissioning the Window Covering Matter Accessory. It shall
↪be commissioned again.");
    WindowBlinds.setLiftPercentage(0); // close the covering
    Matter.decommission();
    button_time_stamp = millis(); // avoid running decommissioning again, reboot
↪takes a second or so
}
}
}

```

Specific endpoint classes The library provides specialized endpoint classes for different device types. Each endpoint type includes specific clusters and functionality relevant to that device category.

Lighting Endpoints:

- **MatterOnOffLight:** Simple on/off light control
- **MatterDimmableLight:** Light with brightness control
- **MatterColorTemperatureLight:** Light with color temperature control
- **MatterColorLight:** Light with RGB color control (HSV color model)
- **MatterEnhancedColorLight:** Enhanced color light with color temperature and brightness control

Sensor Endpoints:

- **MatterTemperatureSensor:** Temperature sensor (read-only)
- **MatterHumiditySensor:** Humidity sensor (read-only)
- **MatterPressureSensor:** Pressure sensor (read-only)
- **MatterContactSensor:** Contact sensor (open/closed state)
- **MatterWaterLeakDetector:** Water leak detector (detected/not detected state)
- **MatterWaterFreezeDetector:** Water freeze detector (detected/not detected state)
- **MatterRainSensor:** Rain sensor (detected/not detected state)
- **MatterOccupancySensor:** Occupancy sensor (occupied/unoccupied state)
- **MatterLightSensor:** Light sensor (read-only)

Control Endpoints:

- **MatterTemperatureControlledCabinet:** Temperature controlled cabinet (setpoint control with min/max limits)
- **MatterFan:** Fan with speed and mode control
- **MatterThermostat:** Thermostat with temperature control and setpoints
- **MatterOnOffPlugin:** On/off plugin unit (power outlet/relay)
- **MatterDimmablePlugin:** Dimmable plugin unit (power outlet/relay with brightness control)
- **MatterGenericSwitch:** Generic switch endpoint (smart button)
- **MatterWindowCovering:** Window covering with lift and tilt control (blinds, shades)

Matter Examples

The Matter library includes a comprehensive set of examples demonstrating various device types and use cases. All examples are available in the [ESP Arduino GitHub repository](#).

Basic Examples:

- **Matter Minimum** - The smallest code required to create a Matter-compatible device. Ideal starting point for understanding Matter basics. [View Matter Minimum code on GitHub](#)
- **Matter Status** - Demonstrates how to check enabled Matter features and connectivity status. Implements a basic on/off light and periodically reports capability and connection status. [View Matter Status code on GitHub](#)
- **Matter Events** - Shows how to monitor and handle Matter events. Provides a comprehensive view of all Matter events during device operation. [View Matter Events code on GitHub](#)
- **Matter Commission Test** - Tests Matter commissioning functionality with automatic decommissioning after a 30-second delay for continuous testing cycles. [View Matter Commission Test code on GitHub](#)

Lighting Examples:

- **Matter On/Off Light** - Creates a Matter-compatible on/off light device with commissioning, device control via smart home ecosystems, and manual control using a physical button with state persistence. [View Matter On/Off Light code on GitHub](#)
- **Matter Dimmable Light** - Creates a Matter-compatible dimmable light device with brightness control. [View Matter Dimmable Light code on GitHub](#)
- **Matter Color Temperature Light** - Creates a Matter-compatible color temperature light device with adjustable color temperature control. [View Matter Color Temperature Light code on GitHub](#)
- **Matter Color Light** - Creates a Matter-compatible color light device with RGB color control (HSV color model). [View Matter Color Light code on GitHub](#)
- **Matter Enhanced Color Light** - Creates a Matter-compatible enhanced color light with color temperature and brightness control. [View Matter Enhanced Color Light code on GitHub](#)
- **Matter Composed Lights** - Creates a Matter node with multiple light endpoints (On/Off Light, Dimmable Light, and Color Light) in a single node. [View Matter Composed Lights code on GitHub](#)
- **Matter On Identify** - Implements the Matter Identify cluster callback for an on/off light device, making the LED blink when the device is identified from a Matter app. [View Matter On Identify code on GitHub](#)

Sensor Examples:

- **Matter Temperature Sensor** - Creates a Matter-compatible temperature sensor device with sensor data reporting to smart home ecosystems. [View Matter Temperature Sensor code on GitHub](#)
- **Matter Humidity Sensor** - Creates a Matter-compatible humidity sensor device with sensor data reporting. [View Matter Humidity Sensor code on GitHub](#)
- **Matter Pressure Sensor** - Creates a Matter-compatible pressure sensor device with automatic simulation of pressure readings. [View Matter Pressure Sensor code on GitHub](#)
- **Matter Contact Sensor** - Creates a Matter-compatible contact sensor device (open/closed state). [View Matter Contact Sensor code on GitHub](#)
- **Matter Occupancy Sensor** - Creates a Matter-compatible occupancy sensor device with automatic simulation of occupancy state changes. [View Matter Occupancy Sensor code on GitHub](#)
- **Matter Occupancy Sensor with HoldTime** - Creates a Matter-compatible occupancy sensor device with HoldTime functionality, automatic simulation of occupancy state changes, HoldTime configuration with persistence across reboots, and HoldTime change callback for real-time updates from Matter controllers. [View Matter Occupancy Sensor with HoldTime code on GitHub](#)
- **Matter Water Leak Detector** - Creates a Matter-compatible water leak detector device with automatic simulation of water leak detection state changes. [View Matter Water Leak Detector code on GitHub](#)
- **Matter Water Freeze Detector** - Creates a Matter-compatible water freeze detector device with automatic simulation of water freeze detection state changes. [View Matter Water Freeze Detector code on GitHub](#)
- **Matter Rain Sensor** - Creates a Matter-compatible rain sensor device with automatic simulation of rain detection state changes. [View Matter Rain Sensor code on GitHub](#)
- **Matter Light Sensor** - Creates a Matter-compatible light (illuminance) sensor device with automatic simulation of illuminance readings. [View Matter Light Sensor code on GitHub](#)

Control Examples:

- **Matter Fan** - Creates a Matter-compatible fan device with speed and mode control. [View Matter Fan code on GitHub](#)
- **Matter Thermostat** - Creates a Matter-compatible thermostat device with temperature setpoint management and simulated heating/cooling systems with automatic temperature regulation. [View Matter Thermostat code on GitHub](#)
- **Matter Temperature Controlled Cabinet** - Creates a Matter-compatible temperature controlled cabinet device with precise temperature setpoint control with min/max limits (temperature_number mode). [View Matter](#)

[Temperature Controlled Cabinet code on GitHub](#)

- **Matter Temperature Controlled Cabinet Levels** - Creates a Matter-compatible temperature controlled cabinet device using predefined temperature levels (temperature_level mode). [View Matter Temperature Controlled Cabinet Levels code on GitHub](#)
- **Matter On/Off Plugin** - Creates a Matter-compatible on/off plugin unit (power relay) device with state persistence for power control applications. [View Matter On/Off Plugin code on GitHub](#)
- **Matter Dimmable Plugin** - Creates a Matter-compatible dimmable plugin unit (power outlet with level control) device with state persistence for dimmable power control applications. [View Matter Dimmable Plugin code on GitHub](#)
- **Matter Smart Button** - Creates a Matter-compatible smart button (generic switch) device that sends button click events to smart home ecosystems and triggers automations. [View Matter Smart Button code on GitHub](#)
- **Matter Window Covering** - Creates a Matter-compatible window covering device with lift and tilt control (blinds, shades) with manual control using a physical button. [View Matter Window Covering code on GitHub](#)
- **Matter Simple Blinds** - A minimal example that only controls lift percentage using a single onGoToLiftPercentage() callback. [View Matter Simple Blinds code on GitHub](#)

Advanced Examples:

- **Matter Lambda Single Callback Many Endpoints** - Demonstrates how to create multiple Matter endpoints in a single node using a shared lambda function callback with capture for efficient callback handling. [View Matter Lambda Single Callback Many Endpoints code on GitHub](#)

Common Problems and Issues

Troubleshooting

Common Issues

Device won't commission

- Ensure the Matter hub is in pairing mode
- Check that Wi-Fi or Thread connectivity is properly configured
- Verify the QR code or pairing code is correct
- For ESP32/ESP32-S2, ensure Wi-Fi credentials are set in the code

Commissioning fails

- Try factory resetting the device by calling `Matter.decommission()`
- Erase flash memory: Arduino IDE Menu->Tools->Erase All Flash Before Sketch Upload: "Enabled"
- Or use esptool: `esptool.py --port <PORT> erase_flash`

Wi-Fi connection issues

- Verify Wi-Fi credentials (SSID and password) are correct
- Check that the device is within range of the Wi-Fi router
- Ensure the Wi-Fi network is 2.4 GHz (Matter requires 2.4 GHz Wi-Fi)

Thread connection issues

- Verify Thread border router is properly configured
- Check that Thread network credentials are correct
- Ensure device supports Thread (ESP32-H2, ESP32-C6 with Thread enabled)

Device not responding

- Check Serial Monitor for error messages (115200 baud)
- Verify endpoint initialization with `begin()` method
- Ensure `Matter.begin()` is called after all endpoints are initialized

Callbacks not firing

- Verify callback functions are registered before commissioning
- Check that callback functions are properly defined
- Ensure endpoint is properly initialized with `begin()`

Factory Reset If you have problems with commissioning or device connectivity, you can try to factory reset the device. This will erase all the Matter network settings and act as a brand new device.


```
Matter.decommission();
```

This will reset the device and it will need to be commissioned again.

Event Monitoring For debugging and monitoring Matter events, you can set up an event callback:

```
Matter.onEvent([](matterEvent_t event, const chip::DeviceLayer::ChipDeviceEvent_
↳ *eventData) {
    Serial.printf("Matter Event: 0x%04X\r\n", event);
    // Handle specific events
    switch(event) {
        case MATTER_COMMISSIONING_COMPLETE:
            Serial.println("Device commissioned!");
            break;
        case MATTER_WIFI_CONNECTIVITY_CHANGE:
            Serial.println("Wi-Fi connectivity changed");
            break;
        // ... handle other events
    }
});
```

This allows you to monitor commissioning progress, connectivity changes, and other Matter events in real-time.

2.5 OpenThread APIs

2.5.1 OpenThread

About

The OpenThread library provides support for creating Thread network devices using ESP32 SoCs with IEEE 802.15.4 radio support. The library offers two different programming interfaces for interacting with the OpenThread stack:

- **Stream-based CLI enhanced with Helper Functions API:** Command-line interface helper functions that send OpenThread CLI commands and parse responses.
- **Classes API:** Object-oriented classes that directly call OpenThread API functions.

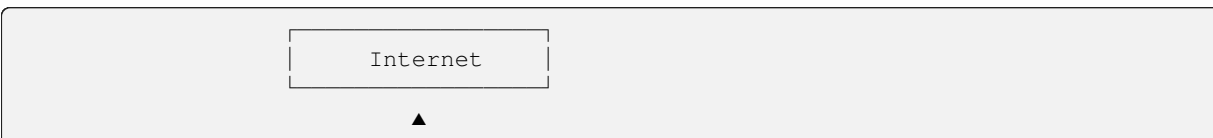
The OpenThread library is built on top of [ESP OpenThread](#) and provides a high-level Arduino-style interface for creating Thread devices.

Thread Protocol Overview Thread is an IPv6-based, low-power wireless mesh networking protocol designed for smart home and IoT applications. It provides secure, reliable, and scalable connectivity for battery-powered devices.

Key Features:

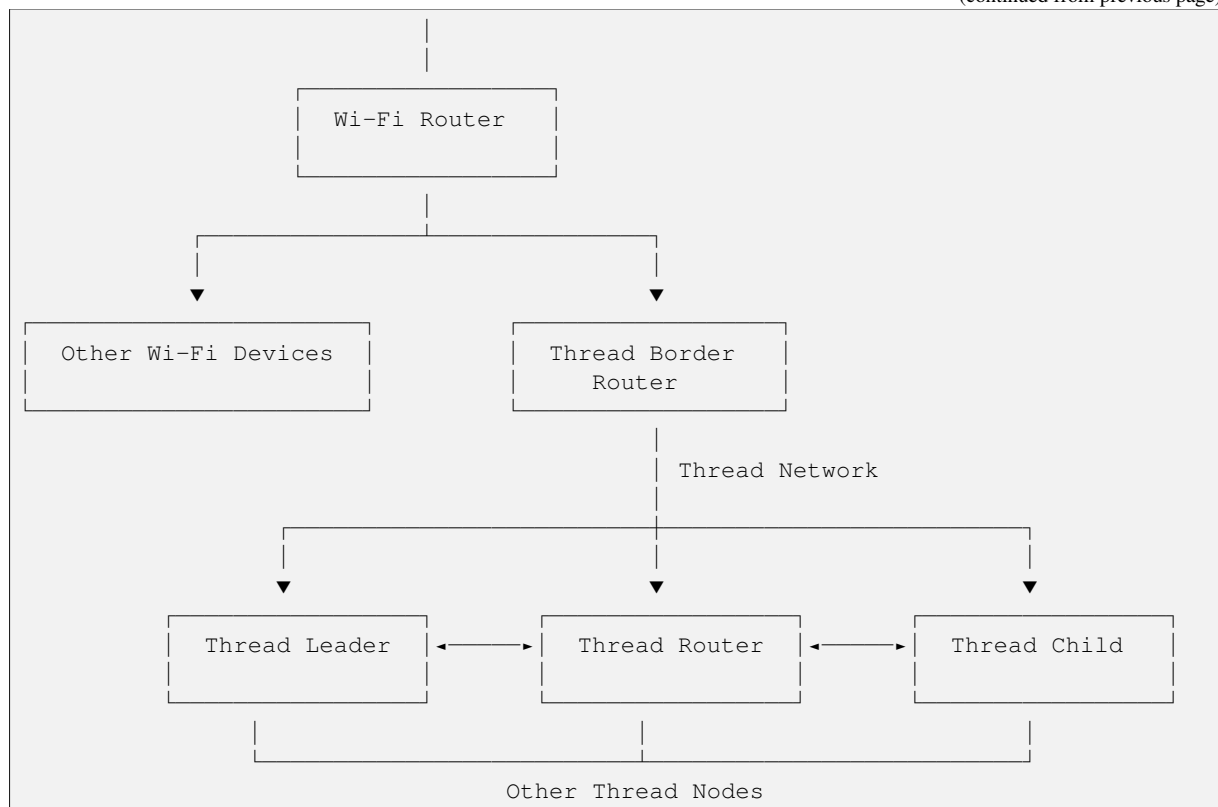
- **IPv6-based:** Native IPv6 addressing and routing.
- **Mesh Networking:** Self-healing mesh topology with automatic routing.
- **Low Power:** Optimized for battery-operated devices.
- **Security:** Built-in security features including encryption and authentication.
- **Scalability:** Supports up to 250+ devices per network.
- **Reliability:** Automatic route discovery and self-healing capabilities.

Thread Network Topology



(continues on next page)

(continued from previous page)

**Thread Device Roles:**

- **Leader:** Manages the Thread network, assigns router IDs, and maintains network state. This device should be powered by a wall adapter.
- **Router:** Extends network range, routes messages, maintains network topology. This device should be powered by a wall adapter.
- **Child:** End device that can sleep for extended periods (battery-powered devices). It can be powered by a battery or a wall adapter.
- **Detached:** Device not currently participating in a Thread network.
- **Disabled:** The Thread stack and interface are disabled.

Other Thread Network Devices:

- **Thread Border Router:** A device that connects a Thread network to other IP-based networks in the external world. The Thread Border Router is connected to both the Thread network and external IP networks (like a Wi-Fi router), enabling Thread devices to communicate with devices on other networks and the Internet. A Border Router can be implemented on a device with any Thread device role (Leader, Router, or Child). It can also act as a gateway to other protocols such as MQTT or Zigbee.

OpenThread Library Structure**The library provides two main programming interfaces:**

- **CLI Helper Functions API:** Functions that interact with OpenThread through the CLI interface * `otGetRespCmd()`: Execute CLI command and get response. * `otExecCommand()`: Execute CLI command with arguments. * `otPrintRespCLI()`: Execute CLI command and print response to Stream. * `OpenThreadCLI`: Stream-based CLI interface class.
- **Classes API:** Object-oriented classes that directly call OpenThread API functions * `OpenThread`: Main class for managing Thread network operations (includes the Joiner / Commissioner roles). * `DataSet`: Class for managing Thread operational datasets. * `OTThreadUDP`: Arduino UDP-compatible class for sending and receiving IPv6 UDP datagrams over the Thread mesh, backed by the raw `otUdpSocket` API (no lwIP).

OpenThread Class The `OpenThread` class is the main entry point for Thread operations using the Classes API. It provides direct access to OpenThread API functions for managing the Thread network.

- **Network Management:** Starting, stopping, and managing the Thread network.
- **Device Role Management:** Getting and monitoring device role (Leader, Router, Child, Detached, Disabled).
- **Dataset Management:** Setting and getting operational dataset parameters (offline via `DataSet + commitDataSet` or live via per-parameter setters).
- **Joiner Role:** Synchronously attach a brand-new device to a Thread network using only a PSKd (Pre-Shared Key for Device).
- **Commissioner Role:** Authorize remote Joiners on the network side using a PSKd.
- **Address Management:** Getting mesh-local addresses, RLOC, and multicast addresses.
- **Network Information:** Getting network name, channel, PAN ID, EUI-64, Thread version, TX power, and other network parameters.

The `OpenThread` class is implemented as a singleton, meaning there's only one instance available globally. You access it directly as `OThread` without creating an instance.

OpenThread Class

About The `OpenThread` class provides direct access to OpenThread API functions for managing Thread network operations. This is the **Classes API** approach, which offers object-oriented methods that directly call OpenThread API functions.

Key Features: * Direct OpenThread API access. * Network management. (start, stop, interface control) * Dataset management. * Live setters that act directly on the running instance. (no `DataSet` required) * Thread Joiner role. (synchronous commissioning of a new device using a PSKd) * Thread Commissioner role. (let new devices attach with just a PSKd) * Address management with caching. * Network information retrieval. * Device role monitoring.

Use Cases: * Thread network configuration and management. * Direct control over Thread operations * Programmatic network setup. * Commissioning new devices over-the-air with a PSKd. * Address and routing information access.

API Reference

Initialization

begin Initializes the OpenThread stack.

```
static void begin(bool OThreadAutoStart = true);
```

- `OThreadAutoStart` - If `true`, automatically starts Thread with default dataset from NVS or ESP-IDF settings (default: `true`)

This function initializes the OpenThread stack and creates the `OpenThread` task. If `OThreadAutoStart` is `true`, it will attempt to start Thread using the active dataset from NVS or ESP-IDF default settings.

`begin()` returns only after the worker task has finished initializing the OpenThread stack and reloaded any persisted Active Operational Dataset from NVS. After it returns successfully, APIs such as `hasActiveDataset()` can reliably query the stack state without an additional delay loop.

Note: This is a static function and should be called before creating an `OpenThread` instance.

end Stops and cleans up the OpenThread stack.

```
static void end();
```

This function stops the OpenThread task and cleans up all OpenThread resources. It should be called when you no longer need the OpenThread stack.

Note: This is a static function.

Network Control

start Starts the Thread network.

```
void start();
```

This function enables the Thread network. The device will attempt to join or form a Thread network based on the active dataset.

Note: The network interface must be brought up separately using `networkInterfaceUp()`.

stop Stops the Thread network.

```
void stop();
```

This function disables the Thread network. The device will leave the Thread network and stop participating in Thread operations.

networkInterfaceUp Brings the Thread network interface up.

```
void networkInterfaceUp();
```

This function enables the Thread IPv6 interface (equivalent to CLI command `ifconfig up`). The device will be able to send and receive IPv6 packets over the Thread network.

networkInterfaceDown Brings the Thread network interface down.

```
void networkInterfaceDown();
```

This function disables the Thread IPv6 interface (equivalent to CLI command `ifconfig down`). The device will stop sending and receiving IPv6 packets.

Dataset Management

commitDataSet Commits an operational dataset to the Thread network.

```
void commitDataSet(const DataSet &dataset);
```

- `dataset` - The `DataSet` object containing the operational dataset parameters

This function sets the active operational dataset for the Thread network. The dataset must be properly configured before committing.

Example:

```
DataSet dataset;  
dataset.initNew();  
dataset.setNetworkName("MyThreadNetwork");  
dataset.setChannel(15);  
dataset.setNetworkKey(networkKey);  
OTThread.commitDataSet(dataset);
```

hasActiveDataset Checks whether OpenThread has a committed Active Operational Dataset.

```
bool hasActiveDataset() const;
```

This function returns `true` when an Active Operational Dataset is available in the OpenThread stack. The dataset may have been loaded from NVS during `OpenThread::begin()` or committed by the application with `commitDataSet()`.

This is useful when a sketch needs to decide whether to resume an existing Thread network or provision a new one:

```
OpenThread::begin(false);

if (OTThread.hasActiveDataset()) {
    OTThread.networkInterfaceUp();
    OTThread.start();
} else {
    DataSet dataset;
    dataset.initNew();
    dataset.setNetworkName("MyThreadNetwork");
    dataset.setChannel(15);
    OTThread.commitDataSet(dataset);
    OTThread.networkInterfaceUp();
    OTThread.start();
}
```

getCurrentDataSet Gets the current active operational dataset.

```
const DataSet &getCurrentDataSet() const;
```

This function returns a reference to a `DataSet` object containing the current active operational dataset parameters.

Network Information

getNetworkName Gets the Thread network name.

```
String getNetworkName() const;
```

This function returns the network name as a `String`.

getExtendedPanId Gets the extended PAN ID.

```
const uint8_t *getExtendedPanId() const;
```

This function returns a pointer to an 8-byte array containing the extended PAN ID.

getNetworkKey Gets the network key.

```
const uint8_t *getNetworkKey() const;
```

This function returns a pointer to a 16-byte array containing the network key.

Note: The network key is stored in static storage and persists after the function returns.

getChannel Gets the Thread channel.

```
uint8_t getChannel() const;
```

This function returns the Thread channel number (11-26).

getPanId Gets the PAN ID.

```
uint16_t getPanId() const;
```

This function returns the PAN ID as a 16-bit value.

Live Setters These setters apply changes directly to the running OpenThread instance. They are an alternative to building an offline `DataSet` and calling `commitDataSet()` - useful when commissioning a new device via the Joiner role, where the dataset is supplied by the commissioner and only a few parameters (channel, PAN ID, extended PAN ID) need to be pre-set on the joiner side.

All setters return an `otError` value (`OT_ERROR_NONE` on success, `OT_ERROR_INVALID_STATE` if OpenThread is not initialized, `OT_ERROR_INVALID_ARGS` for a null argument, `OT_ERROR_FAILED` if the OpenThread API lock could not be acquired). They acquire the OpenThread API lock internally and are safe to call from any FreeRTOS task. The Thread protocol should normally be stopped (`stop()`) before changing these parameters on an already-attached device.

setChannel Sets the IEEE 802.15.4 channel of the running instance.

```
otError setChannel(uint8_t channel);
```

- `channel` - Channel number (11..26).

Wraps `otLinkSetChannel`.

setPanId Sets the PAN ID of the running instance.

```
otError setPanId(uint16_t panid);
```

Wraps `otLinkSetPanId`.

setExtendedPanId Sets the 8-byte Extended PAN ID of the running instance.

```
otError setExtendedPanId(const uint8_t *extpanid);
```

- `extpanid` - Pointer to an 8-byte array (`OT_EXT_PAN_ID_SIZE`).

Wraps `otThreadSetExtendedPanId`.

setNetworkKey Sets the 16-byte Network Key of the running instance.

```
otError setNetworkKey(const uint8_t *key);
```

- `key` - Pointer to a 16-byte array (`OT_NETWORK_KEY_SIZE`).

Wraps `otThreadSetNetworkKey`.

setNetworkName Sets the network name of the running instance.

```
otError setNetworkName(const char *name);
```

- `name` - Null-terminated string, up to 16 ASCII characters.

Wraps `otThreadSetNetworkName`.

Example - Joiner side, pre-configure channel/PAN before joining:

```
OThread.begin(false); // stack up, no DataSet
OThread.setChannel(15); // pre-pick channel
OThread.setPanId(0xFFFF); // pre-pick PAN
uint8_t xp[8] = {0x11, 0x11, 0x11, 0x11, 0x22, 0x22, 0x22, 0x22};
OThread.setExtendedPanId(xp);
OThread.networkInterfaceUp();

otError err = OThread.startJoiner("J01NME"); // run Joiner state machine
if (err == OT_ERROR_NONE) {
    OThread.start(); // enable Thread with provisioned_
    ↪ dataset
}
```

Device Identity These read-only getters expose identity / version / radio parameters of the running instance.

getEui64 Returns the factory-assigned IEEE EUI-64 (the device's hardware extended address).

```
bool getEui64(uint8_t out[8]) const;
String getEui64() const;
```

- The byte-array form copies 8 bytes into `out` and returns `true` on success.
- The `String` form returns the EUI-64 as a 16-character lowercase hex string (empty `String` on failure).

Wraps `otLinkGetFactoryAssignedIeeeEui64`.

getExtendedAddress Returns the **current** IEEE 802.15.4 extended address used by the radio. This may differ from the factory EUI-64 (for example during commissioning the Joiner ID is used in its place).

```
const uint8_t *getExtendedAddress() const;
```

The returned pointer is stack-owned and valid only transiently. Wraps `otLinkGetExtendedAddress`.

getThreadVersion Returns the Thread protocol version implemented by the stack (for example 4 for Thread 1.3).

```
uint16_t getThreadVersion() const;
```

Wraps `otThreadGetVersion`.

getVersionString Returns a human-readable OpenThread stack version string (e.g. "OPENTHREAD/
thread-reference-...; ...").

```
String getVersionString() const;
```

Wraps `otGetVersionString`.

getTxPower Returns the configured radio transmit power, in dBm, or `INT8_MIN` on failure.

```
int8_t getTxPower() const;
```

Wraps `otPlatRadioGetTransmitPower`.

getPollPeriod Returns the Sleepy End Device data poll period, in milliseconds.

```
uint32_t getPollPeriod() const;
```

Wraps `otLinkGetPollPeriod`.

Joiner Role The Joiner role is used by a brand-new device to attach to a Thread network without knowing the network key. It only needs a PSKd (Pre-Shared Key for Device) shared with the network's Commissioner.

The OpenThread Arduino wrapper exposes a **synchronous** `startJoiner` helper: it runs the OpenThread Joiner state machine in the background OT task and blocks the calling task on a FreeRTOS semaphore until the commissioning callback fires (or the timeout elapses).

Order of calls:

1. `OThread.begin(false);` - start the stack without auto-loading a DataSet.
2. (optional) `setChannel / setPanId / setExtendedPanId` so the joiner does not have to scan every channel.
3. `OThread.networkInterfaceUp();` - IPv6 stack must be up before `startJoiner`.
4. `OThread.startJoiner(PSKd);` - blocks until success / failure. Thread must **not** be enabled yet (`start()` must not have been called).
5. On success, `OThread.start();` - enable Thread protocol with the dataset just provisioned by the commissioner.

Required Kconfig: `CONFIG_OPENTHREAD_JOINER=y`.

startJoiner Starts the Joiner role and synchronously waits for completion.

```
otError startJoiner(const char *pskd,
                   const char *provisioningUrl = nullptr,
                   const char *vendorName     = nullptr,
                   const char *vendorModel    = nullptr,
                   const char *vendorSwVersion = nullptr,
                   const char *vendorData     = nullptr,
                   uint32_t   timeoutMs      = 30000);
```

- `pskd` - Pre-Shared Key for Device. ASCII, 6..32 characters, must match the value the commissioner accepted via `addJoiner()`. Must not contain the characters I, O, Q or the digit 0 (base32-thread alphabet).
- `provisioningUrl / vendorName / vendorModel / vendorSwVersion / vendorData` - Optional MeshCoP TLVs passed to the commissioner during the Joiner Finalize message. May be `nullptr`.
- `timeoutMs` - Maximum time the wrapper waits on its semaphore for the `otJoinerCallback` to fire. The OpenThread stack also has its own internal Joiner timers. Default is 30 s.

Returns:

- `OT_ERROR_NONE` - successfully commissioned.
- `OT_ERROR_SECURITY` - PSKd mismatch.
- `OT_ERROR_NOT_FOUND` - no joinable network was discovered.
- `OT_ERROR_RESPONSE_TIMEOUT` - the commissioner stopped responding or the wrapper timeout expired (the wrapper also calls `otJoinerStop` in this case).
- `OT_ERROR_INVALID_STATE` - IPv6 stack not up, or Thread already enabled.

Wraps `otJoinerStart`.

stopJoiner Aborts an in-progress Joiner state machine.

```
void stopJoiner();
```

Wraps `otJoinerStop`.

getJoinerState Returns the live Joiner state (`OT_JOINER_STATE_IDLE / DISCOVER / CONNECT / CONNECTED / ENTRUST / JOINED`).

```
otJoinerState getJoinerState() const;
```

Wraps `otJoinerGetState`.

getJoinerId Returns the device's Joiner ID. By default this is the first 64 bits of SHA-256 (EUI-64) and is also used as the 802.15.4 extended address during commissioning.

```
const otExtAddress *getJoinerId() const;
```

Wraps otJoinerGetId.

Example - Joiner:

```
OThread.begin(false);
OThread.setChannel(15);
OThread.networkInterfaceUp();

otError err = OThread.startJoiner("J01NME");
if (err == OT_ERROR_NONE) {
    OThread.start(); // enable Thread with provisioned dataset
} else {
    Serial.printf("Join failed: %d\r\n", err);
}
```

Commissioner Role The Commissioner is the network-side counterpart of the Joiner: it authorizes new devices to attach using a PSKd, and securely hands the operational dataset to them. A Commissioner must already be **attached** to a Thread network (typically as the Leader).

The Arduino wrapper exposes a **synchronous** startCommissioner: it petitions the Commissioner role and blocks until either OT_COMMISSIONER_STATE_ACTIVE is reached or the petition is rejected.

Order of calls:

1. Form or resume a network (commitDataSet() or NVS resume).
2. OThread.networkInterfaceUp(); + OThread.start(); - bring up Thread and wait until the role is no longer Detached / Disabled.
3. OThread.startCommissioner(); - blocks until the petition succeeds.
4. OThread.addJoiner(PSKd); - open the joiner window for incoming devices.

Required Kconfig: CONFIG_OPENTHREAD_COMMISSIONER=y.

startCommissioner Petitions for the active Commissioner role and waits for the petition to resolve.

```
otError startCommissioner(uint32_t timeoutMs = 30000);
```

- timeoutMs - How long to wait for the petition to be granted before giving up. Default 30 s.

Returns:

- OT_ERROR_NONE - now in OT_COMMISSIONER_STATE_ACTIVE.
- OT_ERROR_REJECTED - petition rejected (another commissioner already active in the partition, or local commissioner state fell back to DISABLED).
- OT_ERROR_RESPONSE_TIMEOUT - petition did not resolve within timeoutMs. The wrapper calls otCommissionerStop to clean up.
- OT_ERROR_INVALID_STATE - device not attached to a network.

Wraps otCommissionerStart.

addJoiner Authorizes a remote joiner that presents the given PSKd.

```
otError addJoiner(const char *pskd,
                  const otExtAddress *eui64 = nullptr,
                  uint32_t timeoutSec = 120);
```

- pskd - PSKd that the joiner must present (must match the joiner side's startJoiner argument).

- `eui64` - EUI-64 of the specific joiner that is allowed, or `nullptr` to accept any joiner (equivalent to `*` in `ot-ctl`).
- `timeoutSec` - How long the joiner entry stays valid on the commissioner (default 120 s). Once expired, the joiner must be added again.

Returns:

- `OT_ERROR_NONE` - joiner authorized.
- `OT_ERROR_NO_BUFS` - commissioner table is full.
- `OT_ERROR_INVALID_ARGS` - invalid EUI-64 or PSKd.
- `OT_ERROR_INVALID_STATE` - commissioner is not active.

Wraps `otCommissionerAddJoiner`.

stopCommissioner Disables the Commissioner role.

```
void stopCommissioner();
```

Wraps `otCommissionerStop`.

getCommissionerState Returns the live Commissioner state (`OT_COMMISSIONER_STATE_DISABLED` / `PETITION` / `ACTIVE`).

```
otCommissionerState getCommissionerState() const;
```

Wraps `otCommissionerGetState`.

Example - Commissioner (run on an attached Leader/Router):

```
// Already attached to the network at this point
// (networkInterfaceUp() + start() completed earlier).
otError err = OThread.startCommissioner();
if (err == OT_ERROR_NONE) {
    OThread.addJoiner("J01NME",      // PSKd
                     nullptr,        // accept any joiner (wildcard)
                     120);           // window: 120 s
}
```

Timeouts at a glance Three independent timeouts are at play during a join:

1. **Wrapper join timeout** (`startJoiner(..., timeoutMs)`) - how long the Arduino wrapper blocks on its FreeRTOS semaphore waiting for the `otJoinerCallback`. Default 30 s. On expiry the wrapper calls `otJoinerStop` and returns `OT_ERROR_RESPONSE_TIMEOUT`.
2. **OpenThread stack timers** - internal MeshCoP / DTLS / channel-scan timers inside the Joiner state machine. NOT configurable from Arduino.
3. **Commissioner joiner window** (`addJoiner(..., timeoutSec)`) - how long the commissioner keeps the joiner entry alive. After this lifetime the entry is removed and the joiner must be re-added.

For interoperability with slow commissioners, set `startJoiner(..., timeoutMs)` slightly larger than `addJoiner(..., timeoutSec) * 1000`.

Device Role

otGetDeviceRole Gets the current device role.

```
static ot_device_role_t otGetDeviceRole();
```

This function returns the current Thread device role:

- `OT_ROLE_DISABLED` - The Thread stack is disabled.
- `OT_ROLE_DETACHED` - Not currently participating in a Thread network.
- `OT_ROLE_CHILD` - The Thread Child role.
- `OT_ROLE_ROUTER` - The Thread Router role.
- `OT_ROLE_LEADER` - The Thread Leader role.

Note: This is a static function.

otGetStringDeviceRole Gets the current device role as a string.

```
static const char *otGetStringDeviceRole();
```

This function returns a human-readable string representation of the current device role.

Note: This is a static function.

otPrintNetworkInformation Prints network information to a Stream.

```
static void otPrintNetworkInformation(Stream &output);
```

- `output` - The Stream object to print to (e.g., `Serial`)

This function prints comprehensive network information including: * Device role. * RLOC16. * Network name. * Channel. * PAN ID. * Extended PAN ID. * Network key.

Note: This is a static function.

Address Management

getMeshLocalPrefix Gets the mesh-local prefix.

```
const otMeshLocalPrefix *getMeshLocalPrefix() const;
```

This function returns a pointer to the mesh-local prefix structure.

getMeshLocalEid Gets the mesh-local EID (Endpoint Identifier).

```
IPAddress getMeshLocalEid() const;
```

This function returns the mesh-local IPv6 address as an `IPAddress` object.

getLeaderRloc Gets the Thread Leader RLOC (Routing Locator).

```
IPAddress getLeaderRloc() const;
```

This function returns the IPv6 address of the Thread Leader as an `IPAddress` object.

getRloc Gets the node RLOC (Routing Locator).

```
IPAddress getRloc() const;
```

This function returns the IPv6 RLOC address of this node as an `IPAddress` object.

getRloc16 Gets the RLOC16 (16-bit Routing Locator).

```
uint16_t getRloc16() const;
```

This function returns the 16-bit RLOC of this node.

Unicast Address Management

getUnicastAddressCount Gets the number of unicast addresses.

```
size_t getUnicastAddressCount() const;
```

This function returns the number of unicast IPv6 addresses assigned to this node. The count is cached for performance.

getUnicastAddress Gets a unicast address by index.

```
IPAddress getUnicastAddress(size_t index) const;
```

- `index` - The index of the address (0-based)

This function returns the unicast IPv6 address at the specified index as an `IPAddress` object.

Note: Addresses are cached for performance. Use `clearUnicastAddressCache()` to refresh the cache.

getAllUnicastAddresses Gets all unicast addresses.

```
std::vector<IPAddress> getAllUnicastAddresses() const;
```

This function returns a vector containing all unicast IPv6 addresses assigned to this node.

Multicast Address Management

getMulticastAddressCount Gets the number of multicast addresses.

```
size_t getMulticastAddressCount() const;
```

This function returns the number of multicast IPv6 addresses subscribed by this node. The count is cached for performance.

getMulticastAddress Gets a multicast address by index.

```
IPAddress getMulticastAddress(size_t index) const;
```

- `index` - The index of the address (0-based)

This function returns the multicast IPv6 address at the specified index as an `IPAddress` object.

Note: Addresses are cached for performance. Use `clearMulticastAddressCache()` to refresh the cache.

getAllMulticastAddresses Gets all multicast addresses.

```
std::vector<IPAddress> getAllMulticastAddresses() const;
```

This function returns a vector containing all multicast IPv6 addresses subscribed by this node.

Cache Management

clearUnicastAddressCache Clears the unicast address cache.

```
void clearUnicastAddressCache() const;
```

This function clears the cached unicast addresses. The cache will be automatically repopulated on the next address access.

clearMulticastAddressCache Clears the multicast address cache.

```
void clearMulticastAddressCache() const;
```

This function clears the cached multicast addresses. The cache will be automatically repopulated on the next address access.

clearAllAddressCache Clears all address caches.

```
void clearAllAddressCache() const;
```

This function clears both unicast and multicast address caches.

Advanced Access

getInstance Gets the OpenThread instance pointer.

```
otInstance *getInstance();
```

This function returns a pointer to the underlying OpenThread instance. This allows direct access to OpenThread API functions for advanced use cases.

Warning: Direct use of the OpenThread instance requires knowledge of the OpenThread API. Use with caution. The Arduino wrapper methods acquire the ESP OpenThread stack lock internally, but raw `ot*` calls made through this pointer are outside that protection and must follow the ESP-IDF OpenThread locking rules.

Operators

bool operator Returns whether OpenThread is started.

```
operator bool() const;
```

This operator returns `true` if OpenThread is started and ready, `false` otherwise.

Example:

```
if (OThread) {  
    Serial.println("OpenThread is ready");  
}
```

Example

Basic Thread Network Setup

```

#include <OThread.h>

void setup() {
    Serial.begin(115200);

    // Initialize OpenThread stack
    OpenThread::begin();

    // Wait for OpenThread to be ready
    while (!OThread) {
        delay(100);
    }

    // Create and configure dataset
    DataSet dataset;
    dataset.initNew();
    dataset.setNetworkName("MyThreadNetwork");
    dataset.setChannel(15);

    // Set network key (16 bytes)
    uint8_t networkKey[16] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
                              0x88, 0x99, 0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};
    dataset.setNetworkKey(networkKey);

    // Apply dataset and start network
    OThread.commitDataSet(dataset);
    OThread.networkInterfaceUp();
    OThread.start();

    // Wait for network to be ready
    while (OpenThread::otGetDeviceRole() == OT_ROLE_DETACHED) {
        delay(100);
    }

    // Print network information
    OpenThread::otPrintNetworkInformation(Serial);

    // Get and print addresses
    Serial.printf("Mesh Local EID: %s\r\n", OThread.getMeshLocalEid().toString().c_
↪str());
    Serial.printf("RLOC: %s\r\n", OThread.getRloc().toString().c_str());
    Serial.printf("RLOC16: 0x%04x\r\n", OThread.getRloc16());
}

```

Monitoring Device Role

```

void loop() {
    ot_device_role_t role = OpenThread::otGetDeviceRole();
    const char *roleStr = OpenThread::otGetStringDeviceRole();

    Serial.printf("Current role: %s\r\n", roleStr);

    switch (role) {
        case OT_ROLE_LEADER:
            Serial.println("This device is the Thread Leader");
            break;
        case OT_ROLE_ROUTER:
            Serial.println("This device is a Thread Router");
            break;
        case OT_ROLE_CHILD:
            Serial.println("This device is a Thread Child");
    }
}

```

(continues on next page)

(continued from previous page)

```

        break;
    case OT_ROLE_DETACHED:
        Serial.println("This device is not attached to a network");
        break;
    case OT_ROLE_DISABLED:
        Serial.println("Thread is disabled");
        break;
}

delay(5000);
}

```

Address Management

```

void printAddresses() {
    // Print unicast addresses
    size_t unicastCount = OThread.getUnicastAddressCount();
    Serial.printf("Unicast addresses: %lu\r\n", (unsigned long)unicastCount);
    for (size_t i = 0; i < unicastCount; i++) {
        Serial.printf(" [%lu] %s\r\n", (unsigned long)i, OThread.
        ↪getUnicastAddress(i).toString().c_str());
    }

    // Print multicast addresses
    size_t multicastCount = OThread.getMulticastAddressCount();
    Serial.printf("Multicast addresses: %lu\r\n", (unsigned long)multicastCount);
    for (size_t i = 0; i < multicastCount; i++) {
        Serial.printf(" [%lu] %s\r\n", (unsigned long)i, OThread.
        ↪getMulticastAddress(i).toString().c_str());
    }

    // Clear cache to force refresh
    OThread.clearAllAddressCache();
}

```

DataSet Class The DataSet class provides a convenient way to manage Thread operational datasets. It allows you to create, configure, and apply operational datasets to the Thread network.

- **Dataset Creation:** Create new operational datasets.
- **Parameter Configuration:** Set network name, channel, PAN ID, network key, and extended PAN ID.
- **Dataset Application:** Apply datasets to the Thread network.
- **Dataset Retrieval:** Get current dataset parameters.

DataSet Class

About The DataSet class provides a convenient way to create, configure, and manage Thread operational datasets. An operational dataset contains all the parameters needed to join or form a Thread network, including network name, channel, PAN ID, network key, and extended PAN ID.

Key Features: * Create new operational datasets. * Configure dataset parameters. * Apply datasets to the Thread network. * Retrieve dataset parameters. * Clear and reset datasets.

Use Cases: * Creating new Thread networks. * Joining existing Thread networks. * Configuring network parameters. * Network migration and reconfiguration.

API Reference

Constructor

DataSet Creates a new DataSet object.

```
DataSet();
```

This constructor creates an empty dataset. You must either call `initNew()` to create a new dataset or configure parameters manually.

Dataset Management

clear Clears all dataset parameters.

```
void clear();
```

This function clears all dataset parameters, resetting the dataset to an empty state.

initNew Initializes a new operational dataset.

```
void initNew();
```

This function creates a new operational dataset with randomly generated values for network key, extended PAN ID, and other parameters. The dataset will be ready to form a new Thread network.

Note: OpenThread must be initialized with `OpenThread::begin()` before calling this function.

Example:

```
DataSet dataset;  
dataset.initNew(); // Creates new dataset with random values  
dataset.setNetworkName("MyNewNetwork");  
dataset.setChannel(15);
```

getDataset Gets the underlying OpenThread dataset structure.

```
const otOperationalDataset &getDataset() const;
```

This function returns a reference to the underlying `otOperationalDataset` structure. This is used internally when applying the dataset to the Thread network.

Note: This function is typically used internally by `OpenThread::commitDataSet()`.

Setters

setNetworkName Sets the network name.

```
void setNetworkName(const char *name);
```

- `name` - The network name string (maximum 16 characters)

This function sets the network name for the Thread network. The network name is a human-readable identifier for the network.

Example:

```
dataset.setNetworkName("MyThreadNetwork");
```

setExtendedPanId Sets the extended PAN ID.

```
void setExtendedPanId(const uint8_t *extPanId);
```

- extPanId - Pointer to an 8-byte array containing the extended PAN ID

This function sets the extended PAN ID, which uniquely identifies the Thread network partition.

Example:

```
uint8_t extPanId[8] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77};  
dataset.setExtendedPanId(extPanId);
```

setNetworkKey Sets the network key.

```
void setNetworkKey(const uint8_t *key);
```

- key - Pointer to a 16-byte array containing the network key.

This function sets the network key, which is used for encryption and authentication in the Thread network.

Example:

```
uint8_t networkKey[16] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,  
                          0x88, 0x99, 0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};  
dataset.setNetworkKey(networkKey);
```

setChannel Sets the Thread channel.

```
void setChannel(uint8_t channel);
```

- channel - The Thread channel number (11-26).

This function sets the IEEE 802.15.4 channel used by the Thread network. Valid channels are 11 through 26.

Example:

```
dataset.setChannel(15); // Use channel 15
```

setPanId Sets the PAN ID.

```
void setPanId(uint16_t panId);
```

- panId - The PAN ID (16-bit value).

This function sets the PAN (Personal Area Network) ID for the Thread network.

Example:

```
dataset.setPanId(0x1234); // Set PAN ID to 0x1234
```

Getters

getNetworkName Gets the network name.

```
const char *getNetworkName() const;
```

This function returns a pointer to the network name string.

Returns: Pointer to the network name, or NULL if not set.

getExtendedPanId Gets the extended PAN ID.

```
const uint8_t *getExtendedPanId() const;
```

This function returns a pointer to the 8-byte extended PAN ID array.

Returns: Pointer to the extended PAN ID array, or NULL if not set.

getNetworkKey Gets the network key.

```
const uint8_t *getNetworkKey() const;
```

This function returns a pointer to the 16-byte network key array.

Returns: Pointer to the network key array, or NULL if not set.

getChannel Gets the Thread channel.

```
uint8_t getChannel() const;
```

This function returns the Thread channel number.

Returns: The channel number (11-26), or 0 if not set.

getPanId Gets the PAN ID.

```
uint16_t getPanId() const;
```

This function returns the PAN ID.

Returns: The PAN ID, or 0 if not set.

Dataset Application

apply Applies the dataset to an OpenThread instance.

```
void apply(otInstance *instance);
```

- `instance` - Pointer to the OpenThread instance.

This function applies the dataset to the specified OpenThread instance, making it the active operational dataset.

Note: This function is typically used internally. For normal use, use `OpenThread::commitDataSet()` instead.

Example:

```
otInstance *instance = OThread.getInstance();  
dataset.apply(instance);
```

Example

Creating a New Network

```

#include <OThread.h>

void setup() {
    Serial.begin(115200);

    // Initialize OpenThread
    OpenThread::begin();
    while (!OThread) {
        delay(100);
    }

    // Create a new dataset
    DataSet dataset;
    dataset.initNew(); // Generate random values

    // Configure network parameters
    dataset.setNetworkName("MyNewThreadNetwork");
    dataset.setChannel(15);

    // Set network key (16 bytes)
    uint8_t networkKey[16] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,
                               0x88, 0x99, 0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff};
    dataset.setNetworkKey(networkKey);

    // Set extended PAN ID (8 bytes)
    uint8_t extPanId[8] = {0x01, 0x23, 0x45, 0x67, 0x89, 0xab, 0xcd, 0xef};
    dataset.setExtendedPanId(extPanId);

    // Set PAN ID
    dataset.setPanId(0x1234);

    // Apply dataset and start network
    OThread.commitDataSet(dataset);
    OThread.networkInterfaceUp();
    OThread.start();

    Serial.println("New Thread network created");
}

```

Joining an Existing Network

```

#include <OThread.h>

void setup() {
    Serial.begin(115200);

    // Initialize OpenThread
    OpenThread::begin();
    while (!OThread) {
        delay(100);
    }

    // Create dataset for existing network
    DataSet dataset;

    // Configure with existing network parameters
    dataset.setNetworkName("ExistingThreadNetwork");
    dataset.setChannel(15);

    // Set the network key from the existing network
    uint8_t networkKey[16] = {0x00, 0x11, 0x22, 0x33, 0x44, 0x55, 0x66, 0x77,

```

(continues on next page)

(continued from previous page)

```

        0x88, 0x99, 0xaa, 0xbb, 0xcc, 0xdd, 0xee, 0xff});
dataset.setNetworkKey(networkKey);

// Set the extended PAN ID from the existing network
uint8_t extPanId[8] = {0x01, 0x23, 0x45, 0x67, 0x89, 0xab, 0xcd, 0xef};
dataset.setExtendedPanId(extPanId);

// Set PAN ID
dataset.setPanId(0x1234);

// Apply dataset and start network
OThread.commitDataSet(dataset);
OThread.networkInterfaceUp();
OThread.start();

Serial.println("Joining existing Thread network");
}

```

Reading Current Dataset

```

void printCurrentDataset() {
    // Get current dataset from OpenThread
    const DataSet &currentDataset = OThread.getCurrentDataSet();

    // Print dataset parameters
    Serial.println("Current Thread Dataset:");
    Serial.printf("  Network Name: %s\r\n", currentDataset.getNetworkName());
    Serial.printf("  Channel: %u\r\n", currentDataset.getChannel());
    Serial.printf("  PAN ID: 0x%04x\r\n", currentDataset.getPanId());

    // Print extended PAN ID
    const uint8_t *extPanId = currentDataset.getExtendedPanId();
    if (extPanId) {
        Serial.print("  Extended PAN ID: ");
        for (int i = 0; i < 8; i++) {
            Serial.printf("%02x", extPanId[i]);
        }
        Serial.println();
    }

    // Print network key (first 4 bytes for security)
    const uint8_t *networkKey = currentDataset.getNetworkKey();
    if (networkKey) {
        Serial.print("  Network Key: ");
        for (int i = 0; i < 4; i++) {
            Serial.printf("%02x", networkKey[i]);
        }
        Serial.println("...");
    }
}

```

Modifying Dataset Parameters

```

void modifyNetworkChannel() {
    // Get current dataset
    DataSet dataset = OThread.getCurrentDataSet();

    // Modify channel
    dataset.setChannel(20); // Change to channel 20
}

```

(continues on next page)

(continued from previous page)

```
// Apply modified dataset
OThread.commitDataSet(dataset);

Serial.println("Network channel changed to 20");
}
```

Best Practices

Dataset Security

- **Network Key:** Keep the network key secure and never expose it in logs or serial output.
- **Extended PAN ID:** Use unique extended PAN IDs to avoid network conflicts.
- **Channel Selection:** Choose channels that avoid interference with Wi-Fi networks (channels 11, 15, 20, 25 are often good choices).

Required Parameters For a dataset to be valid and usable, you typically need:

- **Network Name:** Required for human identification.
- **Network Key:** Required for security (16 bytes).
- **Channel:** Required for radio communication (11-26).
- **Extended PAN ID:** Recommended for network identification (8 bytes).
- **PAN ID:** Optional, will be assigned if not set.

Parameter Validation The DataSet class performs basic validation:

- Network name length is checked (maximum 16 characters).
- Channel range is validated (11-26).
- Null pointer checks for array parameters.

However, it's your responsibility to ensure:

- Network key is properly generated (use `initNew()` for random generation).
- Extended PAN ID is unique within your network environment.
- All required parameters are set before applying the dataset.

OpenThreadCLI The `OpenThreadCLI` class provides a Stream-based interface for interacting with the OpenThread CLI. It allows you to send CLI commands and receive responses programmatically.

- **CLI Interface:** Stream-based interface for sending commands and receiving responses.
- **Command Execution:** Execute OpenThread CLI commands programmatically.
- **Response Handling:** Parse and handle CLI command responses.
- **Console Mode:** Interactive console mode for debugging and testing.

OpenThread CLI

About The OpenThread CLI (Command-Line Interface) provides two ways to interact with the OpenThread stack through CLI commands:

- **CLI Helper Functions API:** Utility functions that execute CLI commands and parse responses.
- **OpenThreadCLI Class:** Stream-based interface for interactive CLI access.

The CLI Helper Functions API is useful for programmatic control using OpenThread CLI commands, while the `OpenThreadCLI` class provides a Stream interface for interactive console access.

CLI Helper Functions API The CLI Helper Functions API consists of utility functions that execute OpenThread CLI commands and handle responses. These functions interact with the OpenThread CLI through the OpenThreadCLI interface.

otGetRespCmd Executes a CLI command and gets the response.

```
bool otGetRespCmd(const char *cmd, char *resp = NULL, uint32_t respTimeout = 5000);
```

- cmd - The CLI command to execute (e.g., "state", "networkname").
- resp - Buffer to store the response (optional, can be NULL).
- respTimeout - Timeout in milliseconds for waiting for response (default: 5000 ms).

This function executes a CLI command and collects all response lines until “Done” or “Error” is received. If resp is not NULL, the response is stored in the buffer.

Returns: true if command executed successfully, false on error or timeout.

Example:

```
char response[256];
if (otGetRespCmd("state", response)) {
    Serial.printf("Thread state: %s\r\n", response);
}

if (otGetRespCmd("networkname", response)) {
    Serial.printf("Network name: %s\r\n", response);
}
```

otExecCommand Executes a CLI command with arguments.

```
bool otExecCommand(const char *cmd, const char *arg, ot_cmd_return_t *returnCode =   
↳ NULL);
```

- cmd - The CLI command to execute. (e.g., "networkname", "channel")
- arg - The command argument. (can be NULL for commands without arguments)
- returnCode - Pointer to ot_cmd_return_t structure to receive error information. (optional)

This function executes a CLI command with an optional argument and returns the success status. If returnCode is provided, it will be populated with error information on failure.

Returns: true if command executed successfully, false on error.

Error Structure:

```
typedef struct {
    int errorCode;           // OpenThread error code
    String errorMessage;    // Error message string
} ot_cmd_return_t;
```

Example:

```
ot_cmd_return_t errorInfo;

// Set network name
if (otExecCommand("networkname", "MyThreadNetwork", &errorInfo)) {
    Serial.println("Network name set successfully");
} else {
    Serial.printf("Error %d: %s\r\n", errorInfo.errorCode, errorInfo.errorMessage.
↳ c_str());
}

// Set channel
```

(continues on next page)

(continued from previous page)

```

if (otExecCommand("channel", "15", &errorInfo)) {
    Serial.println("Channel set successfully");
}

// Bring interface up
if (otExecCommand("ifconfig", "up", NULL)) {
    Serial.println("Interface is up");
}

```

otPrintRespCLI Executes a CLI command and prints the response to a Stream.

```

bool otPrintRespCLI(const char *cmd, Stream &output, uint32_t respTimeout);

```

- cmd - The CLI command to execute
- output - The Stream object to print responses to (e.g., Serial)
- respTimeout - Timeout in milliseconds per response line (default: 5000 ms)

This function executes a CLI command and prints all response lines to the specified Stream until “Done” or “Error” is received.

Returns: true if command executed successfully, false on error or timeout.

Example:

```

// Print all IP addresses
if (otPrintRespCLI("ipaddr", Serial, 5000)) {
    Serial.println("IP addresses printed");
}

// Print all multicast addresses
if (otPrintRespCLI("ipmaddr", Serial, 5000)) {
    Serial.println("Multicast addresses printed");
}

```

OpenThreadCLI Class The OpenThreadCLI class provides a Stream-based interface for interacting with the OpenThread CLI. It allows you to send CLI commands and receive responses programmatically or through an interactive console.

Initialization

begin Initializes the OpenThread CLI.

```

void begin();

```

This function initializes the OpenThread CLI interface. It must be called after `OpenThread::begin()` and before using any CLI functions.

Note: The OpenThread stack must be initialized with `OpenThread::begin()` before initializing the CLI. The Thread protocol itself may still be stopped while CLI commands configure the dataset.

end Stops and cleans up the OpenThread CLI.

```

void end();

```

This function stops the CLI interface and cleans up all CLI resources.

Console Management

startConsole Starts an interactive console for CLI access.

```
void startConsole(Stream &otStream, bool echoback = true, const char *prompt = "ot>
↪ ");
```

- `otStream` - The Stream object for console I/O (e.g., `Serial`).
- `echoback` - If `true`, echo characters back to the console (default: `true`).
- `prompt` - The console prompt string (default: `"ot> "`, can be `NULL` for no prompt).

This function starts an interactive console task that allows you to type CLI commands directly. The console will echo input and display responses.

Example:

```
OTThreadCLI.startConsole(Serial, true, "ot> ");
```

stopConsole Stops the interactive console.

```
void stopConsole();
```

This function stops the interactive console task.

setStream Changes the console Stream object.

```
void setStream(Stream &otStream);
```

- `otStream` - The new Stream object for console I/O.

This function changes the Stream object used by the console.

setEchoBack Changes the echo back setting.

```
void setEchoBack(bool echoback);
```

- `echoback` - If `true`, echo characters back to the console.

This function changes whether characters are echoed back to the console.

setPrompt Changes the console prompt.

```
void setPrompt(char *prompt);
```

- `prompt` - The new prompt string (can be `NULL` for no prompt).

This function changes the console prompt string.

onReceive Sets a callback function for CLI responses.

```
void onReceive(OnReceiveCb_t func);
```

- `func` - Callback function to call when a complete line of output is received.

The callback function is called whenever a complete line of output is received from the OpenThread CLI. This allows you to process CLI responses asynchronously.

Callback Signature:

```
typedef std::function<void(void)> OnReceiveCb_t;
```

Example:

```
void handleCLIResponse() {  
    while (OThreadCLI.available() > 0) {  
        char c = OThreadCLI.read();  
        // Process response character  
    }  
}  
  
OThreadCLI.onReceive(handleCLIResponse);
```

Buffer Management

setTxBufferSize Sets the transmit buffer size.

```
size_t setTxBufferSize(size_t tx_queue_len);
```

- tx_queue_len - The size of the transmit buffer in bytes (default: 256).

This function sets the size of the transmit buffer used for sending CLI commands.

Returns: The actual buffer size set, or 0 on error.

setRxBufferSize Sets the receive buffer size.

```
size_t setRxBufferSize(size_t rx_queue_len);
```

- rx_queue_len - The size of the receive buffer in bytes (default: 1024).

This function sets the size of the receive buffer used for receiving CLI responses.

Returns: The actual buffer size set, or 0 on error.

Stream Interface The `OpenThreadCLI` class implements the Arduino `Stream` interface, allowing you to use it like any other `Stream` object.

write Writes a byte to the CLI.

```
size_t write(uint8_t c);
```

- c - The byte to write

This function writes a single byte to the CLI transmit buffer.

Returns: The number of bytes written (1 on success, 0 on failure).

available Checks if data is available to read.

```
int available();
```

This function returns the number of bytes available in the receive buffer.

Returns: Number of bytes available, or -1 if CLI is not initialized.

read Reads a byte from the CLI.

```
int read();
```

This function reads a single byte from the CLI receive buffer.

Returns: The byte read, or -1 if no data is available.

peek Peeks at the next byte without removing it.

```
int peek();
```

This function returns the next byte in the receive buffer without removing it.

Returns: The byte, or -1 if no data is available.

flush Flushes the transmit buffer.

```
void flush();
```

This function waits for all data in the transmit buffer to be sent.

Operators

bool operator Returns whether the CLI is started.

```
operator bool() const;
```

This operator returns `true` if the CLI is started and ready, `false` otherwise.

Example:

```
if (OThreadCLI) {  
    Serial.println("CLI is ready");  
}
```

Example

Using CLI Helper Functions API

```
#include <OThread.h>  
#include <OThreadCLI.h>  
#include <OThreadCLI_Util.h>  
  
void setup() {  
    Serial.begin(115200);  
  
    // Initialize OpenThread  
    OpenThread::begin();  
    while (!OThread) {  
        delay(100);  
    }  
  
    // Initialize CLI  
    OThreadCLI.begin();  
    while (!OThreadCLI) {  
        delay(100);  
    }  
}
```

(continues on next page)

(continued from previous page)

```

// Get network state
char resp[256];
if (otGetRespCmd("state", resp)) {
    Serial.printf("Thread state: %s\r\n", resp);
}

// Set network name
ot_cmd_return_t errorInfo;
if (otExecCommand("networkname", "MyThreadNetwork", &errorInfo)) {
    Serial.println("Network name set");
} else {
    Serial.printf("Error: %s\r\n", errorInfo.errorMessage.c_str());
}

// Set channel
if (otExecCommand("channel", "15", NULL)) {
    Serial.println("Channel set");
}

// Bring interface up
if (otExecCommand("ifconfig", "up", NULL)) {
    Serial.println("Interface up");
}

// Print IP addresses
otPrintRespCLI("ipaddr", Serial, 5000);
}

```

Using OpenThreadCLI Class

Interactive Console

```

#include <OThread.h>
#include <OThreadCLI.h>

void setup() {
    Serial.begin(115200);

    // Initialize OpenThread
    OpenThread::begin();
    while (!OThread) {
        delay(100);
    }

    // Initialize and start CLI console
    OThreadCLI.begin();
    OThreadCLI.startConsole(Serial, true, "ot> ");

    Serial.println("OpenThread CLI Console Ready");
    Serial.println("Type OpenThread CLI commands (e.g., 'state', 'networkname')");
}

```

Programmatic CLI Access

```

#include <OThread.h>
#include <OThreadCLI.h>

void setup() {

```

(continues on next page)

(continued from previous page)

```

Serial.begin(115200);

// Initialize OpenThread
OpenThread::begin();
while (!OThread) {
    delay(100);
}

// Initialize CLI
OThreadCLI.begin();
while (!OThreadCLI) {
    delay(100);
}

// Send CLI commands programmatically
OThreadCLI.println("state");
delay(100);
while (OThreadCLI.available() > 0) {
    char c = OThreadCLI.read();
    Serial.write(c);
}

// Send command with argument
OThreadCLI.print("networkname ");
OThreadCLI.println("MyThreadNetwork");
delay(100);
while (OThreadCLI.available() > 0) {
    char c = OThreadCLI.read();
    Serial.write(c);
}
}

```

Using Callback for Responses

```

#include <OThread.h>
#include <OThreadCLI.h>

void handleCLIResponse() {
    String response = "";
    while (OThreadCLI.available() > 0) {
        char c = OThreadCLI.read();
        if (c == '\n' || c == '\r') {
            if (response.length() > 0) {
                Serial.printf("CLI Response: %s\r\n", response.c_str());
                response = "";
            }
        } else {
            response += c;
        }
    }
}

void setup() {
    Serial.begin(115200);

    // Initialize OpenThread
    OpenThread::begin();
    while (!OThread) {
        delay(100);
    }
}

```

(continues on next page)

(continued from previous page)

```

// Initialize CLI with callback
OThreadCLI.begin();
OThreadCLI.onReceive(handleCLIResponse);

// Send commands
OThreadCLI.println("state");
delay(500);
OThreadCLI.println("networkname");
delay(500);
}

```

Common OpenThread CLI Commands Here are some commonly used OpenThread CLI commands:

- `state` - Get the current Thread state.
- `networkname <name>` - Set or get the network name.
- `channel <channel>` - Set or get the channel (11-26).
- `panid <panid>` - Set or get the PAN ID.
- `extpanid <extpanid>` - Set or get the extended PAN ID.
- `networkkey <key>` - Set or get the network key.
- `ifconfig up` - Bring the network interface up.
- `ifconfig down` - Bring the network interface down.
- `ipaddr` - List all IPv6 addresses.
- `ipmaddr` - List all multicast addresses.
- `rloc16` - Get the RLOC16.
- `leaderdata` - Get leader data.
- `router table` - Get router table.
- `child table` - Get child table.

For a complete list of OpenThread CLI commands, refer to the [OpenThread CLI Reference](#).

OThreadUDP The `OThreadUDP` class is an Arduino UDP-compatible class that sends and receives IPv6 UDP datagrams directly through the OpenThread `otUdpSocket` API, without going through lwIP. It is the lightest path for application-level UDP traffic over a Thread mesh and is particularly well suited to memory-constrained 802.15.4-only SoCs (ESP32-H2).

- **Familiar API:** Inherits from Arduino's UDP base, so `begin`, `beginPacket`, `write`, `endPacket`, `parsePacket`, `read`, `remoteIP`, `remotePort` work like any other Arduino UDP driver.
- **Native OpenThread:** Uses `otUdpOpen` / `otUdpBind` / `otUdpSend` and exposes one `otUdpSocket` per instance.
- **Multicast:** `beginMulticast(group, port)` joins an IPv6 multicast group via `otIp6SubscribeMulticastAddress`.
- **Tunable RX queue:** `OT_UDP_MAX_PACKET_SIZE` and `OT_UDP_RX_QUEUE_DEPTH` build-time defines control per-instance memory footprint.

OThreadUDP Class

About The `OThreadUDP` class is an Arduino UDP implementation backed directly by the native OpenThread `otUdpSocket` API. It does **not** go through lwIP, so it is the lightest path to send and receive IPv6 UDP datagrams over the Thread mesh and avoids duplicate IP stacks on small SoCs (ESP32-H2, ESP32-C6, ESP32-C5).

Key Features:

- Full Arduino UDP interface (`begin`, `beginMulticast`, `beginPacket`, `write`, `endPacket`, `parsePacket`, `read`, `peek`, `flush`, `remoteIP`, `remotePort`).
- Pure raw `otUdpSocket` - no lwIP dependency.

- Uses a FreeRTOS queue to buffer incoming datagrams between the OpenThread receive callback and the sketch's `loop()`.
- Drop-oldest policy when the RX queue is full, so `parsePacket()` always returns fresh data instead of stalling on stale packets.
- Configurable maximum datagram size and queue depth via build-time defines.

Use Cases:

- IPv6 unicast or multicast UDP messaging between Thread devices.
- Low-overhead device-to-device telemetry over a Thread mesh.
- Anywhere a regular Arduino UDP sketch is needed but on top of Thread instead of Wi-Fi/Ethernet.

API Reference The class inherits from Arduino's abstract UDP base class. All arguments and return values follow the standard Arduino UDP semantics. The methods specific to OpenThread are highlighted.

Constructor

```
OThreadUDP();  
~OThreadUDP();
```

The default constructor does not open any socket - call `begin()` to actually create the underlying `otUdpSocket`. The destructor calls `stop()` automatically.

operator bool

```
operator bool() const;
```

Returns `true` when the underlying `otUdpSocket` is open (i.e. between a successful `begin()` and the matching `stop()`).

Socket Lifecycle**begin (IPv6 address + port)**

```
uint8_t begin(IPAddress addr, uint16_t port);
```

- `addr` - Local IPv6 address to bind to. Use `OT_IN6ADDR_ANY` to accept on any local address.
- `port` - Local UDP port.

Opens the socket (`otUdpOpen`) and binds it (`otUdpBind(..., OT_NETIF_THREAD_INTERNAL)`). Returns 1 on success, 0 on failure.

begin (port only)

```
uint8_t begin(uint16_t port);
```

Convenience overload that binds on `OT_IN6ADDR_ANY`.

beginMulticast

```
uint8_t beginMulticast(IPAddress group, uint16_t port);
```

- `group` - IPv6 multicast group to subscribe to (e.g. `ff03::1`).
- `port` - Local UDP port.

Internally calls `begin(port)` first and then `otIp6SubscribeMulticastAddress` for `group`. The subscription is released symmetrically by `stop()`.

stop

```
void stop();
```

Frees any pending TX message buffer, unsubscribes the multicast group (if one was joined), and closes the `otUdpSocket`. Safe to call multiple times.

Sending Packets**beginPacket (IPAddress)**

```
int beginPacket(IPAddress ip, uint16_t port);
```

- `ip` - Destination IPv6 address.
- `port` - Destination UDP port.

Allocates an `otMessage` via `otUdpNewMessage`, stores the peer address/port, and prepares the socket for `write()` calls. Any previously-pending TX buffer is freed first.

Returns 1 on success, 0 if no message buffer was available.

beginPacket (textual host)

```
int beginPacket(const char *host, uint16_t port);
```

Convenience overload that parses `host` as an IPv6 address using `IPAddress::fromString` and forwards to the `beginPacket(IPAddress, port)` overload. Returns 0 if the address is not a valid IPv6 literal.

write

```
size_t write(uint8_t b);  
size_t write(const uint8_t *buf, size_t size);
```

Appends bytes to the pending TX message using `otMessageAppend`. Returns the number of bytes actually queued (0 on failure or if no `beginPacket` is pending).

endPacket

```
int endPacket();
```

Submits the TX message via `otUdpSend`. On success OpenThread takes ownership of the message buffer; on failure `OTThreadUDP` frees it itself so no buffer leaks. Returns 1 on success, 0 on failure.

Receiving Packets The internal RX callback runs in the OpenThread task with the API lock held. It snapshots the payload plus `mPeerAddr / mPeerPort` into a per-instance FreeRTOS queue. `parsePacket()` then pops one datagram into a per-instance current-packet buffer that `read()`, `available()`, `peek()` and `flush()` walk through.

If the queue is full when a new datagram arrives, the **oldest** entry is dropped to make room. This keeps the queue from becoming a buffer of stale data when the sketch falls behind.

parsePacket

```
int parsePacket();
```

Pops the next received datagram (if any) into the current-packet buffer and returns its length. Returns 0 when the queue is empty. Successive calls drop any unread bytes from the previous packet.

available

```
int available();
```

Returns the number of unread bytes left in the current packet.

read

```
int read();  
int read(unsigned char *buf, size_t len);  
int read(char *buf, size_t len);
```

Reads from the current packet. The byte form returns -1 if no data is left; the buffer forms return the number of bytes copied (0 if no packet is current).

peek

```
int peek();
```

Returns the next byte in the current packet without consuming it (-1 if no byte is available).

flush

```
void flush();
```

Discards any remaining bytes of the current RX packet.

remoteIP

```
IPAddress remoteIP();
```

Returns the sender's IPv6 address of the most recently parsed packet (`IPAddress(IPv6)` if no packet has been parsed yet).

remotePort

```
uint16_t remotePort();
```

Returns the sender's UDP port of the most recently parsed packet (0 if no packet has been parsed yet).

Constants**OT_IN6ADDR_ANY**

```
extern const IPAddress OT_IN6ADDR_ANY;
```

The IPv6 “any” address (`:::`), suitable for binding to all local IPv6 interfaces on a port. Equivalent to `IPAddress(IPv6)` but more self-documenting at call sites.

Build-time tunables The following defines control the RX path; override them on the compiler command line if your traffic profile needs more headroom.

OT_UDP_MAX_PACKET_SIZE

```
#define OT_UDP_MAX_PACKET_SIZE 512 // default
```

Maximum payload bytes of a single received datagram that the RX queue will store. Larger packets are **truncated** to this size on arrival.

OT_UDP_RX_QUEUE_DEPTH

```
#define OT_UDP_RX_QUEUE_DEPTH 4 // default
```

Number of datagrams that may sit in the FreeRTOS RX queue between calls to `parsePacket()`. When the queue is full the **oldest** entry is dropped to make room for the new one.

Memory note: each `OThreadUDP` instance allocates one RX queue of `OT_UDP_RX_QUEUE_DEPTH * (OT_UDP_MAX_PACKET_SIZE + ~20)` bytes once `begin()` succeeds. The default settings cost ~2 KB per instance.

Example

Basic UDP Echo

```
#include <OThread.h>
#include <OThreadUDP.h>

// ff02::1 = link-local all-nodes multicast
const uint8_t serverBytes[16] = {0xff, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
                                0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01};
IPAddress server(IPv6, serverBytes);

OThreadUDP otUdp;

void setup() {
    Serial.begin(115200);

    // Bring up Thread (using a DataSet, or via the Joiner role).
    OThread.begin(false);
    // ... configure dataset and call commitDataSet(), then
    // networkInterfaceUp() + start(); or, for a Joiner device,
    // networkInterfaceUp() + startJoiner() + start() on success.
    OThread.networkInterfaceUp();
    OThread.start();

    // Bind a UDP socket on port 7, any IPv6 local address.
    otUdp.begin(7);
}

void loop() {
    // Send "Hello" to the link-local all-nodes group, port 7.
    otUdp.beginPacket(server, 7);
    otUdp.write((const uint8_t *) "Hello", 5);
    otUdp.endPacket();

    // Drain any pending datagrams.
    while (int n = otUdp.parsePacket()) {
        char buf[64];
        int r = otUdp.read(buf, sizeof(buf) - 1);
        buf[r] = 0;
        Serial.printf("From [%s]:%u -> '%s'\r\n",
                      otUdp.remoteIP().toString().c_str(),
                      otUdp.remotePort(), buf);
    }

    delay(1000);
}
```

Multicast Receiver


```

OTThreadUDP otUdp;
const uint8_t realmLocalBytes[16] = {0xff, 0x03, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
↪0x00,
                                0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
↪0x01};
IPAddress realmLocalAllNodes(IPv6, realmLocalBytes); // ff03::1

void setup() {
    // ... bring up Thread ...
    otUdp.beginMulticast(realmLocalAllNodes, 7);
}

void loop() {
    if (int n = otUdp.parsePacket()) {
        char buf[64];
        int r = otUdp.read(buf, sizeof(buf) - 1);
        buf[r] = 0;
        Serial.printf("Multicast from [%s]:%u -> '%s'\r\n",
                        otUdp.remoteIP().toString().c_str(),
                        otUdp.remotePort(), buf);
    }
}

```

Best Practices

- **Open after attach:** only call `otUdp.begin()` after the device is attached to the network (role is Child / Router / Leader). Binding before attach succeeds, but no traffic will flow until the IPv6 stack acquires its mesh-local address.
- **Avoid blocking the OT task:** the RX callback runs in the OpenThread task with the API lock held - it must finish quickly. Do not call back into `otXXX(...)` functions from a sketch `loop()` while another task holds the lock.
- **Tune queue depth:** if your application can fall behind (e.g. expensive `Serial.print` per packet) and you cannot afford to drop datagrams, increase `OT_UDP_RX_QUEUE_DEPTH` at the cost of RAM.
- **MTU:** Thread message buffers are small. Keep individual datagrams below ~1280 bytes (the IPv6 minimum link MTU) and preferably much smaller for reliability.
- **Multicast scope:** `ff02::` is link-local (single Thread hop), `ff03::` is realm-local (entire Thread partition). Choose according to how far the message must propagate.
- **Application ports:** avoid OpenThread-reserved ports for application sockets. 5683 / 5684 are CoAP/CoAPs ports and 61631 is Thread TMF CoAP. The Native UDP examples use 5050 and 5051.

CLI Helper Functions API The CLI Helper Functions API provides utility functions for executing OpenThread CLI commands and parsing responses. This API is useful when you need to interact with OpenThread through the CLI interface.

- **Command Execution:** Execute CLI commands with arguments.
- **Response Parsing:** Get and parse CLI command responses.
- **Error Handling:** Handle CLI command errors and return codes.

Key Functions:

- `otGetRespCmd()`: Execute CLI command and get response string.
- `otExecCommand()`: Execute CLI command with arguments and error handling.
- `otPrintRespCLI()`: Execute CLI command and print response to Stream.

For detailed documentation on the CLI Helper Functions API, see the [OpenThread CLI](#) documentation.

Supported Hardware

The OpenThread library requires ESP32 SoCs with IEEE 802.15.4 radio support:

- **ESP32-H2**: Native Thread support with IEEE 802.15.4 radio.
- **ESP32-C6**: Thread support with IEEE 802.15.4 radio (when Thread is enabled).
- **ESP32-C5**: Thread support with IEEE 802.15.4 radio (when Thread is enabled).

Note: Thread support must be enabled in the ESP-IDF configuration (`CONFIG_OPENTHREAD_ENABLED`). This is done automatically when using the ESP32 Arduino OpenThread library.

OpenThread Examples

The OpenThread library includes CLI-based and Native API examples demonstrating Thread network setup, CLI command automation, CoAP, UDP, and simple IoT device workflows. All examples are available in the [ESP Arduino GitHub repository](#).

CLI Examples:

- **Simple CLI** - Starts the OpenThread CLI console over Serial for interactive command-line testing. [View Simple CLI code on GitHub](#)
- **Simple Node** - Demonstrates a minimal CLI-driven Thread node setup. [View Simple Node code on GitHub](#)
- **Thread Scan** - Uses OpenThread CLI commands to scan and report nearby Thread networks. [View Thread Scan code on GitHub](#)
- **onReceive** - Shows how to handle asynchronous CLI output using the `OpenThreadCLI` receive callback. [View onReceive code on GitHub](#)

CLI Simple Thread Network Examples:

- **Leader Node** - Creates and starts a Thread network using CLI commands. [View CLI Leader Node code on GitHub](#)
- **Router Node** - Joins the CLI-created Thread network as a router-capable node. [View CLI Router Node code on GitHub](#)
- **Extended Router Node** - Demonstrates a CLI-driven router node with extended network behavior. [View CLI Extended Router Node code on GitHub](#)

CLI CoAP Examples:

- **CoAP Lamp** - Implements a CoAP-controlled lamp using OpenThread CLI commands. [View CoAP Lamp code on GitHub](#)
- **CoAP Switch** - Sends CoAP control commands to the CoAP lamp example over Thread. [View CoAP Switch code on GitHub](#)

CLI UDP Examples:

- **UDP Sensor Collector** - Creates a Thread Leader and UDP sink that receives sensor telemetry and sends acknowledgments. [View UDP Sensor Collector code on GitHub](#)
- **UDP Sensor Node** - Creates a sensor node, optionally configured as a sleepy child, that sends UDP telemetry to the collector. [View UDP Sensor Node code on GitHub](#)

Native API Examples:

- **Native Examples Overview** - Describes the Native API examples and how to choose between them. [View Native examples README on GitHub](#)
- **Native Leader Node** - Creates and starts a Thread network using the OpenThread Classes API. [View Native Leader Node code on GitHub](#)
- **Native Router Node** - Joins the Native API simple Thread network as a router-capable node. [View Native Router Node code on GitHub](#)
- **Thread Commissioning** - Demonstrates a `CommissionerNode` that forms a network and authorizes a `JoinerNode` that joins using only a PSKd. [View Thread Commissioning examples on GitHub](#)
- **UDP Light + Switch** - Demonstrates a Native UDP light server and one or more switch clients using Thread commissioning and application port 5051. [View UDP Light + Switch examples on GitHub](#)
- **UDP Sensor Network** - Demonstrates a Native UDP collector and multiple sensor nodes, including application-level sequence ACKs, application port 5050, and optional Sleepy End Device behavior. [View UDP Sensor Network examples on GitHub](#)

Common Problems and Issues

Troubleshooting

Common Issues

Thread network not starting

- Ensure the device has IEEE 802.15.4 radio support (ESP32-H2, ESP32-C6, ESP32-C5).
- Check that Thread is enabled in ESP-IDF configuration (`CONFIG_OPENTHREAD_ENABLED`).
- Verify that `OpenThread::begin()` is called before using Thread functions.
- Check Serial Monitor for initialization errors.

Device not joining network

- Verify the operational dataset parameters (network name, network key, channel, PAN ID).
- Ensure the device is within range of the Thread network.
- Check that the network key and extended PAN ID match the network you're trying to join.
- Verify the device role is not "Detached".

CLI commands not working

- Ensure `OpenThreadCLI::begin()` is called before using CLI functions.
- Check that `OpenThreadCLI::startConsole()` is called with a valid Stream.
- Verify the CLI is properly initialized and running.
- Check Serial Monitor for CLI initialization errors.

Address not available

- Wait for the device to join the Thread network (role should be Child, Router, or Leader).
- Check that the network interface is up using `networkInterfaceUp()`.
- Verify the device has obtained a mesh-local address.
- Check network connectivity using `getRloc()` or `getMeshLocalEid()`.

Dataset not applying

- Ensure all required dataset parameters are set (network name, network key, channel).
- Verify the dataset is valid before applying.
- Check that OpenThread has been initialized with `OpenThread::begin()` before applying the dataset.
- Apply or update the Active Operational Dataset while Thread is stopped; then bring the interface up and call `OThread.start()`.
- Verify the dataset parameters match the target network.

Unexpected network after reboot

- OpenThread persists the committed Active Operational Dataset in NVS.
- After `OpenThread::begin()` returns, use `OThread.hasActiveDataset()` to check whether a dataset was restored from NVS.
- If a dataset is present, bring the interface up and start Thread to resume the existing network.
- If you want to apply changed dataset constants, clear the stored dataset first (for example with the `OpenThread CLI factoryreset` command).

Joiner fails to attach

- `OT_ERROR_SECURITY`: The PSKd does not match the value the Commissioner used in `addJoiner()`. PSKd is case-sensitive and must use the base32-thread alphabet (no I, O, Q, 0).
- `OT_ERROR_NOT_FOUND`: No Commissioner is advertising a joinable network on the scanned channels. Make sure the Commissioner sketch is running and that `startCommissioner()` returned success.
- `OT_ERROR_RESPONSE_TIMEOUT`: The Commissioner started the handshake but did not finish it in time. Increase the `timeoutMs` argument of `startJoiner` and the `timeoutSec` argument of `addJoiner`, or retry.
- `OT_ERROR_INVALID_STATE`: `networkInterfaceUp()` was not called, or `start()` has already been called (Thread must be stopped when the Joiner runs).
- Build flag: confirm `CONFIG_OPENTHREAD_JOINER=y` is set.

Commissioner cannot start

- `OT_ERROR_INVALID_STATE`: The device must be attached to a Thread network (Leader / Router / Child) before petitioning. Wait until `otGetDeviceRole()` is not `OT_ROLE_DETACHED` / `OT_ROLE_DISABLED`.
- `OT_ERROR_REJECTED`: Another Commissioner is already active in the partition, or the petition was denied. Stop the other Commissioner or wait for it to time out.

- Build flag: confirm `CONFIG_OPENTHREAD_COMMISSIONER=y` is set.

OTThreadUDP packets not received

- Make sure the device has finished attaching (role != Detached) before binding the socket; binding before attach succeeds but no traffic flows until a mesh-local address is acquired.
- If using multicast, double-check the group scope (ff02:: = link-local, ff03:: = realm-local).
- Avoid OpenThread-reserved UDP ports for application traffic. 5683 / 5684 are CoAP/CoAPs ports and 61631 is Thread TMF CoAP; the Native examples use 5050 and 5051.
- If you are seeing dropped or truncated packets, bump `OT_UDP_MAX_PACKET_SIZE` and `OT_UDP_RX_QUEUE_DEPTH` at build time.
- If `otUdpNewMessage()` consistently returns `NULL`, the OpenThread message buffer pool is exhausted - reduce send rate or increase `CONFIG_OPENTHREAD_NUM_MESSAGE_BUFFERS`.

Initialization Order For proper initialization, follow this order:

1. Initialize OpenThread stack: `OpenThread::begin()`
2. Initialize OpenThreadCLI (if using CLI): `OpenThreadCLI::begin()`
3. Start CLI console (if using CLI): `OpenThreadCLI::startConsole()`
4. Configure dataset (if needed): Create and configure `DataSet`
5. Apply dataset (if needed): `OTThread.commitDataSet(dataset)`
6. Bring network interface up: `OTThread.networkInterfaceUp()`
7. Start Thread network: `OTThread.start()`

Joiner Initialization Order (Commissioning) When the device must attach to a network using only a PSKd (and learn the operational dataset from a Commissioner instead of carrying it in source code or NVS), follow this order on the **joiner** side:

1. `OTThread.begin(false);` - initialize the stack without loading a `DataSet` from NVS.
2. (optional) `OTThread.setChannel(...)` / `setPanId(...)` / `setExtendedPanId(...)` - hint the radio so the Joiner does not have to scan every channel.
3. `OTThread.networkInterfaceUp();` - the IPv6 stack must be up before running the Joiner state machine.
4. `OTThread.startJoiner(PSKD);` - blocks until commissioning completes or the timeout expires. Thread must **not** be enabled yet (`start()` must not have been called).
5. On success, `OTThread.start();` - enable Thread protocol with the dataset just provisioned by the Commissioner.

And on the **commissioner** side, after the device has formed or resumed a network and attached (typically as Leader):

1. `OTThread.networkInterfaceUp(); + OTThread.start();` - bring up Thread and wait until the role is no longer Detached / Disabled.
2. `OTThread.startCommissioner();` - blocks until `OT_COMMISSIONER_STATE_ACTIVE`.
3. `OTThread.addJoiner(PSKD);` - authorize an incoming joiner (any EUI-64 by default, valid for 120 s).

For detailed API documentation see [OpenThread Class](#), sections “Joiner Role” and “Commissioner Role”. The `examples/Native/ThreadCommissioning/` directory contains a working `CommissionerNode` / `JoinerNode` pair using this flow. A more elaborate end-to-end demo that combines Joiner / Commissioner with the `OTThreadUDP` socket class lives in `examples/Native/UDP_Light_Switch/` (light server + switch client) - one Leader/Commissioner accepts any number of joiner switches that drive an on-board RGB lamp via realm-local multicast and receive a UDP ACK for every command.

Example

Basic OpenThread Setup Using the Classes API:

```
#include <OTThread.h>

void setup() {
  Serial.begin(115200);
```

(continues on next page)

(continued from previous page)

```

// Initialize OpenThread stack
OpenThread::begin();

// Wait for OpenThread to be ready
while (!OThread) {
    delay(100);
}

// Create and configure dataset
DataSet dataset;
dataset.initNew();
dataset.setNetworkName("MyThreadNetwork");
dataset.setChannel(15);

// Apply dataset and start network
OThread.commitDataSet(dataset);
OThread.networkInterfaceUp();
OThread.start();

// Print network information
OpenThread::otPrintNetworkInformation(Serial);
}

```

Using the CLI Helper Functions API:

```

#include <OThread.h>
#include <OThreadCLI.h>
#include <OThreadCLI_Util.h>

void setup() {
    Serial.begin(115200);

    // Initialize OpenThread stack
    OpenThread::begin();

    // Initialize and start CLI
    OThreadCLI.begin();
    OThreadCLI.startConsole(Serial);

    // Wait for CLI to be ready
    while (!OThreadCLI) {
        delay(100);
    }

    // Execute CLI commands
    char resp[256];
    if (otGetRespCmd("state", resp)) {
        Serial.printf("Thread state: %s\r\n", resp);
    }

    if (otExecCommand("networkname", "MyThreadNetwork", NULL)) {
        Serial.println("Network name set successfully");
    }

    if (otExecCommand("ifconfig", "up", NULL)) {
        Serial.println("Network interface up");
    }
}

```

Joiner-based Commissioning Joiner (new device) - obtains the network key over the air from a Commissioner using only a PSKd:

```
#include <OThread.h>

const char PSKD[] = "J01NME";

void setup() {
    Serial.begin(115200);

    OThread.begin(false);           // stack up, no DataSet
    OThread.setChannel(15);         // optional channel hint
    OThread.networkInterfaceUp();   // IPv6 must be up

    otError err = OThread.startJoiner(PSKD);
    if (err == OT_ERROR_NONE) {
        OThread.start();           // enable Thread with provisioned dataset
        Serial.println("Joined network!");
    } else {
        Serial.printf("Joiner error: %d\r\n", err);
    }
}
```

Commissioner (Leader / attached Router) - admits new joiners via PSKd:

```
#include <OThread.h>

void setup() {
    Serial.begin(115200);

    // ... bring up the device as Leader (DataSet + commitDataSet + start) ...

    // Wait until the device is attached
    while (OThread.otGetDeviceRole() == OT_ROLE_DETACHED ||
           OThread.otGetDeviceRole() == OT_ROLE_DISABLED) {
        delay(100);
    }

    // Become the Commissioner and accept any joiner with PSKd "J01NME"
    if (OThread.startCommissioner() == OT_ERROR_NONE) {
        OThread.addJoiner("J01NME", nullptr, 120); // window: 120 s
    }
}
```

UDP over Thread Sending and receiving IPv6 UDP datagrams over the Thread mesh, using OThreadUDP (raw otUdpSocket, no lwIP):

```
#include <OThread.h>
#include <OThreadUDP.h>

const uint8_t allNodesBytes[16] = {0xff, 0x02, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00,
                                     0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x00, 0x01};
IPAddress allNodes(IPv6, allNodesBytes); // ff02::1
OThreadUDP otUdp;

void setup() {
    Serial.begin(115200);

    // ... bring up Thread (DataSet or Joiner) ...
    OThread.networkInterfaceUp();
    OThread.start();
}
```

(continues on next page)

(continued from previous page)

```

    otUdp.begin(7);                                     // bind to UDP port 7
}

void loop() {
    otUdp.beginPacket(allNodes, 7);
    otUdp.write((const uint8_t *) "hi", 2);
    otUdp.endPacket();

    if (int n = otUdp.parsePacket()) {
        char buf[64];
        int r = otUdp.read(buf, sizeof(buf) - 1);
        buf[r] = 0;
        Serial.printf("From [%s]:%u -> '%s'\r\n",
                      otUdp.remoteIP().toString().c_str(),
                      otUdp.remotePort(), buf);
    }

    delay(1000);
}

```

2.6 Zigbee APIs

2.6.1 Zigbee

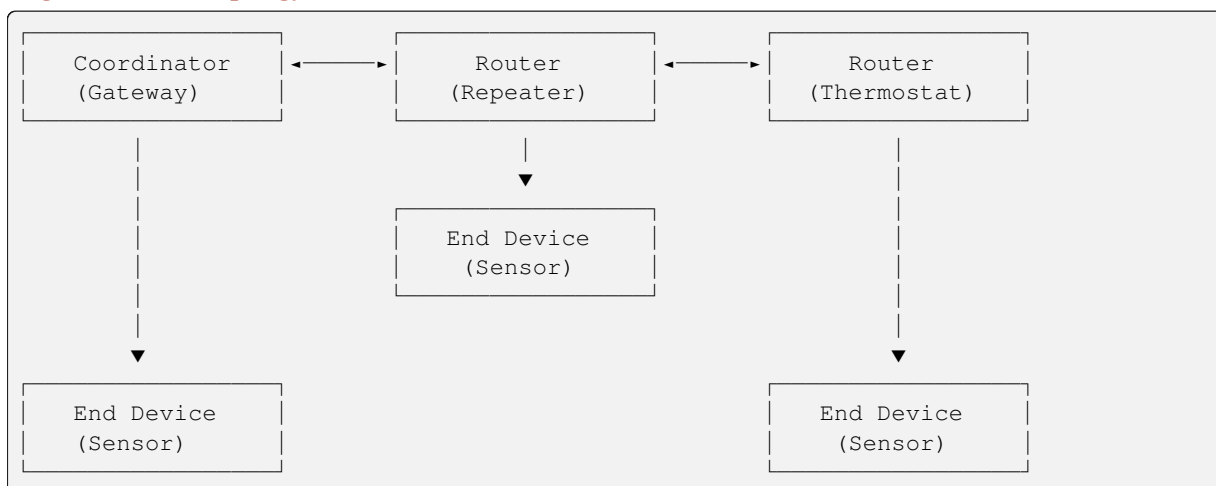
About

The Zigbee library provides support for creating Zigbee 3.0 compatible devices including:

- Support for different Zigbee roles (Coordinator, Router, End Device)
- Network management (scanning, joining, commissioning)
- Multiple endpoint types for various device categories
- OTA (Over-The-Air) update support
- Power management for battery-powered devices
- Time synchronization
- Advanced binding and group management

The Zigbee library is built on top of [ESP-ZIGBEE-SDK](#) and provides a high-level Arduino-style interface for creating Zigbee devices.

Zigbee Network Topology



Device Roles

- **Coordinator:** Forms and manages the network, stores network information
- **Router:** Extends network range, routes messages, mains powered devices (typically lights, switches, etc.)
- **End Device:** Battery-powered devices that can sleep for extended periods (typically sensors)

Zigbee Library Structure

The library is split into three main components:

- **ZigbeeCore:** The main class that manages the Zigbee network
- **ZigbeeEP:** The base class for all Zigbee endpoints, which provides common functionality for all endpoint types
- **Specific endpoint classes:** The classes for all Zigbee endpoints, which provides the specific functionality for each endpoint type

ZigbeeCore The `ZigbeeCore` class is the main entry point for all Zigbee operations. It serves as the central coordinator that manages:

- **Network Operations:** Starting, stopping, and managing the Zigbee network
- **Device Role Management:** Configuring the device as Coordinator, Router, or End Device
- **Endpoint Management:** Adding and managing multiple device endpoints
- **Network Discovery:** Scanning for and joining existing networks
- **OTA Updates:** Managing over-the-air firmware updates
- **Power Management:** Configuring sleep modes for battery-powered devices

The `ZigbeeCore` class is implemented as a singleton, meaning there's only one instance available globally. You access it directly as `Zigbee` without creating an instance.

ZigbeeCore

About The `ZigbeeCore` class is the main entry point for all Zigbee operations. It serves as the central class that manages:

- **Network Operations:** Starting, stopping, and managing the Zigbee network
- **Device Role Management:** Configuring the device as Coordinator, Router, or End Device
- **Endpoint Management:** Adding and managing multiple device endpoints
- **Network Discovery:** Scanning for and joining existing networks

ZigbeeCore APIs

Network Initialization

begin Initializes the Zigbee stack and starts the network.

```
bool begin(zigbee_role_t role = ZIGBEE_END_DEVICE, bool erase_nvs = false);
bool begin(esp_zb_cfg_t *role_cfg, bool erase_nvs = false);
```

- `role` - Device role (default: `ZIGBEE_END_DEVICE`)
- `role_cfg` - Custom role configuration structure
- `erase_nvs` - Whether to erase NVS storage (default: `false`)

This function will return `true` if initialization successful, `false` otherwise.

Available Roles:

- **ZIGBEE_COORDINATOR:** Network coordinator, forms and manages the network

- **ZIGBEE_ROUTER**: Network router, connects to existing network and extends network range and routes messages (if device is mains powered, always use this role)
- **ZIGBEE_END_DEVICE**: End device, connects to existing network (typically battery-powered which can sleep)

Note: Depending on the Zigbee role, proper Zigbee mode and partition scheme must be set in the Arduino IDE.

•**ZIGBEE_COORDINATOR and ZIGBEE_ROUTER:**

- Zigbee mode to Zigbee ZCZR (coordinator/router).
- Partition scheme to Zigbee ZCZR xMB with spiffs (where x is the number of MB of selected flash size).

•**ZIGBEE_END_DEVICE:**

- Zigbee mode to Zigbee ED (end device).
 - Partition scheme to Zigbee xMB with spiffs (where x is the number of MB of selected flash size).
-

start Starts the Zigbee stack again, if it was stopped by calling `stop()`.

```
void start();
```

stop Stops the Zigbee stack. This can be used after calling `begin()` to stop the Zigbee stack. Usage example is to save power or when you need the radio to be available for other tasks.

```
void stop();
```

Network Status

started Checks if the Zigbee stack has been started.

```
bool started();
```

This function will return `true` if Zigbee stack is running, `false` otherwise.

connected Checks if the device is connected to a Zigbee network.

```
bool connected();
```

This function will return `true` if connected to network, `false` otherwise.

getRole Gets the current Zigbee device role.

```
zigbee_role_t getRole();
```

This function will return current device role (`ZIGBEE_COORDINATOR`, `ZIGBEE_ROUTER`, `ZIGBEE_END_DEVICE`).

Endpoint Management

addEndpoint Adds an endpoint to the Zigbee network.

```
bool addEndpoint(ZigbeeEP *ep);
```

- `ep` - Pointer to the endpoint object to add

This function will return `true` if endpoint added successfully, `false` otherwise.

Network Configuration

setPrimaryChannelMask Sets the primary channel mask for network scanning and joining.

```
void setPrimaryChannelMask(uint32_t mask);
```

- `mask` - Channel mask (default: all channels 11-26, mask 0x07FFF800)

setScanDuration Sets the scan duration for network discovery.

```
void setScanDuration(uint8_t duration);
```

- `duration` - Scan duration (1-4, where 1 is fastest, 4 is slowest)

getScanDuration Gets the current scan duration setting.

```
uint8_t getScanDuration();
```

This function will return current scan duration (1-4).

Power Management

setRxOnWhenIdle Sets whether the device keeps its receiver on when idle.

```
void setRxOnWhenIdle(bool rx_on_when_idle);
```

- `rx_on_when_idle` - `true` to keep receiver on, `false` to allow sleep

getRxOnWhenIdle Gets the current receiver idle setting.

```
bool getRxOnWhenIdle();
```

This function will return current receiver idle setting.

setTimeout Sets the timeout for network operations.

```
void setTimeout(uint32_t timeout);
```

- `timeout` - Timeout in milliseconds (default: 30000 ms)

Network Discovery

scanNetworks Scans for available Zigbee networks.

```
void scanNetworks(uint32_t channel_mask = ESP_ZB_TRANSCEIVER_ALL_CHANNELS_MASK,   
↳uint8_t scan_duration = 5);
```

- channel_mask - Channels to scan (default: all channels)
- scan_duration - Scan duration (default: 5)

scanComplete Checks if network scanning is complete.

```
int16_t scanComplete();
```

This function will return: * -2: Scan failed or not started * -1: Scan running * 0: No networks found * >0: Number of networks found

getScanResult Gets the scan results.

```
zigbee_scan_result_t *getScanResult();
```

This function will return pointer to scan results, or NULL if no results.

scanDelete Deletes the scan results from memory.

```
void scanDelete();
```

Network Management (Coordinator only)

setRebootOpenNetwork Opens the network for joining after reboot for a specified time.

```
void setRebootOpenNetwork(uint8_t time);
```

- time - Time in seconds to keep network open after reboot

openNetwork Opens the network for device joining for a specified time.

```
void openNetwork(uint8_t time);
```

- time - Time in seconds to keep network open for device joining

closeNetwork Closes the network to prevent new devices from joining.

```
void closeNetwork();
```

Radio Configuration

setRadioConfig Sets the radio configuration.

```
void setRadioConfig(esp_zb_radio_config_t config);
```

- config - Radio configuration structure

getRadioConfig Gets the current radio configuration.

```
esp_zb_radio_config_t getRadioConfig();
```

This function will return current radio configuration.

Host Configuration

setHostConfig Sets the host configuration.

```
void setHostConfig(esp_zb_host_config_t config);
```

- config - Host configuration structure

getHostConfig Gets the current host configuration.

```
esp_zb_host_config_t getHostConfig();
```

This function will return current host configuration.

Debug and Utilities

setDebugMode Enables or disables debug mode.

```
void setDebugMode(bool debug);
```

- debug - true to enable debug output, false to disable

getDebugMode Gets the current debug mode setting.

```
bool getDebugMode();
```

This function will return current debug mode setting.

factoryReset Performs a factory reset, clearing all network settings.

```
void factoryReset(bool restart = true);
```

- restart - true to restart after reset (default: true)

onGlobalDefaultResponse Sets a global callback for default response messages.

```
void onGlobalDefaultResponse(void (*callback)(zb_cmd_type_t resp_to_cmd, esp_zb_  
↪zcl_status_t status, uint8_t endpoint, uint16_t cluster));
```

- callback - Function pointer to the callback function

This callback will be called for all endpoints when a default response is received.

Utility Functions

formatIEEEAddress Formats an IEEE address for display.

```
static const char *formatIEEEAddress(const esp_zb_ieee_addr_t addr);
```

- `addr` - IEEE address to format

This function will return formatted address string.

formatShortAddress Formats a short address for display.

```
static const char *formatShortAddress(uint16_t addr);
```

- `addr` - Short address to format

This function will return formatted address string.

ZigbeeEP The ZigbeeEP class is the base class for all Zigbee endpoints. It provides common functionality for all endpoint types.

- **Device Information:** Every endpoint can be configured with manufacturer and model information that helps identify the device on the network
- **Binding Management:** Binding allows endpoints to establish direct communication links with other devices. This enables automatic command transmission without requiring manual addressing
- **Power Management:** Endpoints can report their power source type and battery status, which helps the network optimize communication patterns
- **Time Synchronization:** Endpoints can synchronize with network time, enabling time-based operations and scheduling
- **OTA Support:** Endpoints can receive over-the-air firmware updates to add new features or fix bugs
- **Device Discovery:** Endpoints can read manufacturer and model information from other devices on the network

ZigbeeEP

About The ZigbeeEP class is the base class for all Zigbee endpoints. It provides common functionality for all endpoint types.

- **Device Information:** Every endpoint can be configured with manufacturer and model information that helps identify the device on the network
- **Binding Management:** Binding allows endpoints to establish direct communication links with other devices. This enables automatic command transmission without requiring manual addressing
- **Power Management:** Endpoints can report their power source type and battery status, which helps the network optimize communication patterns
- **Time Synchronization:** Endpoints can synchronize with network time, enabling time-based operations and scheduling
- **OTA Support:** Endpoints can receive over-the-air firmware updates to add new features or fix bugs
- **Device Discovery:** Endpoints can read manufacturer and model information from other devices on the network

ZigbeeEP APIs

Device Information

setManufacturerAndModel Sets the manufacturer name and model identifier for the device.

```
bool setManufacturerAndModel(const char *name, const char *model);
```

- `name` - Manufacturer name (max 32 characters)
- `model` - Model identifier (max 32 characters)

This function will return `true` if set successfully, `false` otherwise.

setVersion Sets the application version for the endpoint. The value is stored in the endpoint config and added to the Basic cluster (Application Version attribute). Must be called before `Zigbee.begin()`.

```
void setVersion(uint8_t version);
```

- `version` - Application version (0-255)

setHardwareVersion Sets the hardware version for the endpoint. The value is added to the Basic cluster (Hardware Version attribute). Must be called before `Zigbee.begin()`.

```
void setHardwareVersion(uint8_t version);
```

- `version` - Hardware version (0-255)

getEndpoint Gets the endpoint number assigned to this device.

```
uint8_t getEndpoint();
```

This function will return the endpoint number (1-254).

Binding Management

bound Checks if the endpoint has any bound devices.

```
bool bound();
```

This function will return `true` if the endpoint has bound devices, `false` otherwise.

getBoundDevices Gets the list of devices bound to this endpoint.

```
std::vector<esp_zb_binding_info_t> getBoundDevices();
```

This function will return list of bound device parameters.

printBoundDevices Prints information about bound devices to Serial or a custom Print object.

```
void printBoundDevices(Print &print = Serial);
```

- `print` - Custom Print object (optional, defaults to Serial)

allowMultipleBinding Enables or disables multiple device binding for this endpoint.

```
void allowMultipleBinding(bool bind);
```

- `bind` - `true` to allow multiple bindings, `false` for single binding only

setManualBinding Enables or disables manual binding mode. Manual binding mode is supposed to be used when using ZHA or Z2M where you bind devices manually.

```
void setManualBinding(bool bind);
```

- `bind` - `true` for manual binding, `false` for automatic binding (default)

clearBoundDevices Removes all bound devices from this endpoint.

```
void clearBoundDevices();
```

Binding Status

epAllowMultipleBinding Gets whether multiple device binding is allowed for this endpoint.

```
bool epAllowMultipleBinding();
```

This function will return `true` if multiple bindings are allowed, `false` otherwise.

epUseManualBinding Gets whether manual binding mode is enabled for this endpoint.

```
bool epUseManualBinding();
```

This function will return `true` if manual binding is enabled, `false` otherwise.

Power Management

setPowerSource Sets the power source type for the endpoint.

```
bool setPowerSource(uint8_t source, uint8_t percentage = 0xff, uint8_t voltage = 0xff);
```

- `source` - Power source type (ZB_POWER_SOURCE_MAINS, ZB_POWER_SOURCE_BATTERY, etc.)
- `percentage` - Battery percentage (0-100, default: 0xff)
- `voltage` - Battery voltage in 100 mV units (default: 0xff)

This function will return `true` if set successfully, `false` otherwise.

setBatteryPercentage Sets the current battery percentage.

```
bool setBatteryPercentage(uint8_t percentage);
```

- `percentage` - Battery percentage (0-100)

This function will return `true` if set successfully, `false` otherwise.

setBatteryVoltage Sets the battery voltage.

```
bool setBatteryVoltage(uint8_t voltage);
```

- `voltage` - Battery voltage in 100 mV units (e.g., 35 for 3.5 V)

This function will return `true` if set successfully, `false` otherwise.

reportBatteryPercentage Reports the current battery percentage to the network.

```
bool reportBatteryPercentage();
```

This function will return `true` if reported successfully, `false` otherwise.

Time Synchronization

addTimeCluster Adds time synchronization cluster to the endpoint. When you want to add a server cluster (have the time and GMT offset) fill the time structure with the current time and GMT offset. For client cluster (get the time and GMT offset) keep the default parameters.

```
bool addTimeCluster(tm time = {}, int32_t gmt_offset = 0);
```

- time - Current time structure (default: empty)
- gmt_offset - GMT offset in seconds (default: 0)

This function will return `true` if added successfully, `false` otherwise.

setTime Sets the current time for the endpoint.

```
bool setTime(tm time);
```

- time - Time structure to set

This function will return `true` if set successfully, `false` otherwise.

setTimezone Sets the timezone offset for the endpoint.

```
bool setTimezone(int32_t gmt_offset);
```

- gmt_offset - GMT offset in seconds

This function will return `true` if set successfully, `false` otherwise.

getTime Gets the current network time.

```
struct tm getTime(uint8_t endpoint = 1, int32_t short_addr = 0x0000, esp_zb_ieee_
↪addr_t ieee_addr = {});
```

- endpoint - Target endpoint (default: 1)
- short_addr - Target device short address (default: 0x0000)
- ieee_addr - Target device IEEE address (default: empty)

This function will return network time structure.

getTimezone Gets the timezone offset.

```
int32_t getTimezone(uint8_t endpoint = 1, int32_t short_addr = 0x0000, esp_zb_ieee_
↪addr_t ieee_addr = {});
```

- endpoint - Target endpoint (default: 1)
- short_addr - Target device short address (default: 0x0000)
- ieee_addr - Target device IEEE address (default: empty)

This function will return GMT offset in seconds.

OTA Support

addOTAClient Adds OTA client to the endpoint for firmware updates.

```
bool addOTAClient(uint32_t file_version, uint32_t downloaded_file_ver, uint16_t hw_
↪version, uint16_t manufacturer = 0x1001, uint16_t image_type = 0x1011, uint8_t
↪max_data_size = 223);
```

- file_version - Current firmware version

- `downloaded_file_ver` - Downloaded file version
- `hw_version` - Hardware version
- `manufacturer` - Manufacturer code (default: 0x1001)
- `image_type` - Image type code (default: 0x1011)
- `max_data_size` - Maximum data size for OTA transfer (default: 223)

This function will return `true` if added successfully, `false` otherwise.

requestOTAUpdate Requests OTA update from the server.

```
void requestOTAUpdate();
```

Device Discovery

readManufacturer Reads the manufacturer name from a remote device.

```
char *readManufacturer(uint8_t endpoint, uint16_t short_addr, esp_zb_ieee_addr_t  
↳ ieee_addr);
```

- `endpoint` - Target endpoint number
- `short_addr` - Target device short address
- `ieee_addr` - Target device IEEE address

This function will return pointer to manufacturer string, or `NULL` if read failed.

readModel Reads the model identifier from a remote device.

```
char *readModel(uint8_t endpoint, uint16_t short_addr, esp_zb_ieee_addr_t ieee_  
↳ addr);
```

- `endpoint` - Target endpoint number
- `short_addr` - Target device short address
- `ieee_addr` - Target device IEEE address

This function will return pointer to model string, or `NULL` if read failed.

Event Handling

onIdentify Sets a callback function for identify events.

```
void onIdentify(void (*callback) (uint16_t));
```

- `callback` - Function to call when identify event occurs
- `time` - Identify time in seconds

onDefaultResponse Sets a callback for default response messages for this endpoint.

```
void onDefaultResponse(void (*callback) (zb_cmd_type_t resp_to_cmd, esp_zb_zcl_  
↳ status_t status));
```

- `callback` - Function pointer to the callback function

This callback will be called when a default response is received for this specific endpoint.

Supported Endpoints The Zigbee library provides specialized endpoint classes for different device types. Each endpoint type includes specific clusters and functionality relevant to that device category.

ZigbeeAnalog

About The `ZigbeeAnalog` class provides analog input and output endpoints for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for analog signal processing and control. Analog Input (AI) is meant to be used for sensors that provide an analog signal, such as temperature, humidity, pressure to be sent to the coordinator. Analog Output (AO) is meant to be used for actuators that require an analog signal, such as dimmers, valves, etc. to be controlled by the coordinator.

Common API

Constructor

ZigbeeAnalog Creates a new Zigbee analog endpoint.

```
ZigbeeAnalog(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

Cluster Management

addAnalogInput Adds analog input cluster to the endpoint.

```
bool addAnalogInput();
```

This function will return `true` if successful, `false` otherwise.

addAnalogOutput Adds analog output cluster to the endpoint.

```
bool addAnalogOutput();
```

This function will return `true` if successful, `false` otherwise.

Analog Input API

Configuration Methods

setAnalogInputApplication Sets the application type for the analog input.

```
bool setAnalogInputApplication(uint32_t application_type);
```

- `application_type` - Application type constant (see `esp_zigbee_zcl_analog_input.h` for values)

This function will return `true` if successful, `false` otherwise.

setAnalogInputDescription Sets a custom description for the analog input.

```
bool setAnalogInputDescription(const char *description);
```

- `description` - Description string

This function will return `true` if successful, `false` otherwise.

setAnalogInputResolution Sets the resolution for the analog input.

```
bool setAnalogInputResolution(float resolution);
```

- resolution - Resolution value

This function will return `true` if successful, `false` otherwise.

setAnalogInputMinMax Sets the minimum and maximum values for the analog input.

```
bool setAnalogInputMinMax(float min, float max);
```

- min - Minimum value
- max - Maximum value

This function will return `true` if successful, `false` otherwise.

Value Control

setAnalogInput Sets the analog input value.

```
bool setAnalogInput(float analog);
```

- analog - Analog input value

This function will return `true` if successful, `false` otherwise.

Reporting Methods

setAnalogInputReporting Sets the reporting configuration for analog input.

```
bool setAnalogInputReporting(uint16_t min_interval, uint16_t max_interval, float_  
↪ delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in value to trigger a report

This function will return `true` if successful, `false` otherwise.

reportAnalogInput Manually reports the current analog input value.

```
bool reportAnalogInput();
```

This function will return `true` if successful, `false` otherwise.

Analog Output API

Configuration Methods

setAnalogOutputApplication Sets the application type for the analog output.

```
bool setAnalogOutputApplication(uint32_t application_type);
```

- application_type - Application type constant (see `esp_zigbee_zcl_analog_output.h` for values)

This function will return `true` if successful, `false` otherwise.

setAnalogOutputDescription Sets a custom description for the analog output.

```
bool setAnalogOutputDescription(const char *description);
```

- description - Description string

This function will return `true` if successful, `false` otherwise.

setAnalogOutputResolution Sets the resolution for the analog output.

```
bool setAnalogOutputResolution(float resolution);
```

- resolution - Resolution value

This function will return `true` if successful, `false` otherwise.

setAnalogOutputMinMax Sets the minimum and maximum values for the analog output.

```
bool setAnalogOutputMinMax(float min, float max);
```

- min - Minimum value
- max - Maximum value

This function will return `true` if successful, `false` otherwise.

Value Control

setAnalogOutput Sets the analog output value.

```
bool setAnalogOutput(float analog);
```

- analog - Analog output value

This function will return `true` if successful, `false` otherwise.

getAnalogOutput Gets the current analog output value.

```
float getAnalogOutput();
```

This function will return current analog output value.

Reporting Methods

reportAnalogOutput Manually reports the current analog output value.

```
bool reportAnalogOutput();
```

This function will return `true` if successful, `false` otherwise.

Event Handling

onAnalogOutputChange Sets a callback function to be called when the analog output value changes.

```
void onAnalogOutputChange(void (*callback)(float analog));
```

- callback - Function to call when analog output changes

Example**Analog Input/Output**

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee analog input / output device.
 *
 * The example demonstrates how to use Zigbee library to create a end device.
 * ↪ analog device.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition.
 * ↪ Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 * Modified by Pat Clay
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator/router device mode is not selected in Tools->Zigbee mode
↪ "
#endif

#include "Zigbee.h"

/* Zigbee analog device configuration */
#define ANALOG_DEVICE_ENDPOINT_NUMBER 1

uint8_t analogPin = A0;
uint8_t button = BOOT_PIN;

ZigbeeAnalog zbAnalogDevice = ZigbeeAnalog(ANALOG_DEVICE_ENDPOINT_NUMBER);
ZigbeeAnalog zbAnalogTemp = ZigbeeAnalog(ANALOG_DEVICE_ENDPOINT_NUMBER + 1);
ZigbeeAnalog zbAnalogFan = ZigbeeAnalog(ANALOG_DEVICE_ENDPOINT_NUMBER + 2);
ZigbeeAnalog zbAnalogPercent = ZigbeeAnalog(ANALOG_DEVICE_ENDPOINT_NUMBER + 3);

void onAnalogOutputChange(float analog_output) {
    Serial.printf("Received analog output change: %.1f\r\n", analog_output);
}

void setup() {
    Serial.begin(115200);
    Serial.println("Starting...");

    // Init button switch
```

(continues on next page)

(continued from previous page)

```

pinMode(button, INPUT_PULLUP);

// Set analog resolution to 10 bits
analogReadResolution(10);

// Optional: set Zigbee device name and model
zbAnalogDevice.setManufacturerAndModel("Espressif", "ZigbeeAnalogDevice");

// Set up analog input
zbAnalogDevice.addAnalogInput();
zbAnalogDevice.setAnalogInputApplication(ESP_ZB_ZCL_AI_POWER_IN_WATTS_
↪CONSUMPTION);
zbAnalogDevice.setAnalogInputDescription("Power Consumption (Watts)");
zbAnalogDevice.setAnalogInputResolution(0.01);

// Set up analog output
zbAnalogDevice.addAnalogOutput();
zbAnalogDevice.setAnalogOutputApplication(ESP_ZB_ZCL_AI_RPM_OTHER);
zbAnalogDevice.setAnalogOutputDescription("Fan Speed (RPM)");
zbAnalogDevice.setAnalogOutputResolution(1);

// Set the min and max values for the analog output which is used by HA to limit
↪the range of the analog output
zbAnalogDevice.setAnalogOutputMinMax(-10000, 10000); //-10000 to 10000 RPM

// If analog output cluster is added, set callback function for analog output
↪change
zbAnalogDevice.onAnalogOutputChange(onAnalogOutputChange);

// Set up analog input
zbAnalogTemp.addAnalogInput();
zbAnalogTemp.setAnalogInputApplication(ESP_ZB_ZCL_AI_TEMPERATURE_OTHER);
zbAnalogTemp.setAnalogInputDescription("Temperature");
zbAnalogTemp.setAnalogInputResolution(0.1);

// Set up analog input
zbAnalogFan.addAnalogInput();
zbAnalogFan.setAnalogInputApplication(ESP_ZB_ZCL_AI_RPM_OTHER);
zbAnalogFan.setAnalogInputDescription("RPM");
zbAnalogFan.setAnalogInputResolution(1);

// Set up analog input
zbAnalogPercent.addAnalogInput();
zbAnalogPercent.setAnalogInputApplication(ESP_ZB_ZCL_AI_PERCENTAGE_OTHER);
zbAnalogPercent.setAnalogInputDescription("Percentage");
zbAnalogPercent.setAnalogInputResolution(0.01);

// Add endpoints to Zigbee Core
Zigbee.addEndpoint(&zbAnalogDevice);
Zigbee.addEndpoint(&zbAnalogTemp);
Zigbee.addEndpoint(&zbAnalogFan);
Zigbee.addEndpoint(&zbAnalogPercent);

Serial.println("Starting Zigbee...");
// When all EPs are registered, start Zigbee in Router Device mode
if (!Zigbee.begin(ZIGBEE_ROUTER)) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
} else {
    Serial.println("Zigbee started successfully!");
}

```

(continues on next page)

(continued from previous page)

```

}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println("Connected");

// Optional: Add reporting for analog input
zbAnalogDevice.setAnalogInputReporting(0, 30, 10); // report every 30 seconds_
↳if value changes by 10
}

void loop() {
    static uint32_t timeCounter = 0;

    // Read ADC value and update the analog value every 2s
    if (!(timeCounter++ % 20)) { // delaying for 100ms x 20 = 2s
        float analog = (float)analogRead(analogPin);
        Serial.printf("Updating analog input to %.1f\r\n", analog);
        zbAnalogDevice.setAnalogInput(analog);
        zbAnalogTemp.setAnalogInput(analog / 100);
        zbAnalogFan.setAnalogInput(analog);
        zbAnalogPercent.setAnalogInput(analog / 10);

        // Analog input supports reporting
        zbAnalogDevice.reportAnalogInput();
        zbAnalogTemp.reportAnalogInput();
        zbAnalogFan.reportAnalogInput();
        zbAnalogPercent.reportAnalogInput();
    }

    // Checking button for factory reset and reporting
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        // For demonstration purposes, increment the analog output value by 100
        zbAnalogDevice.setAnalogOutput(zbAnalogDevice.getAnalogOutput() + 100);
        zbAnalogDevice.reportAnalogOutput();
    }
    delay(100);
}
}

```

ZigbeeBinary

About The ZigbeeBinary class provides an endpoint for binary input/output sensors in Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for binary sensors, supporting various application types for HVAC, security, and general binary sensing. Binary Input (BI) is meant to be used for sensors that provide a binary signal, such as door/window sensors, motion detectors, etc. to be sent to the network. Binary Output (BO)

is used for controlling binary devices such as relays, switches, or actuators that can be turned on/off remotely.

API Reference

Constructor

ZigbeeBinary Creates a new Zigbee binary sensor endpoint.

```
ZigbeeBinary(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Binary Input Application Types

HVAC Application Types

```
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_BOILER_STATUS    0x00000003
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_CHILLER_STATUS   0x00000013
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_OCCUPANCY        0x00000031
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_FAN_STATUS       0x00000035
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_FILTER_STATUS    0x00000036
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_HEATING_ALARM    0x0000003E
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_COOLING_ALARM    0x0000001D
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_UNIT_ENABLE      0x00000090
#define BINARY_INPUT_APPLICATION_TYPE_HVAC_OTHER            0x0000FFFF
```

Security Application Types

```
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_GLASS_BREAKAGE_DETECTION_0 0x01000000
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_INTRUSION_DETECTION         0x01000001
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_MOTION_DETECTION            0x01000002
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_GLASS_BREAKAGE_DETECTION_1 0x01000003
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_ZONE_ARMED                  0x01000004
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_GLASS_BREAKAGE_DETECTION_2 0x01000005
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_SMOKE_DETECTION             0x01000006
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_CARBON_DIOXIDE_DETECTION    0x01000007
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_HEAT_DETECTION              0x01000008
#define BINARY_INPUT_APPLICATION_TYPE_SECURITY_OTHER                       0x0100FFFF
```

Binary Output Application Types

HVAC Application Types


```
#define BINARY_OUTPUT_APPLICATION_TYPE_HVAC_BOILER      0x00000003
#define BINARY_OUTPUT_APPLICATION_TYPE_HVAC_CHILLER     0x0000000D
#define BINARY_OUTPUT_APPLICATION_TYPE_HVAC_FAN        0x00000022
#define BINARY_OUTPUT_APPLICATION_TYPE_HVAC_HEATING_VALVE 0x0000002C
#define BINARY_OUTPUT_APPLICATION_TYPE_HVAC_HUMIDIFIER  0x00000033
#define BINARY_OUTPUT_APPLICATION_TYPE_HVAC_PREHEAT    0x00000034
#define BINARY_OUTPUT_APPLICATION_TYPE_HVAC_OTHER      0x0000FFFF
```

Security Application Types

```
#define BINARY_OUTPUT_APPLICATION_TYPE_SECURITY_ARM_DISARM_COMMAND 0x01000000
#define BINARY_OUTPUT_APPLICATION_TYPE_SECURITY_OCCUPANCY_CONTROL  0x01000001
#define BINARY_OUTPUT_APPLICATION_TYPE_SECURITY_ENABLE_CONTROL     0x01000002
#define BINARY_OUTPUT_APPLICATION_TYPE_SECURITY_ACCESS_CONTROL     0x01000003
#define BINARY_OUTPUT_APPLICATION_TYPE_SECURITY_OTHER              0x0100FFFF
```

API Methods

addBinaryInput Adds a binary input cluster to the endpoint.

```
bool addBinaryInput();
```

This function will return `true` if successful, `false` otherwise.

setBinaryInputApplication Sets the application type for the binary input.

```
bool setBinaryInputApplication(uint32_t application_type);
```

- `application_type` - Application type constant (see above)

This function will return `true` if successful, `false` otherwise.

setBinaryInputDescription Sets a custom description for the binary input.

```
bool setBinaryInputDescription(const char *description);
```

- `description` - Description string

This function will return `true` if successful, `false` otherwise.

setBinaryInput Sets the binary input value.

```
bool setBinaryInput(bool input);
```

- `input` - Binary value (true/false)

This function will return `true` if successful, `false` otherwise.

reportBinaryInput Manually reports the current binary input value.

```
bool reportBinaryInput();
```

This function will return `true` if successful, `false` otherwise.

addBinaryOutput Adds a binary output cluster to the endpoint.

```
bool addBinaryOutput();
```

This function will return `true` if successful, `false` otherwise.

setBinaryOutputApplication Sets the application type for the binary output.

```
bool setBinaryOutputApplication(uint32_t application_type);
```

- `application_type` - Application type constant (see Binary Output Application Types above)

This function will return `true` if successful, `false` otherwise.

setBinaryOutputDescription Sets a custom description for the binary output.

```
bool setBinaryOutputDescription(const char *description);
```

- `description` - Description string

This function will return `true` if successful, `false` otherwise.

onBinaryOutputChange Sets a callback function to be called when the binary output value changes.

```
void onBinaryOutputChange(void (*callback) (bool binary_output));
```

- `callback` - Function pointer to callback that receives the new binary output value

setBinaryOutput Sets the binary output value.

```
bool setBinaryOutput(bool output);
```

- `output` - Binary value (true/false)

This function will return `true` if successful, `false` otherwise.

getBinaryOutput Gets the current binary output value.

```
bool getBinaryOutput();
```

This function returns the current binary output state.

reportBinaryOutput Manually reports the current binary output value.

```
bool reportBinaryOutput();
```

This function will return `true` if successful, `false` otherwise.

Example

Binary Input and Output Implementation

```

// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee binary input/output device.
 *
 * The example demonstrates how to use Zigbee library to create an end device_
↪binary sensor/switch device.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee binary sensor device configuration */
#define BINARY_DEVICE_ENDPOINT_NUMBER 1

uint8_t button = BOOT_PIN;

ZigbeeBinary zbBinaryFan = ZigbeeBinary(BINARY_DEVICE_ENDPOINT_NUMBER);
ZigbeeBinary zbBinaryZone = ZigbeeBinary(BINARY_DEVICE_ENDPOINT_NUMBER + 1);
ZigbeeBinary zbBinaryHumidifier = ZigbeeBinary(BINARY_DEVICE_ENDPOINT_NUMBER + 2);

bool zoneStatus = false;

void fanSwitch(bool state) {
    Serial.println("Fan switch changed to: " + String(state));
    // Switch Fan status input signaling the fan status has changed
    zbBinaryFan.setBinaryInput(state);
    zbBinaryFan.reportBinaryInput();
}

void humidifierSwitch(bool state) {
    Serial.println("Humidifier switch changed to: " + String(state));
}

void setup() {
    Serial.begin(115200);
    Serial.println("Starting...");
}

```

(continues on next page)

(continued from previous page)

```

// Init button switch
pinMode(button, INPUT_PULLUP);

// Set analog resolution to 10 bits
analogReadResolution(10);

// Optional: set Zigbee device name and model
zbBinaryFan.setManufacturerAndModel("Espressif", "ZigbeeBinarySensor");

// Set up binary fan status input + switch output (HVAC)
zbBinaryFan.addBinaryInput();
zbBinaryFan.setBinaryInputApplication(BINARY_INPUT_APPLICATION_TYPE_HVAC_FAN_
↪STATUS);
zbBinaryFan.setBinaryInputDescription("Fan Status");

zbBinaryFan.addBinaryOutput();
zbBinaryFan.setBinaryOutputApplication(BINARY_OUTPUT_APPLICATION_TYPE_HVAC_FAN);
zbBinaryFan.setBinaryOutputDescription("Fan Switch");

zbBinaryFan.onBinaryOutputChange(fanSwitch);

// Set up binary zone armed input (Security)
zbBinaryZone.addBinaryInput();
zbBinaryZone.setBinaryInputApplication(BINARY_INPUT_APPLICATION_TYPE_SECURITY_
↪ZONE_ARMED);
zbBinaryZone.setBinaryInputDescription("Zone Armed");

// Set up binary humidifier output (HVAC)
zbBinaryHumidifier.addBinaryOutput();
zbBinaryHumidifier.setBinaryOutputApplication(BINARY_OUTPUT_APPLICATION_TYPE_
↪HVAC_HUMIDIFIER);
zbBinaryHumidifier.setBinaryOutputDescription("Humidifier Switch");

zbBinaryHumidifier.onBinaryOutputChange(humidifierSwitch);

// Add endpoints to Zigbee Core
Zigbee.addEndpoint(&zbBinaryFan);
Zigbee.addEndpoint(&zbBinaryZone);
Zigbee.addEndpoint(&zbBinaryHumidifier);

Serial.println("Starting Zigbee...");
// When all EPs are registered, start Zigbee in End Device mode
if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
} else {
    Serial.println("Zigbee started successfully!");
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println("Connected");
}

void loop() {
    // Checking button for factory reset and reporting
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling

```

(continues on next page)

(continued from previous page)

```

delay(100);
int startTime = millis();
while (digitalRead(button) == LOW) {
    delay(50);
    if ((millis() - startTime) > 3000) {
        // If key pressed for more than 3secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        delay(1000);
        Zigbee.factoryReset();
    }
}

// Toggle fan
zbBinaryFan.setBinaryOutput(!zbBinaryFan.getBinaryOutput());
zbBinaryFan.reportBinaryOutput();

// Toggle zone
zoneStatus = !zoneStatus;
zbBinaryZone.setBinaryInput(zoneStatus);
zbBinaryZone.reportBinaryInput();

// Toggle humidifier
zbBinaryHumidifier.setBinaryOutput(!zbBinaryHumidifier.getBinaryOutput());
zbBinaryHumidifier.reportBinaryOutput();
}
delay(100);
}

```

ZigbeeCarbonDioxideSensor

About The `ZigbeeCarbonDioxideSensor` class provides a CO2 sensor endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for carbon dioxide measurement devices.

API Reference

Constructor

ZigbeeCarbonDioxideSensor Creates a new Zigbee CO2 sensor endpoint.

```
ZigbeeCarbonDioxideSensor(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

API Methods

setCarbonDioxide Sets the CO2 concentration measurement value.

```
bool setCarbonDioxide(float carbon_dioxide);
```

- `carbon_dioxide` - CO2 concentration value in ppm

This function will return `true` if successful, `false` otherwise.

setMinMaxValue Sets the minimum and maximum measurement values.

```
bool setMinMaxValue(float min, float max);
```

- min - Minimum CO2 concentration value in ppm
- max - Maximum CO2 concentration value in ppm

This function will return `true` if successful, `false` otherwise.

setDefaultValue Sets the default (initial) value for the carbon dioxide sensor in ppm. This value will be used as the initial measured value when the device is in factory reset mode and before the sensor provides actual readings.

```
bool setDefaultValue(float defaultValue);
```

- defaultValue - Default CO2 concentration value in ppm

Important: Must be called before adding the EP to Zigbee class. Only effective when the device is in factory reset mode (before commissioning/joining a network).

This function will return `true` if successful, `false` otherwise.

setTolerance Sets the tolerance value for measurements.

```
bool setTolerance(float tolerance);
```

- tolerance - Tolerance value in ppm

This function will return `true` if successful, `false` otherwise.

setReporting Sets the reporting configuration for CO2 measurements.

```
bool setReporting(uint16_t min_interval, uint16_t max_interval, uint16_t delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change required to trigger a report in ppm

Note: Delta reporting is currently not supported by the carbon dioxide sensor.

This function will return `true` if successful, `false` otherwise.

report Manually reports the current CO2 concentration value.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

Example

CO2 Sensor Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
```

(continues on next page)

(continued from previous page)

```

// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee carbon dioxide sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device.
↪carbon dioxide sensor.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition.
↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee carbon dioxide sensor configuration */
#define CARBON_DIOXIDE_SENSOR_ENDPOINT_NUMBER 10
uint8_t button = BOOT_PIN;

ZigbeeCarbonDioxideSensor zbCarbonDioxideSensor = ZigbeeCarbonDioxideSensor(CARBON_
↪DIOXIDE_SENSOR_ENDPOINT_NUMBER);

void setup() {
  Serial.begin(115200);

  // Init button switch
  pinMode(button, INPUT_PULLUP);

  // Optional: set Zigbee device name and model
  zbCarbonDioxideSensor.setManufacturerAndModel("Espressif",
↪"ZigbeeCarbonDioxideSensor");

  // Set minimum and maximum carbon dioxide measurement value in ppm
  zbCarbonDioxideSensor.setMinMaxValue(0, 1500);

  // Add endpoints to Zigbee Core
  Zigbee.addEndpoint(&zbCarbonDioxideSensor);

  Serial.println("Starting Zigbee...");
  // When all EPs are registered, start Zigbee in End Device mode
  if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
  } else {
    Serial.println("Zigbee started successfully!");
  }
  Serial.println("Connecting to network");
}

```

(continues on next page)

(continued from previous page)

```

while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();

// Set reporting interval for carbon dioxide measurement to be done every 30
seconds, must be called after Zigbee.begin()
// min_interval and max_interval in seconds, delta (carbon dioxide change in ppm)
// if min = 1 and max = 0, reporting is sent only when carbon dioxide changes by
delta
// if min = 0 and max = 10, reporting is sent every 10 seconds or when carbon
dioxide changes by delta
// if min = 0, max = 10 and delta = 0, reporting is sent every 10 seconds
regardless of delta change
zbCarbonDioxideSensor.setReporting(0, 30, 0);
}

void loop() {
    static uint32_t timeCounter = 0;
    // Read carbon dioxide sensor every 2s
    if (!(timeCounter++ % 20)) { // delaying for 100ms x 20 = 2s
        // Read sensor value - here is chip temperature used + 300 as a dummy value
        for demonstration
        uint16_t carbon_dioxide_value = 300 + (uint16_t)temperatureRead();
        Serial.printf("Updating carbon dioxide sensor value to %u ppm\r\n", carbon_
        dioxide_value);
        zbCarbonDioxideSensor.setCarbonDioxide(carbon_dioxide_value);
    }

    // Checking button for factory reset and reporting
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        zbCarbonDioxideSensor.report();
    }
    delay(100);
}
}

```

ZigbeeColorDimmableLight

About The ZigbeeColorDimmableLight class provides an endpoint for color dimmable lights in Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for color lighting devices, supporting multiple color modes (RGB/XY, HSV, and Color Temperature), dimming, and scene management.

Features: * On/off control * Brightness level control (0-255) * RGB/XY color control * HSV (Hue/Saturation) color support * Color temperature (mireds) support * Configurable color capabilities (enable/disable color modes) * Separate callbacks for RGB, HSV, and Temperature modes * Automatic color mode switching * Scene and group support * Automatic state restoration * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Smart RGB light bulbs * Color-changing LED strips * Tunable white light bulbs * Full-spectrum color temperature lights * Mood lighting systems * Entertainment lighting * Architectural lighting * Smart home color lighting

API Reference

Constructor

ZigbeeColorDimmableLight Creates a new Zigbee color dimmable light endpoint.

```
ZigbeeColorDimmableLight(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Color Capabilities

setLightColorCapabilities Configures which color modes are supported by the light. Must be called before starting Zigbee. By default, only XY (RGB) mode is enabled.

```
bool setLightColorCapabilities(uint16_t capabilities);
```

- capabilities - Bit flags indicating supported color modes (can be combined with bitwise OR):
 - ZIGBEE_COLOR_CAPABILITY_HUE_SATURATION - Hue/Saturation support
 - ZIGBEE_COLOR_CAPABILITY_X_Y - XY (RGB) support
 - ZIGBEE_COLOR_CAPABILITY_COLOR_TEMP - Color temperature support
 - ZIGBEE_COLOR_CAPABILITY_ENHANCED_HUE - Enhanced hue support
 - ZIGBEE_COLOR_CAPABILITY_COLOR_LOOP - Color loop support

Example:

```
// Enable XY and Temperature modes
light.setLightColorCapabilities(
    ZIGBEE_COLOR_CAPABILITY_X_Y | ZIGBEE_COLOR_CAPABILITY_COLOR_TEMP
);

// Enable all color modes
light.setLightColorCapabilities(
    ZIGBEE_COLOR_CAPABILITY_X_Y |
    ZIGBEE_COLOR_CAPABILITY_HUE_SATURATION |
    ZIGBEE_COLOR_CAPABILITY_COLOR_TEMP
);
```

This function will return true if successful, false otherwise.

Callback Functions

Callback Type Definitions For better type safety and readability, typedefs are provided for all callback functions:

```
typedef void (*ZigbeeColorLightRgbCallback)(bool state, uint8_t red, uint8_t green,
↪ uint8_t blue, uint8_t level);
typedef void (*ZigbeeColorLightHsvCallback)(bool state, uint8_t hue, uint8_t ↪
↪ saturation, uint8_t value);
typedef void (*ZigbeeColorLightTempCallback)(bool state, uint8_t level, uint16_t ↪
↪ color_temperature);
```

These typedefs can be used instead of raw function pointer syntax for better code clarity.

onLightChange Deprecated since version This: method is deprecated and will be removed in a future major version. Use `onLightChangeRgb()` instead.

Sets the legacy callback function for light state changes (RGB mode).

```
void onLightChange(void (*callback) (bool, uint8_t, uint8_t, uint8_t, uint8_t));
```

- callback - Function pointer to the light change callback (state, red, green, blue, level)
 - state - Light state (true = on, false = off)
 - red - Red component (0-255)
 - green - Green component (0-255)
 - blue - Blue component (0-255)
 - level - Brightness level (0-255)

Note: This method is deprecated. Please use `onLightChangeRgb()` for RGB/XY mode callbacks.

onLightChangeRgb Sets the callback function for RGB/XY color mode changes.

```
void onLightChangeRgb(ZigbeeColorLightRgbCallback callback);  
// or using raw function pointer syntax:  
void onLightChangeRgb(void (*callback) (bool, uint8_t, uint8_t, uint8_t, uint8_t));
```

- callback - Function pointer to the RGB light change callback (state, red, green, blue, level)
 - state - Light state (true = on, false = off)
 - red - Red component (0-255)
 - green - Green component (0-255)
 - blue - Blue component (0-255)
 - level - Brightness level (0-255)

onLightChangeHsv Sets the callback function for HSV (Hue/Saturation) color mode changes.

```
void onLightChangeHsv(ZigbeeColorLightHsvCallback callback);  
// or using raw function pointer syntax:  
void onLightChangeHsv(void (*callback) (bool, uint8_t, uint8_t, uint8_t));
```

- callback - Function pointer to the HSV light change callback (state, hue, saturation, value)
 - state - Light state (true = on, false = off)
 - hue - Hue component (0-254)
 - saturation - Saturation component (0-254)
 - value - Value/brightness component (0-255)

onLightChangeTemp Sets the callback function for color temperature mode changes.

```
void onLightChangeTemp(ZigbeeColorLightTempCallback callback);  
// or using raw function pointer syntax:  
void onLightChangeTemp(void (*callback) (bool, uint8_t, uint16_t));
```

- callback - Function pointer to the temperature light change callback (state, level, temperature_mireds)
 - state - Light state (true = on, false = off)
 - level - Brightness level (0-255)
 - temperature_mireds - Color temperature in mireds (inverse of Kelvin)

Control Methods

setLightState Sets the light on/off state.

```
bool setLightState(bool state);
```

- state - Light state (true = on, false = off)

This function will return `true` if successful, `false` otherwise.

setLightLevel Sets the light brightness level.

```
bool setLightLevel(uint8_t level);
```

- level - Brightness level (0-255, where 0 is off, 255 is full brightness)

This function will return `true` if successful, `false` otherwise.

setLightColor (RGB) Sets the light color using RGB values. Requires `ZIGBEE_COLOR_CAPABILITY_X_Y` capability to be enabled.

```
bool setLightColor(uint8_t red, uint8_t green, uint8_t blue);  
bool setLightColor(espRgbColor_t rgb_color);
```

- red - Red component (0-255)
- green - Green component (0-255)
- blue - Blue component (0-255)
- rgb_color - RGB color structure

This function will return `true` if successful, `false` otherwise. Returns `false` if XY capability is not enabled.

setLightColor (HSV) Sets the light color using HSV values. Requires `ZIGBEE_COLOR_CAPABILITY_HUE_SATURATION` capability to be enabled.

```
bool setLightColor(espHsvColor_t hsv_color);
```

- hsv_color - HSV color structure

This function will return `true` if successful, `false` otherwise. Returns `false` if HSV capability is not enabled.

setLightColorTemperature Sets the light color temperature in mireds. Requires `ZIGBEE_COLOR_CAPABILITY_COLOR_TEMP` capability to be enabled.

```
bool setLightColorTemperature(uint16_t color_temperature);
```

- color_temperature - Color temperature in mireds (inverse of Kelvin: mireds = 1000000 / Kelvin)

Example:

```
// Set to 4000K (cool white)  
uint16_t mireds = 1000000 / 4000; // = 250 mireds  
light.setLightColorTemperature(mireds);  
  
// Set to 2700K (warm white)  
mireds = 1000000 / 2700; // = 370 mireds  
light.setLightColorTemperature(mireds);
```

This function will return `true` if successful, `false` otherwise. Returns `false` if color temperature capability is not enabled.

setLightColorTemperatureRange Sets the minimum and maximum color temperature range supported by the hardware.

```
bool setLightColorTemperatureRange(uint16_t min_temp, uint16_t max_temp);
```

- min_temp - Minimum color temperature in mireds
- max_temp - Maximum color temperature in mireds

Example:

```
// Set range for 2000K (warm) to 6500K (cool)
uint16_t min_mireds = 1000000 / 6500; // = 154 mireds
uint16_t max_mireds = 1000000 / 2000; // = 500 mireds
light.setLightColorTemperatureRange(min_mireds, max_mireds);
```

This function will return `true` if successful, `false` otherwise.

setLight Sets all light parameters at once (RGB mode). Requires `ZIGBEE_COLOR_CAPABILITY_X_Y` capability to be enabled.

```
bool setLight(bool state, uint8_t level, uint8_t red, uint8_t green, uint8_t blue);
```

- state - Light state (true/false)
- level - Brightness level (0-255)
- red - Red component (0-255)
- green - Green component (0-255)
- blue - Blue component (0-255)

This function will return `true` if successful, `false` otherwise. Returns `false` if XY capability is not enabled.

State Retrieval Methods

getLightState Gets the current light state.

```
bool getLightState();
```

This function will return current light state (`true` = on, `false` = off).

getLightLevel Gets the current brightness level.

```
uint8_t getLightLevel();
```

This function will return current brightness level (0-255).

getLightColor Gets the current RGB color.

```
espRgbColor_t getLightColor();
```

This function will return current RGB color structure.

getLightRed Gets the current red component.

```
uint8_t getLightRed();
```

This function will return current red component (0-255).

getLightGreen Gets the current green component.

```
uint8_t getLightGreen();
```

This function will return current green component (0-255).

getLightBlue Gets the current blue component.

```
uint8_t getLightBlue();
```

This function will return current blue component (0-255).

getLightColorTemperature Gets the current color temperature.

```
uint16_t getLightColorTemperature();
```

This function will return current color temperature in mireds.

getLightColorMode Gets the current active color mode.

```
uint8_t getLightColorMode();
```

This function will return current color mode: * ZIGBEE_COLOR_MODE_HUE_SATURATION (0x00) - HSV mode * ZIGBEE_COLOR_MODE_CURRENT_X_Y (0x01) - XY/RGB mode * ZIGBEE_COLOR_MODE_TEMPERATURE (0x02) - Temperature mode

getLightColorHue Gets the current hue value (HSV mode).

```
uint8_t getLightColorHue();
```

This function will return current hue value (0-254).

getLightColorSaturation Gets the current saturation value (HSV mode).

```
uint8_t getLightColorSaturation();
```

This function will return current saturation value (0-254).

getLightColorCapabilities Gets the currently configured color capabilities.

```
uint16_t getLightColorCapabilities();
```

This function will return the current color capabilities bit flags.

Utility Methods

restoreLight Restores the light to its last known state. Uses the appropriate callback based on the current color mode.

```
void restoreLight();
```

Color Modes and Automatic Behavior The `ZigbeeColorDimmableLight` class supports three color modes:

- **XY/RGB Mode** (`ZIGBEE_COLOR_MODE_CURRENT_X_Y`) - Uses X/Y coordinates for color representation, internally converted to RGB
- **HSV Mode** (`ZIGBEE_COLOR_MODE_HUE_SATURATION`) - Uses Hue and Saturation values directly, without RGB conversion
- **Temperature Mode** (`ZIGBEE_COLOR_MODE_TEMPERATURE`) - Uses color temperature in mireds

Automatic Mode Switching:

The color mode is automatically updated based on which attributes are set:

- Setting RGB colors via `setLight()` or `setLightColor()` (RGB) → switches to XY mode
- Setting HSV colors via `setLightColor()` (HSV) → switches to HSV mode
- Setting temperature via `setLightColorTemperature()` → switches to Temperature mode
- Receiving Zigbee commands for XY/HSV/TEMP attributes → automatically switches to the corresponding mode

Capability Validation:

All set methods validate that the required capability is enabled before allowing the operation:

- RGB/XY methods require `ZIGBEE_COLOR_CAPABILITY_X_Y`
- HSV methods require `ZIGBEE_COLOR_CAPABILITY_HUE_SATURATION`
- Temperature methods require `ZIGBEE_COLOR_CAPABILITY_COLOR_TEMP`

If a capability is not enabled, the method will return `false` and log an error.

Callback Selection:

The appropriate callback is automatically called based on the current color mode:

- RGB/XY mode → `onLightChangeRgb()` callback
- HSV mode → `onLightChangeHsv()` callback
- Temperature mode → `onLightChangeTemp()` callback

When level or state changes occur, the callback for the current color mode is used automatically.

Note: The legacy `onLightChange()` callback is deprecated and will be removed in a future major version. Always use the mode-specific callbacks (`onLightChangeRgb()`, `onLightChangeHsv()`, or `onLightChangeTemp()`).

Example

Color Dimmable Light Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
```

(continues on next page)

(continued from previous page)

```

* @brief This example demonstrates Zigbee Color Dimmable light bulb with RGB and
* Temperature support.
*
* The example demonstrates how to use Zigbee library to create an end device with
* color dimmable light end point supporting both RGB (X/Y) and Color Temperature
* modes.
* The light bulb is a Zigbee end device, which is controlled by a Zigbee
* coordinator.
*
* Proper Zigbee mode must be selected in Tools->Zigbee mode
* and also the correct partition scheme must be selected in Tools->Partition
* Scheme.
*
* Please check the README.md for instructions and more detailed description.
*
* Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
*/

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee color dimmable light configuration */
#define ZIGBEE_RGB_LIGHT_ENDPOINT 10
uint8_t led = RGB_BUILTIN;
uint8_t button = BOOT_PIN;

ZigbeeColorDimmableLight zbColorLight = ZigbeeColorDimmableLight(ZIGBEE_RGB_LIGHT_
ENDPOINT);

/***** Temperature conversion functions *****/
uint16_t kelvinToMireds(uint16_t kelvin) {
    return 1000000 / kelvin;
}

uint16_t miredsToKelvin(uint16_t mireds) {
    return 1000000 / mireds;
}

/***** RGB LED functions *****/
void setRGBLight(bool state, uint8_t red, uint8_t green, uint8_t blue, uint8_t
level) {
    if (!state) {
        rgbLedWrite(led, 0, 0, 0);
        return;
    }
    float brightness = (float)level / 255;
    rgbLedWrite(led, red * brightness, green * brightness, blue * brightness);
}

/***** Temperature LED functions *****/
void setTempLight(bool state, uint8_t level, uint16_t mireds) {
    if (!state) {
        rgbLedWrite(led, 0, 0, 0);
        return;
    }
    float brightness = (float)level / 255;
    // Convert mireds to color temperature (K) and map to white/yellow

```

(continues on next page)

(continued from previous page)

```

uint16_t kelvin = miredsToKelvin(mireds);
uint8_t warm = constrain(map(kelvin, 2000, 6500, 255, 0), 0, 255);
uint8_t cold = constrain(map(kelvin, 2000, 6500, 0, 255), 0, 255);
rgbLedWrite(led, warm * brightness, warm * brightness, cold * brightness);
}

// Create a task on identify call to handle the identify function
void identify(uint16_t time) {
    static uint8_t blink = 1;
    log_d("Identify called for %u seconds", time);
    if (time == 0) {
        // If identify time is 0, stop blinking and restore light as it was used for
        ↪identify
        zbColorLight.restoreLight();
        return;
    }
    rgbLedWrite(led, 255 * blink, 255 * blink, 255 * blink);
    blink = !blink;
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init RMT and leave light OFF
    rgbLedWrite(led, 0, 0, 0);

    // Init button for factory reset
    pinMode(button, INPUT_PULLUP);

    // Enable both XY (RGB) and Temperature color capabilities
    uint16_t capabilities = ZIGBEE_COLOR_CAPABILITY_X_Y | ZIGBEE_COLOR_CAPABILITY_
    ↪COLOR_TEMP;
    zbColorLight.setLightColorCapabilities(capabilities);

    // Set callback functions for RGB and Temperature modes
    zbColorLight.onLightChangeRgb(setRGBLight);
    zbColorLight.onLightChangeTemp(setTempLight);

    // Optional: Set callback function for device identify
    zbColorLight.onIdentify(identify);

    // Optional: Set Zigbee device name and model
    zbColorLight.setManufacturerAndModel("Espressif", "ZBColorLightBulb");

    // Set min/max temperature range (High Kelvin -> Low Mireds: Min and Max is
    ↪switched)
    zbColorLight.setLightColorTemperatureRange(kelvinToMireds(6500),
    ↪kelvinToMireds(2000));

    // Add endpoint to Zigbee Core
    Serial.println("Adding ZigbeeLight endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbColorLight);

    // When all EPs are registered, start Zigbee in End Device mode
    if (!Zigbee.begin()) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }
    Serial.println("Connecting to network");
}

```

(continues on next page)

(continued from previous page)

```

while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();
}

void loop() {
    // Checking button for factory reset
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        // Increase blightness by 50 every time the button is pressed
        zbColorLight.setLightLevel(zbColorLight.getLightLevel() + 50);
    }
    delay(100);
}

```

ZigbeeColorDimmerSwitch

About The ZigbeeColorDimmerSwitch class provides a color dimmer switch endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for advanced lighting control switches that can control both dimming and color of lights.

Features: * On/off control for bound lights * Brightness level control (0-100%) * Color control (RGB, HSV) * Color temperature control * Scene and group support * Special effects and timed operations * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Smart lighting switches * Color control remotes * Advanced lighting controllers * Smart home lighting automation * Entertainment lighting control

API Reference

Constructor

ZigbeeColorDimmerSwitch Creates a new Zigbee color dimmer switch endpoint.

```
ZigbeeColorDimmerSwitch(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Basic Control Commands

lightToggle Toggles the state of bound lights (on to off, or off to on).

```
void lightToggle();
void lightToggle(uint16_t group_addr);
void lightToggle(uint8_t endpoint, uint16_t short_addr);
void lightToggle(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

lightOn Turns on bound lights.

```
void lightOn();
void lightOn(uint16_t group_addr);
void lightOn(uint8_t endpoint, uint16_t short_addr);
void lightOn(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

lightOff Turns off bound lights.

```
void lightOff();
void lightOff(uint16_t group_addr);
void lightOff(uint8_t endpoint, uint16_t short_addr);
void lightOff(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

Dimmer Control Commands

setLightLevel Sets the brightness level of bound lights.

```
void setLightLevel(uint8_t level);
void setLightLevel(uint8_t level, uint16_t group_addr);
void setLightLevel(uint8_t level, uint8_t endpoint, uint16_t short_addr);
void setLightLevel(uint8_t level, uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- level - Brightness level (0-100, where 0 is off, 100 is full brightness)
- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

setLightLevelStep Sends a level step command to bound lights: changes the current level by a fixed step size in the given direction.

```

void setLightLevelStep(ZigbeeLevelStepDirection direction, uint8_t step_size,
↳uint16_t transition_time);
void setLightLevelStep(ZigbeeLevelStepDirection direction, uint8_t step_size,
↳uint16_t transition_time, uint16_t group_addr);
void setLightLevelStep(ZigbeeLevelStepDirection direction, uint8_t step_size,
↳uint16_t transition_time, uint8_t endpoint, uint16_t short_addr);
void setLightLevelStep(ZigbeeLevelStepDirection direction, uint8_t step_size,
↳uint16_t transition_time, uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);

```

- direction - ZIGBEE_LEVEL_STEP_UP (0) or ZIGBEE_LEVEL_STEP_DOWN (1)
- step_size - Number of level units to step (e.g. 1–254; one step = one unit change in CurrentLevel)
- transition_time - Time to perform the step, in tenths of a second (see below)

Transition time (ZCL Step command):

- Transition time is in tenths of a second (1 unit = 100 ms). It is the time the device should take to perform the step. A step is a change in CurrentLevel of step_size units.
- The device should take as close to this time as it is able. If the device cannot move at a variable rate, it may disregard the transition time and move at a fixed rate.
- Use 0xFFFF (65535) to request the device to move **as fast as it is able** (no specific duration).

Examples:

- Step level up by 10 units over 1 second: `setLightLevelStep(ZIGBEE_LEVEL_STEP_UP, 10, 10);`
- Step level down by 5 units as fast as possible: `setLightLevelStep(ZIGBEE_LEVEL_STEP_DOWN, 5, 0xFFFF);`

Color Control Commands

setLightColor Sets the color of bound lights using RGB values.

```

void setLightColor(uint8_t red, uint8_t green, uint8_t blue);
void setLightColor(uint8_t red, uint8_t green, uint8_t blue, uint16_t group_addr);
void setLightColor(uint8_t red, uint8_t green, uint8_t blue, uint8_t endpoint,
↳uint16_t short_addr);
void setLightColor(uint8_t red, uint8_t green, uint8_t blue, uint8_t endpoint, esp_
↳zb_ieee_addr_t ieee_addr);

```

- red - Red component (0-255)
- green - Green component (0-255)
- blue - Blue component (0-255)
- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

Advanced Control Commands

lightOffWithEffect Turns off lights with a specific effect.

```

void lightOffWithEffect(uint8_t effect_id, uint8_t effect_variant);

```

- effect_id - Effect identifier
- effect_variant - Effect variant

lightOnWithTimedOff Turns on lights with automatic turn-off after specified time.

```
void lightOnWithTimedOff(uint8_t on_off_control, uint16_t time_on, uint16_t time_
↳ off);
```

- on_off_control - Control byte
- time_on - Time to stay on (in 1/10th seconds)
- time_off - Time to stay off (in 1/10th seconds)

lightOnWithSceneRecall Turns on lights with scene recall.

```
void lightOnWithSceneRecall();
```

Example

Color Dimmer Switch Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee color dimmer switch.
 *
 * The example demonstrates how to use Zigbee library to control a RGB light bulb.
 * The RGB light bulb is a Zigbee end device, which is controlled by a Zigbee_
↳ coordinator (Switch).
 * To turn on/off the light, push the button on the switch.
 * To change the color, level, or step the level of the light, send serial_
↳ commands to the switch.
 *
 * By setting the switch to allow multiple binding, so it can bind to multiple_
↳ lights.
 * Also every 30 seconds, all bound lights are printed to the serial console.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↳ Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"
```

(continues on next page)

(continued from previous page)

```

/* Zigbee color dimmer switch configuration */
#define SWITCH_ENDPOINT_NUMBER 5
uint8_t button = BOOT_PIN;

/* Zigbee switch */
ZigbeeColorDimmerSwitch zbSwitch = ZigbeeColorDimmerSwitch(SWITCH_ENDPOINT_NUMBER);

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    //Init button switch
    pinMode(button, INPUT_PULLUP);

    //Optional: set Zigbee device name and model
    zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");

    //Optional to allow multiple light to bind to the switch
    zbSwitch.allowMultipleBinding(true);

    //Add endpoint to Zigbee Core
    Zigbee.addEndpoint(&zbSwitch);

    //Open network for 180 seconds after boot
    Zigbee.setRebootOpenNetwork(180);

    //When all EPs are registered, start Zigbee with ZIGBEE_COORDINATOR mode
    if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }

    Serial.println("Waiting for Light to bound to the switch");
    //Wait for switch to bound to a light:
    while (!zbSwitch.bound()) {
        Serial.printf(".");
        delay(500);
    }
    Serial.println();
}

void loop() {
    // Handle button switch in loop()
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        while (digitalRead(button) == LOW) {
            delay(50);
        }
        // Toggle light
        zbSwitch.lightToggle();
    }
    // Handle serial input to control color and level of the light
    if (Serial.available()) {
        String command = Serial.readString();

        if (command == "on") {
            zbSwitch.lightOn();
        } else if (command == "off") {
            zbSwitch.lightOff();
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

} else if (command == "toggle") {
    zbSwitch.lightToggle();
} else if (command == "red") {
    zbSwitch.setLightColor(255, 0, 0);
} else if (command == "green") {
    zbSwitch.setLightColor(0, 255, 0);
} else if (command == "blue") {
    zbSwitch.setLightColor(0, 0, 255);
} else if (command == "white") {
    zbSwitch.setLightColor(255, 255, 255);
} else if (command == "color") {
    //wait for color value
    Serial.println("Enter red value (0-255):");
    while (!Serial.available()) {
        delay(100);
    }
    int red = Serial.parseInt();
    Serial.println("Enter green value (0-255):");
    while (!Serial.available()) {
        delay(100);
    }
    int green = Serial.parseInt();
    Serial.println("Enter blue value (0-255):");
    while (!Serial.available()) {
        delay(100);
    }
    int blue = Serial.parseInt();
    zbSwitch.setLightColor(red, green, blue);
} else if (command == "level") {
    //wait for level value
    Serial.println("Enter level value (0-255):");
    while (!Serial.available()) {
        delay(100);
    }
    int level = Serial.parseInt();
    zbSwitch.setLightLevel(level);
} else if (command == "stepup") {
    // Step level up by 20 units over 1 second
    zbSwitch.setLightLevelStep(ZIGBEE_LEVEL_STEP_UP, 20, 10);
    Serial.println("Step level up");
} else if (command == "stepdown") {
    // Step level down by 20 units over 1 second
    zbSwitch.setLightLevelStep(ZIGBEE_LEVEL_STEP_DOWN, 20, 10);
    Serial.println("Step level down");
} else if (command == "stepupfast") {
    // Step level up by 10 units as fast as possible (transition_time 0xFFFF)
    zbSwitch.setLightLevelStep(ZIGBEE_LEVEL_STEP_UP, 10, 0xFFFF);
    Serial.println("Step level up (fast)");
} else if (command == "stepdownfast") {
    // Step level down by 10 units as fast as possible
    zbSwitch.setLightLevelStep(ZIGBEE_LEVEL_STEP_DOWN, 10, 0xFFFF);
    Serial.println("Step level down (fast)");
} else if (command == "help") {
    Serial.println("Commands: on, off, toggle, red, green, blue, white, color, ↵
↵level, stepup, stepdown, stepupfast, stepdownfast");
} else {
    Serial.println("Unknown command (type 'help' for list)");
}
}

// print the bound devices (lights) every 30 seconds

```

(continues on next page)

(continued from previous page)

```
static uint32_t last_print = 0;
if (millis() - last_print > 30000) {
    last_print = millis();
    zbSwitch.printBoundDevices(Serial);
}
}
```

ZigbeeContactSwitch

About The ZigbeeContactSwitch class provides a contact switch endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for door/window contact sensors and other binary contact devices.

Features: * Contact state detection (open/closed) * Configurable application types * Automatic reporting capabilities * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Door and window sensors * Security system contacts * Cabinet and drawer sensors * Industrial contact monitoring * Smart home security applications

API Reference

Constructor

ZigbeeContactSwitch Creates a new Zigbee contact switch endpoint.

```
ZigbeeContactSwitch(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

API Methods

setClosed Sets the contact switch to closed state.

```
bool setClosed();
```

This function will return `true` if successful, `false` otherwise.

setOpen Sets the contact switch to open state.

```
bool setOpen();
```

This function will return `true` if successful, `false` otherwise.

report Manually reports the current contact state.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

setIASClientEndpoint Sets the IAS Client endpoint number (default is 1).

```
void setIASClientEndpoint(uint8_t ep_number);
```

- ep_number - IAS Client endpoint number

requestIASZoneEnroll Requests a new IAS Zone enrollment. Can be called to enroll a new device or to re-enroll an already enrolled device.

```
bool requestIASZoneEnroll();
```

This function will return `true` if the enrollment request was sent successfully, `false` otherwise. The actual enrollment status should be checked using the `enrolled()` method after waiting for the enrollment response.

restoreIASZoneEnroll Restores IAS Zone enrollment from stored attributes. This method should be called after rebooting an already enrolled device. It restores the enrollment information from flash memory, which is faster for sleepy devices compared to requesting a new enrollment.

```
bool restoreIASZoneEnroll();
```

This function will return `true` if the enrollment was successfully restored, `false` otherwise. The enrollment information (zone ID and IAS CIE address) must be available in the device's stored attributes for this to succeed.

enrolled Checks if the device is currently enrolled in the IAS Zone.

```
bool enrolled();
```

This function returns `true` if the device is enrolled, `false` otherwise. Use this method to check the enrollment status after calling `requestIASZoneEnroll()` or `restoreIASZoneEnroll()`.

Example

Contact Switch Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates Zigbee contact switch (IAS Zone).
 *
 * The example demonstrates how to use Zigbee library to create a end device.
 * ↳ contact switch.
 * The contact switch is a Zigbee end device, which is reporting data to the
 * ↳ Zigbee network.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition
 * ↳ Scheme.
```

(continues on next page)

(continued from previous page)

```

*
* Please check the README.md for instructions and more detailed description.
*
* Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
*/

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"
#include <Preferences.h>

/* Zigbee contact sensor configuration */
#define CONTACT_SWITCH_ENDPOINT_NUMBER 1
uint8_t button = BOOT_PIN;
uint8_t sensor_pin = 4;

ZigbeeContactSwitch zbContactSwitch = ZigbeeContactSwitch(CONTACT_SWITCH_ENDPOINT_
↪NUMBER);

/* Preferences for storing ENROLLED flag to persist across reboots */
Preferences preferences;

void setup() {
  Serial.begin(115200);

  preferences.begin("Zigbee", false); // Save ENROLLED flag in flash_
↪so it persists across reboots
  bool enrolled = preferences.getBool("ENROLLED"); // Get ENROLLED flag from_
↪preferences
  preferences.end();

  // Init button + switch
  pinMode(button, INPUT_PULLUP);
  pinMode(sensor_pin, INPUT_PULLUP);

  // Optional: set Zigbee device name and model
  zbContactSwitch.setManufacturerAndModel("Espressif", "ZigbeeContactSwitch");

  // Add endpoint to Zigbee Core
  Zigbee.addEndpoint(&zbContactSwitch);

  Serial.println("Starting Zigbee...");
  // When all EPs are registered, start Zigbee in End Device mode
  if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
  } else {
    Serial.println("Zigbee started successfully!");
  }
  Serial.println("Connecting to network");
  while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
  }
  Serial.println();

  // Check if device has been enrolled before restarting - if so, restore IAS Zone_

```

(continues on next page)

(continued from previous page)

```

↪ enroll, otherwise request new IAS Zone enroll
  if (enrolled) {
    Serial.println("Device has been enrolled before - restoring IAS Zone enrollment
↪");
    zbContactSwitch.restoreIASZoneEnroll();
  } else {
    Serial.println("Device is factory new - first time joining network - ↪
↪ requesting new IAS Zone enrollment");
    zbContactSwitch.requestIASZoneEnroll();
  }

  while (!zbContactSwitch.enrolled()) {
    Serial.print(".");
    delay(100);
  }
  Serial.println();
  Serial.println("Zigbee enrolled successfully!");

  // Store ENROLLED flag only if this was a new enrollment (previous flag was ↪
↪ false)
  // Skip writing if we just restored enrollment (flag was already true)
  if (!enrolled) {
    preferences.begin("Zigbee", false);
    preferences.putBool("ENROLLED", true); // set ENROLLED flag to true
    preferences.end();
    Serial.println("ENROLLED flag saved to preferences");
  }
}

void loop() {
  // Checking pin for contact change
  static bool contact = false;
  if (digitalRead(sensor_pin) == HIGH && !contact) {
    // Update contact sensor value
    zbContactSwitch.setOpen();
    contact = true;
  } else if (digitalRead(sensor_pin) == LOW && contact) {
    zbContactSwitch.setClosed();
    contact = false;
  }

  // Checking button for factory reset
  if (digitalRead(button) == LOW) { // Push button pressed
    // Key debounce handling
    delay(100);
    int startTime = millis();
    while (digitalRead(button) == LOW) {
      delay(50);
      if ((millis() - startTime) > 3000) {
        // If key pressed for more than 3secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        // Clear the ENROLLED flag from preferences
        preferences.begin("Zigbee", false);
        preferences.putBool("ENROLLED", false); // set ENROLLED flag to false
        preferences.end();
        Serial.println("ENROLLED flag cleared from preferences");
        delay(1000);
        Zigbee.factoryReset();
      }
    }
  }
}

```

(continues on next page)

(continued from previous page)

```
    delay(100);  
}
```

ZigbeeDimmableLight

About The `ZigbeeDimmableLight` class provides a dimmable light endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for dimmable lighting control with both on/off and brightness level control.

Features: * On/off control * Brightness level control (0-100%) * State and level change callbacks * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Dimmable smart bulbs * LED strips with brightness control * Any device requiring both on/off and dimming functionality

API Reference

Constructor

ZigbeeDimmableLight Creates a new Zigbee dimmable light endpoint.

```
ZigbeeDimmableLight(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

Light Control

setLightState Sets only the light state (on or off) without changing the brightness level.

```
bool setLightState(bool state);
```

- `state` - `true` to turn on, `false` to turn off

This function will return `true` if successful, `false` otherwise.

setLightLevel Sets only the brightness level (0-100) without changing the on/off state.

```
bool setLightLevel(uint8_t level);
```

- `level` - Brightness level (0-100, where 0 is off, 100 is full brightness)

This function will return `true` if successful, `false` otherwise.

setLight Sets both the light state and brightness level simultaneously.

```
bool setLight(bool state, uint8_t level);
```

- `state` - `true` to turn on, `false` to turn off
- `level` - Brightness level (0-100)

This function will return `true` if successful, `false` otherwise.

getLightState Gets the current light state.

```
bool getLightState();
```

This function will return current light state (true = on, false = off).

getLightLevel Gets the current brightness level.

```
uint8_t getLightLevel();
```

This function will return current brightness level (0-100).

restoreLight Restores the light state and triggers any registered callbacks.

```
void restoreLight();
```

Event Handling

onLightChange Sets a callback function to be called when the light state or level changes.

```
void onLightChange(void (*callback) (bool, uint8_t));
```

- callback - Function to call when light state or level changes

Callback Parameters: * bool state - New light state (true = on, false = off) * uint8_t level - New brightness level (0-100)

Example

Dimmable Light Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates Zigbee Dimmable light bulb.
 *
 * The example demonstrates how to use Zigbee library to create an end device with
 * dimmable light end point.
 * The light bulb is a Zigbee end device, which is controlled by a Zigbee
 * ↪ coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition
 * ↪ Scheme.
 */
```

(continues on next page)

(continued from previous page)

```

* Please check the README.md for instructions and more detailed description.
*
* Created by [FaBjE] (https://github.com/FaBjE) based on examples by [Jan_
↪Procházka] (https://github.com/P-R-O-C-H-Y/)
*/

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee dimmable light configuration */
#define ZIGBEE_LIGHT_ENDPOINT 10
uint8_t led = RGB_BUILTIN;
uint8_t button = BOOT_PIN;

ZigbeeDimmableLight zbDimmableLight = ZigbeeDimmableLight(ZIGBEE_LIGHT_ENDPOINT);

/***** RGB LED functions *****/
void setLight(bool state, uint8_t level) {
    if (!state) {
        rgbLedWrite(led, 0, 0, 0);
        return;
    }
    rgbLedWrite(led, level, level, level);
}

// Create a task on identify call to handle the identify function
void identify(uint16_t time) {
    static uint8_t blink = 1;
    log_d("Identify called for %u seconds", time);
    if (time == 0) {
        // If identify time is 0, stop blinking and restore light as it was used for_
↪identify
        zbDimmableLight.restoreLight();
        return;
    }
    rgbLedWrite(led, 255 * blink, 255 * blink, 255 * blink);
    blink = !blink;
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init RMT and leave light OFF
    rgbLedWrite(led, 0, 0, 0);

    // Init button for factory reset
    pinMode(button, INPUT_PULLUP);

    // Set callback function for light change
    zbDimmableLight.onLightChange(setLight);

    // Optional: Set callback function for device identify
    zbDimmableLight.onIdentify(identify);

    // Optional: Set Zigbee device name and model
    zbDimmableLight.setManufacturerAndModel("Espressif", "ZBLightBulb");

```

(continues on next page)

(continued from previous page)

```

// Add endpoint to Zigbee Core
Serial.println("Adding ZigbeeLight endpoint to Zigbee Core");
Zigbee.addEndpoint(&zbdimmableLight);

// When all EPs are registered, start Zigbee in End Device mode
if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();
}

void loop() {
    // Checking button for factory reset
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        // Increase blightness by 50 every time the button is pressed
        zbdimmableLight.setLightLevel(zbdimmableLight.getLightLevel() + 50);
    }
    delay(100);
}
}

```

ZigbeeDoorWindowHandle

About The ZigbeeDoorWindowHandle class provides a door/window handle endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for handle position sensors and other handle-related devices.

Features: * Handle position detection * Multiple position states support * Configurable application types * Automatic reporting capabilities

API Reference

Constructor

ZigbeeDoorWindowHandle Creates a new Zigbee door/window handle endpoint.

```
ZigbeeDoorWindowHandle(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

API Methods

setClosed Sets the door/window handle to closed position.

```
bool setClosed();
```

This function will return `true` if successful, `false` otherwise.

setOpen Sets the door/window handle to open position.

```
bool setOpen();
```

This function will return `true` if successful, `false` otherwise.

setTilted Sets the door/window handle to tilted position.

```
bool setTilted();
```

This function will return `true` if successful, `false` otherwise.

report Manually reports the current handle position.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

setIASClientEndpoint Sets the IAS Client endpoint number (default is 1).

```
void setIASClientEndpoint(uint8_t ep_number);
```

- `ep_number` - IAS Client endpoint number

requestIASZoneEnroll Requests a new IAS Zone enrollment. Can be called to enroll a new device or to re-enroll an already enrolled device.

```
bool requestIASZoneEnroll();
```

This function will return `true` if the enrollment request was sent successfully, `false` otherwise. The actual enrollment status should be checked using the `enrolled()` method after waiting for the enrollment response.

restoreIASZoneEnroll Restores IAS Zone enrollment from stored attributes. This method should be called after rebooting an already enrolled device. It restores the enrollment information from flash memory, which is faster for sleepy devices compared to requesting a new enrollment.

```
bool restoreIASZoneEnroll();
```

This function will return `true` if the enrollment was successfully restored, `false` otherwise. The enrollment information (zone ID and IAS CIE address) must be available in the device's stored attributes for this to succeed.

enrolled Checks if the device is currently enrolled in the IAS Zone.

```
bool enrolled();
```

This function returns `true` if the device is enrolled, `false` otherwise. Use this method to check the enrollment status after calling `requestIASZoneEnroll()` or `restoreIASZoneEnroll()`.

Example *To be added*

ZigbeeElectricalMeasurement

About The `ZigbeeElectricalMeasurement` class provides an endpoint for electrical measurement devices in Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for power monitoring and electrical measurement.

Features: * AC and DC electrical measurements * Voltage, current, and power monitoring * Power factor measurement * Multi-phase support * Configurable measurement ranges * Automatic reporting capabilities * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Smart power monitoring * Energy monitoring systems * Solar panel monitoring * Battery monitoring systems * Electrical load monitoring

Common API

Constructor

ZigbeeElectricalMeasurement Creates a new Zigbee electrical measurement endpoint.

```
ZigbeeElectricalMeasurement(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

DC Electrical Measurement

addDCMeasurement Adds a DC measurement type to the endpoint.

```
bool addDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE measurement_type);
```

- `measurement_type` - DC measurement type constant (`ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE`, `ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT`, `ZIGBEE_DC_MEASUREMENT_TYPE_POWER`)

This function will return `true` if successful, `false` otherwise.

setDCMeasurement Sets the DC measurement value for a specific measurement type.

```
bool setDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE measurement_type, int16_t value);
```

- `measurement_type` - DC measurement type constant (`ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE`, `ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT`, `ZIGBEE_DC_MEASUREMENT_TYPE_POWER`)
- `value` - Measurement value

This function will return `true` if successful, `false` otherwise.

setDCMinMaxValue Sets the minimum and maximum values for a DC measurement type.

```
bool setDCMinMaxValue(ZIGBEE_DC_MEASUREMENT_TYPE measurement_type, int16_t min,
↳ int16_t max);
```

- measurement_type - DC measurement type constant (ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_DC_MEASUREMENT_TYPE_POWER)
- min - Minimum value
- max - Maximum value

This function will return `true` if successful, `false` otherwise.

setDCMultiplierDivisor Sets the multiplier and divisor for scaling DC measurements.

```
bool setDCMultiplierDivisor(ZIGBEE_DC_MEASUREMENT_TYPE measurement_type, uint16_t
↳ multiplier, uint16_t divisor);
```

- measurement_type - DC measurement type constant (ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_DC_MEASUREMENT_TYPE_POWER)
- multiplier - Multiplier value
- divisor - Divisor value

This function will return `true` if successful, `false` otherwise.

setDCReporting Sets the reporting configuration for DC measurements.

```
bool setDCReporting(ZIGBEE_DC_MEASUREMENT_TYPE measurement_type, uint16_t min_
↳ interval, uint16_t max_interval, int16_t delta);
```

- measurement_type - DC measurement type constant (ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_DC_MEASUREMENT_TYPE_POWER)
- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change required to trigger a report

This function will return `true` if successful, `false` otherwise.

reportDC Manually reports a DC measurement value.

```
bool reportDC(ZIGBEE_DC_MEASUREMENT_TYPE measurement_type);
```

- measurement_type - DC measurement type constant (ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_DC_MEASUREMENT_TYPE_POWER)

This function will return `true` if successful, `false` otherwise.

AC Electrical Measurement

addACMeasurement Adds an AC measurement type for a specific phase.

```
bool addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE measurement_type, ZIGBEE_AC_PHASE_
↳ TYPE phase_type);
```

- measurement_type - AC measurement type constant (ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ZIGBEE_AC_MEASUREMENT_TYPE_POWER_FACTOR, ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY)
- phase_type - Phase type constant (ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC, ZIGBEE_AC_PHASE_TYPE_A, ZIGBEE_AC_PHASE_TYPE_B, ZIGBEE_AC_PHASE_TYPE_C)

This function will return `true` if successful, `false` otherwise.

setACMeasurement Sets the AC measurement value for a specific measurement type and phase. For power (`ZIGBEE_AC_MEASUREMENT_TYPE_POWER`) the value is `int16_t`; for other measurement types (voltage, current, power factor, frequency) the value is `uint16_t`.

```
bool setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE measurement_type, ZIGBEE_AC_PHASE_
↳TYPE phase_type, int32_t value);
```

- `measurement_type` - AC measurement type constant (`ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE`, `ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT`, `ZIGBEE_AC_MEASUREMENT_TYPE_POWER`, `ZIGBEE_AC_MEASUREMENT_TYPE_POWER_FACTOR`, `ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY`)
- `phase_type` - Phase type constant (`ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC`, `ZIGBEE_AC_PHASE_TYPE_A`, `ZIGBEE_AC_PHASE_TYPE_B`, `ZIGBEE_AC_PHASE_TYPE_C`)
- `value` - Measurement value (`int16_t` for power, `uint16_t` for voltage, current, power factor, frequency)

This function will return `true` if successful, `false` otherwise.

setACMinMaxValue Sets the minimum and maximum values for an AC measurement type and phase. For power (`ZIGBEE_AC_MEASUREMENT_TYPE_POWER`) min/max are `int16_t`; for other measurement types (voltage, current, power factor, frequency) they are `uint16_t`.

```
bool setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE measurement_type, ZIGBEE_AC_PHASE_
↳TYPE phase_type, int32_t min, int32_t max);
```

- `measurement_type` - AC measurement type constant (`ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE`, `ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT`, `ZIGBEE_AC_MEASUREMENT_TYPE_POWER`, `ZIGBEE_AC_MEASUREMENT_TYPE_POWER_FACTOR`, `ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY`)
- `phase_type` - Phase type constant (`ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC`, `ZIGBEE_AC_PHASE_TYPE_A`, `ZIGBEE_AC_PHASE_TYPE_B`, `ZIGBEE_AC_PHASE_TYPE_C`)
- `min` - Minimum value (`int16_t` for power, `uint16_t` for voltage, current, power factor, frequency)
- `max` - Maximum value (`int16_t` for power, `uint16_t` for voltage, current, power factor, frequency)

This function will return `true` if successful, `false` otherwise.

setACMultiplierDivisor Sets the multiplier and divisor for scaling AC measurements.

```
bool setACMultiplierDivisor(ZIGBEE_AC_MEASUREMENT_TYPE measurement_type, uint16_t
↳multiplier, uint16_t divisor);
```

- `measurement_type` - AC measurement type constant (`ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE`, `ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT`, `ZIGBEE_AC_MEASUREMENT_TYPE_POWER`, `ZIGBEE_AC_MEASUREMENT_TYPE_POWER_FACTOR`, `ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY`)
- `multiplier` - Multiplier value
- `divisor` - Divisor value

This function will return `true` if successful, `false` otherwise.

setACPowerFactor Sets the power factor for a specific phase.

```
bool setACPowerFactor(ZIGBEE_AC_PHASE_TYPE phase_type, int8_t power_factor);
```

- `phase_type` - Phase type constant (`ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC`, `ZIGBEE_AC_PHASE_TYPE_A`, `ZIGBEE_AC_PHASE_TYPE_B`, `ZIGBEE_AC_PHASE_TYPE_C`)
- `power_factor` - Power factor value (-100 to 100)

This function will return `true` if successful, `false` otherwise.

setACReporting Sets the reporting configuration for AC measurements. For power (ZIGBEE_AC_MEASUREMENT_TYPE_POWER) delta is `int16_t`; for other measurement types (voltage, current, power factor, frequency) delta is `uint16_t`.

```
bool setACReporting(ZIGBEE_AC_MEASUREMENT_TYPE measurement_type, ZIGBEE_AC_PHASE_
↳TYPE phase_type, uint16_t min_interval, uint16_t max_interval, int32_t delta);
```

- `measurement_type` - AC measurement type constant (ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ZIGBEE_AC_MEASUREMENT_TYPE_POWER_FACTOR, ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY)
- `phase_type` - Phase type constant (ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC, ZIGBEE_AC_PHASE_TYPE_A, ZIGBEE_AC_PHASE_TYPE_B, ZIGBEE_AC_PHASE_TYPE_C)
- `min_interval` - Minimum reporting interval in seconds
- `max_interval` - Maximum reporting interval in seconds
- `delta` - Minimum change required to trigger a report (`int16_t` for power, `uint16_t` for voltage, current, power factor, frequency)

This function will return `true` if successful, `false` otherwise.

reportAC Manually reports an AC measurement value.

```
bool reportAC(ZIGBEE_AC_MEASUREMENT_TYPE measurement_type, ZIGBEE_AC_PHASE_TYPE_
↳phase_type);
```

- `measurement_type` - AC measurement type constant (ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ZIGBEE_AC_MEASUREMENT_TYPE_POWER_FACTOR, ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY)
- `phase_type` - Phase type constant (ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC, ZIGBEE_AC_PHASE_TYPE_A, ZIGBEE_AC_PHASE_TYPE_B, ZIGBEE_AC_PHASE_TYPE_C)

This function will return `true` if successful, `false` otherwise.

Example

DC Electrical Measurement

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates a Zigbee DC electrical measurement sensor.
 *
 * The example shows how to use the Zigbee library to create an end device that
 * ↳measures
 * DC voltage, current and power using the Electrical Measurement cluster.
 *
 * The device reports:
 * - DC voltage in millivolts (0-5000mV)
 * - DC current in milliamps (0-1000mA)
```

(continues on next page)

(continued from previous page)

```

* - DC power in milliwatts (0-5000mW)
*
* Proper Zigbee mode must be selected in Tools->Zigbee mode
* and also the correct partition scheme must be selected in Tools->Partition_
↪Scheme.
*
* Please check the README.md for instructions and more detailed description.
*
* Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
*/

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee DC measurement device configuration */
#define DC_ELECTRICAL_MEASUREMENT_ENDPOINT_NUMBER 1

uint8_t analogPin = A0;
uint8_t button = BOOT_PIN;

ZigbeeElectricalMeasurement zbElectricalMeasurement =
↪ZigbeeElectricalMeasurement(DC_ELECTRICAL_MEASUREMENT_ENDPOINT_NUMBER);

void setup() {
  Serial.begin(115200);
  Serial.println("Starting...");

  // Init button switch
  pinMode(button, INPUT_PULLUP);

  // Set analog resolution to 10 bits
  analogReadResolution(10);

  // Optional: set Zigbee device name and model
  zbElectricalMeasurement.setManufacturerAndModel("Espressif",
↪"ZigbeeElectricalMeasurementDC");

  // Add analog clusters to Zigbee Analog according your needs
  zbElectricalMeasurement.addDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE);
  zbElectricalMeasurement.addDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT);
  zbElectricalMeasurement.addDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE_POWER);

  // // Optional: set Min/max values for the measurements
  zbElectricalMeasurement.setDCMinMaxValue(ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE, 0,
↪5000); // 0-5.000V
  zbElectricalMeasurement.setDCMinMaxValue(ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT, 0,
↪1000); // 0-1.000A
  zbElectricalMeasurement.setDCMinMaxValue(ZIGBEE_DC_MEASUREMENT_TYPE_POWER, 0,
↪5000); // 0-5.000W

  // // Optional: set Multiplier/Divisor for the measurements
  zbElectricalMeasurement.setDCMultiplierDivisor(ZIGBEE_DC_MEASUREMENT_TYPE_
↪VOLTAGE, 1, 1000); // 1/1000 = 0.001V (1 unit of measurement = 0.001V = 1mV)
  zbElectricalMeasurement.setDCMultiplierDivisor(ZIGBEE_DC_MEASUREMENT_TYPE_
↪CURRENT, 1, 1000); // 1/1000 = 0.001A (1 unit of measurement = 0.001A = 1mA)
  zbElectricalMeasurement.setDCMultiplierDivisor(ZIGBEE_DC_MEASUREMENT_TYPE_POWER,
↪1, 1000); // 1/1000 = 0.001W (1 unit of measurement = 0.001W = 1mW)

```

(continues on next page)

(continued from previous page)

```

// Add endpoints to Zigbee Core
Zigbee.addEndpoint(&zbelectricalMeasurement);

Serial.println("Starting Zigbee...");
// When all EPs are registered, start Zigbee in End Device mode
if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
} else {
    Serial.println("Zigbee started successfully!");
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println("Connected");

// Optional: Add reporting for DC measurements (this is overridden by_
↪ HomeAssistant ZHA if connected to its network)
zbelectricalMeasurement.setDCReporting(ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE, 0, 30,
↪ 10); // report every 30 seconds if value changes by 10 (0.1V)
zbelectricalMeasurement.setDCReporting(ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT, 0, 30,
↪ 10); // report every 30 seconds if value changes by 10 (0.1A)
zbelectricalMeasurement.setDCReporting(ZIGBEE_DC_MEASUREMENT_TYPE_POWER, 0, 30,
↪ 10); // report every 30 seconds if value changes by 10 (0.1W)
}

void loop() {
    static uint32_t timeCounter = 0;
    static uint8_t randomizer = 0;
    // Read ADC value and update the analog value every 2s
    if (!(timeCounter++ % 20)) { // delaying for 100ms x 20 = 2s
        int16_t voltage_mv = (int16_t)(analogReadMilliVolts(analogPin));
        int16_t current_ma = randomizer; //0-255mA
        int16_t power_mw = voltage_mv * current_ma / 1000; //calculate power in mW
        Serial.printf("Updating DC voltage to %d mV, current to %d mA, power to %d mW\
↪ r\n", voltage_mv, current_ma, power_mw);
        zbelectricalMeasurement.setDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE,
↪ voltage_mv);
        zbelectricalMeasurement.setDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT,
↪ current_ma);
        zbelectricalMeasurement.setDCMeasurement(ZIGBEE_DC_MEASUREMENT_TYPE_POWER,
↪ power_mw);
        // Analog input supports reporting
        zbelectricalMeasurement.reportDC(ZIGBEE_DC_MEASUREMENT_TYPE_VOLTAGE);
        zbelectricalMeasurement.reportDC(ZIGBEE_DC_MEASUREMENT_TYPE_CURRENT);
        zbelectricalMeasurement.reportDC(ZIGBEE_DC_MEASUREMENT_TYPE_POWER);

        randomizer += 10;
    }

    // Checking button for factory reset and reporting
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);

```

(continues on next page)

(continued from previous page)

```

    if ((millis() - startTime) > 3000) {
        // If key pressed for more than 3secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        delay(1000);
        Zigbee.factoryReset();
    }
}
}
delay(100);
}

```

AC Electrical Measurement

```

// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee electrical AC measurement device with
 * ↪multi-phase support.
 *
 * The example demonstrates how to use Zigbee library to create a router device
 * ↪that measures
 * AC electrical parameters like voltage, current, power, power factor and
 * ↪frequency across
 * three phases (A, B, C). This allows monitoring of three-phase power systems
 * ↪commonly used
 * in industrial and commercial applications.
 *
 * The device measures:
 * - Per phase: voltage, current, power, power factor
 * - Shared: frequency (common across all phases)
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition
 * ↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

// Recommended to use Router mode, as this type of device is expected to be mains
* ↪powered.
#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator/router mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

```

(continues on next page)

(continued from previous page)

```

/* Zigbee AC measurement device configuration */
#define AC_ELECTRICAL_MEASUREMENT_ENDPOINT_NUMBER 1

uint8_t analogPin = A0;
uint8_t button = BOOT_PIN;

ZigbeeElectricalMeasurement zbElectricalMeasurement =
↳ ZigbeeElectricalMeasurement(AC_ELECTRICAL_MEASUREMENT_ENDPOINT_NUMBER);

void onAnalogOutputChange(float analog_output) {
    Serial.printf("Received analog output change: %.1f\r\n", analog_output);
}

void setup() {
    Serial.begin(115200);
    Serial.println("Starting...");

    // Init button switch
    pinMode(button, INPUT_PULLUP);

    // Set analog resolution to 10 bits
    analogReadResolution(10);

    // Optional: set Zigbee device name and model
    zbElectricalMeasurement.setManufacturerAndModel("Espressif",
↳ "ZigbeeElectricalMeasurementAC");

    // Add analog clusters to Zigbee Analog according your needs
    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE,
↳ ZIGBEE_AC_PHASE_TYPE_A);
    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT,
↳ ZIGBEE_AC_PHASE_TYPE_A);
    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_POWER,
↳ ZIGBEE_AC_PHASE_TYPE_A);

    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE,
↳ ZIGBEE_AC_PHASE_TYPE_B);
    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT,
↳ ZIGBEE_AC_PHASE_TYPE_B);
    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_POWER,
↳ ZIGBEE_AC_PHASE_TYPE_B);

    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE,
↳ ZIGBEE_AC_PHASE_TYPE_C);
    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT,
↳ ZIGBEE_AC_PHASE_TYPE_C);
    zbElectricalMeasurement.addACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_POWER,
↳ ZIGBEE_AC_PHASE_TYPE_C);

    zbElectricalMeasurement.addACMeasurement(
        ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY, ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC
    ); // frequency is not phase specific (shared)

    // Recommended: set Multiplier/Divisor for the measurements (common for all
↳ phases)
    zbElectricalMeasurement.setACMultiplierDivisor(ZIGBEE_AC_MEASUREMENT_TYPE_
↳ VOLTAGE, 1, 100); // 1/100 = 0.01V (1 unit of measurement = 0.01V = 10mV)
    zbElectricalMeasurement.setACMultiplierDivisor(ZIGBEE_AC_MEASUREMENT_TYPE_
↳ CURRENT, 1, 1000); // 1/1000 = 0.001A (1 unit of measurement = 0.001A = 1mA)
    zbElectricalMeasurement.setACMultiplierDivisor(ZIGBEE_AC_MEASUREMENT_TYPE_POWER,
↳

```

(continues on next page)

(continued from previous page)

```

↪1, 10); // 1/10 = 0.1W (1 unit of measurement = 0.1W = 100mW)
    zbElectricalMeasurement.setACMultiplierDivisor(ZIGBEE_AC_MEASUREMENT_TYPE_
↪FREQUENCY, 1, 1000); // 1/1000 = 0.001Hz (1 unit of measurement = 0.001Hz =
↪1mHz)

    // Optional: set Min/max values for the measurements
    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE,
↪ZIGBEE_AC_PHASE_TYPE_A, 0, 30000); // 0-300.00V
    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT,
↪ZIGBEE_AC_PHASE_TYPE_A, 0, 10000); // 0-10.000A
    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_POWER,
↪ZIGBEE_AC_PHASE_TYPE_A, 0, 32000); // 0-3200.0W

    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE,
↪ZIGBEE_AC_PHASE_TYPE_B, 0, 30000); // 0-300.00V
    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT,
↪ZIGBEE_AC_PHASE_TYPE_B, 0, 10000); // 0-10.000A
    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_POWER,
↪ZIGBEE_AC_PHASE_TYPE_B, 0, 32000); // 0-3200.0W

    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE,
↪ZIGBEE_AC_PHASE_TYPE_C, 0, 30000); // 0-300.00V
    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT,
↪ZIGBEE_AC_PHASE_TYPE_C, 0, 10000); // 0-10.000A
    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_POWER,
↪ZIGBEE_AC_PHASE_TYPE_C, 0, 32000); // 0-3200.0W

    zbElectricalMeasurement.setACMinMaxValue(ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY,
↪ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC, 0, 65000); // 0-65.000Hz

    // Add endpoints to Zigbee Core
    Zigbee.addEndpoint(&zbElectricalMeasurement);

    Serial.println("Starting Zigbee...");
    // When all EPs are registered, start Zigbee in Router mode
    if (!Zigbee.begin(ZIGBEE_ROUTER)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    } else {
        Serial.println("Zigbee started successfully!");
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println("Connected");
}

void loop() {
    static uint32_t timeCounter = 0;
    static uint8_t randomizer = 0;
    // Read ADC value and update the analog value every 2s
    if (!(timeCounter++ % 20)) { // delaying for
↪100ms x 20 = 2s
        uint16_t voltage = 23000 + randomizer; // 230.00 V
        uint16_t current = analogReadMilliVolts(analogPin); // demonstrates
↪approx 0-3.3A
        int16_t power = ((voltage / 100) * (current / 1000) * 10); //calculate power
↪in W

```

(continues on next page)

(continued from previous page)

```

uint16_t frequency = 50135; // 50.000 Hz
Serial.printf("Updating AC voltage to %u (0.01V), current to %u (mA), power to
↪ %d (0.1W), frequency to %u (mHz)\r\n", voltage, current, power, frequency);
zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ↪
↪ ZIGBEE_AC_PHASE_TYPE_A, voltage);
zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ↪
↪ ZIGBEE_AC_PHASE_TYPE_A, current);
zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ↪
↪ ZIGBEE_AC_PHASE_TYPE_A, power);

// Phase B demonstrates phase shift
current += 500;
voltage += 500;
power = ((voltage / 100) * (current / 1000) * 10); //calculate power in W

zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ↪
↪ ZIGBEE_AC_PHASE_TYPE_B, voltage);
zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ↪
↪ ZIGBEE_AC_PHASE_TYPE_B, current);
zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ↪
↪ ZIGBEE_AC_PHASE_TYPE_B, power);

// Phase C demonstrates phase shift
current += 500;
voltage += 500;
power = ((voltage / 100) * (current / 1000) * 10); //calculate power in W

zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ↪
↪ ZIGBEE_AC_PHASE_TYPE_C, voltage);
zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ↪
↪ ZIGBEE_AC_PHASE_TYPE_C, current);
zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ↪
↪ ZIGBEE_AC_PHASE_TYPE_C, power);

zbElectricalMeasurement.setACMeasurement(ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY, ↪
↪ ZIGBEE_AC_PHASE_TYPE_NON_SPECIFIC, frequency);

zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_AC_
↪ PHASE_TYPE_A);
zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_AC_
↪ PHASE_TYPE_A);
zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ZIGBEE_AC_
↪ PHASE_TYPE_A);

zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_AC_
↪ PHASE_TYPE_B);
zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_AC_
↪ PHASE_TYPE_B);
zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ZIGBEE_AC_
↪ PHASE_TYPE_B);

zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_VOLTAGE, ZIGBEE_AC_
↪ PHASE_TYPE_C);
zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_CURRENT, ZIGBEE_AC_
↪ PHASE_TYPE_C);
zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_POWER, ZIGBEE_AC_
↪ PHASE_TYPE_C);

zbElectricalMeasurement.reportAC(ZIGBEE_AC_MEASUREMENT_TYPE_FREQUENCY, ZIGBEE_
↪ AC_PHASE_TYPE_NON_SPECIFIC);

```

(continues on next page)

(continued from previous page)

```

    randomizer += 10;
}

// Checking button for factory reset and reporting
if (digitalRead(button) == LOW) { // Push button pressed
    // Key debounce handling
    delay(100);
    int startTime = millis();
    while (digitalRead(button) == LOW) {
        delay(50);
        if ((millis() - startTime) > 3000) {
            // If key pressed for more than 3secs, factory reset Zigbee and reboot
            Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
            delay(1000);
            Zigbee.factoryReset();
        }
    }
}
delay(100);
}

```

ZigbeeFanControl

About The ZigbeeFanControl class provides a Zigbee endpoint for controlling fan devices in a Home Automation (HA) network. This endpoint implements the Fan Control cluster and allows for controlling fan modes and sequences.

Features: * Fan mode control (OFF, LOW, MEDIUM, HIGH, ON, AUTO, SMART) * Fan mode sequence configuration * State change callbacks * Zigbee HA standard compliance

Use Cases: * Smart ceiling fans * HVAC system fans * Exhaust fans * Any device requiring fan speed control

API Reference

Constructor

ZigbeeFanControl Creates a new Zigbee fan control endpoint.

```
ZigbeeFanControl(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Fan Mode Enums

ZigbeeFanMode Available fan modes for controlling the fan speed and operation.

```

enum ZigbeeFanMode {
    FAN_MODE_OFF,           // Fan is off
    FAN_MODE_LOW,           // Low speed
    FAN_MODE_MEDIUM,       // Medium speed
    FAN_MODE_HIGH,         // High speed
    FAN_MODE_ON,           // Fan is on (full speed)
    FAN_MODE_AUTO,         // Automatic mode
    FAN_MODE_SMART,        // Smart mode
};

```

ZigbeeFanModeSequence Available fan mode sequences that define which modes are available for the fan.

```
enum ZigbeeFanModeSequence {
    FAN_MODE_SEQUENCE_LOW_MED_HIGH,           // Low -> Medium -> High
    FAN_MODE_SEQUENCE_LOW_HIGH,               // Low -> High
    FAN_MODE_SEQUENCE_LOW_MED_HIGH_AUTO,      // Low -> Medium -> High -> Auto
    FAN_MODE_SEQUENCE_LOW_HIGH_AUTO,          // Low -> High -> Auto
    FAN_MODE_SEQUENCE_ON_AUTO,                 // On -> Auto
};
```

API Methods

setFanModeSequence Sets the fan mode sequence and initializes the fan control.

```
bool setFanModeSequence(ZigbeeFanModeSequence sequence);
```

- sequence - The fan mode sequence to set

This function will return `true` if successful, `false` otherwise.

getFanMode Gets the current fan mode.

```
ZigbeeFanMode getFanMode();
```

This function will return current fan mode.

getFanModeSequence Gets the current fan mode sequence.

```
ZigbeeFanModeSequence getFanModeSequence();
```

This function will return current fan mode sequence.

Event Handling

onFanModeChange Sets a callback function to be called when the fan mode changes.

```
void onFanModeChange(void (*callback)(ZigbeeFanMode mode));
```

- callback - Function to call when fan mode changes

Example

Fan Control Implementation

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
```

(continues on next page)

(continued from previous page)

```

// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates simple Zigbee fan control.
 *
 * The example demonstrates how to use Zigbee library to create a router device_
↪ fan control.
 * The fan control is a Zigbee router device, which is controlled by a Zigbee_
↪ coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↪ Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee light bulb configuration */
#define ZIGBEE_FAN_CONTROL_ENDPOINT 1

#ifdef RGB_BUILTIN
uint8_t led = RGB_BUILTIN; // To demonstrate the current fan control mode
#else
uint8_t led = 2;
#endif

uint8_t button = BOOT_PIN;

ZigbeeFanControl zbFanControl = ZigbeeFanControl(ZIGBEE_FAN_CONTROL_ENDPOINT);

/***** fan control callback function *****/
void setFan(ZigbeeFanMode mode) {
    switch (mode) {
        case FAN_MODE_OFF:
            rgbLedWrite(led, 0, 0, 0); // Off
            Serial.println("Fan mode: OFF");
            break;
        case FAN_MODE_LOW:
            rgbLedWrite(led, 0, 0, 255); // Blue
            Serial.println("Fan mode: LOW");
            break;
        case FAN_MODE_MEDIUM:
            rgbLedWrite(led, 255, 255, 0); // Yellow
            Serial.println("Fan mode: MEDIUM");
            break;
        case FAN_MODE_HIGH:
            rgbLedWrite(led, 255, 0, 0); // Red
            Serial.println("Fan mode: HIGH");
            break;
        case FAN_MODE_ON:
            rgbLedWrite(led, 255, 255, 255); // White
    }
}

```

(continues on next page)

(continued from previous page)

```

        Serial.println("Fan mode: ON");
        break;
    default: log_e("Unhandled fan mode: %d", mode); break;
    }
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init LED that will be used to indicate the current fan control mode
    rgbLedWrite(led, 0, 0, 0);

    // Init button for factory reset
    pinMode(button, INPUT_PULLUP);

    //Optional: set Zigbee device name and model
    zbFanControl.setManufacturerAndModel("Espressif", "ZBFanControl");

    // Set the fan mode sequence to LOW_MED_HIGH
    zbFanControl.setFanModeSequence(FAN_MODE_SEQUENCE_LOW_MED_HIGH);

    // Set callback function for fan mode change
    zbFanControl.onFanModeChange(setFan);

    //Add endpoint to Zigbee Core
    Serial.println("Adding ZigbeeFanControl endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbFanControl);

    // When all EPs are registered, start Zigbee in ROUTER mode
    if (!Zigbee.begin(ZIGBEE_ROUTER)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();
}

void loop() {
    // Checking button for factory reset
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
    }
    delay(100);
}

```

ZigbeeFlowSensor

About The `ZigbeeFlowSensor` class provides a flow sensor endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for liquid and gas flow measurement devices.

Features: * Flow rate measurement in m³/h * Configurable measurement range * Tolerance and reporting configuration * Automatic reporting capabilities * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Water flow monitoring * Gas flow measurement * Industrial process monitoring * Smart home water management * HVAC system flow monitoring * Agricultural irrigation systems

API Reference

Constructor

ZigbeeFlowSensor Creates a new Zigbee flow sensor endpoint.

```
ZigbeeFlowSensor(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

API Methods

setFlow Sets the flow rate measurement value.

```
bool setFlow(float value);
```

- `value` - Flow rate value in 0.1 m³/h

This function will return `true` if successful, `false` otherwise.

setMinMaxValue Sets the minimum and maximum measurement values.

```
bool setMinMaxValue(float min, float max);
```

- `min` - Minimum flow rate value in 0.1 m³/h
- `max` - Maximum flow rate value in 0.1 m³/h

This function will return `true` if successful, `false` otherwise.

setDefaultValue Sets the default (initial) value for the flow sensor in 0.1 m³/h. This value will be used as the initial measured value when the device is in factory reset mode and before the sensor provides actual readings.

```
bool setDefaultValue(float defaultValue);
```

- `defaultValue` - Default flow rate value in 0.1 m³/h

Important: Must be called before adding the EP to Zigbee class. Only effective when the device is in factory reset mode (before commissioning/joining a network).

This function will return `true` if successful, `false` otherwise.

setTolerance Sets the tolerance value for measurements.

```
bool setTolerance(float tolerance);
```

- tolerance - Tolerance value in 0.01 m³/h

This function will return `true` if successful, `false` otherwise.

setReporting Sets the reporting configuration for flow rate measurements.

```
bool setReporting(uint16_t min_interval, uint16_t max_interval, float delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change required to trigger a report in 0.1 m³/h

This function will return `true` if successful, `false` otherwise.

report Manually reports the current flow rate value.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

Example

Flow + PressureSensor Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee temperature sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device_
↪temperature sensor.
 * The temperature sensor is a Zigbee end device, which is controlled by a Zigbee_
↪coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifdef ZIGBEE_MODE_ED
```

(continues on next page)

(continued from previous page)

```

#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee flow + pressure sensor configuration */
#define FLOW_SENSOR_ENDPOINT_NUMBER 10
#define PRESSURE_SENSOR_ENDPOINT_NUMBER 11

uint8_t button = BOOT_PIN;

ZigbeeFlowSensor zbFlowSensor = ZigbeeFlowSensor(FLOW_SENSOR_ENDPOINT_NUMBER);
ZigbeePressureSensor zbPressureSensor = ZigbeePressureSensor(PRESSURE_SENSOR_
↪ENDPOINT_NUMBER);

void setup() {
    Serial.begin(115200);

    // Init button switch
    pinMode(button, INPUT_PULLUP);

    // Optional: set Zigbee device name and model
    zbFlowSensor.setManufacturerAndModel("Espressif", "ZigbeeFlowSensor");

    // Set minimum and maximum flow measurement value in 0,1 m3/h
    zbFlowSensor.setMinMaxValue(0.0, 100.0);

    // Optional: Set tolerance for flow measurement in 0,1 m3/h
    zbFlowSensor.setTolerance(1.0);

    // Optional: set Zigbee device name and model
    zbPressureSensor.setManufacturerAndModel("Espressif", "ZigbeePressureSensor");

    // Set minimum and maximum pressure measurement value in hPa
    zbPressureSensor.setMinMaxValue(0, 10000);

    // Optional: Set tolerance for pressure measurement in hPa
    zbPressureSensor.setTolerance(1);

    // Add endpoints to Zigbee Core
    Zigbee.addEndpoint(&zbFlowSensor);
    Zigbee.addEndpoint(&zbPressureSensor);

    Serial.println("Starting Zigbee...");
    // When all EPs are registered, start Zigbee in End Device mode
    if (!Zigbee.begin()) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    } else {
        Serial.println("Zigbee started successfully!");
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();

    // Set reporting interval for flow and pressure measurement in seconds, must be_
↪called after Zigbee.begin()

```

(continues on next page)

(continued from previous page)

```

    // min_interval and max_interval in seconds, delta (pressure change in hPa, flow_
    ↪ change in 0,1 m3/h)
    // if min = 1 and max = 0, reporting is sent only when temperature changes by_
    ↪ delta
    // if min = 0 and max = 10, reporting is sent every 10 seconds or temperature_
    ↪ changes by delta
    // if min = 0, max = 10 and delta = 0, reporting is sent every 10 seconds_
    ↪ regardless of delta change
    zbFlowSensor.setReporting(0, 30, 1.0);
    zbPressureSensor.setReporting(0, 30, 1);
}

void loop() {
    static uint32_t timeCounter = 0;

    // Read flow and pressure sensors every 2s
    if (!(timeCounter++ % 20)) { // delaying for 100ms x 20 = 2s
        float flow_value = temperatureRead();
        uint16_t pressure_value = (uint16_t)temperatureRead() * 100; /*100 for_
    ↪ demonstration so the value is in 1000-3000hPa
        Serial.printf("Updating flow sensor value to %.2f m3/h\r\n", flow_value);
        zbFlowSensor.setFlow(flow_value);
        Serial.printf("Updating pressure sensor value to %u hPa\r\n", pressure_value);
        zbPressureSensor.setPressure(pressure_value);
    }

    // Checking button for factory reset and reporting
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        zbFlowSensor.report();
        zbPressureSensor.report();
    }
    delay(100);
}

```

ZigbeeGateway

About The ZigbeeGateway class provides a gateway endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for network coordination and gateway functionality. Gateway is a device that can be used to bridge Zigbee network to other networks (e.g. Wi-Fi, Ethernet, etc.).

API Reference

Constructor

ZigbeeGateway Creates a new Zigbee gateway endpoint.

```
ZigbeeGateway(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Example

Gateway Implementation

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates simple Zigbee Gateway functionality.
 *
 * The example demonstrates how to use Zigbee library on ESP32s to create a Zigbee_
↳ Gateway, updating the time from NTP server.
 * The Gateway is able to communicate with Zigbee end devices and send/receive_
↳ data to/from them.
 * The Gateway is also able to communicate with the cloud or other devices over Wi-
↳ Fi / BLE.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode->Zigbee ZCZR_
↳ (coordinator/router)
 * and also the correct partition scheme must be selected in Tools->Partition_
↳ Scheme->Zigbee ZCZR
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"
#include <WiFi.h>
#include "time.h"
#include "esp_sntp.h"

/* Zigbee gateway configuration */
#define GATEWAY_ENDPOINT_NUMBER 1
#define GATEWAY_RCP_UART_PORT  UART_NUM_1  // UART 0 is used for Serial_
↳ communication
#define GATEWAY_RCP_RX_PIN      4
#define GATEWAY_RCP_TX_PIN      5
```

(continues on next page)

(continued from previous page)

```

ZigbeeGateway zbGateway = ZigbeeGateway(GATEWAY_ENDPOINT_NUMBER);

/* Wi-Fi credentials */
const char *ssid = "your-ssid";
const char *password = "your-password";

/* NTP server configuration */
const char *ntpServer1 = "pool.ntp.org";
const char *ntpServer2 = "time.nist.gov";
const long gmtOffset_sec = 3600;
const int daylightOffset_sec = 3600;
const char *time_zone = "CET-1CEST,M3.5.0,M10.5.0/3"; // TimeZone rule for Europe/
↳Rome including daylight adjustment rules (optional)

/* Time structure */
struct tm timeinfo;

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Initialize Wi-Fi and connect to AP
    WiFi.begin(ssid, password);
    esp_sntp_servermode_dhcp(1); // (optional)

    Serial.print("Connecting to WiFi");

    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("WiFi connected");

    // Initialize Zigbee and Begin Zigbee stack
    // Optional: set Zigbee device name and model
    zbGateway.setManufacturerAndModel("Espressif", "ZigbeeGateway");
    zbGateway.addTimeCluster(timeinfo, gmtOffset_sec);

    // Add endpoint to Zigbee Core
    Serial.println("Adding Zigbee Gateway endpoint");
    Zigbee.addEndpoint(&zbGateway);

    // Optional: Open network for 180 seconds after boot
    Zigbee.setRebootOpenNetwork(180);

    // Set custom radio configuration for RCP communication
    esp_zb_radio_config_t radio_config = ZIGBEE_DEFAULT_UART_RCP_RADIO_CONFIG();
    radio_config.radio_uart_config.port = GATEWAY_RCP_UART_PORT;
    radio_config.radio_uart_config.rx_pin = (gpio_num_t)GATEWAY_RCP_RX_PIN;
    radio_config.radio_uart_config.tx_pin = (gpio_num_t)GATEWAY_RCP_TX_PIN;

    Zigbee.setRadioConfig(radio_config);

    // When all EPs are registered, start Zigbee with ZIGBEE_COORDINATOR or ZIGBEE_
    ↳ROUTER mode
    if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }
}

```

(continues on next page)

(continued from previous page)

```

// set notification call-back function
ntp_set_time_sync_notification_cb(timeavailable);
ntp_set_sync_interval(30000); // sync every 30 seconds

// config time zone and NTP servers
configTime(gmtOffset_sec, daylightOffset_sec, ntpServer1, ntpServer2);
}

void loop() {
  // Nothing to do here in this example
}

void printLocalTime() {
  if (!getLocalTime(&timeinfo)) {
    Serial.println("No time available (yet)");
    return;
  }
  Serial.println(&timeinfo, "%A, %B %d %Y %H:%M:%S");
  zbGateway.setTime(timeinfo);
  Serial.println("Time updated in Zigbee Gateway");
}

// Callback function (gets called when time adjusts via NTP)
void timeavailable(struct timeval *t) {
  Serial.println("Got time adjustment from NTP!");
  printLocalTime();
}

```

ZigbeeIlluminanceSensor

About The `ZigbeeIlluminanceSensor` class provides an endpoint for illuminance sensors in Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for light level measurement devices, supporting ambient light monitoring.

Features: * Illuminance measurement in lux * Configurable measurement range * Tolerance and reporting configuration * Automatic reporting capabilities

API Reference

Constructor

ZigbeeIlluminanceSensor Creates a new Zigbee illuminance sensor endpoint.

```
ZigbeeIlluminanceSensor(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

API Methods

setIlluminance Sets the illuminance measurement value.

```
bool setIlluminance(uint16_t value);
```

- `value` - Illuminance value in lux

This function will return `true` if successful, `false` otherwise.

setMinMaxValue Sets the minimum and maximum measurement values.

```
bool setMinMaxValue(uint16_t min, uint16_t max);
```

- min - Minimum illuminance value in lux
- max - Maximum illuminance value in lux

This function will return `true` if successful, `false` otherwise.

setDefaultValue Sets the default (initial) value for the illuminance sensor. This value will be used as the initial measured value when the device is in factory reset mode and before the sensor provides actual readings.

```
bool setDefaultValue(uint16_t defaultValue);
```

- defaultValue - Default illuminance value in lux

Important: Must be called before adding the EP to Zigbee class. Only effective when the device is in factory reset mode (before commissioning/joining a network).

This function will return `true` if successful, `false` otherwise.

setTolerance Sets the tolerance value for measurements.

```
bool setTolerance(uint16_t tolerance);
```

- tolerance - Tolerance value in lux

This function will return `true` if successful, `false` otherwise.

setReporting Sets the reporting configuration for illuminance measurements.

```
bool setReporting(uint16_t min_interval, uint16_t max_interval, uint16_t delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change required to trigger a report in lux

This function will return `true` if successful, `false` otherwise.

report Manually reports the current illuminance value.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

Example

Illuminance Sensor Implementation

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
```

(continues on next page)

(continued from previous page)

```
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee illuminance sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device_
↪ illuminance sensor.
 * The illuminance sensor is a Zigbee end device, which is controlled by a Zigbee_
↪ coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↪ Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by MikaFromTheRoof (https://github.com/MikaFromTheRoof)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

#define ZIGBEE_ILLUMINANCE_SENSOR_ENDPOINT 9
uint8_t button = BOOT_PIN;
uint8_t illuminance_sensor_pin = 6; // Insert the analog pin to which the sensor_
↪ (e.g. photoresistor) is connected

ZigbeeIlluminanceSensor zbIlluminanceSensor = ZigbeeIlluminanceSensor(ZIGBEE_
↪ ILLUMINANCE_SENSOR_ENDPOINT);

/***** Illuminance sensor *****/
static void illuminance_sensor_value_update(void *arg) {
    for (;;) {
        // read the raw analog value from the sensor
        int lsens_analog_raw = analogRead(illuminance_sensor_pin);
        Serial.printf("[Illuminance Sensor] raw analog value: %d\r\n", lsens_analog_
↪ raw);

        // conversion into zigbee raw illuminance value (typically between 0 in_
↪ darkness and 50000 in direct sunlight)
        // depends on the value range of the raw analog sensor values and will need_
↪ calibration for correct lux values
        // for demonstration purpose map the 12-bit ADC value (0-4095) to Zigbee_
↪ illuminance range (0-50000)
        int lsens_illuminance_raw = map(lsens_analog_raw, 0, 4095, 0, 50000);
        Serial.printf("[Illuminance Sensor] raw illuminance value: %d\r\n", lsens_
↪ illuminance_raw);

        // according to zigbee documentation the formular 10^(lsens_illuminance_raw/
↪ 10000)-1 can be used to calculate lux value from raw illuminance value
        // Note: Zigbee2MQTT seems to be using the formular 10^(lsens_illuminance_raw/
↪ 10000) instead (without -1)
        int lsens_illuminance_lux = round(pow(10, (lsens_illuminance_raw / 10000.0)) -
↪ 1);
    }
}
```

(continues on next page)

(continued from previous page)

```

    Serial.printf("[Illuminance Sensor] lux value: %d lux\r\n", lsens_illuminance_
↳lux);

    // Update illuminance in illuminance sensor EP
    zbIlluminanceSensor.setIlluminance(lsens_illuminance_raw); // use raw_
↳illuminance here!

    delay(1000); // reduce delay (in ms), if you want your device to react more_
↳quickly to changes in illuminance
}
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Optional: configure analog input
    analogSetAttenuation(ADC_11db); // set analog to digital converter (ADC)_
↳attenuation to 11 dB (up to ~3.3V input)
    analogReadResolution(12); // set analog read resolution to 12 bits (value_
↳range from 0 to 4095), 12 is default

    // Init button for factory reset
    pinMode(button, INPUT_PULLUP);

    // Optional: Set Zigbee device name and model
    zbIlluminanceSensor.setManufacturerAndModel("Espressif", "ZigbeeIlluminanceSensor
↳");

    // Optional: Set power source (choose between ZB_POWER_SOURCE_MAINS and ZB_POWER_
↳SOURCE_BATTERY), defaults to unknown
    zbIlluminanceSensor.setPowerSource(ZB_POWER_SOURCE_MAINS);

    // Set minimum and maximum for raw illuminance value (0 min and 50000 max equals_
↳to 0 lux - 100,000 lux)
    zbIlluminanceSensor.setMinMaxValue(0, 50000);

    // Optional: Set tolerance for raw illuminance value
    zbIlluminanceSensor.setTolerance(1);

    // Add endpoint to Zigbee Core
    Serial.println("Adding Zigbee illuminance sensor endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbIlluminanceSensor);

    Serial.println("Starting Zigbee...");
    // When all EPs are registered, start Zigbee in End Device mode
    if (!Zigbee.begin()) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    } else {
        Serial.println("Zigbee started successfully!");
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();

    // Start illuminance sensor reading task

```

(continues on next page)

(continued from previous page)

```

    xTaskCreate(illuminance_sensor_value_update, "illuminance_sensor_update", 3072,
↳NULL, 10, NULL);

    // Set reporting schedule for illuminance value measurement in seconds, must be
↳called after Zigbee.begin()
    // min_interval and max_interval in seconds, delta
    // if min = 1 and max = 0, delta = 1000, reporting is sent when raw illuminance
↳value changes by 1000, but at most once per second
    // if min = 0 and max = 10, delta = 1000, reporting is sent every 10 seconds or
↳if raw illuminance value changes by 1000
    // if min = 0, max = 10 and delta = 0, reporting is sent every 10 seconds
↳regardless of illuminance change
    // Note: On pairing with Zigbee Home Automation or Zigbee2MQTT the reporting
↳schedule will most likely be overwritten with their default settings
    zbIlluminanceSensor.setReporting(1, 0, 1000);
}

/***** Main loop *****/
void loop() {
    // Checking button for factory reset
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3 secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        // force report of illuminance when button is pressed
        zbIlluminanceSensor.report();
    }
    delay(100);
}

```

ZigbeeLight

About The ZigbeeLight class provides a simple on/off light endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for basic lighting control.

Features: * Simple on/off control * State change callbacks

API Reference

Constructor

ZigbeeLight Creates a new Zigbee light endpoint.

```
ZigbeeLight(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Light Control

setLight Sets the light state (on or off).

```
bool setLight (bool state);
```

- state - true to turn on, false to turn off

This function will return true if successful, false otherwise.

getLightState Gets the current light state.

```
bool getLightState();
```

This function will return current light state (true = on, false = off).

restoreLight Restores the light state and triggers any registered callbacks.

```
void restoreLight();
```

Event Handling

onLightChange Sets a callback function to be called when the light state changes.

```
void onLightChange(void (*callback) (bool));
```

- callback - Function to call when light state changes

Example

Basic Light Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates simple Zigbee light bulb.
 *
 * The example demonstrates how to use Zigbee library to create a end device light_
↪bulb.
 * The light bulb is a Zigbee end device, which is controlled by a Zigbee_
↪coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
```

(continues on next page)

(continued from previous page)

```

↪ Scheme.
*
* Please check the README.md for instructions and more detailed description.
*
* Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
*/

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee light bulb configuration */
#define ZIGBEE_LIGHT_ENDPOINT 10
uint8_t led = RGB_BUILTIN;
uint8_t button = BOOT_PIN;

ZigbeeLight zbLight = ZigbeeLight(ZIGBEE_LIGHT_ENDPOINT);

/***** RGB LED functions *****/
void setLED(bool value) {
    digitalWrite(led, value);
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init LED and turn it OFF (if LED_PIN == RGB_BUILTIN, the rgbLedWrite() will
↪ be used under the hood)
    pinMode(led, OUTPUT);
    digitalWrite(led, LOW);

    // Init button for factory reset
    pinMode(button, INPUT_PULLUP);

    //Optional: set Zigbee device name and model
    zbLight.setManufacturerAndModel("Espressif", "ZBLightBulb");

    // Set callback function for light change
    zbLight.onLightChange(setLED);

    //Add endpoint to Zigbee Core
    Serial.println("Adding ZigbeeLight endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbLight);

    // When all EPs are registered, start Zigbee. By default acts as ZIGBEE_END_
↪ DEVICE
    if (!Zigbee.begin()) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();
}

```

(continues on next page)

(continued from previous page)

```

}

void loop() {
  // Checking button for factory reset
  if (digitalRead(button) == LOW) { // Push button pressed
    // Key debounce handling
    delay(100);
    int startTime = millis();
    while (digitalRead(button) == LOW) {
      delay(50);
      if ((millis() - startTime) > 3000) {
        // If key pressed for more than 3secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        delay(1000);
        Zigbee.factoryReset();
      }
    }
    // Toggle light by pressing the button
    zbLight.setLight(!zbLight.getLightState());
  }
  delay(100);
}

```

ZigbeeMultistate

About The ZigbeeMultistate class provides multistate input and output endpoints for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for multistate signal processing and control. Multistate Input (MI) is meant to be used for sensors that provide discrete state values, such as fan speed settings, HVAC modes, or security states to be sent to the coordinator. Multistate Output (MO) is meant to be used for actuators that require discrete state control, such as fan controllers, HVAC systems, or security devices to be controlled by the coordinator.

Note: HomeAssistant ZHA does not support multistate input and output clusters yet.

Common API

Constructor

ZigbeeMultistate Creates a new Zigbee multistate endpoint.

```
ZigbeeMultistate(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Multistate Application Types The ZigbeeMultistate class supports predefined application types with specific state configurations:

```

// Application Type 0: Fan states (Off, On, Auto)
#define ZB_MULTISTATE_APPLICATION_TYPE_0_INDEX      0x0000
#define ZB_MULTISTATE_APPLICATION_TYPE_0_NUM_STATES 3
#define ZB_MULTISTATE_APPLICATION_TYPE_0_STATE_NAMES (const char* const[]) {"Off",
↪ "On", "Auto"}

```

(continues on next page)

(continued from previous page)

```

// Application Type 1: Fan speed states (Off, Low, Medium, High)
#define ZB_MULTISTATE_APPLICATION_TYPE_1_INDEX      0x0001
#define ZB_MULTISTATE_APPLICATION_TYPE_1_NUM_STATES  4
#define ZB_MULTISTATE_APPLICATION_TYPE_1_STATE_NAMES (const char* const[]){ "Off",
↪ "Low", "Medium", "High"}

// Application Type 2: HVAC modes (Auto, Heat, Cool, Off, Emergency Heat, Fan Only,
↪ Max Heat)
#define ZB_MULTISTATE_APPLICATION_TYPE_2_INDEX      0x0002
#define ZB_MULTISTATE_APPLICATION_TYPE_2_NUM_STATES  7
#define ZB_MULTISTATE_APPLICATION_TYPE_2_STATE_NAMES (const char* const[]){ "Auto
↪ ", "Heat", "Cool", "Off", "Emergency Heat", "Fan Only", "Max Heat"}

// Application Type 3: Occupancy states (Occupied, Unoccupied, Standby, Bypass)
#define ZB_MULTISTATE_APPLICATION_TYPE_3_INDEX      0x0003
#define ZB_MULTISTATE_APPLICATION_TYPE_3_NUM_STATES  4
#define ZB_MULTISTATE_APPLICATION_TYPE_3_STATE_NAMES (const char* const[]){
↪ "Occupied", "Unoccupied", "Standby", "Bypass"}

// Application Type 4: Control states (Inactive, Active, Hold)
#define ZB_MULTISTATE_APPLICATION_TYPE_4_INDEX      0x0004
#define ZB_MULTISTATE_APPLICATION_TYPE_4_NUM_STATES  3
#define ZB_MULTISTATE_APPLICATION_TYPE_4_STATE_NAMES (const char* const[]){
↪ "Inactive", "Active", "Hold"}

// Application Type 5: Water system states (Auto, Warm-up, Water Flush,
↪ Autocalibration, Shutdown Open, Shutdown Closed, Low Limit, Test and Balance)
#define ZB_MULTISTATE_APPLICATION_TYPE_5_INDEX      0x0005
#define ZB_MULTISTATE_APPLICATION_TYPE_5_NUM_STATES  8
#define ZB_MULTISTATE_APPLICATION_TYPE_5_STATE_NAMES (const char* const[]){ "Auto
↪ ", "Warm-up", "Water Flush", "Autocalibration", "Shutdown Open", "Shutdown Closed
↪ ", "Low Limit", "Test and Balance"}

// Application Type 6: HVAC system states (Off, Auto, Heat Cool, Heat Only, Cool
↪ Only, Fan Only)
#define ZB_MULTISTATE_APPLICATION_TYPE_6_INDEX      0x0006
#define ZB_MULTISTATE_APPLICATION_TYPE_6_NUM_STATES  6
#define ZB_MULTISTATE_APPLICATION_TYPE_6_STATE_NAMES (const char* const[]){ "Off",
↪ "Auto", "Heat Cool", "Heat Only", "Cool Only", "Fan Only"}

// Application Type 7: Light states (High, Normal, Low)
#define ZB_MULTISTATE_APPLICATION_TYPE_7_INDEX      0x0007
#define ZB_MULTISTATE_APPLICATION_TYPE_7_NUM_STATES  3
#define ZB_MULTISTATE_APPLICATION_TYPE_7_STATE_NAMES (const char* const[]){ "High
↪ ", "Normal", "Low"}

// Application Type 8: Occupancy control states (Occupied, Unoccupied, Startup,
↪ Shutdown)
#define ZB_MULTISTATE_APPLICATION_TYPE_8_INDEX      0x0008
#define ZB_MULTISTATE_APPLICATION_TYPE_8_NUM_STATES  4
#define ZB_MULTISTATE_APPLICATION_TYPE_8_STATE_NAMES (const char* const[]){
↪ "Occupied", "Unoccupied", "Startup", "Shutdown"}

// Application Type 9: Time states (Night, Day, Hold)
#define ZB_MULTISTATE_APPLICATION_TYPE_9_INDEX      0x0009
#define ZB_MULTISTATE_APPLICATION_TYPE_9_NUM_STATES  3
#define ZB_MULTISTATE_APPLICATION_TYPE_9_STATE_NAMES (const char* const[]){ "Night
↪ ", "Day", "Hold"}

// Application Type 10: HVAC control states (Off, Cool, Heat, Auto, Emergency Heat)
#define ZB_MULTISTATE_APPLICATION_TYPE_10_INDEX     0x000A

```

(continues on next page)

(continued from previous page)

```

#define ZB_MULTISTATE_APPLICATION_TYPE_10_NUM_STATES    5
#define ZB_MULTISTATE_APPLICATION_TYPE_10_STATE_NAMES  (const char* const[]){ "Off", "Cool", "Heat", "Auto", "Emergency Heat"}

// Application Type 11: Water control states (Shutdown Closed, Shutdown Open, Satisfied, Mixing, Cooling, Heating, Suppl Heat)
#define ZB_MULTISTATE_APPLICATION_TYPE_11_INDEX        0x000B
#define ZB_MULTISTATE_APPLICATION_TYPE_11_NUM_STATES    7
#define ZB_MULTISTATE_APPLICATION_TYPE_11_STATE_NAMES  (const char* const[]){ "Shutdown Closed", "Shutdown Open", "Satisfied", "Mixing", "Cooling", "Heating", "Suppl Heat"}

// Custom application type for user-defined states
#define ZB_MULTISTATE_APPLICATION_TYPE_OTHER_INDEX      0xFFFF

```

Cluster Management

addMultistateInput Adds multistate input cluster to the endpoint.

```
bool addMultistateInput();
```

This function will return `true` if successful, `false` otherwise.

addMultistateOutput Adds multistate output cluster to the endpoint.

```
bool addMultistateOutput();
```

This function will return `true` if successful, `false` otherwise.

Multistate Input API

Configuration Methods

setMultistateInputApplication Sets the application type for the multistate input.

```
bool setMultistateInputApplication(uint32_t application_type);
```

- `application_type` - Application type constant (see Multistate Application Types above)

This function will return `true` if successful, `false` otherwise.

setMultistateInputDescription Sets a custom description for the multistate input.

```
bool setMultistateInputDescription(const char *description);
```

- `description` - Description string

This function will return `true` if successful, `false` otherwise.

setMultistateInputStates Sets the number of states for the multistate input.

```
bool setMultistateInputStates(uint16_t number_of_states);
```

- `number_of_states` - Number of discrete states (1-65535)

This function will return `true` if successful, `false` otherwise.

Value Control

setMultistateInput Sets the multistate input value.

```
bool setMultistateInput(uint16_t state);
```

- state - State index (0 to number_of_states-1)

This function will return `true` if successful, `false` otherwise.

getMultistateInput Gets the current multistate input value.

```
uint16_t getMultistateInput();
```

This function returns the current multistate input state.

Reporting Methods

reportMultistateInput Manually reports the current multistate input value.

```
bool reportMultistateInput();
```

This function will return `true` if successful, `false` otherwise.

Multistate Output API

Configuration Methods

setMultistateOutputApplication Sets the application type for the multistate output.

```
bool setMultistateOutputApplication(uint32_t application_type);
```

- application_type - Application type constant (see Multistate Application Types above)

This function will return `true` if successful, `false` otherwise.

setMultistateOutputDescription Sets a custom description for the multistate output.

```
bool setMultistateOutputDescription(const char *description);
```

- description - Description string

This function will return `true` if successful, `false` otherwise.

setMultistateOutputStates Sets the number of states for the multistate output.

```
bool setMultistateOutputStates(uint16_t number_of_states);
```

- number_of_states - Number of discrete states (1-65535)

This function will return `true` if successful, `false` otherwise.

Value Control

setMultistateOutput Sets the multistate output value.

```
bool setMultistateOutput(uint16_t state);
```

- state - State index (0 to number_of_states-1)

This function will return `true` if successful, `false` otherwise.

getMultistateOutput Gets the current multistate output value.

```
uint16_t getMultistateOutput();
```

This function returns the current multistate output state.

State Information

getMultistateInputStateNamesLength Gets the number of state names for multistate input.

```
uint16_t getMultistateInputStateNamesLength();
```

This function returns the number of state names configured for multistate input.

getMultistateOutputStateNamesLength Gets the number of state names for multistate output.

```
uint16_t getMultistateOutputStateNamesLength();
```

This function returns the number of state names configured for multistate output.

Reporting Methods

reportMultistateOutput Manually reports the current multistate output value.

```
bool reportMultistateOutput();
```

This function will return `true` if successful, `false` otherwise.

Event Handling

onMultistateOutputChange Sets a callback function to be called when the multistate output value changes.

```
void onMultistateOutputChange(void (*callback)(uint16_t state));
```

- callback - Function to call when multistate output changes, receives the new state value

Example

Multistate Input/Output

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
```

(continues on next page)

(continued from previous page)

```

//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee multistate input / output device.
 *
 * The example demonstrates how to use Zigbee library to create a router_
↳ multistate device.
 * In the example, we have two multistate devices:
 * - zbMultistateDevice: uses defined application states from Zigbee specification
 * - zbMultistateDeviceCustom: uses custom application states (user defined)
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↳ Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * NOTE: HomeAssistant ZHA does not support multistate input and output clusters_
↳ yet.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator/router device mode is not selected in Tools->Zigbee mode
↳ "
#endif

#include "Zigbee.h"

/* Zigbee multistate device configuration */
#define MULTISTATE_DEVICE_ENDPOINT_NUMBER 1

uint8_t button = BOOT_PIN;

// zbMultistateDevice will use defined application states
ZigbeeMultistate zbMultistateDevice = ZigbeeMultistate(MULTISTATE_DEVICE_ENDPOINT_
↳ NUMBER);

// zbMultistateDeviceCustom will use custom application states (user defined)
ZigbeeMultistate zbMultistateDeviceCustom = ZigbeeMultistate(MULTISTATE_DEVICE_
↳ ENDPOINT_NUMBER + 1);

const char *multistate_custom_state_names[6] = {"Off", "On", "UltraSlow", "Slow",
↳ "Fast", "SuperFast"};

void onStateChange(uint16_t state) {
    // print the state
    Serial.printf("Received state change: %u\r\n", state);
    // print the state name using the stored state names
    const char *const *state_names = ZB_MULTISTATE_APPLICATION_TYPE_7_STATE_NAMES;
    if (state_names && state < zbMultistateDevice.
↳ getMultistateOutputStateNamesLength()) {
        Serial.printf("State name: %s\r\n", state_names[state]);
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
    // print state index of possible options
    Serial.printf("State index: %u / %u\r\n", state, zbMultistateDevice.
↪getMultistateOutputStateNamesLength() - 1);
}

void onStateChangeCustom(uint16_t state) {
    // print the state
    Serial.printf("Received state change: %u\r\n", state);
    // print the state name using the stored state names
    if (state < zbMultistateDeviceCustom.getMultistateOutputStateNamesLength()) {
        Serial.printf("State name: %s\r\n", multistate_custom_state_names[state]);
    }
    // print state index of possible options
    Serial.printf("State index: %u / %u\r\n", state, zbMultistateDeviceCustom.
↪getMultistateOutputStateNamesLength() - 1);

    Serial.print("Changing to fan mode to: ");
    switch (state) {
        case 0: Serial.println("Off"); break;
        case 1: Serial.println("On"); break;
        case 2: Serial.println("UltraSlow"); break;
        case 3: Serial.println("Slow"); break;
        case 4: Serial.println("Fast"); break;
        case 5: Serial.println("SuperFast"); break;
        default: Serial.println("Invalid state"); break;
    }
}

void setup() {
    log_d("Starting serial");
    Serial.begin(115200);

    // Init button switch
    log_d("Init button switch");
    pinMode(button, INPUT_PULLUP);

    // Optional: set Zigbee device name and model
    log_d("Set Zigbee device name and model");
    zbMultistateDevice.setManufacturerAndModel("Espressif", "ZigbeeMultistateDevice
↪");

    // Set up analog input
    log_d("Add Multistate Input");
    zbMultistateDevice.addMultistateInput();
    log_d("Set Multistate Input Application");
    zbMultistateDevice.setMultistateInputApplication(ZB_MULTISTATE_APPLICATION_TYPE_
↪0_INDEX);
    log_d("Set Multistate Input Description");
    zbMultistateDevice.setMultistateInputDescription("Fan (on/off/auto)");
    zbMultistateDevice.setMultistateInputStates(ZB_MULTISTATE_APPLICATION_TYPE_0_NUM_
↪STATES);

    // Set up analog output
    log_d("Add Multistate Output");
    zbMultistateDevice.addMultistateOutput();
    log_d("Set Multistate Output Application");
    zbMultistateDevice.setMultistateOutputApplication(ZB_MULTISTATE_APPLICATION_TYPE_
↪7_INDEX);
    log_d("Set Multistate Output Description");
    zbMultistateDevice.setMultistateOutputDescription("Light (high/normal/low)");

```

(continues on next page)

(continued from previous page)

```

    zbMultistateDevice.setMultistateOutputStates(ZB_MULTISTATE_APPLICATION_TYPE_7_
    ↪NUM_STATES);

    // Set up custom output
    log_d("Add Multistate Output");
    zbMultistateDeviceCustom.addMultistateOutput();
    log_d("Set Multistate Output Application");
    zbMultistateDeviceCustom.setMultistateOutputApplication(ZB_MULTISTATE_
    ↪APPLICATION_TYPE_OTHER_INDEX);
    log_d("Set Multistate Output Description");
    zbMultistateDeviceCustom.setMultistateOutputDescription("Fan (on/off/slow/medium/
    ↪fast)");
    zbMultistateDeviceCustom.setMultistateOutputStates(5);

    // Set callback function for multistate output change
    log_d("Set callback function for multistate output change");
    zbMultistateDevice.onMultistateOutputChange(onStateChange);
    zbMultistateDeviceCustom.onMultistateOutputChange(onStateChangeCustom);

    // Add endpoints to Zigbee Core
    log_d("Add endpoints to Zigbee Core");
    Zigbee.addEndpoint(&zbMultistateDevice);
    Zigbee.addEndpoint(&zbMultistateDeviceCustom);

    Serial.println("Starting Zigbee...");
    // When all EPs are registered, start Zigbee in Router Device mode
    if (!Zigbee.begin(ZIGBEE_ROUTER)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    } else {
        Serial.println("Zigbee started successfully!");
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println("Connected");
}

void loop() {
    // Checking button for factory reset and reporting
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        // For demonstration purposes, increment the multistate output/input value by 1
        if (zbMultistateDevice.getMultistateOutput() < zbMultistateDevice.
    ↪getMultistateOutputStateNamesLength() - 1) {
            zbMultistateDevice.setMultistateOutput(zbMultistateDevice.
    ↪getMultistateOutput() + 1);

```

(continues on next page)

(continued from previous page)

```

        zbMultistateDevice.reportMultistateOutput();
        zbMultistateDevice.setMultistateInput(zbMultistateDevice.
↪getMultistateOutput());
        zbMultistateDevice.reportMultistateInput();
    } else {
        zbMultistateDevice.setMultistateOutput(0);
        zbMultistateDevice.reportMultistateOutput();
        zbMultistateDevice.setMultistateInput(zbMultistateDevice.
↪getMultistateOutput());
        zbMultistateDevice.reportMultistateInput();
    }

    if (zbMultistateDeviceCustom.getMultistateOutput() < zbMultistateDeviceCustom.
↪getMultistateOutputStateNamesLength() - 1) {
        zbMultistateDeviceCustom.setMultistateOutput(zbMultistateDeviceCustom.
↪getMultistateOutput() + 1);
        zbMultistateDeviceCustom.reportMultistateOutput();
    } else {
        zbMultistateDeviceCustom.setMultistateOutput(0);
        zbMultistateDeviceCustom.reportMultistateOutput();
    }
}
delay(100);
}

```

ZigbeeOccupancySensor

About The ZigbeeOccupancySensor class provides an endpoint for occupancy sensors in Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for occupancy detection devices, supporting various sensor types for detecting presence.

Features: * Occupancy detection (occupied/unoccupied) * Multiple sensor type support (PIR, ultrasonic, etc.) * Automatic reporting capabilities * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

API Reference

Constructor

ZigbeeOccupancySensor Creates a new Zigbee occupancy sensor endpoint.

```
ZigbeeOccupancySensor(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

API Methods

setOccupancy Sets the occupancy state.

```
bool setOccupancy(bool occupied);
```

- occupied - Occupancy state (true = occupied, false = unoccupied)

This function will return `true` if successful, `false` otherwise.

setSensorType Sets the sensor type.

```
bool setSensorType(uint8_t sensor_type);
```

- `sensor_type` - Sensor type identifier (see `esp_zb_zcl_occupancy_sensing_occupancy_sensor_type_t`)

This function will return `true` if successful, `false` otherwise.

report Manually reports the current occupancy state.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

Example

Occupancy Sensor Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee occupancy sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device_
↳ occupancy sensor.
 * The occupancy sensor is a Zigbee end device, which is reporting data to the_
↳ Zigbee network.
 * Tested with PIR sensor HC-SR501 connected to GPIO4.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↳ Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee occupancy sensor configuration */
#define OCCUPANCY_SENSOR_ENDPOINT_NUMBER 10
uint8_t button = BOOT_PIN;
uint8_t sensor_pin = 4;
```

(continues on next page)

(continued from previous page)

```

ZigbeeOccupancySensor zbOccupancySensor = ZigbeeOccupancySensor(OCCUPANCY_SENSOR_
↪ENDPOINT_NUMBER);

void setup() {
  Serial.begin(115200);

  // Init button + PIR sensor
  pinMode(button, INPUT_PULLUP);
  pinMode(sensor_pin, INPUT);

  // Optional: set Zigbee device name and model
  zbOccupancySensor.setManufacturerAndModel("Espressif", "ZigbeeOccupancyPIRSensor
↪");

  // Add endpoint to Zigbee Core
  Zigbee.addEndpoint(&zbOccupancySensor);

  Serial.println("Starting Zigbee...");
  // When all EPs are registered, start Zigbee in End Device mode
  if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
  } else {
    Serial.println("Zigbee started successfully!");
  }
  Serial.println("Connecting to network");
  while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
  }
  Serial.println();
}

void loop() {
  // Checking PIR sensor for occupancy change
  static bool occupancy = false;
  if (digitalRead(sensor_pin) == HIGH && !occupancy) {
    // Update occupancy sensor value
    zbOccupancySensor.setOccupancy(true);
    zbOccupancySensor.report();
    occupancy = true;
  } else if (digitalRead(sensor_pin) == LOW && occupancy) {
    zbOccupancySensor.setOccupancy(false);
    zbOccupancySensor.report();
    occupancy = false;
  }

  // Checking button for factory reset
  if (digitalRead(button) == LOW) { // Push button pressed
    // Key debounce handling
    delay(100);
    int startTime = millis();
    while (digitalRead(button) == LOW) {
      delay(50);
      if ((millis() - startTime) > 3000) {
        // If key pressed for more than 3secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        delay(1000);
        Zigbee.factoryReset();
      }
    }
  }
}

```

(continues on next page)

(continued from previous page)

```
    }  
  }  
}  
delay(100);  
}
```

ZigbeePM25Sensor

About The `ZigbeePM25Sensor` class provides a PM2.5 air quality sensor endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for particulate matter measurement devices.

Features: * PM2.5 concentration measurement in $\mu\text{g}/\text{m}^3$ * Configurable measurement range * Tolerance and reporting configuration * Automatic reporting capabilities

API Reference

Constructor

ZigbeePM25Sensor Creates a new Zigbee PM2.5 sensor endpoint.

```
ZigbeePM25Sensor(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

API Methods

setPM25 Sets the PM2.5 concentration measurement value.

```
bool setPM25(float pm25);
```

- `pm25` - PM2.5 concentration value in $0.1 \mu\text{g}/\text{m}^3$

This function will return `true` if successful, `false` otherwise.

setMinMaxValue Sets the minimum and maximum measurement values.

```
bool setMinMaxValue(float min, float max);
```

- `min` - Minimum PM2.5 concentration value in $0.1 \mu\text{g}/\text{m}^3$
- `max` - Maximum PM2.5 concentration value in $0.1 \mu\text{g}/\text{m}^3$

This function will return `true` if successful, `false` otherwise.

setDefaultValue Sets the default (initial) value for the PM2.5 sensor in $0.1 \mu\text{g}/\text{m}^3$. This value will be used as the initial measured value when the device is in factory reset mode and before the sensor provides actual readings.

```
bool setDefaultValue(float defaultValue);
```

- `defaultValue` - Default PM2.5 concentration value in $0.1 \mu\text{g}/\text{m}^3$

Important: Must be called before adding the EP to Zigbee class. Only effective when the device is in factory reset mode (before commissioning/joining a network).

This function will return `true` if successful, `false` otherwise.

setTolerance Sets the tolerance value for measurements.

```
bool setTolerance(float tolerance);
```

- tolerance - Tolerance value in 0.1 µg/m³

This function will return `true` if successful, `false` otherwise.

setReporting Sets the reporting configuration for PM2.5 measurements.

```
bool setReporting(uint16_t min_interval, uint16_t max_interval, float delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change required to trigger a report in 0.1 µg/m³

This function will return `true` if successful, `false` otherwise.

report Manually reports the current PM2.5 concentration value.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

Example

PM2.5 Sensor Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates Zigbee PM2.5 sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device PM2.5_
 * sensor.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
 * Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
```

(continues on next page)

(continued from previous page)

```

#endif

#include "Zigbee.h"

/* Zigbee PM2.5 sensor configuration */
#define PM2_5_SENSOR_ENDPOINT_NUMBER 1
uint8_t button = BOOT_PIN;

ZigbeePM25Sensor zbPM25Sensor = ZigbeePM25Sensor(PM2_5_SENSOR_ENDPOINT_NUMBER);

void setup() {
    Serial.begin(115200);

    // Init button switch
    pinMode(button, INPUT_PULLUP);

    // Optional: set Zigbee device name and model
    zbPM25Sensor.setManufacturerAndModel("Espressif", "ZigbeePM25Sensor");

    // Set minimum and maximum PM2.5 measurement value in  $\mu\text{g}/\text{m}^3$ 
    zbPM25Sensor.setMinMaxValue(0, 350);

    // Set tolerance for PM2.5 measurement in  $\mu\text{g}/\text{m}^3$ 
    zbPM25Sensor.setTolerance(0.1);

    // Add endpoints to Zigbee Core
    Zigbee.addEndpoint(&zbPM25Sensor);

    Serial.println("Starting Zigbee...");
    // When all EPs are registered, start Zigbee in End Device mode
    if (!Zigbee.begin()) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    } else {
        Serial.println("Zigbee started successfully!");
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();

    // Set reporting interval for PM2.5 measurement to be done every 30 seconds,
    ↪ must be called after Zigbee.begin()
    // min_interval and max_interval in seconds, delta (PM2.5 change in  $\mu\text{g}/\text{m}^3$ )
    // if min = 1 and max = 0, reporting is sent only when PM2.5 changes by delta
    // if min = 0 and max = 10, reporting is sent every 10 seconds or when PM2.5
    ↪ changes by delta
    // if min = 0, max = 10 and delta = 0, reporting is sent every 10 seconds
    ↪ regardless of delta change
    zbPM25Sensor.setReporting(0, 30, 0);
}

void loop() {
    static uint32_t timeCounter = 0;
    // Read PM2.5 sensor every 2s
    if (!(timeCounter++ % 20)) { // delaying for 100ms x 20 = 2s
        // Read sensor value - here is chip temperature used + 50 as a dummy value for
        ↪ demonstration

```

(continues on next page)

(continued from previous page)

```

float pm25_value = 50.5 + temperatureRead();
Serial.printf("Updating PM2.5 sensor value to %0.1f µg/m³\r\n", pm25_value);
zbPM25Sensor.setPM25(pm25_value);
}

// Checking button for factory reset and reporting
if (digitalRead(button) == LOW) { // Push button pressed
    // Key debounce handling
    delay(100);
    int startTime = millis();
    while (digitalRead(button) == LOW) {
        delay(50);
        if ((millis() - startTime) > 3000) {
            // If key pressed for more than 3secs, factory reset Zigbee and reboot
            Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
            delay(1000);
            Zigbee.factoryReset();
        }
    }
    zbPM25Sensor.report();
}
delay(100);
}

```

ZigbeePowerOutlet

About The ZigbeePowerOutlet class provides a smart power outlet endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for power control, allowing remote on/off control of electrical devices.

Features: * On/off power control * State change callbacks

API Reference

Constructor

ZigbeePowerOutlet Creates a new Zigbee power outlet endpoint.

```
ZigbeePowerOutlet(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Power Control

setState Sets the power outlet state (on or off).

```
bool setState(bool state);
```

- state - true to turn on, false to turn off

This function will return true if successful, false otherwise.

getPowerOutletState Gets the current power outlet state.

```
bool getPowerOutletState();
```

This function will return current power state (true = on, false = off).

restoreState Restores the power outlet state and triggers any registered callbacks.

```
void restoreState();
```

Event Handling

onPowerOutletChange Sets a callback function to be called when the power outlet state changes.

```
void onPowerOutletChange(void (*callback) (bool));
```

- callback - Function to call when power outlet state changes

Example

Smart Power Outlet Implementation

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates simple Zigbee power outlet.
 *
 * The example demonstrates how to use Zigbee library to create a end device power_
↳outlet.
 * The power outlet is a Zigbee end device, which is controlled by a Zigbee_
↳coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↳Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Ludovic Boué (https://github.com/lboué)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee router mode is not selected in Tools->Zigbee mode"
#endif
```

(continues on next page)

(continued from previous page)

```

#include "Zigbee.h"

/* Zigbee power outlet configuration */
#define ZIGBEE_OUTLET_ENDPOINT 1

#ifdef LED_BUILTIN // Use built-in LED if defined for the board
uint8_t led = LED_BUILTIN;
#else
uint8_t led = 2; // Use custom LED pin
#endif

uint8_t button = BOOT_PIN;

ZigbeePowerOutlet zbOutlet = ZigbeePowerOutlet(ZIGBEE_OUTLET_ENDPOINT);

/***** RGB LED functions *****/
void setLED(bool value) {
    digitalWrite(led, value);
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init LED and turn it OFF (if LED_PIN == RGB_BUILTIN, the rgbLedWrite() will
    ↳ be used under the hood)
    pinMode(led, OUTPUT);
    digitalWrite(led, LOW);

    // Init button for factory reset
    pinMode(button, INPUT_PULLUP);

    //Optional: set Zigbee device name and model
    zbOutlet.setManufacturerAndModel("Espressif", "ZBPowerOutlet");

    // Set callback function for power outlet change
    zbOutlet.onPowerOutletChange(setLED);

    //Add endpoint to Zigbee Core
    Serial.println("Adding ZigbeePowerOutlet endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbOutlet);

    // When all EPs are registered, start Zigbee. By default acts as ZIGBEE_END_
    ↳ DEVICE
    if (!Zigbee.begin(ZIGBEE_ROUTER)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();
}

void loop() {
    // Checking button for factory reset
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling

```

(continues on next page)

(continued from previous page)

```

delay(100);
int startTime = millis();
while (digitalRead(button) == LOW) {
    delay(50);
    if ((millis() - startTime) > 3000) {
        // If key pressed for more than 3secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        delay(1000);
        Zigbee.factoryReset();
    }
}
// Toggle state by pressing the button
zbOutlet.setState(!zbOutlet.getPowerOutletState());
}
delay(100);
}

```

ZigbeePressureSensor

About The ZigbeePressureSensor class provides an endpoint for pressure sensors in Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for pressure measurement devices, supporting atmospheric pressure, barometric pressure, and other pressure measurements.

Features: * Pressure measurement in hPa (hectopascals) * Configurable measurement range * Tolerance and reporting configuration * Automatic reporting capabilities

API Reference

Constructor

ZigbeePressureSensor Creates a new Zigbee pressure sensor endpoint.

```
ZigbeePressureSensor(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

API Methods

setPressure Sets the pressure measurement value.

```
bool setPressure(int16_t value);
```

- value - Pressure value in hPa

This function will return `true` if successful, `false` otherwise.

setMinMaxValue Sets the minimum and maximum measurement values.

```
bool setMinMaxValue(int16_t min, int16_t max);
```

- min - Minimum pressure value in hPa
- max - Maximum pressure value in hPa

This function will return `true` if successful, `false` otherwise.

setDefaultValue Sets the default (initial) value for the pressure sensor in 1 hPa. This value will be used as the initial measured value when the device is in factory reset mode and before the sensor provides actual readings.

```
bool setDefaultValue(int16_t defaultValue);
```

- `defaultValue` - Default pressure value in hPa

Important: Must be called before adding the EP to Zigbee class. Only effective when the device is in factory reset mode (before commissioning/joining a network).

This function will return `true` if successful, `false` otherwise.

setTolerance Sets the tolerance value for measurements.

```
bool setTolerance(uint16_t tolerance);
```

- `tolerance` - Tolerance value in hPa

This function will return `true` if successful, `false` otherwise.

setReporting Sets the reporting configuration for pressure measurements.

```
bool setReporting(uint16_t min_interval, uint16_t max_interval, uint16_t delta);
```

- `min_interval` - Minimum reporting interval in seconds
- `max_interval` - Maximum reporting interval in seconds
- `delta` - Minimum change required to trigger a report in hPa

This function will return `true` if successful, `false` otherwise.

report Manually reports the current pressure value.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

Example

Pressure + Flow Sensor Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates Zigbee temperature sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device_
 * ↳temperature sensor.
```

(continues on next page)

(continued from previous page)

```

* The temperature sensor is a Zigbee end device, which is controlled by a Zigbee_
↪coordinator.
*
* Proper Zigbee mode must be selected in Tools->Zigbee mode
* and also the correct partition scheme must be selected in Tools->Partition_
↪Scheme.
*
* Please check the README.md for instructions and more detailed description.
*
* Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
*/

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee flow + pressure sensor configuration */
#define FLOW_SENSOR_ENDPOINT_NUMBER 10
#define PRESSURE_SENSOR_ENDPOINT_NUMBER 11

uint8_t button = BOOT_PIN;

ZigbeeFlowSensor zbFlowSensor = ZigbeeFlowSensor(FLOW_SENSOR_ENDPOINT_NUMBER);
ZigbeePressureSensor zbPressureSensor = ZigbeePressureSensor(PRESSURE_SENSOR_
↪ENDPOINT_NUMBER);

void setup() {
  Serial.begin(115200);

  // Init button switch
  pinMode(button, INPUT_PULLUP);

  // Optional: set Zigbee device name and model
  zbFlowSensor.setManufacturerAndModel("Espressif", "ZigbeeFlowSensor");

  // Set minimum and maximum flow measurement value in 0,1 m3/h
  zbFlowSensor.setMinMaxValue(0.0, 100.0);

  // Optional: Set tolerance for flow measurement in 0,1 m3/h
  zbFlowSensor.setTolerance(1.0);

  // Optional: set Zigbee device name and model
  zbPressureSensor.setManufacturerAndModel("Espressif", "ZigbeePressureSensor");

  // Set minimum and maximum pressure measurement value in hPa
  zbPressureSensor.setMinMaxValue(0, 10000);

  // Optional: Set tolerance for pressure measurement in hPa
  zbPressureSensor.setTolerance(1);

  // Add endpoints to Zigbee Core
  Zigbee.addEndpoint(&zbFlowSensor);
  Zigbee.addEndpoint(&zbPressureSensor);

  Serial.println("Starting Zigbee...");
  // When all EPs are registered, start Zigbee in End Device mode
  if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
  }
}

```

(continues on next page)

(continued from previous page)

```

    Serial.println("Rebooting...");
    ESP.restart();
} else {
    Serial.println("Zigbee started successfully!");
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();

// Set reporting interval for flow and pressure measurement in seconds, must be
↳called after Zigbee.begin()
// min_interval and max_interval in seconds, delta (pressure change in hPa, flow
↳change in 0,1 m3/h)
// if min = 1 and max = 0, reporting is sent only when temperature changes by
↳delta
// if min = 0 and max = 10, reporting is sent every 10 seconds or temperature
↳changes by delta
// if min = 0, max = 10 and delta = 0, reporting is sent every 10 seconds
↳regardless of delta change
zbFlowSensor.setReporting(0, 30, 1.0);
zbPressureSensor.setReporting(0, 30, 1);
}

void loop() {
    static uint32_t timeCounter = 0;

    // Read flow and pressure sensors every 2s
    if (!(timeCounter++ % 20)) { // delaying for 100ms x 20 = 2s
        float flow_value = temperatureRead();
        uint16_t pressure_value = (uint16_t)temperatureRead() * 100; // *100 for
↳demonstration so the value is in 1000-3000hPa
        Serial.printf("Updating flow sensor value to %.2f m3/h\r\n", flow_value);
        zbFlowSensor.setFlow(flow_value);
        Serial.printf("Updating pressure sensor value to %u hPa\r\n", pressure_value);
        zbPressureSensor.setPressure(pressure_value);
    }

    // Checking button for factory reset and reporting
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
        zbFlowSensor.report();
        zbPressureSensor.report();
    }
    delay(100);
}
}

```

ZigbeeRangeExtender

About The ZigbeeRangeExtender class provides a range extender endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for range extender devices that help extend the coverage of Zigbee networks by acting as repeaters.

API Reference

Constructor

ZigbeeRangeExtender Creates a new Zigbee range extender endpoint.

```
ZigbeeRangeExtender(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Example

Range Extender Implementation

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates simple Zigbee Range Extender (router).
 *
 * The example demonstrates how to use Zigbee library to create a Zigbee network_
 * →ragbe extender (router).
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
 * →Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifdef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator/router mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee light bulb configuration */
#define USE_CUSTOM_ZIGBEE_CONFIG 1
#define ZIGBEE_EXTENDER_ENDPOINT 1
```

(continues on next page)

(continued from previous page)

```

#ifndef LED_BUILTIN
#define LED_BUILTIN 4
#endif

uint8_t led = LED_BUILTIN;
uint8_t button = BOOT_PIN;

ZigbeeRangeExtender zbExtender = ZigbeeRangeExtender(ZIGBEE_EXTENDER_ENDPOINT);

/***** Identify *****/
// Create a task on identify call to handle the identify function
void identify(uint16_t time) {
    static uint8_t blink = 1;
    log_d("Identify called for %u seconds", time);
    if (time == 0) {
        digitalWrite(led, LOW);
        return;
    }
    digitalWrite(led, blink);
    blink = !blink;
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init LED and turn it OFF (if LED_PIN == RGB_BUILTIN, the rgbLedWrite() will
    // be used under the hood)
    pinMode(led, OUTPUT);
    digitalWrite(led, LOW);

    // Init button for factory reset
    pinMode(button, INPUT_PULLUP);

    // Optional: Set callback function for device identify
    zbExtender.onIdentify(identify);

    // Optional: Set Zigbee device name and model
    zbExtender.setManufacturerAndModel("Espressif", "ZigbeeRangeExtender");

    // Add endpoint to Zigbee Core
    Serial.println("Adding Zigbee Extender endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbExtender);

#ifdef USE_CUSTOM_ZIGBEE_CONFIG
    // Optional: Create a custom Zigbee configuration for Zigbee Extender
    esp_zb_cfg_t zigbeeConfig = ZIGBEE_DEFAULT_ROUTER_CONFIG();
    zigbeeConfig.nwk_cfg.zcwr_cfg.max_children = 20; // 10 is default

    // When all EPs are registered, start Zigbee with custom config
    if (!Zigbee.begin(&zigbeeConfig)) {
#else
    // When all EPs are registered, start Zigbee as ROUTER device
    if (!Zigbee.begin(ZIGBEE_ROUTER)) {
#endif
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }
    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {

```

(continues on next page)

(continued from previous page)

```

    Serial.print(".");
    delay(100);
}
Serial.println();
}

void loop() {
    // Checking button for factory reset
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(button) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
                Zigbee.factoryReset();
            }
        }
    }
}
}

```

ZigbeeSwitch

About The ZigbeeSwitch class provides a switch endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for controlling other devices (typically lights) through on/off commands.

Features: * On/off control commands for bound devices * Group control support * Direct device addressing

API Reference

Constructor

ZigbeeSwitch Creates a new Zigbee switch endpoint.

```
ZigbeeSwitch(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Basic Control Commands

lightToggle Toggles the state of bound lights (on to off, or off to on).

```

void lightToggle();
void lightToggle(uint16_t group_addr);
void lightToggle(uint8_t endpoint, uint16_t short_addr);
void lightToggle(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);

```

- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

lightOn Turns on bound lights.

```
void lightOn();
void lightOn(uint16_t group_addr);
void lightOn(uint8_t endpoint, uint16_t short_addr);
void lightOn(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

lightOff Turns off bound lights.

```
void lightOff();
void lightOff(uint16_t group_addr);
void lightOff(uint8_t endpoint, uint16_t short_addr);
void lightOff(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- group_addr - Group address to control (optional)
- endpoint - Target device endpoint (optional)
- short_addr - Target device short address (optional)
- ieee_addr - Target device IEEE address (optional)

Advanced Control Commands

lightOffWithEffect Turns off lights with a specific effect.

```
void lightOffWithEffect(uint8_t effect_id, uint8_t effect_variant);
```

- effect_id - Effect identifier
- effect_variant - Effect variant

lightOnWithTimedOff Turns on lights with automatic turn-off after specified time.

```
void lightOnWithTimedOff(uint8_t on_off_control, uint16_t time_on, uint16_t time_
↳ off);
```

- on_off_control - Control byte
- time_on - Time to stay on (in 1/10th seconds)
- time_off - Time to stay off (in 1/10th seconds)

lightOnWithSceneRecall Turns on lights by recalling a scene.

```
void lightOnWithSceneRecall();
```

Example

Basic Switch Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
```

(continues on next page)

(continued from previous page)

```

//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates simple Zigbee light switch.
 *
 * The example demonstrates how to use Zigbee library to control a light bulb.
 * The light bulb is a Zigbee end device, which is controlled by a Zigbee_
↪coordinator (Switch).
 * Button switch and Zigbee runs in separate tasks.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee switch configuration */
#define SWITCH_ENDPOINT_NUMBER 5

#define GPIO_INPUT_IO_TOGGLE_SWITCH BOOT_PIN
#define PAIR_SIZE(TYPE_STR_PAIR)      (sizeof(TYPE_STR_PAIR) / sizeof(TYPE_STR_
↪PAIR[0]))

typedef enum {
    SWITCH_ON_CONTROL,
    SWITCH_OFF_CONTROL,
    SWITCH_ONOFF_TOGGLE_CONTROL,
    SWITCH_LEVEL_UP_CONTROL,
    SWITCH_LEVEL_DOWN_CONTROL,
    SWITCH_LEVEL_CYCLE_CONTROL,
    SWITCH_COLOR_CONTROL,
} SwitchFunction;

typedef struct {
    uint8_t pin;
    SwitchFunction func;
} SwitchData;

typedef enum {
    SWITCH_IDLE,
    SWITCH_PRESS_ARMED,
    SWITCH_PRESS_DETECTED,
    SWITCH_PRESSED,
    SWITCH_RELEASE_DETECTED,

```

(continues on next page)

(continued from previous page)

```

} SwitchState;

static SwitchData buttonFunctionPair[] = {{GPIO_INPUT_IO_TOGGLE_SWITCH, SWITCH_
↪ONOFF_TOGGLE_CONTROL}};

ZigbeeSwitch zbSwitch = ZigbeeSwitch(SWITCH_ENDPOINT_NUMBER);

static bool light_state = false;

/***** Zigbee functions *****/
static void onZbButton(SwitchData *button_func_pair) {
    if (button_func_pair->func == SWITCH_ONOFF_TOGGLE_CONTROL) {
        // Send toggle command to the light
        Serial.println("Toggling light");
        zbSwitch.lightToggle();
    }
}

static void onLightStateChange(bool state) {
    if (state != light_state) {
        light_state = state;
        Serial.printf("Light state changed to %d\r\n", state);
    }
}

/***** Periodic task *****/
void periodicTask(void *arg) {
    while (true) {
        // print the bound lights every 10 seconds
        static uint32_t lastPrint = 0;
        if (millis() - lastPrint > 10000) {
            lastPrint = millis();
            zbSwitch.printBoundDevices(Serial);
        }

        // Poll light state every second
        static uint32_t lastPoll = 0;
        if (millis() - lastPoll > 1000) {
            lastPoll = millis();
            zbSwitch.getLightState();
        }
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

/***** GPIO functions *****/
static QueueHandle_t gpio_evt_queue = NULL;

static void IRAM_ATTR onGpioInterrupt(void *arg) {
    xQueueSendFromISR(gpio_evt_queue, (SwitchData *)arg, NULL);
}

static void enableGpioInterrupt(bool enabled) {
    for (int i = 0; i < PAIR_SIZE(buttonFunctionPair); ++i) {
        if (enabled) {
            enableInterrupt((buttonFunctionPair[i]).pin);
        } else {
            disableInterrupt((buttonFunctionPair[i]).pin);
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    //Optional: set Zigbee device name and model
    zbSwitch.setManufacturerAndModel("Espressif", "ZigbeeSwitch");

    //Optional to allow multiple light to bind to the switch
    zbSwitch.allowMultipleBinding(true);

    zbSwitch.onLightStateChange(onLightStateChange);

    //Add endpoint to Zigbee Core
    Serial.println("Adding ZigbeeSwitch endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbSwitch);

    //Open network for 180 seconds after boot
    Zigbee.setRebootOpenNetwork(180);

    // Init button switch
    for (int i = 0; i < PAIR_SIZE(buttonFunctionPair); i++) {
        pinMode(buttonFunctionPair[i].pin, INPUT_PULLUP);
        /* create a queue to handle gpio event from isr */
        gpio_evt_queue = xQueueCreate(10, sizeof(SwitchData));
        if (gpio_evt_queue == 0) {
            Serial.println("Queue creating failed, rebooting...");
            ESP.restart();
        }
        attachInterruptArg(buttonFunctionPair[i].pin, onGpioInterrupt, (void_
↪ *) (buttonFunctionPair + i), FALLING);
    }

    // When all EPs are registered, start Zigbee with ZIGBEE_COORDINATOR mode
    if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }

    Serial.println("Waiting for Light to bound to the switch");
    //Wait for switch to bound to a light:
    while (!zbSwitch.bound()) {
        Serial.printf(".");
        delay(500);
    }

    // Optional: List all bound devices and read manufacturer and model name
    std::list<zb_device_params_t *> boundLights = zbSwitch.getBoundDevices();
    for (const auto &device : boundLights) {
        Serial.printf("Device on endpoint %u, short address: 0x%x\r\n", device->
↪ endpoint, device->short_addr);
        Serial.printf(
            "IEEE Address: %02X:%02X:%02X:%02X:%02X:%02X:%02X:%02X\r\n", device->ieee_
↪ addr[7], device->ieee_addr[6], device->ieee_addr[5], device->ieee_addr[4],
            device->ieee_addr[3], device->ieee_addr[2], device->ieee_addr[1], device->
↪ ieee_addr[0]
        );
        char *manufacturer = zbSwitch.readManufacturer(device->endpoint, device->short_
↪ addr, device->ieee_addr);
        char *model = zbSwitch.readModel(device->endpoint, device->short_addr, device->

```

(continues on next page)

(continued from previous page)

```

↪ieee_addr);
    if (manufacturer != nullptr) {
        Serial.printf("Light manufacturer: %s\r\n", manufacturer);
    }
    if (model != nullptr) {
        Serial.printf("Light model: %s\r\n", model);
    }
}

Serial.println();

xTaskCreate(periodicTask, "periodicTask", 1024 * 4, NULL, 10, NULL);
}

void loop() {
    // Handle button switch in loop()
    uint8_t pin = 0;
    SwitchData buttonSwitch;
    static SwitchState buttonState = SWITCH_IDLE;
    bool eventFlag = false;

    /* check if there is any queue received, if yes read out the buttonSwitch */
    if (xQueueReceive(gpio_evt_queue, &buttonSwitch, portMAX_DELAY)) {
        pin = buttonSwitch.pin;
        enableGpioInterrupt(false);
        eventFlag = true;
    }
    while (eventFlag) {
        bool value = digitalRead(pin);
        switch (buttonState) {
            case SWITCH_IDLE:
                buttonState = (value == LOW) ? SWITCH_PRESS_
↪DETECTED : SWITCH_IDLE; break;
            case SWITCH_PRESS_DETECTED: buttonState = (value == LOW) ? SWITCH_PRESS_
↪DETECTED : SWITCH_RELEASE_DETECTED; break;
            case SWITCH_RELEASE_DETECTED:
                buttonState = SWITCH_IDLE;
                /* callback to button_handler */
                (*onZbButton)(&buttonSwitch);
                break;
            default: break;
        }
        if (buttonState == SWITCH_IDLE) {
            enableGpioInterrupt(true);
            eventFlag = false;
            break;
        }
        vTaskDelay(10 / portTICK_PERIOD_MS);
    }
}

```

Multi Switch Implementation

```

// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//

```

(continues on next page)

(continued from previous page)

```

// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates simple Zigbee multi-light switch.
 *
 * The example demonstrates how to use Zigbee library to control multiple light
↪bulbs.
 * The light bulbs are Zigbee devices, which are controlled by a Zigbee
↪coordinator/router (Multi-Switch).
 * Settings are stored in NVS to not be lost after power loss.
 * Configuring and controlling the lights is done via serial input.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition
↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"
#include <Preferences.h>

#define ZIGBEE_ROLE ZIGBEE_ROUTER // ZIGBEE_ROUTER for HomeAssistant integration,
↪ZIGBEE_COORDINATOR for running own network

/* Zigbee switch configuration */
#define SWITCH_ENDPOINT_NUMBER 1

uint8_t button = BOOT_PIN;

ZigbeeSwitch zbSwitch = ZigbeeSwitch(SWITCH_ENDPOINT_NUMBER);

int buttonState;
int lastButtonState = LOW;
unsigned long lastDebounceTime = 0; // the last time the output pin was toggled
unsigned long debounceDelay = 50; // the debounce time; increase if the output
↪flickers

// To be stored in NVS to not be lost after power loss
Preferences prefs;

zb_device_params_t light_1;
zb_device_params_t light_2;
zb_device_params_t light_3;

void storeLightParams(zb_device_params_t *light, int light_number) {
    char key[10];
    snprintf(key, sizeof(key), "light_%d", light_number);
    prefs.putBytes(key, light, sizeof(zb_device_params_t));
}

```

(continues on next page)

(continued from previous page)

```

void loadLightParams(zb_device_params_t *light, int light_number) {
    char key[10];
    snprintf(key, sizeof(key), "light_%d", light_number);
    prefs.getBytes(key, light, sizeof(zb_device_params_t));
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Initialize Preferences
    prefs.begin("lights", false); // false means read/write mode

    // Load saved light parameters
    loadLightParams(&light_1, 1);
    loadLightParams(&light_2, 2);
    loadLightParams(&light_3, 3);

    // Init button switch
    pinMode(button, INPUT_PULLUP);

    // Set Zigbee device name and model
    zbSwitch.setManufacturerAndModel("Espressif", "ZBMultiSwitch");

    // Set binding settings depending on the role
    if (ZIGBEE_ROLE == ZIGBEE_COORDINATOR) {
        zbSwitch.allowMultipleBinding(true); // To allow binding multiple lights to
        ↳ the switch
    } else {
        zbSwitch.setManualBinding(true); // Set manual binding to true, so binding is
        ↳ done on Home Assistant side
    }

    // Add endpoint to Zigbee Core
    Serial.println("Adding ZigbeeSwitch endpoint to Zigbee Core");
    Zigbee.addEndpoint(&zbSwitch);

    // When all EPs are registered, start Zigbee with given role
    if (!Zigbee.begin(ZIGBEE_ROLE)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }

    Serial.println("Connecting to network");
    while (!Zigbee.connected()) {
        Serial.print(".");
        delay(100);
    }
    Serial.println();
}

void loop() {
    // Handle button switch in loop()
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        while (digitalRead(button) == LOW) {
            delay(50);
        }
        // Print bound devices
    }
}

```

(continues on next page)

(continued from previous page)

```

Serial.println("Bound devices:");
zbSwitch.printBoundDevices(Serial);
Serial.println("Lights configured:");
Serial.printf("Light 1: %u %s\n", light_1.endpoint, Zigbee.
↪formatIEEEAddress(light_1.ieee_addr));
Serial.printf("Light 2: %u %s\n", light_2.endpoint, Zigbee.
↪formatIEEEAddress(light_2.ieee_addr));
Serial.printf("Light 3: %u %s\n", light_3.endpoint, Zigbee.
↪formatIEEEAddress(light_3.ieee_addr));
}
// Handle serial input to configure and control the lights
if (Serial.available()) {
    String command = Serial.readString();
    Serial.println("Command: " + command);

    if (command == "config") {
        //wait for light number, endpoint and ieee address
        Serial.println("Enter light number (1-3):");
        while (!Serial.available()) {
            delay(100);
        }
        int light_number = Serial.parseInt();
        Serial.println("Enter endpoint:");
        while (!Serial.available()) {
            delay(100);
        }
        int endpoint = Serial.parseInt();
        Serial.println("Enter ieee address:");
        while (!Serial.available()) {
            delay(100);
        }
        String ieee_address = Serial.readStringUntil('\n');
        ieee_address.trim();
        //convert ieee address to uint8_t array (format in string is_
↪00:00:00:00:00:00:00:00)
        uint8_t ieee_address_array[8];
        int index = 0;
        bool valid = true;

        // Check if the string has the correct format (8 hex pairs with colons)
        if (ieee_address.length() != 23) { // 8 pairs * 2 + 7 colons
            Serial.println("Invalid IEEE address format. Expected format:_
↪00:00:00:00:00:00:00:00");
            valid = false;
        } else {
            for (int i = 0; i < ieee_address.length() && index < 8 && valid; i += 3) {
                // Check for colon at expected positions
                if (i > 0 && ieee_address.charAt(i - 1) != ':') {
                    valid = false;
                    break;
                }
                // Convert two hex characters to a byte
                char hex[3] = {ieee_address.charAt(i), ieee_address.charAt(i + 1), '\0'};
                char *endptr;
                long value = strtol(hex, &endptr, 16);
                if (*endptr != '\0' || value < 0 || value > 255) {
                    valid = false;
                    break;
                }
                // Store bytes in reverse order to match Zigbee standard
                ieee_address_array[7 - index++] = (uint8_t)value;
            }
        }
    }
}

```

(continues on next page)

(continued from previous page)

```

    }
}

if (!valid || index != 8) {
    Serial.println("Invalid IEEE address. Please enter a valid address in_
↪format: 00:00:00:00:00:00:00:00");
    return;
}
//set the light parameters
if (light_number == 1) {
    light_1.endpoint = endpoint;
    memcpy(light_1.ieee_addr, ieee_address_array, 8);
    storeLightParams(&light_1, 1);
} else if (light_number == 2) {
    light_2.endpoint = endpoint;
    memcpy(light_2.ieee_addr, ieee_address_array, 8);
    storeLightParams(&light_2, 2);
} else if (light_number == 3) {
    light_3.endpoint = endpoint;
    memcpy(light_3.ieee_addr, ieee_address_array, 8);
    storeLightParams(&light_3, 3);
}
Serial.printf("Light %d configured\n", light_number);
} else if (command == "remove") {
    //wait for light number
    Serial.println("Enter light number (1-3):");
    while (!Serial.available()) {
        delay(100);
    }
    int light_number = Serial.parseInt();
    uint8_t ieee_address_empty[8] = {0, 0, 0, 0, 0, 0, 0, 0};
    if (light_number == 1) {
        light_1.endpoint = 0;
        memcpy(light_1.ieee_addr, ieee_address_empty, 8);
        storeLightParams(&light_1, 1);
    } else if (light_number == 2) {
        light_2.endpoint = 0;
        memcpy(light_2.ieee_addr, ieee_address_empty, 8);
        storeLightParams(&light_2, 2);
    } else if (light_number == 3) {
        light_3.endpoint = 0;
        memcpy(light_3.ieee_addr, ieee_address_empty, 8);
        storeLightParams(&light_3, 3);
    }
    Serial.printf("Light %d removed\n", light_number);
} else if (command == "on") {
    Serial.println(" --> SIG Input : All Lights ON");
    zbSwitch.lightOn();
} else if (command == "off") {
    Serial.println(" --> SIG Input : All Lights OFF");
    zbSwitch.lightOff();
} else if (command == "toggle") {
    Serial.println(" --> SIG Input : All Lights Toggle");
    zbSwitch.lightToggle();
} else if (command == "1on") {
    Serial.println(" --> SIG Input : Light 1 ON");
    zbSwitch.lightOn(light_1.endpoint, light_1.ieee_addr);
} else if (command == "1off") {
    Serial.println(" --> SIG Input : Light 1 OFF");
    zbSwitch.lightOff(light_1.endpoint, light_1.ieee_addr);
} else if (command == "1toggle") {

```

(continues on next page)

(continued from previous page)

```

Serial.println(" --> SIG Input : Light 1 Toggle");
zbSwitch.lightToggle(light_1.endpoint, light_1.ieee_addr);
} else if (command == "2on") {
Serial.println(" --> SIG Input : Light 2 ON");
zbSwitch.lightOn(light_2.endpoint, light_2.ieee_addr);
} else if (command == "2off") {
Serial.println(" --> SIG Input : Light 2 OFF");
zbSwitch.lightOff(light_2.endpoint, light_2.ieee_addr);
} else if (command == "2toggle") {
Serial.println(" --> SIG Input : Light 2 Toggle");
zbSwitch.lightToggle(light_2.endpoint, light_2.ieee_addr);
} else if (command == "3on") {
Serial.println(" --> SIG Input : Light 3 ON");
zbSwitch.lightOn(light_3.endpoint, light_3.ieee_addr);
} else if (command == "3off") {
Serial.println(" --> SIG Input : Light 3 OFF");
zbSwitch.lightOff(light_3.endpoint, light_3.ieee_addr);
} else if (command == "3toggle") {
Serial.println(" --> SIG Input : Light 3 Toggle");
zbSwitch.lightToggle(light_3.endpoint, light_3.ieee_addr);
} else if (command == "freset") {
Serial.println(" --> SIG Input : Factory Reset!");
delay(1500);
Zigbee.factoryReset();
} else if (command == "open_network") {
Serial.println(" --> SIG Input : Open Network");
if (ZIGBEE_ROLE == ZIGBEE_COORDINATOR) {
Zigbee.openNetwork(180);
} else {
Serial.println("Open network is only available for coordinator role");
}
} else {
Serial.println("Unknown command");
}
}
}

```

ZigbeeTempSensor

About The ZigbeeTempSensor class provides a temperature and humidity sensor endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for environmental monitoring with configurable reporting intervals and thresholds.

Features: * Temperature measurement and reporting * Optional humidity measurement * Configurable reporting intervals * Min/max value and tolerance settings

API Reference

Constructor

ZigbeeTempSensor Creates a new Zigbee temperature sensor endpoint.

```
ZigbeeTempSensor(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Temperature Control

setTemperature Sets the temperature value in 0.01°C resolution.

```
bool setTemperature(float value);
```

- value - Temperature value in degrees Celsius

This function will return `true` if successful, `false` otherwise.

setMinMaxValue Sets the minimum and maximum temperature values for the sensor.

```
bool setMinMaxValue(float min, float max);
```

- min - Minimum temperature value in degrees Celsius
- max - Maximum temperature value in degrees Celsius

This function will return `true` if successful, `false` otherwise.

setDefaultValue Sets the default (initial) value for the temperature sensor in 0.01°C resolution. This value will be used as the initial measured value when the device is in factory reset mode and before the sensor provides actual readings.

```
bool setDefaultValue(float defaultValue);
```

- defaultValue - Default temperature value in degrees Celsius

Important: Must be called before adding the EP to Zigbee class. Only effective when the device is in factory reset mode (before commissioning/joining a network).

This function will return `true` if successful, `false` otherwise.

setTolerance Sets the tolerance value for temperature reporting.

```
bool setTolerance(float tolerance);
```

- tolerance - Tolerance value in degrees Celsius

This function will return `true` if successful, `false` otherwise.

setReporting Sets the reporting interval for temperature measurements.

```
bool setReporting(uint16_t min_interval, uint16_t max_interval, float delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in temperature to trigger report (in 0.01°C)

This function will return `true` if successful, `false` otherwise.

reportTemperature Manually reports the current temperature value.

```
bool reportTemperature();
```

This function will return `true` if successful, `false` otherwise.

Humidity Control (Optional)

addHumiditySensor Adds humidity measurement capability to the temperature sensor.

```
void addHumiditySensor(float min, float max, float tolerance);
```

- min - Minimum humidity value in percentage
- max - Maximum humidity value in percentage
- tolerance - Tolerance value in percentage

setHumidity Sets the humidity value in 0.01% resolution.

```
bool setHumidity(float value);
```

- value - Humidity value in percentage (0-100)

This function will return `true` if successful, `false` otherwise.

setHumidityReporting Sets the reporting interval for humidity measurements.

```
bool setHumidityReporting(uint16_t min_interval, uint16_t max_interval, float_
↪delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in humidity to trigger report (in 0.01%)

This function will return `true` if successful, `false` otherwise.

reportHumidity Manually reports the current humidity value.

```
bool reportHumidity();
```

This function will return `true` if successful, `false` otherwise.

Combined Reporting

report Reports both temperature and humidity values if humidity sensor is enabled.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

Example

Temperature Sensor Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
```

(continues on next page)

(continued from previous page)

```

// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee temperature sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device_
↪temperature sensor.
 * The temperature sensor is a Zigbee end device, which is controlled by a Zigbee_
↪coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee temperature sensor configuration */
#define TEMP_SENSOR_ENDPOINT_NUMBER 10
uint8_t button = BOOT_PIN;

// Optional Time cluster variables
struct tm timeinfo;
struct tm *localTime;
int32_t timezone;

ZigbeeTempSensor zbTempSensor = ZigbeeTempSensor(TEMP_SENSOR_ENDPOINT_NUMBER);

/***** Temp sensor *****/
static void temp_sensor_value_update(void *arg) {
    for (;;) {
        // Read temperature sensor value
        float tsens_value = temperatureRead();
        Serial.printf("Updated temperature sensor value to %.2f°C\r\n", tsens_value);
        // Update temperature value in Temperature sensor EP
        zbTempSensor.setTemperature(tsens_value);
        delay(1000);
    }
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init button switch
    pinMode(button, INPUT_PULLUP);

    // Optional: set Zigbee device name and model
    zbTempSensor.setManufacturerAndModel("Espressif", "ZigbeeTempSensor");

    // Set minimum and maximum temperature measurement value (10-50°C is default_

```

(continues on next page)

(continued from previous page)

```

↪range for chip temperature measurement)
zbTempSensor.setMinMaxValue(10, 50);

// Optional: Set default (initial) value for the temperature sensor to 10.0°C to
↪match the minimum temperature measurement value
zbTempSensor.setDefaultValue(10.0);

// Optional: Set tolerance for temperature measurement in °C (lowest possible
↪value is 0.01°C)
zbTempSensor.setTolerance(1);

// Optional: Time cluster configuration (default params, as this device will
↪revieve time from coordinator)
zbTempSensor.addTimeCluster();

// Add endpoint to Zigbee Core
Zigbee.addEndpoint(&zbTempSensor);

Serial.println("Starting Zigbee...");
// When all EPs are registered, start Zigbee in End Device mode
if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
} else {
    Serial.println("Zigbee started successfully!");
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();

// Optional: If time cluster is added, time can be read from the coordinator
timeinfo = zbTempSensor.getTime();
timezone = zbTempSensor.getTimezone();

Serial.println("UTC time:");
Serial.println(&timeinfo, "%A, %B %d %Y %H:%M:%S");

time_t local = mktime(&timeinfo) + timezone;
localTime = localtime(&local);

Serial.println("Local time with timezone:");
Serial.println(localTime, "%A, %B %d %Y %H:%M:%S");

// Start Temperature sensor reading task
xTaskCreate(temp_sensor_value_update, "temp_sensor_update", 3072, NULL, 10,
↪NULL);

// Set reporting interval for temperature measurement in seconds, must be called
↪after Zigbee.begin()
// min_interval and max_interval in seconds, delta (temp change in 0,1 °C)
// if min = 1 and max = 0, reporting is sent only when temperature changes by
↪delta
// if min = 0 and max = 10, reporting is sent every 10 seconds or temperature
↪changes by delta
// if min = 0, max = 10 and delta = 0, reporting is sent every 10 seconds
↪regardless of temperature change
zbTempSensor.setReporting(1, 0, 1);

```

(continues on next page)

(continued from previous page)

```

}

void loop() {
  // Checking button for factory reset
  if (digitalRead(button) == LOW) { // Push button pressed
    // Key debounce handling
    delay(100);
    int startTime = millis();
    while (digitalRead(button) == LOW) {
      delay(50);
      if ((millis() - startTime) > 3000) {
        // If key pressed for more than 3secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        delay(1000);
        Zigbee.factoryReset();
      }
    }
    zbTempSensor.reportTemperature();
  }
  delay(100);
}
}

```

Temperature + Humidity Sleepy Sensor Implementation

```

// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee temperature and humidity sensor Sleepy_
↳device.
 *
 * The example demonstrates how to use Zigbee library to create an end device_
↳temperature and humidity sensor.
 * The sensor is a Zigbee end device, which is reporting data to the Zigbee_
↳network.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↳Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

```

(continues on next page)

(continued from previous page)

```

#include "Zigbee.h"

#define USE_GLOBAL_ON_RESPONSE_CALLBACK 1 // Set to 0 to use local callback.
↳specified directly for the endpoint.

/* Zigbee temperature + humidity sensor configuration */
#define TEMP_SENSOR_ENDPOINT_NUMBER 10

#define uS_TO_S_FACTOR 1000000ULL /* Conversion factor for micro seconds to
↳seconds */
#define TIME_TO_SLEEP 55 /* Sleep for 55s will + 5s delay for
↳establishing connection => data reported every 1 minute */
#define REPORT_TIMEOUT 1000 /* Timeout for response from coordinator in ms */

uint8_t button = BOOT_PIN;

ZigbeeTempSensor zbTempSensor = ZigbeeTempSensor(TEMP_SENSOR_ENDPOINT_NUMBER);

uint8_t dataToSend = 2; // Temperature and humidity values are reported in same
↳endpoint, so 2 values are reported
bool resend = false;

/***** Callbacks *****/
#if USE_GLOBAL_ON_RESPONSE_CALLBACK
void onGlobalResponse(zb_cmd_type_t command, esp_zb_zcl_status_t status, uint8_t
↳endpoint, uint16_t cluster) {
    Serial.printf("Global response command: %d, status: %s, endpoint: %u, cluster: 0x
↳%04x\r\n", command, esp_zb_zcl_status_to_name(status), endpoint, cluster);
    if ((command == ZB_CMD_REPORT_ATTRIBUTE) && (endpoint == TEMP_SENSOR_ENDPOINT_
↳NUMBER)) {
        switch (status) {
            case ESP_ZB_ZCL_STATUS_SUCCESS: dataToSend--; break;
            case ESP_ZB_ZCL_STATUS_FAIL: resend = true; break;
            default: break; // add more statuses like ESP_ZB_ZCL_
↳STATUS_INVALID_VALUE, ESP_ZB_ZCL_STATUS_TIMEOUT etc.
        }
    }
}
#else
void onResponse(zb_cmd_type_t command, esp_zb_zcl_status_t status) {
    Serial.printf("Response command: %d, status: %s\r\n", command, esp_zb_zcl_status_
↳to_name(status));
    if (command == ZB_CMD_REPORT_ATTRIBUTE) {
        switch (status) {
            case ESP_ZB_ZCL_STATUS_SUCCESS: dataToSend--; break;
            case ESP_ZB_ZCL_STATUS_FAIL: resend = true; break;
            default: break; // add more statuses like ESP_ZB_ZCL_
↳STATUS_INVALID_VALUE, ESP_ZB_ZCL_STATUS_TIMEOUT etc.
        }
    }
}
#endif

/***** Temp sensor *****/
static void measureAndSleep(void *arg) {
    // Measure temperature sensor value
    float temperature = temperatureRead();

    // Use temperature value as humidity value to demonstrate both temperature and
↳humidity

```

(continues on next page)

(continued from previous page)

```

float humidity = temperature;

// Update temperature and humidity values in Temperature sensor EP
zbTempSensor.setTemperature(temperature);
zbTempSensor.setHumidity(humidity);

// Report temperature and humidity values
zbTempSensor.report(); // reports temperature and humidity values (if humidity_
↳ sensor is not added, only temperature is reported)
Serial.printf("Reported temperature: %.2f°C, Humidity: %.2f%%\r\n", temperature,
↳ humidity);

unsigned long startTime = millis();
const unsigned long timeout = REPORT_TIMEOUT;

Serial.printf("Waiting for data report to be confirmed \r\n");
// Wait until data was successfully sent
int tries = 0;
const int maxTries = 3;
while (dataToSend != 0 && tries < maxTries) {
    if (resend) {
        Serial.println("Resending data on failure!");
        resend = false;
        dataToSend = 2;
        zbTempSensor.report(); // report again
    }
    if (millis() - startTime >= timeout) {
        Serial.println("\nReport timeout! Report Again");
        dataToSend = 2;
        zbTempSensor.report(); // report again
        startTime = millis();
        tries++;
    }
    Serial.printf(".");
    delay(50); // 50ms delay to avoid busy-waiting
}

// Put device to deep sleep after data was sent successfully or timeout
Serial.println("Going to sleep now");
esp_deep_sleep_start();
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init button switch
    pinMode(button, INPUT_PULLUP);

    // Configure the wake up source and set to wake up every 5 seconds
    esp_sleep_enable_timer_wakeup(TIME_TO_SLEEP * uS_TO_S_FACTOR);

    // Optional: set Zigbee device name and model
    zbTempSensor.setManufacturerAndModel("Espressif", "SleepyZigbeeTempSensor");

    // Set minimum and maximum temperature measurement value (10-50°C is default_
↳ range for chip temperature measurement)
    zbTempSensor.setMinMaxValue(10, 50);

    // Set default (initial) value for the temperature sensor to 10.0°C to match the_
↳ minimum temperature measurement value (default value is 0.0°C)

```

(continues on next page)

(continued from previous page)

```

zbTempSensor.setDefaultValue(10.0);

// Set tolerance for temperature measurement in °C (lowest possible value is 0.
↪01 °C)
zbTempSensor.setTolerance(1);

// Set power source to battery, battery percentage and battery voltage (now 100%
↪and 3.5V for demonstration)
// The value can be also updated by calling zbTempSensor.
↪setBatteryPercentage(percentage) or zbTempSensor.setBatteryVoltage(voltage)
↪anytime after Zigbee.begin()
zbTempSensor.setPowerSource(ZB_POWER_SOURCE_BATTERY, 100, 35);

// Add humidity cluster to the temperature sensor device with min, max,
↪tolerance and default values
zbTempSensor.addHumiditySensor(0, 100, 1, 0.0);

// Set callback for default response to handle status of reported data, there
↪are 2 options.

#if USE_GLOBAL_ON_RESPONSE_CALLBACK
// Global callback for all endpoints with more params to determine the endpoint
↪and cluster in the callback function.
Zigbee.onGlobalDefaultResponse(onGlobalResponse);
#else
// Callback specified for endpoint
zbTempSensor.onDefaultResponse(onResponse);
#endif

// Add endpoint to Zigbee Core
Zigbee.addEndpoint(&zbTempSensor);

// Create a custom Zigbee configuration for End Device with keep alive 10s to
↪avoid interference with reporting data
esp_zb_cfg_t zigbeeConfig = ZIGBEE_DEFAULT_ED_CONFIG();
zigbeeConfig.nwk_cfg.zed_cfg.keep_alive = 10000;

// For battery powered devices, it can be better to set timeout for Zigbee Begin
↪to lower value to save battery
// If the timeout has been reached, the network channel mask will be reset and
↪the device will try to connect again after reset (scanning all channels)
Zigbee.setTimeout(10000); // Set timeout for Zigbee Begin to 10s (default is
↪30s)

// When all EPs are registered, start Zigbee in End Device mode
if (!Zigbee.begin(&zigbeeConfig, false)) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart(); // If Zigbee failed to start, reboot the device and try again
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();
Serial.println("Successfully connected to Zigbee network");

// Start Temperature sensor reading task
xTaskCreate(measureAndSleep, "temp_sensor_update", 3072, NULL, 10, NULL);
}

```

(continues on next page)

(continued from previous page)

```

void loop() {
  // Checking button for factory reset
  if (digitalRead(button) == LOW) { // Push button pressed
    // Key debounce handling
    delay(100);
    int startTime = millis();
    while (digitalRead(button) == LOW) {
      delay(50);
      if ((millis() - startTime) > 10000) {
        // If key pressed for more than 10secs, factory reset Zigbee and reboot
        Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
        delay(1000);
        // Optional set reset in factoryReset to false, to not restart device_
        ↪after erasing nvram, but set it to endless sleep manually instead
        Zigbee.factoryReset(false);
        Serial.println("Going to endless sleep, press RESET button or power off/on_
        ↪the device to wake up");
        esp_sleep_disable_wakeup_source(ESP_SLEEP_WAKEUP_TIMER);
        esp_deep_sleep_start();
      }
    }
    delay(100);
  }
}

```

ZigbeeThermostat

About The ZigbeeThermostat class provides a thermostat endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for thermostats that can bind to temperature and humidity sensors and receive temperature and humidity readings.

Features: * Automatic discovery and binding to temperature sensors * Temperature and humidity data reception from bound sensors * Configurable temperature and humidity reporting intervals * Sensor settings retrieval (min/max/tolerance for temperature and humidity) * Multiple addressing modes (group, specific endpoint, IEEE address)

API Reference

Constructor

ZigbeeThermostat Creates a new Zigbee thermostat endpoint.

```
ZigbeeThermostat(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

Event Handling

onTempReceive Sets a callback function for receiving temperature data.

```
void onTempReceive(void (*callback)(float temperature));
```

- callback - Function to call when temperature data is received
- temperature - Temperature value in degrees Celsius

onTempReceiveWithSource Sets a callback function for receiving temperature data with source information.

```
void onTempReceiveWithSource(void (*callback)(float temperature, uint8_t src_
↳endpoint, esp_zb_zcl_addr_t src_address));
```

- callback - Function to call when temperature data is received
- temperature - Temperature value in degrees Celsius
- src_endpoint - Source endpoint that sent the temperature data
- src_address - Source address information

onTempConfigReceive Sets a callback function for receiving sensor configuration data.

```
void onTempConfigReceive(void (*callback)(float min_temp, float max_temp, float_
↳tolerance));
```

- callback - Function to call when sensor configuration is received
- min_temp - Minimum temperature supported by the sensor
- max_temp - Maximum temperature supported by the sensor
- tolerance - Temperature tolerance of the sensor

onHumidityReceive Sets a callback function for receiving humidity data.

```
void onHumidityReceive(void (*callback)(float humidity));
```

- callback - Function to call when humidity data is received
- humidity - Humidity value in percentage

onHumidityReceiveWithSource Sets a callback function for receiving humidity data with source information.

```
void onHumidityReceiveWithSource(void (*callback)(float humidity, uint8_t src_
↳endpoint, esp_zb_zcl_addr_t src_address));
```

- callback - Function to call when humidity data is received
- humidity - Humidity value in percentage
- src_endpoint - Source endpoint that sent the humidity data
- src_address - Source address information

onHumidityConfigReceive Sets a callback function for receiving humidity sensor configuration data.

```
void onHumidityConfigReceive(void (*callback)(float min_humidity, float max_
↳humidity, float tolerance));
```

- callback - Function to call when humidity sensor configuration is received
- min_humidity - Minimum humidity supported by the sensor
- max_humidity - Maximum humidity supported by the sensor
- tolerance - Humidity tolerance of the sensor

Temperature Data Retrieval

getTemperature Requests temperature data from all bound sensors.

```
void getTemperature();
```

getTemperature (Group) Requests temperature data from a specific group.

```
void getTemperature(uint16_t group_addr);
```

- group_addr - Group address to send the request to

getTemperature (Endpoint + Short Address) Requests temperature data from a specific endpoint using short address.

```
void getTemperature(uint8_t endpoint, uint16_t short_addr);
```

- endpoint - Target endpoint number
- short_addr - Short address of the target device

getTemperature (Endpoint + IEEE Address) Requests temperature data from a specific endpoint using IEEE address.

```
void getTemperature(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- endpoint - Target endpoint number
- ieee_addr - IEEE address of the target device

Humidity Data Retrieval

getHumidity Requests humidity data from all bound sensors.

```
void getHumidity();
```

getHumidity (Group) Requests humidity data from a specific group.

```
void getHumidity(uint16_t group_addr);
```

- group_addr - Group address to send the request to

getHumidity (Endpoint + Short Address) Requests humidity data from a specific endpoint using short address.

```
void getHumidity(uint8_t endpoint, uint16_t short_addr);
```

- endpoint - Target endpoint number
- short_addr - Short address of the target device

getHumidity (Endpoint + IEEE Address) Requests humidity data from a specific endpoint using IEEE address.

```
void getHumidity(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- endpoint - Target endpoint number
- ieee_addr - IEEE address of the target device

Temperature Settings Retrieval

getTemperatureSettings Requests temperature sensor settings from all bound sensors.

```
void getTemperatureSettings();
```

getTemperatureSettings (Group) Requests temperature sensor settings from a specific group.

```
void getTemperatureSettings(uint16_t group_addr);
```

- group_addr - Group address to send the request to

getTemperatureSettings (Endpoint + Short Address) Requests temperature sensor settings from a specific endpoint using short address.

```
void getTemperatureSettings(uint8_t endpoint, uint16_t short_addr);
```

- endpoint - Target endpoint number
- short_addr - Short address of the target device

getTemperatureSettings (Endpoint + IEEE Address) Requests temperature sensor settings from a specific endpoint using IEEE address.

```
void getTemperatureSettings(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- endpoint - Target endpoint number
- ieee_addr - IEEE address of the target device

Humidity Settings Retrieval

getHumiditySettings Requests humidity sensor settings from all bound sensors.

```
void getHumiditySettings();
```

getHumiditySettings (Group) Requests humidity sensor settings from a specific group.

```
void getHumiditySettings(uint16_t group_addr);
```

- group_addr - Group address to send the request to

getHumiditySettings (Endpoint + Short Address) Requests humidity sensor settings from a specific endpoint using short address.

```
void getHumiditySettings(uint8_t endpoint, uint16_t short_addr);
```

- endpoint - Target endpoint number
- short_addr - Short address of the target device

getHumiditySettings (Endpoint + IEEE Address) Requests humidity sensor settings from a specific endpoint using IEEE address.

```
void getHumiditySettings(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr);
```

- endpoint - Target endpoint number
- ieee_addr - IEEE address of the target device

Temperature Reporting Configuration

setTemperatureReporting Configures temperature reporting for all bound sensors.

```
void setTemperatureReporting(uint16_t min_interval, uint16_t max_interval, float_  
↪delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in temperature to trigger a report

setTemperatureReporting (Group) Configures temperature reporting for a specific group.

```
void setTemperatureReporting(uint16_t group_addr, uint16_t min_interval, uint16_t_  
↪max_interval, float delta);
```

- group_addr - Group address to configure
- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in temperature to trigger a report

setTemperatureReporting (Endpoint + Short Address) Configures temperature reporting for a specific endpoint using short address.

```
void setTemperatureReporting(uint8_t endpoint, uint16_t short_addr, uint16_t min_  
↪interval, uint16_t max_interval, float delta);
```

- endpoint - Target endpoint number
- short_addr - Short address of the target device
- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in temperature to trigger a report

setTemperatureReporting (Endpoint + IEEE Address) Configures temperature reporting for a specific endpoint using IEEE address.

```
void setTemperatureReporting(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr,_  
↪uint16_t min_interval, uint16_t max_interval, float delta);
```

- endpoint - Target endpoint number
- ieee_addr - IEEE address of the target device
- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in temperature to trigger a report

Humidity Reporting Configuration

setHumidityReporting Configures humidity reporting for all bound sensors.

```
void setHumidityReporting(uint16_t min_interval, uint16_t max_interval, float_  
↪delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in humidity to trigger a report

setHumidityReporting (Group) Configures humidity reporting for a specific group.

```
void setHumidityReporting(uint16_t group_addr, uint16_t min_interval, uint16_t max_
↳ interval, float delta);
```

- group_addr - Group address to configure
- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in humidity to trigger a report

setHumidityReporting (Endpoint + Short Address) Configures humidity reporting for a specific endpoint using short address.

```
void setHumidityReporting(uint8_t endpoint, uint16_t short_addr, uint16_t min_
↳ interval, uint16_t max_interval, float delta);
```

- endpoint - Target endpoint number
- short_addr - Short address of the target device
- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in humidity to trigger a report

setHumidityReporting (Endpoint + IEEE Address) Configures humidity reporting for a specific endpoint using IEEE address.

```
void setHumidityReporting(uint8_t endpoint, esp_zb_ieee_addr_t ieee_addr, uint16_t_
↳ min_interval, uint16_t max_interval, float delta);
```

- endpoint - Target endpoint number
- ieee_addr - IEEE address of the target device
- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change in humidity to trigger a report

Example

Thermostat Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
/**
 * @brief This example demonstrates simple Zigbee thermostat.
 *
 * The example demonstrates how to use Zigbee library to get data from temperature
 * sensor end device and act as an thermostat.
 * The temperature sensor is a Zigbee end device, which is controlled by a Zigbee_
```

(continues on next page)

(continued from previous page)

```

↪ coordinator (thermostat).
*
* Proper Zigbee mode must be selected in Tools->Zigbee mode
* and also the correct partition scheme must be selected in Tools->Partition_
↪ Scheme.
*
* Please check the README.md for instructions and more detailed description.
*
* Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
*/

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ZCZR
#error "Zigbee coordinator mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

/* Zigbee thermostat configuration */
#define THERMOSTAT_ENDPOINT_NUMBER 1
#define USE_RECEIVE_TEMP_WITH_SOURCE 1
uint8_t button = BOOT_PIN;

ZigbeeThermostat zbThermostat = ZigbeeThermostat(THERMOSTAT_ENDPOINT_NUMBER);

// Save temperature sensor data
float sensor_temp;
float sensor_max_temp;
float sensor_min_temp;
float sensor_tolerance;

struct tm timeinfo = {}; // Time structure for Time cluster

/***** Temperature sensor handling *****/
#if USE_RECEIVE_TEMP_WITH_SOURCE == 0
void receiveSensorTemp(float temperature) {
    Serial.printf("Temperature sensor value: %.2f°C\n", temperature);
    sensor_temp = temperature;
}
#else
void receiveSensorTempWithSource(float temperature, uint8_t src_endpoint, esp_zb_
↪ zcl_addr_t src_address) {
    if (src_address.addr_type == ESP_ZB_ZCL_ADDR_TYPE_SHORT) {
        Serial.printf("Temperature sensor value: %.2f°C from endpoint %u, address 0x
↪ %04x\n", temperature, src_endpoint, src_address.u.short_addr);
    } else {
        Serial.printf(
            "Temperature sensor value: %.2f°C from endpoint %u, address %02x:%02x:%02x:
↪ %02x:%02x:%02x:%02x\n", temperature, src_endpoint,
            src_address.u.ieee_addr[7], src_address.u.ieee_addr[6], src_address.u.ieee_
↪ addr[5], src_address.u.ieee_addr[4], src_address.u.ieee_addr[3],
            src_address.u.ieee_addr[2], src_address.u.ieee_addr[1], src_address.u.ieee_
↪ addr[0]
        );
    }
    sensor_temp = temperature;
}
#endif

void receiveSensorConfig(float min_temp, float max_temp, float tolerance) {
    Serial.printf("Temperature sensor config: min %.2f°C, max %.2f°C, tolerance %.
```

(continues on next page)

(continued from previous page)

```

↪2f°C\n", min_temp, max_temp, tolerance);
    sensor_min_temp = min_temp;
    sensor_max_temp = max_temp;
    sensor_tolerance = tolerance;
}
/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);

    // Init button switch
    pinMode(button, INPUT_PULLUP);

    // Set callback function for receiving temperature from sensor - Use only one_
    ↪option
    #if USE_RECEIVE_TEMP_WITH_SOURCE == 0
        zbThermostat.onTempReceive(receiveSensorTemp); // If you bound only one sensor_
    ↪or you don't need to know the source of the temperature
    #else
        zbThermostat.onTempReceiveWithSource(receiveSensorTempWithSource);
    #endif

    // Set callback function for receiving sensor configuration
    zbThermostat.onTempConfigReceive(receiveSensorConfig);

    //Optional: set Zigbee device name and model
    zbThermostat.setManufacturerAndModel("Espressif", "ZigbeeThermostat");

    //Optional Time cluster configuration
    //example time January 13, 2025 13:30:30 CET
    timeinfo.tm_year = 2025 - 1900; // = 2025
    timeinfo.tm_mon = 0;           // January
    timeinfo.tm_mday = 13;         // 13th
    timeinfo.tm_hour = 12;         // 12 hours - 1 hour (CET)
    timeinfo.tm_min = 30;          // 30 minutes
    timeinfo.tm_sec = 30;          // 30 seconds
    timeinfo.tm_isdst = -1;

    // Set time and gmt offset (timezone in seconds -> CET = +3600 seconds)
    zbThermostat.addTimeCluster(timeinfo, 3600);

    //Add endpoint to Zigbee Core
    Zigbee.addEndpoint(&zbThermostat);

    //Open network for 180 seconds after boot
    Zigbee.setRebootOpenNetwork(180);

    // When all EPs are registered, start Zigbee with ZIGBEE_COORDINATOR mode
    if (!Zigbee.begin(ZIGBEE_COORDINATOR)) {
        Serial.println("Zigbee failed to start!");
        Serial.println("Rebooting...");
        ESP.restart();
    }

    Serial.println("Waiting for Temperature sensor to bound to the thermostat");
    while (!zbThermostat.bound()) {
        Serial.printf(".");
        delay(500);
    }

    Serial.println();
}

```

(continues on next page)

(continued from previous page)

```

// Get temperature sensor configuration for all bound sensors by endpoint number
↳and address
std::list<z_b_device_params_t *> boundSensors = zbThermostat.getBoundDevices();
for (const auto &device : boundSensors) {
    Serial.println("-----");
    if (device->short_addr == 0x0000 || device->short_addr == 0xFFFF) { //End
↳devices never have 0x0000 short address or 0xFFFF group address
        Serial.printf(
            "Device on endpoint %d, IEEE Address: %02X:%02X:%02X:%02X:%02X:%02X:
↳%02X\r\n", device->endpoint, device->ieee_addr[7], device->ieee_addr[6],
            device->ieee_addr[5], device->ieee_addr[4], device->ieee_addr[3], device->
↳ieee_addr[2], device->ieee_addr[1], device->ieee_addr[0]
        );
        zbThermostat.getTemperatureSettings(device->endpoint, device->ieee_addr);
    } else {
        Serial.printf("Device on endpoint %u, short address: 0x%x\r\n", device->
↳endpoint, device->short_addr);
        zbThermostat.getTemperatureSettings(device->endpoint, device->short_addr);
    }
}
}

void loop() {
    // Handle button switch in loop()
    if (digitalRead(button) == LOW) { // Push button pressed
        // Key debounce handling
        while (digitalRead(button) == LOW) {
            delay(50);
        }
        // Set reporting interval for temperature sensor
        zbThermostat.setTemperatureReporting(0, 10, 2);
    }

    // Print temperature sensor data each 10 seconds
    static uint32_t last_print = 0;
    if (millis() - last_print > 10000) {
        last_print = millis();
        int temp_percent = (int)((sensor_temp - sensor_min_temp) / (sensor_max_temp -
↳sensor_min_temp) * 100);
        Serial.printf("Loop temperature info: %.2f°C (%d %%) \n", sensor_temp, temp_
↳percent);
        zbThermostat.printBoundDevices(Serial);
    }
}

```

ZigbeeVibrationSensor

About The ZigbeeVibrationSensor class provides a vibration sensor endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for vibration detection devices.

Features: * Vibration detection and measurement * Configurable sensitivity levels * Multiple detection modes * Automatic reporting capabilities * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Security system vibration detection * Industrial equipment monitoring * Structural health monitoring * Smart home security applications * Machine condition monitoring

API Reference

Constructor

ZigbeeVibrationSensor Creates a new Zigbee vibration sensor endpoint.

```
ZigbeeVibrationSensor(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

API Methods

setVibration Sets the vibration detection state.

```
bool setVibration(bool sensed);
```

- `sensed` - Vibration state (true = sensed, false = not sensed)

This function will return `true` if successful, `false` otherwise.

setIASClientEndpoint Sets the IAS Client endpoint number (default is 1).

```
void setIASClientEndpoint(uint8_t ep_number);
```

- `ep_number` - IAS Client endpoint number

report Manually reports the current vibration state.

```
bool report();
```

This function will return `true` if successful, `false` otherwise.

requestIASZoneEnroll Requests a new IAS Zone enrollment. Can be called to enroll a new device or to re-enroll an already enrolled device.

```
bool requestIASZoneEnroll();
```

This function will return `true` if the enrollment request was sent successfully, `false` otherwise. The actual enrollment status should be checked using the `enrolled()` method after waiting for the enrollment response.

restoreIASZoneEnroll Restores IAS Zone enrollment from stored attributes. This method should be called after rebooting an already enrolled device. It restores the enrollment information from flash memory, which is faster for sleepy devices compared to requesting a new enrollment.

```
bool restoreIASZoneEnroll();
```

This function will return `true` if the enrollment was successfully restored, `false` otherwise. The enrollment information (zone ID and IAS CIE address) must be available in the device's stored attributes for this to succeed.

enrolled Checks if the device is currently enrolled in the IAS Zone.

```
bool enrolled();
```

This function returns `true` if the device is enrolled, `false` otherwise. Use this method to check the enrollment status after calling `requestIASZoneEnroll()` or `restoreIASZoneEnroll()`.

Example**Vibration Sensor Implementation**

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee vibration sensor (IAS Zone).
 *
 * The example demonstrates how to use Zigbee library to create a end device_
↪vibration sensor.
 * The vibration sensor is a Zigbee end device, which is reporting data to the_
↪Zigbee network.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
↪Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"
#include <Preferences.h>

/* Zigbee vibration sensor configuration */
#define VIBRATION_SENSOR_ENDPOINT_NUMBER 1
uint8_t button = BOOT_PIN;
uint8_t sensor_pin = 4;

ZigbeeVibrationSensor zbVibrationSensor = ZigbeeVibrationSensor(VIBRATION_SENSOR_
↪ENDPOINT_NUMBER);

/* Preferences for storing ENROLLED flag to persist across reboots */
Preferences preferences;

void setup() {
    Serial.begin(115200);

    preferences.begin("Zigbee", false); // Save ENROLLED flag in flash_
↪so it persists across reboots
    bool enrolled = preferences.getBool("ENROLLED"); // Get ENROLLED flag from_
↪preferences
    preferences.end();
}
```

(continues on next page)

(continued from previous page)

```

// Init button + sensor
pinMode(button, INPUT_PULLUP);
pinMode(sensor_pin, INPUT);

// Optional: set Zigbee device name and model
zbVibrationSensor.setManufacturerAndModel("Espressif", "ZigbeeVibrationSensor");

// Add endpoint to Zigbee Core
Zigbee.addEndpoint(&zbVibrationSensor);

Serial.println("Starting Zigbee...");
// When all EPs are registered, start Zigbee in End Device mode
if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
} else {
    Serial.println("Zigbee started successfully!");
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();

// Check if device has been enrolled before restarting - if so, restore IAS Zone
↪enroll, otherwise request new IAS Zone enroll
if (enrolled) {
    Serial.println("Device has been enrolled before - restoring IAS Zone enrollment
↪");
    zbVibrationSensor.restoreIASZoneEnroll();
} else {
    Serial.println("Device is factory new - first time joining network -
↪requesting new IAS Zone enrollment");
    zbVibrationSensor.requestIASZoneEnroll();
}

while (!zbVibrationSensor.enrolled()) {
    Serial.print(".");
    delay(100);
}
Serial.println();
Serial.println("Zigbee enrolled successfully!");

// Store ENROLLED flag only if this was a new enrollment (previous flag was
↪false)
// Skip writing if we just restored enrollment (flag was already true)
if (!enrolled) {
    preferences.begin("Zigbee", false);
    preferences.putBool("ENROLLED", true); // set ENROLLED flag to true
    preferences.end();
    Serial.println("ENROLLED flag saved to preferences");
}
}

void loop() {
    // Checking pin for contact change
    static bool sensed = false;
    if (digitalRead(sensor_pin) == HIGH && !sensed) {

```

(continues on next page)

(continued from previous page)

```

// Update contact sensor value
zbVibrationSensor.setVibration(true);
sensed = true;
//if sensed, wait 2 seconds before next sensing
delay(2000);
} else if (digitalRead(sensor_pin) == LOW && sensed) {
  zbVibrationSensor.setVibration(false);
  sensed = false;
  //if not sensed, wait 0,5 seconds before next sensing
  delay(500);
}

// Checking button for factory reset
if (digitalRead(button) == LOW) { // Push button pressed
  // Key debounce handling
  delay(100);
  int startTime = millis();
  while (digitalRead(button) == LOW) {
    delay(50);
    if ((millis() - startTime) > 3000) {
      // If key pressed for more than 3secs, factory reset Zigbee and reboot
      Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
      // Clear the ENROLLED flag from preferences
      preferences.begin("Zigbee", false);
      preferences.putBool("ENROLLED", false); // set ENROLLED flag to false
      preferences.end();
      Serial.println("ENROLLED flag cleared from preferences");
      delay(1000);
      Zigbee.factoryReset();
    }
  }
}
delay(100);
}

```

ZigbeeWindSpeedSensor

About The ZigbeeWindSpeedSensor class provides a wind speed sensor endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for wind speed measurement devices.

Features: * Wind speed measurement in m/s * Configurable measurement range * Tolerance and reporting configuration * Automatic reporting capabilities * Integration with common endpoint features (binding, OTA, etc.) * Zigbee HA standard compliance

Use Cases: * Weather stations * Wind turbine monitoring * Agricultural weather monitoring * Marine applications * Smart home weather systems * Industrial wind monitoring

API Reference

Constructor

ZigbeeWindSpeedSensor Creates a new Zigbee wind speed sensor endpoint.

```
ZigbeeWindSpeedSensor(uint8_t endpoint);
```

- endpoint - Endpoint number (1-254)

API Methods

setWindSpeed Sets the wind speed measurement value.

```
bool setWindSpeed(float value);
```

- value - Wind speed value in 0.01 m/s

This function will return `true` if successful, `false` otherwise.

setMinMaxValue Sets the minimum and maximum measurement values.

```
bool setMinMaxValue(float min, float max);
```

- min - Minimum wind speed value in 0.01 m/s
- max - Maximum wind speed value in 0.01 m/s

This function will return `true` if successful, `false` otherwise.

setDefaultValue Sets the default (initial) value for the wind speed sensor in 0.01 m/s. This value will be used as the initial measured value when the device is in factory reset mode and before the sensor provides actual readings.

```
bool setDefaultValue(float defaultValue);
```

- defaultValue - Default wind speed value in 0.01 m/s

Important: Must be called before adding the EP to Zigbee class. Only effective when the device is in factory reset mode (before commissioning/joining a network).

This function will return `true` if successful, `false` otherwise.

setTolerance Sets the tolerance value for measurements.

```
bool setTolerance(float tolerance);
```

- tolerance - Tolerance value in 0.01 m/s

This function will return `true` if successful, `false` otherwise.

setReporting Sets the reporting configuration for wind speed measurements.

```
bool setReporting(uint16_t min_interval, uint16_t max_interval, float delta);
```

- min_interval - Minimum reporting interval in seconds
- max_interval - Maximum reporting interval in seconds
- delta - Minimum change required to trigger a report in 0.01 m/s

This function will return `true` if successful, `false` otherwise.

reportWindSpeed Manually reports the current wind speed value.

```
bool reportWindSpeed();
```

This function will return `true` if successful, `false` otherwise.

Example

Wind Speed Sensor Implementation

```
// Copyright 2024 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//      http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.

/**
 * @brief This example demonstrates Zigbee windspeed sensor.
 *
 * The example demonstrates how to use Zigbee library to create a end device wind_
 * speed sensor.
 * The wind speed sensor is a Zigbee end device, which is controlled by a Zigbee_
 * coordinator.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition_
 * Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifdef ZIGBEE_MODE_ED
#error "Zigbee coordinator mode is not selected in Tools->Zigbee mode"
#endif

#include "Zigbee.h"

#define BUTTON_PIN 9 //Boot button for C6/H2
#define WIND_SPEED_SENSOR_ENDPOINT_NUMBER 10

ZigbeeWindSpeedSensor zbWindSpeedSensor = ZigbeeWindSpeedSensor(WIND_SPEED_SENSOR_
    ENDPOINT_NUMBER);

/***** WindSpeed sensor *****/
static void windspeed_sensor_value_update(void *arg) {
    for (;;) {
        // Read wind speed sensor value (simulated now by temperature sensor)
        float windspeed = temperatureRead();
        log_v("Wind speed sensor value: %.2fm/s", windspeed);
        // Update windspeed value in Windspeed sensor EP
        zbWindSpeedSensor.setWindSpeed(windspeed);
        delay(1000);
    }
}

/***** Arduino functions *****/
void setup() {
    Serial.begin(115200);
    while (!Serial) {
        delay(10);
    }
}
```

(continues on next page)

(continued from previous page)

```

}

// Init button switch
pinMode(BUTTON_PIN, INPUT);

// Optional: set Zigbee device name and model
zbWindSpeedSensor.setManufacturerAndModel("Espressif", "ZigbeeWindSpeedSensor");

// Set minimum and maximum windspeed measurement value in m/s
zbWindSpeedSensor.setMinMaxValue(0, 50);

// Set tolerance for windspeed measurement in m/s (lowest possible value is 0.01
↪m/s)
zbWindSpeedSensor.setTolerance(1);

// Add endpoint to Zigbee Core
Zigbee.addEndpoint(&zbWindSpeedSensor);

Serial.println("Starting Zigbee...");
// When all EPs are registered, start Zigbee in End Device mode
if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
} else {
    Serial.println("Zigbee started successfully!");
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();

// Start Wind speed sensor reading task
xTaskCreate(windspeed_sensor_value_update, "wind_speed_sensor_update", 3072,
↪NULL, 10, NULL);

// Set reporting interval for windspeed measurement in seconds, must be called
↪after Zigbee.begin()
// min_interval and max_interval in seconds, delta (WindSpeed change in m/s)
// if min = 1 and max = 0, reporting is sent only when windspeed changes by delta
// if min = 0 and max = 10, reporting is sent every 10 seconds or windspeed
↪changes by delta
// if min = 0, max = 10 and delta = 0, reporting is sent every 10 seconds
↪regardless of windspeed change
zbWindSpeedSensor.setReporting(1, 0, 1);
}

void loop() {
    // Checking button for factory reset
    if (digitalRead(BUTTON_PIN) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(BUTTON_PIN) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.println("Resetting Zigbee to factory and rebooting in 1s.");
                delay(1000);
            }
        }
    }
}

```

(continues on next page)

(continued from previous page)

```
        Zigbee.factoryReset();
    }
}
zbWindSpeedSensor.reportWindSpeed();
}
delay(100);
}
```

ZigbeeWindowCovering

About The `ZigbeeWindowCovering` class provides a window covering endpoint for Zigbee networks. This endpoint implements the Zigbee Home Automation (HA) standard for motorized blinds, shades, and other window coverings.

Features: * Position control (lift and tilt) * Multiple window covering types support * Configurable operation modes and limits * Status reporting and callbacks * Safety features and limits

Supported Window Covering Types: * ROLLERSHADE - Lift support * ROLLERSHADE_2_MOTOR - Lift support * ROLLERSHADE_EXTERIOR - Lift support * ROLLERSHADE_EXTERIOR_2_MOTOR - Lift support * DRAPERY - Lift support * AWNING - Lift support * SHUTTER - Tilt support * BLIND_TILT_ONLY - Tilt support * BLIND_LIFT_AND_TILT - Lift and Tilt support * PROJECTOR_SCREEN - Lift support

API Reference

Constructor

ZigbeeWindowCovering Creates a new Zigbee window covering endpoint.

```
ZigbeeWindowCovering(uint8_t endpoint);
```

- `endpoint` - Endpoint number (1-254)

Position Control

setLiftPosition Sets the window covering lift position.

```
bool setLiftPosition(uint16_t lift_position);
```

- `lift_position` - Lift position

This function will return `true` if successful, `false` otherwise.

setLiftPercentage Sets the window covering lift position as a percentage.

```
bool setLiftPercentage(uint8_t lift_percentage);
```

- `lift_percentage` - Lift percentage (0-100, where 0 is fully closed, 100 is fully open)

This function will return `true` if successful, `false` otherwise.

setTiltPosition Sets the window covering tilt position in degrees.

```
bool setTiltPosition(uint16_t tilt_position);
```

- tilt_position - Tilt position in degrees

This function will return `true` if successful, `false` otherwise.

setTiltPercentage Sets the window covering tilt position as a percentage.

```
bool setTiltPercentage(uint8_t tilt_percentage);
```

- tilt_percentage - Tilt percentage (0-100)

This function will return `true` if successful, `false` otherwise.

Configuration

setCoveringType Sets the window covering type.

```
bool setCoveringType(ZigbeeWindowCoveringType covering_type);
```

- covering_type - Window covering type (see supported types above)

This function will return `true` if successful, `false` otherwise.

setConfigStatus Sets the window covering configuration status.

```
bool setConfigStatus(bool operational, bool online, bool commands_reversed, bool ↵  
↵lift_closed_loop, bool tilt_closed_loop, bool lift_encoder_controlled, bool tilt ↵  
↵encoder_controlled);
```

- operational - Operational status
- online - Online status
- commands_reversed - Commands reversed flag
- lift_closed_loop - Lift closed loop flag
- tilt_closed_loop - Tilt closed loop flag
- lift_encoder_controlled - Lift encoder controlled flag
- tilt_encoder_controlled - Tilt encoder controlled flag

This function will return `true` if successful, `false` otherwise.

setMode Sets the window covering operation mode.

```
bool setMode(bool motor_reversed, bool calibration_mode, bool maintenance_mode, ↵  
↵bool leds_on);
```

- motor_reversed - Motor reversed flag
- calibration_mode - Calibration mode flag
- maintenance_mode - Maintenance mode flag
- leds_on - LEDs on flag

This function will return `true` if successful, `false` otherwise.

setLimits Sets the motion limits for the window covering.

```
bool setLimits(uint16_t installed_open_limit_lift, uint16_t installed_closed_limit ↵  
↵lift, uint16_t installed_open_limit_tilt, uint16_t installed_closed_limit_tilt);
```

- installed_open_limit_lift - Installed open limit for lift

- `installed_closed_limit_lift` - Installed closed limit for lift
- `installed_open_limit_tilt` - Installed open limit for tilt
- `installed_closed_limit_tilt` - Installed closed limit for tilt

This function will return `true` if successful, `false` otherwise.

Event Handling

onOpen Sets a callback function to be called when the window covering opens.

```
void onOpen(void (*callback)());
```

- `callback` - Function to call when window covering opens

onClose Sets a callback function to be called when the window covering closes.

```
void onClose(void (*callback)());
```

- `callback` - Function to call when window covering closes

onGoToLiftPercentage Sets a callback function to be called when lift percentage changes.

```
void onGoToLiftPercentage(void (*callback)(uint8_t));
```

- `callback` - Function to call when lift percentage changes

onGoToTiltPercentage Sets a callback function to be called when tilt percentage changes.

```
void onGoToTiltPercentage(void (*callback)(uint8_t));
```

- `callback` - Function to call when tilt percentage changes

onStop Sets a callback function to be called when window covering stops.

```
void onStop(void (*callback)());
```

- `callback` - Function to call when window covering stops

Example

Window Covering Implementation

```
// Copyright 2025 Espressif Systems (Shanghai) PTE LTD
//
// Licensed under the Apache License, Version 2.0 (the "License");
// you may not use this file except in compliance with the License.
// You may obtain a copy of the License at
//
//     http://www.apache.org/licenses/LICENSE-2.0
//
// Unless required by applicable law or agreed to in writing, software
// distributed under the License is distributed on an "AS IS" BASIS,
// WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
// See the License for the specific language governing permissions and
// limitations under the License.
```

(continues on next page)

(continued from previous page)

```

/**
 * @brief This example demonstrates Zigbee Window Covering.
 *
 * The example demonstrates how to use Zigbee library to create a end device.
 * ↳ window covering device.
 * The window covering is a Zigbee end device, which is moving the blinds.
 * ↳ (lift+tilt) and reporting
 * its current position to the Zigbee network.
 *
 * Use setCoveringType() to set the type of covering (blind, shade, etc.).
 *
 * The example also demonstrates how to use the button to manually control the
 * ↳ lift position.
 *
 * Proper Zigbee mode must be selected in Tools->Zigbee mode
 * and also the correct partition scheme must be selected in Tools->Partition
 * ↳ Scheme.
 *
 * Please check the README.md for instructions and more detailed description.
 *
 * Created by hennikul and Jan Procházka (https://github.com/P-R-O-C-H-Y/)
 */

#include <Arduino.h>
#ifndef ZIGBEE_MODE_ED
#error "Zigbee end device mode is not selected in Tools->Zigbee mode"
#endif

#include "ZigbeeCore.h"
#include "ep/ZigbeeWindowCovering.h"

#define ZIGBEE_COVERING_ENDPOINT 10
#define BUTTON_PIN                9 // ESP32-C6/H2 Boot button

#define MAX_LIFT 200 // centimeters from open position (0-900)
#define MIN_LIFT 0

#define MAX_TILT 40 // centimeters from open position (0-900)
#define MIN_TILT 0

uint16_t currentLift = MAX_LIFT;
uint8_t currentLiftPercentage = 100;

uint16_t currentTilt = MAX_TILT;
uint8_t currentTiltPercentage = 100;

ZigbeeWindowCovering zbCovering = ZigbeeWindowCovering(ZIGBEE_COVERING_ENDPOINT);

void setup() {
  Serial.begin(115200);

  // Init button for factory reset
  pinMode(BUTTON_PIN, INPUT_PULLUP);

  // Optional: set Zigbee device name and model
  zbCovering.setManufacturerAndModel("Espressif", "WindowBlinds");

  // Set proper covering type, it defines which attributes are available
  zbCovering.setCoveringType(BLIND_LIFT_AND_TILT);

```

(continues on next page)

(continued from previous page)

```

// Set configuration: operational, online, not commands_reversed, lift / tilt_
↪closed_loop, lift / tilt encoder_controlled
zbCovering.setConfigStatus(true, true, false, true, true, true, true);

// Set mode: not motor_reversed, calibration_mode, not maintenance_mode, not_
↪leds_on
zbCovering.setMode(false, true, false, false);

// Set limits of motion
zbCovering.setLimits(MIN_LIFT, MAX_LIFT, MIN_TILT, MAX_TILT);

// Set callback function for open, close, filt and tilt change, stop
zbCovering.onOpen(fullOpen);
zbCovering.onClose(fullClose);
zbCovering.onGoToLiftPercentage(goToLiftPercentage);
zbCovering.onGoToTiltPercentage(goToTiltPercentage);
zbCovering.onStop(stopMotor);

// Add endpoint to Zigbee Core
Serial.println("Adding ZigbeeWindowCovering endpoint to Zigbee Core");
Zigbee.addEndpoint(&zbCovering);

// When all EPs are registered, start Zigbee. By default acts as ZIGBEE_END_
↪DEVICE
Serial.println("Calling Zigbee.begin()");
if (!Zigbee.begin()) {
    Serial.println("Zigbee failed to start!");
    Serial.println("Rebooting...");
    ESP.restart();
}
Serial.println("Connecting to network");
while (!Zigbee.connected()) {
    Serial.print(".");
    delay(100);
}
Serial.println();

// Set initial position
zbCovering.setLiftPercentage(currentLiftPercentage);
zbCovering.setTiltPercentage(currentTiltPercentage);
}

void loop() {
    // Checking button for factory reset
    if (digitalRead(BUTTON_PIN) == LOW) { // Push button pressed
        // Key debounce handling
        delay(100);
        int startTime = millis();
        while (digitalRead(BUTTON_PIN) == LOW) {
            delay(50);
            if ((millis() - startTime) > 3000) {
                // If key pressed for more than 3secs, factory reset Zigbee and reboot
                Serial.printf("Resetting Zigbee to factory settings, reboot.\n");
                Zigbee.factoryReset();
                delay(30000);
            }
        }
        // Manual lift control simulation by pressing button
        manualControl();
    }
    delay(500);
}

```

(continues on next page)

(continued from previous page)

```

}

void fullOpen() {
    /* This is where you would trigger your motor to go to full open state,
    ↳currentLift should
        be updated until full open has been reached in order to provide feedback to
    ↳controller of actual position
        The stop can be always called, so the movement can be stopped at any time */

    // Our cover updates instantly!
    currentLift = MAX_LIFT;
    currentLiftPercentage = 100;
    Serial.println("Opening cover");
    // Update the current position
    zbCovering.setLiftPercentage(currentLiftPercentage);
}

void fullClose() {
    /* This is where you would trigger your motor to go to full close state,
    ↳currentLift should
        be updated until full close has been reached in order to provide feedback to
    ↳controller of actual position
        The stop can be always called, so the movement can be stopped at any time */

    // Our cover updates instantly!
    currentLift = MIN_LIFT;
    currentLiftPercentage = 0;
    Serial.println("Closing cover");
    // Update the current position
    zbCovering.setLiftPercentage(currentLiftPercentage);
}

void goToLiftPercentage(uint8_t liftPercentage) {
    /* This is where you would trigger your motor to go towards liftPercentage,
    ↳currentLift should
        be updated until liftPercentage has been reached in order to provide feedback
    ↳to controller */

    // Our simulated cover updates instantly!
    currentLift = (liftPercentage * MAX_LIFT) / 100;
    currentLiftPercentage = liftPercentage;
    Serial.printf("New requested lift from Zigbee: %u (%u)\n", currentLift,
    ↳liftPercentage);

    // Update the current position
    zbCovering.setLiftPercentage(currentLiftPercentage); //or setLiftPosition()
}

void goToTiltPercentage(uint8_t tiltPercentage) {
    /* This is where you would trigger your motor to go towards tiltPercentage,
    ↳currentTilt should
        be updated until tiltPercentage has been reached in order to provide feedback
    ↳to controller */

    // Our simulated cover updates instantly!
    currentTilt = (tiltPercentage * MAX_TILT) / 100;
    currentTiltPercentage = tiltPercentage;
    Serial.printf("New requested tilt from Zigbee: %u (%u)\n", currentTilt,
    ↳tiltPercentage);

    // Update the current position

```

(continues on next page)

(continued from previous page)

```

    zbCovering.setTiltPercentage(currentTiltPercentage); //or setTiltPosition()
}

void stopMotor() {
    // Motor can be stopped while moving cover toward current target, when stopped.
    ↳the actual position should be updated
    Serial.println("Stopping motor");
    // Update the current position of both lift and tilt
    zbCovering.setLiftPercentage(currentLiftPercentage);
    zbCovering.setTiltPercentage(currentTiltPercentage);
}

void manualControl() {
    // Simulate lift percentage move by increasing it by 20% each time
    currentLiftPercentage += 20;
    if (currentLiftPercentage > 100) {
        currentLiftPercentage = 0;
    }
    zbCovering.setLiftPercentage(currentLiftPercentage);
    // Also setLiftPosition() can be used to set the exact position instead of.
    ↳percentage
}

```

Specific endpoint classes Library provides the following endpoint classes from lights, switches, sensors, etc. Each endpoint class provides the specific functionality for each endpoint type and inherits from the ZigbeeEP class.

Common Problems and Issues

Troubleshooting

Common Issues

Device won't join network

- Ensure the coordinator is in pairing mode
- Check that the device is configured with the correct role
- Verify the channel mask includes the coordinator's channel

OTA updates fail

- Ensure the OTA server is properly configured
- Check that the device has sufficient memory for the update
- Verify network connectivity

Battery devices not working

- Ensure proper power source configuration
- Check sleep/wake timing settings
- Verify parent device (router/coordinator) is always powered

Binding issues

- Check that both devices support the required clusters
- Verify that binding is enabled on both devices
- Ensure devices are on the same network

Network connectivity problems

- Check that devices are within range
- Verify that routers are properly configured
- Check for interference from other 2.4 GHz devices

Factory Reset If you have problems with connecting to the network, you can try to factory reset the device. This will erase all the network settings and act as brand new device.

```
Zigbee.factoryReset(true); // true = restart after reset
```

Debug Mode For better debugging, you can enable debug mode to get detailed information about network operations. Call debug mode before starting Zigbee. Also selecting zigbee mode with *debug* suffix is recommended.

```
Zigbee.setDebugMode(true);  
// Start Zigbee with debug output  
Zigbee.begin();
```


Chapter 3

Guides

3.1 Coding Guidelines

This page describes the coding standards used in the Arduino ESP32 project. Adherence to these guidelines ensures consistency, portability, and compatibility across all supported ESP-IDF configurations and future hardware targets.

3.1.1 Code Formatting

The project uses [clang-format](#) version 18.1.3 for automatic formatting. The configuration is stored in `.clang-format` at the repository root. A pre-commit hook enforces formatting before every commit, so contributors can also run `clang-format` manually or rely on CI to flag violations.

If you need to ensure that some part of the code is formatted in a specific way, you can use the `// clang-format off` and `// clang-format on` on comments to disable and enable formatting for the specific part of the code. For example:

```
// clang-format off
int x    = 1;
// clang-format on

int y = 2;
```

The style is based on the LLVM preset with the following key rules:

Indentation and Spacing

- **Indent width:** 2 spaces. Tabs are never used.
- **Continuation indent:** 2 spaces for wrapped lines.
- **Column limit:** 160 characters per line.
- **Trailing comments:** 2 spaces before inline `//` comments.
- **Spaces before parentheses:** required after control-flow keywords (`if` `(`, `for` `(`, `while` `(`), not after function names (`foo` `(`).

```
// Correct
if (condition) {
    foo(a, b);
}

// Wrong
if(condition){
    foo (a, b);
}
```

Braces

- **Style:** K&R (opening brace on the same line as the statement).
- **Mandatory braces:** clang-format inserts braces around single-statement bodies automatically (`InsertBraces: true`). Never omit braces even for single-statement `if/for/while` bodies.
- **Short blocks:** empty function/record bodies may be on a single line. Short `if` statements must always span two lines; short `for/while` loops may be on one line.

```
// Correct - braces always present
if (error) {
    return false;
}
for (int i = 0; i < n; i++) continue; // short loop - OK on one line

// Wrong - missing braces
if (error)
    return false;
```

Switch / Case

- Case labels are **indented** inside the `switch`.
- Short case bodies may be on the same line as the label.
- When a case body requires braces (e.g., to declare a variable), the opening brace goes on the **next** line after the label (Allman style for case blocks only).

```
switch (state) {
    case STATE_IDLE:    handle_idle(); break;
    case STATE_ACTIVE:
    {
        int tmp = compute();
        handle_active(tmp);
        break;
    }
    default: break;
}
```

Pointers and References

- Pointer `*` and reference `&` attach to the **variable name**, not the type name (`PointerAlignment: Right`).

```
// Correct
int *ptr;
void func(const char *buf, size_t len);

// Wrong
int* ptr;
void func(const char* buf, size_t len);
```

Include Order

- Include order is **preserved as written** (`SortIncludes: Never`). Do not rely on automatic sorting; group and order headers intentionally.

Line Endings

- All files must use **LF** (Unix) line endings (`LineEnding: LF`).

- Every file must end with a **single newline** (`InsertNewlineAtEOF: true`).

Macro Alignment

- Consecutive `#define` macro definitions are horizontally aligned (`AlignConsecutiveMacros`).

```
#define PIN_SCL    22
#define PIN_SDA    21
#define TIMEOUT    1000
```

3.1.2 Format Specifiers

Arduino ESP32 supports three C standard library configurations:

- **NEWLIB full** –supports all C99 format specifiers including `%hh`, `%h`, `%z`, `%ll`.
- **NEWLIB-nano** –a size-optimised subset; supports only C89 specifiers. Does **not** support `%hh`, `%h`, `%z`, or `%ll`.
- **PICOLIBC** –supports C99 specifiers by default.

The project must also remain forward-compatible with future 64-bit RISC-V targets (LP64 model, where `long` and pointers are 64-bit).

Use the rules in the table below to select the correct format specifier for each type.

C type	Correct specifier	Notes
<code>uint8_t / int8_t</code>	<code>%u / %d</code>	Integer promotion to <code>int</code> makes these safe everywhere. Do not use <code>PRId8 / PRIu8</code> .
<code>uint16_t / int16_t</code>	<code>%u / %d</code>	Same reason. Do not use <code>PRId16 / PRIu16</code> .
<code>uint8_t / uint16_t</code> (hex)	<code>%x / %X</code>	Use width flags as needed: <code>%02x</code> , <code>%04X</code> . Do not use <code>PRId8 / PRIu8</code> .
<code>uint32_t</code>	<code>"%" PRIu32</code>	Must use the <code>PRI</code> macro; expands to <code>"u"</code> or <code>"lu"</code> depending on the library.
<code>int32_t</code>	<code>"%" PRId32</code>	Same rule.
<code>uint32_t</code> (hex)	<code>"%" PRIx32 / "%" PRId32</code>	Same rule.
<code>uint64_t</code>	<code>"%s"</code>	Use <code>u64_to_str(value)</code> or <code>u64_to_String(value)</code> from <code>StringUtils.h</code> . In sketches you can also use <code>Serial.print(value)</code> .
<code>int64_t</code>	<code>"%s"</code>	Use <code>i64_to_str(value)</code> or <code>i64_to_String(value)</code> from <code>StringUtils.h</code> . In sketches you can also use <code>Serial.print(value)</code> .
<code>size_t</code>	<code>%lu</code> with (unsigned long) cast	<code>%zu</code> is not supported on NEWLIB-nano. Cast to (unsigned long) before printing: <code>printf("%lu", (unsigned long)len)</code> . On ILP32 this is a no-op; on LP64 unsigned long matches <code>size_t</code> .
<code>unsigned long / long</code>	<code>%lu / %ld</code>	Correct as-is; no cast needed.
<code>int / unsigned int</code>	<code>%d / %u / %x / %X</code>	Correct as-is.

Note: `#include <inttypes.h>` is required for the `PRId32 / PRIu32 / PRIx32` etc. macros. In most Arduino ESP32 files this is provided transitively by `Arduino.h`, `esp32-hal.h`, or `esp32-hal-log.h` (which

includes `esp_log.h` → `inttypes.h`). For standalone C files that do not include any of those headers, add `#include <inttypes.h>` explicitly.

Printing Enums

The correct specifier for an enum depends on its **underlying type**, not on the range of its values.

C enums and C++ unscoped enums

The underlying type is always `int` (signed 32-bit). Always use `%d`, even when all values are non-negative.

```
typedef enum { STATE_IDLE, STATE_RUNNING, STATE_ERROR } state_t;
state_t state = STATE_RUNNING;
log_d("State: %d", state);

// Wrong - enums are int, not long
log_d("State: %ld", state);
```

C++ scoped enums (enum class)

Scoped enums do **not** implicitly convert to integers. An explicit cast is required. The specifier must match the underlying type of the cast.

```
// Default underlying type: int
enum class Color { Red, Green, Blue };
Color c = Color::Green;
log_d("Color: %d", (int)c);

// Explicit underlying type: uint8_t (promoted to int, %u is safe)
enum class Pin : uint8_t { SCL = 22, SDA = 21 };
Pin p = Pin::SCL;
log_d("Pin: %u", (uint8_t)p);

// Explicit underlying type: uint32_t
enum class Flags : uint32_t { None = 0, Read = 1, Write = 2 };
Flags f = Flags::Read;
log_d("Flags: %" PRIu32, (uint32_t)f);
```

ESP-IDF enums

ESP-IDF types such as `esp_err_t`, `arduino_event_id_t`, and driver event types are C enums (underlying type `int`). Use `%d`. Do **not** use `%ld` or `%" PRIu32` —they are `int`, not `int32_t` or `long`.

```
esp_err_t err = some_function();
log_e("Error: %d (%s)", err, esp_err_to_name(err));

// Wrong
log_e("Error: %ld", err);
```

Key rule: always match the specifier to the *actual underlying type* of the enum value being printed. When in doubt, cast explicitly and use the specifier that corresponds to the cast type.

Examples

```
#include <StringUtils.h>

uint8_t  pin      = 5;
uint16_t addr     = 0x1234;
uint32_t baudrate = 115200;
uint64_t uptime   = 1000000000ULL;
```

(continues on next page)

(continued from previous page)

```

size_t len      = data.size();
int     err      = -1;

char buffer[U64_STR_SIZE];

// Correct
log_d("pin=%u addr=0x%04x baud=%" PRIu32 " up=%s len=%lu err=%d",
      pin, addr, baudrate, u64_to_str(uptime, buffer), (unsigned long)len, err);

// Wrong - uses PRIu8/PRIu16 (breaks NEWLIB-nano), %zu and %llu (breaks NEWLIB-
// nano),
//          and %lu for uint32_t (wrong on PICOLIBC/LP64)
log_d("pin=%" PRIu8 " addr=0x%04" PRIu16 " baud=%lu up=%llu len=%zu err=%d",
      pin, addr, baudrate, uptime, len, err);

```

3.2 Compatibility Guide for ESP32 Arduino Core

3.2.1 Introduction

Welcome to the compatibility guide for library developers aiming to support multiple versions of the ESP32 Arduino core. This documentation provides essential tips and best practices for ensuring compatibility with 2.x, 3.x and future versions of the ESP32 Arduino core.

3.2.2 Code Adaptations

To ensure compatibility with both versions of the ESP32 Arduino core, developers should utilize conditional compilation directives in their code. Below is an example of how to conditionally include code based on the ESP32 Arduino core version:

```

#ifdef ESP_ARDUINO_VERSION_MAJOR
#if ESP_ARDUINO_VERSION >= ESP_ARDUINO_VERSION_VAL(3, 0, 0)
    // Code for version 3.x
#else
    // Code for version 2.x
#endif
#else
    // Code for version 1.x
#endif

```

3.2.3 Version Print

To easily print the ESP32 Arduino core version at runtime, developers can use the `ESP_ARDUINO_VERSION_STR` macro. Below is an example of how to print the ESP32 Arduino core version:

```
Serial.printf(" ESP32 Arduino core version: %s\n", ESP_ARDUINO_VERSION_STR);
```

3.2.4 API Differences

Developers should be aware, that there may be API differences between major versions of the ESP32 Arduino core. For this we created a [Migration guide](#), to help developers transition from between major versions of the ESP32 Arduino core.

3.2.5 Library Testing

We have added an External Library Test CI job, which tests external libraries with the latest version of the ESP32 Arduino core to help developers ensure compatibility with the latest version of the ESP32 Arduino core. If you want to include your library in the External Library Test CI job, please follow the instructions in the [External Libraries Test](#).

3.3 Compile Arduino libs with ESP_LOGx

There are 2 primary approaches and both of them involve editing file `configs/defconfig.common`. Edit the file directly and then build. Later you can `git restore configs/defconfig.common` to go back. Copy the file `cp configs/defconfig.common configs/defconfig.debug` and edit the debug version.

`vim configs/defconfig.common` or `vim configs/defconfig.debug`

Edit **line 44** containing by default `CONFIG_LOG_DEFAULT_LEVEL_ERROR=y` to one of the following lines depending on your desired log level:

```
CONFIG_LOG_DEFAULT_LEVEL_NONE=y # No output
CONFIG_LOG_DEFAULT_LEVEL_ERROR=y # Errors - default
CONFIG_LOG_DEFAULT_LEVEL_WARN=y # Warnings
CONFIG_LOG_DEFAULT_LEVEL_INFO=y # Info
CONFIG_LOG_DEFAULT_LEVEL_DEBUG=y # Debug
CONFIG_LOG_DEFAULT_LEVEL_VERBOSE=y # Verbose
```

Then simply build the libs for all SoCs or one specific SoC. Note that building for all SoCs takes a lot of time, so if you are working only with specific SoC(s), build only for those.

Note: If you have copied the `defconfig` file and the debug settings are in file `configs/defconfig.debug` add flag `debug` to compilation command. Example: `./build.sh debug`

- **Option 1:** Build for all SoCs: `./build.sh`
 - **Option 2:** Build for one SoC: `./build.sh -t <soc>`. The exact text to choose the SoC:
 - `esp32`
 - `esp32s2`
 - `esp32c3`
 - `esp32s3`
 - Example: `./build.sh -t esp32`
 - A wrong format or non-existing SoC will result in the error `sed: can't read sdkconfig: No such file or directory`
-

3.4 Documentation Contribution Guidelines

3.4.1 Introduction

This is a guideline for the Arduino ESP32 project documentation. The idea for this guideline is to show how to start collaborating on the project.

The guideline works to give you the directions and to keep the documentation more concise, helping users to better understand the structure.

3.4.2 About Documentation

We all know how important documentation is. This project is no different.

This documentation was created in a collaborative and open way, letting everyone contribute, from a small typo fix to a new chapter writing. We try to motivate our community by giving all the support needed through this guide.

The documentation is in **American English only**. Future translations can be added when we finish the essential content in English first.

3.4.3 How to Collaborate

Everyone with some knowledge to share is welcome to collaborate.

One thing you need to consider is the fact that your contribution must be concise and assertive since it will be used by people developing projects. The information is very important for everyone, be sure you are not making the developer's life harder!

3.4.4 Documentation Guide

This documentation is based on the [Sphinx](#) with [reStructuredText](#) and hosted by [ReadTheDocs](#).

If you want to get started with [Sphinx](#), see the official documentation:

- [Documentation Index](#)
- [Basics](#)
- [Directives](#)

First Steps

Before starting your collaboration, you need to get the documentation source code from the Arduino-ESP32 project.

- **Step 1** - Fork the [Arduino-ESP32](#) to your GitHub account.
- **Step 2** - Check out the recently created fork.
- **Step 3** - Create a new branch for the changes/addition to the docs.
- **Step 4** - Write!

Requirements

To build the documentation properly, you need to install some packages in your system. Note that depending on your system, you may need to use a virtual environment to install the packages.

```
pip install -r requirements.txt
```

The requirements file is under the `docs` folder.

Using Visual Studio Code

If you are using the Visual Studio Code, you can install some extensions to help you while writing documentation. For reStructuredText, you can install the [reStructuredText Pack](#) extension.

We also recommend you to install some grammar check extension to help you to review English grammar.

Building

To build the documentation and generate the HTML files, you can use the following command inside the `docs` folder. After a successful build, you can check the files inside the `_build/en/generic/html` folder.

```
build-docs -l en
```

This step is essential to ensure that there are no syntax errors and also to see the final result.

If everything is ok, you will see some output logs similar to this one:

```
Running Sphinx v2.3.1
loading pickled environment... done
building [mo]: targets for 0 po files that are out of date
building [html]: targets for 35 source files that are out of date
updating environment: [extensions changed ('sphinx_tabs.tabs')] 41 added, 3_
↳changed, 0 removed
reading sources... [100%] tutorials/tutorials
looking for now-outdated files... none found
pickling environment... done
checking consistency... done
preparing documents... done
writing output... [100%] tutorials/tutorials
generating indices... genindexdone
writing additional pages... searchdone
copying images... [100%] tutorials/../../_static/tutorials/peripherals/tutorial_
↳peripheral_diagram.png
copying static files... .. done
copying extra files... done
dumping search index in English (code: en)... done
dumping object inventory... done
build succeeded.
```

The HTML pages are in `_build/en/generic/html`.

3.4.5 Sections

The Arduino ESP32 is structured in some sections to make it easier to maintain. Here is a brief description of this structure.

API

In this section, you will include all the documentation about drivers, libraries, and any other related to the core.

In this section, we do not add general information. For more general information, we have sections for other related parts, like the FAQ, library builder, troubleshooting, etc.

Boards

Here is the place to add any special guide on the development boards, pin layout, schematics, and any other relevant content.

Common

In this folder, you can add all common information used in several different places. This helps to make documentation easily maintainable.

Guides

This is the place to add the guides for common applications, IDEs configuration, and any other information that can be used as a guideline.

Tutorials

If you want to add a specific tutorial related to the Arduino core for ESP32, this is the place. The intention is not to create a blog or a demo area, but this can be used to add some complex description or to add some more information about APIs.

Images and Assets

All the files used on the documentation must be stored in the `_static` folder. Be sure that the content used is not with any copyright restriction.

3.4.6 Documentation Rules

Here are some guidelines to help you. We also recommend copying a sample file from the same category you are creating.

This will help you to follow the structure as well as to get inspired.

Espressif Manual of Style

The [Espressif Manual of Style](#) (esp-mos) describes established practices for Espressif documentation: punctuation, numbers, units, mathematical expressions, figures, color accessibility, tables, UI elements, and admonitions. Follow it when writing or editing pages here.

Basic Structure

To help you create a new section from scratch, we recommend you include this structure in your content if it applies.

- **About - Brief description of the document.**
 - Description of the peripheral, driver, protocol, including all different modes and configurations.
- **API** - Description of each public function, macros, and structs.
- **Basic Usage**
- **Example Application**

About Section In this section, you need to add a brief description of the API. If you are describing a peripheral API, you should explain a little bit about the peripheral and the working modes, if it's applicable.

API Functions To add a new function description, you must know that the users only have access to the public functions.

Here is an example of how to add the function description from [I2C API](#):

```
setPins
^^^^^^

This function is used to define the ``SDA`` and ``SCL`` pins.

.. note:: Call this function before ``begin`` to change the pins from the default_
↪ones.

.. code-block:: arduino

    bool setPins(int sdaPin, int sclPin);

* ``sdaPin`` sets the GPIO to be used as the I2C peripheral data line.
```

(continues on next page)

(continued from previous page)

```
* ``sclPin`` sets the GPIO to be used as the I2C peripheral clock line.

The default pins may vary from board to board. On the *Generic ESP32* the default
↪ I2C pins are:

* ``sdaPin`` **GPIO21**

* ``sclPin`` **GPIO22**

This function will return ``true`` if the peripheral was configured correctly.
```

Be sure to include a very comprehensive description, add all the parameters in and out, and describe the desired output.

If the function uses a specific structure, you can also describe the structure in the same function block or add a specific section if the structure is shared with other functions.

Basic Usage Some APIs are more complex to use or require more steps in order to configure or initialize. If the API is not straightforward in terms of usability, please consider adding a how-to-use section describing all the steps to get the API configured.

Here is an example:

```
Basic Usage
^^^^^^^^^^

To start using I2C as slave mode on the Arduino, the first step is to include the
↪ ``Wire.h`` header to the sketch.

.. code-block:: arduino

    #include "Wire.h"

Before calling ``begin``, you must create two callback functions to handle the
↪ communication with the master device.

.. code-block:: arduino

    Wire.onReceive(onReceive);

and

.. code-block:: arduino

    Wire.onRequest(onRequest);

The ``onReceive`` will handle the request from the ``master`` device upon a slave
↪ read request and the ``onRequest`` will handle the answer to the master.

Now, we can start the peripheral configuration by calling ``begin`` function with
↪ the device address.

.. code-block:: arduino

    Wire.begin((uint8_t) I2C_DEV_ADDR);

By using ``begin`` without any arguments, all the settings will be done by using
↪ the default values. To set the values on your own, see the function description.
↪ This function is described here: `i2c begin`
```

Example Application It is very important to include at least one application example or a code snippet to help people using the API.

If the API does not have any application example, you can embed the code directly. However, if the example is available, you must include it as a literal block.

```
.. literalinclude:: ../../../../libraries/WiFi/examples/WiFiAccessPoint/  
↳WiFiAccessPoint.ino  
   :language: arduino
```

3.4.7 Sphinx Basics

Heading Levels

The heading levels used on this documentation are:

- **H1:** - (Dash)
- **H2:** * (Asterisk)
- **H3:** ^ (Circumflex)
- **H4:** # (Sharp)

Code Block

To add a code block, you can use the following structure:

```
.. code-block:: arduino  
   bool begin(); //Code example
```

Links

To include links to external content, you can use two ways.

- First option:

```
`Arduino Wire Library`_  
_Arduino Wire Library: https://www.arduino.cc/en/reference/wire
```

- Second option:

```
`Arduino Wire Library <https://www.arduino.cc/en/reference/wire>`_
```

Images

To include images in the docs, first, add all the files into the `_static` folder with a filename that makes sense for the topic.

After that, you can use the following structure to include the image in the docs.

```
.. figure:: ../../_static/arduino_i2c_master.png  
   :align: center  
   :width: 720  
   :figclass: align-center
```

You can adjust the `width` according to the image size.

Be sure the file size does not exceed 600 kB.

Support

If you need support on the documentation, you can ask a question in the discussion [here](#).

3.4.8 Additional Guidelines

If you want to contribute with code on the Arduino ESP32 core, be sure to follow the [ESP-IDF Documenting Code](#) as a reference.

3.5 Arduino IDE Tools Menu

3.5.1 Introduction

This guide is a walkthrough of the Arduino IDE configuration menu for the ESP32 System on Chip (SoC's). In this guide, you will see the most relevant configuration to get your project optimized and working.

Since some boards and SoC's may vary in terms of hardware configuration, be sure you know all the board characteristics that you are using, like flash memory size, SoC variant (ESP32 family), PSRAM, etc.

Note: To help you identify the characteristics, you can see the [Espressif Product Selector](#).

3.5.2 Arduino IDE

The Arduino IDE is widely used for ESP32 on Arduino development and offers a wide variety of configurations.

3.5.3 Tools Menu

To properly configure your project build and flash, some settings must be done in order to get it compiled and flashed without any issues. Some boards are natively supported and almost no configuration is required. However, if your is not yet supported or you have a custom board, you need to configure the environment by yourself.

For more details or to add a new board, see the [boards.txt](#) file.

3.5.4 Generic Options

Most of the options are available for every ESP32 family. Some options will be available only for specific targets, like the USB configuration.

Board

This option is the target board and must be selected in order to get all the default configuration settings. Once you select the correct board, you will see that some configurations will be automatically selected, but be aware that some boards can have multiple versions (i.e different flash sizes).

To select the board, go to `Tools -> Board -> ESP32 Arduino` and select the target board.

If your board is not present on this list, you can select the generic `ESP32-XX Dev Module`.

Currently, we have one generic development module for each of the supported targets.

If the board selected belongs to another SoC family, you will see the following information at the build output:

```
A fatal error occurred: This chip is ESP32 not ESP32-S2. Wrong --chip argument?
```

Upload Speed

To select the flashing speed, change the `Tools -> Upload Speed`. This value will be used for flashing the code to the device.

Note: If you have issues while flashing the device at high speed, try to decrease this value. This could be due to the external serial-to-USB chip limitations.

CPU Frequency

On this option, you can select the CPU clock frequency. This option is critical and must be selected according to the high-frequency crystal present on the board and the radio usage (Wi-Fi and Bluetooth).

In some applications, reducing the CPU clock frequency is recommended in order to reduce power consumption.

If you don't know why you should change this frequency, leave the default option.

Flash Frequency

Use this function to select the flash memory frequency. The frequency will be dependent on the memory model.

- **40 MHz**
- **80 MHz**

If you don't know if your memory supports **80 MHz**, you can try to upload the sketch using the **80 MHz** option and watch the log output via the serial monitor.

Note: In some boards/SoC, the flash frequency is automatically selected according to the flash mode. In some cases (i.e ESP32-S3), the flash frequency is up to 120 MHz.

Flash Mode

This option is used to select the SPI communication mode with the flash memory.

Depending on the application, this mode can be changed in order to increase the flash communication speed.

- **QIO - Quad I/O Fast Read**
 - Four SPI pins are used to write to the flash and to read from the flash.
- **DIO - Dual I/O Fast Read**
 - Two SPI pins are used to write to the flash and to read from the flash.
- **QOUT - Quad Output Fast Read**
 - Four SPI pins are used to read the flash data.
- **DOOUT - Dual Output Fast Read**
 - Two SPI pins are used to read flash data.
- **OPI - Octal I/O**
 - Eight SPI pins are used to write and to read from the flash.

If you don't know how the board flash is physically connected or the flash memory model, try the **QIO** at **80 MHz** first.

Flash Size

This option is used to select the flash size. The flash size should be selected according to the flash model used on your board.

- **2 MB** (16 Mb)

- **4 MB** (32 Mb)
- **8 MB** (64 Mb)
- **16 MB** (128 Mb)

If you choose the wrong size, you may have issues when selecting the partition scheme.

Embedded Flash Some SoC has embedded flash. The ESP32-S3 is a good example.

Note: Check the manufacturer part number of your SoC/module to see the right version.

Example: **ESP32-S3FH4R2**

This particular ESP32-S3 variant comes with 4 MB Flash and 2 MB PSRAM.

Options for Embedded Flash

- **Fx4** 4 MB Flash (*QIO*)
- **Fx8** 8 MB Flash (*QIO*)
- **V** 1.8 V SPI

The **x** stands for the temperature range specification.

- **H** High Temperature (*-40 to 85°C*)
- **N** Low Temperature (*-40 to 65°C*)

For more details, please see the corresponding datasheet at [Espressif Product Selector](#).

Partition Scheme

This option is used to select the partition model according to the flash size and the resources needed, like storage area and OTA (Over The Air updates).

Note: Be careful selecting the right partition according to the flash size. If you select the wrong partition, the system will crash.

Core Debug Level

This option is used to select the Arduino core debugging level to be printed to the serial debug.

- **None** - Prints nothing.
- **Error** - Only at error level.
- **Warning** - Only at warning level and above.
- **Info** - Only at info level and above.
- **Debug** - Only at debug level and above.
- **Verbose** - Prints everything.

PSRAM

The PSRAM is an internal or external extended RAM present on some boards, modules or SoC.

This option can be used to `Enable` or `Disable` PSRAM. In some SoCs, you can select the PSRAM mode as the following.

- **QSPI PSRAM** - Quad PSRAM
- **OPI PSRAM** - Octal PSRAM

Embedded PSRAM Some SoC has embedded PSRAM. The ESP32-S3 is a good example.

Example: **ESP32-S3FH4R2**

This particular ESP32-S3 comes with 4 MB Flash and 2 MB PSRAM.

Options for Embedded Flash and PSRAM

- **R2** 2 MB PSRAM (*QSPI*)
- **R8** 8 MB PSRAM (*OPI*)
- **V** 1.8 V SPI

The **x** stands for the temperature range specification.

- **H** High Temperature (*-40 to 85°C*)
- **N** Low Temperature (*-40 to 65°C*)

For more details, please see the corresponding datasheet at [Espressif Product Selector](#).

Arduino Runs On

This function is used to select the core that runs the Arduino core. This is only valid if the target SoC has 2 cores.

When you have some heavy task running, you might want to run this task on a different core than the Arduino tasks. For this reason, you have this configuration to select the right core.

Events Run On

This function is also used to select the core that runs the Arduino events. This is only valid if the target SoC has 2 cores.

Erase All Flash Before Sketch Upload

This option selects the flash memory region to be erased before uploading the new sketch.

- **Disabled** - Upload the sketch without erasing all flash contents. (Default)
- **Enabled** - Erase all flash contents before uploading the sketch.

Port

This option is used to select the serial port to be used on the flashing and monitor.

3.5.5 USB Options

Some ESP32 families have a USB peripheral. This peripheral can be used for flashing and debugging.

To see the supported list for each SoC, see this section: [Libraries](#).

The USB option will be available only if the correct target is selected.

USB CDC On Boot

The USB Communications Device Class, or USB CDC, is a class used for basic communication to be used as a regular serial controller (like RS-232).

This class is used for flashing the device without any other external device attached to the SoC.

This option can be used to `Enable` or `Disable` this function at the boot. If this option is `Enabled`, once the device is connected via USB, one new serial port will appear in the list of the serial ports. Use this new serial port for flashing the device.

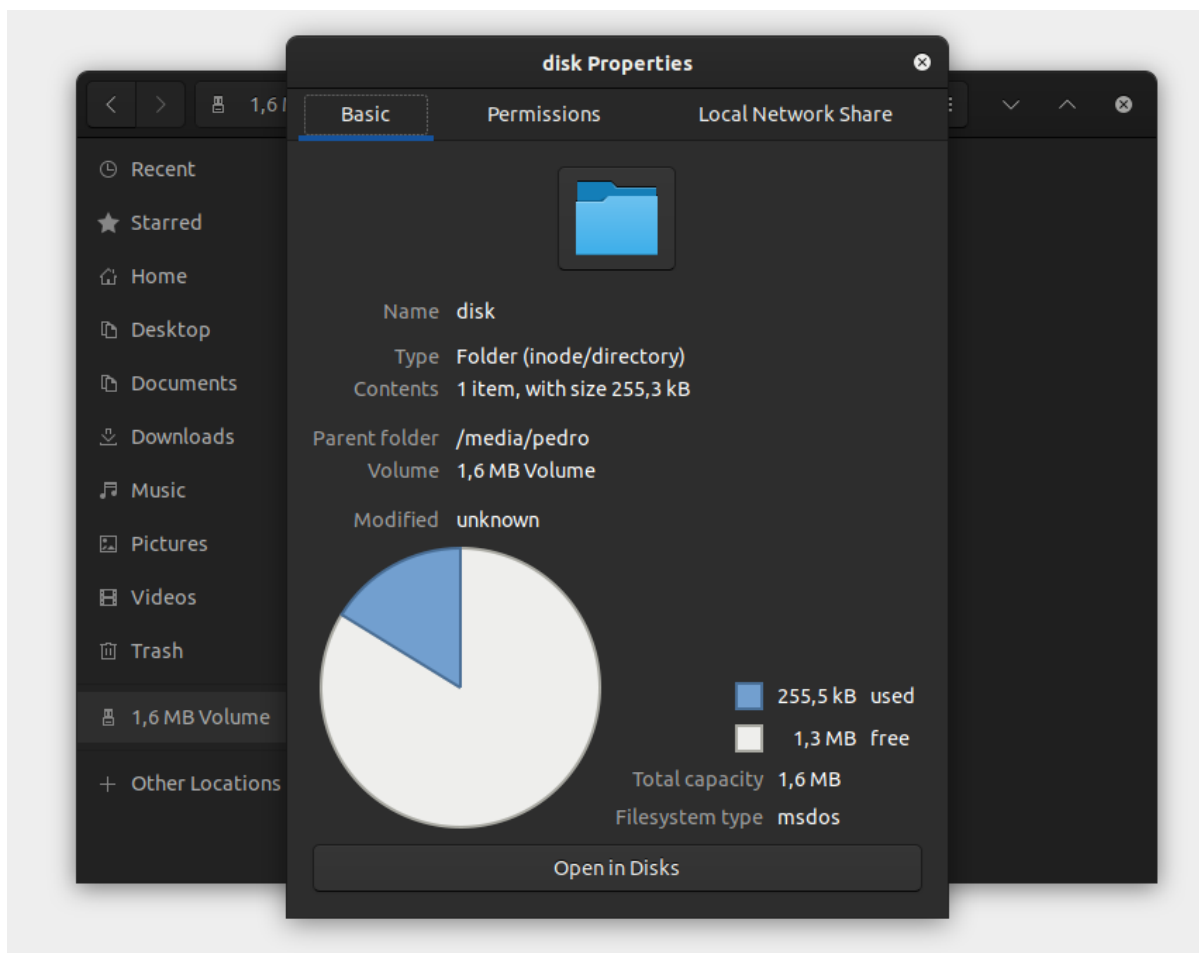
This option can be used as well for debugging via the Serial Monitor using **CDC** instead of the **UART0**.

To use the UART as serial output, you can use `Serial0.print("Hello World!");` instead of `Serial.print("Hello World!");` which will be printed using USB CDC.

USB Firmware MSC On Boot

The USB Mass Storage Class, or USB MSC, is a class used for storage devices, like a USB flash drive.

This option can be used to **Enable** or **Disable** this function at the boot. If this option is **Enabled**, once the device is connected via USB, one new storage device will appear in the system as a storage drive. Use this new storage drive to write and read files or to drop a new firmware binary to flash the device.



USB DFU On Boot

The USB Device Firmware Upgrade is a class used for flashing the device through USB.

This option can be used to **Enable** or **Disable** this function at the boot. If this option is **Enabled**, once the device is connected via USB, the device will appear as a USB DFU capable device.

Chapter 4

Tutorials

4.1 Basic Tutorial

4.1.1 Introduction

This is the basic tutorial and should be used as template for other tutorials.

4.1.2 Requirements

- Arduino IDE
- ESP32 Board
- Good USB Cable

4.1.3 Steps

Here are the steps for this tutorial.

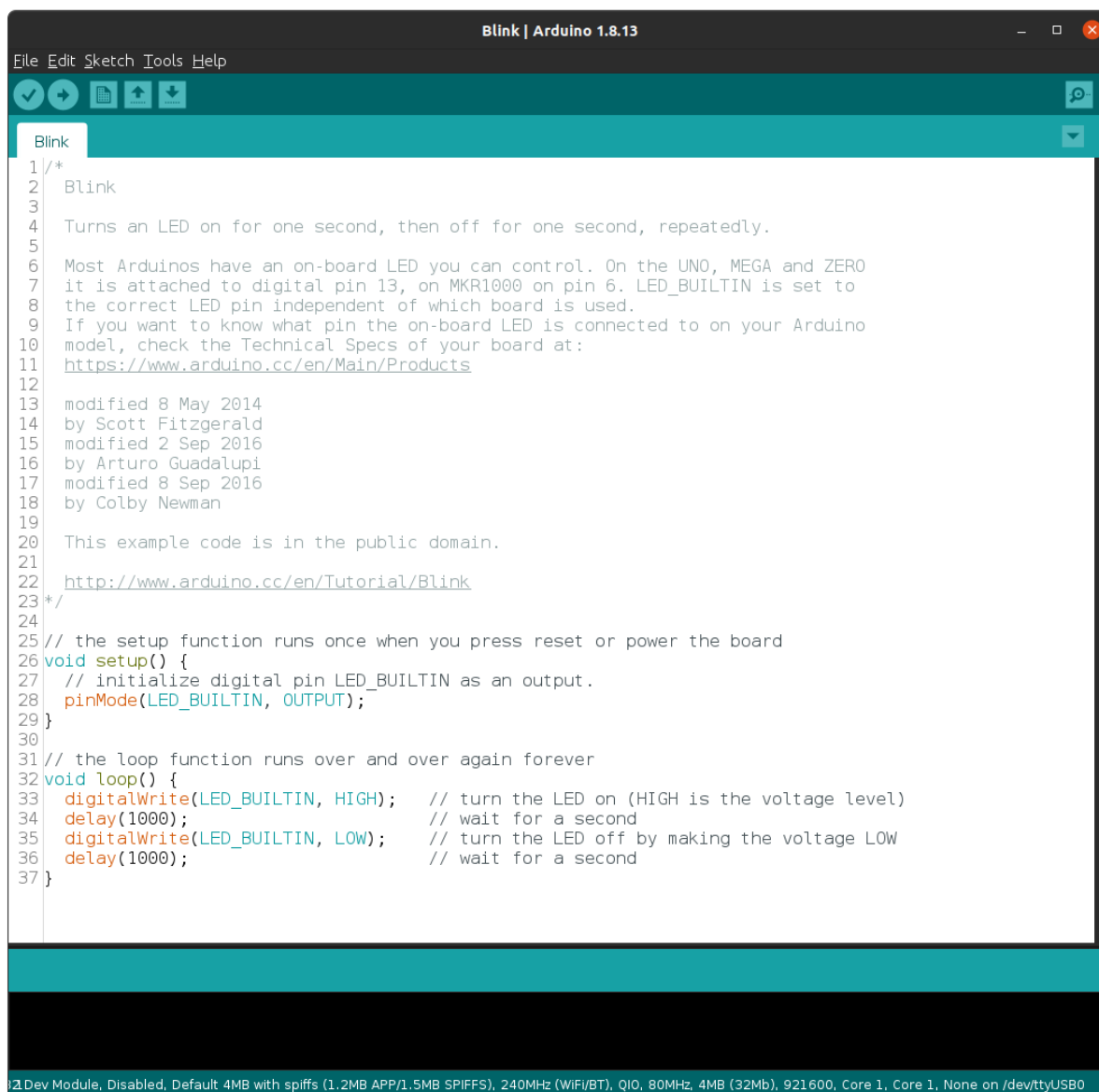
1. Open the Arduino IDE
2. Build and Flash the *blink* project.

4.1.4 Code

Listing 1: Blink.ino

```
/*  
Blink  
  
Turns an LED on for one second, then off for one second, repeatedly.  
  
Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO  
it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to  
the correct LED pin independent of which board is used.  
If you want to know what pin the on-board LED is connected to on your Arduino  
model, check the Technical Specs of your board at:  
https://www.arduino.cc/en/Main/Products  
  
modified 8 May 2014  
by Scott Fitzgerald  
modified 2 Sep 2016
```

(continues on next page)

A screenshot of the Arduino IDE interface. The title bar reads "Blink | Arduino 1.8.13". The menu bar includes "File", "Edit", "Sketch", "Tools", and "Help". Below the menu bar is a toolbar with icons for saving, running, uploading, and downloading. The main text area shows the "Blink" sketch. The sketch includes a multi-line comment explaining the purpose of the code, the hardware it works on, and the license. It also contains the C++ code for the setup and loop functions. The setup function initializes the built-in LED as an output. The loop function turns the LED on for one second and off for one second, repeating this process. The status bar at the bottom shows hardware details: "1 Dev Module, Disabled, Default 4MB with spiiffs (1.2MB APP/1.5MB SPIFFS), 240MHz (WiFi/BT), QIO, 80MHz, 4MB (32Mb), 921600, Core 1, Core 1, None on /dev/ttyUSB0".

```
1/*  
2  Blink  
3  
4  Turns an LED on for one second, then off for one second, repeatedly.  
5  
6  Most Arduinos have an on-board LED you can control. On the UNO, MEGA and ZERO  
7  it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is set to  
8  the correct LED pin independent of which board is used.  
9  If you want to know what pin the on-board LED is connected to on your Arduino  
10 model, check the Technical Specs of your board at:  
11 https://www.arduino.cc/en/Main/Products  
12  
13 modified 8 May 2014  
14 by Scott Fitzgerald  
15 modified 2 Sep 2016  
16 by Arturo Guadalupi  
17 modified 8 Sep 2016  
18 by Colby Newman  
19  
20 This example code is in the public domain.  
21  
22 http://www.arduino.cc/en/Tutorial/Blink  
23*/  
24  
25// the setup function runs once when you press reset or power the board  
26void setup() {  
27  // initialize digital pin LED_BUILTIN as an output.  
28  pinMode(LED_BUILTIN, OUTPUT);  
29}  
30  
31// the loop function runs over and over again forever  
32void loop() {  
33  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)  
34  delay(1000); // wait for a second  
35  digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the voltage LOW  
36  delay(1000); // wait for a second  
37}
```

1 Dev Module, Disabled, Default 4MB with spiiffs (1.2MB APP/1.5MB SPIFFS), 240MHz (WiFi/BT), QIO, 80MHz, 4MB (32Mb), 921600, Core 1, Core 1, None on /dev/ttyUSB0

(continued from previous page)

```
by Arturo Guadalupi
modified 8 Sep 2016
by Colby Newman

This example code is in the public domain.

http://www.arduino.cc/en/Tutorial/Blink
*/

// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);                     // wait for a second
  digitalWrite(LED_BUILTIN, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);                     // wait for a second
}
```

4.1.5 Log Output

If the log output from the serial monitor is relevant, please add here:

```
I (0) cpu_start: App cpu up.
I (418) cpu_start: Pro cpu start user code
I (418) cpu_start: cpu freq: 160000000
```

4.1.6 Resources

- [ESP32 Datasheet](#) (Datasheet)

4.2 Blink Interactive Tutorial

4.2.1 Introduction

This is the interactive blink tutorial using [Wokwi](#). For this tutorial, you don't need the ESP32 board or the Arduino toolchain.

Note: If you don't want to use this tutorial with the simulation, you can copy and paste the [Example Code](#) from [Wokwi](#) editor and use it on the *Arduino IDE*.

4.2.2 About this Tutorial

This tutorial is the most basic for any get started. In this tutorial, we will show how to set a GPIO pin as an output to drive a LED to blink each 1 second.

4.2.3 Step by step

In order to make this simple blink tutorial, you' ll need to do the following steps.

1. Define the GPIO for the LED.

```
#define LED 2
```

This `#define LED 2` will be used to set the GPIO2 as the LED output pin.

2. Setup.

Inside the `setup()` function, we need to add all things we want to run once during the startup. Here we' ll add the `pinMode` function to set the pin as output.

```
void setup() {  
    pinMode(LED, OUTPUT);  
}
```

The first argument is the GPIO number, already defined and the second is the mode, here defined as an output.

3. Main Loop.

After the `setup`, the code runs the `loop` function infinitely. Here we will handle the GPIO in order to get the LED blinking.

```
void loop() {  
    digitalWrite(LED, HIGH);  
    delay(100);  
    digitalWrite(LED, LOW);  
    delay(100);  
}
```

The first function is the `digitalWrite()` with two arguments:

- GPIO: Set the GPIO pin. Here defined by our LED connected to the GPIO2.
- State: Set the GPIO state as HIGH (ON) or LOW (OFF).

This first `digitalWrite` we will set the LED ON.

After the `digitalWrite`, we will set a `delay` function in order to wait for some time, defined in milliseconds.

Now we can set the GPIO to LOW to turn the LED off and `delay` for more few milliseconds to get the LED blinking.

4. Run the code.

To run this code, you' ll need a development board and the Arduino toolchain installed on your computer. If you don' t have both, you can use the simulator to test and edit the code.

4.2.4 Simulation

This simulator is provided by [Wokwi](#) and you can test the blink code and play with some modifications to learn more about this example.

Change the parameters, like the delay period, to test the code right on your browser. You can add more LEDs, change the GPIO, and more.

4.2.5 Example Code

Here is the full blink code.

```
#define LED 2

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, HIGH);
  delay(100);
  digitalWrite(LED, LOW);
  delay(100);
}
```

4.2.6 Resources

- [ESP32 Datasheet](#) (Datasheet)
- [Wokwi](#) (Wokwi Website)

4.3 USB CDC and DFU Flashing

4.3.1 Introduction

Since the ESP32-S2 introduction, Espressif has been working on USB peripheral support for some of the SoC families, including the ESP32-C3 and the ESP32-S3.

This new peripheral allows a lot of new possibilities, including flashing the firmware directly to the SoC without any external USB-to-Serial converter.

In this tutorial, you will be guided on how to use the embedded USB to flash the firmware.

The current list of supported SoCs:

SoC	USB Peripheral Support
ESP32-S2	CDC and DFU
ESP32-C3	CDC only
ESP32-S3	CDC and DFU
ESP32-C6	CDC only
ESP32-H2	CDC only

It's important that your board includes the USB connector attached to the embedded USB from the SoC. If your board doesn't have the USB connector, you can attach an external one to the USB pins.

These instructions will only work on the supported devices with the embedded USB peripheral. This tutorial will not work if you are using an external USB-to-serial converter like FTDI, CP210x, CH340, etc.

For a complete reference to the Arduino IDE tools menu, please see the [Tools Menu](#) reference guide.

4.3.2 USB DFU

The USB DFU (Device Firmware Upgrade) is a class specification from the USB standard that adds the ability to upgrade the device firmware by the USB interface.

Flashing Using DFU

Note: DFU is only supported by the ESP32-S2 and ESP32-S3. See the table of supported SoCs.

To use the USB DFU to flash the device, you will need to configure some settings in the Arduino IDE according to the following steps:

1. Enter into Download Mode manually

This step is done only for the first time you flash the firmware in this mode. To enter into the download mode, you need to press and hold BOOT button and press and release the RESET button.

To check if this procedure was done correctly, now you will see the new USB device listed in the available ports. Select this new device in the **Port** option.

2. Configure the USB DFU

In the next step you can set the USB DFU as default on BOOT and for flashing.

Go to the Tools menu in the Arduino IDE and set the following options:

For ESP32-S2

- USB DFU On Boot -> Enable
- Upload Mode -> Internal USB

For ESP32-S3

- USB Mode -> USB-OTG (TinyUSB)
- USB DFU On Boot -> Enabled

3. Flash Now you can upload your sketch to the device. After flashing, you need to manually reset the device.

Note: On the USB DFU, you can't use the USB for the serial output for the logging, just for flashing. To enable the serial output, use the CDC option instead. If you want to use the USB DFU for just upgrading the firmware using the manual download mode, this will work just fine, however, for developing please consider using USB CDC.

4.3.3 USB CDC

The USB CDC (Communications Device Class) allows you to communicate to the device like in a serial interface. This mode can be used on the supported targets to flash and monitor the device in a similar way on devices that uses the external serial interfaces.

To use the USB CDC, you need to configure your device in the Tools menu:

1. Enter into Download Mode manually

Similar to the DFU mode, you will need to enter into download mode manually. To enter into the download mode, you need to press and hold BOOT button and press and release the RESET button.

To check if this procedure was done correctly, now you will see the new USB device listed in the available ports. Select this new device in the **Port** option.

2. Configure the USB CDC

For ESP32-S2

- USB CDC On Boot -> Enabled
- Upload Mode -> Internal USB

For ESP32-C3

- USB CDC On Boot -> Enabled

For ESP32-S3

- USB CDC On Boot -> Enabled
- Upload Mode -> UART0 / Hardware CDC

3. Flash and Monitor

You can now upload your sketch to the device. After flashing for the first time, you need to manually reset the device.

This procedure enables the flashing and monitoring through the internal USB and does not require you to manually enter into the download mode or to do the manual reset after flashing.

To monitor the device, you need to select the USB port and open the Monitor tool selecting the correct baud rate (usually 115200) according to the `Serial.begin()` defined in your code.

4.3.4 Hardware

If you are developing a custom hardware using the compatible SoC, and want to remove the external USB-to-Serial chip, this feature will completely substitute the needs of the external chip. See the SoC datasheet for more details about this peripheral.

4.4 GPIO Matrix and Pin Mux

4.4.1 Introduction

This is a basic introduction to how the peripherals work in the ESP32. This tutorial can be used to understand how to define the peripheral usage and its corresponding pins.

In some microcontrollers' architecture, the peripherals are attached to specific pins and cannot be redefined to another one.

For example:

The XYZ MCU defines that the I2C peripheral SDA signal is the IO5 on the physical pin 10 and the SCL is on the IO6 and physical pin 11.

This means that, in your hardware project, you **NEED** to use these pins as the I2C and this cannot be changed due to the internal architecture. In this case, you must be very careful during the hardware design to not make any mistake by switching the SDA and SCL connections. Firmware will not help you if you do so.

4.4.2 GPIO Matrix and Pin Mux

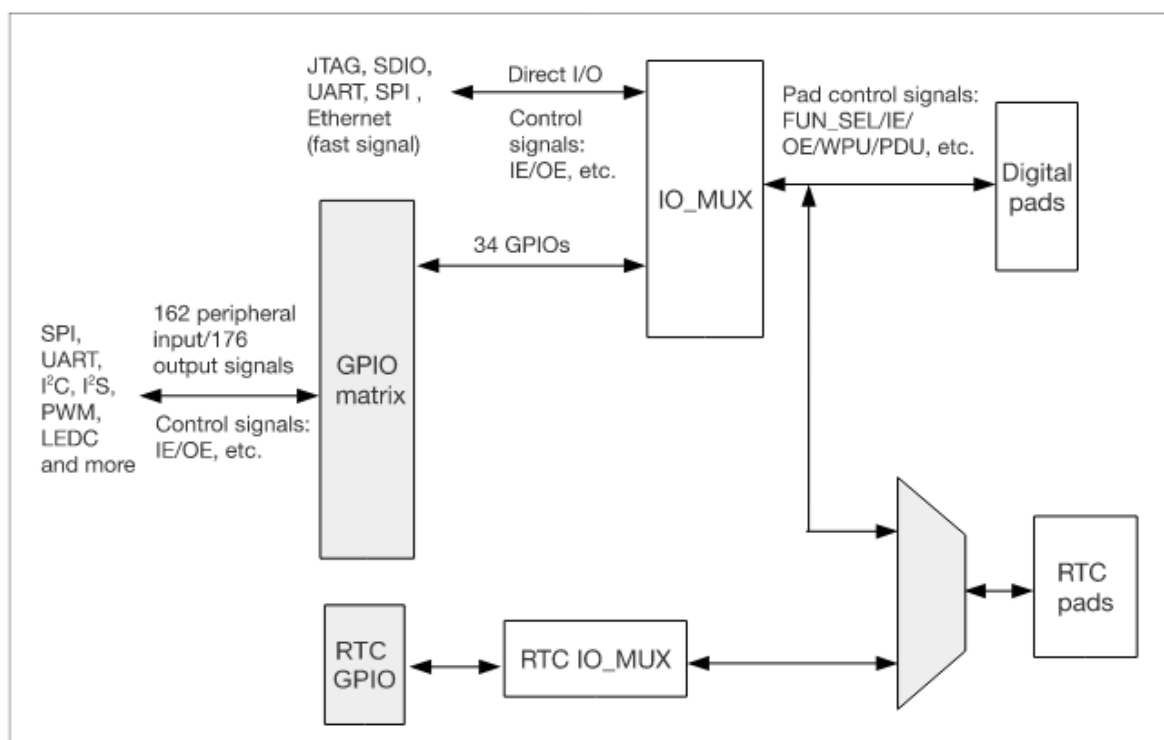
The ESP32 architecture includes the capability of configuring some peripherals to any of the GPIOs pins, managed by the **IO MUX GPIO**. Essentially, this capability means that we can route the internal peripheral into a different physical pin using the IO MUX and the GPIO Matrix.

It means that in the scenario of the XYZ MCU, in the ESP32 we can use any of the GPIOs to route the SDA (input/output) and the SCL (output).

To use this functionality, we must be aware of some precautions:

- Some of the GPIOs are **INPUT** only.
- Some peripherals have output signals and must be used on GPIOs capable to be configured as **OUTPUT**.
- Some peripherals, mostly the high speed ones, ADC, DAC, Touch, and JTAG use dedicated GPIOs pins.
- Some pins are used to connect flash memory on the module - this prevents them from any other use - if a peripheral is routed to one of these pins the device will not be able to boot.

Warning: Before assigning the peripheral pins in your design, double check if the pins you're using are appropriate. The input-only pins cannot be used for peripherals that require output or input/output signals.



The greatest advantage of this functionality is the fact that we don't need to be fully dependent on the physical pin, since we can change according to our needs. This can facilitate the hardware design routing or in some cases, fix some pin swap mistake during the hardware design phase.

4.4.3 Peripherals

Here is the basic peripherals list present on the [ESP32](#). The peripheral list may vary from each ESP32 SoC family. To see all peripherals available on the [ESP32-S2](#) and [ESP32-C3](#), check each of the datasheets.

Peripheral Table

Type	Function
ADC	Dedicated GPIOs
DAC	Dedicated GPIOs
Touch Sensor	Dedicated GPIOs
JTAG	Dedicated GPIOs
SD/SDIO/MMC HostController	Dedicated GPIOs
Motor PWM	Any GPIO
SDIO/SPI SlaveController	Dedicated GPIOs
UART	Any GPIO[1]
I2C	Any GPIO
I2S	Any GPIO
LED PWM	Any GPIO
RMT	Any GPIO
GPIO	Any GPIO
Parallel QSPI	Dedicated GPIOs
EMAC	Dedicated GPIOs
Pulse Counter	Any GPIO
TWAI	Any GPIO
USB	Dedicated GPIOs

[1] except for the download/programming mode decided by the bootloader.

This table is present on each datasheet provided by Espressif.

4.4.4 Usage Examples

In the Arduino Uno, we have the I2C pins defined by hardware, A4 is the SDA and A5 the SCL. In this case, we do not need to set these pins in the `Wire.begin()` ; function, because they are already into the Wire library.

```
void setup()
{
    Wire.begin(); // join i2c bus (address optional for master)
}
```

Now, for the ESP32, the default pins for the I2C are SDA (GPIO21) and SCL (GPIO22). We can use a different pin as alternative for the default ones if you need to change the pins. To change the pins, we must call the `Wire.setPins(int sda, int scl)` ; function before calling `Wire.begin()` ;.

```
int sda_pin = 16; // GPIO16 as I2C SDA
int scl_pin = 17; // GPIO17 as I2C SCL

void setup()
{
    Wire.setPins(sda_pin, scl_pin); // Set the I2C pins before begin
    Wire.begin(); // join i2c bus (address optional for master)
}
```

A similar approach also applies for the other peripherals.

4.4.5 Datasheet

- [ESP32](#) (Datasheet)
- [ESP32-C2](#) (Datasheet)
- [ESP32-C3](#) (Datasheet)

- [ESP32-C5](#) (Datasheet)
- [ESP32-C6](#) (Datasheet)
- [ESP32-C61](#) (Datasheet)
- [ESP32-H2](#) (Datasheet)
- [ESP32-P4](#) (Datasheet)
- [ESP32-S2](#) (Datasheet)
- [ESP32-S3](#) (Datasheet)

4.4.6 Resources

4.5 Partition Table

4.5.1 Introduction

Partition table is used to define the flash memory organization and the different kind of data will be stored on each partition.

You can use one of the available partition table scheme or create your own. You can see all the different schemes on the [tools/partitions](#) folder or by the Arduino IDE tools menu *Tools -> Partition Scheme*.

The partition table is created by a .CSV (Comma-separated Values) file with the following structure:

```
# ESP-IDF Partition Table
# Name, Type, SubType, Offset, Size, Flags
```

Where:

1. **Name**

Is the partition name and must be a unique name. This name is not relevant for the system and the size must be at maximum of 16-chars (no special chars).

2. **Type**

This is the type of the partition. This value can be `data` or `app`.

- `app` type is used to define the partition that will store the application.
- `data` type can be used to define the partition that stores general data, not the application.

3. **SubType**

The SubType defines the usage of the `app` and `data` partitions.

data

`ota`

The `ota` subtype is used to store the OTA information. This partition is used only when the OTA is used to select the initialization partition, otherwise no need to add it to your custom partition table. The size of this partition should be a fixed size of 8 kB (0x2000 bytes).

`nvs`

The `nvs` partition subtype is used to define the partition to store general data, like the Wi-Fi data, device PHY calibration data and any other data to be stored on the non-volatile memory. This kind of partition is suitable for small custom configuration data, cloud certificates, etc. Another usage for the NVS is to store sensitive data, since the NVS supports encryption. It is highly recommended to add at least one `nvs` partition, labeled with the name `nvs`, in your custom partition tables with size of at least 12 kB (0x3000 bytes). If needed, you can increase the size of the `nvs` partition. The recommended size for this partition is from 12 kB to 64 kB. Although larger NVS partitions can be defined, we recommend using FAT or SPIFFS filesystem for storage of larger amounts of data.

`coredump`

The `coredump` partition subtype is used to store the core dump on the flash. The core dump is used to analyze critical errors like crash and panic. This function must be enabled in the project configuration menu and set the data destination to flash. The recommended size for this partition is 64 kB (0x10000).

`nvs_keys`

The `nvs_keys` partition subtype is used to store the keys when the NVS encryption is used. The size for this partition is 4 kB (0x1000).

`fat`

The `fat` partition subtype defines the FAT filesystem usage, and it is suitable for larger data and if this data is often updated and changed. The FAT FS can be used with wear leveling feature to increase the erase/modification cycles per memory sector and encryption for sensitive data storage, like cloud certificates or any other data that may be protected. To use FAT FS with wear leveling see the example.

`spiffs`

The `spiffs` partition subtype defines the SPI flash filesystem usage, and it is also suitable for larger files and it also performs the wear leveling and file system consistency check. The SPIFFS do not support flash encryption.

app

`factory`

The `factory` partition subtype is the default application. The bootloader will set this partition as the default application initialization if no OTA partition is found, or the OTA partitions are empty. If the OTA partition is used, the `ota_0` can be used as the default application and the `factory` can be removed from the partition table to save memory space.

`ota_0` to `ota_15`

The `ota_x` partition subtype is used for the Over-the air update. The OTA feature requires at least two `ota_x` partition (usually `ota_0` and `ota_1`) and it also requires the `ota` partition to keep the OTA information data. Up to 16 OTA partitions can be defined but only two are needed for basic OTA feature.

`test`

The `test` partition subtype is used for factory test procedures.

4. Offset

The offset defines the partition start address. The offset is defined by the sum of the offset and the size of the earlier partition.

Note: Offset must be multiple of 4 kB (0x1000) and for app partitions it must be aligned by 64 kB (0x10000). If left blank, the offset will be automatically calculated based on the end of the previous partition, including any necessary alignment, however, the offset for the first partition must be always set as **0x9000** and for the first application partition **0x10000**.

5. Size

Size defines the amount of memory to be allocated on the partition. The size can be formatted as decimal, hex numbers (0x prefix), or using unit prefix K (kilo) or M (mega) i.e: 4096 = 4K = 0x1000.

6. Flags

The last column in the CSV file is the flags and it is currently used to define if the partition will be encrypted by the flash encryption feature.

For example, **the most common partition** is the `default_8MB.csv` (see [tools/partitions](#) folder for some examples):

```
# Name, Type, SubType, Offset, Size, Flags
nvs, data, nvs, 0x9000, 0x5000,
otadata, data, ota, 0xe000, 0x2000,
app0, app, ota_0, 0x10000, 0x330000,
app1, app, ota_1, 0x340000, 0x330000,
spiffs, data, spiffs, 0x670000, 0x190000,
```

4.5.2 Using a Custom Partition Scheme

To create your own partition table, you can create the `partitions.csv` file **in the same folder you created your sketch**. The build system will automatically pick the partition table file and use it instead of the predefined ones.

Here is an example you can use for a custom partition table:

```
# Name,      Type, SubType, Offset,  Size, Flags
nvs,         data, nvs,      36K,    20K,
otadata,     data, ota,       56K,    8K,
app0,        app,  ota_0,     64K,    2M,
app1,        app,  ota_1,      ,    2M,
spiffs,      data, spiffs,    ,    8M,
```

This partition will use about 12 MB of the 16 MB flash. The offset will be automatically calculated after the first application partition and the units are in K and M.

An alternative is to create the new partition table as a new file in the [tools/partitions](#) folder and edit the [boards.txt](#) file to add your custom partition table.

Another alternative is to create the new partition table as a new file, and place it in the [variants](#) folder under your boards folder, and edit the [boards.txt](#) file to add your custom partition table, noting that in order for the compiler to find your custom partition table file you must use the `‘.build.custom_partitions=’` option in the boards.txt file, rather than the standard `‘.build.partitions=’` option. The `‘.build.variant=’` option has the name of the folder holding your custom partition table in the variants folder.

An example of the PartitionScheme listing using the ESP32-S3 Dev Module as a reference, would be to have the following:

Custom Partition - CSV file in /variants/custom_esp32s3/ folder

```
esp32s3.build.variant=custom_esp32s3
--
esp32s3.menu.PartitionScheme.huge_app=Custom Huge APP (3MB No OTA/1MB SPIFFS)
esp32s3.menu.PartitionScheme.huge_app.build.custom_partitions=custom_huge_app
esp32s3.menu.PartitionScheme.huge_app.upload.maximum_size=3145728
```

4.5.3 Examples

2 MB no OTA

```
# Name,      Type, SubType, Offset,  Size, Flags
nvs,         data, nvs,      36K,    20K,
factory,     app,  factory,  64K,   1900K,
```

4 MB no OTA

```
# Name,      Type, SubType, Offset,  Size, Flags
nvs,         data, nvs,      36K,    20K,
factory,     app,  factory,  64K,   4000K,
```

4 MB with OTA

```
# Name,      Type, SubType, Offset,  Size, Flags
nvs,         data, nvs,      36K,    20K,
otadata,     data, ota,       56K,    8K,
app0,        app,  ota_0,     64K,   1900K,
app1,        app,  ota_1,      ,   1900K,
```

8 MB no OTA with Storage

```
# Name,      Type, SubType, Offset,  Size, Flags
nvs,         data, nvs,      36K,    20K,
factory,     app,  factory,  64K,    2M,
spiffs,      data, spiffs,    ,    5M,
```

8 MB with OTA and Storage

#	Name,	Type,	SubType,	Offset,	Size,	Flags
nvs,		data,	nvs,	36K,	20K,	
otadata,		data,	ota,	56K,	8K,	
app0,		app,	ota_0,	64K,	2M,	
app1,		app,	ota_1,	,	2M,	
spiffs,		data,	spiffs,	,	3M,	

4.5.4 Reference

This documentation was based on the [How to use custom partition tables on ESP32](#) article.

4.6 Preferences

4.6.1 Introduction

The Preferences library is unique to arduino-esp32. It should be considered as the replacement for the Arduino EEPROM library.

It uses a portion of the on-board non-volatile memory (NVS) of the ESP32 to store data. This data is retained across restarts and loss of power events to the system.

Preferences works best for storing many small values, rather than a few large values. If you need to store large amounts of data, consider using a file system library such as LittleFS.

The Preferences library is usable by all ESP32 variants.

4.6.2 Preferences Attributes

Preferences data is stored in NVS in sections called a “namespace” . Within each namespace are a set of key-value pairs. The “key” is the name of the data item and the “value” is, well, the value of that piece of data. Kind of like variables. The key is the name of the variable and the value is its value. Like variables, a key-value pair has a data type.

Multiple namespaces are permitted within NVS. The name of each namespace must be unique. The keys within that namespace are unique to that namespace. Meaning the same key name can be used in multiple namespaces without conflict.

Namespace and key names are case sensitive.

Each key name must be unique within a namespace.

Namespace and key names are character strings and are limited to a maximum of 15 characters.

Only one namespace can be open (in use) at a time.

4.6.3 Library Overview

Library methods are provided to:

- create a namespace;
- open and close a namespace;
- store and retrieve data within a namespace for supported data types;
- determine if a key value has been initialized;
- delete a key-value pair;
- delete all key-value pairs in a namespace;
- determine data types stored against a key;
- determine the number of key entries available in the namespace.

Preferences directly supports the following data types:

Table 1: **Table 1 —Preferences Types**

Preferences Type	Data Type	Size (bytes)
Bool	bool	1
Char	int8_t	1
UChar	uint8_t	1
Short	int16_t	2
UShort	uint16_t	2
Int	int32_t	4
UInt	uint32_t	4
Long	int32_t	4
ULong	uint32_t	4
Float	float_t	4
Long64	int64_t	8
ULong64	uint64_t	8
Double	double_t	8
String	const char*	variable
	String	
Bytes	uint8_t	variable

String values can be stored and retrieved either as an Arduino String or as a null terminated `char` array (C-string).

Bytes type is used for storing and retrieving an arbitrary number of bytes in a namespace.

4.6.4 Workflow

Preferences workflow, once everything is initialized, is pretty simple.

To store a value:

- Open the namespace in read-write mode.
- Put the value into the key.
- Close the namespace.

To retrieve a value:

- Open the namespace in read-only mode.
- Use the key to get the value.
- Close the namespace.

(Technically, you can retrieve a value if the namespace is open in either read-only or read-write mode but it's good practice to open the namespace in read-only mode if you are only retrieving values.)

When storing information, a “`put [PreferencesType]`” method referenced to its key is used.

When retrieving information a “`get [PreferencesType]`” method referenced to its key is used.

Ensuring that the data types of your “`get`’s” and “`put`’s” all match, you’re good to go.

The nuance is in initializing everything at the start.

Before you can store or retrieve anything using Preferences, both the namespace and the key within that namespace need to exist. So the workflow is:

1. Create or open the namespace.
2. Test for the existence of a key that should exist if the namespace has been initialized.
3. If that key does not exist, create the key(s).
4. Carry on with the rest of your sketch where data can now be stored and retrieved from the namespace.

Each step is discussed below.

Note: From here on when referring in general to a method used to store or retrieve data we'll use the shorthand “putX” and “getX” where the “X” is understood to be a Preferences Type; Bool, UInt, Char, and so on from the Preferences Types table above.

Create or Open the Namespace

In your sketch, first insert a declaration of a Preferences object by including a line like;

```
Preferences mySketchPrefs;    // "mySketchPrefs" is the name of the Preferences_
↪object.                      // Can be whatever you want.
```

This object is used with the Preferences methods to access the namespace and the key-value pairs it contains.

A namespace is made available for use with the `.begin` method:

```
mySketchPrefs.begin("myPrefs", false)
```

If the namespace does not yet exist, this will create and then open the namespace `myPrefs`.

If the namespace already exists, this will open the namespace `myPrefs`.

If the second argument is `false` the namespace is opened in read-write (RW) mode —values can be stored in to and retrieved from the namespace. If it is `true` the namespace is opened in read-only (RO) mode —values can be retrieved from the namespace but nothing can be stored.

Test for Initial Existence of Your Key(s)

When the ESP32 boots, there is no inherent way to know if this is the very first time it has ever powered on or if it is a subsequent launch and it has run its sketch before. We can use Preferences to store information that is retained across reboots that we can read, and based on that, decide if this is a first-time run and take the required actions if so.

We do this by testing for the existence of a certain key within a namespace. If that key exists, it is safe to assume the key was created during the first-time run of the sketch and so the namespace has already been initialized.

To determine if a key exists, use:

```
isKey("myTestKey")
```

This returns `true` if `"myTestKey"` exists in the namespace, and `false` if it does not.

By example, consider this code segment:

```
Preferences mySketchPrefs;
String doesExist;

mySketchPrefs.begin("myPrefs", false);    // open (or create and then open if it_
↪does not                                // yet exist) the namespace "myPrefs" in_
↪RW mode.

bool doesExist = mySketchPrefs.isKey("myTestKey");

if (doesExist == false) {
    /*
       If doesExist is false, we will need to create our
       namespace key(s) and store a value into them.
    */
}
```

(continues on next page)

(continued from previous page)

```
// Insert your "first time run" code to create your keys & assign their values.  
↳ below here.  
}  
else {  
    /*  
        If doesExist is true, the key(s) we need have been created before  
        and so we can access their values as needed during startup.  
    */  
  
    // Insert your "we've been here before" startup code below here.  
}
```

Creating Namespace Keys and Storing Values

To create a key, we use one of the `.putX` methods, matching "X" to the Preferences Type of the data we wish to store:

```
myPreferences.putX("myKeyName", value)
```

If "myKeyName" does not exist in the namespace, it is first created and then `value` is stored against that keyname. The namespace must be open in RW mode to do this. Note that `value` is not optional and must be provided with every `".putX"` statement. Thus every key within a namespace will always hold a valid value.

An example is:

```
myPreferences.putFloat("pi", 3.14159265359);    // stores an float_t data type  
                                              // against the key "pi".
```

Reading Values From a Namespace

Once a key exists in a namespace and the namespace is open, its value is retrieved using one of the `getX` methods, matching "X" to the type of data stored against that key.

```
myPreferences.getX("myKeyName")
```

Like so:

```
float_t myFloat = myPreferences.getFloat("pi");
```

This will retrieve the `float_t` value from the namespace key "pi" and assign it to the `float_t` type variable `myFloat`.

Summary

So the basics of using Preferences are:

1. You cannot store into or retrieve from a key-value pair until a namespace is created and opened and the key exists in that namespace.
2. If the key already exists, it was created the first time the sketch was run.
3. A key value can be retrieved regardless of the mode in which the namespace was opened, but a value can only be stored if the namespace is open in read-write mode.
4. Data types of the "get" s and "put" s must match.
5. Remember the 15 character limit for namespace and key names.

4.6.5 Real World Example

Here is part of a `setup()` function that uses Preferences.

Its purpose is to set either a factory default configuration if the system has never run before, or use the last configuration if it has.

When started, the system has no way of knowing which of the above conditions is true. So the first thing it does after opening the namespace is check for the existence of a key that we have predetermined can only exist if we have previously run the sketch. Based on its existence we decide if a factory default set of operating parameters should be used (and in so doing create the namespace keys and populate the values with defaults) or if we should use operating parameters from the last time the system was running.

```
#include <Preferences.h>

#define RW_MODE false
#define RO_MODE true

Preferences stcPrefs;

void setup() {

    // not the complete setup(), but in setup(), include this...

    stcPrefs.begin("STCPrefs", RO_MODE);           // Open our namespace (or create_
↪ it                                           // if it doesn't exist) in RO_
↪ mode.

    bool tpInit = stcPrefs.isKey("nvsInit");       // Test for the existence
↪ key.                                       // of the "already initialized"

    if (tpInit == false) {
        // If tpInit is 'false', the key "nvsInit" does not yet exist therefore this
        // must be our first-time run. We need to set up our Preferences namespace_
↪ keys. So...
        stcPrefs.end();                       // close the namespace in RO_
↪ mode and...
        stcPrefs.begin("STCPrefs", RW_MODE);    // reopen it in RW mode.

        // The .begin() method created the "STCPrefs" namespace and since this is our
        // first-time run we will create
        // our keys and store the initial "factory default" values.
        stcPrefs.putUChar("curBright", 10);
        stcPrefs.putString("talChan", "one");
        stcPrefs.putLong("talMax", -220226);
        stcPrefs.putBool("ctMde", true);

        stcPrefs.putBool("nvsInit", true);       // Create the "already_
↪ initialized"                               // key and store a value.

        // The "factory defaults" are created and stored so...
        stcPrefs.end();                       // Close the namespace in RW_
↪ mode and...
        stcPrefs.begin("STCPrefs", RO_MODE);    // reopen it in RO mode so the_
↪ setup code                               // outside this first-time run
↪ 'if' block                               // can retrieve the run-time_
↪ values
```

(continues on next page)

(continued from previous page)

```

// from the "STCPrefs"
↪namespace.
    }

    // Retrieve the operational parameters from the namespace
    // and save them into their run-time variables.
    currentBrightness = stcPrefs.getUChar("curBright"); //
    tChannel = stcPrefs.getString("talChan");           // The LHS variables were
↪defined
    tChanMax = stcPrefs.getLong("talMax");               // earlier in the sketch.
    ctMode = stcPrefs.getBool("ctMde");                 //

    // All done. Last run state (or the factory default) is now restored.
    stcPrefs.end();                                     // Close our preferences
↪namespace.

    // Carry on with the rest of your setup code...

    // When the sketch is running, it updates any changes to an operational
↪parameter
    // to the appropriate key-value pair in the namespace.
}

```

4.6.6 Utility Functions

There are a few other functions useful when working with namespaces.

Deleting key-value Pairs

```
preferences.clear();
```

- Deletes *all* the key-value pairs in the currently opened namespace.
 - The namespace still exists.
 - The namespace must be open in read-write mode for this to work.

```
preferences.remove("keyname");
```

- Deletes the “keyname” and value associated with it from the currently opened namespace.
 - The namespace must be open in read-write mode for this to work.
 - Tip: use this to remove the “test key” to force a “factory reset” during the next reboot (see the *Real World Example* above).

If either of the above are used, the key-value pair will need to be recreated before using it again.

Determining the Number of Available Keys

For each namespace, Preferences keeps track of the keys in a key table. There must be an open entry in the table before a key can be created. This method will return the number of entries available in the table.

```
freeEntries()
```

To send to the serial monitor the number of available entries the following could be used.

```
Preferences mySketchPrefs;
```

(continues on next page)

(continued from previous page)

```
mySketchPrefs.begin("myPrefs", true);
size_t whatsLeft = freeEntries(); // this method works regardless of the mode
↳ in which the namespace is opened.
Serial.printf("There are: %lu entries available in the namespace table.\n",
↳ (unsigned long) whatsLeft);
mySketchPrefs.end();
```

The number of available entries in the key table changes depending on the number of keys in the namespace and also the dynamic size of certain types of data stored in the namespace. Details are in the [Preferences API Reference](#).

Do note that the number of entries in the key table does not guarantee that there is room in the opened NVS namespace for all the data to be stored in that namespace. Refer to the Espressif [Non-volatile storage library](#) documentation for full details.

Determining the Type of a key-value Pair

Keeping track of the data types stored against a key-value pair is one of the bookkeeping tasks left to you. Should you want to discover the Preferences data type stored against a given key, use this method:

```
getType("myKey")
```

As in:

```
PreferenceType whatType = getType("myKey");
```

The value returned is a `PreferenceType` value that maps to a Preferences Type. Refer to the description in the [Preferences API Reference](#) for details.

4.6.7 Working with Large Data

Recall that the Preferences library works best for storing many small values, rather than a few large values. Regardless, it may be desirable to store larger amounts of arbitrary data than what is provided by the basic types in the Preferences Types table above.

The library provides the following methods to facilitate this.

```
putBytes("myBytesKey", value, valueLen)
getBytes("myBytesKey", buffer, valueLen)
getBytesLength("myBytesKey")
```

The `put` and `get Bytes` methods store and retrieve the data. The `getBytesLength` method is used to find the size of the data stored against the key (which is needed to retrieve Bytes data).

As the names of the methods imply, they operate on variable length bytes of data (often referred to as a “blob”) and not on individual elements of a certain data type.

Meaning if you store for example an array of type `int16_t` against a Bytes type key, the value of that key becomes a series of bytes with no associated data type. Or if you like, all data stored as a blob gets converted to a series of `uint8_t` type bytes.

As a result, when using the `getBytes` method to retrieve the value of the key, what is returned to the buffer is a series of `uint8_t` bytes. It is up to you to manage the data types and size of the arrays and buffers when retrieving Bytes data.

Fortunately this is not as difficult as it may sound as the `getBytesLength` method and the `sizeof` operator help with keeping track of it all.

This is best explained with an example. Here the Bytes methods are used to store and retrieve an array, while ensuring the data type is preserved.


```

/*
 * An example sketch using the Preferences "Bytes" methods
 * to store and retrieve an arbitrary number of bytes in
 * a namespace.
 */

#include <Preferences.h>

#define RO_MODE true
#define RW_MODE false

void setup() {

    Preferences mySketchPrefs;

    Serial.begin(115200);
    delay(250);

    mySketchPrefs.begin("myPrefs", RW_MODE);    // open (or create) the namespace
                                                // "myPrefs" in RW mode
    mySketchPrefs.clear();                      // delete any previous keys in this_
↪ namespace

    // Create an array of test values. We're using hex numbers
    // throughout to better show how the bytes move around.
    int16_t myArray[] = { 0x1112, 0x2122, 0x3132, 0x4142, 0x5152, 0x6162, 0x7172 };

    Serial.println("Printing myArray...");
    for (int i = 0; i < sizeof(myArray) / sizeof(int16_t); i++) {
        Serial.print(myArray[i], HEX); Serial.print(", ");
    }
    Serial.println("\r\n");

    // In the next statement, the second sizeof() needs
    // to match the data type of the elements of myArray
    Serial.print("The number of elements in myArray is: ");
    Serial.println( sizeof(myArray) / sizeof(int16_t) );
    Serial.print("But the size of myArray in bytes is: ");
    Serial.println( sizeof(myArray) );
    Serial.println("");

    Serial.println(
        "Storing myArray into the Preferences namespace \"myPrefs\" against the key \"
↪ myPrefsBytes\".");
    // Note: in the next statement, to store the entire array, we must use the
    // size of the array in bytes, not the number of elements in the array.
    mySketchPrefs.putBytes( "myPrefsBytes", myArray, sizeof(myArray) );
    Serial.print("The size of \"myPrefsBytes\" is (in bytes): ");
    Serial.println( mySketchPrefs.getBytesLength("myPrefsBytes") );
    Serial.println("");

    int16_t myIntBuffer[20] = {}; // No magic about 20. Just making a buffer_
↪ (array) big enough.
    Serial.println("Retrieving the value of myPrefsBytes into myIntBuffer.");
    Serial.println("    - Note the data type of myIntBuffer matches that of myArray
↪ ");
    mySketchPrefs.getBytes("myPrefsBytes", myIntBuffer,
                           mySketchPrefs.getBytesLength("myPrefsBytes"));

    Serial.println("Printing myIntBuffer...");
    // In the next statement, sizeof() needs to match the data type of the_
↪ elements of myArray

```

(continues on next page)

(continued from previous page)

```

    for (int i = 0; i < mySketchPrefs.getBytesLength("myPrefsBytes") /
    ↪ sizeof(int16_t); i++) {
        Serial.print(myIntBuffer[i], HEX); Serial.print(", ");
    }
    Serial.println("\r\n");

    Serial.println(
        "We can see how the data from myArray is actually stored in the namespace as
    ↪ follows.");
    uint8_t myByteBuffer[40] = {}; // No magic about 40. Just making a buffer
    ↪ (array) big enough.
    mySketchPrefs.getBytes("myPrefsBytes", myByteBuffer,
                           mySketchPrefs.getBytesLength("myPrefsBytes"));

    Serial.println("Printing myByteBuffer...");
    for (int i = 0; i < mySketchPrefs.getBytesLength("myPrefsBytes"); i++) {
        Serial.print(myByteBuffer[i], HEX); Serial.print(", ");
    }
    Serial.println("");
}

void loop() {
    ;
}

```

The resulting output is:

```

Printing myArray...
1112, 2122, 3132, 4142, 5152, 6162, 7172,

The number of elements in myArray is: 7
But the size of myArray in bytes is: 14

Storing myArray into the Preferences namespace "myPrefs" against the key
↪ "myPrefsBytes".
The size of "myPrefsBytes" is (in bytes): 14

Retrieving the value of myPrefsBytes into myIntBuffer.
- Note the data type of myIntBuffer matches that of myArray
Printing myIntBuffer...
1112, 2122, 3132, 4142, 5152, 6162, 7172,

We can see how the data from myArray is actually stored in the namespace as
↪ follows.
Printing myByteBuffer...
12, 11, 22, 21, 32, 31, 42, 41, 52, 51, 62, 61, 72, 71,

```

You can copy the sketch and change the data type and values in `myArray` and follow along with the code and output to see how the Bytes methods work. The data type of `myIntBuffer` should be changed to match that of `myArray` (and check the “`sizeof()`”s where indicated in the comments).

The main takeaway is to remember you’re working with bytes and so attention needs to be paid to store all the data based on the size of its type and to manage the buffer size and data type for the value retrieved.

4.6.8 Multiple Namespaces

As stated earlier, multiple namespaces can exist in the Preferences NVS partition. However, only one namespace at a time can be open (in use).

If you need to access a different namespace, close the one before opening the other. For example:

```

Preferences currentNamespace;

currentNamespace.begin("myNamespace", false);
    // do stuff...

currentNamespace.end();                                // closes 'myNamespace'

currentNamespace.begin("myOtherNamespace", false);    // opens a different
↳ Preferences namespace.                               // do other stuff...

currentNamespace.end();                                // closes 'myOtherNamespace'
↳ '

```

Here the “currentNamespace” object is reused, but different Preferences objects can be declared and used. Just remember to keep it all straight as all “putX’s” and “getX’s”, etc. will only operate on the single currently opened namespace.

4.6.9 A Closer Look at getX

Methods in the Preferences library return a status code that can be used to determine if the method completed successfully. This is described in the [Preferences API Reference](#).

Assume we have a key named `favorites` that contains a value of a `String` data type.

After executing the statement:

```
dessert = mySketchPrefs.getString("favorites");
```

the variable `dessert` will contain the value of the string stored against the key “favorites”.

But what if something went wrong and the `getString` call failed to retrieve the key value? How would we be able to detect the error?

With Preferences, the `getX` methods listed in Table 2 below will return a default value if an error is encountered.

Table 2: Table 2 —getX Methods Defaults

Preferences Type	Default Return Value
Char, UChar, Short, UShort, Int, UInt, Long, ULong, Long64, ULong64	0
Bool	false
Float	NAN
Double	
String (String)	""
String (* buf)	\0

Thus to detect an error we could compare the value returned against its default return value and if they are equal assume an error occurred and take the appropriate action.

But what if a method default return value is also a potential legitimate value? How can we then know if an error occurred?

As it turns out, the complete form of the `getX` methods for each of the Preferences Types in Table 2 is:

```
preferences.getX("myKey", myDefault)
```

In this form the method will return either the value associated with “myKey” or, if an error occurred, return the value myDefault, where myDefault must be the same data type as the getX.

Returning to the example above:

```
dessert = mySketchPrefs.getString("favorites", "gravel");
```

will assign to the variable dessert the String gravel if an error occurred, or the value stored against the key favorites if not.

If we predetermine a default value that is outside all legitimate values, we now have a way to test if an error actually occurred.

In summary, if you need to confirm that a value was retrieved without error from a namespace, use the complete form of the getX method with a predetermined default “this can only happen if an error” value and compare that against the value returned by the call. Otherwise, you can omit the default value as the call will return the default for that particular getX method.

Additional detail is given in the [Preferences API Reference](#).

4.6.10 Advanced Item

In the arduino-esp32 implementation of Preferences there is no method to completely remove a namespace. As a result, over the course of a number of projects, it is possible that the ESP32 NVS Preferences partition becomes cluttered or full.

To completely erase and reformat the NVS memory used by Preferences, create and run a sketch that contains:

```
#include <nvs_flash.h>

void setup() {

    nvs_flash_erase();           // erase the NVS partition and...
    nvs_flash_init();           // initialize the NVS partition.
    while (true);

}

void loop() {
    ;
}
```

Warning: You should download a new sketch to your board immediately after running the above or else it will reformat the NVS partition every time it is powered up or restarted!

4.6.11 Resources

- [Preferences API Reference](#)
- [Non-volatile storage library](#) (espressif-IDF API Reference)
- [Official ESP-IDF documentation](#) (espressif-IDF Reference)

4.6.12 Contribute

To contribute to this project, see [How to contribute](#).

If you have any **feedback** or **issue** to report on this tutorial, please open an issue or fix it by creating a new PR. Contributions are more than welcome!

Before creating a new issue, be sure to try the Troubleshooting and to check if the same issue was already created by someone else.

Chapter 5

Advanced Utilities

5.1 Library Builder

5.1.1 About

Espressif provides a macOS and Linux [tool](#) to simplify building your own compiled libraries for use in Arduino IDE (or your favorite IDE).

This tool can be used to change the project or a specific configuration according to your needs.

5.1.2 Installing

To install the Library Builder into your environment, please, follow the instructions below.

- Clone the ESP32 Arduino lib builder:

```
git clone https://github.com/espressif/esp32-arduino-lib-builder
```

- Go to the esp32-arduino-lib-builder folder:

```
cd esp32-arduino-lib-builder
```

- Build:

```
./build.sh
```

If everything works, you may see the following message: Successfully created esp32 image.

Dependencies

To build the library you will need to install some dependencies. Maybe you already have installed it, but it is a good idea to check before building.

- Install all dependencies (**Ubuntu**):

```
sudo apt-get install git wget curl libssl-dev libncurses-dev flex bison gperf.  
↪ cmake ninja-build ccache jq
```

- Install Python and upgrade pip:

```
sudo apt-get install python3  
sudo pip install --upgrade pip
```

- Install all required packages:

```
pip install --user setuptools pyserial click cryptography future pyparsing_
↳pyelftools
```

5.1.3 Building

If you have all the dependencies met, it is time to build the libraries.

To build using the default configuration:

```
./build.sh
```

Custom Build

There are some options to help you create custom libraries. You can use the following options:

Usage

```
build.sh [-s] [-A arduino_branch] [-I idf_branch] [-i idf_commit] [-c path] [-t
↳<target>] [-b <build|menuconfig|idf_libs|copy_bootloader|mem_variant>] [config ..
↳.]
```

Skip Install/Update Skip installing/updating of ESP-IDF and all components

```
./build.sh -s
```

This option can be used if you already have the ESP-IDF and all components already in your environment.

Set Arduino-ESP32 Branch Set which branch of arduino-esp32 to be used for compilation

```
./build.sh -A <arduino_branch>
```

Set ESP-IDF Branch Set which branch of ESP-IDF is to be used for compilation

```
./build.sh -I <idf_branch>
```

Set the ESP-IDF Commit Set which commit of ESP-IDF to be used for compilation

```
./build.sh -i <idf_commit>
```

Deploy Deploy the build to github arduino-esp32

```
./build.sh -d
```

Set the Arduino-ESP32 Destination Folder Set the arduino-esp32 folder to copy the result to. ex. '\$HOME/Arduino/hardware/espressif/esp32'

```
./build.sh -c <path>
```

This function is used to copy the compiled libraries to the Arduino folder.

Set the Target Set the build target(chip). ex. 'esp32s3'

```
./build.sh -t <target>
```

This build command will build for the ESP32-S3 target. You can specify other targets.

- esp32
- esp32c2
- esp32c3
- esp32c5
- esp32c6
- esp32c61
- esp32h2
- esp32p4
- esp32s2
- esp32s3

Set Build Type Set the build type. ex. 'build' to build the project and prepare for uploading to a board.

Note: This command depends on the `-t` argument.

```
./build.sh -t esp32 -b <build|menuconfig|idf_libs|copy_bootloader|mem_variant>
```

Additional Configuration Specify additional configs to be applied. ex. `qio 80m` to compile for QIO Flash at 80 MHz.

Note: This command requires the `-b` to work properly.

```
./build.sh -t esp32 -b idf_libs qio 80m
```

5.1.4 User Interface

Starting from `arduino-esp32` version 3.0.0 (IDF v5.1), there is also a terminal user interface that can be used to configure the libraries to be compiled.

It allows the user to select the targets to compile, change the configuration options and compile the libraries. It has mouse support and can be pre-configured using command line arguments.

For more information and troubleshooting, check [the documentation](#).

To use the terminal user interface, make sure to have `python>=3.9`, all the previous dependencies and install the textual library:

```
pip install --user textual
```

You can then run the UI using the following command:

```
./tools/config_editor/app.py
```

Pre-Configuring the UI

The UI can be pre-configured using command line arguments. The following arguments are available:

- `-t, --target <target>`: Comma-separated list of targets to be compiled. Choose from: *all, esp32, esp32c2, esp32c3, esp32c5, esp32c6, esp32c61, esp32h2, esp32s2, esp32s3*. Default: all except *esp32c2* and *esp32c61*;
- `--copy, --no-copy`: Enable/disable copying the compiled libraries to `arduino-esp32`. Enabled by default;
- `-c, --arduino-path <path>`: Path to `arduino-esp32` directory. Default: OS dependent;
- `-A, --arduino-branch <branch>`: Branch of the `arduino-esp32` repository to be used. Default: set by the build script;
- `-I, --idf-branch <branch>`: Branch of the ESP-IDF repository to be used. Default: set by the build script;
- `-i, --idf-commit <commit>`: Commit of the ESP-IDF repository to be used. Default: set by the build script;
- `-D, --debug-level <level>`: Debug level to be set in ESP-IDF. Choose from: *default, none, error, warning, info, debug, verbose*. Default: *default*.

Please note that all these options can be changed in the UI itself and are only used for automation purposes.

Screens

There are many screens in the UI that are used to configure the libraries to be compiled. Note that in all screens you can also use the shortcut keys shown in the footer bar to navigate.

The UI consists of the following screens:

- **Main Menu**: The main screen shows buttons to navigate to the other screens.
- **Compile Screen**: The compile screen shows the output of the compilation process and any errors that may have occurred.
- **Sdkconfig Editor**: The `sdkconfig` editor screen is a simple text editor that shows you the `sdkconfig` files that will be used for compilation. You can edit the files here to customize the generated libraries.
- **Settings Screen**: The settings screen allows you to change the settings of the compilation process. Here you can change:
 - The targets that the libraries will be compiled for. To save time, you can compile the libraries only for the target you are using;
 - Whether the compiled libraries will be copied to the `arduino-esp32` directory after compilation so that they can be used in the Arduino IDE;
 - The path to the `arduino-esp32` directory. This will be automatically set if the `arduino-esp32` repository is in one of the default locations. If not, you can set it manually here. If using the docker image, it should not be changed as the mount point is fixed;
 - The branch of the `arduino-esp32` repository to be used. This is useful if you want to compile the libraries for a specific branch or pull request of the `arduino-esp32` repository. Leave empty to use the default branch for this ESP-IDF version;
 - The branch of the ESP-IDF repository to be used. This is useful if you want to compile the libraries for a specific branch of the ESP-IDF repository. Leave empty to use the default branch for this IDF version;
 - The commit of the ESP-IDF repository to be used. This is useful if you want to compile the libraries for a specific commit on the selected branch. Leave empty to use the latest commit;
 - The debug level to be set in ESP-IDF.

5.1.5 Docker Image

You can use a docker image for building the static libraries of ESP-IDF components for use in Arduino projects. This image contains a copy of the `esp32-arduino-lib-builder` repository and already includes or will obtain all the required tools and dependencies to build the Arduino static libraries.

The current supported architectures by the Docker image are:

- `amd64`
- `arm64`

Note: Building the libraries using the Docker image is much slower than building them natively on the host machine. It is recommended to use the Docker image only when the host machine does not meet the requirements for building the libraries (e.g., building on Windows).

Tags

Multiple tags of this image are maintained:

- `latest`: tracks master branch of the Lib Builder. Note that the `latest` tag is not recommended for use as, depending on the development stage of the Lib Builder, it might not be stable or might not contain the latest changes;
- `release-vX.Y`: tracks `release/vX.Y` branch of the Lib Builder.

Note: Versions of Lib Builder released before this feature was introduced do not have corresponding Docker image versions. You can check the up-to-date list of available tags at <https://hub.docker.com/r/esp8266/arduino-lib-builder/tags>.

Usage

Before using the `esp8266/arduino-lib-builder` Docker image locally, make sure you have Docker installed and running on your machine. Follow the instructions at <https://docs.docker.com/install/>, if it is not installed yet.

If using the image in a CI environment, consult the documentation of your CI service on how to specify the image used for the build process.

Building the Libraries You have two options to run the Docker image to build the libraries. Manually or using the provided run script.

To run the Docker image manually, use the following command from the root of the `arduino-esp32` repository:

```
docker run --rm -it -v $PWD:/arduino-esp32 -e TERM=xterm-256color esp8266/arduino-lib-builder:release-v5.5
```

This will start the Lib Builder UI for compiling the libraries. The above command explained:

- `docker run`: Runs a command in a container;
- `--rm`: Optional. Automatically removes the container when it exits. Remove this flag if you plan to use the container multiple times;
- `-i` Run the container in interactive mode;
- `-t` Allocate a pseudo-TTY;
- `-e TERM=xterm-256color`: Optional. Sets the terminal type to `xterm-256color` to display colors correctly;
- `-v $PWD:/arduino-esp32`: Optional. Mounts the current folder at `/arduino-esp32` inside the container. If not provided, the container will not copy the compiled libraries to the host machine;
- `esp8266/arduino-lib-builder:release-v5.5`: uses Docker image `esp8266/arduino-lib-builder` with tag `release-v5.5`. The `latest` tag is implicitly added by Docker when no tag is specified. It is recommended to use a specific version tag to ensure reproducibility of the build process.

Warning: The `-v` option is used to mount a folder from the host machine to the container. Make sure the folder already exists on the host machine before running the command. Otherwise, the folder will be created with root permissions and files generated inside the container might cause permission issues and compilation errors.

Note: When the mounted directory `/arduino-esp32` contains a git repository owned by a different user (UID) than the one running the Docker container, git commands executed within `/arduino-esp32` might fail, displaying an error message `fatal: detected dubious ownership in repository at '/arduino-esp32'`. To resolve this issue, you can designate the `/arduino-esp32` directory as safe by setting the `LIBBUILDER_GIT_SAFE_DIR` environment variable during the Docker container startup. For instance, you can achieve this by including `-e LIBBUILDER_GIT_SAFE_DIR='/arduino-esp32'` as a parameter. Additionally, multiple directories can be specified by using a `:` separator. To entirely disable this git security check, `*` can be used.

After running the above command, you will be inside the container and the libraries can be built using the user interface.

By default the docker container will run the user interface script. If you want to run a specific command, you can pass it as an argument to the `docker run` command. For example, to run a terminal inside the container, you can run:

```
docker run -it espressif/esp32-arduino-lib-builder:release-v5.5 /bin/bash
```

Running the Docker image using the provided run script will depend on the host OS. Use the following command from the root of the `arduino-esp32` repository to execute the image in a Linux or macOS environment for the `release-v5.5` tag:

```
curl -LJO https://raw.githubusercontent.com/espressif/esp32-arduino-lib-builder/
↪refs/heads/release/v5.5/tools/docker/run.sh
chmod +x run.sh
./run.sh $PWD
```

For Windows, use the following command in PowerShell from the root of the `arduino-esp32` repository:

```
Invoke-WebRequest -Uri "https://raw.githubusercontent.com/espressif/esp32-arduino-
↪lib-builder/refs/heads/release/v5.5/tools/docker/run.ps1" -OutFile "run.ps1"
.\run.ps1 $pwd
```

As the script is unsigned, you may need to change the execution policy of the current session before running the script. To do so, run the following command in PowerShell:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

Warning: It is always a good practice to understand what the script does before running it. Make sure to analyze the content of the script to ensure it is safe to run and won't cause any harm to your system.

Building Custom Images

To build a custom Docker image, you need to clone the Lib Builder repository and use the provided Dockerfile in the Lib Builder repository. The Dockerfile is located in the `tools/docker` directory.

The [Docker file in the Lib Builder repository](#) provides several build arguments which can be used to customize the Docker image:

- `LIBBUILDER_CLONE_URL`: URL of the repository to clone Lib Builder from. Can be set to a custom URL when working with a fork of Lib Builder. The default is `https://github.com/espressif/esp32-arduino-lib-builder.git`;
- `LIBBUILDER_CLONE_BRANCH_OR_TAG`: Name of a git branch or tag used when cloning Lib Builder. This value is passed to the `git clone` command using the `--branch` argument. The default is `master`;
- `LIBBUILDER_CHECKOUT_REF`: If this argument is set to a non-empty value, `git checkout $LIBBUILDER_CHECKOUT_REF` command performs after cloning. This argument can be set to the SHA of the specific commit to check out, for example, if some specific commit on a release branch is desired;

- `LIBBUILDER_CLONE_SHALLOW`: If this argument is set to a non-empty value, `--depth=1 --shallow-submodules` arguments are used when performing `git clone`. Depth can be customized using `LIBBUILDER_CLONE_SHALLOW_DEPTH`. Doing a shallow clone significantly reduces the amount of data downloaded and the size of the resulting Docker image. However, if switching to a different branch in such a “shallow” repository is necessary, an additional `git fetch origin <branch>` command must be executed first;
- `LIBBUILDER_CLONE_SHALLOW_DEPTH`: This argument specifies the depth value to use when doing a shallow clone. If not set, `--depth=1` will be used. This argument has effect only if `LIBBUILDER_CLONE_SHALLOW` is used. Use this argument if you are building a Docker image for a branch, and the image has to contain the latest tag on that branch. To determine the required depth, run `git describe` for the given branch and note the offset number. Increment it by 1, then use it as the value of this argument. The resulting image will contain the latest tag on the branch, and consequently `git describe` command inside the Docker image will work as expected;

To use these arguments, pass them via the `--build-arg` command line option. For example, the following command builds a Docker image with a shallow clone of Lib Builder from a specific repository and branch:

```
docker buildx build -t lib-builder-custom:master \
  --build-arg LIBBUILDER_CLONE_BRANCH_OR_TAG=master \
  --build-arg LIBBUILDER_CLONE_SHALLOW=1 \
  --build-arg LIBBUILDER_CLONE_URL=https://github.com/espressif/esp32-arduino-
↳ lib-builder \
  tools/docker
```

5.2 Arduino as an ESP-IDF component

5.2.1 About

You can use the Arduino framework as an ESP-IDF component. This allows you to use the Arduino framework in your ESP-IDF projects with the full flexibility of the ESP-IDF.

This method is recommended for advanced users. To use this method, you will need to have the ESP-IDF toolchain installed.

For a simplified method, see [Installing using Boards Manager](#).

If you plan to use these modified settings multiple times, for different projects and targets, you can recompile the Arduino core with the new settings using the Arduino Static Library Builder. For more information, see the [Lib Builder documentation](#).

Note: Latest Arduino Core ESP32 version (3.3.10) is now compatible with ESP-IDF v5.5. Please consider this compatibility when using Arduino as a component in ESP-IDF.

For easiest use of Arduino framework as a ESP-IDF component, you can use the [IDF Component Manager](#) to add the Arduino component to your project. This will automatically clone the repository and its submodules. You can find the Arduino component in the [ESP Registry](#) together with dependencies list and examples.

5.2.2 Installation

1. Download and install [ESP-IDF](#).
 - For more information see [Get Started](#).

Installing using IDF Component Manager

To add the Arduino component to your project using the IDF Component Manager, run the following command in your project directory:

```
idf.py add-dependency "espressif/arduino-esp32^3.3.10"
```

Or you can start a new project from a template with the Arduino component:

```
idf.py create-project-from-example "espressif/arduino-esp32^3.3.10:hello_world"
```

Manual installation of Arduino framework

1. Create a blank ESP-IDF project (use `sample_project` from `/examples/get-started`) or choose one of the examples.
2. In the project folder, create a new folder called `components` and clone this repository inside the newly created folder.

```
mkdir -p components && \  
cd components && \  
git clone https://github.com/espressif/arduino-esp32.git arduino && \  
cd arduino && \  
git submodule update --init --recursive && \  
cd ../../ && \  
idf.py menuconfig
```

Note: If you use Arduino with ESP-IDF often, you can place the `arduino` folder into global `components` folder.

If you're targeting the ESP32-S2 or ESP32-S3 and you want to use USBHID classes such as `USBHID`, `USBHIDConsumerControl`, `USBHIDGamepad`, `USBHIDKeyboard`, `USBHIDMouse`, `USBHIDSystemControl`, or `USBHIDVendor`:

1. Clone these nested repos somewhere:

```
git clone https://github.com/espressif/esp32-arduino-lib-builder.git esp32-arduino-  
↳lib-builder && \  
git clone https://github.com/hathach/tinyusb.git esp32-arduino-lib-builder/  
↳components/arduino_tinyusb/tinyusb
```

2. In the project folder, edit `CMakeLists.txt` and add the following before the `project()` line:

```
set(EXTRA_COMPONENT_DIRS <path to esp32-arduino-lib-builder/components/arduino_  
↳tinyusb>)
```

5.2.3 Configuration

Depending on one of the two following options, in the `menuconfig` set the appropriate settings.

Go to the section `Arduino Configuration` --->

1. For usage of `app_main()` function - Turn off `Autostart Arduino setup and loop on boot`
2. For usage of `setup()` and `loop()` functions - Turn on `Autostart Arduino setup and loop on boot`

Experienced users can explore other options in the `Arduino` section.

After the setup you can save and exit:

- Save [S]
- Confirm default filename [Enter]
- Close confirmation window [Enter] or [Space] or [Esc]
- Quit [Q]

As the Arduino libraries use C++ features, you will need to swap some file extensions from `.c` to `.cpp`:

- In main folder rename file *main.c* to *main.cpp*.
- In main folder open file *CMakeLists.txt* and change *main.c* to *main.cpp* as described below.

Option 1. Using Arduino `setup()` and `loop()`

Your *main.cpp* should be formatted like any other sketch. Don't forget to include *Arduino.h*.

```
//file: main.cpp
#include "Arduino.h"

void setup(){
  Serial.begin(115200);
  while(!Serial){
    ; // wait for serial port to connect
  }
}

void loop(){
  Serial.println("loop");
  delay(1000);
}
```

Option 2. Using ESP-IDF `appmain()`

In *main.cpp* you need to implement `app_main()` and call `initArduino()` ; in it.

Keep in mind that `setup()` and `loop()` will not be called in this case. Furthermore the `app_main()` is single execution as a normal function so if you need an infinite loop as in Arduino place it there.

```
//file: main.cpp
#include "Arduino.h"

extern "C" void app_main()
{
  initArduino();

  // Arduino-like setup()
  Serial.begin(115200);
  while(!Serial){
    ; // wait for serial port to connect
  }

  // Arduino-like loop()
  while(true){
    Serial.println("loop");
  }

  // WARNING: if program reaches end of function app_main() the MCU will restart.
}
```

Build, flash and monitor

- For both options use command `idf.py -p <your-board-serial-port> flash monitor`
- The project will build, upload and open the serial monitor to your board
 - Some boards require button combo press on the board: press-and-hold Boot button + press-and-release RST button, release Boot button
 - After a successful flash, you may need to press the RST button again
 - To terminate the serial monitor press `Ctrl + ]`

5.2.4 Logging To Serial

If you are writing code that does not require Arduino to compile and you want your *ESP_LOGx* macros to work in Arduino IDE, you can enable the compatibility by adding the following lines:

```
#ifndef ARDUINO_ARCH_ESP32
#include "esp32-hal-log.h"
#endif
```

5.2.5 FreeRTOS Tick Rate (Hz)

The Arduino component requires the FreeRTOS tick rate *CONFIG_FREERTOS_HZ* set to 1000 Hz in *make menuconfig* -> *Component config* -> *FreeRTOS* -> *Tick rate*.

5.2.6 Compilation Errors

As commits are made to ESP-IDF and submodules, the codebases can develop incompatibilities that cause compilation errors. If you have problems compiling, follow the instructions in [Issue #1142](#) to roll ESP-IDF back to a different version.

5.2.7 Adding arduino library

There are few approaches:

1. Add global library to `components/arduino-esp32/libraries/new_library`
2. Add local project library to `examples/your_project/main/libraries/new_library`

1 Adding global library

Download the library:

```
cd ~/esp/esp-idf/components/arduino/
git clone --recursive git@github.com:Author/new_library.git libraries/new_library
```

Edit file `components/arduino-esp32/CMakeLists.txt`

Get the source file list with shell command:

```
find libraries/new_library/src/ -name '*.c' -o -name '*.cpp'
libraries/new_library/src/new_library.cpp
libraries/new_library/src/new_library_extra_file.c
```

Locate block which starts with `set (LIBRARY_SRCS` and copy the list there. Now it should look something like this:

```
set (LIBRARY_SRCS
  libraries/ArduinoOTA/src/ArduinoOTA.cpp
  libraries/AsyncUDP/src/AsyncUDP.cpp
  libraries/new_library/src/new_library.cpp
  libraries/new_library/src/new_library_extra_file.c
```

After this add the library path to block which starts with `set (includedirs`. It should look like this:

```
set (includedirs
  variants/${CONFIG_ARDUINO_VARIANT}/
  cores/esp32/
  libraries/ArduinoOTA/src
```

(continues on next page)

(continued from previous page)

```
libraries/AsyncUDP/src  
libraries/new_library/src
```

2 Adding local library

Download the library:

```
cd ~/esp/esp-idf/examples/your_project  
mkdir components  
git clone --recursive git@github.com:Author/new_library.git components/new_library
```

Create new CMakeLists.txt in the library folder: components/new_library/CMakeLists.txt

```
idf_component_register(SRCS "new_library.cpp" "another_source.c"  
                        INCLUDE_DIRS "."  
                        REQUIRES arduino-esp32  
                        )
```

You can read more about CMakeLists in the IDF documentation regarding the [Build System](#)

5.2.8 Tip

If you want to use arduino-esp32 both as an ESP-IDF component and with Arduino IDE you can simply create a symlink:

```
ln -s ~/Arduino/hardware/espressif/esp32 ~/esp/esp-idf/components/arduino-esp32
```

This will allow you to install new libraries as usual with Arduino IDE. To use them with IDF component, use `add_lib.sh -e ~/Arduino/libraries/New_lib`

5.3 OTA Web Update

OTAWebUpdate is done with a web browser that can be useful in the following typical scenarios:

- Once the application developed and loading directly from Arduino IDE is inconvenient or not possible
- after deployment if user is unable to expose Firmware for OTA from external update server
- provide updates after deployment to small quantity of modules when setting an update server is not practicable

For more information about the update process, please refer to the [OTA API reference](#) section of the ESP-IDF documentation.

5.3.1 Requirements

- The ESP and the computer must be connected to the same network

5.3.2 Implementation

The sample implementation has been done using:

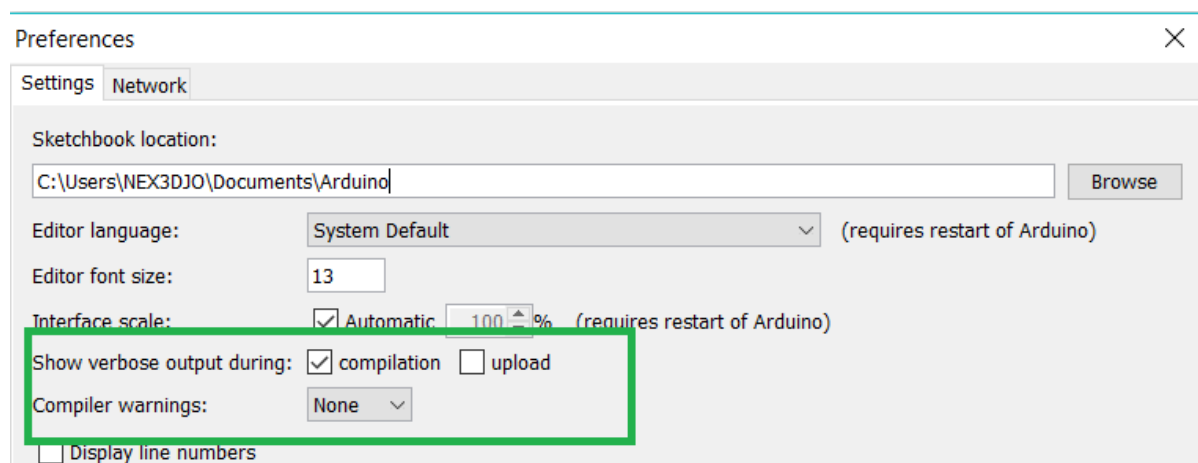
- Example sketch ``OTAWebUpdater.ino``.
- ESP32 Board.

You can also use another module if it meets Flash chip size of the sketch

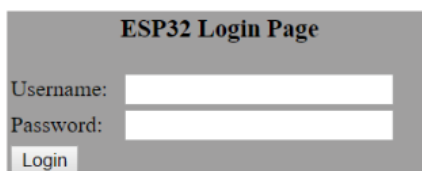
Before you begin, please make sure that you have the following software installed:

- Arduino IDE
- **Host software depending on O/S you use**
 - [Avahi](#) for Linux
 - [Bonjour](#) for Windows
 - Mac OSX and iOS - support is already built in / no any extra s/w is required

Prepare the sketch and configuration for initial upload with a serial port - Start Arduino IDE and load sketch OTAWebUpdater.ino available under File > Examples > OTAWebUpdater.ino - Update ssid and pass in the sketch so the module can join your Wi-Fi network - Open File > Preferences, look for “Show verbose output during:” and check out “compilation” option



- Upload sketch (Ctrl+U)
- Now open web browser and enter the url, i.e. <http://esp32.local>. Once entered, browser should display a form



ESP32 Login Page

Username:

Password:

- username = admin
- password = admin

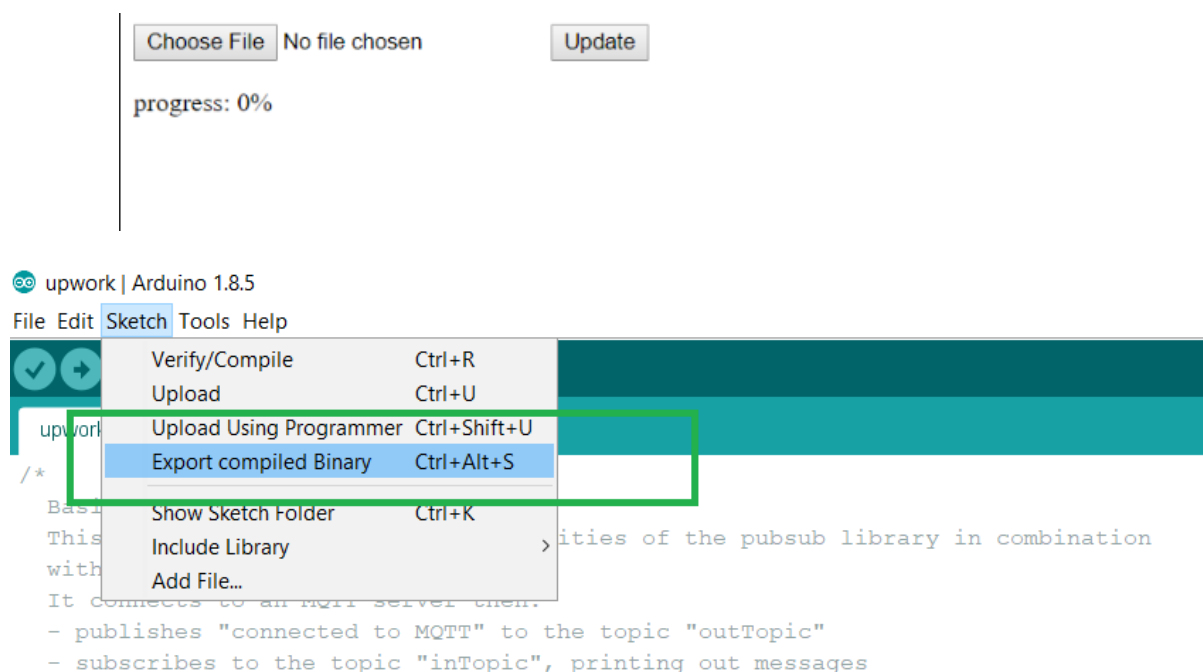
Note: If entering “<http://ESP32.local>” does not work, try replacing “ESP32” with module’s IP address. This workaround is useful in case the host software installed does not work.

Now click on the Login button and browser will display an upload form

For Uploading the New Firmware, you need to provide the Binary File of your Code.

Exporting Binary file of the Firmware (Code) - Open up the Arduino IDE - Open up the Code, for Exporting up Binary file - Now go to Sketch > export compiled Binary

- Binary file is exported to the same Directory where your code is present



Once you are comfortable with this procedure, go ahead and modify `OTAWebUpdater.ino` sketch to print some additional messages and compile it. Then, export the new binary file and upload it using web browser to see entered changes on a Serial Monitor.

5.4 makeEspArduino

The `makeEspArduino` is a generic makefile for any ESP8266/ESP32 Arduino project. Using it instead of the Arduino IDE makes it easier to do automated and production builds.

Chapter 6

Third Party Tools

Here you will find documentation pages for third party tools that can be used with the ESP32 Arduino Core.

Warning: These tools are **not maintained by the ESP32 Arduino Core team**, so we cannot provide support or guarantee that they will work as expected. Each tool documentation should be provided and maintained by the community.

6.1 pioarduino - (p)eople (i)nitiated (o)ptimized (arduino)

Warning: This tool is **not maintained by the ESP32 Arduino Core team**, so we cannot provide support or guarantee that it will work as expected.

Note: This is a work in progress documentation and we will appreciate your help! We are looking for contributors!

6.1.1 About

For more information, please refer to the [official documentation](#).

6.2 Wokwi

Warning: This tool is **not maintained by the ESP32 Arduino Core team**, so we cannot provide support or guarantee that it will work as expected.

Note: This is a work in progress documentation and we will appreciate your help! We are looking for contributors!

6.2.1 About

Wokwi is an online Electronics simulator. You can use it to simulate Arduino, ESP32, and many other popular boards, parts and sensors.

The advantages of using Wokwi include:

- Immediate start: No need to wait for components or download large software. Everything required is available in your browser, enabling you to begin coding your IoT project within seconds.
- Safe experimentation: Virtual hardware cannot be damaged, allowing users to experiment freely without the risk of destroying components. Mistakes can be easily undone.
- Easy collaboration: Sharing a link to your Wokwi project facilitates obtaining help and feedback from others.
- Code reliability: Helps in distinguishing between hardware and software issues, thereby increasing confidence in your code.
- Unlimited resources: Access to an unlimited number of parts without concerns about cost or availability.
- Supportive community: A maker-friendly environment where users can share projects, seek assistance, and find inspiration.

Unique features provided by Wokwi:

- Wi-Fi simulation - Connect your simulated project to the internet. You can use MQTT, HTTP, NTP, and many other network protocols.
- Virtual Logic Analyzer - Capture digital signals in your simulation (e.g. UART, I2C, SPI) and analyze them on your computer.
- Advanced debugging with GDB - Powerful Arduino debugger for advanced users.
- SD card simulation - Store and retrieve files and directories from your code. Paying users can also upload binary files (such as images)
- Chips API - Create your own custom chips and parts, and share them with the community.
- Visual Studio Code integration - Simulate your embedded projects directly from VS Code.

6.2.2 Pricing

Wokwi is free for personal use. For commercial users and professionals, please check out the paid plans in the [pricing page](#).

6.2.3 Learn more

For more information, please refer to the [official Wokwi website](#) and the [Wokwi documentation](#).

Chapter 7

Migration Guides

7.1 Migration from 2.x to 3.0

7.1.1 Introduction

This is a guide to highlight **breaking changes** in the API and build system to help the migration of projects from versions 2.X (based on ESP-IDF 4.4) to version 3.0 (based on ESP-IDF 5.1) of the Arduino ESP32 core.

All the examples on the version 3.0.0 were updated to be compatible to the new API. The old examples from the versions below 3.0.0 will be not compatible with the version 3.0.0 or newer releases.

For more information about all changes and new features, check project [RELEASE NOTES](#).

7.1.2 Build System

Compilation Flags

Functional changes

- If your project uses extra flags in the compilation process, it will now overwrite [some required default flags](#). To ensure your project compiles correctly, make sure to have the `-MMD -c` flags in your C and C++ extra flags.

7.1.3 APIs

ADC

Removed APIs

- `analogSetClockDiv`
- `adcAttachPin`
- `analogSetVRefPin`

BLE

Changes in APIs

- Changed APIs return and parameter type from `std::string` to Arduino style `String`.
- Changed UUID data type from `uint16_t` to `BLEUUID` class.
- `BLEScan::start` and `BLEScan::getResults` methods return type changed from `BLEScanResults` to `BLEScanResults*`.

Hall Sensor

Hall sensor is no longer supported.

Removed APIs

- `hallRead`

I2S

The I2S driver has been completely redesigned and refactored to use the new ESP-IDF driver. For more information about the new API, check [I2S](#).

LEDC

The LEDC API has been changed in order to support the Peripheral Manager and make it easier to use, as LEDC channels are now automatically assigned to pins. For more information about the new API, check [LED Control \(LEDC\)](#).

Removed APIs

- `ledcSetup`
- `ledcAttachPin`

New APIs

- `ledcAttach` used to set up the LEDC pin (merged `ledcSetup` and `ledcAttachPin` functions).
- `ledcOutputInvert` used to attach the interrupt to a timer using arguments.
- `ledcFade` used to set up and start a fade on a given LEDC pin.
- `ledcFadeWithInterrupt` used to set up and start a fade on a given LEDC pin with an interrupt.
- `ledcFadeWithInterruptArg` used to set up and start a fade on a given LEDC pin with an interrupt using arguments.

Changes in APIs

- `ledcDetachPin` renamed to `ledcDetach`.
- In all functions, input parameter `channel` has been changed to `pin`.

RMT

For more information about the new API, check [Remote Control \(RMT\)](#).

Removed APIs

- `_rmtDumpStatus`
- `rmtSetTick`
- `rmtWriteBlocking`
- `rmtEnd`
- `rmtBeginReceive`
- `rmtReadData`

New APIs

- `rmtSetEOT`
- `rmtWriteAsync`
- `rmtTransmitCompleted`
- `rmtSetRxMinThreshold`

Changes in APIs

- In all functions, input parameter `rmt_obj_t* rmt` has been changed to `int pin`.
- `rmtInit` return parameter changed to `bool`.
- `rmtInit` input parameter `bool tx_not_rx` has been changed to `rmt_ch_dir_t channel_direction`.
- `rmtInit` new input parameter `uint32_t frequency_Hz` to set frequency of RMT channel (as function `rmtSetTick` was removed).
- `rmtWrite` now sending data in blocking mode. It only returns after sending all data or through a timeout. For Async mode use the new `rmtWriteAsync` function.
- `rmtWrite` new input parameter `uint32_t timeout_ms`.
- `rmtLoop` renamed to `rmtWriteLooping`.
- `rmtRead` input parameters changed to `int pin, rmt_data_t* data, size_t *num_rmt_symbols, uint32_t timeout_ms`.
- `rmtReadAsync` input parameters changed to `int pin, rmt_data_t* data, size_t *num_rmt_symbols`.
- `rmtSetRxThreshold` renamed to `rmtSetRxMaxThreshold` and input parameter `uint32_t` value has been changed to `uint16_t idle_thres_ticks`.
- `rmtSetCarrier` input parameters `uint32_t low, uint32_t high` have been changed to `uint32_t frequency_Hz, float duty_percent`.

SigmaDelta

SigmaDelta has been refactored to use the new ESP-IDF driver. For more information about the new API, check [SigmaDelta](#).

Removed APIs

- `sigmaDeltaSetup`
- `sigmaDeltaRead`

New APIs

- `sigmaDeltaAttach` used to set up the SigmaDelta pin (channel is acquired automatically).
- `timerGetFrequency` used to get the actual frequency of the timer.
- `timerAttachInterruptArg` used to attach the interrupt to a timer using arguments.

Changes in APIs

- `sigmaDeltaDetachPin` renamed to `sigmaDeltaDetach`.
- `sigmaDeltaWrite` input parameter `channel` has been changed to `pin`.

Timer

Timer has been refactored to use the new ESP-IDF driver and its API got simplified. For more information about the new API check [Timer](#).

Removed APIs

- `timerGetConfig`
- `timerSetConfig`
- `timerSetDivider`
- `timerSetCountUp`
- `timerSetAutoReload`
- `timerGetDivider`
- `timerGetCountUp`
- `timerGetAutoReload`
- `timerAlarmEnable`
- `timerAlarmDisable`
- `timerAlarmWrite`
- `timerAlarmEnabled`
- `timerAlarmRead`
- `timerAlarmReadMicros`
- `timerAlarmReadSeconds`
- `timerAttachInterruptFlag`

New APIs

- `timerAlarm` used to set up Alarm for the timer and enable it automatically (merged `timerAlarmWrite` and `timerAlarmEnable` functions).
- `timerGetFrequency` used to get the actual frequency of the timer.
- `timerAttachInterruptArg` used to attach the interrupt to a timer using arguments.

Changes in APIs

- `timerBegin` has now only 1 parameter (frequency). There is an automatic calculation of the divider using different clock sources to achieve the selected frequency.
- `timerAttachInterrupt` has now only 2 parameters. The `edge` parameter has been removed.

UART (HardwareSerial)

Changes in APIs

- `setHwFlowCtrlMode` input parameter `uint8_t mode` has been changed to `SerialHwFlowCtrl mode`.
- `setMode` input parameter `uint8_t mode` has been changed to `SerialMode mode`.

Functional changes

- Default pins for some SoCs have been changed to avoid conflicts with other peripherals: * ESP32' s UART1 RX and TX pins are now GPIO26 and GPIO27, respectively; * ESP32' s UART2 RX and TX pins are now GPIO4 and GPIO25, respectively; * ESP32-S2' s UART1 RX and TX pins are now GPIO4 and GPIO5, respectively.
- It is now possible to detach UART0 pins by calling `end()` with no previous call of `begin()`.
- It is now possible to call `setPins()` before `begin()` or in any order.
- `setPins()` will detach any previous pins that have been changed.
- `begin(baud, rx, tx)` will detach any previous attached pins.
- `setPins()` or `begin(baud, rx, tx)` when called at first, will detach console RX0/TX0, attached in boot.
- Any pin set as -1 in `begin()` or `setPins()` won' t be changed nor detached.
- `begin(baud)` will not change any pins that have been set before this call, through a previous `begin(baud, rx, tx)` or `setPin()`.
- If the application only uses RX or TX, `begin(baud, -1, tx)` or `begin(baud, rx)` will change only the assigned pin and keep the other unchanged.

Wi-Fi

Functional changes

- In Arduino (and other frameworks) the method named `flush()` is intended to send out the transmit buffer content. `WiFiClient` and `WiFiUDP` method `flush()` won't clear the receive buffer anymore. A new method called `clear()` is now used for that. Currently `flush()` does nothing in `WiFiClient`, `WiFiClientSecure` and `WiFiUDP`.
- `WiFiServer` has functions `accept()` and `available()` with the same functionality. In Arduino, `available()` should work differently so it is now deprecated.
- `WiFiServer` had unimplemented write functions inherited from `Print` class. These are now removed. Also unimplemented method `stopAll()` is removed. The methods were unimplemented because `WiFiServer` doesn't manage connected `WiFiClient` objects for print-to-all-clients functionality.

Chapter 8

Frequently Asked Questions

8.1 How to modify an sdkconfig option in Arduino?

Arduino-esp32 project is based on ESP-IDF. While ESP-IDF supports configuration of various compile-time options (known as “Kconfig options” or “sdkconfig options”) via a “menuconfig” tool, this feature is not available in Arduino IDE.

To use the arduino-esp32 core with a modified sdkconfig option, you need to use ESP-IDF to compile Arduino libraries. Please see [Arduino as an ESP-IDF component](#) and [Library Builder](#) for the two solutions available.

Note that modifying `sdkconfig` or `sdkconfig.h` files found in the arduino-esp32 project tree **does not** result in changes to these options. This is because ESP-IDF libraries are included into the arduino-esp32 project tree as pre-built libraries.

8.2 How to compile libs with different debug level?

The short answer is `esp32-arduino-lib-builder/configs/defconfig.common:44`. A guide explaining the process can be found here [<guides/core_debug>](#)

Chapter 9

Troubleshooting

9.1 Common Issues

Here are some of the most common issues around the ESP32 development using Arduino.

Note: Please consider contributing if you have found any issues with the solution here.

9.1.1 Installing

Here are the common issues during the installation.

Slow or unstable downloads

Users in China might have troubles with connection and download speeds using GitHub. Please use our Jihulab mirror as the repository source:

<https://jihulab.com/esp-mirror/esp8266/arduino-esp8266.git>

JSON files for the boards manager are available here:

- Stable release:

```
https://jihulab.com/esp-mirror/esp8266/arduino-esp8266/-/raw/gh-pages/package_
↪esp8266_index_cn.json
```

- Development release:

```
https://jihulab.com/esp-mirror/esp8266/arduino-esp8266/-/raw/gh-pages/package_
↪esp8266_dev_index_cn.json
```

9.1.2 Building

Missing Python: “python” : executable file not found in \$PATH

You are trying to build your sketch using Ubuntu and this message appears:

```
"exec: "python": executable file not found in $PATH
Error compiling for board ESP32 Dev Module"
```

Solution To avoid this error, you can install the `python-is-python3` package to create the symbolic links.

```
sudo apt install python-is-python3
```

If you are not using Ubuntu, you can check if you have the Python correctly installed or the presence of the symbolic links/environment variables.

9.1.3 Flashing

Why is my board not flashing/uploading when I try to upload my sketch?

To be able to upload the sketch via the serial interface, the ESP32 must be in the download mode. The download mode allows you to upload the sketch over the serial port, and to get into it, you need to keep the **GPIO0** in LOW while resetting (**EN** pin) the cycle. If you are trying to upload a new sketch and your board is not responding, there are some possible reasons.

Possible fatal error message from the Arduino IDE:

A fatal error occurred: Failed to connect to ESP32: Timed out waiting for packet header

Solution Here are some steps that you can try:

- Check your USB cable and try a new one (some cables are only for charging and there is no data connection).
- Change the USB port - prefer direct connection to the computer and avoid USB hubs. Some USB ports may share the power source with other ports used, for example, for charging a phone.
- Check your power supply.
- Make sure that nothing is connected to pins labeled **TX** and **RX**. Please refer to the pin layout table - some TX and RX pins may not be labeled on the dev board.
- In some instances, you must keep **GPIO0** LOW during the uploading process via the serial interface.
- Hold down the “**BOOT**” button on your ESP32 board while uploading/flashing.
- Solder a **10 uF** capacitor in parallel with **RST** and **GND**.
- If you are using external power connected to pins, it is easy to confuse pins **CMD** (which is usually next to the 5V pin) and **GND**.

In some development boards, you can try adding the reset delay circuit, as described in the *Power-on Sequence* section on the [ESP32 Hardware Design Guidelines](#) to get into the download mode automatically.

Arduino IDE 1.x: {placeholder} expansion in upload recipes

This section documents how **Arduino IDE 1.8.x** (Java IDE) expands properties in `platform.txt` when you **Upload** a sketch. It explains why some patterns that work with **arduino-cli** or **Arduino IDE 2.x** fail on IDE 1.x, and records approaches we tried while wiring the `flasher.py` upload wrapper.

This does **not** apply to the compile step: builds use **arduino-builder**, which merges `platform.txt` and `boards.txt` much like **arduino-cli**. Only the **upload** step uses the Java path described below.

How IDE 1.8.x builds the upload command On upload, `SerialUploader` (Arduino IDE 1.8.19) assembles a property map and runs `StringReplacer` on `tools.<upload.tool>.upload.pattern`:

1. Start from `PreferencesData.getMap()` (IDE settings and a few runtime.* keys, such as `runtime.platform.path`).
2. Merge **board** preferences from `boards.txt` (including menu selections), e.g. `upload.speed`, `build.mcu`.
3. Merge **only the active upload tool** subtree: `targetPlatform.getTool(upload.tool)`, which strips the `tools.<tool>.` prefix.
Example: `tools.esptool_py.cmd` becomes the short key `{cmd}` inside the pattern.
4. Replace `{name}` tokens in the pattern for up to 10 passes (`StringReplacer.formatAndSplit`).

So at upload time the recipe sees **board prefs + one tool's shorthands + runtime keys**, not the full `platform.txt` file.

What works on IDE 1.x (reference patterns) These patterns are known to expand correctly:

- **Board-scoped** names from `boards.txt`: `{upload.speed}`, `{upload.flags}`, `{upload.erase_cmd}`, `{build.mcu}`, etc. Each `esptool_py` board must define `upload.erase_cmd` at board level (empty by default); the `EraseFlash` menu overrides it with `-e`.
- **Runtime** keys set when you select the board/port: `{runtime.platform.path}`, `{serial.port}`, `{build.path}` (set on upload).
- **Tool shorthands** for the **selected** `upload.tool` only, e.g. `{path}`, `{cmd}`, `{upload.pattern_args}`, `{network_cmd}` (from `tools.esptool_py.network_cmd`).

The core uses `upload.flash_prefix` the same way as `network_cmd`: define it under each upload tool that needs the flasher wrapper, then reference `{upload.flash_prefix}` in `upload.pattern` / `program.pattern`:

```
tools.esptool_py.upload.flash_prefix=python3 "{runtime.platform.path}/tools/
↪flasher.py"
tools.esptool_py.upload.flash_prefix.windows="{runtime.platform.path}\\tools\
↪flasher.exe"
tools.esptool_py.upload.pattern={upload.flash_prefix} --esptool "{path}/{cmd}" --
↪build-dir "{build.path}" {upload.pattern_args}
tools.esptool_py.program.pattern={upload.flash_prefix} --esptool "{path}/{cmd}" --
↪build-dir "{build.path}" {program.pattern_args}
```

A second upload tool (`esptool_py_app_only`, used e.g. by the `kode dot board`) needs its **own** `upload.flash_prefix` because IDE 1.x never loads another tool's keys during upload.

Approaches that do not work on IDE 1.x (and why) The following are real configurations we tried for a single shared flasher command. They may work with `arduino-cli` or IDE 2.x but fail on IDE 1.8.x upload for the reasons given.

{tools.flasher.cmd} or a global tools.flasher.cmd property

```
tools.flasher.cmd=python3 "{runtime.platform.path}/tools/flasher.py"
tools.esptool_py.upload.pattern={tools.flasher.cmd} --esptool "{path}/{cmd}" ...
```

Result: Upload runs a program literally named `{tools.flasher.cmd}` (file not found).

Why: Only keys from the **active** tool subtree are merged as short names. Top-level `tools.flasher.*` is not part of `getTool("esptool_py")`. The full name `tools.flasher.cmd` is not in the upload property map at all.

{flasher.cmd} with tools.esptool_py.flasher.cmd duplicated on every upload tool

```
tools.esptool_py.flasher.cmd=python3 "{runtime.platform.path}/tools/flasher.py"
tools.esptool_py_app_only.flasher.cmd=python3 "{runtime.platform.path}/tools/
↪flasher.py"
tools.esptool_py.upload.pattern={flasher.cmd} --esptool "{path}/{cmd}" ...
```

Result: Works on IDE 1.x, but the flasher command must be **copied once per upload tool** (and per OS variant).

Why: Same as above: each `upload.tool` value only exposes **its** `tools.<that_tool>.*` keys. There is no cross-tool alias.

{build.flasher.cmd} defined once in platform.txt

```
build.flasher.cmd=python3 "{runtime.platform.path}/tools/flasher.py"
tools.esptool_py.upload.pattern={build.flasher.cmd} --esptool "{path}/{cmd}" ...
```


Result: Unexpanded `{build.flasher.cmd}` in the executed command (or upload failure).

Why: `getBoardPreferences()` only contains `build.*` entries from **boards.txt** (and menus), not arbitrary `build.*` defaults from `platform.txt`. Unless every board defines `build.flasher.cmd`, the key is missing at upload time.

`{upload.flasher_cmd}` defined once at the top of `platform.txt`

```
upload.flasher_cmd=python3 "{runtime.platform.path}/tools/flasher.py"
tools.esptool_py.upload.pattern={upload.flasher_cmd} --esptool "{path}/{cmd}" ...
```

Result: Same as `build.flasher.cmd`: placeholder not expanded.

Why: Platform-level `upload.*` keys are not merged into board preferences. Only `upload.*` from **boards.txt** (e.g. `esp32.upload.speed`) appear in the upload map—compare with `{upload.speed}`, which works because boards define it.

Tool alias to a platform property: `tools.esptool_py.upload.flasher_cmd={upload.flasher_cmd}`

```
upload.flasher_cmd=python3 "{runtime.platform.path}/tools/flasher.py"
tools.esptool_py.upload.flasher_cmd={upload.flasher_cmd}
tools.esptool_py.upload.pattern={upload.flasher_cmd} --esptool "{path}/{cmd}" ...
```

Result: `{upload.flasher_cmd}` stays literal in the final command.

Why: After step 3 above, the tool map contains `upload.flasher_cmd={upload.flasher_cmd}` (a self-reference). The upload map still does **not** contain the resolved `upload.flasher_cmd` from the top of `platform.txt`, so `StringReplacer` never substitutes the Python path. `arduino-cli` avoids this by expanding the whole platform when loading properties; IDE 1.x upload does not.

`kodedot.upload.flasher_cmd` in `boards.txt` (board-only definition)

```
kodedot.upload.flasher_cmd=python3 "{runtime.platform.path}/tools/flasher.py"
kodedot.upload.tool=esptool_py_app_only
tools.esptool_py_app_only.upload.pattern={upload.flasher_cmd} --esptool "{path}/
↳{cmd}" ...
```

Result: Can work for that **one** board.

Why: Board keys are merged in step 2. This only fixes boards that define the property; it duplicates the flasher path in `boards.txt` instead of `platform.txt` and does not help other boards or tools.

Inlining `{runtime.platform.path}/tools/flasher.py` in every `upload.pattern`

```
tools.esptool_py.upload.pattern=python3 "{runtime.platform.path}/tools/flasher.py"
↳--esptool "{path}/{cmd}" ...
tools.esptool_py.program.pattern=python3 "{runtime.platform.path}/tools/flasher.py
↳" --esptool "{path}/{cmd}" ...
tools.esptool_py_app_only.upload.pattern=python3 "{runtime.platform.path}/tools/
↳flasher.py" --esptool "{path}/{cmd}" ...
```

Result: Works on IDE 1.x.

Why: `{runtime.platform.path}` is in the upload map. The downside is repeating the same flasher invocation in every pattern (and `.windows` variants), which is why `upload.flash_prefix` is preferred within each tool.

Summary

Approach	IDE 1.8.x upload	up- load	Note
{tools.*} global tool	No		Not in active tool map
{build.*} in platform only	No		Board map has no platform build.* defaults
{upload.*} in platform only	No		Board map has no platform upload.* defaults
Tool alias to platform key	No		Alias value never resolved
tools.<tool>.upload.* short-hand	Yes		One definition per upload tool
{runtime.platform.path} inline	Yes		Inline in pattern
Board upload.* in boards.txt	Yes		Per-board duplication

To have **one** flasher definition for the whole core you would need either a single upload tool for all boards or a change in IDE 1.x (not something this core can assume). The maintained approach is `upload.flash_prefix` per upload tool, mirroring `network_cmd`.

A small script in the core tree simulates IDE 1.8.19 upload expansion for regression checks (run from the `esp32 hardware` folder):

```
python3 .github/scripts/ci_testing/test_ide1_upload_pattern.py
```

9.1.4 Hardware

Why is my computer not detecting my board?

If your board is not being detected after connecting to the USB, you can try the following:

Solution

- Check if the USB driver is missing. - [USB Driver Download Link](#)
- Check your USB cable and try a new one.
- Change the USB port.
- Check your power supply.
- Check if the board is damaged or defective.

9.1.5 Wi-Fi

Why does the board not connect to WEP/WPA- “encrypted” Wi-Fi?

Please note that WEP/WPA has significant security vulnerabilities, and its use is strongly discouraged. The support may, therefore, be removed in the future. Please migrate to WPA2 or newer.

Solution Nevertheless, it may be necessary to connect to insecure networks. To do this, the security requirement of the ESP32 must be lowered to an insecure level by using:

```
WiFi.setMinSecurity(WIFI_AUTH_WEP); // Lower min security to WEP.
// or
WiFi.setMinSecurity(WIFI_AUTH_WPA_PSK); // Lower min security to WPA.
```

Why does the board not connect to WPA3-encrypted Wi-Fi?

WPA3 support is resource-intensive and may not be compiled into the used SDK.

Solution

- Check WPA3 support by your SDK.
- Compile your custom SDK with WPA3 support.

Sample code to check SDK WPA3 support at compile time:

```
#ifndef CONFIG_ESP32_WIFI_ENABLE_WPA3_SAE
#warning "No WPA3 support."
#endif
```

Serial not printing

I have uploaded firmware to the ESP32 device, but I don't see any response from a Serial.print (HardwareSerial).

Solution Newer ESP32 variants have two possible USB connectors - USB and UART. The UART connector will go through a USB->UART adapter, and will typically present itself with the name of that mfr (eg, Silicon Labs CP210x UART Bridge). The USB connector can be used as a USB-CDC bridge and will appear as an Espressif device (Espressif USB JTAG/serial debug unit). On Espressif devkits, both connections are available, and will be labeled. ESP32 can only use UART, so will only have one connector. Other variants with one connector will typically be using USB. Please check in the product [datasheet](<https://products.espressif.com>) or [hardware guide](<https://www.espressif.com/en/products/devkits>) to find Espressif products with the appropriate USB connections for your needs. If you use the UART connector, you should disable USB-CDC on boot under the Tools menu (-D ARDUINO_USB_CDC_ON_BOOT=0). If you use the USB connector, you should have that enabled (-D ARDUINO_USB_CDC_ON_BOOT=1) and set USB Mode to "Hardware CDC and JTAG" (-D ARDUINO_USB_MODE=1). USB-CDC may not be able to initialize in time to catch all the data if your device is in a tight reboot loop. This can make it difficult to troubleshoot initialization issues.

9.1.6 SPIFFS mount failed

When you come across an error like this:

```
E (588) SPIFFS: mount failed, -10025
[E] [SPIFFS.cpp:47] begin(): Mounting SPIFFS failed! Error: -1
```

Try enforcing format on fail in your code by adding `true` in the `begin` method such as this:

```
SPIFFS.begin(true);
```

See the method prototype for reference: `bool begin(bool formatOnFail=false, const char * basePath="/spiffs", uint8_t maxOpenFiles=10, const char * partitionLabel=NULL);`

9.1.7 SD card mount fail

Even though you made sure that the pins are correctly connected, and not using restricted pins, you may still get an error such as this:

```
[ 1065] [E] [sd_diskio.cpp:807] sdcard_mount(): f_mount failed: (3) The physical
↪ drive cannot work
```

Most of the problems originate from a poor connection caused by prototyping cables/wires, and one of the best solutions is to **solder all the connections** or use good quality connectors.

Note that with SD_MMC lib all the data pins need to be pulled up with an external 10k to 3.3V. This applies especially to card's D3 which needs to be pulled up even when using 1-bit line connection and the D3 is not used.

If you want to try the software approach before soldering, try manually specifying SPI pins, like this:

```
int SD_CS_PIN = 19;
SPI.begin(18, 36, 26, SD_CS_PIN);
SPI.setDataMode(SPI_MODE0);
SD.begin(SD_CS_PIN);
```

ESP32-S3 is rebooting even with a bare minimum sketch

Some ESP32-S3 boards are equipped with Quad SPI (QSPI) or Octal SPI (OPI) PSRAM. If you upload such a board with default settings for ESP32-S3, it will result in rebooting with a message similar to this:

https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/api-guides/flash_psram_config.html

```
E (124) esp_core_dump_flash: Core dump flash config is corrupted! CRC=0x7bd5c66f_
↳ instead of 0x0
Rebooting...
???ESP-ROM:esp32s3-20210327
Build:Mar 27 2021
rst:0xc (RTC_SW_CPU_RST),boot:0x18 (SPI_FAST_FLASH_BOOT)
Saved PC:0x40376af0
SPIWP:0xee
Octal Flash Mode Enabled
For OPI Flash, Use Default Flash Boot Mode
mode:SLOW_RD, clock div:1
load:0x3fce3808,len:0x44c
load:0x403c9700,len:0xbec
load:0x403cc700,len:0x2920
entry 0x403c98d8

assert failed: do_core_init startup.c:326 (flash_ret == ESP_OK)
```

To fix the issue, you will need to find out the precise module you are using and set **PSRAM** in the Arduino IDE Tools according to the following table.

9.1.8 How to determine the module version:

- First determine if you have a **WROOM-1** or **WROOM-2** module - this is written on the module shielding almost at the top, right under the ESP logo and company name (Espressif) right after the ESP32-S3 - for example ESP32-S3-WROOM-2.
- Then locate the version code on left bottom corner on the module shielding. The markings are very small and it might be really difficult to read with naked eyes - try using a camera with careful lighting.

With this knowledge find your module in the table and note what is written in the **PSRAM** column.

- If the results is empty (-) you don't need to change anything
- For QSPI go to Tools > PSRAM > QSPI PSRAM
- For OPI go to Tools > PSRAM > OPI PSRAM

Note that WROOM-2 has always OPI.

Module	Code	Flash Mode	PSRAM
WROOM-1	N4	QSPI	•
WROOM-1	N8	QSPI	•
WROOM-1	N16	QSPI	•
WROOM-1	H4	QSPI	•
WROOM-1	N4R2	QSPI	QSPI
WROOM-1	N8R2	QSPI	QSPI
WROOM-1	N16R2	QSPI	QSPI
WROOM-1	N4R8	QSPI	OPI
WROOM-1	N8R8	QSPI	OPI
WROOM-1	N16R8	QSPI	OPI
WROOM-2	N16R8V	OPI	OPI
WROOM-2	N16R8V	OPI	OPI
WROOM-2	N32R8V	OPI	OPI

9.1.9 Further Help

If you encounter any other issues or need further assistance, please consult the [ESP32 Arduino Core](#) documentation or seek help from the [ESP32 community forums](#).

Chapter 10

Contributions Guide

We welcome contributions to the Arduino ESP32 project!

10.1 How to Contribute

Contributions to the Arduino ESP32 (fixing bugs, adding features, adding documentation) are welcome. We accept contributions via [Github Pull Requests](#).

10.2 Before Contributing

Before sending us a Pull Request, please consider this:

- All contributions must be written in English to ensure effective communication and support. Pull Requests written in other languages will be closed, with a request to rewrite them in English.
- Is the contribution entirely your own work, or is it already licensed under an LGPL 2.1 compatible Open Source License? If not, cannot accept it.
- Is the code adequately commented and can people understand how it is structured?
- Is there documentation or examples that go with code contributions?
- Are comments and documentation written in clear English, with no spelling or grammar errors?
- Example contributions are also welcome.
 - If you are contributing by adding a new example, please use the [Arduino style guide](#) and the example guideline below.
- If the contribution contains multiple commits, are they grouped together into logical changes (one major change per pull request)? Are any commits with names like “fixed typo” [squashed into previous commits](#)?

If you’re unsure about any of these points, please open the Pull Request anyhow and then ask us for feedback.

10.3 Pull Request Process

After you open the Pull Request, there will probably be some discussion in the comments field of the request itself.

Once the Pull Request is ready to merge, it will first be merged into our internal git system for “in-house” automated testing.

If this process passes, it will be merged into the public GitHub repository.

10.4 Example Contribution Guideline

10.4.1 Checklist

- Check if your example proposal has no similarities to the project (**already existing examples**)
- Use the [Arduino style guide](#)
- Add the header to all source files
- Add the README.md file
- Add inline comments if needed
- Test the example

10.4.2 Header

All the source files must include the header with the example name and license, if applicable. You can change this header as you wish, but it will be reviewed by the community and may not be accepted.

Ideally, you can add some description about the example, links to the documentation, or the author's name. Just have in mind to keep it simple and short.

Header Example

```
/* Wi-Fi FTM Initiator Arduino Example

This example code is in the Public Domain (or CC0 licensed, at your option.)

Unless required by applicable law or agreed to in writing, this
software is distributed on an "AS IS" BASIS, WITHOUT WARRANTIES OR
CONDITIONS OF ANY KIND, either express or implied.
*/
```

10.4.3 README file

The README.md file should contain the example details.

Please see the recommended README.md file in the [example template folder](#).

10.4.4 Inline Comments

Inline comments are important if the example contains complex algorithms or specific configurations that the user needs to change.

Brief and clear inline comments are really helpful for the example understanding and its fast usage.

See the [FTM example](#) as a reference:

```
// Number of FTM frames requested in terms of 4 or 8 bursts (allowed values - 0,
↳ (No pref), 16, 24, 32, 64)
```

Also:

```
const char * WIFI_FTM_SSID = "WiFi_FTM_Responder"; // SSID of AP that has FTM_
↳ Enabled
const char * WIFI_FTM_PASS = "ftm_responder"; // STA Password
```

10.4.5 Testing

Be sure you have tested the example in all the supported targets. If the example has some specific hardware requirements, edit/add the ci.yml in the same folder as the sketch to specify the regular expression for the required configurations from sdkconfig. This will ensure that the CI system will run the test only on the targets that have the required configurations.

You can check the available configurations in the `sdkconfig` file in the `tools/esp32-arduino-libs/<target>` folder.

Here is an example of the `ci.yml` file where the example requires Wi-Fi to work properly:

```
requires:
  - CONFIG_SOC_WIFI_SUPPORTED=y
```

Note: The list of configurations will be checked against the `sdkconfig` file in the target folder. If the configuration is not present in the `sdkconfig`, the test will be skipped for that target. That means that the test will only run on the targets that have **ALL** the required configurations.

Also, by default, the “match start of line” character (^) will be added to the beginning of each configuration. That means that the configuration must be at the beginning of the line in the `sdkconfig` file.

Sometimes, the example might not be supported by some target, even if the target has the required configurations (like resources limitations or requiring a specific SoC). To avoid compilation errors, you can add the target to the `ci.yml` file so the CI system will force to skip the test on that target.

Here is an example of the `ci.yml` file where the example is requires Wi-Fi to work properly but is also not supported by the ESP32-S2 target:

```
requires:
  - CONFIG_SOC_WIFI_SUPPORTED=y
targets:
  esp32s2: false
```

You also need to add this information in the `README.md` file, on the **Supported Targets**, and in the example code as an inline comment. For example, in the sketch:

```
/*
  THIS FEATURE REQUIRES WI-FI SUPPORT AND IS NOT AVAILABLE FOR ESP32-S2 AS IT DOES
  ↳NOT HAVE ENOUGH RAM.
*/
```

And in the `README.md` file:

Currently, this example requires Wi-Fi and supports the following targets.

Supported Targets	ESP32	ESP32-S3	ESP32-C3	ESP32-C6
-----	-----	-----	-----	-----

By default, the CI system will use the FQBNs specified in the `.github/scripts/sketch_utils.sh` file to compile the sketches. Currently, the default FQBNs are:

- `espressif:esp32:esp32:PSRAM=enabled`
- `espressif:esp32:esp32s2:PSRAM=enabled`
- `espressif:esp32:esp32s3:PSRAM=opi,USBMode=default`
- `espressif:esp32:esp32c3`
- `espressif:esp32:esp32c6`
- `espressif:esp32:esp32h2`
- `espressif:esp32:esp32p4:USBMode=default,ChipVariant=postv3`

There are two ways to alter the FQBNs used to compile the sketches: by using the `fqbn` or `fqbn_append` fields in the `ci.yml` file.

If you just want to append a string to the default FQBNs, you can use the `fqbn_append` field. For example, to add the `DebugLevel=debug` to the FQBNs, you would use:

```
fqbn_append: DebugLevel=debug
```


If you want to override the default FQBNs, you can use the `fqbn` field. It is a dictionary where the key is the target name and the value is a list of FQBNs. The FQBNs in the list will be used in sequence to compile the sketch. For example, to compile a sketch for ESP32-S2 with and without PSRAM enabled, you would use:

```
fqbn:
  esp32s2:
    - espressif:esp32:esp32s2:PSRAM=enabled,FlashMode=dio
    - espressif:esp32:esp32s2:PSRAM=disabled,FlashMode=dio
```

Note: The FQBNs specified in the `fqbn` field will also override the options specified in the `fqbn_append` field. That means that if the `fqbn` field is specified, the `fqbn_append` field will be ignored and will have no effect.

10.4.6 Example Template

The example template can be found [here](#) and can be used as a reference.

10.5 Documentation

If you are contributing to the documentation, please follow the instructions described in the [documentation guidelines](#) to properly format and test your changes.

10.6 Testing and CI

In our repository, we have a Continuous Integration (CI) system that runs tests and code formatting fixes on every Pull Request. This system will run the tests on all supported targets and check if everything is working as expected.

We have many types of tests and checks, including:

- Compilation tests;
- Runtime tests;
- Documentation checks;
- Code style checks;
- And more.

After you have made your changes, you should test them. You can do this in different ways depending on the type of change you have made.

10.6.1 Examples

The easiest way to test an example is to load it into the Arduino IDE and run it on your board. If you have multiple boards, you should test it on all of them to ensure that the example works on all supported targets.

You can refer to the [Example Contribution Guideline](#) section for more information on how to write and test examples.

10.6.2 Library Changes

If you have made changes to a library, you should test it on all supported targets. You can do this by loading the library examples (or creating new ones) into the Arduino IDE and running them on your board. If you have multiple boards, you should test it on all of them to ensure that the library works as expected on all targets.

You can also add a new test suite to automatically check the library. You can refer to the [Adding a New Test Suite](#) section for more information.

10.6.3 Core Changes

If you have made changes to the core, it is important to ensure that the changes do not break the existing functionality. You can do this by running the tests on all supported targets. You can refer to the [Runtime Tests](#) section for more information.

10.6.4 CI

In our repository, we have a Continuous Integration (CI) system that runs tests and fixes on every Pull Request. This system will run the tests on all supported targets and check if everything is working as expected. You can read a more technical explanation in the [CI README](#).

We have many types of tests and checks, including:

- Compilation tests;
- Runtime tests;
- Documentation checks;
- Code style checks;
- And more.

Let's go deeper into each type of test in the CI system:

10.6.5 Compilation Tests

The compilation tests are the first type of tests in the CI system. They check if the code compiles on all supported targets. If the code does not compile, the CI system will fail the test and the Pull Request will not be merged. This is important to ensure that the code is compatible with all supported targets and no broken code is merged.

It will go through all the sketches in the repository and check if they compile on all supported targets. This process is automated and controlled by GitHub Actions. The CI system will run the `arduino-cli` tool to compile the sketches on all supported targets.

Testing it locally using the CI scripts would be too time consuming, so we recommend running the tests locally using the Arduino IDE with a sketch that uses the changes you made (you can also add the sketch as an example if your change is not covered by the existing ones). Make sure to set `Compiler Warnings` to `All` in the Arduino IDE to catch any potential issues.

10.6.6 Runtime Tests

Another type of test is the runtime tests. They check if the code runs and behaves as expected on all supported targets. If the code does not run as expected, the CI system will fail the test and the Pull Request will not be merged. This is important to ensure that the code works as expected on all supported targets and no unintended crashes or bugs are introduced.

These tests are specialized sketches that run on the target board and check if the code behaves as expected. This process is automated and controlled by the `pytest_embedded` tool. You can read more about this tool in its [documentation](#).

The tests are divided into two categories inside the `tests` folder:

- `validation`: These tests are used to validate the behavior of the Arduino core and libraries. They are used to check if the core and libraries are working as expected;
- `performance`: These tests are used to check the performance of the Arduino core and libraries. They are used to check if the changes made to the core and libraries have any big impact on the performance. These tests usually run for a longer time than the validation tests and include common benchmark tools like [CoreMark](#).

Note: Keep in mind that to run the CI scripts locally, you need to have the Go-based `yq` from [mikefarah/yq](#) installed and not the Python-based `yq` from PyPI or [kislyuk/yq](#). The syntax is slightly different and will not work with the CI

scripts. You can install the Go-based `yq` through `brew`, `chocolatey` or downloading the binary directly from the [releases page](#).

To run the runtime tests locally, first install the required dependencies by running:

```
pip install -U -r tests/requirements.txt
```

Before running the test, we need to build it by running:

```
./github/scripts/tests_build.sh -s <test_name> -t <target>
```

The `<test_name>` is the name of the test you want to run (you can check the available tests in the `tests/validation` and `tests/performance` folders), and the `<target>` is the target board you want to run the test on. For example, to run the `uart` test on the ESP32-C3 target, you would run:

```
./github/scripts/tests_build.sh -s uart -t esp32c3
```

You should see the output of the build process and the test binary should be generated in the `~/ .arduino/tests/<target_chip>/<test_name>/build.tmp` folder.

Now that the test is built, you can run it in the target board. Connect the target board to your computer and run:

```
./github/scripts/tests_run.sh -s <test_name> -t <target>
```

For example, to run the `uart` test on the ESP32-C3 target, you would run:

```
./github/scripts/tests_run.sh -s uart -t esp32c3
```

The test will run on the target board and you should see the output of the test in the terminal:

```
lucassvaz@Lucas--MacBook-Pro esp32 % ./github/scripts/tests_run.sh -s uart -t_
↳esp32c3
Sketch uart test type: validation
Running test: uart -- Config: Default
pytest tests --build-dir /Users/lucassvaz/.arduino/tests/esp32c3/uart/build.tmp -k_
↳test_uart --junit-xml=/Users/lucassvaz/Espressif/Arduino/hardware/espressif/_
↳esp32/tests/validation/uart/esp32c3/uart.xml --embedded-services esp,arduino
=====
↳test session starts_
↳=====
platform darwin -- Python 3.12.3, pytest-8.2.2, pluggy-1.5.0
rootdir: /Users/lucassvaz/Espressif/Arduino/hardware/espressif/esp32/tests
configfile: pytest.ini
plugins: cov-5.0.0, embedded-1.11.5, anyio-4.4.0
collected 15 items / 14 deselected / 1 selected

tests/validation/uart/test_uart.py::test_uart
-----
↳----- live log setup -----
↳-----
2024-08-22 11:49:30 INFO Target: esp32c3, Port: /dev/cu.usbserial-2120
PASSED
↳
↳
↳[100%]
-----
↳----- live log teardown -----
↳-----
2024-08-22 11:49:52 INFO Created unity output junit report: /private/var/folders/_
↳vp/g9wctsvn7b91k3pv_7cwpt_h0000gn/T/pytest-embedded/2024-08-22_14-49-30-392993/_
↳test_uart/dut.xml
```

(continues on next page)

(continued from previous page)

```
----- generated xml file: /Users/
↳ lucassvaz/Espressif/Arduino/hardware/espressif/esp32/tests/validation/uart/
↳ esp32c3/uart.xml -----
=====
↳ 1 passed, 14 deselected in 22.18s_
=====
```

After the test is finished, you can check the output in the terminal and the generated XML file in the test folder. Additionally, for performance tests, you can check the generated JSON file in the same folder.

You can also run the tests in [Wokwi](#) or [Espressif's QEMU](#) by using the `-W` and `-Q` flags respectively. You will need to have the Wokwi and/or QEMU installed in your system and set the `WOKWI_CLI_TOKEN` and/or `QEMU_PATH` environment variables. The `WOKWI_CLI_TOKEN` is the CI token that can be obtained from the [Wokwi website](#) and the `QEMU_PATH` is the path to the QEMU binary.

For example, to run the `uart` test using Wokwi, you would run:

```
WOKWI_CLI_TOKEN=<your_wokwi_token> ./github/scripts/tests_run.sh -s uart -t_
↳ esp32c3 -W
```

And to run the `uart` test using QEMU, you would run:

```
QEMU_PATH=<path_to_qemu_binary> ./github/scripts/tests_run.sh -s uart -t esp32c3 -
↳ Q
```

Note: Not all tests are supported by Wokwi and QEMU. QEMU is only supported for ESP32 and ESP32-C3 targets. Wokwi support depends on the [currently implemented peripherals](#).

Adding a New Test Suite

If you want to add a new test suite, you can create a new folder inside `tests/validation` or `tests/performance` with the name of the test suite. You can use the `hello_world` test suite as a starting point and the other test suites as a reference.

A test suite contains the following files:

- `test_<test_name>.py`: Is the test file that contains the test cases. Required.
- `<test_name>.ino`: This is the sketch that will be tested. Required.
- `ci.yml`: The file that specifies how the test suite will be run in the CI system. Optional.
- `diagram.<target>.json`: The diagram file that specifies the connections between the components in Wokwi. Optional.
- Any other files that are needed for the test suite.

You can read more about the test python API in the [pytest-embedded documentation](#). For more information about the Unity testing framework, you can check the [Unity documentation](#).

After creating the test suite, make sure to test it locally and run it in the CI system to ensure that it works as expected.

CI YAML File The `ci.yml` file is used to specify how the test suite and sketches will be handled by the CI system. It can contain the following fields:

- `multi_device`: A dictionary that matches the device names to their sketch directories. Each device must have its own sketch folder. There are two devices in a multi-device test suite: `device0` and `device1`. This field is only valid for multi-device test suites.
- `requires`: A list of configurations in `sdkconfig` that are required to run the test suite. The test suite will only run on the targets that have **ALL** the required configurations. By default, no configurations are required.

- `requires_any`: A list of configurations in `sdkconfig` that are required to run the test suite. The test suite will only run on the targets that have **ANY** of the required configurations. By default, no configurations are required.
- `targets`: A dictionary that specifies the targets for which the tests will be run. The key is the target name and the value is a boolean that specifies if the test should be run for that target. By default, all targets are enabled as long as they have the required configurations specified in the `requires` field. This field is also valid for examples.
- `platforms`: A dictionary that specifies the supported platforms. The key is the platform name and the value is a boolean that specifies if the platform is supported. By default, all platforms are assumed to be supported.
- `tags`: A list of tags that all runners will require when running the test suite in hardware. By default, no tags are required.
- `soc_tags`: A dictionary that specifies SoC-specific tags required for hardware runners. The key is the target name and the value is a list of tags. By default, no SoC-specific tags are required.
- `fqbn_append`: A string to be appended to the default FQBNs. By default, no string is appended. This has no effect if `fqbn` is specified.
- `fqbn`: A dictionary that specifies the FQBNs that will be used to compile the sketch. The key is the target name and the value is a list of FQBNs. The [default FQBNs](#) are used if this field is not specified. This overrides the default FQBNs and the `fqbn_append` field.
- `libs`: A list of libraries that are required to run the test suite. The libraries will be installed automatically if they are not already present. Libraries are installed using the `arduino-cli lib install` command, so you can specify libraries by name + version (e.g., `AudioZero@1.0.0`) or by URL (e.g., `https://github.com/arduino-libraries/WiFi101.git`). More information can be found in the [Arduino CLI documentation](#).

The `wifi` test suite is a good example of how to use the `ci.yml` file:

```
tags:
  - wifi_router

fqbn:
  esp32:
    - espressif:esp32:esp32:PSRAM=enabled,PartitionScheme=huge_app,FlashMode=dio
    - espressif:esp32:esp32:PSRAM=disabled,PartitionScheme=huge_app,FlashMode=dio
  esp32s2:
    - espressif:esp32:esp32s2:PSRAM=enabled,PartitionScheme=huge_app,FlashMode=dio
    - espressif:esp32:esp32s2:PSRAM=disabled,PartitionScheme=huge_app,FlashMode=dio
  esp32s3:
    - espressif:esp32:esp32s3:PSRAM=opi,USBMode=default,PartitionScheme=huge_app,
    ↪FlashMode=qio
    - espressif:esp32:esp32s3:PSRAM=disabled,USBMode=default,PartitionScheme=huge_
    ↪app,FlashMode=qio
    - espressif:esp32:esp32s3:PSRAM=enabled,USBMode=default,PartitionScheme=huge_
    ↪app,FlashMode=qio

platforms:
  gemu: false

requires_any:
  - CONFIG_SOC_WIFI_SUPPORTED=y
  - CONFIG_ESP_HOSTED_ENABLED=y
```

Adding a Multi-Device Test Suite

Multi-device tests allow you to test interactions between multiple ESP32 devices, such as Wi-Fi AP/client communication or BLE connections. Each device runs its own sketch, and the test coordinates communication between them.

Creating a Multi-Device Test To create a multi-device test, follow these steps. The examples below use the `wifi_ap` test as a reference, but you should create your own test with an appropriate name:

1. **Create the test directory structure:** Create a folder for each device sketch within your test directory. For example, using the `wifi_ap` test structure:

```
mkdir -p tests/validation/wifi_ap/ap
mkdir -p tests/validation/wifi_ap/client
```

2. **Create the ``ci.yml`` file:** Create `ci.yml` in your test directory with a `multi_device` configuration. Example from the `wifi_ap` test:

```
multi_device:
  device0: ap
  device1: client

platforms:
  qemu: false
  wokwi: false

requires:
  - CONFIG_SOC_WIFI_SUPPORTED=y
```

The `multi_device` field maps device names to their sketch directories. Each device must have its own sketch folder.

3. **Create device sketches:** Create an `.ino` file in each sketch directory with the same name as the directory. Example from the `wifi_ap` test, `tests/validation/wifi_ap/ap/ap.ino`:

```
void setup() {
  Serial.begin(115200);
  Serial.println("[AP] Device ready for WiFi credentials");
}

void loop() {
  // Your AP logic
}
```

And `tests/validation/wifi_ap/client/client.ino` (example):

```
void setup() {
  Serial.begin(115200);
  Serial.println("[CLIENT] Device ready for WiFi credentials");
}

void loop() {
  // Your client logic
}
```

4. **Create the test file:** Create `test_<your_test_name>.py` in your test directory. Example from the `wifi_ap` test, `tests/validation/wifi_ap/test_wifi_ap.py`:

```
def test_wifi_ap(dut):
    ap = dut[0]
    client = dut[1]

    # Wait for devices to be ready
    ap.expect_exact("[AP] Device ready for WiFi credentials")
    client.expect_exact("[CLIENT] Device ready for WiFi credentials")

    # Your test logic here
    # Example: Send commands and verify responses
    ap.write("command")
    client.expect("expected_response")
```

The `dut` parameter is a list where each element corresponds to a device in the order specified in `ci.yml`.

Note: The examples above use the `wifi_ap` test as a reference. You can find the complete working example in `tests/validation/wifi_ap/`. When creating your own test, replace `wifi_ap` with your test name and

adapt the device names (`ap`, `client`) to match your use case.

Building Multi-Device Tests To build a specific multi-device test (using `wifi_ap` as an example):

```
./.github/scripts/tests_build.sh -s wifi_ap -t esp32
```

To build all tests (chunk build):

```
./.github/scripts/tests_build.sh -c -type validation -t esp32 -i 0 -m 1
```

The build output will be stored in a nested layout:

```
$HOME/.arduino/tests/<target>/<test_name>/<sketch_name>/build.tmp
```

For example, for a `wifi_ap` test on `esp32`: * **Device0** (`ap`): `$HOME/.arduino/tests/esp32/wifi_ap/ap/build.tmp` * **Device1** (`client`): `$HOME/.arduino/tests/esp32/wifi_ap/client/build.tmp`

Running Multi-Device Tests Before running multi-device tests, you need to set up the environment variables for the serial ports:

```
export ESPPORT1=/dev/ttyUSB0 # Port for device0
export ESPPORT2=/dev/ttyUSB1 # Port for device1
```

Then, to run a specific test (using `wifi_ap` as an example):

```
./.github/scripts/tests_run.sh -s wifi_ap -t esp32
```

To run all tests (chunk run):

```
./.github/scripts/tests_run.sh -c -type validation -t esp32 -i 0 -m 1
```

Note: Both `ESPPORT1` and `ESPPORT2` environment variables must be set before running multi-device tests. The sketch directory name must match the value in the `multi_device` configuration in `ci.yml`, and the `.ino` file must have the same name as the directory.

10.6.7 Documentation Checks

The CI also checks the documentation for any compilation errors. This is important to ensure that the documentation layout is not broken. To build the documentation locally, please refer to the [documentation guidelines](#).

10.6.8 Code Style Checks

For checking the code style and other code quality checks, we use pre-commit hooks. These hooks will be automatically run by the CI when a Pull Request is marked as `Status: Pending Merge`. You can check which hooks are being run in the `.pre-commit-config.yaml` file.

Currently, we have hooks for the following tasks:

- Formatters for C, C++, Python, Bash, JSON, Markdown and ReStructuredText files;
- Linters for Python, Bash and prose (spoken language);
- Checking for spelling errors in the code and documentation;
- Removing trailing whitespaces and tabs in the code;
- Checking for the presence of private keys and other sensitive information in the code;
- Fixing the line endings and end of files (EOF) in the code;

- And more.

You can read more about the pre-commit hooks in the [pre-commit documentation](#).

If you want to run the pre-commit hooks locally, you first need to install the required dependencies by running:

```
pip install -U -r tools/pre-commit/requirements.txt
```

Then, you can run the pre-commit hooks staging your changes and running:

```
pre-commit run
```

To run a specific hook, you can use the hook name as an argument. For example, to run the `codespell` hook, you would run:

```
pre-commit run codespell
```

If you want to run the pre-commit hooks automatically against the changed files on every `git commit`, you can install the pre-commit hooks by running:

```
pre-commit install
```

10.7 Legal Part

Before a contribution can be accepted, you will need to sign our contributor agreement. You will be prompted for this automatically as part of the Pull Request process.

Chapter 11

External Libraries Testing

11.1 About

External libraries testing is a compilation test for listed libraries on arduino-esp32 master branch. The test was made for users, so they can check which libraries are compiling without errors on which SoC. External libraries test is running periodically (once a week) against master branch and can also run on PR by adding a label `lib_test`. The test is running on all supported ESP32 chips.

Note: As the test is just a compilation of example, that does not guarantee that the library/sketch will run without any problems after flashing it on your device.

11.2 How to Add Library to Test

To add a library to the CI test you need to add your library to the [lib.json](#) file located in `./github/workflows/`.

Note: Please add the libraries to the [lib.json](#) in alphabetical order, thanks.

11.2.1 List of parameters:

Where the library will be installed from (use only 1 option):

- `name` - Name of the Library in Arduino Library Manager.
- `source-url` - URL to the library github repository (example: “<https://github.com/Arduino-IRremote/Arduino-IRremote.git>”). Use when your Library is not listed in Arduino Library Manager.

Required:

- `exclude_targets` - List of targets to be excluded from testing. Use only when the SoC dont support used peripheral.
- `sketch_path` - Path / paths to the sketch / sketches to be tested.

Optional:

- `version` - Version of the library.
- `required-libs` - Libraries that are necessary to be installed to compile the tested library.
- `destination-name` - Folder name used for the installation of library (use only when needed).

11.2.2 Example of library addition from Arduino Library Manager with ESP32-S2 excluded:

```
{
  "name": "ArduinoBLE",
  "exclude_targets": [
    "esp32s2"
  ],
  "sketch_path": [
    "~/Arduino/libraries/ArduinoBLE/examples/Central/Scan/Scan.ino"
  ]
}
```

11.2.3 Example of library addition from Github URL with required library:

```
{
  "source-url": "https://github.com/me-no-dev/ESPAsyncWebServer.git",
  "required-libs": [
    { "source-url": "https://github.com/me-no-dev/AsyncTCP.git" }
  ],
  "exclude_targets": [],
  "sketch_path": [
    "~/Arduino/libraries/ESPAsyncWebServer/examples/CaptivePortal/
→CaptivePortal.ino",
    "~/Arduino/libraries/ESPAsyncWebServer/examples/ESP_
→AsyncFSBrowser/ESP_AsyncFSBrowser.ino",
    "~/Arduino/libraries/ESPAsyncWebServer/examples/regex_patterns/
→regex_patterns.ino",
    "~/Arduino/libraries/ESPAsyncWebServer/examples/simple_server/
→simple_server.ino"
  ]
},
```

11.2.4 Submit a PR

- Open a PR with the changes and someone from Espressif team will add a label `lib_test` to the PR and CI will run the test to check, if the addition is fine and the library/example is compiling.
- After merging your PR, the next scheduled test will test your library and add the results to the [LIBRARIES_TEST.md](#).

11.3 Test Results

11.3.1 Icons meaning

There is a number before each icon, which means the number of examples which were compiled with that result.

-  - Compilation was successful.
-  - Compilation was successful, but some warnings occurs.
-  - Compilation failed.
- N/A - Not tested (target is in `exclude_targets` list).

11.3.2 Scheduled test result

You can check the results in [LIBRARIES_TEST.md](#).

The results file example:

External libraries build test

Library	ESP32	ESP32C3	ESP32S2	ESP32S3
Adafruit NeoPixel	1	1	1	1
FastLED	1	1	1	1
IRremote	1	1	1	1
ESP32Servo	4	4	4	4
ArduinoBLE	1	1	1	1

Generated on: Mar-31-2023 10:11:12 / [GitHub Action Link](#)

11.3.3 Pull Request test result

If the test run on Pull Request, it will compile all libraries and sketches 2 times (before/after changes in PR) to see, if the PR is breaking/fixing libraries. In the table the results are in order BEFORE -> AFTER.

External libraries build test

Library	ESP32	ESP32C3	ESP32S2	ESP32S3
Adafruit NeoPixel	1 -> 1	1 -> 1	1 -> 1	1 -> 1
FastLED	1 -> 1	1 -> 1	1 -> 1	1 -> 1
IRremote	1 -> 1	1 -> 1	1 -> 1	1 -> 1
ESP32Servo	4 -> 4	4 -> 4	4 -> 4	4 -> 4
ArduinoBLE	1 -> 1	1 -> 1	N/A	1 -> 1